

The Complete LangGraph Blueprint



Build 50+ AI Agents for
Business Success

James Karanja Maina

The Complete LangGraph Blueprint

Build 50+ AI Agents for Business Success



By James Karanja Maina

Hak Cipta © 2025 Semua hak dilindungi. Tidak ada bagian dari buku ini yang boleh diperbanyak, didistribusikan, atau ditransmisikan dalam bentuk atau dengan cara apa pun, termasuk fotokopi, perekaman, atau metode elektronik atau mekanis lainnya, tanpa izin tertulis sebelumnya dari penerbit, kecuali dalam bentuk kutipan singkat yang disertakan dalam ulasan kritis dan penggunaan non-komersial lain tertentu yang diizinkan oleh undang-undang hak cipta.

Untuk permintaan izin, silakan hubungi penerbit di alamat di bawah ini. Informasi Penerbit: Nama Penerbit: Zavora Technologies Ltd Penulis: James Karanja Maina Alamat Penerbit: P.O. Box 60058, 00100, Nairobi, Kenya Kota Penerbit: Nairobi Negara Penerbit: Kenya Email Penerbit: info@zavora.ai Email Penulis: james.karanja@zavora.ai Website: www.zavora.ai Penyangkalan Informasi yang diberikan dalam buku ini dimaksudkan hanya untuk tujuan edukasi dan tidak boleh dianggap sebagai saran bisnis, hukum, atau profesional. Penulis dan penerbit menolak segala tanggung jawab yang timbul dari penggunaan atau penerapan informasi yang terdapat dalam buku ini. Merek Dagang Semua nama merek, nama produk, dan nama perusahaan yang disebutkan dalam buku ini adalah merek dagang atau merek dagang terdaftar milik pemiliknya masing-masing. Penggunaan nama-nama ini tidak menyiratkan adanya afiliasi dengan atau dukungan dari mereka. Edisi Pertama: Januari 2025 Terima kasih telah membaca dan mendukung karya ini. Untuk informasi dan pembaruan lebih lanjut, kunjungi: <https://www.zavora.ai>

Table of Contents

Prakata Source Code Mailing List

Chapter 1 Introduction to LangGraph
and AI Agents

1.1 Selamat Datang di Dunia LangGraph dan AI
Agents 1.2 Programming Large Language Models 1.3
Apa itu LangGraph? 1.4 Prinsip Inti LangGraph 1.5
Mengapa Menggunakan LangGraph untuk AI Agents?
1.6 Bagaimana AI Agents Diuntungkan oleh Struktur
LangGraph 1.7 Aplikasi Dunia Nyata dari AI Agents
dengan LangGraph Chapter 2: Setting Up Your
Development Environment

Mengapa Virtual Environment? 2.1: Menginstal
Python 2.2: Menyiapkan Virtual Environment 2.3:
Menginstal LangGraph dan Library yang
Diperlukan 2.4: Menyiapkan Visual Studio Code
(VS Code) Lesson 1: Membangun Agent Pertama
Anda: "Hello World" 2.5 Memvisualisasikan Graph
Anda 2.6 Mencoba Menambahkan Node Lain ke
Graph Workflow 2.7 Kode Lengkap untuk Agent
"Hello World" Penutup Akhir: Apa yang Kita
Bangun dan Pelajari Bagaimana Semuanya
Bekerja Bersama: Chapter 3 Memahami
Konsep-Konsep Programming Kunci

Apa itu Bahasa Programming?

3.5. Class di Python: Fondasi Object-Oriented Programming 3.5.7
Class and Static Methods 3.5.8 Magic Methods 3.5.9 Properties:
3.5.10 Praktik Terbaik dengan Class 3.5.11. Modul Eksternal
Umum yang Digunakan dalam LangGraph 3.5.12 Map, Filter, dan
Reduce 3.6 Penanganan Exception Tingkat Lanjut Chapter 4
Elemen Inti LangGraph

Konsep Kunci: Nodes, Edges, States, dan Graphs 4.1. State -
Melacak Data 4.2. Nodes - Logika Bisnis 4.3. Edges -
menghubungkan node-node 4.4. Graph 4.5. Input and Output
Schemas dari sebuah graph: Manajemen State yang Fleksibel
dalam LangGraph 4.6. Bagaimana LangGraph Menyatukan
Semuanya 4.7. Pengambilan Keputusan dengan Conditional Edges
Chapter 5 Membangun AI Agent Pertama Anda

5.1 Pendahuluan: Kekuatan AI Agents 5.2
Menyiapkan OpenAI API Key dan Environment
Anda 5.3 Dasar-Dasar Messaging di LangGraph 5.4
Pemrosesan Input Pengguna secara Berkelanjutan
5.5 Pengantar Tools dalam AI Agents Chapter 6
Memperkenalkan Memory dalam AI Agents

6.1 Masalahnya: Agent Tanpa Memory 6.2
Meningkatkan Agent dengan Short-Term Memory 6.3
Bagaimana Short-Term Memory Bekerja

6.5 Contoh Kode Lengkap: AI Agent dengan Short-Term Memory
6.6 Memory di Beberapa Sesi 6.7 Bagian Penjelasan: Detail Teknis tentang Checkpointers dan InMemoryStore 6.7.2. InMemoryStore
Chapter 6: Quiz Chapter 7 Advanced Routing and Customization of AI Agents

7.1 Pengantar Routing dalam AI Agents 7.2 tools_condition Bawaan untuk Conditional Routing 7.3 Contoh Custom Conditional Routing: Weather and Calculator Nodes 7.4 Streaming di LangGraph 7.5 Integrasi API Eksternal: Gambaran Umum 7.6 Integrasi Weather API: Contoh Langkah demi Langkah 7.7 Integrasi API Dinamis dengan Input Pengguna 7.8 Integrasi Calculator API 7.9 Menggabungkan Beberapa Integrasi API Chapter 7 Quiz: API Integrations, Routing, and Streaming Chapter 8 Foundational AI Agent Architectures - ReAct

Pola ReAct (Reason + Act) ReAct Agents di LangGraph Opsi Implementasi ReAct Agent Internal dan Custom di LangGraph Praktik Terbaik untuk Membangun ReAct Agents 10 Contoh ReAct Agents di LangGraph Chapter 9 Human-in-the-Loop Agents: Menggabungkan Feedback Manusia untuk Pengambilan Keputusan yang Lebih Cerdas

9.1 Apa itu Human-in-the-Loop? 9.2 Konsep Inti: Breakpoints, Checkpoints, dan State Editing 9.3 Contoh 1: Mengimplementasikan Breakpoints Sederhana

9.4 Contoh ReAct Agent dengan Use Case Pembelian Saham
Finansial 9.5 Memahami dan Menggunakan "Interrupt After" di
LangGraph 9.6 Mengedit State Graph Selama Eksekusi 9.7 Lima
Contoh Breakpoint Sederhana 9.8 Dynamic Breakpoints: Konsep
dan Penggunaan 9.9 Menunggu Input Pengguna: Konsep dan
Implementasi Contoh Praktis: ReAct Agent dengan Input Manusia
Penjelasan: Aspek Teknis Human-in-the-Loop (HITL) di LangGraph
Chapter 9 Quiz: Human-in-the-Loop Agents Chapter 10
Plan-and-Execute Agents

10.1 Pendahuluan 10.2 Mengapa Menggunakan
Plan-and-Execute? 10.3 Konsep Plan-and-Execute 10.4
Implementasi LangGraph dari AI Agent Plan and Execute
Contoh Praktis 1. Agent Troubleshooting dan Diagnostik IT
Contoh Praktis 2. Agent Otomasi Workflow Bisnis Contoh
Praktis 3: Chatbot Customer Support Penjelasan: Arsitektur
Plan-and-Execute Chapter 10 Quiz: Plan-and-Execute
Agents Chapter 11 Agentic Retrieval-Augmented
Generation (RAG) di LangGraph

Apa itu RAG dan Konsep Dasarnya Embeddings dalam
Riset: Paper dan Konsep Kunci Embedding Models dari
Large Language Models (LLMs) Vector Stores dalam
Sistem RAG Vector Stores Populer Menghasilkan
Jawaban atas Pertanyaan menggunakan RAG dengan
LLM RAG Praktis - Mengajukan Pertanyaan pada
Dokumen PDF yang Panjang

Memahami Workflow RAG di LangGraph: Retrieval AI Agent Ulasan
Chapter: Workflow RAG di LangGraph Pengantar Arsitektur RAG
Tingkat Lanjut Quiz tentang Workflow RAG di LangGraph Chapter
12 Advanced RAG Architectures (Self-RAG, Corrective RAG and
Adaptive RAG)

12.1 Pengantar Self-Reflective RAG 12.2 Corrective
Retrieval-Augmented Generation (CRAG) 12.3 Adaptive RAG
Bagian Penjelasan: Konsep-Konsep di Chapter 12 - Advanced RAG
Architectures Chapter 12 Quiz: Advanced RAG Architectures
(Self-RAG, Corrective RAG, Adaptive RAG) Chapter 13 Multiagent
Architectures

13.1 Agent Supervisor Contoh Praktis 1:
Jaringan Research Assistant Contoh Praktis 2:
Otomasi Customer Service Contoh Praktis 3:
Analisis Saham Contoh Praktis 4: Manajemen
Portofolio Chapter 14 Hierarchical Agent Teams

Bagian 14.1: Membangun Hierarchical Agent Teams
Contoh Arsitektur Hierarchical Agent Team Langkah
1: Membangun Research Team Langkah 2:
Membangun Document Authoring Team Langkah 3:
Mengintegrasikan Team dengan Overall Supervisor
Chapter 15 Specialized Agent Architectures

15.1 Event-Driven Agents

15.2 Cognitive Architectures 15.3
Model-Based Agents 15.4 AI-Powered
Code Generation Agents 15.5
AI-Enhanced Pair Programming Tools
Chapter 16 Testing AI Agents

16.1 Pentingnya Testing dalam Sistem AI 16.2 Unit
Testing untuk Agents Chapter 17 Frontend
Development for LangGraph-Powered AI Agents

17.1 Pengantar Interaksi Frontend dan AI Agent 17.2 Mengekspos
LangGraph Agents sebagai API 17.3 Menyiapkan Frontend
Development Environment 17.4 Membangun Frontend Interface 17.5
Menghubungkan Frontend ke LangGraph Agent 17.6 Men-deploy
Aplikasi Lengkap 17.7 Ringkasan dan Praktik Terbaik 17.8
Menyiapkan Asynchronous Streaming dengan LangGraph dan
WebSockets 17.9 Mengintegrasikan Streaming di Frontend 17.10
Pengantar Streaming Responses 17.11 Menyiapkan Frontend
dengan React 17.12 Ringkasan dan Praktik Terbaik untuk Streaming
Responses 17.13 Menyiapkan Backend dengan Next.js dan
Frontend dengan Next.js AI SDK 17.14 Ringkasan dan Praktik
Terbaik 17.15 Menyiapkan LangGraph Backend dengan Next.js dan
AI SDK 17.16 Ringkasan dan Praktik Terbaik Chapter 18 Building
Agents with NVIDIA NeMo Inference Models (NIMs)

18.2 Inference Menggunakan Direct API Calls 18.3: Inference Menggunakan LangChain 18.4 Self-Hosting NIMs 18.4 Membangun AI-Powered Campaign Generator dengan LangGraph 18.5 Bagian Penjelasan: Konsep Programming Kunci dalam Campaign Generator Chapter 18 Quiz: Building Agents with NVIDIA NeMo Inference Models (NIMs) Chapter 19 Testing AI Agents with Test-Driven Development (TDD)

19.1 Memahami Test-Driven Development (TDD) 19.2 Project Setup 19.3 Memahami Siklus TDD Test-Driven Development mengikuti siklus yang sederhana namun kuat: 1. Menulis test yang gagal (failing test) 2. Menulis kode minimal untuk melewati test 3. Melakukan refactor sambil menjaga test tetap green Mari kita mulai dengan contoh sederhana untuk mengilustrasikan siklus ini. Contoh 1: Basic Message Processing Node 19.4 Memahami State Management 19.5 Menguji LLM Interactions Membangun Customer Support AI Agent dengan Test-Driven Development Chapter 20 Deploying AI Agents into Production

20.1 Strategi Deployment 20.2 Continuous Integration and Continuous Deployment (CI/CD) 20.3 Menskalakan AI Agents 20.4 Pertimbangan Keamanan Chapter 21 Performance Monitoring and Maintenance

21.1 Tools dan Teknik Monitoring

21.2 Metrik Performa untuk AI Agents

21.3 Troubleshooting dan Debugging

Chapter 22 Case Studies and

Applications

22.1 Otomasi Customer Support: The AI Help Desk 22.3

Efisiensi Operasional: Mengotomasi Workflow Internal

22.4 Pelajaran Nyata yang Dipetik Chapter 23 Final

Thoughts and Next Steps

23.1 Masa Depan AI Agents 1. Tren 1: Multi-Modal

AI Agents 2. Tren 2: Federated Learning untuk

Privacy-Enhanced AI 3. Tren 3: Explainable AI (XAI)

4. Tren 4: Edge AI 5. Tren 5: AI Agents di Metaverse

23.2 Melanjutkan Perjalanan Anda 23.3 Penutup &

Kata-Kata Akhir Terima Kasih atas Dukungan Anda!

Kami Ingin Mendengar Feedback Anda!

Preface

Kita berada di ambang era AI agent. Pada tahun 2025, sistem AI otonom akan secara fundamental mengubah cara bisnis beroperasi dan cara kita menyelesaikan masalah kompleks. Namun bagi banyak orang, membangun solusi AI yang praktis tetap merupakan lompatan yang menantang dari teori ke implementasi. The Complete LangGraph Blueprint menjembatani kesenjangan ini. Dengan menggunakan framework open-source LangChain dan LangGraph, buku ini menyediakan panduan praktis untuk membangun lebih dari 50 AI agent yang siap produksi. Setiap chapter bergerak dari konsep ke kode, berfokus pada aplikasi dunia nyata yang menciptakan nilai langsung. Ini bukan sekadar buku lain tentang potensi AI-ini adalah framework praktis Anda untuk membangun masa depan.

Selamat datang di The Complete LangGraph Blueprint.

JAMES KARANJA

James Karanja Maina
www.zavora.ai

Source Code

Source code lengkap untuk buku ini dapat diperoleh dari GitHub di alamat berikut:
<https://github.com/jkmaina/LangGraphProjects>

Mailing List

Untuk mendapatkan versi terbaru buku ini, bergabunglah dengan mailing list dengan mengirim email ke james.karanja@zavora.ai. Ini akan memberikan Anda hak eksklusif atas edisi PDF (jika tersedia) dari buku-buku dalam seri "The Complete AI Blueprint" yang akan menggabungkan pembaruan mendatang tentang AI Agents, LangGraph dan LangChain, beserta pembaruan kode dan konten yang relevan. Berlangganan untuk memastikan Anda selalu diperbarui.

Untuk men-deploy AI Agents Anda di cloud, daftarkan diri Anda untuk platform AI Agent yang akan datang di www.zavora.ai

Part 1: Foundations of LangGraph and

AI Agent Building

(Chapters 1-3)

Chapter 1

Introduction to LangGraph and AI Agents

1.1 Selamat Datang di Dunia LangGraph dan AI Agents

Selamat datang dalam perjalanan Anda ke dunia AI agents yang menakjubkan bersama LangGraph! Buku ini akan memandu Anda melalui proses membangun 50 AI agent dengan kompleksitas yang terus meningkat, menggunakan pendekatan terstruktur yang dimulai dari dasar-dasar. Baik Anda seorang pemula mutlak atau seorang programmer yang ingin meningkatkan keterampilan Anda, buku ini dirancang untuk membuat yang kompleks menjadi sederhana dan yang menantang menjadi dapat dicapai.

Pada chapter pertama ini, kami akan mengenalkan Anda pada konsep inti LangGraph dan AI agents. Anda akan belajar apa itu LangGraph, bagaimana ia memungkinkan pembuatan workflow yang dinamis, dan mengapa ia sangat cocok untuk mengembangkan AI agents. Pada akhir chapter ini, Anda akan memiliki pemahaman tingkat tinggi tentang arsitektur LangGraph, jenis-jenis masalah yang dapat diselesaikannya, dan industri-industri di mana AI agents memberikan dampak yang signifikan. Anda juga akan menyiapkan

development environment Anda dan membangun agent "Hello World" sederhana untuk mendapatkan pengalaman langsung sejak awal.

1.2 Programming Large Language Models

Large Language Models (LLMs) seperti GPT-4, Gemini dan Claude sangat kuat. Mereka dapat memahami dan menghasilkan teks yang menyerupai manusia, membuatnya cocok untuk berbagai aplikasi, dari chatbot hingga pembuatan konten. Namun, agar bisnis dapat memanfaatkan potensi penuhnya, penting untuk menstrukturkan bagaimana model-model ini berinteraksi dengan sistem lain, seperti database, API, atau sistem retrieval.

Pendekatan Tradisional: Chains

Secara tradisional, workflow yang menggunakan LLM, seperti ChatGPT, mengikuti paradigma "chain". Pendekatan ini telah sangat sukses secara luas, terutama dengan ChatGPT, yang menjadi populer karena kemampuannya melakukan tugas-tugas kompleks melalui rangkaian langkah yang sederhana dan linear. Berikut cara kerjanya secara umum:

1. **Input:** The system receives a user's question.
2. **Retrieve:** Relevant documents or data related to the question are found.
3. **Generate:** The retrieved data is then passed to the LLM (such as ChatGPT) to generate a well-informed response.

Memperkenalkan LangChain

LangChain adalah framework populer yang memfasilitasi pembuatan dan pengelolaan chain semacam itu. Ia menyediakan tools untuk mendefinisikan urutan panggilan LLM dan mengintegrasikannya dengan sistem eksternal seperti API dan database. Pendekatan berbasis chain dari LangChain menawarkan keandalan, karena rangkaian langkah yang sama berjalan pada setiap invocation, memastikan hasil yang konsisten.

Linear Chain Workflow

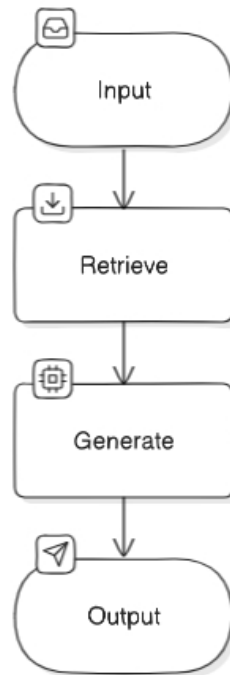


Figure 1: Sebuah linear chain workflow dengan node yang mewakili setiap langkah (Input ? Retrieve ? Generate) yang terhubung oleh edge.

Namun, Chains Memiliki Keterbatasan

Meskipun chain dapat diandalkan, mereka kurang fleksibel. Mereka tidak dapat menyesuaikan alurnya berdasarkan konten yang sedang diproses atau membuat keputusan otonom tentang langkah mana yang harus dijalankan selanjutnya. Kekakuan ini bisa menjadi keterbatasan yang signifikan dalam aplikasi kompleks atau dinamis di mana adaptabilitas sangat penting.

Memperkenalkan AI Agents: Dynamic Control Flow

Agents membawa hal ini selangkah lebih jauh. Alih-alih mengikuti chain yang tetap, agent dapat menentukan urutan langkahnya sendiri berdasarkan input yang diterimanya. Ini berarti:

- **Dynamic Routing:** Sebuah agent dapat memilih di antara beberapa jalur tergantung pada situasinya.
- **Tool Selection:** Ia dapat memutuskan tools atau API mana yang akan dipanggil berdasarkan tugas yang dihadapi.
- **Adaptive Responses:** Ia dapat menentukan apakah jawaban yang dihasilkan sudah cukup atau apakah pemrosesan lebih lanjut diperlukan.

Introducing Agents: Dynamic Control Flow

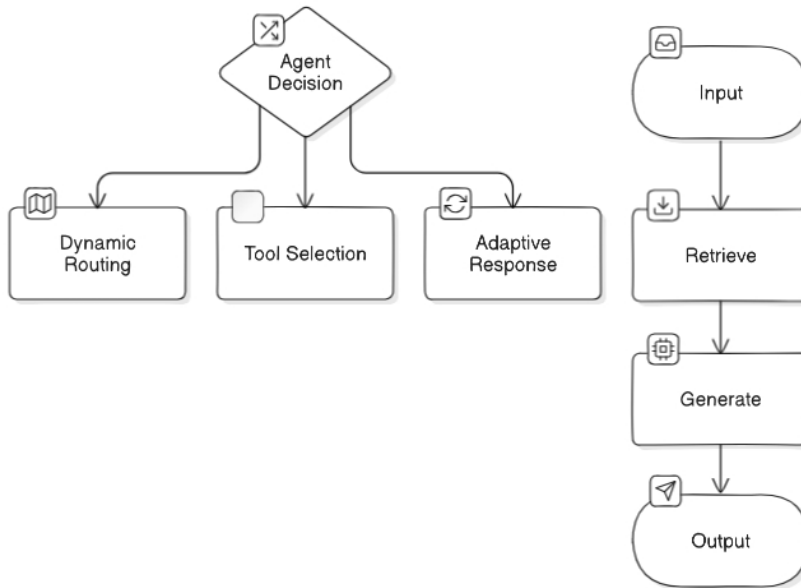


Figure 2: Memperkenalkan Agents

Mengapa AI Agents?

Agents menawarkan tingkat fleksibilitas dan kecerdasan yang lebih tinggi. Mereka dapat menangani interaksi yang lebih kompleks dan bervariasi, membuatnya cocok untuk aplikasi yang membutuhkan adaptabilitas dan pengambilan keputusan yang bernuansa.

1.3 Apa itu LangGraph?

LangGraph adalah framework open-source yang dirancang untuk menyederhanakan pengembangan AI agents. Berbeda dengan framework programming tradisional, di mana kontrol mengalir secara linear dari satu tugas ke tugas berikutnya, LangGraph menstrukturkan workflow sebagai graph. Dalam workflow LangGraph, tugas-tugas direpresentasikan sebagai node dan koneksi di antaranya sebagai edge. Pendekatan berbasis graph ini membuat LangGraph sangat cocok untuk AI agents yang perlu mengelola pengambilan keputusan dan pemrosesan data yang kompleks dan dinamis.

Mengapa Pendekatan Berbasis Graph?

LangGraph terinspirasi oleh Pregel (framework pemrosesan graph skala besar yang dikembangkan oleh Google) dan Apache Beam (framework pemrosesan batch dan real-time open source). Antarmuka publiknya mendapat inspirasi dari NetworkX. LangGraph dibangun oleh LangChain Inc, pencipta LangChain, tetapi dapat digunakan tanpa LangChain atau dikombinasikan dengannya untuk hasil terbaik. Untuk memahami mengapa LangGraph menggunakan pendekatan berbasis graph, mari kita lihat sekilas apa itu graph dan bagaimana cara kerjanya. Dalam ilmu komputer, sebuah graph adalah kumpulan node (atau vertex) dan edge (atau link) yang menghubungkan node-node ini.

Graph memungkinkan banyak jalur dan koneksi, menjadikannya ideal untuk workflow yang melibatkan percabangan, looping, atau tugas-tugas konkuren.

Dalam LangGraph:

- Node merepresentasikan tugas atau tindakan individual. Setiap node dapat menjalankan fungsi tertentu, seperti memproses data atau membuat keputusan. - Edge mendefinisikan hubungan antar node dan mengontrol aliran data. Mereka dapat dikonfigurasi dengan kondisi, memungkinkan workflow mengikuti jalur yang berbeda berdasarkan kriteria tertentu.

Fleksibilitas ini memungkinkan Anda membuat workflow yang tidak sepenuhnya linear, tetapi justru dapat bercabang dan beradaptasi dengan berbagai skenario.

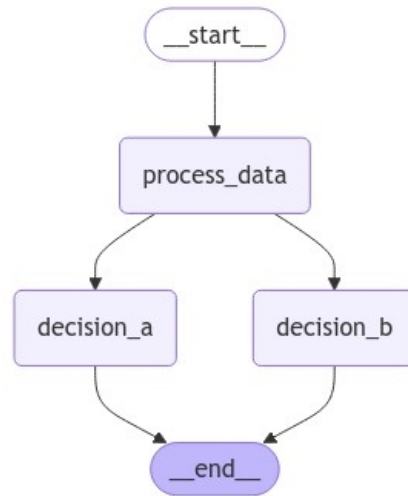


Figure 3: Sebuah diagram sederhana dengan tiga node yang terhubung oleh panah. Node pertama berlabel "START," yang terhubung melalui sebuah edge ke "Process Data." Node ini kemudian terpecah menjadi dua jalur (edge), satu menuju node "Decision A" dan yang lainnya ke "Decision B". Kedua node kemudian terhubung ke node END yang menyimpulkan eksekusi graph.

Diagram menunjukkan alur dasar di LangGraph dengan lima langkah utama. Dimulai dengan node START, yang mewakili awal dari proses. Dari sana, ia berpindah ke Process Data, di mana penanganan data awal terjadi. Proses kemudian bercabang menjadi dua keputusan: Decision A dan Decision B, yang mewakili pilihan atau jalur yang berbeda berdasarkan data. Kedua keputusan pada akhirnya mengarah ke node END, yang menandakan penyelesaian proses. Alur ini menunjukkan bagaimana LangGraph dapat menangani pemrosesan data dan pengambilan keputusan dengan cara yang sederhana dan terstruktur.

1.4 Prinsip Inti LangGraph

LangGraph dibangun berdasarkan beberapa prinsip inti yang membuatnya ideal untuk mengembangkan agent yang andal dan dinamis:

1. **Controllability:** Define the flow of your application precisely using nodes (tasks) and edges (connections). This allows for both deterministic and conditional workflows.

2. **Persistence:** Maintain the state of your application across different sessions or interactions using various storage options like databases.
3. **Human-in-the-Loop:** Incorporate human oversight or intervention within the workflow, enhancing reliability and decision-making.
4. **Streaming:** Provide real-time updates and allow for continuous interaction between the agent and users or other systems.

1.5 Mengapa Menggunakan LangGraph untuk AI Agents?

AI agents adalah program yang dapat menjalankan tugas secara otonom atau semi-otonom, sering menggunakan teknik artificial intelligence seperti machine learning dan natural language processing (NLP) untuk membuat keputusan dan berinteraksi dengan pengguna. LangGraph sangat cocok untuk membangun AI agents karena memungkinkan Anda membangun workflow yang fleksibel, modular, dan skalabel.

Membangun agent yang memiliki tingkat kontrol yang tinggi terkadang dapat menimbulkan tantangan seperti:

- **Masalah Keandalan:** Kontrol yang lebih besar dapat memunculkan ketidakpastian, terutama jika agent membuat keputusan yang salah.
- **Non-Determinisme:** LLM dapat menghasilkan output yang berbeda untuk input yang sama, sehingga menyebabkan workflow yang tidak konsisten.
- **Kesalahan Pemilihan Tool:** Memilih tools atau langkah yang salah dapat mengganggu workflow.

1.6 Bagaimana AI Agents Diuntungkan oleh Struktur LangGraph

AI agents sering perlu menangani tugas-tugas seperti:

1. **Decision-Making:** Evaluating conditions and choosing between multiple actions.
2. **Data Processing:** Analyzing input data and transforming it into useful information.

3. **Interaction with External Systems:** Fetching data from APIs, logging information, or performing other I/O operations.

Di LangGraph, masing-masing tugas ini dapat diurai menjadi node-node individual, yang terhubung oleh edge yang menentukan aliran informasi. Misalnya, bayangkan membangun chatbot untuk customer service. Bot ini mungkin perlu untuk:

- **Memahami Pertanyaan Pengguna:** Mengidentifikasi jenis pertanyaan atau permintaan yang diajukan pengguna.
- **Merutekan Pertanyaan:** Menentukan apakah pertanyaan dapat ditangani secara otomatis atau perlu dieskalasi ke agent manusia.
- **Mencatat Interaksi:** Merekam percakapan untuk referensi atau analisis di masa depan.

Masing-masing tugas ini dapat direpresentasikan oleh node terpisah di LangGraph, yang dapat dengan mudah diperbarui atau dimodifikasi tanpa memengaruhi bagian lain dari workflow. Modularitas ini membuatnya mudah untuk mengembangkan kemampuan bot dari waktu ke waktu. Perlu menambahkan fitur yang merekomendasikan produk kepada pelanggan berdasarkan pertanyaan mereka? Cukup tambahkan node baru, hubungkan ke workflow yang ada, dan Anda siap!

1.7 Aplikasi Dunia Nyata dari AI Agents dengan LangGraph

AI agents yang dibangun dengan LangGraph dapat diterapkan di berbagai industri, termasuk:

- **Customer Service:** Chatbot yang menangani pertanyaan pelanggan, memberikan respons instan, dan mengeskalasi masalah bila diperlukan.
- **Healthcare:** Agent yang menganalisis data pasien, menyediakan pemantauan kesehatan, atau membantu diagnostik awal.
- **Finance:** Agent yang memantau tren pasar, mendeteksi transaksi yang mencurigakan, atau mengelola portofolio.
- **Retail:** Sistem rekomendasi cerdas yang menyarankan produk berdasarkan perilaku pelanggan, mengelola inventaris, atau mengoptimalkan harga.

Setiap aplikasi ini mengandalkan kemampuan AI agent untuk memproses informasi, membuat keputusan, dan mengambil tindakan yang sesuai. Framework LangGraph mendukung hal ini dengan memungkinkan Anda membangun workflow kompleks yang

dapat menangani beberapa titik keputusan dan aliran data, membuatnya ideal untuk mengembangkan AI agents yang sofistikasi.

Ringkasan Singkat

Pada titik ini, Anda seharusnya memiliki pemahaman dasar tentang:

- Apa itu LangGraph dan mengapa ia menggunakan pendekatan berbasis graph.
- Bagaimana node dan edge membentuk fondasi arsitektur LangGraph.
- Mengapa LangGraph sangat cocok untuk membangun AI agents, terutama untuk tugas-tugas yang melibatkan pengambilan keputusan, pemrosesan data, dan interaksi dengan sistem eksternal.

Chapter 2:

Setting Up Your Development Environment

Di bagian selanjutnya, kami akan memandu Anda melalui penyiapan development environment Anda sehingga Anda dapat mulai membangun dengan LangGraph. Pada akhir chapter ini, Anda akan membuat agent "Hello World" sederhana yang mengenalkan Anda pada syntax dan workflow LangGraph.

Untuk mulai membangun dengan LangGraph, penting untuk menyiapkan development environment yang akan membuat pengalaman coding Anda lebih lancar dan terorganisir. Di bagian ini, kami akan membahas langkah-langkah untuk menginstal Python, membuat virtual environment untuk mengelola dependency, dan menyiapkan Visual Studio Code (VS Code) sebagai Integrated Development Environment (IDE) Anda.

Mengapa Virtual Environment?

Virtual environment adalah direktori mandiri yang memungkinkan Anda mengelola dependency spesifik proyek secara terpisah dari environment Python global di komputer Anda. Isolasi ini memastikan bahwa perubahan pada dependency satu proyek tidak akan memengaruhi proyek lainnya. Untuk proyek LangGraph, virtual environment akan membantu Anda mengelola dan menginstal library yang Anda butuhkan dengan mudah.

2.1: Menginstal Python

LangGraph memerlukan Python 3.10 atau lebih tinggi. Jika Anda belum menginstal Python, ikuti langkah-langkah berikut:

1. Download Python:

Visit [python.org](https://www.python.org/downloads/) (<https://www.python.org/downloads/>) and download the latest version for your operating system. Instructions are provided for most operating systems you might be using.



Figure 4: Installing python from the python website.

2. **Install Python:** Run the installer and follow the on-screen instructions. Be sure to check the box or agree to the question that says “**Add Python to PATH**” during installation. This will make it easier to run Python from the command line.
3. **Verify the install:** To verify the installation, open your terminal (or Command Prompt on Windows) and type:

```
terminal  
python --version
```

You should see a version number starting with “3.10” or higher. If you encounter issues, check Python’s installation guide (<https://docs.python.org/3/using/index.html>) for troubleshooting tips.

2.2: Menyiapkan Virtual Environment

Setelah Python terinstal, saatnya membuat virtual environment khusus untuk proyek LangGraph Anda. Ini akan menjaga semua library yang diperlukan tetap terisolasi dan terorganisir di dalam folder proyek.

Buat Folder Proyek: Mulailah dengan membuat folder untuk proyek LangGraph Anda. Anda bisa menamainya seperti LangGraphProjects.

```
terminal
mkdir LangGraphProjects
cd LangGraphProjects
```

Buat Virtual Environment:

Setelah folder proyek Anda dibuat, Anda dapat menyiapkan virtual environment dengan menjalankan perintah berikut:

```
terminal
python -m venv langgraph_env
```

Perintah ini membuat direktori baru bernama langgraph_env di dalam folder proyek Anda, yang akan berisi instalasi Python terpisah beserta library apa pun yang Anda instal.

Aktifkan Virtual Environment:

Di Windows, gunakan perintah berikut:

```
Command prompt
langgraph_env\Scripts\activate
```

```
Command Prompt

D:\>mkdir LangGraphProjects

D:\>cd LangGraphProjects

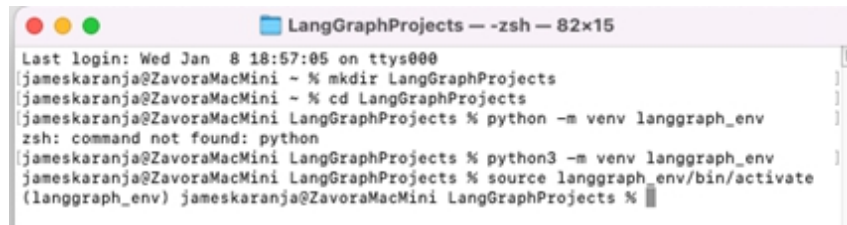
D:\LangGraphProjects>python -m venv langgraph_env
Could not find platform independent libraries <prefix>

D:\LangGraphProjects>langgraph_env\Scripts\activate

(langgraph_env) D:\LangGraphProjects>
```

Di macOS/Linux, gunakan perintah ini:

```
Terminal  
source langgraph_env/bin/activate
```



```
LangGraphProjects — -zsh — 82x15  
Last login: Wed Jan  8 18:57:05 on ttys000  
jameskaranja@ZavoraMacMini ~ % mkdir LangGraphProjects  
jameskaranja@ZavoraMacMini ~ % cd LangGraphProjects  
jameskaranja@ZavoraMacMini LangGraphProjects % python -m venv langgraph_env  
zsh: command not found: python  
jameskaranja@ZavoraMacMini LangGraphProjects % python3 -m venv langgraph_env  
jameskaranja@ZavoraMacMini LangGraphProjects % source langgraph_env/bin/activate  
(langgraph_env) jameskaranja@ZavoraMacMini LangGraphProjects %
```

Gambar 5: Cara mengaktifkan python environment proyek untuk mempelajari LangGraph dalam kursus ini. Semua contoh sebaiknya dikerjakan di dalam folder ini dan di dalam environment yang telah diaktifkan. Ini memastikan Anda tidak memengaruhi proyek python lainnya.

Setelah diaktifkan, prompt terminal Anda seharusnya menampilkan (langgraph_env) di bagian awal, menandakan bahwa Anda kini bekerja di dalam virtual environment. Semua paket Python yang Anda instal akan khusus untuk environment ini, sehingga instalasi Python global Anda tetap tidak terpengaruh.

Troubleshooting Tip: If you encounter an error, double-check the commands and make sure you are in the correct project directory. On Windows, you can manually create the folder using file explorer as an alternative to the command prompt.

2.3: Menginstal LangGraph dan Library yang Diperlukan

Dengan virtual environment sudah diaktifkan, Anda siap menginstal LangGraph. Ini dapat dilakukan menggunakan package manager Python, pip, yang sudah terinstal secara bawaan bersama Python.

Instal LangGraph:

```
Terminal  
pip install langgraph
```

Perintah ini akan mengunduh dan menginstal LangGraph, beserta semua dependency yang dibutuhkannya.

Instal Library Tambahan (Opsional):

Tergantung kebutuhan proyek Anda, Anda mungkin juga ingin menginstal library untuk bekerja dengan API eksternal atau pemrosesan data. Untuk saat ini, mari kita instal requests, sebuah library populer untuk membuat HTTP request, yang akan berguna pada latihan-latihan berikutnya.

```
Terminal  
pip install requests
```

2.4: Menyiapkan Visual Studio Code (VS Code)

VS Code adalah IDE yang ringan namun powerful yang menyediakan fitur-fitur penting seperti syntax highlighting, debugging, dan version control. IDE ini gratis, open-source, dan sangat cocok untuk pengembangan Python.

1. Download VS Code:

Go to [Visual Studio Code's website \(https://code.visualstudio.com\)](https://code.visualstudio.com) and download the installer for your operating system. Run the installer and follow the on-screen instructions to complete the setup.

2. Open the folder where we created the project “LangGraphProjects”. Your vscode should look like below

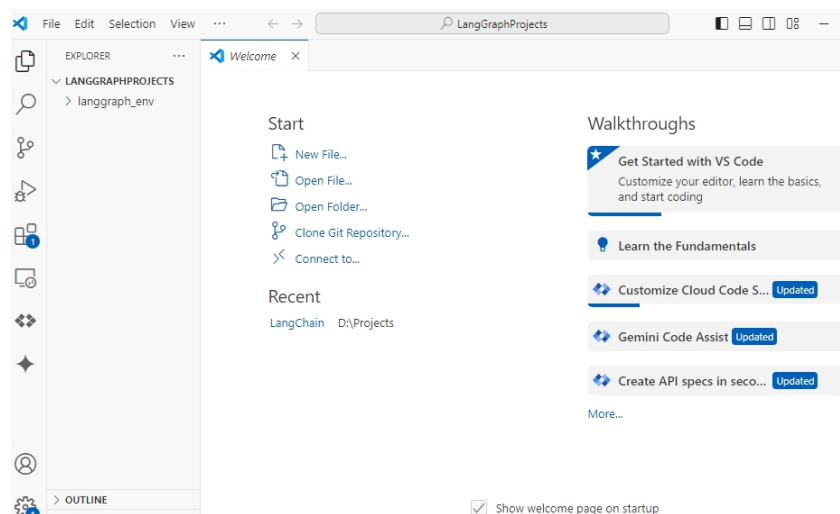


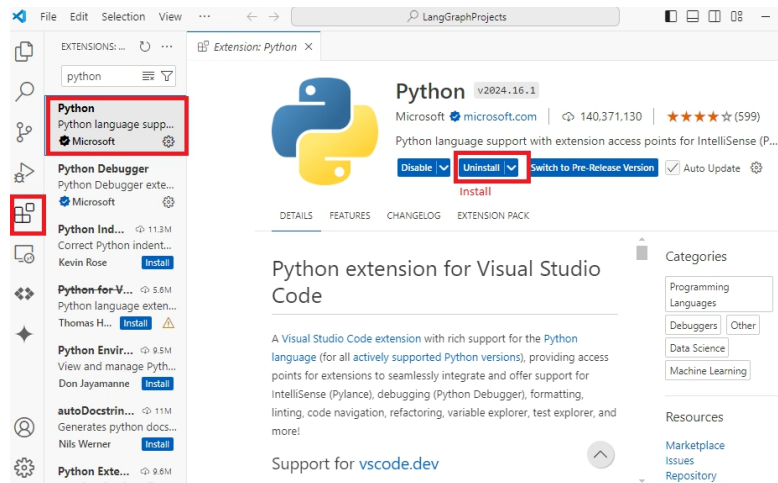
Figure 6: Visual Studio Code with the LangGraphProjects folder opened using a light theme. Your VS Code might show a light theme or a dark mode theme, but it's the same content: Themes are just a matter of personal preference on how it looks.

3. Install the Python Extension:

Open VS Code, and go to the Extensions view by clicking on the Extensions icon in the sidebar or pressing **Ctrl+Shift+X**. Search for “Python” by Microsoft and click **Install**. This extension provides

Python-specific features, such as IntelliSense (code completion), linting, and debugging.

Figure 7: How to open the extensions icon in the sidebar, search for “Python” by Microsoft, and click Install. This is a recommended step for python programming in vscode.



4. Configure VS Code to Use the Virtual Environment:

With the Python extension installed, you can set VS Code to use your virtual environment. Open the Command Palette by pressing **Ctrl+Shift+P**, then type and select “Python: Select Interpreter.” Choose the Python interpreter located in the `langgraph_env` folder. This ensures that VS Code runs Python commands within your virtual environment.

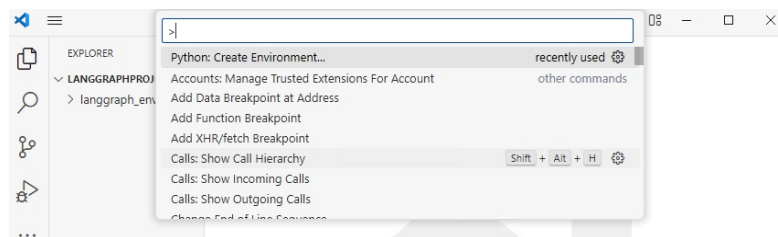


Figure 8: Creating a python environment in VS Code for the LangGraphProjects folder.

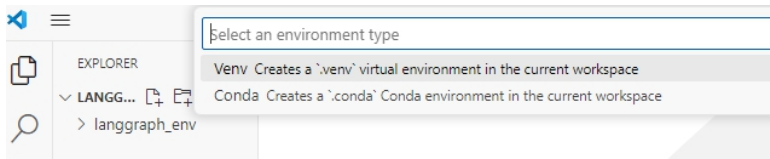


Figure 9: Selecting the environment. Choose between:

- **Venv** : A virtual environment using Python's **venv** module. This is useful for creating isolated environments for project-specific dependencies.
- **Conda** : A Conda environment, ideal for users who prefer managing packages and environments with AnaConda, particularly for data science and machine learning workflows.

In our project, we recommend using **Venv** for the sake of making things simple. You may use Conda if you prefer it.

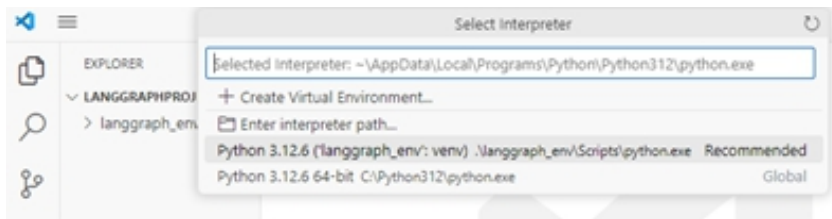


Figure 10: Choose the Python interpreter located in the **langgraph_env** folder. This ensures that VS Code runs Python commands within your virtual environment.

Ringkasan Singkat

Sampai saat ini, Anda seharusnya sudah:

- Menginstal Python dan membuat virtual environment untuk mengisolasi proyek LangGraph Anda.
- Menginstal LangGraph dan library tambahan yang diperlukan untuk membuat API request.
- Menyiapkan VS Code dengan ekstensi Python dan mengonfigurasinya untuk menggunakan virtual environment Anda.

Dengan semuanya sudah siap, Anda kini sudah siap untuk mulai membangun agent LangGraph! Pada bagian berikutnya, kita akan membahas cara membuat agent pertama Anda-sebuah

program "Hello World" sederhana yang akan memperkenalkan Anda pada syntax dan workflow LangGraph.

Lesson 1: Membangun Agent Pertama Anda: "Hello World"

Dengan development environment Anda yang sudah siap, saatnya mulai membangun agent LangGraph pertama Anda-sebuah program "Hello World" sederhana. Latihan ini akan memperkenalkan Anda pada komponen inti LangGraph: node, edge, dan state. Anda akan belajar bagaimana elemen-elemen ini bekerja sama untuk mengontrol alur data melalui agent Anda.

Langkah 1: Memahami Komponen Inti LangGraph

Agent LangGraph dibangun menggunakan tiga komponen utama:

- **Nodes:** Mewakili tugas atau aksi, seperti memproses input atau mengambil keputusan.- **Edges:** Mendefinisikan koneksi antar node, mengontrol urutan aksi.- **States:** Menyimpan dan meneruskan data antar node, memungkinkan agent mempertahankan konteks sepanjang workflow.

Pada contoh ini, Anda akan membuat sebuah node yang memodifikasi pesan sapaan di dalam state dan menghubungkannya untuk membentuk workflow sederhana.

Langkah 2: Mendefinisikan State

Pertama, mari kita definisikan sebuah state untuk menyimpan pesan sapaan Anda. Kita akan menggunakan

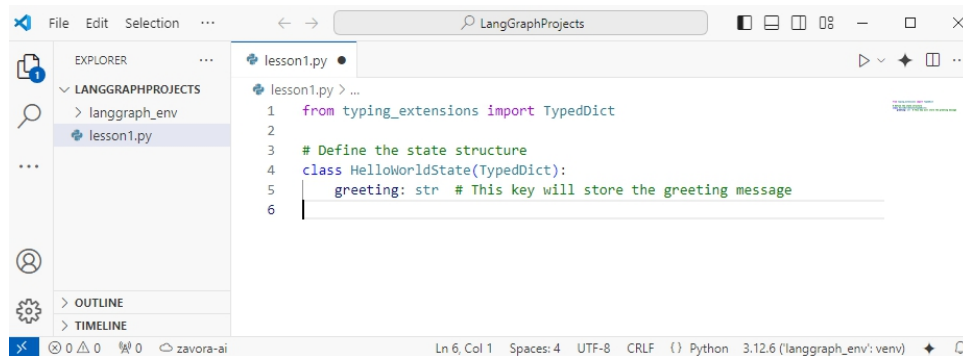
TypedDict milik Python untuk menyusunnya sebagai dictionary dengan key tertentu. Dalam LangGraph, sebuah state merepresentasikan data terkini aplikasi dan sering didefinisikan dengan TypedDict atau BaseModel milik Pydantic untuk memastikan konsistensi dan validasi data.

```
lesson1.py
from typing_extensions import TypedDict
# Define the state structure
class HelloWorldState(TypedDict):
    greeting: str # This key will store the greeting message
```

State HelloWorldState kini didefinisikan sebagai objek mirip dictionary di mana field greeting akan menyimpan sebuah string. Anda akan menginisialisasi pesan ini dan memodifikasinya di dalam workflow.

Gambar 1.1: Di dalam folder VSCode Anda, buat sebuah file bernama lesson1.py lalu tambahkan kode di atas sesuai instruksi di atas. Anda dapat menyalin atau menempel untuk mempercepat

waktu pengetikan, meskipun saya secara pribadi menyarankan untuk mengetik ulang kodenya karena hal ini memperkuat proses belajar dan membuat Anda lebih memahaminya.



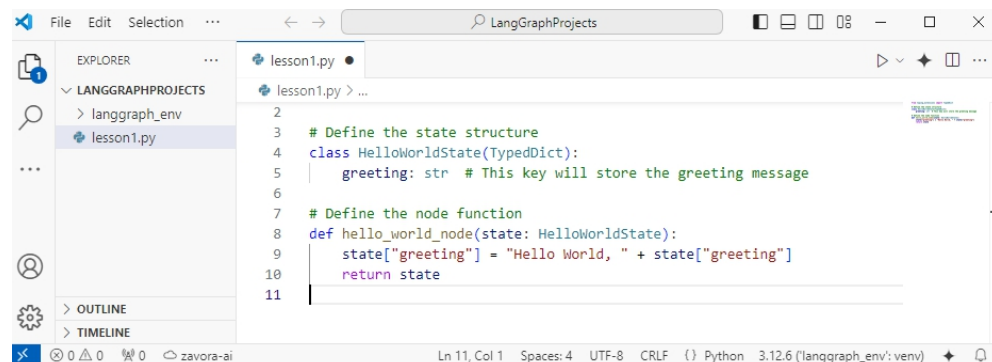
Langkah 3: Membuat Fungsi Node "Hello World"

Selanjutnya, kita akan mendefinisikan sebuah function untuk bertindak sebagai node di LangGraph. Node ini akan memperbarui key greeting di dalam state dengan menambahkan "Hello World" di awal pesan yang sudah ada.

```
lesson1.py continued..
# Define the node function
def hello_world_node(state: HelloWorldState):
    state["greeting"] = "Hello World, " + state["greeting"]
    return state
```

Function ini menerima state, menambahkan "Hello World" di awal greeting, lalu mengembalikan state yang telah diperbarui. Setiap node di LangGraph memiliki akses ke state, yang memungkinkannya membaca dan memodifikasi data sesuai kebutuhan.

Gambar 1.2: Di dalam lesson1.py Anda, kita akan terus menambahkan kode sesuai instruksi. Disarankan agar Anda mengikuti dan mempraktikkannya secara langsung seiring kita melangkah maju.



Langkah 4: Menyiapkan Struktur Graph

Dengan state dan node sudah didefinisikan, saatnya membangun workflow LangGraph. Anda akan membuat sebuah graph yang menghubungkan node-node dengan edge, membangun jalur dari awal hingga akhir.

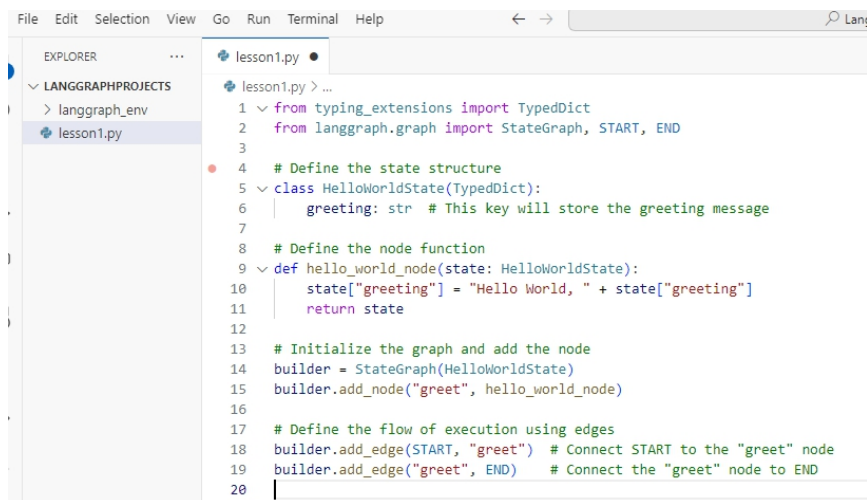
```
lesson1.py continued..

#Add import at beginning of lesson1.py file
from langgraph.graph import StateGraph, START, END
# Initialize the graph and add the node
builder = StateGraph(HelloWorldState)
builder.add_node("greet", hello_world_node)
# Define the flow of execution using edges
builder.add_edge(START, "greet") # Connect START to the "greet" node
builder.add_edge("greet", END) # Connect the "greet" node to END
```

Berikut penjelasan tiap bagiannya:

- `StateGraph(HelloWorldState)`: Menginisialisasi graph dengan schema `HelloWorldState`, yang berarti graph ini akan mengharapkan data yang terstruktur sesuai state ini.
- `add_node("greet", hello_world_node)`: Menambahkan fungsi `hello_world_node` sebagai node di dalam graph, dengan label "greet".
- `add_edge(...)`: Menetapkan alur eksekusi dari `START` ke "greet" lalu ke `END`, membentuk jalur yang lurus.

Figure 1.3: Here is how your code should look like in VS Code so far



Langkah 5: Mengompilasi dan Menjalankan Graph

Dengan graph telah dikonfigurasi, Anda kini dapat mengompilasinya dan menjalankan workflow dengan memanggil graph menggunakan sebuah initial state. Ini akan memicu node untuk memproses data dan menghasilkan

output.

```
lesson1.py continued..  
  
# Compile and run the graph  
graph = builder.compile()  
result = graph.invoke({"greeting": "from LangGraph!"})  
print(result)
```

Output yang diharapkan akan terlihat seperti ini:

```
{'greeting': 'Hello World, from LangGraph!'}
```

Ini menunjukkan bagaimana state diteruskan melalui node "greet" dan dimodifikasi untuk menyertakan "Hello World."

Anda dapat menjalankan program dan melihat hasilnya di jendela terminal, seperti yang ditandai di bawah ini.

2.5 Memvisualisasikan Graph Anda. Kita menggunakan `MermaidDrawMethod` untuk membuat visualisasi PNG dari graph. Ini memberikan representasi grafis dari workflow, membantu Anda melihat koneksi antar node secara visual. Kita akan menggunakan kembali method ini untuk memvisualisasikan graph di sepanjang buku ini.

```
lesson1.py lanjutan..  
  
#Kode untuk memvisualisasikan graph, kita akan menggunakannya kembali di semua  
lesson berikutnya  
from langchain_core.runnables.graph import MermaidDrawMethod  
  
import random  
  
import os  
  
mermaid_png=graph.get_graph(xray=1).draw_mermaid_png(draw_method=MermaidDrawMethod.API)  
  
# Membuat folder output jika belum ada, untuk saat ini kita bisa menyimpannya di folder saat ini yang direpresentasikan oleh .  
  
output_folder = "."  
  
os.makedirs(output_folder, exist_ok=True)  
  
filename = os.path.join(output_folder, f"graph_{random.randint(1, 100000)}.png")  
  
with open(filename, 'wb') as f:  
  
    f.write(mermaid_png)  
  
if sys.platform.startswith('darwin'):  
  
    subprocess.call(('open', filename))  
  
elif sys.platform.startswith('linux'):  
  
    subprocess.call(('xdg-open', filename))  
  
elif sys.platform.startswith('win'):  
  
    os.startfile(filename)
```

File lengkapnya ditampilkan di bawah ini dan juga dapat diperoleh dari GitHub:

<https://github.com/jkmaina/LangGraphProjects/blob/main/chapter2/lesson1.py>

lesson1.py: complete example.

```
import random
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_core.runnables.graph import MermaidDrawMethod
import os
import subprocess
import sys

# Define the state structure
class HelloWorldState(TypedDict):
    greeting: str # This key will store the greeting message

# Define the node function
def hello_world_node(state: HelloWorldState):
    state["greeting"] = "Hello World, " + state["greeting"]
    return state

# Initialize the graph and add the node
builder = StateGraph(HelloWorldState)
builder.add_node("greet", hello_world_node)

# Define the flow of execution using edges
builder.add_edge(START, "greet") # Connect START to the "greet" node
builder.add_edge("greet", END) # Connect the "greet" node to END

# Compile and run the graph
graph = builder.compile()
result = graph.invoke({"greeting": "from LangGraph!"})
print(result)

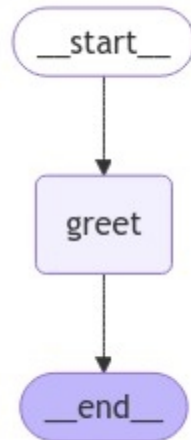
#Code to visualize the graph, we will re-use this in all future lessons
mermaid_png=graph.get_graph(xray=1).draw_mermaid_png(draw_method=MermaidDrawMethod.API)

# Create an output folder if it doesn't exist, for now we can save in the current folder represented by .
output_folder = "."
os.makedirs(output_folder, exist_ok=True)
filename = os.path.join(output_folder, f"graph_{random.randint(1, 100000)}.png")
with open(filename, 'wb') as f:
    f.write(mermaid_png)

if sys.platform.startswith('darwin'):
    subprocess.call(('open', filename))
elif sys.platform.startswith('linux'):
    subprocess.call(('xdg-open', filename))
```

```
elif sys.platform.startswith('win'):  
    os.startfile(filename)
```

Jalankan proyeknya. Anda seharusnya mendapatkan file graph berikut yang dibuat secara otomatis di folder proyek Anda dan ditampilkan secara otomatis di layar Anda.

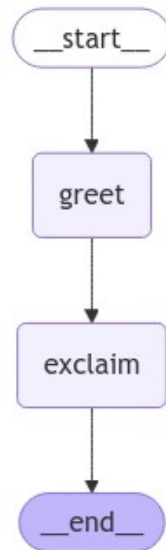


Apa yang Anda Lihat di Visualisasi:

- **Node START terhubung ke node greet, yang kemudian mengarah ke node END.-**
Alur ini mencerminkan urutan yang Anda definisikan, menunjukkan bagaimana data bergerak dari awal ke modifikasi sapaan lalu ke akhir.

2.6 Coba Tambahkan Node Lain ke Workflow Graph

Untuk melihat betapa mudahnya memperluas sebuah workflow, mari kita tambahkan node baru yang menambahkan tanda seru pada sapaan tersebut. Ini akan menunjukkan bagaimana Anda dapat memperluas proyek LangGraph Anda dengan menghubungkan node secara berantai.



Definisikan Node Baru: Tepat di bawah node `hello_world_node` di dalam kode `lesson1.py` Anda, tambahkan node baru di bawah ini. Fungsi baru ini menambahkan tanda seru di akhir sapaan.

```
lesson1.py continued..  
  
# Define an additional node function  
def exclamation_node(state: HelloWorldState):  
    state["greeting"] += "!"  
    return state
```

Tambahkan Node ke Graph dan Definisikan Edge Baru: Sekarang, tambahkan node baru ke graph dan hubungkan ke alur tersebut. Modifikasi node `greet` agar terhubung ke node `exclaim`.

```
lesson1.py continued..  
  
# Define the flow of execution using edges  
builder.add_edge(START, "greet") # Connect START to the "greet" node  
builder.add_node("exclaim", exclamation_node) # Add a new node  
# Update the edges  
builder.add_edge("greet", "exclaim") # Connect "greet" to "exclaim"  
builder.add_edge("exclaim", END) # Update the flow to end after "exclaim"
```

Kompilasi dan Jalankan Graph yang Sudah Diperbarui:

```
lesson1.py continued..  
  
# Compile and run the graph  
graph = builder.compile()  
result = graph.invoke({"greeting": "from LangGraph!"})  
# Output the result
```



```
print(result)
# Output: {'greeting': 'Hello World, from LangGraph!!'}
```

Pengaturan ini menggambarkan betapa mudahnya Anda dapat memperluas workflow LangGraph dengan menghubungkan node secara berantai untuk melakukan berbagai tugas.

2.7 Kode Lengkap untuk Agent "Hello World"

Berikut adalah keseluruhan kode, menggabungkan setiap langkah di atas, termasuk eksperimen opsional dengan node exclaim:

```
lesson1b.py full example

#lesson1b.py
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_core.runnables.graph import MermaidDrawMethod
from display_graph import display_graph
# Define the state structure
class HelloWorldState(TypedDict):
    greeting: str # This key will store the greeting message
# Define an additional node function
def exclamation_node(state: HelloWorldState):
    state["greeting"] += "!"
    return state
# Define the node function
def hello_world_node(state: HelloWorldState):
    state["greeting"] = "Hello World, " + state["greeting"]
    return state
# Initialize the graph and add the node
builder = StateGraph(HelloWorldState)
builder.add_node("greet", hello_world_node)
# Define the flow of execution using edges
builder.add_edge(START, "greet") # Connect START to the "greet" node
builder.add_node("exclaim", exclamation_node) # Add a new node
# Update the edges
builder.add_edge("greet", "exclaim") # Connect "greet" to "exclaim"
builder.add_edge("exclaim", END) # Update the flow to end after "exclaim"
# Compile and run the graph
graph = builder.compile()
result = graph.invoke({"greeting": "from LangGraph!!"})
# Output the result
print(result)
# Output: {'greeting': 'Hello World, from LangGraph!!'}
```

```
#Visualize the graph  
display_graph(graph)
```

Rangkuman Akhir: Apa yang Kita Bangun dan Pelajari

- **Nodes, Edges, dan States:** Ini adalah blok bangunan dari LangGraph. Anggap node sebagai tugas, edge sebagai jalur yang menghubungkan tugas-tugas, dan state sebagai wadah untuk data yang bergerak melalui workflow.- **Membuat Node Sederhana:** Anda membuat sebuah function yang menambahkan "Hello World" ke sebuah pesan. Function ini adalah node kita, dan ini menunjukkan bagaimana node memproses dan memperbarui data.- **Membangun Alur dengan Graph:** Dengan menghubungkan node menggunakan edge, Anda mendefinisikan bagaimana data bergerak dari awal hingga akhir dalam workflow Anda.

Bagaimana Semuanya Bekerja Bersama:

- **Anda melihat bagaimana LangGraph menggunakan sebuah state (data) yang diperbarui saat bergerak melalui node (tugas), semuanya terhubung oleh edge (jalur).** Ini memungkinkan Anda membangun agent yang dapat menangani tugas selangkah demi selangkah.- **Output akhir, {'greeting': 'Hello World, from LangGraph!'}**, menunjukkan bagaimana data dimodifikasi saat melewati workflow sederhana Anda.

Ringkasan

Anda kini telah membangun agent LangGraph pertama Anda dan menambahkan kompleksitas dengan beberapa node:

- **Mendefinisikan sebuah state untuk menyimpan dan memperbarui data.**- **Membuat dan menghubungkan node untuk memanipulasi state.**- **Menjalankan workflow untuk melihat bagaimana LangGraph memproses dan memodifikasi data melalui node dan edge.**

Chapter 3

Memahami Dasar-Dasar Pemrograman

Concepts

Sebelum kita menyelami lebih dalam pembuatan agent LangGraph, mari kita berhenti sejenak dan menjelajahi beberapa konsep pemrograman mendasar. Penjelasan ini akan mencakup dasar-dasar pemrograman Python, struktur data, dan elemen-elemen inti LangGraph. Baik Anda baru mengenal coding maupun butuh penyegaran singkat, halaman ini akan membantu Anda merasa lebih percaya diri saat kita melangkah maju di dalam buku ini.

Apa Itu Bahasa Pemrograman?

Bahasa pemrograman adalah sekumpulan instruksi yang kita gunakan untuk berkomunikasi dengan komputer. Python, bahasa yang kita gunakan dalam buku ini, dikenal mudah dibaca dan ramah bagi pemula. Bahasa ini memungkinkan Anda menulis kode yang mudah dipahami, yang menjadikannya pilihan populer untuk membangun AI agent.

Konsep Kunci dalam Python

Berikut adalah beberapa konsep dasar dalam Python yang akan sering Anda temui saat bekerja dengan LangGraph:

3.1. Variabel

Sebuah variabel adalah nama yang menyimpan sebuah nilai. Anggap sebagai wadah yang menyimpan informasi. Anda dapat membuat sebuah variabel dengan sekadar memberikan sebuah nilai padanya, seperti ini:

```
greeting = "Hello World"
```

Pada contoh ini:

- `greeting` adalah nama variabel.- `"Hello World"` adalah nilainya.- `=` Operator = memberikan nilai tersebut kepada variabel.

Setelah sebuah variabel diberi nilai, Anda dapat merujuk atau memanipulasi nilai tersebut nanti di dalam program:

```
Lesson2.py  
print(greeting) # Output: Hello World
```

`print` adalah sebuah function khusus dalam python yang menampilkan nilai sebuah variabel di output.

Python mendukung berbagai tipe data termasuk:

- **Integer (int):** Bilangan bulat (misalnya, 5, -3).- **Floating-point number (float):** Angka dengan desimal (misalnya, 3.14, -0.5).- **String (str):** Teks yang diapit tanda kutip (misalnya, "Hello, World!").- **Boolean (bool):** True atau False.- **NoneType (None):** Merepresentasikan tidak adanya nilai.- **Tuple:** kumpulan elemen yang berurutan dan tidak dapat diubah (immutable) yang diapit dengan () misalnya (1, "hello", 3.14).- **List:** list adalah kumpulan variabel yang diapit dengan [] dan dipisahkan dengan koma, misalnya `colors = ['red', 'blue', 'green']`- **Set:** Kumpulan yang tidak berurutan, misalnya `items = {"start", "end"}`

Salah satu fitur utama Python adalah sistem dynamic typing-nya. Ini berarti Anda tidak perlu mendeklarasikan tipe sebuah variabel saat membuatnya. Python secara otomatis menentukan tipenya berdasarkan nilai yang diberikan:

```
Lesson2.py continued
```

```

age = 25      # Integer
price = 19.99 # Float
name = "Alice" # String
is_active = True # Boolean
unknown = None # NoneType
my_tuple = (1, "hello", 3.14) # Tuple
print(age) # output is 25
print(price) # output is 19.99
print(name) # output is "Alice"
print(is_active) # output is True
print(my_tuple[0]) # output is 1
print(my_tuple[1]) # output is Hello

```

Fleksibilitas ini membuat Python mudah digunakan, tetapi juga memerlukan pengelolaan yang cermat untuk menghindari error terkait tipe data. Anda dapat mengubah tipe sebuah variabel dengan memberinya nilai dari tipe yang berbeda:

```

Lesson2.py continued

data = "Hello"
print(type(data)) # Output: <class 'str'>
data = 100
print(type(data)) # Output: <class 'int'>
data = "Hello Again"
print(type(data)) # Output: <class 'str'>

```

3.1.1 Penamaan Variabel - Praktik Terbaik:

Berikut adalah aturan utama dan praktik terbaik untuk penamaan variabel di Python:

- **Karakter yang Diizinkan:** Nama variabel dapat mencakup huruf, angka, dan underscore (_). Namun, nama variabel tidak boleh dimulai dengan angka.

```

Lesson2.py continued

user_name = "Alice" # Valid
user2 = "Bob"      # Valid
2user = "Charlie"  # Invalid

```

- **Case Sensitivity:** Python bersifat case-sensitive, artinya UserName dan username akan dianggap sebagai dua variabel yang berbeda.

```
Lesson2.py continued  
  
UserName = "Alice"  
username = "Bob"  
print(UserName) # Output: Alice  
print(username) # Output: Bob
```

- **Snake Case:** Sebagai konvensi, nama variabel di Python biasanya ditulis dalam snake_case (huruf kecil dengan underscore di antara kata).

```
Lesson2.py continued  
  
first_name = "Alice"  
account_balance = 1000.00
```

- **Nama yang Bermakna:** Gunakan nama variabel yang deskriptif dan mencerminkan nilai yang disimpannya atau tujuan yang dilayaninya. Ini membuat kode Anda lebih mudah dipahami.
- **Hindari Menggunakan Keyword Python:** Keyword adalah kata-kata yang dicadangkan dalam Python yang memiliki makna khusus, seperti if, else, for, dll. Hindari menggunakan kata-kata ini sebagai nama variabel untuk mencegah syntax error.

```
Lesson2.py continued  
  
#Invalid variable names  
if = 5 # SyntaxError  
class = 10 # SyntaxError
```

3.1.2 Scope Variabel: Variabel Global dan Lokal

Scope sebuah variabel menentukan di mana variabel tersebut dapat diakses di dalam sebuah program. Python memiliki dua scope utama untuk variabel: global dan lokal. Sebuah variabel global didefinisikan di luar function apa pun dan dapat diakses dari bagian mana pun dari kode tersebut. Variabel global tersedia di sepanjang program.

```
Lesson2.py continued  
  
greeting = "Hello, World!" #This variable can be accessed by many functions  
def say_hello():
```

```
print(greeting)
say_hello() # Output: Hello, World!
```

Sebuah variabel lokal didefinisikan di dalam sebuah function dan hanya dapat diakses di dalam function tersebut. Setelah function selesai dieksekusi, variabel lokal tersebut akan dibuang.

```
Lesson2.py continued
def greet():
    message = "Hi there!"
    print(message)
```

Pada contoh di atas, message adalah sebuah variabel lokal dan hanya ada di dalam function greet().

3.1.3 Manajemen Memori: Python secara otomatis menangani alokasi dan dealokasi memori untuk variabel melalui garbage collection. Proses ini menghapus objek yang tidak terpakai dari memori, membebaskan resource.

3.1.4 Konstanta: Sebagai konvensi, konstanta ditulis dengan semua huruf kapital dengan underscore di antara kata, sebagai penanda kepada programmer lain bahwa nilai-nilai ini tidak boleh diubah.

```
Lesson2.py continued
PI = 3.14159 #ALL CAPS indicates its value should not be changed
MAX_USERS = 100
```

3.1.5 Unpacking Variabel: Python mendukung unpacking beberapa nilai ke beberapa variabel sekaligus, yang dapat menyederhanakan assignment dan membuat kode Anda lebih rapi.

```
Lesson2.py continued

#9. Unpacking a tuple
coordinates = (10, 20)
x, y = coordinates
print(x) # Output: 10
print(y) # Output: 20

#10. Unpacking a list
names = ["Alice", "Bob", "Charlie"]
first, second, third = names
```

```
print(first) # Output: Alice
```

You can also use the `*` operator to capture excess items during unpacking:

```
numbers = [1, 2, 3, 4, 5]
first, *middle, last = numbers
print(middle) # Output: [2, 3, 4]
```

3.2. Function

Sebuah function adalah blok kode yang menjalankan tugas tertentu. Function memungkinkan Anda menggunakan kembali kode, sehingga lebih mudah untuk mengelola dan mengorganisir program Anda. Di Python, Anda mendefinisikan sebuah function menggunakan keyword `def`, diikuti dengan nama function dan tanda kurung `()`:

```
Lesson2.py continued
def say_hello():
    print("Hello World!")
```

Function ini, `say_hello`, menampilkan teks "Hello World!" ketika dipanggil. Body function diindentasi di bawah definisinya, dan function berakhir ketika indentasi berhenti.

Untuk mengeksekusi kode di dalam sebuah function, Anda memanggil function tersebut dengan menggunakan namanya diikuti tanda kurung:

```
Lesson2.py continued
say_hello()
```

3.2.1 Parameter memungkinkan Anda meneruskan data ke dalam function, membuatnya lebih fleksibel. Saat mendefinisikan sebuah function, Anda menentukan parameter sebagai placeholder di dalam tanda kurung.

```
Lesson2.py continued
def greet(name):
    print(f"Hello, {name}!")
```

Saat memanggil function tersebut, Anda memberikan argument, yaitu nilai-nilai aktual yang diberikan ke parameter tersebut.

```
Lesson2.py continued
greet("Alice") # Output: Hello, Alice!
```


3.2.2 Positional argument adalah cara paling umum untuk meneruskan data ke dalam function. Urutan argument pada pemanggilan function harus sesuai dengan urutan parameter pada definisi function.

Urutan argument pada pemanggilan function harus sesuai dengan urutan parameter pada definisi function.

```
Lesson2.py continued
def add(a, b):
    return a + b
print(add(5, 3)) # Output: 8
```

3.2.3 Keyword Argument

Anda juga dapat menentukan argument berdasarkan namanya, terlepas dari urutannya, dengan menggunakan keyword argument. Ini membuat pemanggilan function lebih jelas dan memungkinkan Anda menentukan hanya sebagian parameter jika parameter tersebut memiliki nilai default.

```
Lesson2.py continued
def create_user(name, age=18):
    print(f"User: {name}, Age: {age}")
create_user(name="Bob", age=25) # Output: User: Bob, Age: 25
create_user(name="Alice")      # Output: User: Alice, Age: 18
```

3.2.4 Nilai Parameter Default

Nilai parameter default memungkinkan Anda mendefinisikan parameter opsional yang tidak perlu ditentukan pada setiap pemanggilan function.

```
Lesson2.py continued
def greet(name="World"):
    print(f"Hello, {name}!")
greet()      # Output: Hello, World!
greet("Alice") # Output: Hello, Alice!
```

3.2.5 Argument dengan Panjang Variabel

Terkadang, Anda mungkin tidak tahu sebelumnya berapa banyak argument yang akan diteruskan ke sebuah function. Python menyediakan syntax khusus untuk menangani argument dengan panjang variabel, menggunakan `*args` untuk positional argument dan `**kwargs` untuk keyword argument.

Menggunakan *args untuk Positional Argument dengan Panjang Variabel

Syntax *args memungkinkan sebuah function menerima sejumlah positional argument yang tidak ditentukan jumlahnya, yang dapat diakses sebagai sebuah tuple di dalam function.

```
Lesson2.py continued
def sum_all(*args):
    total = sum(args)
    print(f"Total: {total}")
sum_all(1, 2, 3, 4) # Output: Total: 10
```

Menggunakan **kwargs untuk Keyword Argument dengan Panjang Variabel

Syntax **kwargs memungkinkan sebuah function menerima sejumlah keyword argument yang tidak ditentukan jumlahnya, yang dapat diakses sebagai sebuah dictionary di dalam function.

```
Lesson2.py continued
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
print_info(name="Alice", age=30, country="Wonderland")
# Output:
# name: Alice
# age: 30
# country: Wonderland
```

3.2.4 Return Statement

Sebuah function dapat mengembalikan sebuah nilai menggunakan return statement. Jika tidak ada return statement yang ditentukan, function tersebut akan mengembalikan None secara default.

```
Lesson2.py continued
def add(a, b):
    return a + b
result = add(5, 3)
print(result) # Output: 8
```

Beberapa nilai dapat dikembalikan dari sebuah function sebagai sebuah tuple:

```
Lesson2.py continued
```

```
def get_user():
    return "Alice", 30
name, age = get_user()
print(name, age) # Output: Alice 30
```

3.2.5 Fungsi Anonim dengan Ekspresi Lambda

Fungsi lambda adalah fungsi anonim yang ringkas, didefinisikan dengan kata kunci lambda. Fungsi ini ideal untuk fungsi singkat dan sekali pakai yang diteruskan sebagai argumen.

```
Lesson2.py continued

add = lambda x, y: x + y #Take two numbers, add them together and return the value
print(add(3, 5)) # Output: 8
# Using lambda in a higher-order function
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x ** 2, numbers))
print(squared) # Output: [1, 4, 9, 16]
```

3.2.6 Decorator: Decorator adalah fungsi tingkat tinggi (higher-order function) yang memodifikasi perilaku fungsi lain. Decorator berguna untuk memperluas fungsionalitas, seperti logging, caching, atau kontrol akses, tanpa mengubah kode fungsi aslinya.

```
Lesson2.py continued

def log_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"Executing {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_decorator
def greet(name):
    print(f"Hello, {name}!")

greet("Alice") # Output: Executing greet
# Hello, Alice!
```

Dalam LangGraph, decorator dapat digunakan untuk membungkus fungsi node, menambahkan perilaku logging atau penanganan error.

3.2.7 Fungsi Rekursif: Fungsi rekursif adalah fungsi yang memanggil dirinya sendiri. Fungsi rekursif berguna untuk menyelesaikan masalah yang dapat dipecah menjadi subproblem yang lebih kecil dan serupa, seperti menghitung faktorial atau menelusuri tree.

```
Lesson2.py continued

def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)
print(factorial(5)) # Output: 120
```

Saat menggunakan rekursi, pastikan ada base case untuk mencegah rekursi tak terbatas.

3.2.8 Docstring dan Dokumentasi

Docstring adalah string yang mendeskripsikan tujuan dan perilaku suatu fungsi. Menyertakan docstring membuat kode Anda lebih self-documenting, dan dapat diakses melalui fungsi bawaan Python `help()`.

```
Lesson2.py continued

def greet(name):
    """Greets a person by name."""
    print(f"Hello, {name}!")
help(greet) # Output:
# Help on function greet in module __main__:
# greet(name)
# Greets a person by name.
```

Dengan menyertakan docstring, Anda mempermudah orang lain (dan diri Anda sendiri) untuk memahami fungsi-fungsi Anda dan perilaku yang diharapkan.

3.3. Dictionary

Dictionary adalah struktur data dalam Python yang menyimpan data sebagai pasangan **key-value**. Setiap key dikaitkan dengan sebuah value, sehingga memudahkan pencarian informasi berdasarkan key tertentu. Berikut contohnya:

```
Lesson2.py continued
```

```
person = { "name": "Alice", "age": 30, "is_active": True }
```

Dalam dictionary ini, "name", "age", dan "is_active" adalah key. "Alice", 30, dan True adalah value yang terkait dengan key-key tersebut.

person["name"] akan mengembalikan "Alice", dan person["age"] akan mengembalikan 30.

Dictionary sangat berguna untuk menyimpan dan mengakses informasi yang saling terkait. Oleh karena itu, dictionary dalam Python adalah kumpulan item yang tidak berurutan (unordered), di mana setiap item disimpan sebagai pasangan key-value. Key harus unik dan immutable (misalnya, string, angka, atau tuple), sedangkan value dapat berupa tipe data apa pun, termasuk list, dictionary lain, atau bahkan function.

Beberapa cara untuk membuat dictionary:

```
Lesson2.py continued

#Using curly braces
user_info = { "name": "Alice", "age": 30 }

#Using the dict() constructor
user_info = dict([("name", "Alice"), ("age", 30)])

user_info = dict(name="Alice", age=30)
```

Anda dapat mengakses value dalam dictionary dengan merujuk pada key-nya di dalam tanda kurung siku [].

```
Lesson2.py continued

user_info = { "name": "Alice", "age": 30 }
print(user_info["name"]) # Output: Alice
```

Untuk menghindari error, Anda dapat menggunakan method get(), yang mengembalikan None (atau default yang ditentukan) jika key tidak ada:

```
Lesson2.py continued

print(user_info.get("location", "Unknown")) # Output: Unknown
```

Anda dapat menambah, memperbarui, dan menghapus item dalam dictionary dengan mudah:

3.3.1 Menambahkan item: Cukup dengan memberikan value ke sebuah key. Jika key sudah ada, value-nya akan diperbarui; jika key belum ada, pasangan key-value baru akan ditambahkan.

```
Lesson2.py continued
```

```
user_info["location"] = "Wonderland" # Adds a new key-value pair
user_info["age"] = 31                # Updates the value for the existing key
print(user_info)
```

3.3.2 Menghapus item dari dictionary: Python menyediakan beberapa metode untuk menghapus item dari dictionary:

pop(key): Menghapus item dengan key yang ditentukan dan mengembalikan value-nya.

```
Lesson2.py continued

age = user_info.pop("age")
print(age) # Output: 31
print(user_info)
# Output: {'name': 'Alice', 'location': 'Wonderland'}
del: Deletes an item or the entire dictionary.
del user_info["location"]
del user_info # Deletes the entire dictionary
popitem(): Removes and returns the last inserted key-value pair.
last_item = user_info.popitem()
```

clear(): Mengosongkan dictionary.

```
Lesson2.py continued

user_info.clear()
```

3.3.3 Method dictionary: Dictionary Python dilengkapi dengan banyak method bawaan yang memudahkan pengerjaan dengan dictionary secara efisien.

keys(), values(), dan items()

Method-method ini digunakan untuk mengambil key, value, dan pasangan key-value secara berturut-turut dari sebuah dictionary:

```
Lesson2.py continued

user_info = {"name": "James", "age": 45}
keys = user_info.keys() # Returns a view of the keys
print(keys)
values = user_info.values() # Returns a view of the values
print(values)
items = user_info.items() # Returns a view of the key-value pairs
```

```
print(items)
```

Anda dapat melakukan iterasi pada dictionary menggunakan for loop. Anda dapat memilih untuk melakukan iterasi pada key, value, atau keduanya.

```
Lesson2.py continued

for key in user_info:
    print(key)
for value in user_info.values():
    print(value)
for key, value in user_info.items():
    print(key, value)
```

update()

Method update() memungkinkan Anda menambahkan pasangan key-value dari dictionary lain atau iterable berisi pasangan key-value. Jika ada key yang tumpang tindih, value-nya akan diperbarui.

```
Lesson2.py continued

user_info.update({"location": "Nairobi", "age": 32})
```

setdefault()

Method setdefault() mengembalikan value dari key yang ditentukan. Jika key tersebut tidak ada, method ini akan menyisipkan key tersebut dengan value yang ditentukan.

```
Lesson2.py continued

user_info.setdefault("is_active", True)
```

3.3.4 Nested Dictionary

Sebuah dictionary dapat berisi dictionary lain, membentuk struktur bersarang (nested). Nested dictionary berguna untuk menyimpan data yang kompleks dan umum digunakan dalam LangGraph untuk mengelola state dengan beberapa lapisan informasi.

```
Lesson2.py continued

user_info = {
    "name": "Alice",
    "contact": {
```

```
        "email": "alice@example.com",
        "phone": "123-456-7890"
    }
}

print(user_info["contact"]["email"]) # Output: alice@example.com
```

3.3.5 Mengakses Value Nested Dictionary

Anda dapat mengakses value nested dictionary dengan merangkai pemanggilan `[]` atau `get()`.

```
Lesson2.py continued

email = user_info["contact"]["email"]
phone = user_info.get("contact", {}).get("phone", "Not available")
print(email, phone)
```

3.3.6 Memodifikasi Value Nested Dictionary

Anda dapat memperbarui value bersarang dengan mengaksesnya melalui key masing-masing.

```
Lesson2.py continued

user_info["contact"]["email"] = "new_email@example.com"
```

3.3.7 Dictionary Comprehension

Python mendukung dictionary comprehension, yang memungkinkan Anda membuat dictionary dengan cara yang ringkas dan mudah dibaca. Ini sangat berguna untuk transformasi data dalam LangGraph. Dictionary comprehension adalah teknik untuk membuat dictionary Python dalam satu baris. Metode ini membuat dictionary dari objek iterable, seperti list, tuple, atau dictionary lain.

```
Lesson2.py continued

# Creating a dictionary with squares of numbers
squares = {x: x ** 2 for x in range(5)}
print(squares)
# Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

3.3.8 Menyalin Dictionary

Ketika Anda memberikan (assign) sebuah dictionary ke variabel baru, sebenarnya Anda membuat referensi ke dictionary asli. Oleh karena itu, perubahan pada variabel

baru juga akan memengaruhi dictionary aslinya. Untuk membuat salinan terpisah, gunakan method `copy()`.

```
Lesson2.py continued

user_info_copy = user_info.copy()
user_info_copy["name"] = "Bob"
print(user_info["name"]) # Output: Alice
print(user_info_copy["name"]) # Output: Bob
```

Untuk deep copy (nested dictionary), gunakan modul `copy`.

```
Lesson2.py continued

import copy
deep_copy = copy.deepcopy(user_info)
print(deep_copy)
```

3.3.9 Menggabungkan Dictionary: Python 3.9 memperkenalkan operator `|` untuk menggabungkan dua dictionary. Operator `|=` memperbarui sebuah dictionary secara langsung (in place).

```
Lesson2.py continued

dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}
# Merging
merged = dict1 | dict2
# Output: {'a': 1, 'b': 3, 'c': 4}
# In-place merge
dict1 |= dict2
# Output: {'a': 1, 'b': 3, 'c': 4}
```

3.4. Apa itu TypedDict?

TypedDict adalah fitur yang diperkenalkan dalam modul `typing` Python (dan kemudian `typing_extensions` untuk kompatibilitas mundur) yang memungkinkan Anda mendefinisikan dictionary dengan struktur tertentu.

Dalam Python, TypedDict adalah jenis dictionary khusus di mana Anda mendefinisikan key spesifik dan tipe value yang dapat dimiliki masing-masing key. Ini membantu memastikan struktur data Anda digunakan secara konsisten, yang dapat mencegah error. Berikut cara mendefinisikan TypedDict:

```
Lesson2.py continued

from typing_extensions import TypedDict
class Person(TypedDict):
    name: str
    age: int
```

Dalam contoh ini, TypedDict Person menentukan bahwa dictionary tersebut harus memiliki key name (berisi string) dan key age (berisi integer). Jika Anda mencoba menyimpan tipe data lain, Python akan memunculkan error, membantu Anda menjaga kode tetap bersih dan bebas error.

```
Lesson2.py continued

person : Person = {"name": "Alice", "age": 30}
print(person["name"]) # Output: Alice
```

Sebagai alternatif, Anda dapat menggunakan fungsi factory TypedDict, yang berguna saat membuat tipe secara dinamis atau ketika nama tipe tidak dapat menjadi nama class yang valid.

```
Lesson2.py continued

Person = TypedDict('Person', {'id': int, 'name': str, 'age': int})
```

3.4.1 TypedDict dengan Nested Dictionary

TypedDict sangat berguna ketika Anda perlu mendefinisikan struktur bersarang (nested) yang kompleks, yang umum ditemukan dalam LangGraph untuk merepresentasikan state dengan beberapa lapisan informasi.

Dengan nested TypedDict, Anda dapat memastikan bahwa struktur dictionary bagian luar maupun bagian dalam mematuhi tipe yang telah ditentukan.

```
Lesson2.py continued

class Address(TypedDict):
    street: str
    city: str
    zip_code: int
class UserProfile(TypedDict):
    username: str
    email: str
    address: Address
```

```

profile: UserProfile = {
    "username": "johndoe",
    "email": "johndoe@example.com",
    "address": {
        "street": "123 Elm St",
        "city": "Metropolis",
        "zip_code": 12345
    }
}

```

3.4.2 Menggunakan TypedDict dalam Function

TypedDict sangat bernilai dalam parameter function dan tipe return. Menentukan TypedDict sebagai type hint memungkinkan Anda memastikan bahwa dictionary apa pun yang diteruskan ke atau dikembalikan oleh function mengikuti struktur yang diharapkan.

4.3 TypedDict Sebagai Parameter Function

```

Lesson2.py continued

class User(TypedDict):
    name: str
    age: int
    is_active: bool

def display_user_info(user: User) -> None:
    print(f"Name: {user['name']}, Age: {user['age']}, Active: {user['is_active']}")

user_data = {"name": "Alice", "age": 30, "is_active": True}
display_user_info(user_data)

3.4.3 TypedDict As Return Types

def create_user(name: str, age: int) -> User:
    return {"name": name, "age": age, "is_active": True}

new_user = create_user("Bob", 25)
print(new_user)

#Output {'name': 'Bob', 'age': 25, 'is_active': True}

```

Dengan menentukan User sebagai tipe return, Anda memastikan bahwa dictionary yang dikembalikan oleh create_user harus sesuai dengan struktur yang didefinisikan oleh User.

3.4.4 Menggunakan TypedDict dengan Nilai Default

Berbeda dengan dictionary biasa, TypedDict tidak mendukung penetapan nilai default secara langsung untuk key-nya. Namun, Anda dapat menggunakan `setdefault()` atau menginisialisasi dictionary dengan helper function untuk memastikan nilai default.

Dengan menggunakan helper function, Anda dapat menginisialisasi TypedDict dengan nilai default sesuai kebutuhan.

```
Lesson2.py continued

class ServerConfig(TypedDict):
    host: str
    port: int
    use_ssl: bool
def create_default_server_config() -> ServerConfig:
    return {
        "host": "localhost",
        "port": 8080,
        "use_ssl": False
    }
config = create_default_server_config()
```

3.4.5 Inheritance dengan TypedDict

Inheritance pada TypedDict memungkinkan Anda memperluas TypedDict yang sudah ada dengan key tambahan. Ini berguna untuk membuat dictionary khusus berdasarkan tipe dictionary dasar.

```
Lesson2.py continued

class BaseUser(TypedDict):
    username: str
    email: str
class AdminUser(BaseUser):
    access_level: int
admin: AdminUser = {
    "username": "admin",
    "email": "admin@example.com",
    "access_level": 5
}
```

Di sini, AdminUser mewarisi struktur dari BaseUser dan menambahkan key access_level tambahan.

3.5. Class dalam Python: Fondasi Object-Oriented Programming

Class adalah tulang punggung Object-Oriented Programming (OOP) dalam Python. Class memungkinkan Anda membuat struktur data kustom yang merepresentasikan entitas dunia nyata, dengan data (attribute) dan fungsionalitas (method) yang dienkapsulasi di dalamnya. Ini membuat class sangat kuat untuk mengelola perilaku kompleks dan menjaga kode tetap terorganisir. Dalam LangGraph, class sering digunakan untuk merepresentasikan berbagai entitas, seperti node, state, dan agent. Bagian ini akan memandu Anda melalui dasar-dasar class Python, termasuk attribute, method, inheritance, encapsulation, dan konsep lanjutan seperti class method dan static method.

3.5.1 Apa itu Class?

Class adalah cetak biru (blueprint) untuk membuat object, yang merupakan instance dari class tersebut. Class mendefinisikan sekumpulan attribute dan method yang akan dimiliki oleh object yang dibuat. Menggunakan class memungkinkan Anda mengelompokkan data dan fungsionalitas yang terkait, memberikan cara terstruktur untuk memodelkan konsep dunia nyata.

```
Lesson2.py continued

class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Creating an object of the Dog class
my_dog = Dog("Buddy", 3)
print(my_dog.name) # Output: Buddy
print(my_dog.age)  # Output: 3
```

Dalam contoh ini:

- **Dog** adalah class-nya.- **__init__()** adalah method khusus (initializer) yang dijalankan saat Anda membuat instance dari class tersebut. Method ini menginisialisasi attribute object.- **my_dog** adalah instance (atau object) dari class **Dog**.

3.5.2 Attribute dan Method:

Attribute adalah variabel yang dimiliki oleh sebuah class. Ada dua jenis attribute:

- **Instance Attribute:** Didefinisikan di dalam method dan spesifik untuk setiap instance dari class.- **Class Attribute:** Didefinisikan langsung di dalam class dan dibagikan di antara semua instance.

```
Lesson2.py continued

class Dog:
    species = "Canis lupus" # Class attribute
    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute
# Accessing attributes
dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)
print(dog1.species) # Output: Canis lupus
print(dog1.name) # Output: Buddy
```

3.5.2.1 Method

Method adalah function yang didefinisikan di dalam class yang mendeskripsikan perilaku dari object yang dibuat dari class tersebut.

```
Lesson2.py continued

class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def bark(self):
        return f"{self.name} says woof!"
dog = Dog("Buddy", 3)
print(dog.bark()) # Output: Buddy says woof!
```

3.5.3 Method __init__()

Method `__init__()` adalah method khusus dalam class Python yang dikenal sebagai initializer atau constructor. Method ini dipanggil secara otomatis ketika instance baru dari class dibuat, dan memungkinkan Anda menginisialisasi instance attribute.

```
Lesson2.py continued

class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
my_car = Car("Toyota", "Corolla", 2021)
print(my_car.make) # Output: Toyota
```

3.5.4 Kata Kunci self

Dalam Python, self merujuk pada instance dari class dan digunakan untuk mengakses instance attribute dan method dari dalam class. Saat Anda mendefinisikan method dalam sebuah class, self adalah parameter pertama, yang merepresentasikan instance yang memanggil method tersebut.

3.5.5 Inheritance

Inheritance memungkinkan Anda membuat class baru berdasarkan class yang sudah ada, mewarisi attribute dan method-nya. Ini mendorong penggunaan ulang kode (code reuse) dan membangun hubungan antar class.

```
Lesson2.py continued

class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "Some generic sound"
class Dog(Animal): # Dog inherits from Animal
    def speak(self):
        return f"{self.name} says woof!"
dog = Dog("Buddy")
print(dog.speak()) # Output: Buddy says woof!
```

Dalam contoh ini:

- **Animal** adalah base class atau parent class.- **Dog** adalah derived class atau child class, yang mewarisi properti dari Animal dan meng-override method **speak**.

3.5.5.1 Multiple Inheritance

Python mendukung multiple inheritance, di mana sebuah class dapat mewarisi dari lebih dari satu base class. Ini berguna dalam sistem yang kompleks, meskipun dapat menimbulkan ambiguitas jika tidak dikelola dengan hati-hati.

```
Lesson2.py continued

class Walker:
    def walk(self):
        return "Walking..."
class Swimmer:
    def swim(self):
        return "Swimming..."
class Amphibian(Walker, Swimmer):
    pass
frog = Amphibian()
print(frog.walk()) # Output: Walking...
print(frog.swim()) # Output: Swimming...
```

3.5.6 Encapsulation dan Access Modifier

Encapsulation adalah praktik membatasi akses ke attribute atau method tertentu untuk melindungi integritas data. Python menggunakan underscore untuk menandai level akses yang berbeda:

- **Public Attribute:** Dapat diakses dari mana saja (default di Python).-
- Protected Attribute:** Diawali dengan satu underscore (_). Ini adalah konvensi untuk menandakan bahwa attribute tersebut sebaiknya tidak diakses secara langsung.-
- Private Attribute:** Diawali dengan dua underscore (__). Attribute ini mengalami name-mangling dan tidak dapat diakses secara langsung dari luar class.

```
Lesson2.py continued

class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self._balance = balance # Protected

    def __withdraw(self, amount): # Private method
        if amount <= self._balance:
            self._balance -= amount
            return self._balance
        return "Insufficient funds"
```



```
account = BankAccount("Alice", 1000)
print(account._balance) # Accessing a protected attribute (not recommended)
```

3.5.7 Class Method dan Static Method

3.5.7.1 Class Method

Class method adalah method yang terikat pada class, bukan pada instance. Method ini memiliki akses ke class itu sendiri melalui parameter cls dan didefinisikan dengan decorator @classmethod.

```
Lesson2.py continued
class Person:
    species = "Homo sapiens"

    @classmethod
    def set_species(cls, new_species):
        cls.species = new_species

Person.set_species("Homo sapiens sapiens")
print(Person.species) # Output: Homo sapiens sapiens
```

3.5.7.2 Static Method

Static method adalah fungsi utilitas yang tidak memerlukan akses ke attribute class atau instance. Static method didefinisikan dengan decorator @staticmethod.

```
Lesson2.py continued
class Math:
    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(3, 5)) # Output: 8
```

3.5.8 Magic Method

Magic method (juga dikenal sebagai dunder method) adalah method khusus dalam Python yang diawali dan diakhiri dengan double underscore (__). Method ini memungkinkan Anda

mendefinisikan perilaku class Anda dalam berbagai operasi bawaan, seperti +, -, atau print().

3.5.8.1 __str__:

Mendefinisikan perilaku untuk str() dan print() agar mengembalikan representasi string yang mudah dibaca pengguna (user-friendly). __repr__:

Mengembalikan representasi string yang tidak ambigu yang dapat digunakan untuk membuat ulang object tersebut.

```
Lesson2.py continued

class Car:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def __str__(self):
        return f"{self.make} {self.model}"

    def __repr__(self):
        return f"Car('{self.make}', '{self.model}')"

car = Car("Toyota", "Corolla")
print(car) # Output: Toyota Corolla
print(repr(car)) # Output: Car('Toyota', 'Corolla')
```

3.5.8.2 Operator Overloading:

Magic method juga memungkinkan Anda mendefinisikan perilaku untuk operator bawaan.

```
Lesson2.py continued

class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

v1 = Vector(1, 2)
v2 = Vector(3, 4)
result = v1 + v2
print(result.x, result.y) # Output: 4 6
```

3.5.9 Property:

Property dalam Python memungkinkan Anda mengontrol akses attribute. Decorator `@property` memungkinkan Anda mendefinisikan method yang diakses seperti attribute, membuat kode Anda lebih bersih dan lebih aman.

```
Lesson2.py continued

class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    @property
    def area(self):
        return self.width * self.height

rect = Rectangle(5, 10)
print(rect.area) # Output: 50
```

Anda juga dapat mendefinisikan setter untuk property guna mengontrol bagaimana attribute diatur.

```
Lesson2.py continued

class Rectangle:
    def __init__(self, width, height):
        self._width = width
        self._height = height

    @property
    def width(self):
        return self._width

    @width.setter
    def width(self, value):
        if value > 0:
            self._width = value
        else:
```

```
raise ValueError("Width must be  
positive")
```

3.5.10 Best Practice dengan Class

- **Gunakan Nama Class yang Deskriptif:** Pilih nama yang secara jelas mendeskripsikan tujuan class tersebut.
- **Enkapsulasi Data:** Gunakan private attribute dan public method untuk melindungi data dan mengontrol akses.
- **Utamakan Composition daripada Inheritance:** Gunakan inheritance secukupnya dan utamakan composition untuk mengurangi kompleksitas dan meningkatkan fleksibilitas.
- **Gunakan Property untuk Akses Terkontrol:** Gunakan property untuk mengelola akses ke instance attribute dengan aman.
- **Dokumentasikan Class Anda:** Sertakan docstring untuk menjelaskan tujuan dan penggunaan class serta method Anda.

3.5.11. Modul Eksternal Umum yang Digunakan dalam LangGraph

LangGraph memanfaatkan beberapa modul Python eksternal untuk meningkatkan fungsionalitasnya, menyederhanakan proses, dan mengelola tugas-tugas yang kompleks. Modul-modul ini mendukung berbagai fungsionalitas, termasuk data typing, pemrograman asinkron, interaksi API, dan visualisasi data. Memahami modul-modul ini akan memberi Anda alat yang dibutuhkan untuk membangun aplikasi LangGraph yang tangguh. Berikut ini, kita akan menjelajahi beberapa modul eksternal yang paling umum digunakan dalam pengembangan LangGraph.

a) typing dan typing_extensions

LangGraph sering membutuhkan type hint untuk memastikan konsistensi data dan menjaga keterbacaan kode. Modul typing dan typing_extensions sangat berharga untuk menentukan dan menegakkan tipe, khususnya untuk struktur data kompleks seperti dictionary dan function.

Komponen Utama:

- **TypedDict:** Memungkinkan dictionary dengan pasangan key-value spesifik dan type hint.
- **List, Dict, Tuple:** Type hint untuk struktur data dasar.
- **Union:** Memungkinkan sebuah variabel menjadi salah satu dari beberapa tipe yang ditentukan.
- **Optional:** Digunakan ketika sebuah variabel dapat bernilai None.

```
Lesson2.py continued

from typing import List, Dict, Optional

def process_data(data: Dict[str, Optional[List[int]]]) -> None:
    print(data)
```

Di Mana Digunakan dalam LangGraph:

Modul-modul ini penting saat mendefinisikan struktur data kompleks untuk node state atau parameter, memungkinkan pengecekan tipe yang kuat dan kejelasan kode.

b) `asyncio`

Aplikasi LangGraph sering melibatkan tugas yang dapat memperoleh manfaat dari eksekusi asinkron, seperti network request atau pemrosesan data secara konkuren. Modul `asyncio` adalah library bawaan Python untuk pemrograman asinkron, memungkinkan operasi non-blocking dan penanganan tugas I/O-bound secara efisien.

Komponen Utama:

- **`async` dan `await`:** Untuk mendefinisikan fungsi asinkron.-
- `asyncio.run()`:** Menjalankan event loop asinkron utama.-
- `gather()`:** Menjalankan beberapa task asinkron secara konkuren.

```
Lesson2.py continued

import asyncio

async def fetch_data():
    await asyncio.sleep(1)
    return "Data fetched!"

async def main():
    results = await asyncio.gather(fetch_data(), fetch_data())
    print(results)

asyncio.run(main())
```

Di Mana Digunakan dalam LangGraph:

Gunakan asyncio ketika Anda perlu melakukan beberapa tugas secara bersamaan, seperti memproses data secara paralel atau menunggu beberapa respons API.

c) requests

Meskipun LangGraph sendiri mungkin tidak selalu memerlukan panggilan API langsung, hal yang umum adalah berinteraksi dengan API eksternal untuk pengambilan data, model serving, atau integrasi dengan layanan lain. Modul requests adalah library utama untuk menangani HTTP request dalam Python.

Komponen Utama:

- **get(), post(), put(), delete():** Method untuk membuat HTTP request.- **json():** Mengurai (parse) respons JSON langsung menjadi dictionary Python.- **headers, params, data:** Untuk menyesuaikan request.

```
Lesson2.py continued
import requests
try:
    response = requests.get("https://api.example.com/data")
    if response.status_code == 200:
        data = response.json()
        print(data)
except requests.exceptions.RequestException as e:
    print(f"Error: {e}")
```

Di Mana Digunakan dalam LangGraph:

Gunakan requests untuk setiap interaksi HTTP yang dibutuhkan oleh node LangGraph Anda, seperti mengambil data dari API eksternal atau mengirim hasil ke sebuah web service.

d) pydantic

Pydantic adalah library validasi data yang berbasis pada type hint Python. Library ini sangat berguna dalam LangGraph ketika Anda perlu memastikan bahwa data yang masuk atau keluar dari sebuah node memenuhi kriteria tertentu. Model Pydantic dapat menegakkan typing yang ketat, menyediakan nilai default, dan memvalidasi data terhadap skema.

Komponen Utama:

- **BaseModel:** Ini adalah class fondasi tempat semua model Pydantic mewarisi. Ketika Anda mendefinisikan class yang mewarisi dari BaseModel, Anda dapat menentukan tipe data yang diharapkan untuk setiap attribute. Pydantic kemudian akan menegakkan tipe-tipe ini saat Anda membuat instance dari class tersebut.
- **Field:** Fungsi ini memungkinkan Anda menambahkan metadata pada field model, seperti constraint (misalnya, menetapkan nilai minimum) dan nilai default. Dalam contoh di bawah, `Field(..., gt=0)` menentukan bahwa field `age` harus lebih besar dari nol.
- **ValidationError:** Jika data yang diteruskan ke model Pydantic tidak sesuai dengan skema yang dibutuhkan, sebuah `ValidationError` akan dimunculkan. Exception ini memberikan informasi rinci tentang field mana yang gagal validasi dan alasannya.

```
Lesson2.py continued

from pydantic import BaseModel, Field, ValidationError

class User(BaseModel):
    name: str
    age: int = Field(..., gt=0)

try:
    user = User(name="Alice", age=-1)
except ValidationError as e:
    print(e)
```

Di Mana Digunakan dalam LangGraph:

Pydantic ideal untuk memvalidasi input node, memastikan data state sesuai dengan tipe yang diharapkan, dan melakukan serialisasi struktur data yang kompleks.

e) logging

Logging sangat penting dalam aplikasi LangGraph, terutama saat melakukan deployment di lingkungan produksi, karena memberikan wawasan tentang operasi aplikasi, membantu mendiagnosis masalah, dan membantu menjaga sistem tetap stabil. Modul logging Python adalah alat bawaan yang memungkinkan logging yang komprehensif dan dapat disesuaikan, sehingga memudahkan pelacakan dan pengelolaan log untuk berbagai tingkat keparahan (severity).

Penjelasan Komponen Utama:

1. **basicConfig()** : This function sets up the basic configuration for logging, such as the log level, format, and output destination. By setting the **level**, you control which messages are recorded, filtering out messages below the specified level.
2. **Logging Levels**: These predefined levels determine the severity of messages to log:
 - **DEBUG**: Detailed information for diagnosing problems.
 - **INFO**: General information about program execution.
 - **WARNING**: Indicates a potential problem or something unexpected.
 - **ERROR**: A significant issue that has prevented some part of the program from functioning.
 - **CRITICAL**: A serious error, likely leading to program termination.
3. **getLogger()** : This function returns a logger instance associated with a particular module or component. Using loggers allows you to create different log streams for different parts of your application, giving you fine-grained control over your logging.

Berikut contoh yang menunjukkan cara mengonfigurasi dan menggunakan logging:

```
Lesson2.py continued

import logging

# Configure basic logging settings
logging.basicConfig(level=logging.INFO) # Sets logging to capture INFO and above
messages
# Create a logger instance
logger = logging.getLogger(__name__)
# Log an informational message
logger.info("This is an informational message.")
```

Dalam contoh ini:

- `basicConfig(level=logging.INFO)` mengonfigurasi sistem logging untuk menangkap pesan INFO, WARNING, ERROR, dan CRITICAL, sementara pesan DEBUG diabaikan.

- `getLogger(__name__)` membuat instance logger yang spesifik untuk modul saat ini, memungkinkan Anda mengorganisasi log berdasarkan modul jika diperlukan.- `logger.info("This is an informational message.")` menghasilkan pesan informasi, yang akan ditampilkan di konsol sesuai konfigurasi.

Logging sangat berguna untuk:

- **Melacak Alur Eksekusi:** Anda dapat menambahkan pernyataan log di titik-titik penting dalam node LangGraph untuk mengikuti bagaimana data mengalir melalui graph dan memverifikasi bahwa proses berjalan sebagaimana diharapkan.- **Mencatat Error:** Catat pesan error di dalam bagian penanganan error pada node untuk menangkap detail tentang masalah yang terjadi, sehingga memudahkan proses debugging.- **Menangkap Peristiwa Sistem:** Catat peristiwa penting, seperti inisialisasi node, penyelesaian tugas signifikan, atau interaksi dengan layanan eksternal, untuk memantau perilaku aplikasi seiring waktu.

Sebagai contoh, dalam node LangGraph, Anda dapat menggunakan logging untuk:

- Mencatat kapan sebuah node mulai dan selesai memproses.- Mencatat input dan output dari node untuk memudahkan debugging dan memahami transformasi data yang terjadi.- Melacak error dan exception yang dimunculkan dalam node, menyediakan stack trace atau konteks tambahan untuk menyederhanakan penyelesaian masalah.

Dengan menempatkan pernyataan log secara strategis, Anda dapat memperoleh wawasan berharga tentang operasi aplikasi LangGraph Anda, sehingga lebih mudah untuk memantau, melakukan debugging, dan memelihara aplikasi.

f) subprocess

Modul subprocess memungkinkan Anda memunculkan (spawn) proses baru, menghubungkan ke pipe input/output/error-nya, dan mendapatkan return code-nya. Ini berguna dalam LangGraph ketika node perlu menjalankan perintah shell atau menjalankan script eksternal.

Komponen Utama:

- **subprocess.run():** Menjalankan sebuah perintah dan menunggu hingga selesai.- **subprocess.Popen():** Kontrol yang lebih lanjut atas proses, memungkinkan eksekusi asinkron.

- **check_output():** Menjalankan sebuah perintah dan mengembalikan outputnya.

```
Lesson2.py continued

import sys
import subprocess

result = subprocess.run(["echo", "Hello, LangGraph!"], capture_output=True, text=True,
executable=sys.executable)
print(result.stdout)
```

Di Mana Digunakan dalam LangGraph:

Gunakan subprocess untuk tugas seperti menjalankan perintah shell, menjalankan script tambahan (auxiliary), atau mengintegrasikan dengan command-line tool secara langsung dari node LangGraph.

g) Modul json

Modul json adalah library bawaan Python untuk bekerja dengan data JSON (JavaScript Object Notation). JSON adalah format data ringan yang mudah dibaca dan ditulis, sehingga ideal untuk pertukaran data antar aplikasi. Dalam LangGraph, json sering digunakan untuk melakukan serialisasi dan deserialisasi data, khususnya saat berinteraksi dengan API atau menyimpan data terstruktur ke disk.

Komponen Utama:

- **json.load():** Membaca data JSON dari sebuah file dan mengubahnya (deserialize) menjadi dictionary Python.- **json.dump():** Melakukan serialisasi object Python ke format JSON dan menuliskannya ke sebuah file.- **json.loads():** Mengubah (deserialize) string berformat JSON menjadi dictionary Python.- **json.dumps():** Melakukan serialisasi object Python menjadi string berformat JSON.

```
Lesson2.py continued

# Reading from a JSON file
with open("data.json", "r") as f:
    loaded_data = json.load(f)
print(loaded_data) # Output: {'name': 'Alice', 'age': 30}
```

Di Mana Menggunakan json dalam LangGraph:

Gunakan modul json untuk tugas yang melibatkan pembacaan dan penulisan data terstruktur. Kasus penggunaan umum meliputi:

- **Menyimpan dan mengambil data state node, yaitu persistence: File JSON dapat mempertahankan (persist) state dari node dalam aplikasi LangGraph.- Berinteraksi dengan API: JSON sering menjadi format data standar untuk komunikasi API, sehingga Anda akan sering melakukan serialisasi dan deserialisasi data saat mengirim atau menerima API request.- Manajemen Konfigurasi: File JSON digunakan untuk mengelola konfigurasi dalam proyek LangGraph karena kesederhanaannya dan dukungan luas di berbagai platform.**

h) Modul yaml

Modul yaml, yang tersedia melalui library PyYAML, adalah alat lain untuk melakukan serialisasi dan deserialisasi data. YAML (YAML Ain't Markup Language) mirip dengan JSON tetapi lebih mudah dibaca manusia berkat sintaksnya dan dukungan untuk komentar. Dalam LangGraph, yaml umum digunakan untuk file konfigurasi, karena keterbacaannya membuatnya ideal untuk pengaturan yang mungkin perlu disesuaikan secara manual. Untuk menggunakan yaml, Anda perlu menginstal library PyYAML:

```
Terminal  
pip install PyYAML
```

Komponen Utama:

- **yaml.safe_load(): Membaca data YAML dari sebuah file dan mengubahnya (deserialize) menjadi dictionary Python. Method safe_load direkomendasikan karena menghindari potensi risiko keamanan yang terkait dengan method load.- yaml.dump(): Melakukan serialisasi object Python ke format YAML dan menuliskannya ke sebuah file.- yaml.safe_load_all(): Membaca dan melakukan deserialisasi beberapa dokumen dari sebuah stream YAML.**

```
Lesson2.py continued  
  
import yaml  
# Writing to a YAML file
```

```

config = {"name": "Alice", "roles": ["admin", "user"]}
try:
    with open("config.yaml", "w") as f:
        yaml.dump(config, f)

    # Reading from a YAML file
    with open("config.yaml", "r") as f:
        loaded_config = yaml.safe_load(f)
    print(loaded_config) # Output: {'name': 'Alice', 'roles': ['admin', 'user']}
except Exception as e:
    print(f"Error: {e}")

```

Di Mana Menggunakan yaml di LangGraph:

Modul yaml sangat cocok untuk manajemen konfigurasi karena keterbacaannya dan dukungannya terhadap struktur data yang kompleks. Kasus penggunaan yang umum meliputi:

- **File Konfigurasi:** File YAML sering digunakan untuk menyimpan konfigurasi dalam proyek LangGraph karena lebih mudah dibaca dibandingkan JSON dan mendukung fitur seperti komentar, yang berguna untuk dokumentasi.
- **Setting untuk Node atau Pipeline yang Kompleks:** Untuk workflow LangGraph yang memerlukan setting yang rumit, YAML dapat memberikan cara yang bersih dan mudah dibaca untuk mengelola dan memahami konfigurasi.
- **Struktur Bersarang:** YAML menangani struktur bersarang dan tipe data kompleks dengan lebih elegan dibandingkan JSON, sehingga lebih disukai ketika konfigurasi bersifat hierarkis.

i) pandas

Untuk tugas manipulasi dan analisis data dalam LangGraph, pandas adalah alat yang sangat berharga. pandas menyediakan struktur data yang kuat seperti DataFrame dan berbagai fungsi untuk pemrosesan data.

Komponen Utama:

- **DataFrame :** Struktur data utama untuk data tabular.
- **read_csv() / to_csv() :** Membaca dari dan menulis ke file CSV.

- **groupby() , merge() , pivot() : Metode untuk mentransformasi dan mengagregasi data.**

```
Lesson2.py continued  
  
import pandas as pd  
  
df = pd.read_csv("data.csv")  
print(df.head())
```

Di Mana Digunakan dalam LangGraph:

Gunakan pandas untuk memproses dan mentransformasi dataset besar, seperti mengagregasi data dari berbagai sumber atau mempersiapkan data untuk input model di node LangGraph.

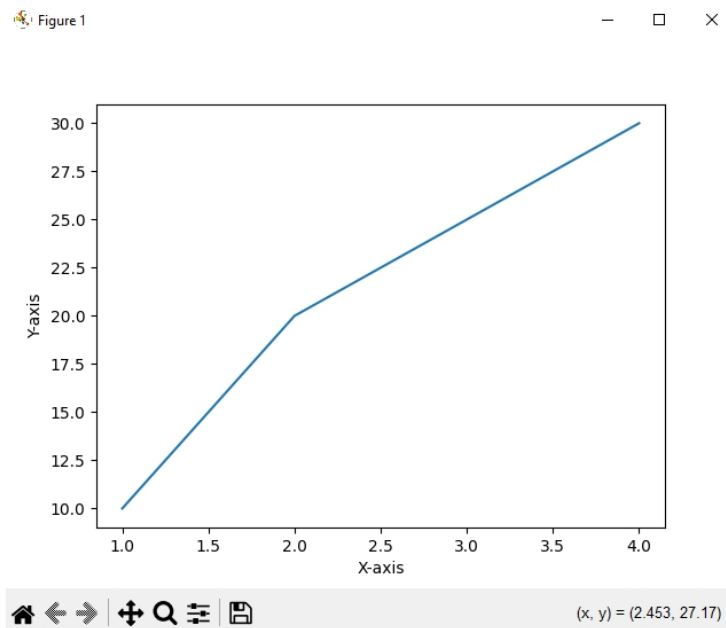
j) **matplotlib dan seaborn**

LangGraph sering melibatkan visualisasi data, baik untuk debugging, analisis, atau menyajikan hasil. matplotlib dan seaborn adalah library populer untuk membuat visualisasi statis, animasi, dan interaktif.

Komponen Utama:

- **matplotlib.pyplot : Module untuk membuat plot dasar seperti line, scatter, dan bar chart.**- **seaborn : Dibangun di atas matplotlib dengan plotting statistik yang lebih canggih.**

```
Lesson2.py continued  
  
import matplotlib.pyplot as plt  
  
plt.plot([1, 2, 3, 4], [10, 20, 25, 30])  
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")  
plt.show()  
  
#Output
```



Gunakan matplotlib dan seaborn untuk membuat visualisasi yang dapat membantu menafsirkan data yang diproses dalam LangGraph, seperti memplot hasil dari sebuah node atau memvisualisasikan perubahan state seiring waktu.

k) sqlalchemy dan sqlite3

Untuk aplikasi yang membutuhkan penyimpanan data persisten, seperti mencatat log state atau menyimpan hasil, LangGraph dapat berinteraksi dengan database. sqlalchemy menyediakan lapisan Object-Relational Mapping (ORM) untuk bekerja dengan database SQL, sedangkan sqlite3 adalah database ringan yang sudah termasuk dalam standard library Python.

Komponen Utama:

- **sqlalchemy.create_engine()** : Menghubungkan ke database.-
- sessionmaker()** : Mengelola session database.- **sqlite3.connect()** :
- Terhubung langsung ke database SQLite.

```
Lesson2.py continued

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

try:
    engine = create_engine("sqlite:///data.db")
    Session = sessionmaker(bind=engine)
```

```

session = Session()
except Exception as e:
    print(f"Error: {e}")

```

Gunakan library ini untuk menyimpan dan mengambil data dari database, mengelola persistensi state aplikasi, atau melacak metrik jangka panjang di berbagai session LangGraph.

l) Modul os: Operasi Sistem File

Modul os menyediakan cara untuk berinteraksi dengan sistem operasi. Modul ini memungkinkan manipulasi file dan direktori, akses environment variable, dan operasi tingkat sistem. Modul ini sering digunakan dalam proyek LangGraph untuk tugas seperti mengelola file dan direktori, membuat folder output, dan menangani path.

Fungsi Utama:

- **os.path.join()** : Menggabungkan beberapa komponen path menjadi satu path.- **os.makedirs()** : Membuat direktori (dan direktori induk yang diperlukan) jika belum ada.- **os.environ** : Mengakses environment variable.- **os.remove()** : Menghapus file.- **os.rename()** : Mengganti nama file atau direktori.

```

Lesson2.py continued

import os
try:
    #Create a folder
    output_dir = "output"
    os.makedirs(output_dir, exist_ok=True)

    # Get an environment variable
    api_key = os.environ.get("API_KEY", "default_value")
except Exception as e:
    print(f"Error: {e}")

```

Penggunaan Umum dalam LangGraph:

- **Mengelola file output: Membuat dan mengatur direktori output untuk log, graph, atau ekspor data.- Mengatur environment variable: Mengonfigurasi API key atau setting khusus environment untuk node LangGraph.- Manipulasi file: Menghapus atau mengganti nama file sebagai bagian dari workflow pemrosesan data.**

m) Modul sys: Parameter dan Fungsi Khusus Sistem

Modul sys menyediakan akses ke parameter dan fungsi khusus sistem yang berinteraksi dengan runtime environment Python. Modul ini sering digunakan untuk menangani argumen command-line, mengontrol interpreter, dan mengelola input/output stream.

Fungsi Utama:

- **sys.argv : Mengambil daftar argumen command-line yang diberikan ke script.- sys.exit() : Mengakhiri program, opsional dengan status exit tertentu.- sys.path : Mengakses daftar direktori yang dicari Python untuk module, berguna untuk menambahkan path secara dinamis.- sys.platform : Mengembalikan string yang mengidentifikasi platform tempat script dijalankan.**

```
Lesson2.py continued

import sys
# Check if enough arguments were passed
if len(sys.argv) < 2:
    print("Usage: python script.py <filename>")
    sys.exit("Please provide a file path as an argument.")
# Accessing command-line arguments
filename = sys.argv[1]
print(f"Processing file: {filename}")
#Exit management
def process_data(data):
    if not data:
        print("Error: No data provided.")
        sys.exit(1) # Exit with an error status

    print("Data processed successfully.")
    sys.exit(0) # Exit with a success status
```



```
# Simulating a scenario with empty data
data = None
process_data(data)
```

Penggunaan Umum dalam LangGraph:

- **Penanganan argumen command-line:** Memungkinkan script LangGraph menerima input pengguna langsung dari command line, yang berguna untuk kustomisasi.- **Manajemen exit:** Mengontrol penghentian script, terutama saat menemukan error atau kondisi yang tidak valid.- **Perilaku khusus platform:** Menyesuaikan perilaku code berdasarkan sistem operasi (misalnya, menangani path file secara berbeda di Windows vs. Linux).

n) Modul random: Pembuatan Angka Acak

Modul random menyediakan fungsi untuk menghasilkan angka acak dan memilih item secara acak. Ini dapat berguna dalam LangGraph untuk membuat test case acak, sampling data, atau mengimplementasikan proses stokastik di dalam graph.

Fungsi Utama:

- **random.randint()** : Menghasilkan integer acak dalam rentang tertentu.- **random.choice()** : Memilih elemen acak dari sequence yang tidak kosong.- **random.shuffle()** : Mengacak urutan list di tempat (in place).- **random.uniform()** : Menghasilkan angka floating-point acak dalam rentang tertentu.

```
Lesson2.py continued

import random
# Generate a random integer between 1 and 100
random_number = random.randint(1, 100)
print(random_number)

# Choose a random item from a list
options = ["apple", "banana", "cherry"]
```

```

fruit = random.choice(options)

print(fruit)
# Shuffle a list
numbers = [1, 2, 3, 4, 5]
shuffle = random.shuffle(numbers)

print(shuffle)
#Uniform float numbers
random_float = random.uniform(0, 1)
print(random_float)

```

Penggunaan Umum dalam LangGraph:

- **Sampling data:** Memilih subset data secara acak untuk keperluan pemrosesan atau pengujian.- **Mengacak sequence:** Mengacak urutan item dalam sebuah list, seperti mengacak test case atau membuat output yang bervariasi.- **Proses stokastik:** Memasukkan keacakan ke dalam pengambilan keputusan atau mensimulasikan peristiwa acak di dalam node.

o) Modul datetime: Bekerja dengan Tanggal dan Waktu

Modul datetime menyediakan class untuk memanipulasi tanggal dan waktu. Modul ini penting untuk tugas yang melibatkan timestamping, penjadwalan, atau pengukuran interval waktu. Dalam LangGraph, datetime berguna untuk logging, mengelola event berbasis waktu, atau memproses data time-series.

Komponen Utama:

- **datetime.datetime.now()** : Mengambil tanggal dan waktu saat ini.- **datetime.timedelta** : Merepresentasikan durasi, yang dapat digunakan untuk aritmetika tanggal.- **datetime.datetime.strptime()** : Mengonversi string menjadi objek datetime berdasarkan format tertentu.- **datetime.datetime.strftime()** : Memformat objek datetime menjadi string.

```

Lesson2.py continued

from datetime import datetime, timedelta

# Get the current date and time

```

```

now = datetime.now()

# Calculate a future date by adding a timedelta
future_date = now + timedelta(days=5)
print(future_date.strftime("%Y-%m-%d"))

```

Penggunaan Umum dalam LangGraph:

- **Timestamping event:** Mencatat waktu ketika node tertentu dijalankan atau ketika peristiwa tertentu terjadi.- **Penjadwalan berbasis waktu:** Memicu atau menunda node berdasarkan interval waktu, seperti menjalankan tugas setiap 15 menit.- **Parsing dan pemformatan tanggal:** Memproses string tanggal menjadi objek datetime untuk kalkulasi, atau mengonversi objek datetime kembali menjadi string untuk pelaporan.

p) Modul collections: Struktur Data Khusus

Modul collections menyediakan struktur data khusus yang memperluas tipe bawaan Python, seperti deque , Counter , defaultdict , dan namedtuple . Struktur ini dioptimalkan untuk tugas tertentu dan dapat membuat code lebih efisien dan mudah dibaca.

Komponen Utama:

- **Counter :** Subclass dictionary untuk menghitung objek yang hashable.- **defaultdict :** Subclass dictionary yang menyediakan nilai default untuk key yang belum ada.- **deque :** Queue dua arah (double-ended) yang dioptimalkan untuk append dan pop yang cepat.- **namedtuple :** Factory function untuk membuat subclass tuple dengan field bernama.

```

Lesson2.py continued

from collections import Counter, defaultdict

# Counting occurrences of items in a list
words = ["apple", "banana", "apple", "cherry"]
word_count = Counter(words)

# Using defaultdict to handle missing keys
fruits = defaultdict(int)
fruits["apple"] += 1

```

```
print(fruits) # Output: defaultdict(<class 'int'>, {'apple': 1})
```

Penggunaan Umum dalam LangGraph:

- **Menghitung dan mentabulasi item:** Gunakan Counter untuk dengan cepat menghitung item, seperti menghitung jenis data yang melewati node.-
Menangani nilai default: Gunakan defaultdict pada skenario di mana key yang hilang sering terjadi, seperti saat mengakumulasi hasil.- **Mengelola data terurut:** Gunakan deque untuk pemrosesan data berbasis queue dalam workflow graph.

q) Modul itertools: Fungsi Iterator

Modul itertools menyediakan sekumpulan tool yang cepat dan efisien memori untuk membuat dan bekerja dengan iterator. Fungsi-fungsi ini dapat membantu mengelola looping dan iterasi dalam workflow LangGraph yang kompleks.

Komponen Utama:

- **itertools.chain()** : Menggabungkan beberapa iterator menjadi satu sequence.-
itertools.cycle() : Mengulang sebuah iterable tanpa henti.- **itertools.groupby()** : Mengelompokkan item dalam iterable berdasarkan fungsi key tertentu.-
itertools.permutations() : Menghasilkan permutasi dari sebuah sequence.

```
Lesson2.py continued  
  
from itertools import cycle  
  
# Cycling through a list indefinitely  
colors = cycle(["red", "green", "blue"])  
for _ in range(6):  
    print(next(colors))
```

Penggunaan Umum dalam LangGraph:

- **Menghasilkan sequence tanpa batas:** Gunakan cycle untuk mengulang sequence, yang berguna untuk penjadwalan round-robin.- **Looping yang efisien:** Gunakan chain untuk mengiterasi beberapa list tanpa membuat struktur data tambahan.

- **Membuat permutasi dan kombinasi:** Gunakan permutations untuk menguji berbagai konfigurasi atau menyelesaikan masalah kombinatorial di dalam node LangGraph.

3.5.12 Map, Filter, dan Reduce

Tool functional programming ini ideal untuk memproses list dan iterable lain dalam LangGraph.

- **map():** Menerapkan fungsi ke semua item dalam sebuah iterable.- **filter():** Menyaring item berdasarkan sebuah fungsi.- **reduce():** Mengakumulasi item menjadi satu hasil.

Fungsi map() menerapkan fungsi tertentu ke setiap item dalam sebuah iterable (seperti list) dan mengembalikan map object (yang dapat dikonversi menjadi list, tuple, dll). Fungsi ini berguna untuk mentransformasi data, seperti mengkuadratkan angka atau mengonversi string menjadi huruf kapital.

```
Lesson2.py continued
# List of numbers to square
numbers = [1, 2, 3, 4]

# Using map to square each number
squares = list(map(lambda x: x**2, numbers))
print(squares) # Output: [1, 4, 9, 16]

# List of strings
words = ["hello", "world", "langgraph"]

# Using map to convert each string to uppercase
uppercase_words = list(map(lambda word: word.upper(), words))
print(uppercase_words) # Output: ['HELLO', 'WORLD', 'LANGGRAPH']

# List of dictionaries with user data
users = [
    {"name": "Alice", "age": 25},
    {"name": "Bob", "age": 30},
    {"name": "Charlie", "age": 35}
]

# Using map to extract only the 'name' field from each dictionary
names = list(map(lambda user: user["name"], users))
print(names) # Output: ['Alice', 'Bob', 'Charlie']
```

Fungsi filter() membuat iterable baru dengan elemen dari iterable asli yang mengembalikan True saat diteruskan ke fungsi yang ditentukan. Fungsi ini umum digunakan untuk menyaring item berdasarkan kondisi tertentu.

```
Lesson2.py continued

# List of numbers to filter
numbers = [1, 2, 3, 4, 5, 6]
# Using filter to keep only even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # Output: [2, 4, 6]

# List of names
names = ["Alice", "Bob", "Charlie", "Dave"]
# Using filter to keep names with more than 3 characters
long_names = list(filter(lambda name: len(name) > 3, names))
print(long_names) # Output: ['Alice', 'Charlie', 'Dave']
# List of dictionaries with user data
users = [
    {"name": "Alice", "age": 25},
    {"name": "Bob", "age": 20},
    {"name": "Charlie", "age": 35}
]
# Using filter to keep only users who are 30 years or older
adult_users = list(filter(lambda user: user["age"] >= 30, users))
print(adult_users) # Output: [{'name': 'Charlie', 'age': 35}]
```

Dalam workflow LangGraph yang lebih kompleks, Anda mungkin menemukan bahwa mengombinasikan map() , filter() , dan reduce() berguna untuk melakukan beberapa operasi secara berurutan. Berikut contoh yang mendemonstrasikan penggunaan ketiga fungsi tersebut bersama-sama.

```
Lesson2.py continued

from functools import reduce
# List of numbers
numbers = [1, 2, 3, 4, 5, 6]

# Filtering even numbers, mapping to squares, and reducing to sum
sum_of_squares_of_evens = reduce(
    lambda x, y: x + y,
    map(lambda x: x**2, filter(lambda x: x % 2 == 0, numbers))
)
print(sum_of_squares_of_evens) # Output: 56 (2^2 + 4^2 + 6^2)
```

Dalam contoh ini:

1. `filter()` selects the even numbers: `[2, 4, 6]` .
2. `map()` squares each even number: `[4, 16, 36]` .
3. `reduce()` sums the squares: `4 + 16 + 36 = 56` .

3.5.13 Dasar-Dasar Asynchronous Programming untuk LangGraph

Asynchronous programming adalah konsep kunci untuk mengoptimalkan tugas yang melibatkan waktu tunggu, seperti network request, file I/O, atau pemrosesan data. Dalam LangGraph, asynchronous programming memungkinkan eksekusi paralel tugas secara efisien, meningkatkan performa dengan menghindari operasi yang bersifat blocking. Python menyediakan keyword `async` dan `await` untuk mengaktifkan asynchronous programming, memungkinkan code berjalan tanpa menunggu tugas yang berjalan lama untuk selesai.

3.5.13.1 Dasar-Dasar Async/Await

Keyword `async` dan `await` digunakan untuk mendefinisikan fungsi asynchronous dan mengelola operasi asynchronous. Fungsi asynchronous yang didefinisikan dengan `async def` dapat dijeda dan dilanjutkan, memungkinkan code lain berjalan selama waktu tunggu.

Contoh 1: Fungsi Asynchronous Dasar

Dalam contoh ini, kita membuat fungsi asynchronous yang mensimulasikan pengambilan data dengan menunggu satu detik sebelum mengembalikan sebuah pesan.

```
Lesson2.py continued

import asyncio

async def fetch_data():
    await asyncio.sleep(1) # Simulates a non-blocking wait
    return "Data fetched!"
# Run the asynchronous function
result = asyncio.run(fetch_data())
print(result) # Output: Data fetched!
```

Berikut:

- `async def fetch_data()` mendefinisikan fungsi asynchronous.-
`await asyncio.sleep(1)` menunda eksekusi selama satu detik tanpa memblokir code lain.-
`asyncio.run()` menjalankan fungsi asynchronous.

Contoh 2: Menjalankan Beberapa Tugas Asynchronous Sekuensial: Anda dapat merangkai fungsi asynchronous untuk dieksekusi satu demi satu.

```
Lesson2.py continued

async def task1():
    await asyncio.sleep(1)
    print("Task 1 completed")
async def task2():
    await asyncio.sleep(2)
    print("Task 2 completed")
async def main():
    await task1()
    await task2()
asyncio.run(main())
```

Dalam contoh ini, `task1` dan `task2` dijalankan secara sekuensial, sehingga total runtime adalah 3 detik. Menggunakan `await` memastikan setiap fungsi selesai sebelum fungsi berikutnya dimulai.

3.5.14 Concurrent Programming dengan asyncio

Concurrency memungkinkan Anda menjalankan beberapa tugas pada waktu yang sama. Di `asyncio`, fungsi `gather()` memungkinkan Anda menjalankan beberapa fungsi asynchronous secara concurrent. Ini berguna dalam `LangGraph` untuk melakukan pemrosesan paralel, seperti mengambil data dari beberapa sumber secara bersamaan.

```
Lesson2.py continued

async def task(name):
    await asyncio.sleep(1)
    print(f"Task {name} completed")

async def main():
    await asyncio.gather(task("A"), task("B"), task("C"))
```



```
asyncio.run(main())  
#Output:  
# Task A completed  
# Task B completed  
# Task C completed
```

Semua tugas dimulai pada waktu yang sama, sehingga selesai bersamaan setelah satu detik. Ini lebih efisien dibandingkan menjalankannya secara sekuensial, yang akan memerlukan waktu 3 detik.

Contoh 3: Melakukan Operasi Asynchronous I/O

Bayangkan Anda ingin mensimulasikan pengambilan data dari tiga sumber berbeda secara paralel:

```
Lesson2.py continued  
  
async def fetch_source(source):  
    print(f"Fetching from {source}...")  
    await asyncio.sleep(2)  
    print(f"Completed fetching from {source}")  
  
async def main():  
    sources = ["Source 1", "Source 2", "Source 3"]  
    await asyncio.gather(*(fetch_source(source) for source in sources))  
  
asyncio.run(main())  
  
#Output  
# Fetching from Source 1...  
# Completed fetching from Source 1  
# Fetching from Source 2...  
# Completed fetching from Source 2  
# Fetching from Source 3...  
# Completed fetching from Source 3
```

Di sini, ketiga tugas `fetch_source` berjalan secara paralel, selesai dalam 2 detik, dibandingkan dengan 6 detik jika dilakukan secara sekuensial.

3.5.15 Error Handling dalam Asynchronous Code

Error handling dalam fungsi asynchronous dikelola dengan blok `try / except`, sama seperti pada fungsi synchronous. Namun, penting untuk menempatkan blok ini di dalam fungsi `async` itu sendiri untuk error handling yang non-blocking.

Contoh 4: Menangani Error dalam Tugas Asynchronous

```
Lesson2.py continued

async def faulty_task():
    try:
        raise ValueError("An error occurred")
    except ValueError as e:
        print(f"Error: {e}")

asyncio.run(faulty_task())
```

Dalam contoh ini, error apa pun yang muncul di dalam `faulty_task` ditangkap oleh blok `except`, sehingga mencegah program crash.

Contoh 5: Error Handling dalam Tugas Concurrent dengan `gather()`

Saat menggunakan `gather()`, Anda dapat memilih untuk menangani error secara individual atau membiarkan fungsi tersebut meneruskannya (`propagate`). Secara default, jika ada tugas yang memunculkan error, `gather()` berhenti dan memunculkan error pertama yang ditemukan. Untuk menangani error secara individual, gunakan `return_exceptions=True`.

```
Lesson2.py continued

async def task1():
    raise ValueError("Error in Task 1")

async def task2():
    await asyncio.sleep(1)
    return "Task 2 completed successfully"

async def main():
    results = await asyncio.gather(task1(), task2(), return_exceptions=True)
    for result in results:
        if isinstance(result, Exception):
            print(f"Handled exception: {result}")
        else:
            print(result)

asyncio.run(main())
#Output
# Error: ValueError("Error in Task 1")
# Task 2 completed successfully
```

Asynchronous programming dalam LangGraph memungkinkan operasi yang efisien dan non-blocking, membuat workflow lebih cepat dan responsif, terutama pada tugas yang melibatkan operasi I/O, network call, atau operasi apa pun dengan latency tinggi. Dengan menggunakan asyncio, Anda dapat menangani beberapa tugas secara concurrent, meningkatkan skalabilitas dan performa proyek LangGraph Anda.

3.5.15 Contoh Asynchronous yang Relevan dengan LangGraph

Dalam LangGraph, operasi asynchronous sangat berguna ketika berurusan dengan node yang membutuhkan data dari API eksternal. Dengan menjalankan beberapa tugas pengambilan data secara concurrent, Anda dapat mengurangi total waktu eksekusi dan meningkatkan efisiensi.

Contoh 6: Mengambil Data untuk Beberapa Node secara Concurrent di LangGraph

Anggaplah Anda memiliki workflow LangGraph di mana tiga node berbeda mengambil data dari tiga API berbeda. Dengan mengambilnya secara asynchronous, Anda menghindari menunggu setiap node selesai sebelum melanjutkan ke node berikutnya.

Lesson2.py continued

```
import asyncio

async def fetch_data_for_node(node_id, api_url):
    print(f"Node {node_id}: Fetching data from {api_url}...")
    await asyncio.sleep(2) # Simulates a network delay
    print(f"Node {node_id}: Completed data fetch from {api_url}")
    return f"Data from {api_url}"

async def main():
    # URLs for data fetching in different nodes
    nodes = {
        "Node 1": "https://api.example.com/data1",
        "Node 2": "https://api.example.com/data2",
        "Node 3": "https://api.example.com/data3"
    }

    # Concurrently fetch data for all nodes
    results = await asyncio.gather(
        *(fetch_data_for_node(node_id, url) for node_id, url in nodes.items())
    )

    print("All data fetched:", results)

asyncio.run(main())
#Node Node 1: Fetching data from https://api.example.com/data1...
```

```
#Node Node 2: Fetching data from https://api.example.com/data2...
#Node Node 3: Fetching data from https://api.example.com/data3...
#Node Node 1: Completed data fetch from https://api.example.com/data1
#Node Node 2: Completed data fetch from https://api.example.com/data2
#Node Node 3: Completed data fetch from https://api.example.com/data3
#All data fetched: ['Data from https://api.example.com/data1', 'Data from https://api.example.com/data2', 'Data from https://api.example.com/data3']
```

Dalam contoh ini:

- `fetch_data_for_node` mensimulasikan pengambilan data dari API untuk node tertentu.- `asyncio.gather` menjalankan semua tugas `fetch_data_for_node` secara concurrent, mengambil data untuk semua node secara paralel.- Setup ini ideal untuk workflow LangGraph di mana node bergantung pada data eksternal dan pengambilan data secara paralel diperlukan untuk meminimalkan waktu tunggu.

3.5.16 Manipulasi String

String adalah tipe data fundamental dalam Python dan digunakan secara luas di hampir setiap program. Memahami cara memanipulasi string sangat penting saat membangun agent yang berinteraksi dengan input pengguna, menghasilkan teks, atau memproses data dari API.

3.5.16.1 Operasi String Dasar

Mari kita mulai dengan beberapa operasi string umum seperti concatenation, slicing, dan pemformatan dasar.

Contoh 1: Concatenation dan Repetition

Concatenation dan repetition adalah operasi string fundamental yang digunakan untuk menggabungkan atau mengulang string.

```
Lesson2.py continued

# Concatenation
greeting = "Hello" + " " + "World"
print(greeting) # Outputs: Hello World

# Repetition
echo = "Echo " * 3
print(echo) # Outputs: Echo Echo Echo
```

Contoh 2: String Slicing

Anda dapat mengekstrak bagian dari sebuah string menggunakan slicing.

```
Lesson2.py continued  
  
text = "LangGraph"  
# Extracting a substring  
print(text[0:4]) # Outputs: Lang  
print(text[-5:]) # Outputs: Graph
```

Contoh 3: String Formatting

String formatting penting untuk membuat string yang dinamis. Dalam Python modern, sintaks f-string sangat disarankan.

```
Lesson2.py continued  
  
name = "Alice"  
age = 30  
# Using f-strings for formatting  
print(f"My name is {name}, and I am {age} years old.") # Outputs: My name is Alice,  
and I am 30 years old.
```

Use Case dalam LangGraph:

String concatenation, slicing, dan formatting berguna untuk menghasilkan prompt atau output teks secara dinamis dalam agent LangGraph. Misalnya, saat chatbot merespons query pengguna, chatbot mungkin perlu memformat dan menggabungkan berbagai string berdasarkan input.

3.5.16.2 Operasi String Tingkat Lanjut

Mari kita masuk ke operasi string yang lebih tingkat lanjut seperti searching, splitting, joining, dan memanipulasi case.

Contoh 4: Mencari di dalam String

Anda dapat mencari substring di dalam sebuah string menggunakan metode seperti `find()` dan `in`.

```
Lesson2.py continued  
  
text = "LangGraph is a powerful framework."  
# Searching for a substring  
print(text.find("powerful")) # Outputs: 13
```

```
print("Graph" in text)    # Outputs: True
```

Contoh 5: Splitting dan Joining String

Splitting memecah sebuah string menjadi list of substring, sedangkan joining menggabungkan elemen-elemen dari sebuah list menjadi satu string.

```
Lesson2.py continued
# Splitting a string into a list
words = text.split(" ")
print(words) # Outputs: ['LangGraph', 'is', 'a', 'powerful', 'framework.']

# Joining a list into a string
sentence = " ".join(words)
print(sentence) # Outputs: LangGraph is a powerful framework.
```

Contoh 6: Mengubah Case String

Python menyediakan metode untuk memanipulasi case dari string.

```
Lesson2.py continued
text = "LangGraph"
print(text.upper()) # Outputs: LANGGRAPH
print(text.lower()) # Outputs: langgraph
print(text.title()) # Outputs: Langgraph
```

Use Case dalam LangGraph:

Operasi string ini khususnya berguna untuk tugas pemrosesan teks dalam LangGraph. Misalnya, saat mem-parsing query pengguna, memecah teks menjadi kata, menormalkan case, dan menggabungkan token adalah langkah-langkah yang umum dilakukan.

3.6 Advanced Exception Handling

Dalam sistem yang kompleks, error handling sangat penting untuk keandalan (robustness). Mekanisme exception handling Python memungkinkan Anda menangkap dan menangani error dengan baik.

3.6.1 Memunculkan Custom Exception

Anda dapat memunculkan exception Anda sendiri menggunakan keyword `raise`. Ini berguna ketika Anda ingin menerapkan aturan tertentu dalam program Anda.

Contoh 7: Memunculkan Custom Exception

```
Lesson2.py continued

def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    elif age < 18:
        print("Not eligible to vote")
    else:
        print("Eligible to vote")

try:
    check_age(-5)
except ValueError as e:
    print(e) # Outputs: Age cannot be negative
```

Custom exception berguna dalam LangGraph ketika Anda perlu menangani kasus tertentu seperti input yang tidak valid atau API call yang gagal.

3.6.2 try-except-else-finally

Struktur try-except lengkap dapat menyertakan blok else dan finally untuk menambahkan kontrol lebih lanjut.

- **else** : Dieksekusi jika tidak ada exception yang terjadi.- **finally** : Dieksekusi apa pun yang terjadi, sering digunakan untuk cleanup.

Contoh 8: Exception Handling Lengkap

```
Lesson2.py continued

try:
    number = int(input("Enter a number: "))
    result = 10 / number
except ValueError:
    print("You must enter a valid integer.")
except ZeroDivisionError:
    print("You can't divide by zero.")
else:
    print(f"Result: {result}")
finally:
    print("Execution completed.")
```

Struktur ini memastikan bahwa setiap error ditangani, dan blok finally memastikan bahwa tugas cleanup penting (seperti menutup file atau koneksi) dilakukan.

3.6.3 Chained Exception

Python memungkinkan Anda memunculkan exception baru sambil tetap mempertahankan exception aslinya. Ini berguna untuk debugging.

Contoh 9: Chained Exception

```
Lesson2.py continued

try:
    number = int(input("Enter a number: "))
    result = 10 / number
except ZeroDivisionError as e:
    raise RuntimeError("Failed to divide") from e
```

Dalam LangGraph, chaining exception membantu ketika satu node gagal karena error pada node sebelumnya, dengan tetap mempertahankan konteks error untuk debugging.

3.6.4 Exception Handling dalam Workflow LangGraph

Dalam sebuah agent LangGraph, Anda mungkin perlu menangani exception yang muncul selama workflow. Ini dapat mencakup menangkap masalah network, input tidak valid, atau error pada API eksternal.

Contoh 10: Exception Handling di LangGraph

```
Lesson2.py continued

from langgraph import Graph, Node

class APIRequestNode(Node):
    def run(self):
        try:
            data = self.make_request()
            self.send_output(data)
        except TimeoutError:
            self.send_output("The request timed out.")
        except ValueError as e:
            self.send_output(f"Invalid data: {e}")
```



```
        finally:  
            self.log("Request completed.")  
  
graph = Graph()  
node = APIRequestNode()  
graph.add_node(node)
```

Dalam contoh ini, sebuah node LangGraph membuat request API dan menangani potensi timeout atau data tidak valid dengan baik, mencatat hasilnya di blok finally .

Bagian 2: Konsep Menengah LangGraph dan Agent

Design (Chapters 4-7)

Chapter 4

Core Elements of LangGraph

Dalam bab ini, kita akan menyelami blok bangunan fundamental dari LangGraph. Memahami elemen-elemen inti ini sangat penting karena mereka membentuk dasar dari setiap AI agent atau workflow yang Anda buat. Jangan khawatir jika Anda baru mengenal konsep-konsep ini - kita akan membahas semuanya langkah demi langkah dan menyediakan contoh coding praktis untuk memandu Anda.

Konsep Utama: Node, Edge, State, dan Graph

LangGraph, seperti namanya, berputar di sekitar graph, yaitu jaringan dari task yang saling terhubung yang dapat digunakan untuk membangun workflow bagi AI agent. Workflow ini bergantung pada empat komponen utama:

1. **State**
2. **Nodes**
3. **Edges**
4. **Graphs**

Mari kita jelajahi komponen-komponen ini secara detail.

4.1. State - Melacak Data

State adalah struktur data bersama yang menyimpan informasi saat mengalir melalui node-node dalam sebuah graph. State memungkinkan bagian-bagian berbeda dari workflow untuk

"mengingat" dan membagikan informasi di berbagai node. Anggap saja sebagai memori dari workflow Anda. Dalam LangGraph, state biasanya berupa dictionary Python, objek TypedDict, atau model Pydantic. Struktur ini menyimpan data yang dapat diperbarui atau dimodifikasi oleh berbagai node dalam workflow Anda.

Berikut contoh sederhana dari sebuah state:

```
Lesson3a.py

from typing_extensions import TypedDict

# Define a simple state structure
class HelloWorldState(TypedDict):
    message: str # This will store a message
```

Dalam contoh ini, HelloWorldState mendefinisikan sebuah state yang dapat menyimpan pesan string. Seiring workflow berjalan, berbagai node dapat membaca, memperbarui, atau memanipulasi pesan ini.

Contoh Praktis: Bayangkan sebuah workflow di mana satu node dimulai dengan menyapa pengguna, dan node lain menambahkan informasi lebih lanjut ke sapaan tersebut. State memungkinkan data ini bertahan dan diteruskan dari satu langkah ke langkah berikutnya.

```
Lesson3a.py continued

# Initializing the state
state = {"message": "Hi! "}
```

4.2. Node - Logika Bisnis

Sebuah node dalam LangGraph bertanggung jawab untuk menjalankan tugas tertentu. Node adalah blok bangunan yang merepresentasikan setiap aksi atau titik keputusan dalam sebuah workflow. Misalnya, satu node mungkin mengambil input pengguna, sementara node lain memproses input tersebut.

Node dalam LangGraph biasanya diimplementasikan sebagai function yang menerima sebuah state dan mengembalikan versi state yang telah dimodifikasi. Berikut contoh sebuah node yang menambahkan teks ke pesan yang tersimpan dalam state:

```
Lesson3a.py continued

def hello_world_node(state: HelloWorldState):
```

```
state["message"] = "Hello World"
return state
```

Di sini, `hello_world_node` adalah sebuah node yang menerima sebuah state (dictionary), menambahkan string teks ke pesan dalam state, dan mengembalikan state yang telah diperbarui. Dengan memecah workflow menjadi langkah-langkah yang lebih kecil seperti ini, Anda dapat membangun fungsionalitas yang lebih kompleks seiring waktu.

Output yang dikembalikan terlihat seperti ini

```
{"message": "Hello World"}
```

Ini berarti Anda dapat menulis ulang node di atas seperti berikut:

```
Lesson3a.py continued
def hello_world_node(state: HelloWorldState):
    return {"message": "Hello World"}
```

Node ditambahkan ke graph menggunakan function `add_node`

```
Lesson3a.py continued
graph_builder.add_node("greet_user", greet_user)
```

4.3. Edge - menghubungkan node

Edge mendefinisikan koneksi antar node. Edge mengontrol bagaimana data mengalir dari satu node ke node berikutnya dan menentukan urutan tugas dijalankan. Secara sederhana, edge adalah tanda panah yang menunjukkan jalur dari satu node ke node lain dalam sebuah workflow.

Misalnya, jika Anda ingin workflow Anda dimulai dengan node sapaan dan kemudian berpindah ke node lain yang memproses input pengguna, edge akan mendefinisikan transisi ini.

Contoh sebuah edge yang menghubungkan dua node, menghubungkan node `raise_invoice` ke `receive_payment`.

```
Lesson3a.py continued
graph_builder = StateGraph(UserState)
graph_builder.add_edge("raise_invoice", "raise_invoice")
```

```
graph_builder.add_edge("receive_payment", END)
```

Mari kita fokus pada tiga jenis node dan edge penting dalam LangGraph:

1. **Start Node**
2. **End Node**
3. **Conditional Edge**
4. **Conditional Entry point**

4.3.1 Start Node: Awal dari Workflow

Start Node menandai titik masuk (entry point) dari graph. Ini adalah node pertama yang mengaktifkan eksekusi workflow. Anggap saja sebagai trigger yang menggerakkan proses, menghubungkan node awal tempat workflow dimulai.

Dalam LangGraph, awal dari sebuah workflow sering dinyatakan dengan konstanta `START`. Start Node mendefinisikan koneksi dari titik awal ini ke node pertama dalam workflow Anda. Berikut contohnya:

```
Lesson3a.py continued  
  
# Connect the start point to the first node  
graph_builder.add_edge(START, "greet_user") # Start edge
```

Dalam contoh ini, edge menghubungkan titik awal (`START`) ke node `"greet_user"`. Ketika graph dijalankan, edge ini memicu proses sapaan, berpindah dari `START` ke task pertama.

Sebagai alternatif, entry point juga dapat didefinisikan menggunakan fungsi `set_entry_point`

```
Lesson3a.py continued  
  
graph_builder.set_entry_point("greet_user")
```

4.3.2 End Node: Menyelesaikan Workflow

End Node merepresentasikan akhir dari workflow. Ini menandakan bahwa workflow telah mencapai task terakhirnya dan tidak ada aksi lebih lanjut yang akan dilakukan. Seperti halnya start edge, End Node direpresentasikan oleh konstanta `END` dalam LangGraph.

Anda dapat mendefinisikan end node dengan menghubungkan node terakhir dari workflow Anda ke penanda END. Contohnya:

```
Lesson3a.py continued  
  
# Connect the last node to the end point  
graph_builder.add_edge("say_goodbye", END)
```

Dalam kasus ini, setelah node "say_goodbye" menyelesaikan tugasnya, workflow mencapai END, menandakan selesainya proses tersebut.

Sebagai alternatif, titik akhir dari sebuah graph dapat dinyatakan menggunakan set_finish_point

```
Lesson3a.py continued  
  
graph_builder.set_finish_point("say_goodbye")
```

4.3.3 Conditional Edges: Menambahkan Titik Keputusan

Conditional edges adalah salah satu fitur paling powerful dalam LangGraph. Fitur ini memungkinkan workflow untuk bercabang berdasarkan kondisi tertentu, sehingga memungkinkan perilaku yang lebih dinamis. Dengan Conditional Edges, Anda dapat mendefinisikan jalur berbeda yang mungkin diambil oleh workflow tergantung pada state dari data atau hasil dari sebuah node.

Misalnya, anggaplah Anda memiliki sebuah node yang memeriksa apakah seorang pengguna adalah pelanggan premium. Tergantung apakah pengguna tersebut premium atau tidak, Anda ingin workflow mengambil jalur yang berbeda. Berikut cara mengimplementasikan conditional edges:

```
Lesson3a.py continued  
  
# Define two nodes that handle different cases  
  
def check_subscription(state: UserState):  
    if state["is_premium"]:  
        return "premium_greeting"  
    else:  
        return "regular_greeting"  
    ...  
  
graph_builder.add_conditional_edges("greet_user", check_subscription)
```

...

Dalam kasus ini, graph akan memutuskan jalur mana yang harus diikuti berdasarkan kondisi yang didefinisikan dalam edge. Node `check_subscription` akan mengevaluasi status langganan pengguna, dan graph akan mengikuti edge yang sesuai.

4.3.4 Prinsip-Prinsip Edge dalam LangGraph

Edge bukan sekadar koneksi antar task - edge mendefinisikan logika dan alur dari workflow Anda. Berikut adalah prinsip-prinsip inti yang perlu dipahami saat bekerja dengan edge dalam LangGraph:

- **Sequential Flow:** Edge memastikan bahwa task dieksekusi dalam urutan yang benar. Tanpa edge, tidak akan ada kontrol atas bagaimana task dipicu dan dieksekusi.
- **Conditional Flow:** Conditional edges memungkinkan logika percabangan, di mana workflow dapat mengikuti jalur berbeda berdasarkan evaluasi kondisi atau data tertentu. Ini membuat workflow menjadi adaptif dan cerdas.
- **Completion Control:** End Edge mendefinisikan kapan workflow berakhir. Tanpa end edge, graph akan terus berjalan tanpa henti atau gagal menandakan penyelesaiannya dengan benar.
- **Error Handling:** Edge dapat dirancang untuk menangani error atau exception, mengarahkan workflow ke pemulihan atau jalur alternatif ketika terjadi hasil yang tidak terduga.

4.3.5 Contoh Praktis: Menggunakan Start, End, dan Conditional Edges

Berikut adalah workflow sederhana yang menggunakan start, end, dan conditional edges untuk menyapa pengguna secara berbeda tergantung apakah mereka pelanggan premium:

```
Lesson3a.py complete code example

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# Define the state for user data
class UserState(TypedDict):
    is_premium: bool
```

```

        message: str

# Define nodes
def greet_user(state: UserState):
    state["message"] = "Welcome!"
    return state

def premium_greeting(state: UserState):
    state["message"] += " Thank you for being a premium user!"
    return state

def regular_greeting(state: UserState):
    state["message"] += " Enjoy your time here!"
    return state

# Define a decision node to choose the path based on user type
def check_subscription(state: UserState):
    if state["is_premium"]:
        return "premium_greeting"
    else:
        return "regular_greeting"

# Build the graph
graph_builder = StateGraph(UserState)
graph_builder.add_node("greet_user", greet_user)
graph_builder.add_node("check_subscription", check_subscription)
graph_builder.add_node("premium_greeting", premium_greeting)
graph_builder.add_node("regular_greeting", regular_greeting)
# Add edges to control the flow
graph_builder.add_edge(START, "greet_user") # Start edge
graph_builder.add_conditional_edges("greet_user", check_subscription)
graph_builder.add_edge("premium_greeting", END) # End edge for premium users
graph_builder.add_edge("regular_greeting", END) # End edge for regular users

# Compile and run the graph for a premium user
graph = graph_builder.compile()
result = graph.invoke({"is_premium": True, "message": ""})
print(result) # Output: {'message': 'Welcome! Thank you for being a premium user!'}

# Compile and run the graph for a regular user
result = graph.invoke({"is_premium": False, "message": ""})
print(result) # Output: {'message': 'Welcome! Enjoy your time here!'}

```


Dalam contoh ini, kita menggunakan Start Edge untuk memulai workflow, Conditional Edge untuk memilih di antara dua jalur, dan End Edge untuk mengakhiri workflow. Hasilnya berubah berdasarkan status langganan pengguna, menunjukkan bagaimana workflow yang dinamis dapat dibuat dengan edge.

4.4. Graph

Sebuah Graph dalam LangGraph adalah struktur menyeluruh yang menyatukan node, edge, dan state menjadi sebuah workflow yang koheren. Graph berfungsi sebagai blueprint untuk aktivitas agent, mendefinisikan bagaimana task diurutkan, jalur mana yang dapat diambil oleh agent, dan bagaimana informasi mengalir melalui keseluruhan proses.

Graph dapat berupa sederhana atau kompleks, tergantung pada workflow yang diinginkan. Graph bisa saja hanya terdiri dari beberapa node yang terhubung secara linear, atau bisa juga mencakup banyak cabang, jalur kondisional, dan loop yang memungkinkan perilaku yang lebih canggih. Sebuah graph menyediakan kerangka kerja tempat node beroperasi, dengan edge yang mengarahkan alur state melalui task-task yang terhubung.

4.4.1 Jenis-Jenis Graph yang Umum dalam LangGraph:

a. StateGraph

Anda mulai membangun sebuah graph dengan graph builder yang menentukan tipe graph. Class StateGraph adalah class graph utama yang digunakan. Class ini diparameterisasi oleh objek State yang didefinisikan oleh pengguna.

```
Lesson3b.py
graph_builder = StateGraph(HelloWorldState)
```

b. MessageGraph

Class MessageGraph adalah tipe graph khusus yang dibuat khusus untuk menangani daftar messages, secara khusus cocok untuk kasus penggunaan chat. State dari sebuah MessageGraph HANYA berupa daftar messages. Class ini jarang digunakan kecuali untuk chatbot, karena sebagian besar aplikasi membutuhkan State yang lebih kompleks daripada sekadar daftar messages.

```
Lesson3b.py continued
```

```
graph_builder = StateGraph(MessageGraph)
```

4.4.2 Meng-compile Graph

Setelah Anda mendefinisikan node dan edge pada graph builder, Anda meng-compile-nya menjadi sebuah graph.

```
Lesson3b.py continued  
  
graph = graph_builder.compile()
```

Meng-compile sebuah graph melakukan hal berikut:

- Memeriksa dan memvalidasi struktur graph untuk memastikan semua node dimanfaatkan sepenuhnya dalam edge dan semua edge mendefinisikan workflow yang lengkap dengan benar.
- Menentukan argumen memory seperti checkpointer-
- Mendefinisikan breakpoint untuk aksi intervensi human in the loop.

Dalam LangGraph, sebuah graph sederhana yang lengkap mungkin terlihat seperti ini:

```
Lesson3b.py continued  
  
from typing_extensions import TypedDict  
from langgraph.graph import StateGraph, START, END  
  
class HelloWorldState(TypedDict):  
    """The state"""  
    message: str # This state key will store the message  
  
def hello_world_node(state: HelloWorldState):  
    """The node function"""  
    state["message"] += "Hello World" #This node does this simple task.  
    return state  
  
#Define the graph  
graph_builder = StateGraph(HelloWorldState)  
graph_builder.add_node("hello_world", hello_world_node)  
graph_builder.add_edge(START, "hello_world")  
graph_builder.add_edge("hello_world", END)  
  
# Compile and run the graph
```

```
graph = graph_builder.compile()
result = graph.invoke({"message": "Hi! "})
# Output the result
print(result)
# Output: {'greeting': 'Hi! Hello World'}
```

4.4.3 State Channels

Node biasanya berkomunikasi menggunakan satu schema. Misalnya, Anda mungkin memiliki node-node di bawah ini yang keduanya berbagi HelloWorldState yang sama. state["message"] menjadi sebuah state channel yang mengaktifkan salah satu node begitu data diterima dari edge.

Lesson3b.py continued

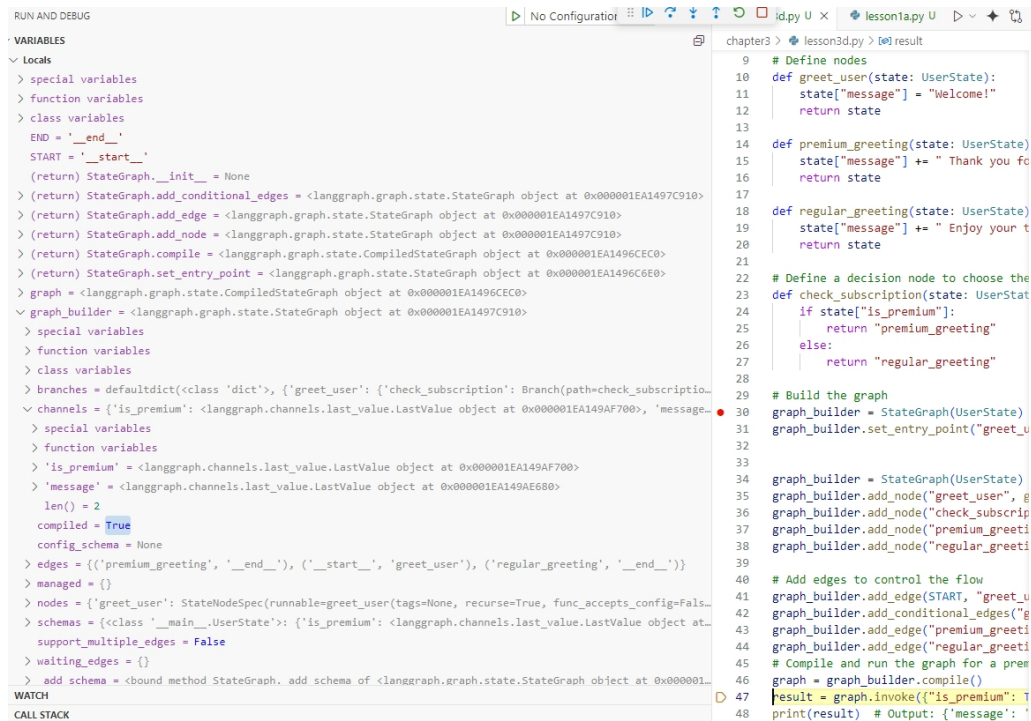
```
def hello_world_node(state: HelloWorldState):
    state["message"] += "Hello World"

def welcome_user_node(state: HelloWorldState):
    state["message"] += "Welcome to this app"
```

Ketika Anda memiliki variabel privat di dalam sebuah node yang tidak diperlukan dalam output graph, variabel-variabel ini menjadi private channel yang dapat merujuk pada sebuah private schema.

4.4.4 Debug Symbols dari sebuah LangGraph

Ketika Anda men-debug sebuah workflow LangGraph, Anda dapat melihat nilai aktual dari variabel-variabel dalam workflow Anda, termasuk start dan end node, node-node, edge-edge, channel yang digunakan dalam graph, state schema, dan masih banyak lagi.



The screenshot displays a code editor with two panels. The left panel shows the 'RUN AND DEBUG' interface with a 'VARIABLES' section expanded, listing various objects and their memory addresses. The right panel shows the source code of 'lesson3d.py', which defines a LangGraph workflow. The code includes functions for 'greet_user', 'premium_greeting', and 'regular_greeting', and a 'check_subscription' function. It also shows the graph building process and the final output.

```
chapter3 > lesson3d.py [x] result
9 # Define nodes
10 def greet_user(state: UserState):
11     state["message"] = "Welcome!"
12     return state
13
14 def premium_greeting(state: UserState):
15     state["message"] += " Thank you for premium!"
16     return state
17
18 def regular_greeting(state: UserState):
19     state["message"] += " Enjoy your time!"
20     return state
21
22 # Define a decision node to choose the greeting
23 def check_subscription(state: UserState):
24     if state["is_premium"]:
25         return "premium_greeting"
26     else:
27         return "regular_greeting"
28
29 # Build the graph
30 graph_builder = StateGraph(UserState)
31 graph_builder.set_entry_point("greet_user")
32
33 graph_builder = StateGraph(UserState)
34 graph_builder.add_node("greet_user", greet_user)
35 graph_builder.add_node("check_subscription", check_subscription)
36 graph_builder.add_node("premium_greeting", premium_greeting)
37 graph_builder.add_node("regular_greeting", regular_greeting)
38
39 # Add edges to control the flow
40 graph_builder.add_edge(START, "greet_user")
41 graph_builder.add_conditional_edges("greet_user", check_subscription, {"premium_greeting": "premium_greeting", "regular_greeting": "regular_greeting"})
42 graph_builder.add_edge("premium_greeting", "regular_greeting")
43 graph_builder.add_edge("regular_greeting", "regular_greeting")
44
45 # Compile and run the graph for a premium user
46 graph = graph_builder.compile()
47 result = graph.invoke({"is_premium": True})
48 print(result) # Output: {'message': 'Welcome! Thank you for premium! Enjoy your time! Enjoy your time!'}
```

4.5. Input dan Output Schema dari sebuah Graph: Flexible State Management dalam LangGraph

Dalam LangGraph, state memainkan peran penting dalam bagaimana node berkomunikasi dan berbagi data sepanjang workflow. LangGraph menawarkan fleksibilitas dalam bagaimana state dikelola, memungkinkan schema input dan output yang berbeda-beda. Fleksibilitas ini memungkinkan developer untuk menggunakan subset data yang berbeda untuk bagian-bagian graph yang berbeda, meningkatkan modularitas dan kontrol alur data.

Pada bagian ini, kita akan mengeksplorasi bagaimana Anda dapat merancang workflow yang menggunakan schema berbeda untuk input, output, dan komunikasi internal dengan memperkenalkan internal state channel dan multiple state schema. Ini memungkinkan perilaku yang lebih kompleks di mana node berkomunikasi menggunakan state privat (internal) maupun eksternal (output).

4.5.1 Peran Input dan Output Schema

Dalam sebuah workflow LangGraph, input dan output schema mendefinisikan struktur data yang masuk dan keluar dari node. Dengan menentukan input dan output schema yang berbeda, Anda dapat mengontrol data apa yang dilewatkan antar node dan bagaimana hasil akhir dikembalikan.

- **Input Schema:** Menentukan data apa yang diterima sebuah node atau graph sebagai input.
- **Output Schema:** Mendefinisikan struktur data yang dikembalikan oleh node atau graph.

Pemisahan ini berguna ketika Anda perlu:

1. **Hide Internal Details:** For example, if a node needs to process intermediate data that shouldn't be exposed to the rest of the graph.
2. **Define Flexible Workflows:** Different parts of the graph can work with different types of data without interfering with each other.

4.5.2 Mendefinisikan Multiple State Schema

Mari kita lihat contoh di mana kita mendefinisikan schema `OverallState`, yang berisi semua field yang relevan dengan operasi graph. Dari overall

state ini, kita dapat mengekstrak subset untuk membentuk input dan output schema untuk node-node.

```
Lesson3b.py continued  
  
from typing_extensions import TypedDict  
  
# Define the input and output types for the overall state state  
class OverallState(TypedDict):  
    partial_message: str  
    user_input: str  
    message_output: str
```

Dalam kasus ini:

- OverallState berisi semua data yang relevan dengan graph.-
InputState adalah versi sederhana dari OverallState yang hanya mencakup input pengguna.-
OutputState adalah subset lain yang hanya berisi output pesan akhir.-
PrivateState adalah state internal yang digunakan untuk komunikasi antar node tanpa mengekspos informasi ini pada input atau output.

```
Lesson3b.py continued  
  
class InputState(TypedDict):  
    user_input: str  
class OutputState(TypedDict):  
    message_output: str  
class PrivateState(TypedDict):  
    private_message: str
```

4.5.3 Contoh: Membangun Graph dengan Multiple State Schema

Untuk memahami bagaimana state input, output, dan privat berinteraksi, mari kita bangun sebuah workflow yang memproses input pengguna dengan menambahkan kata-kata ke dalamnya, menggunakan transformasi internal, dan mengembalikan pesan akhir.

Mendefinisikan Node

Kita mulai dengan mendefinisikan node, masing-masing beroperasi pada bagian state yang berbeda.

1. **add_world:** This node takes the **InputState** , adds the word "World" to the user input, and returns the modified **OverallState** .
2. **add_exclamation:** This node works on the **OverallState** , adds an exclamation mark (!), and stores the result in the internal **PrivateState** .
3. **finalize_message:** This node retrieves the message from the private state and prepares it for output, returning an **OutputState** .

Lesson3b.py continued

```
# Define nodes (functions) that operate on the states
def add_world(state: InputState) -> OverallState:
    partial_message = state["user_input"] + " World"
    print(f"Node 1 - add_world: Transformed '{state['user_input']}' to '{partial_message}'")
    return {"partial_message": partial_message, "user_input": state["user_input"], "message_output": ""}

def add_exclamation(state: OverallState) -> PrivateState:
    private_message = state["partial_message"] + "!"
    print(f"Node 2 - add_exclamation: Transformed '{state['partial_message']}' to '{private_message}'")
    return {"private_message": private_message}

def finalize_message(state: PrivateState) -> OutputState:
    message_output = state["private_message"]
    print(f"Node 3 - finalize_message: Finalized message to '{message_output}'")
    return {"message_output": message_output}
```

Membangun Graph

Selanjutnya, kita menggunakan StateGraph untuk mendefinisikan keseluruhan workflow, menentukan state input, output, dan overall.

Lesson3b.py continued

```
# Create the graph builder with nodes and edges
```

```

builder = StateGraph(OverallState, input=InputState, output=OutputState)
builder.add_node("add_world", add_world)
builder.add_node("add_exclamation", add_exclamation)
builder.add_node("finalize_message", finalize_message)
# Define the edges between nodes
builder.add_edge(START, "add_world")
builder.add_edge("add_world", "add_exclamation")
builder.add_edge("add_exclamation", "finalize_message")
builder.add_edge("finalize_message", END)
# Compile and run the graph
graph = builder.compile()
result = graph.invoke({"user_input": "Hello"})
print(result) # Output: {'message_output': 'Hello World!'}

```

Dalam contoh ini, kita mendemonstrasikan bagaimana node dapat bekerja dengan subset data yang berbeda:

- **InputState** dilewatkan ke node pertama (**add_world**) untuk memproses input pengguna.
- **OverallState** digunakan secara internal oleh **add_world** dan **add_exclamation** untuk mengelola dan memanipulasi data intermediate.
- **PrivateState** digunakan sebagai channel komunikasi internal antara **add_exclamation** dan **finalize_message**, memungkinkan node terakhir mengakses data yang telah ditransformasikan tanpa mengekspos data tersebut ke bagian lain dari graph.
- **OutputState** dikembalikan oleh node **finalize_message**, berisi hasil dari workflow.

Poin-Poin Penting:

1. **State Flexibility:** A node can write to any state channel in the graph, even if that channel wasn't included in its input schema. For example, **add_world** writes to **partial_message** in the **OverallState**, even though it only received the **InputState**.
2. **Internal Communication:** The use of **PrivateState** allows nodes to communicate internally without exposing this information outside the graph. This is useful for separating private transformations from the public-facing output.

3. **State Union:** The *graph's state is the union of all state channels defined at initialization*. Even though we define different input and output schemas, the nodes can still access the overall state and manipulate different parts of it.

Key Takeaways

- **Input dan Output Schema** memungkinkan Anda mengontrol data yang masuk dan keluar dari setiap node, memberikan fleksibilitas dalam bagaimana node berinteraksi dengan state.- **Internal State Channel** memungkinkan node untuk melakukan pemrosesan **intermediate** yang tidak memengaruhi input atau output secara keseluruhan. Ini membuat workflow menjadi lebih bersih dan modular.- **Sistem state management LangGraph** memungkinkan node berbagi data sambil menyembunyikan detail tertentu ketika diperlukan, memfasilitasi workflow yang kompleks dengan mudah.

Dengan menguasai input dan output schema dalam LangGraph, Anda dapat membangun agent AI yang fleksibel, scalable, dan powerful yang mengelola alur data kompleks dengan presisi.

4.6. Bagaimana LangGraph Menyatukan Semuanya

LangGraph dirancang untuk membangun workflow menggunakan komponen-komponen ini. Ketika Anda mendefinisikan sebuah agent LangGraph, Anda membuat node (function) yang menjalankan task, edge yang menentukan alur, dan state yang menyimpan data. LangGraph menggabungkan elemen-elemen ini ke dalam sebuah struktur graph, memungkinkan agent untuk menangani workflow kompleks yang dapat bercabang, melakukan loop, atau beradaptasi berdasarkan data.

Berikut adalah representasi visual dari bagaimana elemen-elemen ini bekerja sama dalam sebuah workflow LangGraph sederhana:

1. **Start** → 2. **Node 1** (modifies the state) → 3. **Node 2** (uses the modified state) → 4. **End**

Setiap node memiliki akses ke state, dan edge mendefinisikan jalur di mana state mengalir. Dengan menyusun workflow dengan cara ini, LangGraph

memudahkan pembuatan agent AI yang dapat menangani task kompleks dan membuat keputusan berdasarkan data yang berubah-ubah.

4.6.1 Mengaktifkan sebuah node

Ketika graph dimulai, semua node dalam keadaan tidak aktif. Node hanya mulai bekerja (menjadi aktif) ketika menerima informasi baru atau pesan dari node lain. Anggap saja seperti sinyal untuk mulai bekerja.

4.6.2 Super Steps - beberapa node berjalan pada waktu yang sama. Satu atau lebih node dapat menjadi aktif pada saat yang sama. Sebuah super-step ibarat sebuah ronde atau fase di mana node (task) tertentu dalam graph dieksekusi bersama-sama. Jika node dapat berjalan secara bersamaan, mereka termasuk dalam super-step yang sama. Jika mereka perlu saling menunggu, mereka berada di super-step yang terpisah.

Selama sebuah super-step, semua node yang telah menerima pesan bekerja pada waktu yang sama. Masing-masing menjalankan function-nya dan mengirimkan update atau hasil. Misalnya, satu node mungkin melakukan perhitungan, dan node lain mungkin memproses sepotong teks.

Pada akhir sebuah super-step, graph memeriksa apakah ada node yang tidak menerima pesan baru. Node-node ini kemudian "vote to halt" (memilih untuk berhenti), yang berarti mereka kembali menjadi tidak aktif.

4.6.3 Kapan Graph Berhenti?

Keseluruhan proses berhenti ketika tidak ada lagi node yang aktif, dan tidak ada pesan yang sedang dalam perjalanan (in transit). Ini berarti semua task telah selesai, dan tidak ada informasi baru yang mengalir melalui graph.

4.7. Pengambilan Keputusan dengan Conditional Edges

Dalam LangGraph, conditional edges memungkinkan agent Anda untuk memilih di antara beberapa jalur berdasarkan kondisi tertentu. Ini membuat workflow menjadi lebih dinamis, memungkinkan agent beradaptasi dengan skenario yang berbeda dan menangani berbagai input secara cerdas. Pada bagian ini, Anda akan memperluas agent "Hello World" untuk menyertakan logika kondisional, yang akan menentukan jalur mana yang diambil oleh workflow berdasarkan konten dari state.

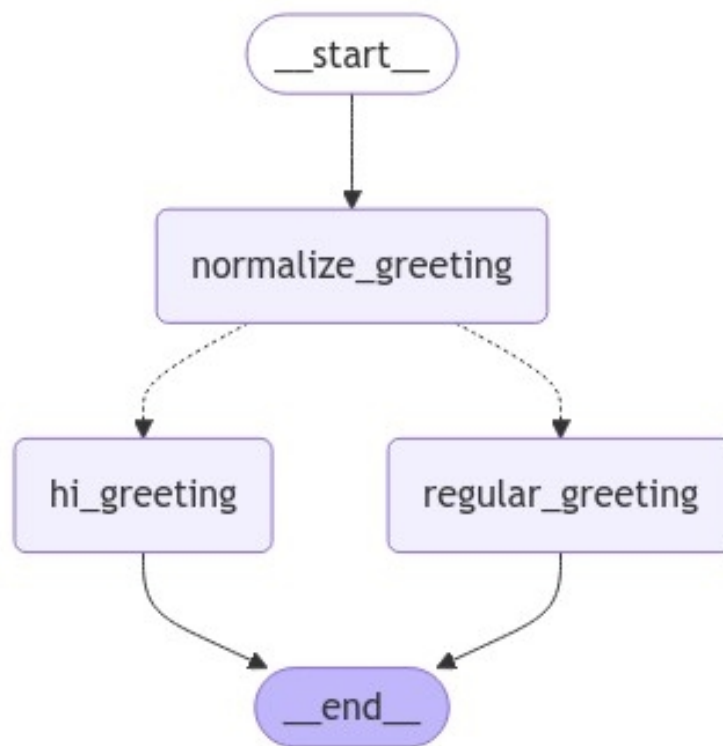
4.7.1 Memahami Conditional Edges

Sebuah conditional edge ibarat sebuah titik percabangan dalam workflow Anda. Alih-alih selalu bergerak dalam satu arah, sebuah conditional edge mengevaluasi sebuah kondisi dan memutuskan jalur mana yang harus diikuti berdasarkan hasilnya. Fleksibilitas ini penting untuk membangun agent AI yang perlu menangani berbagai jenis data atau merespons berbagai input pengguna.

Misalnya, sebuah chatbot layanan pelanggan mungkin mengikuti satu jalur jika pertanyaan pengguna terkait dengan penagihan (billing) dan jalur lain jika terkait dengan dukungan teknis. Dalam LangGraph, conditional edges memungkinkan Anda mengatur jenis titik keputusan ini, sehingga memungkinkan pembuatan workflow yang merespons data dalam state secara dinamis.

Menyiapkan Conditional Greeting Agent. Dalam contoh ini, kita akan menyiapkan sebuah graph yang:

- Normalizes the greeting to ensure that it's lowercase, making it easier to process.
- Evaluates whether the normalized greeting contains the word "hi."
- Branches to different greeting nodes based on this evaluation.



1. Mendefinisikan Struktur State

Pertama, kita perlu mendefinisikan struktur state yang akan digunakan oleh agent untuk melacak pesan sapaan. Dalam contoh ini, state kita hanya akan menyimpan sebuah string sapaan.

```
Lesson3b.py continued
from typing_extensions import TypedDict
# Define the state structure
class GreetingState(TypedDict):
    greeting: str
```

2. Membuat Node Function untuk Setiap Jenis Sapaan

Selanjutnya, kita akan mendefinisikan tiga node function terpisah:

- `normalize_greeting_node`: Node ini mengubah teks sapaan menjadi huruf kecil untuk memastikan konsistensi.-
- `hi_greeting_node`: Node ini menambahkan pesan khusus jika sapaan mengandung "hi."- `regular_greeting_node`: Node ini menambahkan pesan standar untuk sapaan tanpa "hi."

```
Lesson3b.py continued
# Define a preprocessing node to normalize the greeting
def normalize_greeting_node(state):
    # Transform the greeting to lowercase
    state["greeting"] = state["greeting"].lower()
    return state # Return the updated state dictionary
# Define a node for the "Hi" greeting
def hi_greeting_node(state):
    state["greeting"] = "Hi there, " + state["greeting"]
    return state # Return the updated state dictionary
# Define a node for a standard greeting
def regular_greeting_node(state):
    state["greeting"] = "Hello, " + state["greeting"]
    return state # Return the updated state dictionary
```

3. Menyiapkan Function Logika Kondisional

Function kondisional, `choose_greeting_node`, akan memeriksa sapaan yang telah dinormalisasi dan memutuskan apakah akan bercabang ke `hi_greeting_node` atau `regular_greeting_node`.

Lesson3b.py continued

```
# Define the conditional function to choose the appropriate greeting
def choose_greeting_node(state):
    # Choose the node based on whether "hi" is in the normalized greeting
    return "hi_greeting" if "hi" in state["greeting"] else "regular_greeting"
```

4. Membangun Graph dengan Percabangan Kondisional

Sekarang kita akan menyiapkan `LangGraph`, dimulai dari langkah preprocessing (`normalize_greeting_node`), lalu menggunakan `add_conditional_edges` untuk mengevaluasi jalur mana yang harus diikuti.

Lesson3b.py continued

```
from langgraph.graph import StateGraph, START, END

# Initialize the StateGraph
builder = StateGraph(GreetingState)
builder.add_node("normalize_greeting", normalize_greeting_node)
builder.add_node("hi_greeting", hi_greeting_node)
builder.add_node("regular_greeting", regular_greeting_node)
# Add the START to normalization node, then conditionally branch based on the
transformed greeting
builder.add_edge(START, "normalize_greeting")
builder.add_conditional_edges(
    "normalize_greeting", choose_greeting_node, ["hi_greeting", "regular_greeting"]
)
builder.add_edge("hi_greeting", END)
builder.add_edge("regular_greeting", END)
```

Dalam pengaturan ini:

- Workflow dimulai dengan `normalize_greeting_node`, yang memastikan sapaan berada dalam huruf kecil.- Setelah normalisasi, `choose_greeting_node` menentukan langkah berikutnya berdasarkan keberadaan "hi" dalam sapaan.

- Agent kemudian melanjutkan ke `hi_greeting_node` atau `regular_greeting_node`, tergantung pada hasil evaluasi.

5. Meng-compile dan Menguji Agent

Terakhir, kita meng-compile dan menguji graph dengan berbagai sapaan untuk melihat bagaimana perilakunya berdasarkan input.

```
Lesson3b.py continued

# Compile and run the graph
graph = builder.compile()
# Test with a greeting containing "Hi" in various forms (e.g., uppercase, mixed case)
result = graph.invoke({"greeting": "HI THERE!"})
print(result) # Expected Output: {'greeting': 'Hi there, hi there!'}
# Test with a greeting not containing "Hi"
result = graph.invoke({"greeting": "Good morning!"})
print(result) # Expected Output: {'greeting': 'Hello, good morning!'}
```

Ringkasan Konsep Kunci

- **Conditional Edges:** Kita menggunakan `add_conditional_edges` untuk menambahkan logika percabangan setelah langkah normalisasi sapaan.
- **State Transformation:** `normalize_greeting_node` memastikan konsistensi dengan mengubah sapaan menjadi huruf kecil sebelum pemeriksaan kondisional apa pun.
- **Dynamic Path Selection:** Function `choose_greeting_node` memungkinkan workflow untuk beradaptasi berdasarkan konten sapaan, mengarahkannya ke node yang sesuai.

Dengan menggunakan conditional edges dan langkah preprocessing, agent AI ini kini dapat menangani berbagai input sapaan dengan cara yang lebih robust. Pendekatan ini memastikan bahwa meskipun pengguna memasukkan "Hi" dalam huruf besar atau campuran, agent akan mengenalinya dan merespons dengan sesuai.

Kode Lengkap untuk Contoh Logika Kondisional yang Diperluas

Berikut adalah kode lengkapnya, siap untuk Anda uji dan eksperimenkan dengan kondisi tambahan sesuai kebutuhan Anda:

```
Lesson3b.py full example code
```

```

from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict

# Define the state structure
class GreetingState(TypedDict):
    greeting: str

# Define a preprocessing node to normalize the greeting
def normalize_greeting_node(state):
    # Transform the greeting to lowercase
    state["greeting"] = state["greeting"].lower()
    return state # Return the updated state dictionary

# Define a node for the "Hi" greeting
def hi_greeting_node(state):
    state["greeting"] = "Hi there, " + state["greeting"]
    return state # Return the updated state dictionary

# Define a node for a standard greeting
def regular_greeting_node(state):
    state["greeting"] = "Hello, " + state["greeting"]
    return state # Return the updated state dictionary

# Define the conditional function to choose the appropriate greeting
def choose_greeting_node(state):
    # Choose the node based on whether "hi" is in the normalized greeting
    return "hi_greeting" if "hi" in state["greeting"] else "regular_greeting"

# Initialize the StateGraph
builder = StateGraph(GreetingState)
builder.add_node("normalize_greeting", normalize_greeting_node)
builder.add_node("hi_greeting", hi_greeting_node)
builder.add_node("regular_greeting", regular_greeting_node)

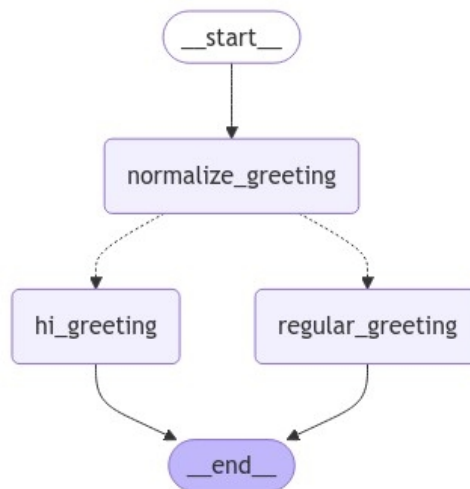
# Add the START to normalization node, then conditionally branch based on the
# transformed greeting
builder.add_edge(START, "normalize_greeting")
builder.add_conditional_edges(
    "normalize_greeting", choose_greeting_node, ["hi_greeting", "regular_greeting"]
)
builder.add_edge("hi_greeting", END)
builder.add_edge("regular_greeting", END)

# Compile and run the graph
graph = builder.compile()

```

```
# Test with a greeting containing "Hi" in various forms (e.g., uppercase, mixed case)
result = graph.invoke({"greeting": "HI THERE!"})
print(result) # Expected Output: {'greeting': 'Hi there, hi there!'}
# Test with a greeting not containing "Hi"
result = graph.invoke({"greeting": "Good morning!"})
print(result) # Expected Output: {'greeting': 'Hello, good morning!'}
```

Output:



Ringkasan dan Langkah Selanjutnya

Dalam contoh yang diperluas ini, Anda telah mempelajari cara untuk:

- Membuat beberapa jalur percabangan berdasarkan kondisi yang berbeda.
- Menggunakan conditional edges untuk memetakan kondisi tertentu ke node yang sesuai.
- Menguji dan memverifikasi alur agent LangGraph Anda dengan input yang berbeda.

Selanjutnya, kita akan menjelajahi bagaimana memperkenalkan modifikasi state dan penanganan error ke dalam workflow, yang akan memungkinkan agent Anda mengelola data dan merespons dengan baik terhadap situasi yang tidak terduga. Ini akan membantu membuat agent LangGraph Anda menjadi lebih robust dan serbaguna.

Quiz: Konsep Inti LangGraph

Bagian 1: Konsep dan Struktur Inti LangGraph

1. Which of the following statements about nodes in LangGraph is FALSE?

- A) Nodes can modify the state data.
- B) Nodes define decision points that determine the path of the workflow.
- C) Nodes can connect directly to the **END** node.
- D) Nodes can be conditional or unconditional.

2. In LangGraph, which element would most likely cause an infinite loop if improperly configured?

- A) START node
- B) Conditional Edge
- C) State Dictionary
- D) Visualization

3. **True or False:** In a LangGraph workflow, all nodes must be directly connected to either the START node or the END node.

Bagian 2: Logika Kondisional dan Percabangan

4. If the following conditional function is used in a LangGraph workflow, which greeting will be used if state["greeting"] contains both "hi" and "hello"?

```
Lesson3b.py continued
def choose_greeting(state):
    if "hi" in state["greeting"]:
        return "hi_greeting"
    elif "hello" in state["greeting"]:
        return "hello_greeting"
    else:
        return "generic_greeting"
```

- A) hi_greeting
- B) hello_greeting
- C) generic_greeting
- D) It will result in an error due to ambiguity.

4. **Which of the following best describes the behavior of the conditional edge in LangGraph?**

- A) It only allows the workflow to proceed if a specific condition is true.
- B) It enables branching by evaluating the state and determining the next node based on predefined conditions.
- C) It defines the start and end points of a workflow.
- D) It modifies the state based on conditional logic.

5. Given the code below, what would happen if "personalized_hi_greeting" is selected as the next node, but there is no direct edge from this node to END ?

Quiz

```
builder.add_conditional_edges(  
    "normalize_greeting_node", choose_greeting,  
    ["personalized_hi_greeting",  
     "personalized_hello_greeting", END])
```

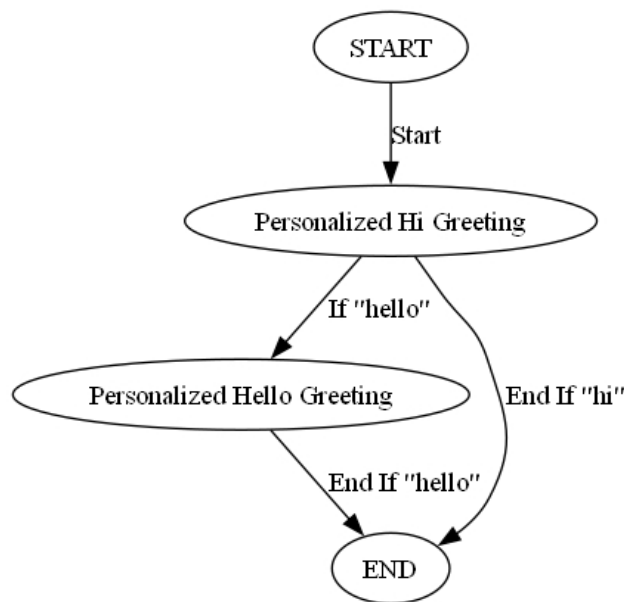
- A) The workflow will skip to the next available node automatically.
- B) The workflow will attempt to execute, but a **GraphRecursionError** will likely occur.
- C) The workflow will end immediately.
- D) The workflow will loop back to the START node.

7. **Which of these is NOT a best practice when defining state in LangGraph workflows?**
- A) Keeping state variables as specific as possible to ensure clarity.
 - B) Storing all potential data in the state dictionary at the start, even if it's not used immediately.
 - C) Updating the state within each node that processes it.
 - D) Using type definitions like `TypedDict` to structure the state.
8. **If an AI agent needs to remember user preferences across multiple interactions, where should this information be stored?**
- A) In a local variable inside each node
 - B) Within the state dictionary, passed along through the workflow
 - C) In a separate file that each node can access
 - D) Within the visualization settings
9. **True or False:** If a node modifies the state in a way that is unexpected by the subsequent nodes, it can lead to an `InvalidUpdateError`.

10. **Why might you use a tool like Graphviz to visualize your LangGraph workflow?**
- A) To automatically debug errors in the code
 - B) To ensure the workflow logic is visually correct and flows as expected
 - C) To speed up the execution time of the workflow

D) To make the workflow more accessible to users without technical knowledge

11. In the visualization below, if the arrow from **Personalized Hi Greeting** to **Personalized Hello Greeting** is missing, which of the following is most likely true?



A) The workflow has no path to execute **Personalized Hello Greeting**.

B) **Personalized Hello Greeting** will be executed as the next node regardless.

C) The workflow will automatically connect **Personalized Hi Greeting** to **END**.

D) The visualization will still work but produce inaccurate output.

12. **True or False:** In LangGraph, visualization directly affects how the workflow executes.

Bagian 5: Pertanyaan Aplikasi dan Skenario Lanjutan

13. Consider a scenario where you have the following conditional function:

Quiz

```
def choose_path(state):  
    if state["greeting"].startswith("hello"):  
        return "hello_path"  
    if "goodbye" in state["greeting"]:  
        return "goodbye_path"  
    return END
```

Jika input sapaan adalah "hello and goodbye," jalur mana yang akan dipilih?

- A) hello_path
- B) goodbye_path
- C) END
- D) Both hello_path and goodbye_path

14. What would happen if you accidentally removed the edge between **Personalized Greeting and **END** ?**

- A) The workflow would still reach **END** through conditional paths.
- B) The workflow would terminate prematurely.
- C) A **GraphRecursionError** could occur if **Normalize Greeting** is revisited.
- D) The workflow would follow a default path to the **END** .

15. Which of the following actions would likely improve performance if the workflow involves multiple branching conditions and paths?

- A) Reducing the number of nodes by merging tasks into fewer nodes
- B) Storing state data as global variables
- C) Using more complex conditions within each node

D) Increasing the number of conditional edges for more precise routing

Kunci Jawaban

1. B) Nodes define decision points that determine the path of the workflow.
2. B) Conditional Edge
3. False
4. A) hi_greeting
5. B) It enables branching by evaluating the state and determining the next node based on predefined conditions.
6. B) The workflow will attempt to execute, but a **GraphRecursionError** will likely occur.
7. B) Storing all potential data in the state dictionary at the start, even if it's not used immediately.
8. B) Within the state dictionary, passed along through the workflow
9. True
10. B) To ensure the workflow logic is visually correct and flows as expected
11. A) The workflow has no path to execute **Personalized Hello Greeting**.
12. False
13. A) hello_path
14. C) A **GraphRecursionError** could occur if **Personalized HI Greeting** is revisited.
15. A) Reducing the number of nodes by merging tasks into fewer nodes

Chapter 5

Building Your First AI Agent

Tujuan:

Di bab ini, Anda akan membuat agent AI pertama yang berfungsi penuh menggunakan LangGraph. Agent tersebut akan didukung oleh Large Language Model (LLM), yang akan menyediakan kecerdasan untuk menafsirkan input pengguna dan menghasilkan respons. LLM juga akan mampu menggunakan tools, seperti API eksternal, untuk melakukan tugas-tugas tertentu.

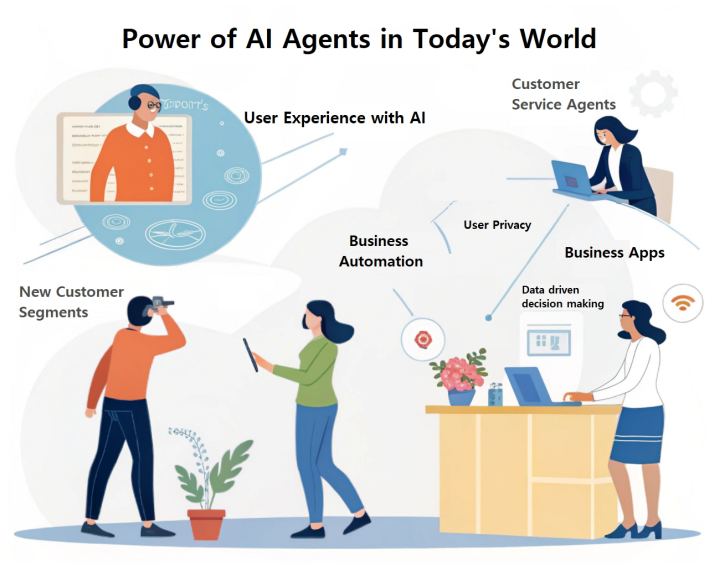
Pada akhir bab ini, Anda akan:

- Mengintegrasikan LLM ke dalam LangGraph.- Mengelola alur perpesanan (human messages, system prompts, dan AI responses).- Menyiapkan pemrosesan input pengguna secara berkelanjutan.- Mengelola API key dan konfigurasi environment secara aman.

5.1 Pendahuluan: Kekuatan AI Agents

Di dunia saat ini, AI agents menjadi bagian integral dari kehidupan kita, mulai dari layanan pelanggan hingga berinteraksi dengan pengguna dan mengotomatiskan tugas dalam operasi bisnis. Agent-agent ini dibangun untuk melakukan tugas secara otonom, berinteraksi dengan manusia, dan terkadang bahkan mengambil keputusan berdasarkan input yang mereka terima. Yang membuat mereka cerdas adalah kombinasi dari natural language

processing, logika pengambilan keputusan, dan kemampuan untuk mengambil tindakan dengan berinteraksi dengan tools dan sistem eksternal.



Di bab ini, Anda akan mengambil langkah pertama untuk membangun AI agent semacam ini dengan menggunakan LangGraph sebagai framework dan LLM sebagai mesin kecerdasan intinya. Agent Anda akan:

1. Interact with users by understanding their queries.
2. Use the LLM to process natural language, turning human-like text into actionable data.
3. Trigger tool nodes to perform specific tasks, such as retrieving information from external APIs, scheduling events, or performing computations.

AI agent ini akan sederhana secara konsep tetapi kuat dalam eksekusinya. Dengan menggabungkan kemampuan LLM dan tool nodes di LangGraph, Anda akan membuka potensi penuh dari autonomous AI agents.

5.2 Menyiapkan OpenAI API Key dan Environment Anda

Sebelum kita mulai coding, Anda akan memerlukan OpenAI API key untuk memanggil LLM. Berikut cara menyiapkannya dengan aman.

1. Obtain an OpenAI API Key:

- Sign up or log in to [OpenAI Platform](#) and navigate to the API keys section to generate your API key.

2. Store the API Key Securely:

- API Keys should never be stored as part of your main code. They are best stored as secrets in your environment as environment variables, or secrets with your cloud provider, like AWS or Google secrets.
- In this case, we store the API key in an `.env` file to keep it secure. This will prevent hard coding and leaking of sensitive information in your code.

Buat file `.env` di direktori project Anda:

```
terminal  
touch .env
```

Tambahkan API key Anda ke file `.env`:

```
terminal  
OPENAI_API_KEY="your-api-key-here"
```

Kemudian, muat API key dari file `.env` di dalam kode Anda:

```
#lesson4a.py  
  
from dotenv import load_dotenv  
import os  
  
# Load environment variables from .env file  
load_dotenv()  
  
# Get the API key from environment variables  
api_key = os.getenv("OPENAI_API_KEY")  
  
# Check if API key exists  
if api_key is None:  
    print("Error: OPENAI_API_KEY not found in environment variables")  
else:  
    print("API key loaded successfully")
```

Dengan pengaturan ini, Anda dapat memuat OpenAI API key Anda secara aman saat script dijalankan.

5.3 Dasar-Dasar Perpesanan LangGraph

Di LangGraph, interaksi antara pengguna dan AI agent dikelola melalui serangkaian messages. Ini meliputi:

- **Human Messages:** Input dari pengguna, seperti pertanyaan atau perintah.
- **AI Messages:** Respons yang dihasilkan oleh LLM.
- **System Messages:** Pesan yang dikirim oleh agent untuk memberikan konteks atau instruksi tambahan kepada LLM (misalnya, mengatur perilaku atau kepribadian).
- **Prompt Templates:** Digunakan untuk memformat dan menyusun input ke LLM, membantunya menghasilkan respons yang lebih akurat.

Untuk saat ini, mari kita fokus pada Human Message dan AI Message untuk membangun interaksi bolak-balik yang sederhana.

5.3.1 Kesederhanaan Penggunaan LangGraph untuk Membuat AI Agents

LangGraph adalah framework yang kokoh untuk membangun AI agents yang memanfaatkan graph-based workflows. Framework ini berpusat pada konsep node dan edge, yang masing-masing merepresentasikan aksi dan alur data. Di bab ini, Anda akan fokus pada penggunaan tiga jenis node utama:

1. **LLM Node:** This node uses an LLM (e.g., GPT-4o) to process text inputs and generate responses.
2. **Tool Node:** This node is responsible for triggering external actions, such as calling an API or executing a function.
3. **Decision Node:** This node helps the agent decide the next step based on the output from the LLM or tool node.

Komponen-komponen ini akan bekerja bersama untuk menciptakan sistem yang dinamis dan cerdas yang dapat memahami input pengguna dan mengambil tindakan yang bermakna.

5.3.2 Studi Kasus AI Agent: Question & Answer AI Agent

Agar contoh ini konkret, kita akan membangun Q&A agent praktis yang dapat:

1. Accept a user's question.
2. Use an LLM to process the question and formulate a response.
3. Trigger tool nodes when needed (e.g., fetching weather information or news articles).
4. Respond back to the user with a detailed answer.

Mendefinisikan Masalah untuk Question and Answer (Q&A) AI Agent

Studi kasus ini sangat praktis dan meletakkan dasar untuk agent yang lebih kompleks di bab-bab berikutnya. Ini juga menunjukkan betapa kuatnya sebuah AI agent ketika menggabungkan pemahaman natural language dengan tugas-tugas yang dapat ditindaklanjuti.

Sebelum masuk ke pembuatan agent, mari kita definisikan apa yang akan dilakukannya. Tujuan kita adalah membuat Q&A agent yang dapat:

1. **Understand questions** from a user, such as “What’s the weather today?” or “Tell me a joke.”
2. **Generate intelligent responses** using an LLM as the intelligence provider.
3. **Trigger tool nodes** when necessary to fetch real-time information (e.g., calling an external weather API or stock prices).

Dalam contoh ini, LLM akan bertanggung jawab untuk memahami maksud (intent) pengguna, dan tool nodes akan digunakan untuk menjalankan tugas-tugas tertentu.

Menyiapkan Graph

Sekarang setelah kita memiliki tujuan yang jelas, kita dapat mulai menyiapkan LangGraph kita. Sebuah graph di LangGraph terdiri dari node dan edge yang mendefinisikan bagaimana data bergerak melalui sistem. Untuk agent ini, Anda akan membuat graph yang terdiri dari node-node berikut:

1. **Input Node:** This node will accept user input (the question). It will pass this input to the LLM for processing.
2. **LLM Node:** The LLM will take the user input, process it, and generate a response. The LLM node can either generate a direct

response or trigger tool nodes depending on the question.

3. **Tool Nodes:** These nodes will be triggered based on the LLM's output. For example, if the LLM detects that the user is asking for the weather, it will trigger a tool node that fetches the weather data.
4. **Output Node:** Once the response has been generated, it will be returned to the user.

Membuat AI Agent Dasar Kita akan mulai dengan menyiapkan workflow LangGraph untuk berinteraksi dengan LLM. Dalam kasus ini, kita hanya berfokus pada pemanggilan LLM untuk merespons query pengguna.

```
#lesson5a.py

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START, END
# Initialize the LLM model (using OpenAI in this example)
model = ChatOpenAI(model="gpt-4o-mini", api_key="YOUR_API_KEY")
# Node function to handle the user query and call the LLM
def call_llm(state: MessagesState):
    messages = state["messages"]
    response = model.invoke(messages[-1].content)
    return {"messages": [response]}
# Define the graph
workflow = StateGraph(MessagesState)
# Add the node to call the LLM
workflow.add_node("call_llm", call_llm)
# Define the edges (start -> LLM -> end)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
# Compile the workflow
app = workflow.compile()
# Example input message from the user
input_message = {
    "messages": [("human", "What is the capital of Kenya?")]
}
# Run the workflow
for chunk in app.stream(input_message, stream_mode="values"):
```

```
chunk["messages"][-1].pretty_print()
```

Berikut adalah rincian dari apa yang sedang terjadi:

- Kita menggunakan ChatOpenAI, sebuah LLM, untuk menangani interaksi berbasis bahasa.- Fungsi call_llm mengirimkan input pengguna ke model dan mengembalikan respons yang dihasilkan.- Kita mendefinisikan graph sederhana dengan satu node (call_llm) dan menghubungkannya ke titik start dan end.- Kita mengajukan pertanyaan kepada LLM sebagai input_message dan menangkap respons sebagai stream hasil.

Output yang diharapkan dari menjalankan kode ini adalah sebagai berikut:

```
=====Human
Message=====
Apa ibu kota Kenya?
```

```
=====AI
Message=====
=====Ibu kota Kenya adalah Nairobi.
```

5.4 Pemrosesan Input Pengguna Berkelanjutan

Salah satu fitur penting dari AI agent yang tangguh adalah kemampuannya untuk menangani input pengguna secara berkelanjutan, di mana pengguna dapat mengajukan beberapa pertanyaan dalam satu sesi. Kita akan memperluas agent kita untuk mendukung interaction loop ini, memproses input pengguna hingga percakapan diakhiri.

```
#lesson5b.py

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START, END
from dotenv import load_dotenv
import os

# Load API key from .env file
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

```

# Initialize the LLM (using OpenAI's GPT-4o)
model = ChatOpenAI(model="gpt-4o-mini", api_key=api_key)
# Node function to handle the user query and call the LLM
def call_llm(state: MessagesState):
    messages = state["messages"]
    response = model.invoke(messages[-1].content)
    return {"messages": [response]}
# Define the graph
workflow = StateGraph(MessagesState)
# Add the node to call the LLM
workflow.add_node("call_llm", call_llm)
# Define the edges (start -> LLM -> end)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
# Compile the workflow
app = workflow.compile()
# Function to continuously take user input
def interact_with_agent():
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Ending the conversation.")
            break
        input_message = {
            "messages": [("human", user_input)]
        }
        for chunk in app.stream(input_message, stream_mode="values"):
            chunk["messages"][-1].pretty_print()
# Start interacting with the agent
interact_with_agent()

```

Penjelasan:

- **Langkah 1: Pengaturan Aman:** Kita memuat OpenAI API key dari file .env untuk menjaganya tetap aman.
- **Langkah 2: LLM Node:** Fungsi call_llm memproses input pengguna melalui LLM dan mengembalikan respons AI.

- **Langkah 3: Pengaturan Graph:** Kita membuat StateGraph dan menambahkan node untuk memanggil LLM. Graph terhubung dari start hingga end, memproses input pengguna dan mengembalikan respons.-
- Langkah 4: Input Berkelanjutan:** Fungsi `interact_with_agent` menerima input pengguna secara berkelanjutan, memungkinkan pengguna mengajukan beberapa pertanyaan dalam satu sesi. Percakapan berakhir ketika pengguna mengetik "exit" atau "quit."

5.4.1 Menjalankan Agent

Sekarang setelah AI agent siap, Anda dapat menjalankan script dan mulai berinteraksi dengan agent. Sebagai contoh:

You: Apa ibu kota

Kenya?=====A

I

Message=====

Ibu kota Kenya adalah

Nairobi.=====Hu

man

Message=====

Ceritakan sebuah

lelucon=====Ai

Message=====

Mengapa orang-orangan sawah memenangkan penghargaan? Karena dia luar biasa di bidangnya!

Anda dapat terus mengajukan pertanyaan, dan agent akan terus merespons hingga Anda mengetik "exit" atau "quit."

Ringkasan Bagian

Di bagian pertama ini, kita telah berhasil membangun AI agent dasar yang:

- Menerima input pengguna secara berkelanjutan.-
- Memanggil LLM untuk menghasilkan respons yang cerdas.-
- Mengelola API key secara aman melalui `.env`.

5.5 Pengantar Tools di AI Agents

Meskipun LLM (Large Language Model) unggul dalam menghasilkan respons berdasarkan pemahaman bahasa, LLM memiliki keterbatasan. Misalnya, LLM tidak memiliki informasi real-time, dan tidak dapat melakukan tugas tertentu seperti memanggil API atau menjalankan perhitungan. Di sinilah tools berperan.

Tools memungkinkan AI agent untuk:

- Mengambil data real-time (misalnya, informasi cuaca, harga saham).-
- Melakukan tugas tertentu (misalnya, mengirim email, menjadwalkan acara).-
- Mengambil informasi dari database atau API eksternal.

Di bagian ini, kita akan memperkenalkan ToolNode, sebuah node LangGraph yang dirancang khusus untuk memanggil tools eksternal, dan mengintegrasikannya ke dalam agent kita yang sudah ada.

5.5.1 Mendefinisikan Tool

Untuk mendemonstrasikan integrasi tool, mari kita definisikan sebuah tool sederhana yang mengambil informasi cuaca untuk kota tertentu. Tool ini akan mensimulasikan layanan cuaca dengan mengembalikan respons yang telah ditentukan sebelumnya untuk kota-kota tertentu.

Mendefinisikan Tool Menggunakan @tool Decorator

LangGraph menyediakan cara yang mudah untuk mendefinisikan tools menggunakan @tool decorator, yang memudahkan untuk mengubah fungsi Python menjadi callable tools.

```
#lesson5c.py

from langchain_core.tools import tool
# Define a tool to get the weather for a city
@tool
def get_weather(location: str):
    """Fetch the current weather for a specific location."""
    weather_data = {
        "San Francisco": "It's 60 degrees and foggy.",
        "New York": "It's 90 degrees and sunny.",
        "London": "It's 70 degrees and cloudy."
    }
    return weather_data.get(location, "Weather information is unavailable for this location.")
```


Penjelasan:

- **@tool Decorator:** Decorator ini menandai fungsi sebagai tool yang dapat digunakan di dalam LangGraph. Anda bisa membayangkannya seperti memberi tag pada fungsi sehingga LangGraph tahu bahwa ia dapat memanggilnya nanti.-

Logika Fungsi: Fungsi `get_weather` menerima `location` sebagai input dan mengembalikan cuaca untuk lokasi tersebut. Jika lokasi tidak ditemukan dalam dictionary, fungsi ini mengembalikan pesan default.

5.5.2 Mengintegrasikan Tools dengan ToolNode

Sekarang setelah kita memiliki tool untuk mengambil informasi cuaca, kita perlu mengintegrasikannya ke dalam agent kita. LangGraph menyediakan jenis node bernama `ToolNode`, yang bertanggung jawab untuk memanggil tools eksternal seperti yang baru saja kita definisikan.

Langkah 2: Menyiapkan ToolNode

```
#lesson5d.py
from langgraph.prebuilt import ToolNode
# Create a ToolNode with the weather tool
tool_node = ToolNode([get_weather])
```

Penjelasan:

- **ToolNode:** Node ini dirancang khusus untuk memanggil tools eksternal. Node ini menerima daftar tools sebagai input. Dalam kasus ini, kita meneruskan tool `get_weather` ke `ToolNode`.

Sekarang setelah `ToolNode` kita siap, kita dapat mengintegrasikannya ke dalam workflow LangGraph kita.

5.5.3 Memperbarui LLM untuk Menggunakan Tools

Di bagian pertama, agent kita hanya menggunakan LLM untuk merespons query pengguna. Sekarang, kita akan memperbarui graph sehingga agent juga dapat memanggil tool cuaca jika pengguna meminta informasi cuaca.

```
#lesson5d.py continued
model=ChatOpenAI(model="gpt-4o-mini",api_key=api_key).bind_tools([get_weather])
# Step 4: Function to handle user queries and process LLM + tool results
def call_llm(state: MessagesState):
    messages = state["messages"]
```

```

# The LLM will decide if it should invoke a tool based on the user input
response = model.invoke(messages[-1].content)
# If the LLM generates a tool call, handle it
if response.tool_calls:
    tool_result = tool_node.invoke({"messages": [response]})
    # Append the tool's output to the response message
    tool_message = tool_result["messages"][-1].content
    response.content += f"\nTool Result: {tool_message}"
    return {"messages": [response]}
# Step 5: Create the LangGraph workflow
workflow = StateGraph(MessagesState)
# Step 6: Add the LLM node to the workflow
workflow.add_node("call_llm", call_llm)
# Step 7: Define edges to control the flow (LLM -> End)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
# Step 8: Compile the workflow
app = workflow.compile()

```

Penjelasan:

- **Metode bind_tools:** Metode bind_tools digunakan untuk mengikat tool get_weather ke LLM. Dengan melakukan ini, LLM menjadi sadar akan tool tersebut dan dapat memilih kapan memanggilnya berdasarkan input pengguna. Jika query pengguna tentang cuaca, LLM akan secara otomatis memanggil tool get_weather dan menyertakan hasilnya dalam responsnya.- **Pembaruan Graph:** Kita menambahkan node baru ke graph, call_llm, yang bertanggung jawab untuk berinteraksi dengan tool. Ini berarti setelah LLM memproses input pengguna, ia akan memutuskan apakah akan memanggil tool tersebut.-

Kontrol Alur: Jika LLM mendeteksi bahwa pengguna bertanya tentang cuaca, ia akan memanggil tool cuaca. Setelah tool selesai dieksekusi, percakapan akan berakhir.

5.5.4 Menangani Input Berkelanjutan dengan Tools

Mari kita perluas fungsi input berkelanjutan kita untuk menangani baik query umum (diproses oleh LLM) maupun query spesifik (diproses oleh tool).

```
#lesson5d.py continued

# Function to continuously take user input and decide between LLM and tool calls
def interact_with_agent():
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Ending the conversation.")
            break
        input_message = {
            "messages": [("human", user_input)]
        }
        for chunk in app.stream(input_message, stream_mode="values"):
            chunk["messages"][-1].pretty_print()
# Start interacting with the agent
interact_with_agent()
```

Penjelasan:

- **Continuous Input Loop:** Agent mendengarkan input pengguna secara terus-menerus, memprosesnya melalui graph dan menentukan apakah akan memanggil LLM atau tool berdasarkan permintaan pengguna.

5.5.5 Menjalankan Agent dengan Tools

Sekarang setelah semuanya siap, mari kita jalankan agent dan ajukan pertanyaan terkait cuaca.

Kode lengkapnya:

```
#lesson5d.py complete code example

# Import necessary libraries
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_core.tools import tool
from langgraph.prebuilt import ToolNode
from dotenv import load_dotenv
import os

# Step 1: Load the API key from .env
load_dotenv()
```

```

api_key = os.getenv("OPENAI_API_KEY")
# Step 2: Define the tool to get weather information
@tool
def get_weather(location: str):
    """Fetch the current weather for a specific location."""
    weather_data = {
        "San Francisco": "It's 60 degrees and foggy.",
        "New York": "It's 90 degrees and sunny.",
        "London": "It's 70 degrees and cloudy."
    }
    return weather_data.get(location, "Weather information is unavailable for this location.")
# Step 3: Initialize the LLM (OpenAI's GPT-4o-mini model) and bind the tool
tool_node = ToolNode([get_weather], handle_tool_errors=False)
model = ChatOpenAI(model="gpt-4o-mini",
api_key=api_key).bind_tools([get_weather])
# Step 4: Function to handle user queries and process LLM + tool results
def call_llm(state: MessagesState):
    messages = state["messages"]
    # The LLM will decide if it should invoke a tool based on the user input
    response = model.invoke(messages[-1].content)
    # If the LLM generates a tool call, handle it
    if response.tool_calls:
        tool_result = tool_node.invoke({"messages": [response]})
        # Append the tool's output to the response message
        tool_message = tool_result["messages"][-1].content
        response.content += f"\nTool Result: {tool_message}"
    return {"messages": [response]}
# Step 5: Create the LangGraph workflow
workflow = StateGraph(MessagesState)
# Step 6: Add the LLM node to the workflow
workflow.add_node("call_llm", call_llm)
# Step 7: Define edges to control the flow (LLM -> End)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
# Step 8: Compile the workflow
app = workflow.compile()

```

```

# Step 9: Function to interact with the agent continuously
def interact_with_agent():
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Ending the conversation.")
            break
        # Prepare the user input for processing
        input_message = {
            "messages": [("human", user_input)]
        }
        # Process the input through the workflow and return the response
        for chunk in app.stream(input_message, stream_mode="values"):
            chunk["messages"][-1].pretty_print()
# Step 10: Start interacting with the AI agent
interact_with_agent()

```

Output yang

Diharapkan:=====

Human

Message=====Bag

aimana cuaca di

Nairobi?=====AI

Message=====Cua

ca di San Francisco adalah: 27 derajat dan cerah.

5.5.6 Bagaimana Sistem Memutuskan untuk Menggunakan Tool

Anda mungkin bertanya-tanya, "Bagaimana sistem tahu kapan harus memanggil tool dan kapan cukup menghasilkan respons?" Proses pengambilan keputusan ini didasarkan pada output LLM. Berikut cara kerjanya:

1. The user asks a question, and the **LLM node** processes the input.
2. The LLM generates a response. If the response contains a **tool call** (e.g., "Check the weather for San Francisco"), the graph triggers the **ToolNode**.

3. The **ToolNode** calls the specified tool (in this case, `get_weather`) and returns the result.

Interaksi antara LLM dan tools inilah yang memberi agent Anda kemampuan untuk memahami input pengguna dan mengambil tindakan nyata berdasarkan pemahaman tersebut.

5.5.7 Menangani Errors di Tools

LangGraph menyediakan penanganan error bawaan untuk tools. Jika terjadi kesalahan selama eksekusi tool (misalnya, API tidak dapat diakses), LangGraph akan menangani error tersebut dan mengembalikan pesan yang bermakna kepada pengguna. Anda dapat menyesuaikan penanganan error dengan mengonfigurasi ToolNode untuk menangani atau meneruskan error.

```
#lesson5d.py continued  
  
# ToolNode with error handling disabled (propagating errors to the user)  
tool_node = ToolNode([get_weather], handle_tool_errors=False)
```

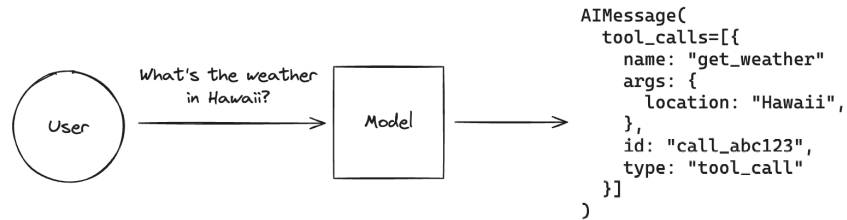
Dalam kasus ini, jika tool cuaca mengalami error, agent akan memberi tahu pengguna bahwa ada yang salah, alih-alih menangani error tersebut secara diam-diam.

5.5.9 Penjelasan: Dasar-Dasar Tool Calling di LangGraph: Gambaran Sederhana

Di LangGraph, tools adalah fungsi bantuan yang dapat dipanggil oleh AI agent untuk melakukan tugas tertentu, seperti mengambil data atau memproses input pengguna. Berikut adalah panduan yang disederhanakan untuk memahami cara kerja tool calling, langkah demi langkah.

Apa itu Tool Calling di LangGraph?

Di LangGraph, tools adalah fungsi bantuan yang dapat dipanggil oleh AI agent untuk melakukan tugas tertentu. Tools ini memungkinkan agent untuk mengakses data eksternal, melakukan komputasi tertentu, atau mengambil tindakan yang tidak dapat ditangani oleh agent sendirian. Misalnya, Anda dapat membuat tool yang mencari entri dalam database, memeriksa tanggal yang tersedia dalam kalender, atau mengambil data keuangan.



Persyaratan Utama

Sebelum masuk ke prosesnya, penting untuk memahami bahwa agar ToolNode berfungsi, state graph harus berisi kunci messages. Kunci ini menyimpan daftar interaksi, yang mencakup tool calls, input pengguna, dan hasil.

- **Tool:** Sebuah fungsi yang melakukan tugas, seperti mengambil entri database atau melakukan perhitungan.- **ToolNode:** Sebuah node yang bertanggung jawab untuk mengeksekusi tools ketika AI agent memintanya.- **StateGraph:** State dari workflow AI, yang mencakup kunci messages untuk menyimpan tool calls dan respons.

Contoh: Mengambil Profil Pengguna dari Database

Mari kita gunakan contoh yang lebih praktis: mengambil profil pengguna dari database. Dalam contoh ini, AI agent akan memanggil tool yang mencari informasi pengguna berdasarkan user ID.

Mendefinisikan Tool Anda

Kita mendefinisikan tool menggunakan `@tool` decorator, yang menandainya sebagai dapat dipanggil oleh ToolNode. Dalam kasus ini, tool akan mencari profil pengguna dalam database yang telah ditentukan sebelumnya (disimulasikan di sini dengan sebuah dictionary).

```
#lesson5e.py

from langchain_core.tools import tool
# Define a tool to get user profile by user ID
@tool
def get_user_profile(user_id: str):
    user_data = {
        "101": {"name": "Alice", "age": 30, "location": "New York"},
        "102": {"name": "Bob", "age": 25, "location": "San Francisco"}
```

```
}  
return user_data.get(user_id, "User profile not found.")
```

Di sini, tool `get_user_profile` menerima user ID sebagai input dan mengembalikan informasi profil pengguna. Jika user ID tidak ditemukan, tool ini mengembalikan pesan "User profile not found".

Menyiapkan ToolNode

Sekarang setelah kita mendefinisikan tool kita, langkah selanjutnya adalah menyiapkan ToolNode. ToolNode bertanggung jawab untuk memanggil tool `get_user_profile` ketika AI agent memintanya.

```
#lesson5e.py continued  
  
from langgraph.prebuilt import ToolNode  
# Set up the ToolNode with the get_user_profile tool  
tools = [get_user_profile]  
tool_node = ToolNode(tools)
```

Di sini, kita meneruskan tool `get_user_profile` kita ke ToolNode. Node ini akan memproses tool call dan mengembalikan hasilnya.

Membuat AIMessage untuk Tool Calling

Selanjutnya, kita perlu membuat AIMessage yang memberi tahu AI agent tool mana yang harus dipanggil dan menyediakan input yang diperlukan (dalam kasus ini, user ID). AIMessage ini akan menjadi bagian dari state graph di bawah kunci `messages`.

```
#lesson5e.py continued  
  
from langchain_core.messages import AIMessage  
# Create a message with a tool call for retrieving user profile  
message_with_tool_call = AIMessage(  
    content="",  
    tool_calls=[  
        {  
            "name": "get_user_profile",  
            "args": {"user_id": "101"},  
            "id": "tool_call_id",  
            "type": "tool_call"  
        }  
    ]  
)
```



```
)
```

AIMessage ini meminta AI agent untuk memanggil tool `get_user_profile` dengan user ID 101.

Menyiapkan StateGraph

Sebelum kita dapat memanggil tool, kita perlu menyiapkan StateGraph. Graph ini menyimpan state dari workflow AI, termasuk daftar messages (yang mencakup tool call).

```
#lesson5e.py continued

# Initialize the state with a messages key
state = {
    "messages": [message_with_tool_call]
}
```

StateGraph harus memiliki kunci `messages`, yang berisi daftar interaksi (seperti tool call yang baru saja kita buat).

Memanggil Tool Menggunakan ToolNode

Sekarang, kita dapat menggunakan ToolNode untuk memproses state dan memanggil tool. ToolNode akan melihat pesan terakhir dalam state, menemukan tool call, dan mengeksekusi tool yang sesuai (`get_user_profile`).

```
#lesson5e.py continued

# Use ToolNode to invoke the tool and update the state
result = tool_node.invoke(state)
print(result)
```

Ini akan memanggil tool `get_user_profile` dengan user ID 101 dan mengembalikan profil pengguna untuk Alice.

Contoh Output

Kode lengkapnya:

```
#lesson5e.py complete code example

from langchain_core.tools import tool
from langgraph.prebuilt import ToolNode
from langchain_core.messages import AIMessage
```

```

# Step 1: Define the tool
@tool
def get_user_profile(user_id: str):
    """Fetch the profile of a user by user ID."""
    user_data = {
        "101": {"name": "Alice", "age": 30, "location": "New York"},
        "102": {"name": "Bob", "age": 25, "location": "San Francisco"}
    }
    return user_data.get(user_id, "User profile not found.")

# Step 2: Set up the ToolNode with the get_user_profile tool
tools = [get_user_profile]
tool_node = ToolNode(tools)

# Step 3: Create an AIMessage for the tool call
message_with_tool_call = AIMessage(
    content="",
    tool_calls=[{
        "name": "get_user_profile",
        "args": {"user_id": "101"},
        "id": "tool_call_id",
        "type": "tool_call"
    }]
)

# Step 4: Set up the state with the messages key
state = {
    "messages": [message_with_tool_call]
}

# Step 5: Invoke the ToolNode with the state and get the result
result = tool_node.invoke(state)

# Output the result
print(result)
{'messages': [ToolMessage(content="{'name': 'Alice', 'age': 30, 'location': 'New York'}",
    name='get_user_profile', tool_call_id='tool_call_id')]}

```

Setelah menjalankan kode, outputnya akan terlihat seperti ini:

```
#lesson5e.py output
```

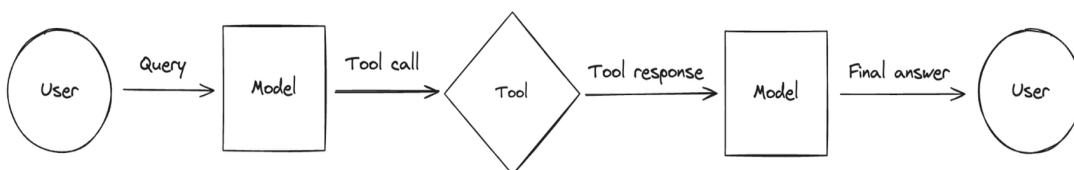
```
{'messages': [ToolMessage(content='{\"name\": \"Alice\", \"age\": 30, \"location\": \"New York\"}',name='get_user_profile', tool_call_id='tool_call_id')]}
```

Ini menunjukkan bahwa ToolNode berhasil memanggil tool `get_user_profile` dan mengembalikan profil Alice.

Ringkasan Tool Calling

1. **Tools** in LangGraph are functions that can be called by the AI agent to perform specific tasks.
2. The **ToolNode** is responsible for invoking the tools when requested.
3. **StateGraph** must have a `messages` key, where tool calls and results are stored.
4. The **AIMessage** tells the AI agent which tool to call and what input to pass.
5. The **ToolNode** updates the state with the result of the tool call.

Dengan memastikan StateGraph Anda memiliki kunci `messages` yang diperlukan, Anda dapat dengan mudah menggunakan tools seperti `get_user_profile` atau fungsionalitas kustom lainnya dalam workflow LangGraph Anda. Panduan yang direvisi ini memberikan pemahaman yang jelas tentang bagaimana ToolNode berinteraksi dengan state untuk memproses dan mengembalikan hasil tool call.



Ringkasan Bab 5

Di bagian kedua ini, kita memperluas kemampuan AI agent kita dengan:

- Mendefinisikan tool untuk mengambil informasi cuaca real-time.- Mengintegrasikan tool ke dalam workflow LangGraph kita menggunakan ToolNode.

- Memperbarui agent untuk memutuskan kapan memanggil LLM dan kapan menggunakan tool.

Dengan menambahkan tools ke AI agent Anda, Anda memungkinkannya melakukan tugas yang lebih kompleks dan berguna, seperti mengambil data real-time, melakukan perhitungan, atau berinteraksi dengan sistem eksternal. Ini meletakkan dasar untuk membangun agent yang bahkan lebih canggih di bab-bab berikutnya.

Apa Selanjutnya?

Sekarang setelah Anda memiliki pemahaman yang kuat tentang cara mengintegrasikan LLMs dan tools ke dalam workflow LangGraph Anda, kita akan menjelajahi state management dan persistence di bab berikutnya. Ini akan memungkinkan agent Anda mengingat informasi antar interaksi dan sesi, semakin meningkatkan fungsionalitas dan pengalaman pengguna mereka.

Chapter 6

Memperkenalkan Memory dalam AI Agents

Di bab ini, kita akan fokus pada short-term memory dalam AI agents, sebuah fitur utama yang memungkinkan agent mengingat interaksi dalam satu sesi tetapi tidak di berbagai sesi. Short-term memory membantu agent mempertahankan konteks selama percakapan, membuatnya lebih koheren dan responsif terhadap input pengguna.

Kita akan mulai dengan mendemonstrasikan AI agent tanpa memory apa pun dan menunjukkan bagaimana penambahan short-term memory meningkatkan interaksi. Kita akan menggunakan built-in memory tools milik LangGraph untuk mencapai ini.

6.1 Masalahnya: Agent Tanpa Memory

Pertama-tama, mari kita buat AI agent dasar yang tidak memiliki memory. Agent ini memproses setiap query pengguna secara individual, tanpa pengetahuan tentang interaksi sebelumnya.

Contoh: AI Agent Dasar Tanpa Memory

```
#lesson6a.py

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START, END
from dotenv import load_dotenv
import os
```

```

# Load API key from .env file
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
# Initialize the LLM (using OpenAI's GPT-4o-mini)
model = ChatOpenAI(model="gpt-4o-mini", api_key=api_key)
# Node function to handle the user query and call the LLM
def call_llm(state: MessagesState):
    messages = state["messages"]
    response = model.invoke(messages[-1].content)
    return {"messages": [response]}
# Define the graph
workflow = StateGraph(MessagesState)
# Add the node to call the LLM
workflow.add_node("call_llm", call_llm)
# Define the edges (start -> LLM -> end)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
# Compile the workflow
app = workflow.compile()
# Function to continuously take user input
def interact_with_agent():
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Ending the conversation.")
            break
        input_message = {
            "messages": [("human", user_input)]
        }
        for chunk in app.stream(input_message, stream_mode="values"):
            chunk["messages"][-1].pretty_print()
# Start interacting with the agent
interact_with_agent()

```

Dalam kasus ini:

- Agent menerima input pengguna, mengirimkannya ke language model, dan merespons.

- **Tanpa memori: Agent melupakan segalanya setelah setiap interaksi, memperlakukan setiap input sebagai query baru yang terisolasi.**

Contoh Interaksi Tanpa Memori:

Anda: Siapa nama saya?Agent: Maaf, saya tidak tahu nama Anda.

Anda: Nama saya John.Agent: Senang bertemu Anda, John.

Anda: Siapa nama saya?Agent: Maaf, saya tidak tahu nama Anda.

Perhatikan bahwa agent tidak dapat mengingat bahwa pengguna baru saja menyebutkan namanya beberapa saat yang lalu.

6.2 Meningkatkan Agent dengan Short-Term Memory

Sekarang, mari kita tambahkan short-term memory menggunakan MemorySaver dari LangGraph. Dengan short-term memory, agent dapat mengingat percakapan selama sesi berlangsung, tetapi akan melupakan semuanya setelah sesi berakhir.

Contoh: Menambahkan Short-Term Memory dengan MemorySaver

```
#lesson6b.py

from langgraph.checkpoint.memory import MemorySaver
# Update node function to invoke all messages to give the LLM context
def call_llm(state: MessagesState):
    messages = state["messages"]
    response = model.invoke(messages)
    return {"messages": [response]}
# Initialize the checkpoint for short-term memory
checkpointer = MemorySaver()
# Compile the workflow with short-term memory
app_with_memory = workflow.compile(checkpointer=checkpointer)
def interact_with_agent_with_memory():
    # Use a thread ID to simulate a continuous session
    thread_id = "session_1"
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Ending the conversation.")
```

```

        break
    input_message = {
        "messages": [("human", user_input)]
    }
    # Invoke the graph with short-term memory enabled
    config = {"configurable": {"thread_id": thread_id}}
    for chunk in app_with_memory.stream(input_message, config=config,
stream_mode="values"):
        chunk["messages"][-1].pretty_print()
    # Start interacting with the memory-enabled agent
    interact_with_agent_with_memory()

```

Contoh Interaksi dengan Short-Term Memory:Anda: Siapa nama saya?Agent: Maaf, saya tidak tahu nama Anda.

Anda: Nama saya John.Agent: Senang bertemu Anda, John.

Anda: Siapa nama saya?Agent: Nama Anda John.

Di sini, agent sekarang mengingat bahwa nama Anda adalah "John" selama sesi berlangsung, menunjukkan manfaat dari short-term memory. Konteks sesi tetap ada selama sesi masih aktif.

6.3 Cara Kerja Short-Term Memory

Dengan menambahkan MemorySaver, kita dapat menyimpan state percakapan dalam sesi tersebut. Di balik layar, LangGraph menyimpan checkpoint-snapshot dari state percakapan-di setiap langkah. Checkpoint ini terkait dengan thread ID, yang mensimulasikan sebuah sesi. Selama thread ID tetap sama, agent akan mengingat interaksi-interaksi sebelumnya.

- Checkpointers: Menyimpan state percakapan.- Thread ID: Mensimulasikan sebuah sesi, memungkinkan agent mempertahankan konteks.- Short-Term Memory: Setelah sesi berakhir, memori akan dibuang.

6.4 Mendemonstrasikan Short-Term Memory Melalui Beberapa Pertanyaan

Mari kita lihat bagaimana agent berperilaku saat menjawab beberapa pertanyaan.

Contoh Interaksi:

Anda: Apa ibu kota Prancis?Agent: Ibu kota Prancis adalah Paris.

Anda: Dan siapa nama saya?Agent: Maaf, saya tidak tahu nama Anda.

Anda: Nama saya Sarah James.Agent: Senang bertemu Anda, Sarah.

Anda: Apa nama depan saya?Agent: Nama depan Anda adalah Sarah.

Anda: Apa nama belakang saya?Agent: Nama belakang Anda adalah James.

Dalam sesi ini:

- Agent dengan benar mengingat nama pengguna ("Sarah") setelah diberikan.- Agent juga menangani query lain (misalnya, ibu kota negara) tanpa kehilangan memori tentang nama pengguna selama sesi berlangsung.

6.5 Contoh Kode Lengkap: AI Agent dengan Short-Term Memory

Berikut adalah kode lengkap untuk membuat AI agent dengan short-term memory menggunakan MemorySaver dari LangGraph:

```
#lesson6c.py

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.checkpoint.memory import MemorySaver
from dotenv import load_dotenv
import os

# Load API key from .env file
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")

# Initialize the LLM (using OpenAI's GPT-4o-mini)
```

```

model = ChatOpenAI(model="gpt-4o-mini", api_key=api_key)
# Node function to handle the user query and call the LLM
def call_llm(state: MessagesState):
    messages = state["messages"]
    response = model.invoke(messages)
    return {"messages": [response]}
# Define the graph
workflow = StateGraph(MessagesState)
# Add the node to call the LLM
workflow.add_node("call_llm", call_llm)
# Define the edges (start -> LLM -> end)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
# Initialize the checkpointer for short-term memory
checkerpoint = MemorySaver()
# Compile the workflow with short-term memory
app_with_memory = workflow.compile(checkpointer=checkerpoint)
# Function to interact with the agent using short-term memory
def interact_with_agent_with_memory():
    # Use a thread ID to simulate a continuous session
    thread_id = "session_2"
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Ending the conversation.")
            break
        input_message = {
            "messages": [("human", user_input)]
        }
        # Invoke the graph with short-term memory enabled
        config = {"configurable": {"thread_id": thread_id}}
        for chunk in app_with_memory.stream(input_message, config=config,
stream_mode="values"):
            chunk["messages"][-1].pretty_print()
# Start interacting with the memory-enabled agent
interact_with_agent_with_memory()

```

6.6 Memori Lintas Beberapa Sesi

Pada bagian sebelumnya, kita telah membahas short-term memory, di mana agent mengingat interaksi dalam satu sesi. Namun, dalam banyak kasus, sangat berguna bagi agent untuk mempertahankan memori lintas beberapa sesi. Hal ini memungkinkan agent mengingat detail pengguna atau riwayat percakapan meskipun pengguna terhubung kembali atau memulai sesi baru. Pada bagian ini, kita akan membahas bagaimana LangGraph memungkinkan persistensi memori lintas sesi menggunakan memory store-nya.

6.6.1 Memori dan Checkpoint Lintas Sesi

LangGraph menggunakan checkpointer dan thread ID untuk menyimpan state percakapan di setiap interaksi (disebut super-step). Setiap interaksi atau sesi dapat dikaitkan dengan thread tertentu, memungkinkan agent mengingat state percakapan yang tepat ketika thread ID tersebut digunakan kembali. Hal ini berguna saat Anda memerlukan memori yang bertahan lintas sesi yang berbeda, yang sering terjadi pada aplikasi layanan pelanggan atau asisten pribadi.

Cara Kerjanya:

1. **Checkpointers** store the state of the conversation (e.g., user inputs, agent responses) at each interaction.
2. **Thread IDs** uniquely identify a session or conversation, allowing the agent to restore the conversation from a previous checkpoint when the same thread ID is provided.

Mari kita demonstrasikan bagaimana agent menyimpan dan memulihkan memori lintas sesi yang berbeda menggunakan thread ID.

6.6.2 Contoh: Persistensi In-Memory Lintas Sesi

Kita akan memodifikasi agent yang sudah ada untuk mempertahankan memori antar sesi yang berbeda dengan menggunakan thread ID untuk menghubungkan percakapan. Perlu diingat bahwa ketika program berakhir, data akan hilang dari memori, karena semua sesi berada in-memory yaitu berada di RAM dan tidak disimpan ke disk.

Contoh Kode: Menggunakan Checkpointer untuk Memori Lintas Sesi

```
#lesson6d.py  
  
from langchain_openai import ChatOpenAI
```

```

from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.checkpoint.memory import MemorySaver
from dotenv import load_dotenv
import os

# Load API key from .env file
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
# Initialize the LLM (using OpenAI's GPT-4o-mini)
model = ChatOpenAI(model="gpt-4o-mini", api_key=api_key)
# Node function to handle the user query and call the LLM
def call_llm(state: MessagesState):
    messages = state["messages"]
    response = model.invoke(messages)
    return {"messages": [response]}
# Define the graph
workflow = StateGraph(MessagesState)
# Add the node to call the LLM
workflow.add_node("call_llm", call_llm)
# Define the edges (start -> LLM -> end)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
# Initialize the MemorySaver checkpointer
checkerpoint = MemorySaver()
# Compile the workflow with short-term memory
app_with_memory = workflow.compile(checkpointer=checkerpoint)
# Function to simulate interacting with the agent across sessions
def interact_with_agent_across_sessions():
    while True:
        # Simulate a new session by allowing the user to input a thread ID
        thread_id = input("Enter thread ID (or 'new' for a new session): ")
        if thread_id.lower() in ["exit", "quit"]:
            print("Ending the conversation.")
            break
        if thread_id.lower() == "new":
            thread_id = f"session_{os.urandom(4).hex()}" # Generate a unique session
ID
        while True:

```

```

user_input = input("You: ")
if user_input.lower() in ["exit", "quit", "end session"]:
    print(f"Ending session {thread_id}.")
    break
input_message = {
    "messages": [("human", user_input)]
}
# Invoke the graph with the correct thread ID to maintain memory across
sessions

config = {"configurable": {"thread_id": thread_id}}
for chunk in app_with_memory.stream(input_message, config=config,
stream_mode="values"):
    chunk["messages"][-1].pretty_print()
# Start interacting with the memory-persistent agent
interact_with_agent_across_sessions()

```

Fitur Utama:

1. **Session-based Memory:** Users can reconnect to the agent using a thread ID (e.g., `session_1`), and the agent will recall the conversation context from the last interaction associated with that ID.
2. **New Sessions:** Users can create a new session by generating a new thread ID, allowing the agent to start fresh without any prior memory.

Contoh Interaksi Dengan Persistensi In-Memory:Anda:

Nama saya Sarah.Agent: Senang bertemu Anda, Sarah!

Ada yang bisa saya bantu hari ini?Anda: Akhiri sesi.

Mengakhiri sesi session_1.Masukkan thread ID
(atau 'new' untuk sesi baru): session_1Anda: Siapa
nama saya?Agent: Nama Anda Sarah.Masukkan
thread ID (atau 'new' untuk sesi baru):
session_2Anda: Siapa nama saya?

Agent: Maaf, tapi saya tidak memiliki akses ke informasi pribadi...

Dalam contoh ini, meskipun sesi telah berakhir dan dilanjutkan kembali nanti, agent tetap mengingat nama pengguna, berkat memori yang terkait dengan thread_id. Memori setiap sesi bersifat independen dan memori yang tersimpan di session_1 tidak tersedia di session_2.

6.6.3 Cara Kerja Persistensi In-Memory di LangGraph

Checkpoint dan Thread

- **Checkpoint:** LangGraph menyimpan state percakapan di setiap interaksi sebagai checkpoint. Setiap checkpoint berisi konteks percakapan, termasuk input pengguna dan respons agent.
- **Thread ID:** Setiap sesi dikaitkan dengan thread ID. Ketika thread ID yang sama digunakan kembali, LangGraph memulihkan konteks percakapan dari checkpoint terakhir yang terkait dengan thread tersebut.

Sebagai contoh:

- Jika Anda memulai sesi dengan thread_id = "session_1", LangGraph akan menyimpan semua interaksi dalam sesi ini.
- Jika Anda kembali lagi nanti dan memulai sesi baru menggunakan thread_id yang sama, agent akan memulihkan memori dari interaksi terakhir di session_1.

Menyelesaikan Memori Lintas Sesi

Memori dapat disimpan lintas sesi yang berbeda, sehingga memungkinkan untuk melanjutkan percakapan setelah sesi berakhir. Hal ini sangat berguna untuk workflow yang berjalan lama, aplikasi layanan pelanggan, atau skenario di mana pengguna berinteraksi dengan agent dari waktu ke waktu.

6.6.4 Memory Store In-Memory untuk Informasi Persisten

Terkadang, agent perlu mengingat informasi pengguna tertentu (misalnya, preferensi, data pribadi) di seluruh sesi, bahkan ketika thread baru dimulai. Untuk mencapai hal ini, LangGraph menyediakan Memory Store.

Memory Store memungkinkan agent menyimpan informasi yang dapat dibagikan lintas sesi (thread) yang berbeda untuk pengguna yang sama. Sebagai contoh, agent

dapat menyimpan preferensi makanan atau aktivitas favorit pengguna, yang akan tetap ada di semua percakapan mendatang, terlepas dari session ID.

Contoh: Menyimpan Informasi Lintas Sesi Menggunakan Memory Store

```
#lesson6e.py

from langgraph.store.memory import InMemoryStore
import uuid

# Initialize an in-memory store to store user information across sessions
in_memory_store = InMemoryStore()

# Function to store user information across sessions
def store_user_info(state: MessagesState, config, *, store=in_memory_store):
    user_id = config["configurable"]["user_id"]
    namespace = (user_id, "memories")
    # Create a memory based on the conversation
    memory_id = str(uuid.uuid4())
    memory = {"user_name": state["user_name"]}
    # Save the memory to the in-memory store
    store.put(namespace, memory_id, memory)
    return {"messages": ["User information saved."]}

# Function to retrieve stored user information
def retrieve_user_info(state: MessagesState, config, *, store=in_memory_store):
    user_id = config["configurable"]["user_id"]
    namespace = (user_id, "memories")
    # Retrieve the stored memories
    memories = store.search(namespace)
    if memories:
        info = f"Hello {memories[-1].value['user_name']}, welcome back!"
    else:
        info = "I don't have any information about you yet."
    return {"messages": [info]}
```

Contoh ini menunjukkan bagaimana menyimpan informasi khusus pengguna dalam memory store, memungkinkan agent mempertahankan informasi penting lintas sesi dan thread. Informasi yang tersimpan dapat diakses kembali nanti, terlepas dari session ID, dengan menggunakan user ID yang sama.

Kode lengkapnya:

```
#lesson6e.py complete example code

from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_openai import ChatOpenAI
import uuid
from dotenv import load_dotenv
import os

# Load API key from .env file
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")

# Initialize the LLM (using OpenAI's GPT-4o-mini)
model = ChatOpenAI(model="gpt-4o-mini", api_key=api_key)

# Initialize an in-memory store to store user information across sessions
in_memory_store = InMemoryStore()

# Function to store user information across sessions
def store_user_info(state: MessagesState, config, *, store=in_memory_store):
    user_id = config["configurable"]["user_id"]
    namespace = (user_id, "memories")
    # Store user's name in memory
    memory_id = str(uuid.uuid4())
    user_name = state["user_name"]
    memory = {"user_name": user_name}
    # Save the memory in the in-memory store
    store.put(namespace, memory_id, memory)
    return {"messages": ["User information saved."]}

# Function to retrieve stored user information
def retrieve_user_info(state: MessagesState, config, *, store=in_memory_store):
    user_id = config["configurable"]["user_id"]
    namespace = (user_id, "memories")
    # Retrieve stored user info
    memories = store.search(namespace)
    if memories:
        info = f"Hello {memories[-1].value['user_name']}, welcome back!"
    else:
```



```

        info = "I don't have any information about you yet."
        return {"messages": [info]}
# Function to handle storing or retrieving user info based on input
def call_model(state: MessagesState, config):
    last_message = state["messages"][-1].content.lower()
    if "remember my name" in last_message:
        # Store user's name in state and in memory
        user_name = last_message.split("remember my name is")[-1].strip()
        state["user_name"] = user_name
        return store_user_info(state, config)
    if "what's my name" in last_message or "what is my name" in last_message:
        # Retrieve the user's name from memory
        return retrieve_user_info(state, config)
    # Default LLM response for other inputs
    return {"messages": ["I didn't understand your request."]}
# Build the LangGraph workflow
workflow = StateGraph(MessagesState)
workflow.add_node("call_model", call_model)
workflow.add_edge(START, "call_model")
workflow.add_edge("call_model", END)
# Compile the graph with memory management
app_with_memory = workflow.compile(checkpointer=MemorySaver(),
store=in_memory_store)
# Simulate sessions
def simulate_sessions():
    # First session: store user's name
    config = {"configurable": {"thread_id": "session_1", "user_id": "user_123"}}
    input_message = {"messages": [{"type": "user", "content": "Remember my name is Alice"}]}
    for chunk in app_with_memory.stream(input_message, config=config,
stream_mode="values"):
        chunk["messages"][-1].pretty_print()
    # Second session: retrieve user's name
    config = {"configurable": {"thread_id": "session_2", "user_id": "user_123"}}
    input_message = {"messages": [{"type": "user", "content": "What's my name?"}]}
    for chunk in app_with_memory.stream(input_message, config=config,
stream_mode="values"):

```

```
chunk["messages"][-1].pretty_print()
# Run the session simulations
simulate_sessions()
```

6.6.5 Penjelasan Kode

a) **store_user_info:**

- i) This function is responsible for storing the user's name in the memory store. It generates a unique memory ID and saves the user's name in the namespace for the user_id.
- ii) The namespace is constructed using the user_id, ensuring that informasi disimpan dan diambil untuk pengguna tertentu.

retrieve_user_info:

- i) This function retrieves the user's name from the memory store. It searches the store for any saved information under the specific user_id namespace.
- ii) If the name is found, it returns the stored name; otherwise, it memberi tahu pengguna bahwa tidak ada informasi yang

tersedia.

c) **call_model:**

- i) This function is the central point that decides whether to store or retrieve user information based on the user's input.
- ii) If the user says "Remember my name is ...", the agent stores the name.
- iii) If the user asks "What's my name?", the agent retrieves the stored

nama.

d) **Penyimpanan**

Memori:

- i) The InMemoryStore is used to manage memory persistence, and the agent saves and retrieves the user's name across sessions using the store.
- ii) The MemorySaver checkpoint is responsible for managing short-term memory within the session, while the memory store handles information across different sessions.

Pada bagian ini, kami telah mendemonstrasikan cara mempertahankan memori lintas sesi menggunakan thread ID dan sistem checkpoint dari LangGraph. Kami juga membahas penggunaan memory store untuk menyimpan informasi khusus pengguna lintas thread dan sesi yang berbeda. Dengan menggabungkan checkpoint dan memory store, Anda dapat membangun AI agent yang tidak hanya mengingat percakapan

dalam satu sesi tetapi juga mengingat informasi penting tentang pengguna lintas berbagai interaksi, meningkatkan pengalaman pengguna secara keseluruhan.

6.7 Bagian Penjelasan: Detail Teknis tentang Checkpointer dan InMemoryStore

Pada bagian penjelasan ini, kita akan membahas secara mendalam dua komponen teknis penting yang mendukung memori di LangGraph: Checkpointer dan InMemoryStore. Memahami komponen-komponen ini akan membantu Anda membangun AI agent yang mampu mengelola memori lintas sesi dan workflow secara efisien.

6.7.1. Checkpointer

Checkpointer bertanggung jawab untuk menyimpan state dari sebuah graph di setiap super-step dalam workflow. Setiap checkpoint adalah snapshot dari state graph saat ini dan berisi informasi penting seperti konfigurasi, metadata, dan nilai state. Checkpointer memungkinkan LangGraph mempertahankan short-term memory dengan menyimpan state graph dan memanggilnya kembali saat diperlukan.

Properti Utama dari Checkpoint

Sebuah checkpoint direpresentasikan oleh objek StateSnapshot dan berisi properti-properti penting berikut:

- **Config:** Konfigurasi yang terkait dengan checkpoint ini, termasuk `thread_id` dan `checkpoint_id` opsional.
- **Metadata:** Metadata memberikan detail tentang sumber checkpoint dan kemajuan graph pada titik ini.
- **Values:** Values merepresentasikan state terkini dari channel dalam graph pada saat checkpoint diambil.
- **Next:** Sebuah tuple berisi nama-nama node yang akan dieksekusi selanjutnya dalam graph.
- **Tasks:** Sebuah tuple objek `PregelTask` yang berisi informasi tentang task berikutnya yang akan dieksekusi dalam graph. Ini juga menyimpan data error jika eksekusi gagal atau terganggu.

Setiap checkpoint merepresentasikan state graph pada super-step tertentu dan dapat diputar ulang (replayed) atau diperbarui.

Konsep Python yang Digunakan:

- **TypedDict:** Sebuah fitur dari modul typing Python yang digunakan untuk mendefinisikan struktur tertentu untuk dictionary. Di LangGraph, ini digunakan untuk mendefinisikan struktur state graph.
- **Reducers:** Fungsi yang menggabungkan atau mereduksi nilai state. Sebagai contoh, sebuah list reducer dapat menggabungkan item-item list lama dan baru.

Struktur Checkpoint (StateSnapshot)

```
#lesson6f.py

StateSnapshot()
    values={'foo': 'b', 'bar': ['a', 'b']}, # Current state values
    next=(), # Nodes to execute next (empty means graph execution is complete)
    config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id':
'12345'}}, # Config details
    metadata={'source': 'loop', 'writes': {'node_b': {'foo': 'b', 'bar': ['b']}}, 'step': 2}, #
Metadata info
    created_at='2024-10-02T18:22:31.590602+00:00', # Creation timestamp
    parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id':
'12344'}}, # Parent checkpoint
    tasks=() # Tasks to be executed next
)
```

Contoh: Menangkap Checkpoint

Mari kita buat graph sederhana dan amati bagaimana checkpoint disimpan pada berbagai tahap eksekusi. Kita akan mengilustrasikan sebuah graph dengan dua node dan menggunakan checkpointer MemorySaver untuk menangkap dan menampilkan checkpoint.

```
#lesson6f.py

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from typing import TypedDict
# Define the state schema using TypedDict
class State(TypedDict):
    foo: str
    bar: list[str]
# Define node functions to update the state
def node_a(state: State):
```

```

        return {"foo": "a", "bar": ["a"]}
def node_b(state: State):
    return {"foo": "b", "bar": ["b"]}
# Build the workflow graph
workflow = StateGraph(State)
workflow.add_node(node_a)
workflow.add_node(node_b)
workflow.add_edge(START, "node_a")
workflow.add_edge("node_a", "node_b")
workflow.add_edge("node_b", END)
# Initialize the checkpointer
checkerpoint = MemorySaver()
graph = workflow.compile(checkpointer=checkerpoint)
# Run the graph with thread_id and capture checkpoints
config = {"configurable": {"thread_id": "1"}}
graph.invoke({"foo": "", "bar": []}, config)

```

Output: Checkpoint yang Dihasilkan

1. **Initial State:** Before `node_a` executes.

```

#lesson6f.py

StateSnapshot(
    values={'foo': '', 'bar': []},
    next=('node_a',),
    metadata={'step': 0},
    tasks=(PregelTask(name='node_a',))
)

```

2. **After `node_a` Executes:** State is updated by `node_a`.

```

#lesson6f.py

StateSnapshot(
    values={'foo': 'a', 'bar': ['a']},
    next=('node_b',),
    metadata={'step': 1},
    tasks=(PregelTask(name='node_b',))
)

```

3. After **node_b** Executes: Final state of the graph.

```
#lesson6f.py

StateSnapshot(
    values={'foo': 'b', 'bar': ['a', 'b']},
    next=(),
    metadata={'step': 2},
    tasks=()
)
```

Setiap StateSnapshot menampilkan state (foo dan bar), node selanjutnya yang akan dieksekusi (next), dan metadata (step, tasks).

Mendapatkan State Saat Ini:

Anda dapat mengambil state terbaru dari graph dengan memanggil graph.get_state().

```
#lesson6f.py

config = {"configurable": {"thread_id": "1"}}
latest_state = graph.get_state(config)
print(latest_state.values) # {'foo': 'b', 'bar': ['a', 'b']}
```

Mendapatkan Riwayat State:

Anda dapat mengambil seluruh riwayat state menggunakan graph.get_state_history():

```
#lesson6f.py

config = {"configurable": {"thread_id": "1"}}
state_history = graph.get_state_history(config)
for snapshot in state_history:
    print(snapshot.values)
```

Output:[

```
{'foo': 'b', 'bar': ['a',
'b']},{'foo': 'a', 'bar':
['a']},{'foo': '', 'bar': []}]
```

Ini menunjukkan bagaimana state berkembang selama eksekusi.

6.7.2. InMemoryStore

InMemoryStore memungkinkan Anda mempertahankan informasi lintas thread dan sesi yang berbeda. Sementara checkpointer terikat pada sesi (thread) tertentu, memory store dapat menyimpan informasi pengguna, preferensi, dan riwayat antar sesi yang berbeda.

Konsep Utama:

- **Namespace:** Di **InMemoryStore**, memori disimpan menggunakan sebuah namespace yang biasanya mencakup `user_id` untuk mengidentifikasi memori secara unik.- **Put dan Search:** Ini adalah operasi utama. `put()` menyimpan sebuah memori, dan `search()` mengambilnya kembali.

Konsep Python yang Digunakan:

- **UUID:** Sebuah modul Python untuk menghasilkan ID memori unik untuk setiap memori.- **Dict:** Objek memori disimpan sebagai dictionary untuk memungkinkan penyimpanan pasangan key-value yang fleksibel.

Contoh: Menggunakan **InMemoryStore** untuk Menyimpan dan Mengambil Memori

```
#lesson6f.py

from langgraph.store.memory import InMemoryStore
import uuid

# Initialize the memory store
in_memory_store = InMemoryStore()

# Define a user namespace (user_id + "memories")
user_id = "1"
namespace_for_memory = (user_id, "memories")

# Store a memory (food preference)
memory_id = str(uuid.uuid4())
memory = {"food_preference": "I like pizza"}
in_memory_store.put(namespace_for_memory, memory_id, memory)

# Retrieve the stored memories
memories = in_memory_store.search(namespace_for_memory)
```

```
print(memories[-1].dict()) # {'value': {'food_preference': 'I like pizza'}, ...}
```

Output:

```
{  
  'value': {'food_preference': 'I like pizza'},  
  'key': '07e0caf4-1631-47b7-b15f-65515d4c1843',  
  'namespace': ['1', 'memories'],  
  'created_at': '2024-10-02T17:22:31.590602+00:00',  
  'updated_at': '2024-10-02T17:22:31.590605+00:00'  
}
```

Di sini, put() menambahkan memori ke store, dan search() mengambilnya kembali.

6.7.3. Menggunakan Checkpointer dan InMemoryStore Bersama-sama

Dalam aplikasi dunia nyata, Anda sering menggabungkan checkpointer untuk memori berbasis sesi dan InMemoryStore untuk memori persisten lintas sesi.

Contoh Lengkap: Graph dengan Checkpointer dan Memory Store

```
#lesson6f.py  
  
from langgraph.checkpoint.memory import MemorySaver  
from langgraph.store.memory import InMemoryStore  
# Initialize the checkpointer and memory store  
checkpointer = MemorySaver()  
in_memory_store = InMemoryStore()  
# Compile the graph with both  
graph = workflow.compile(checkpointer=checkpointer, store=in_memory_store)  
# Invoke the graph with thread_id and user_id  
config = {"configurable": {"thread_id": "session_1", "user_id": "1"}}  
graph.invoke({"foo": ""}, config)
```

Dalam pengaturan ini:

- **Checkpointer** menangani persistensi state dalam sesi.- **InMemoryStore** mengelola memori jangka panjang lintas sesi menggunakan user_id.

Mengakses Memori dalam Sebuah Node

```
#lesson6f.py  
  
def update_memory(state: MessagesState, config: RunnableConfig, *, store: BaseStore):
```



```

user_id = config["configurable"]["user_id"]
namespace = (user_id, "memories")
# Store a memory
memory_id = str(uuid.uuid4())
store.put(namespace, memory_id, {"favorite_food": "pizza"})
# Retrieve stored memories
memories = store.search(namespace)
return [{"messages": [{"f"I remember you like
{memories[-1].value['favorite_food']}"]}]}

```

Output: Saat Dipanggil dalam Dua Thread

Terpisah Thread 1: Anda: Apa makanan favorit

saya? Agent: Saya ingat Anda suka pizza.

Thread 2: Anda: Apa makanan favorit

saya? Agent: Saya ingat Anda suka
pizza.

Meskipun pengguna memulai sesi baru dengan thread yang berbeda, agent tetap mengingat memori tersebut karena disimpan di bawah user_id yang sama.

6.7.4. Library Checkpointer Utama dan Interface

LangGraph menyediakan beberapa library checkpointer, masing-masing dioptimalkan untuk kasus penggunaan yang berbeda:

- **MemorySaver: Checkpointer in-memory untuk pengujian dan eksperimen.**
- **SQLiteSaver: Menyimpan checkpoint di SQLite, ideal untuk produksi skala kecil.**
- **PostgresSaver: Checkpointer berbasis Postgres yang digunakan pada lingkungan produksi skala besar.**

Semua checkpointer mengimplementasikan metode-metode utama berikut:

- **put(): Menyimpan checkpoint dengan config dan metadata.**
- **get_state(): Mengambil checkpoint terbaru untuk thread_id tertentu.**
- **get_state_history(): Mengambil semua checkpoint untuk thread_id tertentu.**

Metode-metode ini memungkinkan manajemen state yang fleksibel dan mendukung replay atau update state pada titik mana pun dalam eksekusi graph.

Bab 6: Kuis

Uji pengetahuan Anda tentang Bab 6 dan pastikan Anda memahami konsep-konsep kunci tentang memori dalam AI agent, checkpoint, dan memory store. Setiap pertanyaan membahas bagian fundamental dari konsep-konsep yang baru saja Anda pelajari.

Pertanyaan Pilihan Ganda:

1. What does short-term memory in an AI agent refer to?

- a) The ability to remember information across multiple sessions
- b) The ability to remember information only during a single session
- c) The ability to recall data from memory without user input
- d) The ability to store all user data permanently

Answer: b) The ability to remember information only during a single session

2. Which of the following tools is used to implement short-term memory in LangGraph?

- a) InMemoryStore
- b) SQLiteSaver
- c) MemorySaver
- d) MemoryCache

Answer: c) MemorySaver

3. What is a checkpoint in the context of LangGraph?

- a) A fixed point where the user input is stored
- b) A snapshot of the graph state saved at each step of the workflow
- c) A method for retrieving past sessions
- d) A storage method for long-term user preferences

Answer: b) A snapshot of the graph state saved at each step of the workflow

4. What does the `thread_id` help with in LangGraph workflows?

- a) Creating new nodes in a graph
- b) Storing data across multiple sessions
- c) Replaying a specific session's memory
- d) Uniquely identifying a session and associating it with memory

Answer: d) Uniquely identifying a session and associating it with memory

5. Which method is used to retrieve all previous checkpoints in a session's execution?

- a) `graph.get_all_states()`
- b) `graph.invoke(config)`
- c) `graph.get_state_history(config)`
- d) `graph.search(config)`

Answer: c) `graph.get_state_history(config)`

6. Which Python feature is used to structure the state in LangGraph graphs?

- a) NamedTuple
- b) List
- c) TypedDict
- d) Dictionary

Answer: c) TypedDict

7. What is the purpose of an InMemoryStore in LangGraph?

- a) To store state during a single session
- b) To provide memory persistence across multiple sessions and threads
- c) To speed up the LLM's response time
- d) To store temporary files during graph execution

Answer: b) To provide memory persistence across multiple sessions and threads

Tugas Praktik: Membangun AI Agent yang Berfungsi Penuh

Dalam tugas praktik ini, Anda akan membuat sebuah AI agent menggunakan LangGraph yang menggabungkan semua fitur yang telah Anda pelajari dari Bab 1 hingga 6, termasuk:

- Manajemen state- Short-term memory- Long-term memory lintas sesi

Tujuan:

1. Basic Agent Behavior:

- Create an AI agent that can handle user queries and responds with answers.
- Use a Language Model (LLM) such as OpenAI's GPT models.

2. State Management:

- Design a workflow with nodes that manage and update the conversation state.

3. Short-Term Memory:

- Implement short-term memory using **MemorySaver** to keep the context of the conversation within a session.
- The agent should remember user input and context during the session.

4. Long-Term Memory:

- Use **InMemoryStore** to store persistent user information (e.g., user preferences, name) across sessions.
- Allow the agent to recall details from previous sessions using the `user_id` and `thread_id`.

5. Custom Node Behavior:

- Create custom nodes to handle specific actions like storing user information and recalling it when necessary.

6. Checkpoints and Replays:

- Make sure the agent can replay sessions by using checkpoints and thread identifiers.

Langkah-langkah untuk Menyelesaikan Tugas:

1. Menyiapkan Environment Anda:

Pastikan Anda telah menyiapkan LangGraph, LangChain, dan model LLM yang diperlukan (seperti GPT-4o-mini dari OpenAI). Install dependency jika diperlukan:

```
terminal  
pip install langgraph langchain_openai python-dotenv
```

Langkah 2: Mengimplementasikan Short-Term Memory

Short-term memory akan menggunakan MemorySaver untuk mengingat interaksi dalam satu sesi. Anda perlu menggunakan thread_id untuk mengidentifikasi sesi dan menangkap state percakapan.

```
#lesson6f.py  
  
from langgraph.graph import StateGraph, MessagesState, START, END  
from langgraph.checkpoint.memory import MemorySaver  
from langchain_openai import ChatOpenAI  
from dotenv import load_dotenv  
import os  
  
# Load environment variables  
load_dotenv()  
  
# Initialize API key  
api_key = os.getenv("OPENAI_API_KEY")  
  
# Initialize OpenAI model  
model = ChatOpenAI(model="gpt-4o-mini", api_key=api_key)  
  
# Define the graph workflow  
workflow = StateGraph(MessagesState)
```

```

# Node to process user query and return the LLM response
def call_llm(state: MessagesState):
    messages = state["messages"]
    response = model.invoke(messages)
    return {"messages": [response]}

# Add nodes and define edges in the graph
workflow.add_node("call_llm", call_llm)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)

# Initialize the MemorySaver for short-term memory
checkpointer = MemorySaver()
app_with_memory = workflow.compile(checkpointer=checkpointer)

# Simulate conversation with short-term memory
def interact_with_agent():
    thread_id = "session_1"
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            break
        input_message = {
            "messages": [("human", user_input)]
        }
        config = {"configurable": {"thread_id": thread_id}}
        for chunk in app_with_memory.stream(input_message, config=config,
stream_mode="values"):
            chunk["messages"][-1].pretty_print()

interact_with_agent()

```

Dalam kode ini:

- **Node call_llm** mengirim query pengguna ke model OpenAI dan menerima respons.- **MemorySaver** menyimpan state percakapan, memungkinkan agent mengingat interaksi dalam sesi tersebut.

Langkah 3: Mengimplementasikan Long-Term Memory

Untuk mempertahankan informasi lintas sesi, kita akan menggunakan InMemoryStore. Ini memungkinkan agent mengingat informasi pengguna pada sesi-sesi mendatang.

```
#lesson6f.py

from langgraph.store.memory import InMemoryStore
import uuid

# Initialize the in-memory store
in_memory_store = InMemoryStore()

# Function to store user information
def store_user_info(state: MessagesState, config, *, store=in_memory_store):
    user_id = config["configurable"]["user_id"]
    namespace = (user_id, "memories")
    memory_id = str(uuid.uuid4())
    memory = {"user_name": state["user_name"]}
    store.put(namespace, memory_id, memory)
    return {"messages": ["User information saved."]}

# Function to retrieve stored user information
def retrieve_user_info(state: MessagesState, config, *, store=in_memory_store):
    user_id = config["configurable"]["user_id"]
    namespace = (user_id, "memories")
    memories = store.search(namespace)
    if memories:
        info = f"Hello {memories[-1].value['user_name']}, welcome back!"
    else:
        info = "I don't have any information about you yet."
    return {"messages": [info]}
```

Dalam kode ini:

- **InMemoryStore** digunakan untuk menyimpan dan mengambil informasi khusus pengguna lintas sesi menggunakan `user_id`.

Langkah 4: Menggabungkan Short-Term dan Long-Term Memory

Sekarang, kita akan menggabungkan kedua pendekatan memori untuk menciptakan agent yang berfungsi penuh yang mampu melakukan short-term dan long-term memory.

```
terminal

from langgraph.graph import StateGraph, MessagesState, START, END
```

```

from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import MemorySaver
import uuid

# Initialize the in-memory store
in_memory_store = InMemoryStore()

# Function to store user information
def store_user_info(state: MessagesState, config, *, store=in_memory_store):
    user_id = config["configurable"]["user_id"]
    namespace = (user_id, "memories")
    memory_id = str(uuid.uuid4())
    memory = {"user_name": state["user_name"]}
    store.put(namespace, memory_id, memory)
    return {"messages": ["User information saved."]}

# Function to retrieve stored user information
def retrieve_user_info(state: MessagesState, config, *, store=in_memory_store):
    user_id = config["configurable"]["user_id"]
    namespace = (user_id, "memories")
    memories = store.search(namespace)
    if memories:
        info = f"Hello {memories[-1].value['user_name']}, welcome back!"
    else:
        info = "I don't have any information about you yet."
    return {"messages": [info]}

# Function to manage user input and memory storage/retrieval
def call_model(state: MessagesState, config):
    last_message = state["messages"][-1].content.lower()

    # Store the user's name
    if "remember my name" in last_message:
        user_name = last_message.split("remember my name is")[-1].strip()
        state["user_name"] = user_name
        return store_user_info(state, config)

    # Retrieve the user's name
    if "what's my name" in last_message or "what is my name" in last_message:
        return retrieve_user_info(state, config)

    # Default LLM response for other inputs
    return {"messages": ["I didn't understand your request."]}

# Define the graph workflow

```



```

workflow = StateGraph(MessagesState)
workflow.add_node("call_model", call_model)
workflow.add_edge(START, "call_model")
workflow.add_edge("call_model", END)
# Compile the graph with both checkpointer and memory store
app_with_memory = workflow.compile(checkpointer=MemorySaver(),
store=in_memory_store)
def simulate_sessions():
    # First session: store user's name
    config = {"configurable": {"thread_id": "session_1", "user_id": "user_123"}}
    input_message = {"messages": [("human", "Remember my name is Alice")]}
    for chunk in app_with_memory.stream(input_message, config=config,
stream_mode="values"):
        chunk["messages"][-1].pretty_print()
    # Second session: retrieve user's name
    config = {"configurable": {"thread_id": "session_2", "user_id": "user_123"}}
    input_message = {"messages": [("human", "What's my name?")]}
    for chunk in app_with_memory.stream(input_message, config=config,
stream_mode="values"):
        chunk["messages"][-1].pretty_print()
simulate_sessions()

```

Expected Output:

Session 1: You: Ingat nama saya Alice
Agent: Informasi Anda telah disimpan!

Session 2: You: Siapa nama saya?
Agent: Halo Alice, selamat datang kembali!

Tugas ini menunjukkan bagaimana mengimplementasikan baik short-term memory (dalam satu session) maupun long-term memory (lintas session) menggunakan MemorySaver dan InMemoryStore milik LangGraph.

Chapter 7

Advanced Routing and Customization of AI Agents

Pada bab ini, kita akan memperluas konsep dasar yang telah diperkenalkan pada bab-bab sebelumnya, dengan fokus pada routing tingkat lanjut, custom workflow, structured output, dan integrasi dengan API umum. Konsep-konsep ini akan memungkinkan Anda membangun AI agent yang lebih dinamis, adaptif, dan disesuaikan menggunakan LangGraph. Kita juga akan membahas conditional logic, yang memungkinkan Anda mengontrol bagaimana agent memproses query pengguna dan respons dari API eksternal. Pada akhir bab ini, Anda akan mampu mengembangkan AI agent yang secara cerdas melakukan routing input pengguna ke workflow yang tepat, menangani interaksi yang kompleks, dan bahkan melakukan streaming data secara real-time kepada pengguna.

7.1 Pengantar Routing pada AI Agent

Routing pada AI agent mengacu pada kemampuan untuk mengarahkan input pengguna melalui jalur, workflow, atau task yang berbeda berdasarkan kriteria tertentu. Ini adalah komponen krusial untuk menangani percakapan yang kompleks, yang memungkinkan agent untuk:

- Memutuskan task atau service mana yang akan dipanggil berdasarkan input pengguna.
- Menangani berbagai jenis query dalam satu session.
- Mengintegrasikan service eksternal (misalnya, weather, search) dan melakukan routing input dengan tepat.

Mengapa Routing Itu Penting?

AI agent sering kali perlu menangani berbagai macam permintaan pengguna, mulai dari pengambilan informasi sederhana hingga operasi yang lebih kompleks seperti melakukan perhitungan atau melakukan query ke API eksternal. Tanpa routing, agent akan kesulitan mengelola berbagai jenis task dan sering kali akan memberikan respons yang salah atau tidak lengkap. Routing memastikan agent memproses setiap query melalui workflow atau task yang tepat.

Contoh Use Case

Bayangkan sebuah virtual assistant yang dapat menangani baik query cuaca maupun rekomendasi restoran. Masing-masing task ini memerlukan API yang berbeda, logika yang berbeda, dan kemungkinan output yang berbeda pula. Agent perlu memutuskan, berdasarkan input pengguna, workflow mana yang harus diikuti.

Skenario 1:

- **User Query:** "Bagaimana cuaca di New York?"? **Workflow:** Agent melakukan routing ini ke weather API.

Skenario 2:

- **User Query:** "Rekomendasikan restoran yang bagus di New York."? **Workflow:** Agent melakukan routing ini ke API rekomendasi restoran.

Tanpa routing, agent tidak dapat membedakan secara cerdas antara workflow-workflow ini dan akan memperlakukan semua input dengan cara yang sama, yang menyebabkan respons yang tidak sesuai atau salah.

7.2 tools_condition Bawaan untuk Conditional Routing

tools_condition adalah tool bawaan (pre-built) LangGraph untuk menangani conditional routing, yang sangat berguna ketika AI agent Anda memiliki tool (seperti API atau utility function) dan Anda ingin memutuskan secara kondisional apakah akan memanggil sebuah tool atau merespons secara langsung.

Contoh: Conditional Routing Antara Tool dan Respons Langsung

Berikut adalah contoh sederhana di mana kita menggunakan tools_condition untuk melakukan routing antara memanggil sebuah tool (dalam kasus ini, sebuah fungsi perkalian dasar) atau

merespons langsung kepada pengguna.

```
#lesson7a.py

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode, tools_condition
# Initialize the LLM and define a basic multiplication tool
llm = ChatOpenAI(model="gpt-4o-mini")
# Define a multiplication tool
def multiply(a: int, b: int) -> int:
    """
    Multiplies two numbers.
    """
    return a * b
# Bind the LLM with the tool
llm_with_tools = llm.bind_tools([multiply])
# Node that calls the LLM with tools bound
def tool_calling_llm(state: MessagesState):
    """
    Node that calls the LLM with tools bound.
    """
    return {"messages": [llm_with_tools.invoke(state["messages"])]}
# Build the workflow
builder = StateGraph(MessagesState)
builder.add_node("tool_calling_llm", tool_calling_llm)
builder.add_node("tools", ToolNode([multiply])) # Tool node for handling tool
invocation
# Define edges to connect the nodes
builder.add_edge(START, "tool_calling_llm")
# Add the conditional edge based on tool usage
builder.add_conditional_edges(
    "tool_calling_llm",
    tools_condition, # Condition to decide if the assistant should call the tool
)
builder.add_edge("tools", END) # If tool is called, terminate after tool execution
# Compile the graph
```

```
graph = builder.compile()
# Simulate invoking the graph
def simulate():
    user_input = {"messages": [("human", "Can you multiply 3 by 5?")]}
    result = graph.invoke(user_input)
    return result["messages"][-1].pretty_print()
print(simulate())
```

Poin Penting:

- **tools_condition** : Tool condition bawaan ini mengevaluasi apakah pesan terbaru dari assistant melibatkan pemanggilan sebuah tool. Jika ya, ia melakukan routing ke node tool; jika tidak, percakapan dapat berakhir atau di-routing ke tempat lain.- **ToolNode** : Sebuah node bawaan (pre-built) yang dirancang untuk memanggil tool tertentu, seperti fungsi perkalian dalam kasus ini.

7.3 Contoh Custom Conditional Routing: Node Weather dan Calculator

Pada contoh ini, kita akan membangun AI agent dasar yang:

- Melakukan routing query terkait cuaca ke sebuah node yang mensimulasikan pengambilan informasi cuaca.- Melakukan routing query perhitungan ke node calculator yang menangani operasi aritmatika sederhana.- Menangani query yang tidak dikenali dengan respons default.

Langkah 1: Mendefinisikan Node

Kita akan mulai dengan mendefinisikan tiga node:

1. **weather_node** : Handles weather-related queries.
2. **calculator_node** : Handles basic arithmetic operations like addition.
3. **default_node** : Provides a fallback for any unrecognized user input.

```
#lesson7b.py
from langgraph.graph import StateGraph, MessagesState, START, END
# Define a node to simulate a weather response
```

```

def weather_node(state: MessagesState):
    return {"messages": ["It's sunny with a temperature of 25°C."]}
# Define a node to handle basic arithmetic calculations
def calculator_node(state: MessagesState):
    user_query = state["messages"][-1].content.lower()
    if "add" in user_query:
        numbers = [int(s) for s in user_query.split() if s.isdigit()]
        result = sum(numbers)
        return {"messages": [f"The result of addition is {result}."]}
    return {"messages": ["I can only perform addition for now."]}
# Define a default node to handle unrecognized inputs
def default_node(state: MessagesState):
    return {"messages": ["Sorry, I don't understand that request."]}

```

Langkah 2: Membuat Routing Function

Selanjutnya, kita perlu membuat sebuah routing function yang memutuskan node mana yang akan menerima query pengguna berdasarkan konten query tersebut. Fungsi ini akan bertanggung jawab untuk memeriksa apakah query tersebut tentang cuaca, aritmatika, atau bukan keduanya, lalu melakukan routing sesuai dengan itu.

```

#lesson7b.py
# Custom routing function to decide which node to route to
def routing_function(state: MessagesState):
    last_message = state["messages"][-1].content.lower()
    if "weather" in last_message:
        return "weather_node" # Route to weather node
    elif "add" in last_message or "calculate" in last_message:
        return "calculator_node" # Route to calculator node
    return "default_node" # Route to default node for unrecognized inputs

```

routing_function memeriksa pesan terakhir dari pengguna dan:

- Melakukan routing ke node weather jika pesan mengandung kata "weather".
- Melakukan routing ke node calculator jika pesan mengandung kata terkait aritmatika seperti "add" atau "calculate".

- Melakukan routing ke node default jika input tidak sesuai dengan kondisi yang diharapkan.

Langkah 3: Membangun Workflow Graph

Sekarang, kita akan membangun workflow LangGraph menggunakan node yang telah kita buat. Kita akan menggunakan conditional edge untuk melakukan routing secara dinamis berdasarkan hasil dari `routing_function` .

```
#lesson7b.py continued

# Build the workflow graph
builder = StateGraph(MessagesState)
builder.add_node("weather_node", weather_node)
builder.add_node("calculator_node", calculator_node)
builder.add_node("default_node", default_node)
builder.add_node("routing_function", routing_function)

# Set up the edges for routing
builder.add_conditional_edges(START, routing_function) # Route based on the routing
function
builder.add_edge("weather_node", END) # Route to end after weather node
builder.add_edge("calculator_node", END) # Route to end after calculator node
builder.add_edge("default_node", END) # Route to end after default node

# Compile the graph
app = builder.compile()
```

Pada workflow ini:

- Kita mulai dengan node **START**, yang melakukan routing ke `routing_function` .- Conditional edge menentukan node mana yang akan dituju selanjutnya berdasarkan output dari `routing_function` .- Setiap node (`weather_node` , `calculator_node` , `default_node`) terhubung ke node **END** setelah eksekusi.

Langkah 4: Mensimulasikan Interaksi dengan Agent

Sekarang, kita akan membuat fungsi sederhana untuk mensimulasikan interaksi dengan agent. Fungsi ini memungkinkan kita untuk memasukkan input dan melihat bagaimana agent melakukan routing terhadap

query.

```
#lesson7b.py continued

# Simulate interaction with the agent
def simulate_interaction():
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Exiting...")
            break
        input_message = {"messages": [("human", user_input)]}
        for result in app.stream(input_message, stream_mode="values"):
            result["messages"][-1].pretty_print()
# Start interacting with the agent
simulate_interaction()
```

Fungsi interaktif ini:

- Menerima input pengguna.- Mengirimkan input tersebut ke graph untuk diproses.- Menampilkan respons dari node yang sesuai.

Kode lengkapnya adalah sebagai berikut:

```
#lesson7b.py complete code example

from langgraph.graph import StateGraph, MessagesState, START, END
# Define a node to simulate a weather response
def weather_node(state: MessagesState):
    return {"messages": ["It's sunny with a temperature of 25°C."]}
# Define a node to handle basic arithmetic calculations
def calculator_node(state: MessagesState):
    user_query = state["messages"][-1].content.lower()
    if "add" in user_query:
        numbers = [int(s) for s in user_query.split() if s.isdigit()]
        result = sum(numbers)
        return {"messages": [f"The result of addition is {result}."]}
    return {"messages": ["I can only perform addition for now."]}
# Define a default node to handle unrecognized inputs
def default_node(state: MessagesState):
```



```

    return {"messages": ["Sorry, I don't understand that request."]}
# Custom routing function to decide which node to route to
def routing_function(state: MessagesState):
    last_message = state["messages"][-1].content.lower()
    if "weather" in last_message:
        return "weather_node" # Route to weather node
    elif "add" in last_message or "calculate" in last_message:
        return "calculator_node" # Route to calculator node
    return "default_node" # Route to default node for unrecognized inputs
# Build the workflow graph
builder = StateGraph(MessagesState)
builder.add_node("weather_node", weather_node)
builder.add_node("calculator_node", calculator_node)
builder.add_node("default_node", default_node)
builder.add_node("routing_function", routing_function)
# Set up the edges for routing
builder.add_conditional_edges(START, routing_function)
builder.add_edge("weather_node", END)
builder.add_edge("calculator_node", END)
builder.add_edge("default_node", END) # Edge to end after default node
# Compile the graph
app = builder.compile()
# Simulate interaction with the agent
def simulate_interaction():
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Exiting...")
            break
        input_message = {"messages": [("human", user_input)]}
        for result in app.stream(input_message, stream_mode="values"):
            result["messages"][-1].pretty_print()
# Start interacting with the agent
simulate_interaction()

```

Contoh Interaksi

Mari kita jalankan kode ini dan jelajahi beberapa kemungkinan interaksi dengan agent.

1. Weather Query

You: What's the weather today?

Agent: It's sunny with a temperature of 25°C.

2. Calculation Query

You: Can you add 10 and 20?

Agent: The result of the addition is 30.

3. Unrecognized Input

You: Hello, how are you?

Agent: Sorry, I don't understand that request.

Poin-Poin Penting

Pada bagian ini, kita telah menunjukkan:

1. **Conditional Routing:** How to dynamically route between nodes based on the content of user input using custom routing functions.
2. **Custom Nodes:** How to create nodes that handle specific tasks, such as returning weather information or performing arithmetic operations.
3. **Default Responses:** How to ensure that your AI agent gracefully handles unrecognized input by routing it to a fallback node.

Conditional routing memungkinkan AI agent Anda menangani berbagai jenis query dengan mudah dan memastikan bahwa perilaku agent tetap fleksibel dan responsif.

7.4 Streaming pada LangGraph

Streaming pada LangGraph adalah fitur yang memungkinkan pembaruan secara real-time dari state graph atau output node selama workflow sedang dieksekusi. Ini sangat berguna ketika menangani dataset yang besar, proses yang berjalan lama (long-running), atau ketika feedback yang berkelanjutan diperlukan, seperti pada chatbot, pemrosesan data langsung (live), atau sistem monitoring real-time.

Pada LangGraph, Anda dapat melakukan streaming untuk hal berikut:

1. **Full State Streaming:** Streams the entire state of the graph after each node execution.
2. **Updates Streaming:** Streams only the updates to the state after each node execution.

3. **LLM Token Streaming:** Streams the tokens produced by the Language Learning Model (LLM) during its generation process.

Pada bagian ini, kita akan membahas setiap mode streaming ini secara rinci, dengan menyertakan contoh untuk menunjukkan cara mengimplementasikannya pada AI agent Anda.

7.4.1 Full State Streaming

Pada full state streaming, seluruh state dari graph dikirim kembali setelah setiap eksekusi node. Ini berguna ketika Anda ingin memonitor semua perubahan yang terjadi pada graph setelah setiap eksekusi node.

Contoh: Full State Streaming

Mari kita lihat contoh dasar yang menunjukkan cara melakukan streaming full state dari sebuah graph. Kita akan mendefinisikan sebuah graph dengan weather tool sederhana dan melakukan streaming seluruh state setelah setiap node dieksekusi.

```
#lesson7c.py

import operator
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_openai import ChatOpenAI
from typing import Annotated
from typing_extensions import TypedDict

# Define the state schema
class State(TypedDict):
    messages: Annotated[list, operator.add]

# Define a node to handle weather queries
def weather_node(state: State):
    return {"messages": ["The weather is sunny and 25°C."]}

# Define a node to handle calculator queries
def calculator_node(state: State):
    return {"messages": ["The result of 2 + 2 is 4."]}

# Define the workflow graph
workflow = StateGraph(State)
workflow.add_node("weather_node", weather_node)
workflow.add_node("calculator_node", calculator_node)

# Set the edges for the graph
```

```

workflow.add_edge(START, "weather_node")
workflow.add_edge("weather_node", "calculator_node")
workflow.add_edge("calculator_node", END)
# Compile the workflow
app = workflow.compile()
# Simulate interaction and stream the full state
def simulate_interaction():
    input_message = {"messages": [("human", "Tell me the weather")]}
    # Stream the full state of the graph
    for result in app.stream(input_message, stream_mode="values"):
        print(result) # Print the full state after each node
simulate_interaction()

```

Pada contoh ini, setelah setiap node (weather_node dan calculator_node) dieksekusi, seluruh state dari graph di-streaming kembali. Full state ini mencakup semua pesan yang telah dihasilkan sejauh ini.

Output:

```

{'messages': [('human', 'Tell me the weather')]}
{'messages': [('human', 'Tell me the weather'), 'The weather is sunny and 25°C.']}
{'messages': [('human', 'Tell me the weather'), 'The weather is sunny and 25°C.', 'The result of 2 + 2 is 4.']}

```

7.4.2 Updates Streaming

Berbeda dengan full state streaming, updates streaming hanya mengirimkan kembali perubahan yang terjadi pada graph setelah setiap eksekusi node. Ini efisien ketika Anda hanya perlu melihat apa yang berubah, bukan seluruh state.

Contoh: Updates Streaming

```

#lesson7d.py

import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_openai import ChatOpenAI
# Define the state schema
class State(TypedDict):
    messages: Annotated[list, operator.add]

```

```

# Define the same nodes as before
def weather_node(state: State):
    return {"messages": ["It's 25°C and sunny."]}
def calculator_node(state: State):
    return {"messages": ["2 + 2 equals 4."]}
# Define the graph
workflow = StateGraph(State)
workflow.add_node("weather_node", weather_node)
workflow.add_node("calculator_node", calculator_node)
workflow.add_edge(START, "weather_node")
workflow.add_edge("weather_node", "calculator_node")
workflow.add_edge("calculator_node", END)
app = workflow.compile()
def simulate_interaction():
    input_message = {"messages": [("human", "Tell me the weather")]}
    # Stream updates after each node
    for result in app.stream(input_message, stream_mode="updates"):
        print(result)
simulate_interaction()

```

Di sini, hanya update setelah setiap node yang akan di-streaming kembali, sehingga lebih efisien pada skenario di mana Anda tidak memerlukan seluruh state.

Output:

```

{'weather_node': {'messages': ["It's 25°C and sunny."]} }
{'calculator_node': {'messages': ['2 + 2 equals 4.']} }

```

7.4.3 LLM Token Streaming

Melakukan streaming token LLM sangat berguna ketika menggunakan language model untuk menghasilkan respons yang panjang. Alih-alih menunggu seluruh respons selesai dihasilkan, Anda dapat melakukan streaming token saat token tersebut dihasilkan.

Contoh: Streaming Token LLM

Pada contoh ini, kita akan menggunakan model OpenAI GPT-4 untuk menunjukkan token streaming saat model tersebut menghasilkan respons.

```
#lesson7e.py
```

```

import operator
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_openai import ChatOpenAI
from typing import Annotated
from typing_extensions import TypedDict
import asyncio
from langgraph.graph.message import add_messages
from langchain_core.messages import AIMessageChunk, HumanMessage
# Define the state schema
class State(TypedDict):
    messages: Annotated[list, add_messages]
# Initialize the LLM
model = ChatOpenAI(model="gpt-4o-mini")
# Define a node to handle LLM queries
async def call_llm(state: State):
    messages = state["messages"]
    response = await model.ainvoke(messages)
    return {"messages": [response]}
# Define the graph
workflow = StateGraph(State)
workflow.add_node("call_llm", call_llm)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
app = workflow.compile()
# Simulate interaction and stream tokens
async def simulate_interaction():
    input_message = {"messages": [("human", "Tell me a very long joke")]}
    first = True
    # Stream LLM tokens
    async for msg, metadata in app.astream(input_message,
stream_mode="messages"):
        if msg.content and not isinstance(msg, HumanMessage):
            print(msg.content, end="|", flush=True)
        if isinstance(msg, AIMessageChunk):
            if first:
                gathered = msg
                first = False

```

```

else:
    gathered = gathered + msg
    if msg.tool_call_chunks:
        print(gathered.tool_calls)
asyncio.run(simulate_interaction())

```

Output (Token Streaming):

Tentu! Ini lah lelucon yang panjang, dan ringan untukmu:

[Pada] suatu ketika di sebuah desa kecil, hiduplah seorang pria bernama Bob. Bob dikenal di seluruh desa karena kemampuannya yang luar biasa dalam bercerita lelucon. Ia bisa membuat bahkan orang yang paling pemarah pun tertawa. Suatu hari, desa itu memutuskan untuk mengadakan sebuah festival dan mereka ingin Bob menjadi penghibur utama.

[Dengan] penuh semangat, Bob mulai merencanakan serangkaian lelucon yang akan membuat festival itu tak terlupakan. Ia memutuskan untuk menceritakan sebuah kisah, bukan sekadar lelucon satu kalimat. Ia berpikir, "Jika aku bisa merajut sebuah cerita dengan lelucon-leluconku, ini akan jadi hit!"

Pada contoh ini, Anda dapat mengamati token yang di-streaming dari LLM, yang memungkinkan pengguna melihat respons saat sedang dihasilkan.

7.4.4 Menggabungkan Mode Streaming

Anda dapat menggabungkan mode streaming yang berbeda tergantung pada use case Anda. Misalnya, Anda mungkin ingin melakukan streaming updates sekaligus token LLM ketika menggunakan sebuah language model.

Contoh: Menggabungkan Updates dan Token Streaming

```

#lesson7f.py

import operator
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_openai import ChatOpenAI
from typing import Annotated
from typing_extensions import TypedDict
import asyncio
from langgraph.graph.message import add_messages
from langchain_core.messages import AIMessageChunk, HumanMessage

# Define the state schema
class State(TypedDict):
    messages: Annotated[list, add_messages]

```

```

# Initialize the LLM with the correct model name and streaming enabled
model = ChatOpenAI(
    model="gpt-4",
    streaming=True
)

# Define a node to handle LLM queries
async def call_llm(state: State):
    messages = state["messages"]
    response = await model.ainvoke(messages)
    return {"messages": [response]}

# Define the graph
workflow = StateGraph(State)
workflow.add_node("call_llm", call_llm)
workflow.add_edge(START, "call_llm")
workflow.add_edge("call_llm", END)
app = workflow.compile()

# Simulate interaction and stream tokens
async def simulate_interaction():
    input_message = {"messages": [HumanMessage(content="Tell me a very long joke")]}

    # Stream LLM tokens
    async for msg, metadata in app.astream(input_message, stream_mode=
["messages", "updates"]):
        # Check if we have metadata for call_llm
        if isinstance(metadata, dict) and 'call_llm' in metadata:
            # Extract the message from call_llm metadata
            ai_message = metadata['call_llm']['messages'][0]
            if ai_message.content:
                print(ai_message.content, end="|", flush=True)

if __name__ == "__main__":
    asyncio.run(simulate_interaction())

```

Ringkasan Mode Streaming:

- **Full State Streaming:** Melakukan streaming seluruh state setelah setiap eksekusi node (`stream_mode="values"`).

- **Updates Streaming:** Hanya melakukan streaming update setelah setiap eksekusi node (`stream_mode="updates"`).- **LLM Token Streaming:** Melakukan streaming token yang dihasilkan oleh language model (`stream_mode="messages"` dengan model yang mendukung token streaming).- Anda dapat menggabungkan mode streaming menggunakan `stream_mode=["messages","updates"]`

Dengan menggunakan mode streaming ini, Anda dapat membangun aplikasi yang lebih responsif dan real-time yang memberikan feedback berkelanjutan seiring berjalannya workflow.

7.4.6 Streaming Custom Data

LangGraph juga mendukung streaming custom data. Selain melakukan streaming state graph atau token, Anda juga dapat mengonfigurasi node Anda untuk melakukan streaming data custom apa pun yang dibutuhkan aplikasi Anda.

Contoh: Streaming Custom Data

Pada contoh ini, kita akan melakukan streaming update progres custom untuk sebuah node yang mensimulasikan task yang berjalan lama (long-running).

```
#lesson7g.py

from langgraph.graph import StateGraph, MessagesState, START, END
from time import sleep
from langgraph.types import StreamWriter

# Define a custom node to simulate a long-running task
def long_running_node(state: MessagesState, writer: StreamWriter):
    for i in range(1, 6):
        sleep(1) # Simulate a delay
        writer({"progress": f"Processing step {i}/5"})
    return {"messages": ["Task completed!"]}

# Define the graph
workflow = StateGraph(MessagesState)
workflow.add_node("long_running_node", long_running_node)
workflow.add_edge(START, "long_running_node")
workflow.add_edge("long_running_node", END)

# Compile the graph
app = workflow.compile()

# Simulate interaction and stream custom progress updates
```

```
def simulate_interaction():
    input_message = {"messages": [("human", "Start the long-running task")]}
    for result in app.stream(input_message, stream_mode=["custom", "updates"]):
        if "progress" in result[-1]:
            print(result[-1]) # Stream custom progress updates
        else:
            print(result[-1]) # Stream final message
    simulate_interaction()
```

Penjelasan:

- **long_running_node** menghasilkan (yield) update progres pada setiap langkah task (Processing step 1/5 , Processing step 2/5 , dst.).- Update progres custom ini di-streaming kembali kepada pengguna secara real-time, memberikan feedback kepada pengguna mengenai perkembangan task tersebut.- Setelah task selesai, pesan akhir di-streaming: "Task completed!" .

Output:

```
Processing step 1/5
Processing step 2/5
Processing step 3/5
Processing step 4/5
Processing step 5/5
Task completed!
```

Contoh ini menunjukkan bagaimana Anda dapat melakukan streaming custom data, seperti indikator progres, sebagai tambahan dari streaming state atau token standar.

7.4.7 Menonaktifkan Streaming untuk Model yang Tidak Mendukungnya

Tidak semua model atau workflow mendukung streaming. Misalnya, beberapa model mungkin perlu mengembalikan output-nya sekaligus. Dalam kasus seperti ini, Anda mungkin perlu menonaktifkan streaming untuk node atau model tertentu yang tidak mendukungnya.

Contoh: Menonaktifkan Streaming untuk Node Tertentu

Pada contoh ini, kita akan menonaktifkan streaming untuk sebuah node tertentu sambil tetap mengaktifkannya untuk sisa graph.

```
#lesson7g.py

from langgraph.graph import StateGraph, MessagesState, START, END
```

```

from langchain_openai import ChatOpenAI
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
# First example with streaming enabled
llm_streaming = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=1,
    streaming=True,
    callbacks=[StreamingStdOutCallbackHandler()]
)
# Second example with streaming disabled
llm_no_streaming = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=1,
    streaming=False
)
def create_graph(llm):
    graph_builder = StateGraph(MessagesState)
    def chatbot(state: MessagesState):
        messages = state["messages"]
        if not isinstance(messages, list):
            messages = [messages]
        return {"messages": llm.invoke(messages)}
    graph_builder.add_node("chatbot", chatbot)
    graph_builder.add_edge(START, "chatbot")
    graph_builder.add_edge("chatbot", END)
    return graph_builder.compile()
input = {
    "messages": [
        {
            "role": "user",
            "content": "how many r's are in strawberry? Explain in three
paragraphs."
        }
    ]
}
print("With streaming enabled:")
graph_streaming = create_graph(llm_streaming)

```

```

for output in graph_streaming.stream(input):
    if isinstance(output, dict) and 'chatbot' in output:
        # We don't need to print here as StreamingStdOutCallbackHandler handles it
        pass
print("\n\nWith streaming disabled:")
graph_no_streaming = create_graph(llm_no_streaming)
for output in graph_no_streaming.stream(input):
    if isinstance(output, dict) and 'chatbot' in output:
        message = output['chatbot']['messages']
        print(message.content, end="", flush=True)

```

Penjelasan:

- streaming=False secara eksplisit menonaktifkan streaming dari model. Ketika graph mencapai node ini, ia akan mengeluarkan seluruh pesan sekaligus, bukan melakukan streaming token atau update.

Pengaturan ini memungkinkan Anda mengontrol node mana yang melakukan streaming dan mana yang tidak, tergantung pada kemampuan model atau tool yang Anda gunakan.

7.4.8 Pertimbangan Praktis untuk Streaming

Ketika mengimplementasikan streaming pada LangGraph, pertimbangkan hal-hal berikut:

1. **Latency:** For long-running tasks or real-time data, streaming provides continuous feedback, reducing the perceived latency for the user.
2. **Efficiency:** Streaming updates rather than full state can reduce overhead and bandwidth when processing large graphs or complex workflows.
3. **Custom Data:** Streaming custom data allows for flexible feedback mechanisms, such as showing progress bars, intermediate results, or live updates during execution.
4. **Disabling Streaming:** Not all models support streaming, so it's important to handle cases where streaming should be disabled to avoid issues.

Integrasi API Tingkat Lanjut pada LangGraph

Pada bagian ini, kita akan membahas integrasi API real-time menggunakan LangGraph. Menghubungkan API eksternal adalah cara yang ampuh untuk memperluas fungsionalitas AI agent Anda, memungkinkannya mengambil data langsung (live), berinteraksi dengan service pihak ketiga, dan memberikan respons dinamis berdasarkan informasi dunia nyata. Kita akan menjelajahi cara untuk:

- Terhubung ke sebuah API eksternal.- Menggunakan data respons di dalam workflow.- Menggabungkan respons API dengan logika graph yang sudah ada.

7.5 Integrasi API Eksternal: Gambaran Umum

Integrasi API memungkinkan AI agent Anda untuk:

- **Mengambil data langsung (live):** Misalnya, mendapatkan informasi cuaca real-time dari sebuah weather API atau kurs mata uang dari sebuah financial service.- **Mengirim data (Post data):** Misalnya, mengirim informasi atau tindakan pengguna ke sebuah web service untuk diproses lebih lanjut.- **Berinteraksi dengan tool pihak ketiga:** Seperti terhubung dengan database eksternal atau akun pengguna untuk mengambil data yang dipersonalisasi.

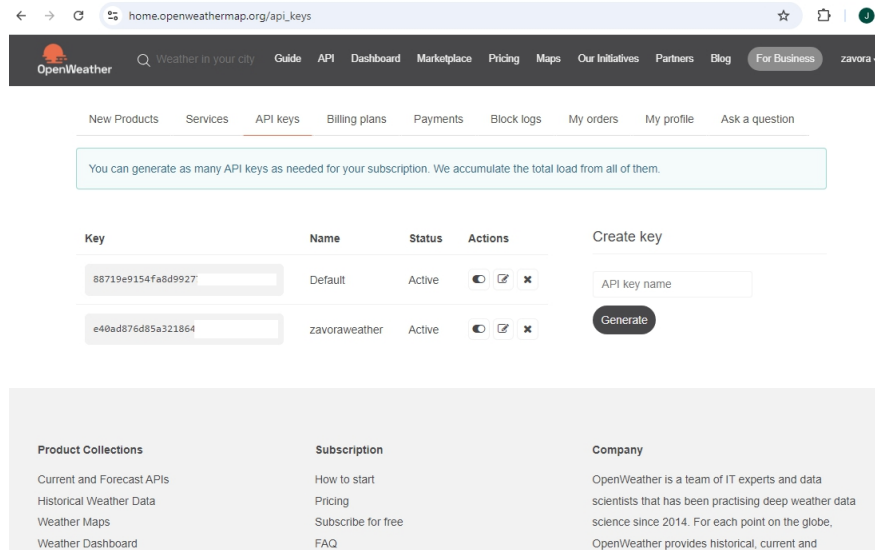
Pada LangGraph, pemanggilan API biasanya dilakukan di dalam custom node, yang kemudian memproses respons dari API tersebut dan melakukan routing state berdasarkan data yang dikembalikan. Menggabungkan pemanggilan API dengan conditional routing dan streaming memungkinkan agent Anda menjadi lebih dinamis dan responsif.

7.6 Integrasi Weather API: Contoh Langkah demi Langkah

Pada contoh ini, kita akan mengintegrasikan sebuah live weather API untuk memberikan informasi cuaca real-time sebagai respons terhadap query pengguna. Kita akan menggunakan OpenWeatherMap API sebagai contoh.

Langkah 1: Menyiapkan Weather API

Pertama, Anda memerlukan sebuah API key dari OpenWeatherMap. Anda dapat mendaftar dan mendapatkan API key gratis di https://home.openweathermap.org/api_keys.



Langkah 2: Mendefinisikan Node untuk Memanggil Weather API

Selanjutnya, kita akan mendefinisikan sebuah node yang:

- Mengirimkan request ke OpenWeatherMap API.-
- Mem-parsing respons tersebut.- Mengembalikan pesan berisi data cuaca.

```
#lesson7i.py

import requests
from dotenv import load_dotenv
import os

# Load environment variables
load_dotenv()
weather_api_key = os.getenv("OPENWEATHER_API_KEY")

# Define the node to fetch live weather data
def live_weather_node(state):
    city = "London" # You can replace this with dynamic input from the user
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={weather_api_key}&units=metric"

    # Make the API call
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        temperature = data['main']['temp']
```

```

        description = data['weather'][0]['description']
        return {"messages": [f"The weather in {city} is {temperature}°C with {description}."]}
    else:
        return {"messages": ["Sorry, I couldn't fetch the weather information."]}

```

Langkah 3: Menambahkan Node ke Graph

Selanjutnya, kita akan menambahkan node weather ini ke workflow LangGraph kita, memastikan bahwa graph dapat melakukan routing query ke node weather dan mengembalikan data cuaca sebagai respons.

```

#lesson7i.py continued

from langgraph.graph import StateGraph, MessagesState, START, END
# Define the graph workflow
builder = StateGraph(MessagesState)
# Add the weather node
builder.add_node("live_weather_node", live_weather_node)
# Set up the edges
builder.add_edge(START, "live_weather_node")
builder.add_edge("live_weather_node", END)
# Compile the graph
app = builder.compile()
# Simulate interaction with the weather API
def simulate_interaction():
    input_message = {"messages": [("human", "Tell me the weather in London")]}
    # Process the input and stream the result
    for result in app.stream(input_message, stream_mode="values"):
        result["messages"][-1].pretty_print()
    simulate_interaction()

```

Output:

The weather in London is 18°C with scattered clouds.

7.7 Integrasi API Dinamis dengan Input Pengguna

Pada contoh di atas, kita melakukan hard-code nama kota menjadi "London". Namun, pada sebagian besar skenario dunia nyata, Anda akan menginginkan kota tersebut disediakan secara dinamis

oleh pengguna. Mari kita perbarui node tersebut untuk mengekstrak nama kota dari input pengguna dan mengambil data cuaca untuk lokasi tersebut.

Langkah 1: Mengekstrak Kota dari Input Pengguna

Kita akan memodifikasi `live_weather_node` untuk secara dinamis mengambil nama kota dari query pengguna dan menyesuaikan pemanggilan API sesuai dengan itu.

```
#lesson7i.py continued

def live_weather_node(state):
    last_message = state["messages"][-1].content.lower()
    # Extract city name from user query
    if "in" in last_message:
        city = last_message.split("in")[-1].strip()
    else:
        city = "London" # Default city
    # API call
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={weather_api_key}&units=metric"
    # Make the API call
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        temperature = data['main']['temp']
        description = data['weather'][0]['description']
        return {"messages": [f"The weather in {city} is {temperature}°C with {description}."]}
    else:
        return {"messages": ["Sorry, I couldn't fetch the weather information."]}
```

Langkah 2: Memperbarui Workflow

Kita tidak perlu banyak mengubah pengaturan workflow, tetapi sekarang pengguna dapat menentukan sebuah kota secara dinamis, dan node tersebut akan mengambil data cuaca untuk lokasi tersebut.

Contoh Interaksi:

You: What's the weather in Paris?

Agent: The weather in Paris is 20°C with clear sky.

7.8 Integrasi Calculator API

Pada bagian ini, kita akan mengintegrasikan calculator API dasar yang dapat menangani operasi aritmatika seperti penjumlahan, pengurangan, perkalian, dan pembagian. Kita akan menggunakan API eksternal seperti Math.js untuk melakukan perhitungan.

Langkah 1: Definisikan Calculator Node

```
#lesson7i.py continued

# Define the node to fetch live calculator results
def calculator_node(state):
    last_message = state["messages"][-1].content.lower()
    # Extract the arithmetic expression from the user query
    expression = last_message.split("calculate")[-1].strip()
    # URL-encode the expression to ensure it's safe for use in the query string
    encoded_expression = urllib.parse.quote(expression)
    # Make the API call to the math.js API with the URL-encoded expression
    url = f"http://api.mathjs.org/v4/?expr={encoded_expression}"
    response = requests.get(url)
    if response.status_code == 200:
        result = response.text
        return {"messages": [f"The result of {expression} is {result}."]}
    else:
        return {"messages": ["Sorry, I couldn't calculate that."]}
```

Langkah 2: Tambahkan Node ke Workflow

```
#lesson7i.py continued

# Define the graph workflow
builder = StateGraph(MessagesState)
# Add the calculator node
builder.add_node("calculator_node", calculator_node)
# Set up the edges
builder.add_edge(START, "calculator_node")
builder.add_edge("calculator_node", END)
# Compile the graph
app = builder.compile()
# Simulate interaction with the calculator API
```

```
def simulate_interaction():
    input_message = {"messages": [("human", "Calculate 5 + 3 * 2")]}
    # Process the input and stream the result
    for result in app.stream(input_message, stream_mode="values"):
        result["messages"][-1].pretty_print()
    simulate_interaction()

Output:
The result of 5 + 3 * 2 is 11.
```

7.9 Menggabungkan Beberapa Integrasi API

Setelah membahas cara mengintegrasikan baik weather API maupun calculator API, mari kita gabungkan keduanya menjadi satu workflow. Hal ini akan memungkinkan agent menangani kedua jenis query dan secara dinamis merutekan input ke API yang sesuai berdasarkan input pengguna.

Langkah 1: Buat Routing Function

Kita akan membuat routing function yang:

- Merutekan query terkait cuaca ke weather node.
- Merutekan query terkait perhitungan ke calculator node.
- Merutekan input yang tidak dikenali ke default node.

```
#lesson7i.py continued

# Define the routing function
def routing_function(state):
    last_message = state["messages"][-1].content.lower()
    if "weather" in last_message:
        return "live_weather_node"
    elif "calculate" in last_message:
        return "calculator_node"
    return "default_node"
```

Langkah 2: Bangun Workflow Lengkap

```
#lesson7i.py continued

# Define the graph workflow
```

```

builder = StateGraph(MessagesState)
# Add the nodes
builder.add_node("live_weather_node", live_weather_node)
builder.add_node("calculator_node", calculator_node)
builder.add_node("default_node", lambda state: {"messages": ["Sorry, I don't understand
that request."]}))
# Add conditional edges for routing
builder.add_conditional_edges(START, routing_function)
# Set up the edges
builder.add_edge("live_weather_node", END)
builder.add_edge("calculator_node", END)
builder.add_edge("default_node", END)
# Compile the graph
app = builder.compile()
# Simulate interaction with both APIs
def simulate_interaction():
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Exiting...")
            break
        input_message = {"messages": [("human", user_input)]}
        # Stream the result
        for result in app.stream(input_message, stream_mode="values"):
            result["messages"][-1].pretty_print()
simulate_interaction()

```

Contoh Interaksi:

You: What's the weather in New York?

Agent: The weather in New York is 22°C with scattered clouds.

You: Calculate 10 / 2

Agent: The result of 10 / 2 is 5.

Ringkasan

Pada bagian ini, kita telah membahas cara mengintegrasikan API eksternal ke dalam workflow LangGraph. Kita telah mendemonstrasikan:

- Integrasi weather API menggunakan OpenWeatherMap.- Integrasi calculator API menggunakan Math.js.- Dynamic routing antar node berbeda berdasarkan input pengguna.

Kombinasi integrasi API ini memungkinkan agent Anda menangani berbagai jenis query yang lebih luas dan memberikan informasi dinamis secara real-time.

Pada bagian berikutnya, kita akan membahas error handling dan mekanisme retry untuk membuat workflow Anda lebih robust dan fault-tolerant.

Kuis Bab 7: Integrasi API, Routing, dan Streaming

Pertanyaan 1:

Apa tujuan utama mengintegrasikan API eksternal ke dalam workflow LangGraph?

- A. Untuk mengurangi jumlah kode yang diperlukan- B. Untuk memungkinkan interaksi real-time dengan layanan eksternal, seperti mengambil data live atau melakukan perhitungan- C. Untuk mempercepat eksekusi workflow- D. Untuk memungkinkan debugging workflow

Jawaban:

B. Untuk memungkinkan interaksi real-time dengan layanan eksternal, seperti mengambil data live atau melakukan perhitungan.

Pertanyaan 2:

Apa peran conditional edge dalam workflow LangGraph?

- A. Untuk menentukan urutan eksekusi node- B. Untuk merutekan workflow ke node yang berbeda berdasarkan state graph atau logika kustom- C. Untuk mendefinisikan pesan kustom untuk node- D. Untuk menangani exception dalam workflow

Jawaban:

B. Untuk merutekan workflow ke node yang berbeda berdasarkan state graph atau logika kustom.

Pertanyaan 3:

Modul Python mana yang umum digunakan untuk memastikan bahwa parameter, seperti input pengguna, di-URL-encode dengan benar dalam sebuah panggilan API?

- A. json-
B. urllib.parse-
C. os- D. re

Jawaban:

B. urllib.parse

Pertanyaan 4:

Apa yang dilakukan oleh method `add_conditional_edges()` dalam workflow LangGraph?

- A. Menambahkan edge antar node yang dieksekusi secara kondisional berdasarkan hasil routing function.- B. Menggabungkan dua workflow yang berbeda.- C. Memicu panggilan API saat dipanggil.- D. Menonaktifkan streaming untuk node yang tidak mendukungnya.

Jawaban:

A. Menambahkan edge antar node yang dieksekusi secara kondisional berdasarkan hasil routing function.

Pertanyaan 5:

Bagaimana cara menonaktifkan streaming untuk node tertentu di LangGraph yang tidak mendukung streaming?

- A. Dengan menghapus argumen `stream_mode` dari definisi node- B. Dengan mengatur `stream_mode` menjadi "disable" pada konfigurasi graph- C. Dengan mengecualikan node tersebut dari workflow- D. Dengan mendefinisikan node tersebut sebagai bagian dari graph non-streaming terpisah

Jawaban:

B. Dengan mengatur `stream_mode` menjadi "disable" pada konfigurasi graph.

Pertanyaan 6:

Mana dari berikut ini yang paling tepat menggambarkan tujuan dari function `requests.get()` dalam konteks integrasi API?

- A. Digunakan untuk melakukan GET request ke API eksternal untuk mengambil data.- B. Digunakan untuk menyimpan hasil dari workflow dalam sebuah dictionary.- C. Digunakan untuk merutekan data antar node di LangGraph.- D. Digunakan untuk melakukan LLM invocation di dalam LangGraph.

Jawaban:

A. Digunakan untuk melakukan GET request ke API eksternal untuk mengambil data.

Pertanyaan 7:

Dalam konteks streaming di LangGraph, apa yang dilakukan oleh pengaturan `stream_mode="values"`?

- A. Mengalirkan (stream) seluruh state graph setelah setiap node dieksekusi.- B. Hanya mengalirkan data yang ter-tokenisasi dari LLM.- C. Mengalirkan data kustom yang dihasilkan di dalam node.- D. Menonaktifkan streaming dan mengembalikan hasil akhir di akhir eksekusi.

Jawaban:

A. Mengalirkan (stream) seluruh state graph setelah setiap node dieksekusi.

Pertanyaan 8:

Function Python apa yang akan Anda gunakan untuk mengekstrak ekspresi aritmatika `"5 + 3"` dari input pengguna, `"Calculate 5 + 3"`?

- A. `input.split()`- B. `re.match()`- C. `input.split("calculate")[-1].strip()`- D. `re.split()`

Jawaban:

C. `input.split("calculate")[-1].strip()`

Pertanyaan 9:

Mana dari mode berikut yang BUKAN merupakan stream mode yang valid di LangGraph?

- A. `"values"`- B. `"updates"`- C. `"tokens"`- D. `"complete"`

Jawaban:

D. `"complete"`

Pertanyaan 10:

Apa yang dilakukan `urllib.parse.quote()` di Python?

- A. Mengonversi objek Python menjadi string yang di-encode dalam format JSON.- B. Mengirim HTTP request ke API eksternal.

- C. Melakukan URL-encode pada string, memastikan karakter khusus diformat dengan benar dalam sebuah URL.- D. Mengonversi teks menjadi huruf kecil.

Jawaban:

C. Melakukan URL-encode pada string, memastikan karakter khusus diformat dengan benar dalam sebuah URL.

Tugas Praktik: Integrasi API dan Conditional Routing

Ikhtisar Tugas:

Untuk tugas ini, Anda akan membangun sebuah AI agent yang:

- **Terintegrasi dengan dua API eksternal: weather API dan calculator API.- Secara dinamis merutekan query pengguna ke API yang tepat berdasarkan input.- Mengimplementasikan streaming untuk update progres dan respons kustom berdasarkan input pengguna.**

Persyaratan:

1. Use the **OpenWeatherMap API** to fetch live weather information based on the city provided by the user.
2. Use the **Math.js API** to perform arithmetic operations such as addition, subtraction, multiplication, and division.
3. Route the user's queries to the appropriate API (weather or calculator) using **conditional routing**.
4. Implement streaming to:
 - Stream the result of the **weather API** as soon as it's available.
 - Stream **progress updates** when handling calculator queries with multiple steps.
5. Ensure proper **URL-encoding** for the API requests.

Contoh Interaksi Pengguna:

Anda: Bagaimana cuaca di London?

Agent: Cuaca di London adalah 18°C dengan awan tersebar (scattered clouds).

Anda: Calculate $5 + 3 * 2$

Agent: Hasil dari $5 + 3 * 2$ adalah 11.

Kode Solusi:

```
#lesson71.py full solution code

import requests
import urllib.parse
from langgraph.graph import StateGraph, MessagesState, START, END
from dotenv import load_dotenv
import os

# Load API keys from environment variables
load_dotenv()
weather_api_key = os.getenv("OPENWEATHER_API_KEY")

# Define the node for fetching live weather data
def live_weather_node(state):
    last_message = state["messages"][-1].content.lower()
    # Extract city name from user query
    if "in" in last_message:
        city = last_message.split("in")[-1].strip().replace("?", "")
    else:
        city = "London" # Default city
    # URL-encode the city name for the API request
    city_encoded = urllib.parse.quote(city)
    # Fetch the weather data from OpenWeatherMap API
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city_encoded}&appid={weather_api_key}&units=metric"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        temperature = data['main']['temp']
        description = data['weather'][0]['description']
        return {"messages": [f"The weather in {city} is {temperature}°C with {description}."]}
    else:
```

```

        return {"messages": ["Sorry, I couldn't fetch the weather
information."]}
# Define the node for calculator operations using Math.js API
def calculator_node(state):
    last_message = state["messages"][-1].content.lower()
    # Extract the arithmetic expression from the user query
    expression = last_message.split("calculate")[-1].strip()
    # URL-encode the expression for the API request
    encoded_expression = urllib.parse.quote(expression)
    # Fetch the result from Math.js API
    url = f"http://api.mathjs.org/v4/?expr={encoded_expression}"
    response = requests.get(url)
    if response.status_code == 200:
        result = response.text
        return {"messages": [f"The result of {expression} is
{result}."]}
    else:
        return {"messages": ["Sorry, I couldn't calculate that."]}
# Define a default node for unrecognized inputs
def default_node(state):
    return {"messages": ["Sorry, I don't understand that request."]}
# Define the routing function to route the user query to the
appropriate node
def routing_function(state):
    last_message = state["messages"][-1].content.lower()
    if "weather" in last_message:
        return "live_weather_node"
    elif "calculate" in last_message:
        return "calculator_node"
    return "default_node"
# Build the LangGraph workflow
builder = StateGraph(MessagesState)
builder.add_node("live_weather_node", live_weather_node)
builder.add_node("calculator_node", calculator_node)
builder.add_node("default_node", default_node)
# Add conditional edges to route the queries
builder.add_conditional_edges(START, routing_function)

```

```

builder.add_edge("live_weather_node", END)
builder.add_edge("calculator_node", END)
builder.add_edge("default_node", END)
# Compile the graph
app = builder.compile()
# Simulate interaction with the user
def simulate_interaction():
    while True:
        user_input = input("You: ")
        if user_input.lower() in ["exit", "quit"]:
            print("Exiting...")
            break
        input_message = {"messages": [("human", user_input)]}
        # Stream the result
        for result in app.stream(input_message,
stream_mode="values"):
            result["messages"][-1].pretty_print()
simulate_interaction()

```

Penjelasan Solusi:

1. **Weather API Node:** This node fetches the current weather for the city provided by the user using the **OpenWeatherMap API**. The city name is dynamically extracted from the user's query.
2. **Calculator API Node:** This node handles arithmetic calculations via the **Math.js API**. It extracts the mathematical expression from the user query and performs URL encoding to ensure special characters (such as `+`, `*`, `/`) are properly formatted in the API request.
3. **Routing Function:** The routing function inspects the user's query and routes it to the appropriate node. If the query contains the word "weather", it routes to the weather node. If the query contains "calculate", it routes to the calculator node. For unrecognized queries, it routes to the default node.
4. **Streaming:** The graph streams the results from the API calls back to the user in real-time. This allows for faster feedback and more interactive experiences.

Part 3: LangGraph Agent Design Patterns
and Architectures
(Chapters 8-11)

Chapter 8

Arsitektur Fondasi AI Agent -

ReAct

Pendahuluan

Pada bab ini, kita akan menjelajahi arsitektur agent inti yang menjadi fondasi sistem AI yang dinamis dan fleksibel menggunakan LangGraph. Arsitektur-arsitektur ini dirancang untuk membantu AI agent melakukan tugas secara otonom atau semi-otonom, beradaptasi dengan lingkungan yang berubah, dan membuat keputusan berdasarkan data real-time. Dengan memahami arsitektur fondasi ini, Anda akan siap mengembangkan AI agent canggih Anda sendiri yang mampu menangani berbagai skenario bisnis.

Kita akan membahas lima arsitektur utama, masing-masing sesuai untuk jenis workflow dan aplikasi yang berbeda:

1. **ReAct (Reason + Act) Pattern** – A simple, transparent decision-making framework.
2. **Plan-and-Execute Agents** – Agents that plan multi-step workflows and execute them.
3. **Hierarchical Agent Teams** – A supervisor-worker model for complex task management.
4. **Agentic Retrieval-Augmented Generation (RAG)** – A framework that combines information retrieval and generation.
5. **Corrective Retrieval-Augmented Generation (CRAG) with Self-Reflection** – A refined version of RAG, incorporating self-assessment.

Mari kita mulai.

Pola ReAct (Reason + Act)

Ikhtisar

Arsitektur ReAct menggabungkan reasoning dengan action, memungkinkan agent untuk "berpikir dengan lantang" dengan menghasilkan thought dan menjalankan action berdasarkan thought tersebut. Transparansi dalam pengambilan keputusan ini membuat agent lebih akuntabel, karena mencatat proses reasoning-nya di setiap langkah.

Arsitektur ini paling cocok untuk tugas di mana agent perlu mengevaluasi beberapa opsi lalu memilih tindakan terbaik. Sebagai contoh, arsitektur ini dapat digunakan dalam aplikasi layanan pelanggan di mana agent harus mengevaluasi query pelanggan dan memutuskan cara merespons.

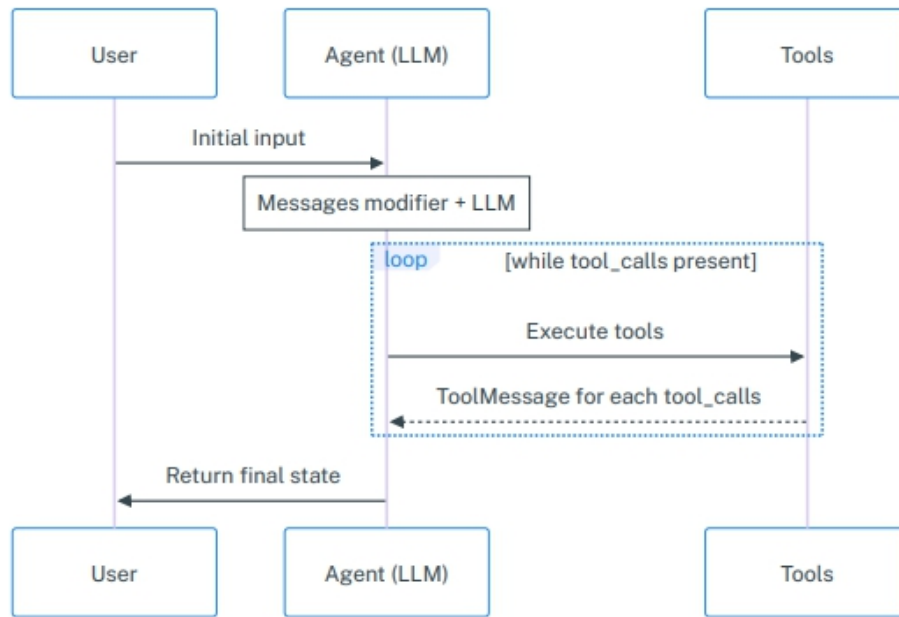
Latar Belakang Historis

ReAct agent merepresentasikan terobosan dalam integrasi reasoning dan acting pada sistem AI, dirancang untuk membuat keputusan dan mengambil tindakan secara real time berdasarkan input dinamis dan data eksternal. Arsitektur ini diperkenalkan dalam paper ReAct oleh Shunyu Yao dkk. (2023) dari Princeton University dan Google Research. Framework ReAct menggabungkan verbal reasoning dan action spesifik-tugas secara berselang-seling (interleaved), memungkinkan sinergi antara reasoning trace dan action. Hal ini menghasilkan kemampuan penyelesaian tugas yang lebih robust, terutama pada tugas yang dinamis atau padat pengetahuan (knowledge-intensive).

Secara historis, sistem AI berfokus baik pada reasoning (misalnya, Chain-of-Thought, atau CoT prompting) maupun pada eksekusi action (misalnya, reinforcement learning). Pendekatan-pendekatan ini, meski efektif secara terpisah, sering gagal ketika tugas membutuhkan kombinasi reasoning dan action. Sebagai contoh, model berbasis reasoning murni sering menghasilkan halusinasi, sementara model berbasis action kekurangan konteks dan perencanaan. ReAct mengatasi kekurangan ini dengan menciptakan feedback loop antara reasoning (thought) dan action (menjalankan tugas dan mengumpulkan informasi tambahan). Arsitektur ini telah terbukti efektif pada lingkungan interaktif seperti ALFWorld dan tugas padat pengetahuan seperti HotPotQA.

Konsep Kunci ReAct

- **Sinergi Reasoning dan Action:** Agent bernalar melalui data atau opsi yang tersedia lalu memilih action berdasarkan reasoning tersebut.
- **Pengambilan Keputusan yang Transparan:** Agent mencatat thought dan action-nya, memberikan rekam jejak yang jelas tentang bagaimana keputusan dibuat.
- **Proses Iteratif:** Siklus reasoning dan action dapat diulang, memungkinkan agent menyempurnakan pendekatannya jika action awal tidak menghasilkan hasil yang diinginkan.

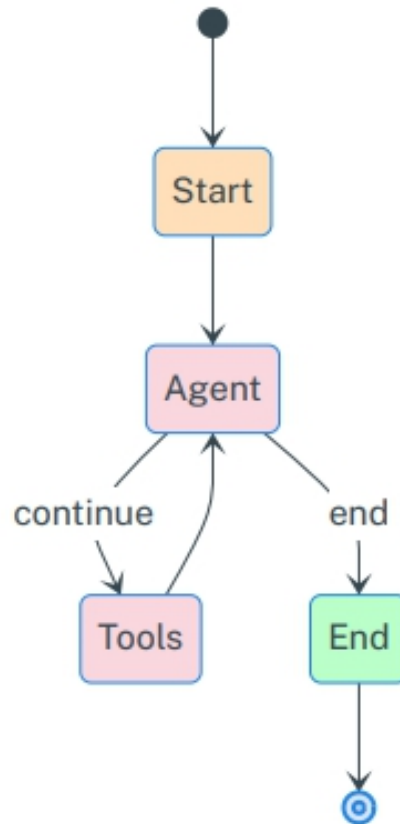


Langkah-Langkah Implementasi

1. **Define the Environment and Agent Goal:** First, provide a description of the environment in which the agent operates. Define the goal that the agent needs to achieve.
2. **List Available Actions:** Enumerate all possible actions the agent can undertake. These might include querying a database, calling an API, or responding to a user query.
3. **Record Observations:** Log actions taken and the resulting observations (e.g., did the action succeed or fail? What new data is available?).
4. **Generate Reasoning:** Based on the observations, prompt the agent to generate thoughts or reason about the next action. This process continues until the goal is achieved.
5. **Execute Actions:** Once the agent completes the reasoning process, it chooses an action and executes it.

ReAct Agent di LangGraph

LangGraph mengimplementasikan arsitektur ReAct dengan cara yang sangat fleksibel, memungkinkan pengguna membuat agent dinamis yang mampu melakukan reasoning, tool calling, dan pengambilan keputusan. LangGraph ReAct agent menggunakan struktur modular di mana sebuah large language model (LLM) bernalar melalui langkah-langkah dan secara dinamis memutuskan apakah akan memanggil tool, mengambil data, atau menghentikan proses.



Komponen Kunci ReAct di LangGraph:

1. **Reasoning:** The agent generates thoughts based on the input and decides on potential actions.
2. **Tool Calling:** Based on reasoning, the agent decides to call specific tools, such as APIs or knowledge bases, and integrates the tool output back into the reasoning loop.
3. **Observations:** The output from tools is passed back to the reasoning phase, allowing the agent to refine its thinking and proceed with further actions or finalize its response.
4. **Memory:** The agent can retain information across multiple interactions, allowing it to make informed decisions based on previous steps .

Opsi Implementasi ReAct Agent Internal dan Kustom di LangGraph

LangGraph menyediakan ReAct agent pre-built yang terintegrasi dengan tool dan API eksternal, memungkinkan eksekusi tugas yang mulus. Sebuah LangGraph ReAct agent dapat memutuskan secara otonom apakah akan memanggil tool berdasarkan input pengguna dan kemudian melanjutkan berdasarkan output tool.

Anda juga dapat membuat ReAct agent Anda sendiri di LangGraph menggunakan kemampuan berikut:

- **System Prompt:** Mendefinisikan tujuan atau perilaku tingkat tinggi dari agent.- **Tools:** Function, API, atau sumber daya lain yang telah didefinisikan sebelumnya yang dapat dipanggil oleh agent.

- **Graph Structure:** Sebuah graph didefinisikan di mana setiap langkah (node) dapat melakukan reasoning, action, atau memanggil sebuah tool. Agent terus melakukan siklus melalui langkah-langkah ini hingga tidak diperlukan lagi tool call.

Praktik Terbaik untuk Membangun ReAct Agent

1. **Structured Thought and Action Cycles:** Ensure that the agent's reasoning steps are separated from action steps. This allows for clear identification of what the agent is thinking and what actions are being taken.
2. **Effective Tool Integration:** ReAct agents should have access to a rich set of tools to call, enabling them to fetch external data when needed. In LangGraph, tools can range from simple Python functions to complex APIs.
3. **Use of Memory:** For longer conversations or tasks, it's crucial to maintain memory, allowing the agent to "remember" past steps and interactions. LangGraph provides memory modules for both short-term and long-term retention.
4. **Iterative Thought-Action Loops:** Design your ReAct agents to cycle through reasoning and action multiple times as necessary, allowing them to refine their understanding of the task with each loop.
5. **Error Handling and Reflection:** Implement feedback mechanisms where the agent can identify and recover from errors, or ask for clarification if unsure about the next step .

10 Contoh ReAct Agent di LangGraph

Contoh 1: ReAct Agent Dasar dengan Built-In Tools

Ikhtisar:

Pada contoh ini, kita akan membuat sebuah ReAct agent sederhana yang melakukan operasi aritmatika dasar menggunakan built-in tools. Agent akan bernalar tentang query pengguna dan memanggil tool yang sesuai untuk menjalankan perhitungan.

Konsep Kunci:

- Menggunakan built-in ReAct agent di LangGraph.- Tool calling dasar (penjumlahan, perkalian, pembagian).

```
#lesson8a.py

from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
# Define tools
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b
def multiply(a: int, b: int) -> int:
    """Multiply two numbers together."""
    return a * b
```

```

tools = [add, multiply]
# Initialize the LLM
llm = ChatOpenAI(model="gpt-4o-mini")
# Create the ReAct agent
graph = create_react_agent(model=llm, tools=tools)
# User input
inputs = {"messages": [("user", "Add 3 and 4. Multiply the result by 2.")]}
# Run the ReAct agent
messages = graph.invoke(inputs)
for message in messages["messages"]:
    print(message.content)

```

Output=====Hu
man
Message=====

Add 3 and 4. Multiply the result by
2.=====Ai
Message=====

===Tool Calls:

add (call_y8FflKxbYsM6dysTSh5YykDE)Call ID:
call_y8FflKxbYsM6dysTSh5YykDE
Args:

a: 3b:

4=====T
ool

Message=====

====Name: add

7=====

=Ai

Message=====

====Tool Calls:

multiply (call_KzbjH61WxKq2t4aksEOh2wwc)Call ID:
call_KzbjH61WxKq2t4aksEOh2wwc
Args:

a: 7b:

2=====T
ool

Message=====

====Name: multiply

14=====Ai
Message=====

===

Hasil dari penjumlahan 3 dan 4 adalah 7. Ketika hasil tersebut dikalikan 2, Anda mendapatkan 14.

Pada contoh ini, kita membuat sebuah ReAct agent dasar yang melakukan aritmatika sederhana menggunakan built-in tools (penjumlahan dan perkalian). Langkah-langkah kuncinya adalah:

1. **User Input:** The user inputs a request in natural language, such as *"Add 3 and 4. Multiply the result by 2."*
2. **Reasoning (Agent Node):** The ReAct agent first processes the input with the language model (in this case, GPT-4o-mini) to determine whether it can respond directly or if it needs to call a tool.
3. **Tool Call (Tools Node):** If the model identifies a need for a tool (such as performing a calculation), it triggers the relevant tool (e.g., **add** or **multiply**).
4. **Result Integration:** After the tool completes its execution, the result is passed back to the agent for further reasoning.
5. **Final Output:** Once all necessary tool calls are completed, the agent provides a final response to the user.

Siklus sederhana reasoning, tool calling, dan responding ini membentuk inti dari framework ReAct. Agent terus melakukan looping antara reasoning dan acting (memanggil tool) hingga tidak ada lagi action yang diperlukan.

Bagaimana Internal ReAct Agent Tool Bekerja

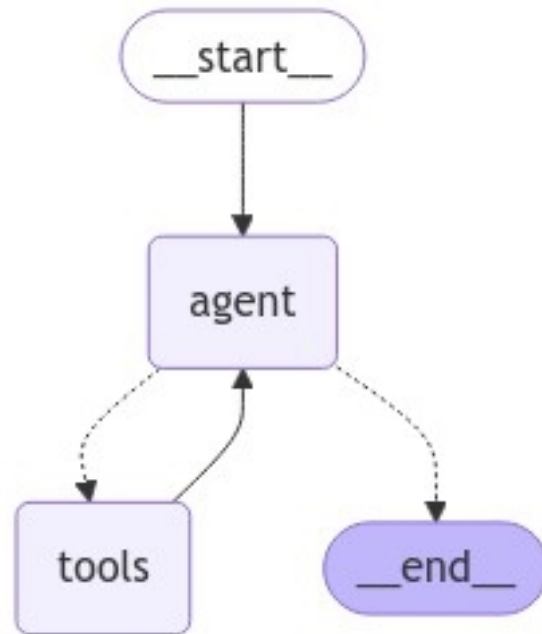
Function `create_react_agent` mendefinisikan logika internal dari ReAct agent. Berikut adalah rincian komponen-komponen kuncinya:

Function signature menyediakan parameter untuk mengonfigurasi ReAct agent:

- **model** : Language model (misalnya, GPT-4) yang digunakan untuk reasoning.-
- tools** : Daftar tool (misalnya, `add` , `multiply`) yang dapat dipanggil oleh agent untuk bertindak.-
- state_schema** : Parameter opsional yang mendefinisikan struktur state graph, yang mencakup pelacakan message dan metadata lainnya.-
- messages_modifier** : Deprecated. Sebelumnya digunakan untuk memodifikasi message sebelum diteruskan ke language model.-
- state_modifier** : Opsional. Memungkinkan modifikasi pada state graph sebelum dikirim ke language model.-
- checkpoint** , **store** : Mengelola penyimpanan dan pengambilan state untuk persistence antar session.-
- interrupt_before** , **interrupt_after** : Titik dalam alur di mana interupsi eksternal (misalnya, konfirmasi pengguna) dapat disisipkan.-
- debug** : Flag untuk mengaktifkan mode debug.

Desain ReAct Agent:

Visualisasi dari agent graph adalah sebagai berikut:



ReAct agent dirancang untuk terus melakukan looping antara reasoning (agent node) dan acting (tools node). Alur kerja utama dari agent ini dijelaskan sebagai berikut:

- **Agent Node:** Agent memanggil language model dengan daftar message saat ini (input pengguna + respons sebelumnya) dan bernalar apakah akan memanggil sebuah tool atau merespons langsung.- **Tools Node:** Jika language model menentukan bahwa sebuah tool harus dipanggil (misalnya, untuk melakukan operasi aritmatika), tools node dipicu, yang menjalankan tool tersebut (misalnya, add) dan mengembalikan hasilnya ke agent.- **Conditional Routing:** Agent terus memanggil tool dan bernalar berdasarkan output tool hingga tidak ada lagi tool call yang diperlukan. Setelah agent menentukan bahwa semua action yang diperlukan telah selesai, agent melanjutkan ke end node dan mengembalikan hasil akhir.- **StateGraph:** Sebuah StateGraph dibuat untuk mendefinisikan node dan edge dari graph. Graph ini merepresentasikan alur reasoning dan action pada ReAct agent.
 - **Nodes:** Two main nodes are defined: the **agent** node (which invokes the language model) and the **tools** node (which handles tool calls).
 - **Edges:** Conditional edges determine how the agent moves between the reasoning and acting phases. Specifically, if the agent decides to call a tool, the flow moves from the **agent** node to the **tools** node. After a tool is executed, the flow returns to the **agent** node to either reason further or finalize the response.

Contoh Conditional Edge:

```
#lesson8a.py continued

workflow.add_conditional_edges(
    "agent", # Start from the agent node
```

```
    should_continue # Decide whether to call tools or end the process
)
```

Mekanisme Looping: ReAct agent terus melakukan looping antara agent node dan tools node selama agent memutuskan untuk memanggil tool. Loop ini memungkinkan agent menangani permintaan kompleks yang membutuhkan beberapa action sebelum selesai.

Pembuatan Graph Otomatis

Pada contoh ini, function `create_react_agent` dari `LangGraph` secara otomatis membangun sebuah graph untuk ReAct agent berdasarkan tool dan language model yang disediakan. Graph ini mengikuti struktur tertentu:

- Agent Node ? Tools Node ? Agent Node ? End Node

Graph milik agent menangani logika untuk menentukan kapan harus memanggil tool dan kapan harus berhenti. Setiap kali agent node memproses input (query pengguna atau output tool), agent menentukan apakah action lebih lanjut (tool call) diperlukan. Jika tidak ada action lebih lanjut yang dibutuhkan, graph berpindah ke end node, menyelesaikan alur.

Graph
VisualizationstateDiagram-v2

```
[*] --> Start
Start --> Agent
Agent --> Tools
Tools --> continue
continue --> Agent
Agent --> End
End --> [*]
```

- **Agent Node:** Memanggil LLM dengan input dan menentukan apakah diperlukan tool call.
- **Tools Node:** Menjalankan tool sesuai kebutuhan dan mengembalikan hasilnya ke agent.
- **Looping Behavior:** Agent terus memanggil tool dan bernalar atas hasilnya hingga tidak ada lagi tool call yang diperlukan. Proses kemudian berhenti.

Rincian Contoh Output

Mari kita uraikan sebuah contoh di mana pengguna meminta ReAct agent untuk "Add 3 and 4. Multiply the result by 2."

1. Initial Input:

- User input: *"Add 3 and 4. Multiply the result by 2."*
- The agent processes this input and identifies two operations: addition and multiplication.

2. Tool Call 1 (Addition):

- Tool called: `add(3, 4)`.
- Output: 7.

3. Tool Call 2 (Multiplication):

- Tool called: `multiply(7, 2)` .
- Output: 14.

4. Final Output:

- The agent returns: *"The result of adding 3 and 4 and then multiplying the result by 2 is 14."*

Sepanjang proses ini, agent node menangani reasoning atas input dan output, dan tools node menjalankan operasi spesifik (penjumlahan dan perkalian).

Contoh 2: ReAct Agent untuk Product Inquiry dengan Single-Thread Memory

Ikhtisar:

Pada contoh ini, kita akan merevisi product inquiry agent untuk menggunakan single-thread memory melalui MemorySaver untuk menyimpan dan mengingat query pengguna. Agent akan mampu mengingat pertanyaan pengguna sebelumnya dalam satu thread (percakapan), mengambil informasi produk, dan memberikan respons yang lebih personal berdasarkan memory.

Konsep Kunci:

- Menggunakan single-thread memory untuk mempertahankan informasi pengguna sepanjang sebuah session.- Bernalar tentang pertanyaan produk berdasarkan input pengguna dan memanggil tool untuk mengambil detail produk.- Mengintegrasikan MemorySaver milik LangGraph untuk mengelola memory dalam satu session percakapan.

Implementasi:

```
#lesson8a.py continued

from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.memory import MemorySaver
# Define tools
def product_info(product_name: str) -> str:
    """Fetch product information."""
    product_catalog = {
        "iPhone 20": "The latest iPhone features an A15 chip and improved camera.",
        "MacBook": "The new MacBook has an M2 chip and a 14-inch Retina display.",
    }
    return product_catalog.get(product_name, "Sorry, product not found.")
# Initialize the memory saver for single-thread memory
checkpointer = MemorySaver()
# Initialize the language model
llm = ChatOpenAI(model="gpt-4o-mini")
# Create the ReAct agent with the memory saver
```

```

graph = create_react_agent(model=llm, tools=[product_info], checkpointer=checkpointer)
# Set up thread configuration to simulate single-threaded memory
config = {"configurable": {"thread_id": "thread-1"}}
# User input: initial inquiry
inputs = {"messages": [("user", "Hi, I'm James. Tell me about the new iPhone 20.)"]}
messages = graph.invoke(inputs, config=config)
for message in messages["messages"]:
    message.pretty_print()
# User input: repeated inquiry (memory recall)
inputs2 = {"messages": [("user", "Tell me more about the iPhone 20.)"]}
messages2 = graph.invoke(inputs2, config=config)
for message in messages2["messages"]:
    message.pretty_print()

```

Penjelasan Langkah-Langkah Kunci:

1. Single-Thread Memory with **MemorySaver** :

- The **MemorySaver** is used to manage memory within a single conversation session. It stores user interactions under a unique thread ID (in this case, "thread-1"), allowing the agent to recall information within the same conversation.
- **Thread Configuration:** We configure the ReAct agent to use a specific thread ID ("thread-1") to simulate a single-threaded conversation.

2. Reasoning and Tool Call:

- The ReAct agent first reasons about the user's query (e.g., "Tell me about the new iPhone") and decides to call the **product_info** tool with the product name "iPhone" .
- The tool retrieves the relevant product information, which the agent then provides to the user.

3. Memory Storage:

- After responding to the user's initial query, the agent stores the query and its response in memory. This memory is saved under the thread "thread-1" .

4. Memory Recall:

- When the user asks a follow-up question (e.g., "Tell me more about the iPhone"), the agent checks the memory stored in the same thread ("thread-1"). It recalls that the user previously asked about the iPhone and uses this information to provide a personalized response.
- The agent still has the option to call the **product_info** tool again if further details are needed.

Rincian Contoh Output:

1. Initial Inquiry:

- **User Input:** *"Tell me about the new iPhone."*
- **Agent Response:** *"The latest iPhone features an A15 chip and improved camera."*
- The agent stores this query and response in memory for future reference.

```
=====
Human Message=====Tell
me about the new iPhone
20.=====Ai
Message=====The
latest iPhone 20 features an A15 chip and improved camera.
```

2. Follow-Up Inquiry (with Memory Recall):

- **User Input:** *"Tell me more about the iPhone."*
- **Agent Response:** *"You previously asked about the iPhone. Here's more information: The latest iPhone features an A15 chip and improved camera."*
- The agent recalls the previous query and provides a personalized response.

```
=====Human Message=====Tell me
more about the iPhone 20.=====Ai
Message=====You previously asked about the iPhone 20. Here's more
information: The latest iPhone features an A15 chip and improved camera.
```

Bagaimana Single-Thread Memory Bekerja:

1. **MemorySaver Integration:** The **MemorySaver** checkpoint ensures that memory is stored and persisted within a single conversation session, identified by the thread ID (**"thread-1"**). This allows the agent to remember previous interactions within that specific session but forgets once the session (thread) ends.
2. **Single-Threaded Memory Behavior:**
 - **Thread ID:** Each session is tied to a unique thread ID, ensuring that the agent's memory is scoped to that particular conversation. In this example, the agent remembers the user's initial query about the iPhone within the **"thread-1"** session.
 - **Memory Recall:** When the user asks a follow-up question within the same thread, the agent retrieves and uses the stored information to provide a more context-aware and personalized response.

3. Memory Usage in ReAct Design:

- **Reasoning (Agent Node):** The agent reasons over the user's input and determines whether to recall memory or call a tool. In this case, it checks whether the query is related to a previous one stored in memory.
- **Tool Call (Tools Node):** If the query requires retrieving product information, the agent calls the `product_info` tool to fetch product details. It then stores the user query and the response in memory.
- **Memory Check:** In future queries within the same thread, the agent checks the memory for previously stored queries. If relevant, the agent recalls the memory and provides a tailored response.

Visualisasi Graph dengan Integrasi Memory:

```
stateDiagram-v2
    [*] --> Start
    Start --> Agent
    Agent --> Tools : continue
    Tools --> Memory : store or recall query
    Memory --> Agent : recall memory
    Agent --> End : end
    End --> [*]
```

- **Agent Node:** Agent memproses input pengguna dan memutuskan apakah akan?
Memanggil sebuah tool (seperti mengambil informasi produk).? Mengingat
memory jika relevan dengan percakapan saat ini.- **Tools Node:** Node ini
menjalankan tool (dalam hal ini, `product_info`), yang mengambil detail produk
berdasarkan pertanyaan pengguna.- **Memory Node:** Jika memory relevan, agent
memeriksa single-thread memory yang disimpan oleh `MemorySaver` dan mengingat
query sebelumnya. Memory ini dibatasi (scoped) ke thread tertentu, memastikan agent
hanya mengingat interaksi dalam session yang sama.- **End Node:** Setelah reasoning,
tool call, dan pengambilan memory selesai, agent memberikan respons akhir dan
menyelesaikan percakapan.

Contoh 3: ReAct Agent Kompleks untuk Product Inquiry dan Ketersediaan Stok dengan Memory dan Dynamic Decision-Making

Ikhtisar:

Pada contoh ini, kita akan membuat sebuah ReAct agent yang lebih kompleks yang menangani pertanyaan produk sekaligus memeriksa ketersediaan stok menggunakan proses dynamic decision-making. Agent akan menggunakan single-thread memory untuk mengingat query pengguna, mengambil informasi produk, dan secara dinamis memutuskan apakah akan memeriksa ketersediaan stok produk berdasarkan permintaan pengguna. Setup ini menunjukkan cara mengintegrasikan beberapa tool, bernalar tentang action, dan secara dinamis memanggil tool yang sesuai tergantung pada pertanyaan lanjutan pengguna.

Konsep Kunci:

- **Dynamic decision-making:** Agent akan bernalar berdasarkan input pengguna dan memutuskan apakah akan memanggil tool tambahan (misalnya, memeriksa ketersediaan stok).

- **Memory with MemorySaver** : Agent akan mengingat query pengguna sebelumnya dalam satu sesi dan memberikan respons yang dipersonalisasi.- **Tool Integration**: Beberapa tool akan digunakan, termasuk satu untuk mengambil informasi produk dan lainnya untuk memeriksa ketersediaan stok.

Implementasi:

```
#lesson8c.py

from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.memory import MemorySaver

# Define tools
def product_info(product_name: str) -> str:
    """Fetch product information."""
    product_catalog = {
        "iPhone": "The latest iPhone features an A15 chip and improved camera.",
        "MacBook": "The new MacBook has an M2 chip and a 14-inch Retina display.",
    }
    return product_catalog.get(product_name, "Sorry, product not found.")
def check_stock(product_name: str) -> str:
    """Check product stock availability."""
    stock_data = {
        "iPhone": "In stock.",
        "MacBook": "Out of stock.",
    }
    return stock_data.get(product_name, "Stock information unavailable.")

# Initialize the memory saver for single-thread memory
checkpoint = MemorySaver()

# Initialize the language model
llm = ChatOpenAI(model="gpt-4o-mini")

# Create the ReAct agent with tools and memory
tools = [product_info, check_stock]
graph = create_react_agent(model=llm, tools=tools, checkpoint=checkpoint)

# Set up thread configuration for single-thread memory
config = {"configurable": {"thread_id": "thread-3"}}

# User input: initial inquiry about product information
inputs = {"messages": [("user", "Tell me about the new iPhone.")]}
messages = graph.invoke(inputs, config=config)
for message in messages["messages"]:
    print(message.content)

# User input: follow-up inquiry about stock availability (memory recall and dynamic decision-making)
inputs2 = {"messages": [("user", "Is the iPhone in stock?")]}
messages2 = graph.invoke(inputs2, config=config)
for message in messages2["messages"]:
    print(message.content)
```

Penjelasan Langkah-Langkah Kunci:

1. User Input (Initial Inquiry about Product Information):

- The user initially asks for product details: *"Tell me about the new iPhone."*
- The ReAct agent reasons that this is a product inquiry and calls the `product_info` tool with the product name `"iPhone"`.
- The tool retrieves the product information: *"The latest iPhone features an A15 chip and improved camera."*
- The agent stores the query and response in memory under the single-threaded session ID (`"thread-3"`).

2. Tool Call for Product Information:

- The agent triggers the `product_info` tool to retrieve product details from the catalog and returns the result to the user.

3. Memory Storage:

- The agent uses `MemorySaver` to store the query *"Tell me about the new iPhone."* and the corresponding response in the single-thread memory associated with `"thread-3"`. This allows the agent to recall the user's previous query and provide context in future interactions.

4. User Input (Follow-Up Inquiry about Stock):

- The user then asks a follow-up question: *"Is the iPhone in stock?"*
- The agent recalls the previous query from memory and dynamically decides to call the `check_stock` tool to retrieve stock availability information for the iPhone.
- The tool returns the stock status: *"In stock."*
- The agent responds with the stock availability and incorporates memory to reference the earlier conversation.

Rincian Contoh Output:

1. Initial Inquiry (Product Information):

- **User Input:** *"Tell me about the new iPhone."*
- **Agent Response:** *"The latest iPhone features an A15 chip and improved camera."*
- The agent stores this query and response in memory.

```
=====Human
Message=====
=Ceritakan tentang iPhone
baru.=====Ai
Message=====
===
```

iPhone terbaru dilengkapi chip A15 dan kamera yang lebih baik.

2. Follow-Up Inquiry (Stock Availability Check):

- **User Input:** *"Is the iPhone in stock?"*
- **Agent Response:** *"You previously asked about the iPhone. It's currently in stock."*
- The agent recalls the previous query from memory, checks the stock availability using the `check_stock` tool, and provides the personalized response.

```
=====Human
Message=====Apakah
iPhone tersedia
stoknya?=====Ai
Message=====Anda
sebelumnya bertanya tentang iPhone. Saat ini stoknya
tersedia.
```

Cara Kerja Pengambilan Keputusan Dinamis:

1. Reasoning and Acting with Dynamic Tool Calls:

- After the first inquiry, the agent reasons that the user is asking about product details and calls the `product_info` tool.
- When the user follows up with a question about stock availability, the agent dynamically decides to call the `check_stock` tool to get the stock information based on the context of the previous query.
- This dynamic decision-making capability allows the ReAct agent to flexibly respond to different types of queries and call the appropriate tools depending on user needs.

2. Memory Integration:

- The ReAct agent uses `MemorySaver` to recall the user's previous query from the same conversation thread (`"thread-3"`), enabling it to provide contextually aware responses.
- By recalling the previous product inquiry, the agent provides a more seamless and personalized conversation, referencing the iPhone and its availability based on the earlier interaction.

Graph Workflow dengan Dynamic Tool Calls:

Dalam contoh ini, ReAct agent membuat graph dengan pengambilan keputusan dinamis di mana agent dapat menalar (reason) input pengguna dan memutuskan apakah akan memanggil tool atau mengingat memory:

```
stateDiagram-v2
[*] --> Start
Start --> Agent
Agent --> Tools : retrieve product info
Tools --> Memory : store query
```

Memory --> Agent : recall memory
Agent --> Tools : check stock availability
Tools --> Agent : respond with stock info
Agent --> End : end
End --> [*]

- **Agent Node:** Memproses input pengguna dan menentukan apakah akan memanggil tool atau mengingat memory.- **Tools Node:** Menjalankan tools (product_info dan check_stock) berdasarkan langkah penalaran, mengambil detail produk dan memeriksa ketersediaan stok.- **Memory Node:** Menyimpan query pengguna dan respons dalam memory serta mengambilnya kembali saat dibutuhkan untuk interaksi berikutnya.- **End Node:** Agent memberikan respons akhir kepada pengguna dan menyelesaikan percakapan.

Cara Desain ReAct Agent Menangani Query Dinamis:

- **Conditional Logic untuk Tool Calls:** ReAct agent menggunakan pengambilan keputusan dinamis untuk memutuskan tool mana yang akan dipanggil berdasarkan input pengguna dan konteks percakapan. Jika pengguna meminta detail produk, agent memanggil product_info . Jika pengguna melanjutkan dengan pertanyaan tentang stok, agent memanggil check_stock .- **Single-Threaded Memory:** Memory dibatasi pada percakapan (thread) saat ini, memungkinkan agent mengingat interaksi sebelumnya dan memberikan respons yang dipersonalisasi berdasarkan query sebelumnya. Hal ini membuat interaksi lebih menarik dan ramah pengguna.

Contoh 4: ReAct Agent dengan Multi-Step Reasoning dan Dynamic Action Sub-Graphs

Tujuan:

- Kita akan membuat agent yang mampu menalar input kompleks dan melakukan aksi dinamis berdasarkan permintaan pengguna.- Agent akan membuat beberapa keputusan selama fase penalaran (misalnya, memutuskan apakah akan mengambil informasi dari beberapa tool).- Sub-graphs akan menangani task spesifik seperti mengambil informasi dari tools atau melakukan beberapa aksi berdasarkan prompt yang berbeda.

Rencana:

1. **Parent Graph:** Responsible for high-level reasoning and orchestrating multiple sub-graphs.
2. **Reasoning Phase:** Dynamically decide which sub-graphs to call based on user input.
3. **Sub-Graphs:** Handle specific actions, including tool calls or acting on the user's query.
4. **Memory:** Store and recall data in the sub-graphs (optional extension).

Implementasi:

```

#lesson8a.py continued

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START, END
from typing import TypedDict

# Define shared state for the agent and sub-graphs
class ReActAgentState(TypedDict):
    message: str # Shared key between the parent graph and sub-graphs
    action: str # Sub-graph specific key (what action to perform)
    sub_action: str # Additional sub-action to perform in a more complex scenario

# Reasoning Node 1: Determines which action the agent should take based on the user query
def reasoning_node(state: ReActAgentState):
    query = state["message"]
    if "weather" in query:
        return {"action": "fetch_weather"}
    elif "news" in query:
        return {"action": "fetch_news"}
    elif "recommend" in query:
        return {"action": "recommendation", "sub_action": "book"}
    else:
        return {"action": "unknown"}

# Sub-graph for fetching weather information (acting phase)
def weather_subgraph_node(state: ReActAgentState):
    # Simulating a weather tool call
    return {"message": "The weather today is sunny."}

# Sub-graph for fetching news information (acting phase)
def news_subgraph_node(state: ReActAgentState):
    # Simulating a news tool call
    return {"message": "Here are the latest news headlines."}

# Sub-graph for providing a book recommendation (acting phase)
def recommendation_subgraph_node(state: ReActAgentState):
    if state.get("sub_action") == "book":
        return {"message": "I recommend reading 'The Pragmatic Programmer'."}
    else:
        return {"message": "I have no other recommendations at the moment."}

# Build sub-graph for fetching weather information
weather_subgraph_builder = StateGraph(ReActAgentState)
weather_subgraph_builder.add_node("weather_action", weather_subgraph_node)
weather_subgraph_builder.set_entry_point("weather_action")
weather_subgraph = weather_subgraph_builder.compile()

# Build sub-graph for fetching news information
news_subgraph_builder = StateGraph(ReActAgentState)
news_subgraph_builder.add_node("news_action", news_subgraph_node)
news_subgraph_builder.set_entry_point("news_action")
news_subgraph = news_subgraph_builder.compile()

# Build sub-graph for recommendations (e.g., book recommendation)

```

```

recommendation_subgraph_builder = StateGraph(ReActAgentState)
recommendation_subgraph_builder.add_node("recommendation_action", recommendation_subgraph_node)
recommendation_subgraph_builder.set_entry_point("recommendation_action")
recommendation_subgraph = recommendation_subgraph_builder.compile()
# Define dynamic reasoning node in the parent graph to route to the correct sub-graph
def reasoning_state_manager(state: ReActAgentState):
    if state["action"] == "fetch_weather":
        return weather_subgraph
    elif state["action"] == "fetch_news":
        return news_subgraph
    elif state["action"] == "recommendation":
        return recommendation_subgraph
    else:
        return None
# Create the parent graph
parent_builder = StateGraph(ReActAgentState)
parent_builder.add_node("reasoning", reasoning_node)
parent_builder.add_node("action_dispatch", reasoning_state_manager)
# Define edges in the parent graph
parent_builder.add_edge(START, "reasoning")
parent_builder.add_edge("reasoning", "action_dispatch")
# Compile the parent graph
react_agent_graph = parent_builder.compile()
# Test the agent with a weather-related query
inputs_weather = {"message": "What is the weather today?"}
result_weather = react_agent_graph.invoke(inputs_weather)
print(result_weather["message"])
# Test the agent with a news-related query
inputs_news = {"message": "Give me the latest news."}
result_news = react_agent_graph.invoke(inputs_news)
print(result_news["message"])
# Test the agent with a recommendation-related query
inputs_recommendation = {"message": "Can you recommend a good book?"}
result_recommendation = react_agent_graph.invoke(inputs_recommendation)
print(result_recommendation["message"])

```

Penjelasan Konsep Kunci:

1. Shared State Keys (**ReActAgentState**):

- The **message** key is shared between the parent graph and sub-graphs. It contains the user's input and the agent's response.
- The **action** and **sub_action** keys are used by the reasoning node to decide which specific sub-graph should be invoked.

2. Parent Graph:

- The parent graph begins with a **reasoning node** that analyzes the user's input and determines which action to perform.
- The **reasoning_state_manager** then dynamically routes the workflow to the appropriate sub-graph based on the reasoning node's decision (e.g., fetching weather or news).

3. Sub-Graphs for Acting:

- Each sub-graph handles a specific action based on the reasoning phase's result. For example, the **weather sub-graph** simulates fetching weather information, while the **news sub-graph** simulates fetching news headlines.
- The **recommendation sub-graph** is designed to provide specific recommendations (e.g., books) and can be extended to handle more complex reasoning scenarios.

4. Dynamic Routing:

- The parent graph dynamically routes control to the appropriate sub-graph using the **action** key. This approach makes the agent flexible, as it can easily be extended to support additional actions by adding more sub-graphs.

Contoh Output:

Weather Query:

User Input: What is the weather today?

Assistant Response: The weather today is sunny.

News Query:

User Input: Give me the latest news.

Assistant Response: Here are the latest news headlines.

Recommendation Query:

User Input: Can you recommend a good book?

Assistant Response: I recommend reading 'The Pragmatic Programmer'.

Fitur Dynamic ReAct Agent:

1. Modular Sub-Graphs:

- Each sub-graph is modular and can be easily extended or replaced with more complex functionality (e.g., real API calls for weather or news).

2. Dynamic Reasoning:

- The parent graph can dynamically decide which action to perform based on the user's input, making the agent flexible and adaptive to a wide variety of queries.

3. Expandable:

- This structure is highly expandable. For example, you can add more sub-graphs for other tasks like retrieving calendar events, setting reminders, or even providing personalized recommendations based on user history.

4. Memory:

- You can integrate memory (like `MemorySaver`) to store previous user interactions and provide personalized responses based on the user's preferences or past queries.

Contoh 5: Advanced ReAct Agent dengan Multiple Sub-Graphs dan Contextual Memory

Tujuan:

- Dalam contoh ini, kita akan membuat advanced ReAct agent yang tidak hanya menalar dan bertindak secara dinamis tetapi juga menggabungkan contextual memory.- Agent akan menggunakan memory untuk mengingat interaksi sebelumnya, sehingga dapat memberikan respons yang dipersonalisasi berdasarkan query sebelumnya.- Proses penalaran akan lebih kompleks, mempertimbangkan baik input pengguna saat ini maupun riwayat pengguna.- Beberapa sub-graphs akan menangani aspek yang berbeda, seperti melakukan query ke tools eksternal (misalnya, cuaca, berita) dan mengelola preferensi pengguna atau interaksi sebelumnya.

Rencana:

1. **Parent Graph:** Responsible for managing user interactions and invoking sub-graphs.
2. **Contextual Memory:** Memory will store past interactions, and reasoning will be adapted based on previous conversations.
3. **Sub-Graphs:** Handle specific actions, such as fetching data from tools (e.g., APIs) or making decisions based on context.
4. **Complex Reasoning:** The agent will reason not just on current input but also on past user interactions.

Implementasi:

```
#lesson8d.py

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.checkpoint.memory import MemorySaver
from typing import TypedDict

# Define shared state for the agent and sub-graphs, including memory
class ReActAgentState(TypedDict):
    message: str    # Current user message
    action: str     # Action determined by reasoning
    sub_action: str # Sub-action determined by reasoning
    memory: dict    # Memory of past interactions
```

```

# Reasoning node that adapts based on user input and past interactions
def reasoning_node(state: ReActAgentState):
    query = state["message"]
    # Check if there is any past context stored in memory
    past_interactions = state.get("memory", {})
    # Personalized decision-making based on past interactions
    if "weather" in query:
        return {"action": "fetch_weather"}
    elif "news" in query:
        return {"action": "fetch_news"}
    elif "recommend" in query:
        if past_interactions.get("favorite_genre") == "science":
            return {"action": "recommendation", "sub_action": "science_book"}
        else:
            return {"action": "recommendation", "sub_action": "general_book"}
    else:
        return {"action": "unknown"}
# Sub-graph for fetching weather information
def weather_subgraph_node(state: ReActAgentState):
    return {"message": "The weather today is sunny."}
# Sub-graph for fetching news information
def news_subgraph_node(state: ReActAgentState):
    return {"message": "Here are the latest news headlines."}
# Sub-graph for providing a general book recommendation
def general_recommendation_node(state: ReActAgentState):
    return {"message": "I recommend reading 'The Pragmatic Programmer.'"}
# Sub-graph for providing a science book recommendation based on user preferences
def science_recommendation_node(state: ReActAgentState):
    return {"message": "Since you like science, I recommend 'A Brief History of Time' by Stephen Hawking."}
# Sub-graph for updating memory (e.g., user preference updates)
def update_memory_node(state: ReActAgentState):
    if "recommend" in state["message"]:
        # Example of updating user's favorite genre in memory
        state["memory"]["favorite_genre"] = "science"
    return state
# Build sub-graphs for actions and memory
weather_subgraph_builder = StateGraph(ReActAgentState)
weather_subgraph_builder.add_node("weather_action", weather_subgraph_node)
weather_subgraph_builder.set_entry_point("weather_action")
weather_subgraph = weather_subgraph_builder.compile()
news_subgraph_builder = StateGraph(ReActAgentState)
news_subgraph_builder.add_node("news_action", news_subgraph_node)
news_subgraph_builder.set_entry_point("news_action")

```

```

news_subgraph = news_subgraph_builder.compile()
general_recommendation_builder = StateGraph(ReActAgentState)
general_recommendation_builder.add_node("general_recommendation_action",
general_recommendation_node)
general_recommendation_builder.set_entry_point("general_recommendation_actio
n")
general_recommendation_subgraph = general_recommendation_builder.compile()
science_recommendation_builder = StateGraph(ReActAgentState)
science_recommendation_builder.add_node("science_recommendation_action",
science_recommendation_node)
science_recommendation_builder.set_entry_point("science_recommendation_actio
n")
science_recommendation_subgraph = science_recommendation_builder.compile()
# Memory update sub-graph
memory_update_builder = StateGraph(ReActAgentState)
memory_update_builder.add_node("update_memory_action",
update_memory_node)
memory_update_builder.set_entry_point("update_memory_action")
memory_update_subgraph = memory_update_builder.compile()
# Define dynamic reasoning node in the parent graph to route to the correct sub-
graph
def reasoning_state_manager(state: ReActAgentState):
    if state["action"] == "fetch_weather":
        return weather_subgraph
    elif state["action"] == "fetch_news":
        return news_subgraph
    elif state["action"] == "recommendation":
        if state["sub_action"] == "science_book":
            return science_recommendation_subgraph
        else:
            return general_recommendation_subgraph
    else:
        return None
# Create the parent graph
parent_builder = StateGraph(ReActAgentState)
parent_builder.add_node("reasoning", reasoning_node)
parent_builder.add_node("action_dispatch", reasoning_state_manager)
parent_builder.add_node("update_memory", memory_update_subgraph)
# Define edges in the parent graph
parent_builder.add_edge(START, "reasoning")
parent_builder.add_edge("reasoning", "action_dispatch")
parent_builder.add_edge("action_dispatch", "update_memory")
# Compile the parent graph
react_agent_graph = parent_builder.compile()
# Initialize memory

```

```

checkpointer = MemorySaver()
# Test the agent with a weather-related query (memory will not affect this)
inputs_weather = {"message": "What is the weather today?", "memory": {}}
result_weather = react_agent_graph.invoke(inputs_weather)
print(result_weather["message"])
# Test the agent with a recommendation query (first time, no memory)
inputs_recommendation_first = {"message": "Can you recommend a good book?",
"memory": {}}
result_recommendation_first =
react_agent_graph.invoke(inputs_recommendation_first)
print(result_recommendation_first["message"])
# Simulate memory update after the recommendation (user prefers science books)
inputs_recommendation_second = {"message": "Can you recommend another
book?", "memory": {"favorite_genre": "science"}}
result_recommendation_second =
react_agent_graph.invoke(inputs_recommendation_second)
print(result_recommendation_second["message"])

```

Test Output: Weather Query: User Input:

Bagaimana cuaca hari ini? Assistant

Response: Cuaca hari ini cerah.

First Recommendation Query (Without Memory): User Input: Bisakah kamu merekomendasikan buku yang bagus? Assistant Response: Saya merekomendasikan membaca 'The Pragmatic Programmer'.

Second Recommendation Query (With Memory): User Input: Bisakah kamu merekomendasikan buku lain? Assistant Response: Karena kamu menyukai sains, saya merekomendasikan 'A Brief History of Time' karya Stephen Hawking

Konsep Kunci:

1. Memory Integration (**MemorySaver**):

- We integrate memory by using **MemorySaver** to recall past user preferences (such as favorite book genres) and update them dynamically during the conversation.
- For instance, the agent updates the user's memory based on their interactions and provides personalized recommendations in future conversations.

2. Reasoning and Actions:

- The reasoning node (**reasoning_node**) dynamically chooses actions based on the user's input and previous memory. If the user previously indicated a

- preference for science books, the agent recommends books from that genre.
- Sub-graphs handle specific actions, such as providing general book recommendations or weather updates, and update the memory after performing those actions.

3. Sub-Graphs for Actions:

- The agent includes multiple sub-graphs for actions like fetching weather information, providing news, and making personalized book recommendations.
- Each sub-graph is modular, allowing us to easily extend or replace functionality.

4. Updating Memory:

- After the reasoning and action phases, the agent uses the **memory update sub-graph** to modify the user's memory based on the current conversation. This ensures that future interactions are influenced by past user preferences.

Contoh 6: Dynamic Pricing Agent dengan LLM Reasoning dan External Tool Calls

Ikhtisar:

Dalam contoh ini, kita membuat Dynamic Pricing Agent yang menggunakan LLM (Large Language Model) untuk penalaran, dan terintegrasi dengan mock competitor pricing API untuk mengambil data harga kompetitor secara real-time. Agent menyesuaikan harganya secara dinamis berdasarkan permintaan pasar dan harga kompetitor. Agent ini mengikuti framework ReAct, yang berarti agent akan:

1. **Reason:** The LLM will reason about the pricing strategy based on the demand and competitor pricing.
2. **Act:** If necessary, the agent will call an external tool (mock API) to fetch competitor pricing.
3. **Observe:** The LLM will then observe the results from the tool call and either reason again or finalize the product price.

Ini adalah contoh bagaimana agent berbasis LLM dapat terintegrasi dengan tools dunia nyata untuk pengambilan keputusan dinamis.

Teknologi Kunci:

1. **LLM for Reasoning:** The agent uses the LLM to decide on the pricing strategy.
2. **External Tool (Mock API):** A mock API simulates fetching competitor pricing data.
3. **StateGraph:** The agent's workflow is managed using a state graph that flows between reasoning, acting, and observing.

Implementasi:

1. Mendefinisikan Environment Pricing Agent

Kita perlu mendefinisikan environment dan tools yang akan digunakan agent. Untuk ini, kita akan menggunakan model LangChain (seperti GPT-4) dan mock APIs.

```
#lesson8e.py continued

from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
import requests

# Define the mock APIs for demand and competitor pricing
def get_demand_data(product_id: str) -> dict:
    """Mock demand API to get demand data for a product."""
    # In real use, replace with an actual API call
    return {"product_id": product_id, "demand_level": "high"}
def get_competitor_pricing(product_id: str) -> dict:
    """Mock competitor pricing API."""
    # In real use, replace with an actual API call
    return {"product_id": product_id, "competitor_price": 95.0}
# List of tools for the agent to call
tools = [get_demand_data, get_competitor_pricing]
```

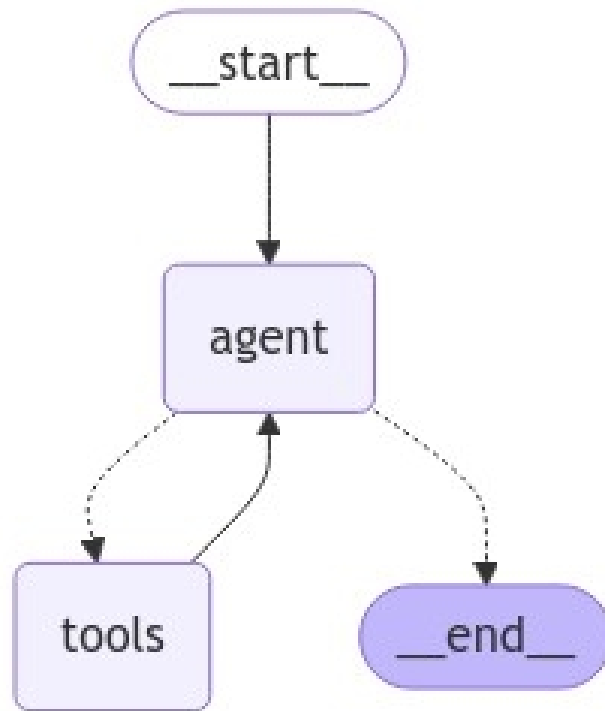
2. Membuat ReAct Agent dengan Tool Integration

Menggunakan pattern ReAct dari LangGraph, kita membuat agent yang berinteraksi dengan tools untuk mengambil data pasar dan memutuskan harga.

```
#lesson8e.py continued

# Define the agent using the ReAct pattern
model = ChatOpenAI(model="gpt-4")
# Create the ReAct agent with tools for demand and competitor pricing
graph = create_react_agent(model, tools=tools)
# Define the initial state of the agent
initial_messages = [
    ("system", "You are an AI agent that dynamically adjusts product prices based on market demand and competitor prices."),
    ("user", "What should be the price for product ID '12345'?")
]
# Stream agent responses and decisions
inputs = {"messages": initial_messages}
# Simulate the agent reasoning, acting (calling tools), and observing
for state in graph.stream(inputs, stream_mode="values"):
    message = state["messages"][-1] # Get the latest message in the interaction
    if isinstance(message, tuple):
        print(message)
    else:
        message.pretty_print() # Print the response from the agent
```

Graph yang dihasilkan terlihat seperti ini:



3. Reasoning Logic dalam LLM

Agent akan memanggil demand dan competitor pricing APIs sebagai tools, kemudian menalar berdasarkan hasilnya. Sebagai contoh, jika harga kompetitor lebih rendah dan permintaan tinggi, agent mungkin menyarankan untuk menurunkan harga.

LLM akan menghasilkan jenis penalaran berikut:

1. **Fetch Competitor Data:** The agent uses a tool (competitor pricing API) to get competitor prices.
2. **Evaluate Market Demand:** The agent calls the demand API to assess current demand levels.
3. **Adjust Pricing:** Based on the demand and competitor data, the LLM decides whether to increase or decrease the price.

Contoh penalaran yang dihasilkan oleh LLM:

- "Harga kompetitor lebih rendah, dan permintaan tinggi, jadi saya akan sedikit menurunkan harga kita agar tetap kompetitif."

4. Mengamati dan Mengevaluasi Ulang

Agent akan mengamati hasilnya dan menyempurnakan pengambilan keputusannya. Jika harga kompetitor berubah setelah pemanggilan API, agent akan mengevaluasi ulang dan menyesuaikan strategi harganya lagi.

Contoh 7: Sentiment Analysis Agent Menggunakan LLM dan Custom Sentiment Analysis Tools

Ikhtisar:

Dalam contoh ini, kita membuat Sentiment Analysis Agent yang menggunakan LLM (Large Language Model) untuk memproses feedback pelanggan dan terintegrasi dengan custom sentiment analysis tools untuk mengklasifikasikan feedback sebagai positif, netral, atau negatif. Berdasarkan sentimen tersebut, agent menghasilkan respons yang sesuai menggunakan framework ReAct (Reason, Act, Observe). Agent menalar feedback pelanggan, memanggil sentiment analysis tool, dan memberikan respons.

Teknologi Kunci:

1. **LLM for Reasoning:** The LLM processes customer feedback and decides whether to invoke the sentiment analysis tool or respond directly.
2. **Sentiment Analysis Tool:** Custom sentiment analysis logic (using `TextBlob`) to determine whether the sentiment is positive, neutral, or negative.
3. **StateGraph:** The agent's workflow is managed using a state graph that integrates reasoning, acting (sentiment analysis), and responding.

Implementasi:

1. Mendefinisikan Agent State:

State melacak feedback pelanggan dan urutan message dari LLM.

```
#lesson8e.py continued

from typing import Annotated, Sequence, TypedDict
from langchain_core.messages import BaseMessage
from langgraph.graph.message import add_messages
# Define the state for the agent
class AgentState(TypedDict):
    """The state of the agent."""
    # `add_messages` is a reducer that manages the message sequence.
    messages: Annotated[Sequence[BaseMessage], add_messages]
```

2. Mendefinisikan Sentiment Analysis Tool:

Kita mengimplementasikan custom tool menggunakan `TextBlob` untuk menganalisis sentimen feedback pelanggan.

```
#lesson8e.py continued

from langchain_core.tools import tool
from textblob import TextBlob
@tool
def analyze_sentiment(feedback: str) -> str:
    """Analyze customer feedback sentiment with custom logic."""
    analysis = TextBlob(feedback)
    if analysis.sentiment.polarity > 0.5:
        return "positive"
```



```

elif analysis.sentiment.polarity == 0.5:
    return "neutral"
else:
    return "negative"

```

3. Mendefinisikan Response Generation Tool:

Setelah sentimen dianalisis, kita menggunakan tool kedua untuk menghasilkan respons berdasarkan sentimen tersebut.

```

#lesson8e.py continued

@tool
def respond_based_on_sentiment(sentiment: str) -> str:
    """Respond to the customer based on the analyzed sentiment."""
    if sentiment == "positive":
        return "Thank you for your positive feedback!"
    elif sentiment == "neutral":
        return "We appreciate your feedback."
    else:
        return "We're sorry to hear that you're not satisfied. How can we help?"

```

4. Mengikat (Bind) Tools ke LLM:

Kita mengikat sentiment analysis tool dan response tool ke LLM, sehingga LLM dapat memanggil tools ini saat dibutuhkan.

```

#lesson8e.py continued

from langchain_openai import ChatOpenAI
from langchain_core.messages import ToolMessage, SystemMessage
tools = [analyze_sentiment, respond_based_on_sentiment]
# Initialize the LLM
llm = ChatOpenAI(model="gpt-4o-mini")
llm = llm.bind_tools(tools)
# Create a dictionary of tools by their names for easy lookup
tools_by_name = {tool.name: tool for tool in tools}

```

5. Mendefinisikan Tool Execution Node:

Kita mendefinisikan node untuk menjalankan tools saat LLM menentukan bahwa sebuah tool perlu dipanggil.

```

#lesson8e.py continued

import json
from langchain_core.messages import ToolMessage
# Tool node to handle sentiment analysis and response generation
def tool_node(state: AgentState):
    outputs = []
    for tool_call in state["messages"][-1].tool_calls:
        tool_result = tools_by_name[tool_call["name"]].invoke(tool_call["args"])

```

```

        outputs.append(
            ToolMessage(
                content=json.dumps(tool_result),
                name=tool_call["name"],
                tool_call_id=tool_call["id"],
            )
        )
    return {"messages": outputs}

```

6. Mendefinisikan LLM Reasoning Node:

Node ini menggunakan LLM untuk menalar feedback pelanggan dan baik merespons langsung atau memanggil tool.

```

#lesson8e.py continued

from langchain_core.runnables import RunnableConfig
# LLM reasoning node to process the feedback and call tools if needed
def call_model(state: AgentState, config: RunnableConfig):
    system_prompt = SystemMessage(
        content="You are a helpful assistant tasked with responding to customer feedback."
    )
    response = llm.invoke([system_prompt] + state["messages"], config)
    return {"messages": [response]}

```

7. Mendefinisikan State Transitions:

Kita mendefinisikan kondisi yang menentukan apakah LLM harus terus memanggil tools atau memberikan respons akhir.

```

#lesson8e.py continued

def should_continue(state: AgentState):
    last_message = state["messages"][-1]
    # If there is no tool call, then we finish
    if not last_message.tool_calls:
        return "end"
    else:
        return "continue"

```

8. Membangun StateGraph:

Kita membangun state graph untuk mengatur fase penalaran, pemanggilan tool, dan observasi.

```

#lesson8e.py continued

from langgraph.graph import StateGraph, END, START
workflow = StateGraph(AgentState)
workflow.add_node("agent", call_model)
workflow.add_node("tools", tool_node)

```

```

workflow.set_entry_point("agent")
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "continue": "tools",
        "end": END,
    },
)
workflow.add_edge("tools", "agent")
# Compile the graph
graph = workflow.compile()

```

9. Memvisualisasikan Graph (Opsional):

Anda dapat memvisualisasikan workflow graph untuk pemahaman alur yang lebih baik.

```

#lesson8e.py continued

import os
from display_graph import display_graph
display_graph(graph, file_name=os.path.basename(__file__))

```

10. Menginisialisasi Agent dan Menjalankannya:

Kita menginisialisasi agent dengan feedback pelanggan dan menjalankan agent untuk menganalisis sentimen dan menghasilkan respons.

```

#lesson8e.py continued

# Initialize the agent's state with the user's feedback
initial_state = {"messages": [("user", "The product was great but the delivery was poor.")]}
# Helper function to print the conversation
def print_stream(stream):
    for s in stream:
        message = s["messages"][-1]
        if isinstance(message, tuple):
            print(message)
        else:
            message.pretty_print()
# Run the agent
print_stream(graph.stream(initial_state, stream_mode="values"))

```

Penjelasan:

1. Sentiment State:

- The state tracks the messages exchanged between the user and the agent, including feedback, sentiment analysis, and responses.

2. LLM-Based Reasoning:

- The LLM processes the customer feedback and determines whether a tool needs to be invoked for sentiment analysis or if it can generate a response directly.

3. Sentiment Analysis and Response Generation:

- The sentiment analysis tool (`analyze_sentiment`) classifies feedback as positive, neutral, or negative.
- Based on the result, the response tool (`respond_based_on_sentiment`) generates a response tailored to the customer's sentiment.

4. StateGraph:

- The graph manages the transitions between the LLM reasoning, sentiment analysis, and response generation, ensuring a structured process for the agent to follow.

Contoh Output:

Dengan feedback "The product was great but the delivery was poor", agent akan menganalisis sentimen dan menghasilkan output berikut:

```
=====Human
Message=====

Produknya bagus tapi pengirimannya
buruk.=====Ai
Message=====
Tool Calls:

analyze_sentiment (call_ztXfoVKMspW1uekBhP9KWwAJ)Call ID:
call_ztXfoVKMspW1uekBhP9KWwAJ
  Args:
feedback: The product was great but the delivery was poor.
respond_based_on_sentiment (call_FEVrtQYE6bXLjpxftMLBhC59)Call ID:
call_FEVrtQYE6bXLjpxftMLBhC59
  Args:
sentiment:
mixed=====Tool
Message=====Na
me: respond_based_on_sentiment

"Kami turut prihatin karena Anda tidak puas. Bagaimana kami bisa
membantu?" =====Ai
Message=====

Kami turut prihatin karena Anda tidak puas. Bagaimana kami bisa membantu?
```

Contoh 8: Personalized Product Recommendation Agent Menggunakan LLM dan Memory

Ikhtisar:

Contoh ini mendemonstrasikan Personalized Product Recommendation Agent yang menggunakan LLM (Large Language Model) untuk penalaran dan mengintegrasikan sistem memory untuk melacak preferensi pengguna dari waktu ke waktu. Agent memberikan rekomendasi produk berdasarkan riwayat interaksi pengguna dan preferensi saat ini. Agent ini mengikuti framework ReAct (Reason, Act, Observe), di mana LLM menalar preferensi pengguna, memanggil tools (misalnya, product search atau recommendation engines), dan memperbarui memory pengguna untuk mempersonalisasi interaksi di masa depan.

Teknologi Kunci:

1. **LLM for Reasoning:** The LLM processes user input and decides whether to recommend a product directly or query a recommendation engine.
2. **Memory Integration:** The agent uses a memory system to store user preferences and adapt its recommendations based on past interactions.
3. **StateGraph:** The agent's workflow is managed using a state graph to handle reasoning, acting (recommendation generation), and memory updates.

Implementasi:

1. Mendefinisikan Recommendation State:

Kita mendefinisikan state schema yang melacak preferensi pengguna saat ini, proses penalaran, dan produk yang direkomendasikan.

```
#lesson8e.py continued

from typing import TypedDict
# Define the state for product recommendation
class RecommendationState(TypedDict):
    user_id: str    # User identifier
    preference: str  # User's current preference (e.g., genre, category)
    reasoning: str   # Reasoning process from LLM
    recommendation: str # Final product recommendation
    memory: dict     # User memory to store preferences
```

2. Mendefinisikan Recommendation Tools:

Kita membuat tool untuk menghasilkan rekomendasi produk berdasarkan preferensi pengguna.

```
#lesson8e.py continued

from langchain_core.tools import tool
# Tool function: Recommend a product based on the user's preference
@tool
def recommend_product(preference: str) -> str:
    """Recommend a product based on the user's preferences."""
```

```

product_db = {
    "science": "I recommend 'A Brief History of Time' by Stephen Hawking.",
    "technology": "I recommend 'The Innovators' by Walter Isaacson.",
    "fiction": "I recommend 'The Alchemist' by Paulo Coelho."
}

return product_db.get(preference, "I recommend exploring our latest products!")

```

3. Mengikat (Bind) Tools ke LLM:

Kita mengikat product recommendation tool ke LLM, sehingga LLM dapat memanggil tool tersebut saat dibutuhkan.

```

#lesson8e.py continued

from langchain_openai import ChatOpenAI
from langchain_core.messages import ToolMessage, SystemMessage
# Initialize the LLM
llm = ChatOpenAI(model="gpt-4o-mini")
llm = llm.bind_tools([recommend_product])

```

4. Mendefinisikan Memory Update Function:

Kita mengimplementasikan fungsi untuk memperbarui memory pengguna dengan preferensi mereka saat ini, sehingga interaksi di masa depan dipersonalisasi berdasarkan pilihan sebelumnya.

```

#lesson8e.py continued

# Tool function: Update the user's memory with the latest preference
def update_memory(state: RecommendationState):
    # Store the user's preference in the memory
    state["memory"][state["user_id"]] = state["preference"]
    return state

```

5. Mendefinisikan Tool Execution Node:

Kita mendefinisikan node yang menjalankan recommendation tool saat LLM menentukan bahwa rekomendasi produk dibutuhkan.

```

#lesson8e.py continued

import json
from langchain_core.messages import ToolMessage
# Tool node to handle product recommendation
def tool_node(state: RecommendationState):
    tool_result = recommend_product.invoke({"preference": state["preference"]})
    return {
        "messages": [
            ToolMessage(
                content=json.dumps(tool_result),
                name="recommend_product",

```

```

        tool_call_id="recommendation_call"
    )
]
}

```

6. Mendefinisikan LLM Reasoning Node:

Node ini menggunakan LLM untuk menalar input pengguna, memutuskan apakah rekomendasi dibutuhkan, dan memanggil recommendation tool jika perlu.

```

#lesson8e.py continued

from langchain_core.runnables import RunnableConfig
# LLM reasoning node to process user input and generate product recommendations
def call_model(state: RecommendationState, config: RunnableConfig):
    system_prompt = SystemMessage(
        content=f"You are a helpful assistant. Recommend a product based on the user's preference for {state['preference']}."
    )
    response = llm.invoke([system_prompt], config)
    state["reasoning"] = response["message"]
    return {"messages": [response]}

```

7. Mendefinisikan State Transitions:

Kita mendefinisikan kondisi yang menentukan apakah LLM harus memanggil product recommendation tool atau merespons langsung.

```

#lesson8e.py continued

# Conditional function to determine whether to call the tool or end
def should_continue(state: RecommendationState):
    last_message = state["reasoning"]
    if "recommend a product" in last_message:
        return "continue"
    else:
        return "end"

```

8. Membangun StateGraph:

Kita membangun state graph yang mengatur fase penalaran, pembuatan rekomendasi, dan pembaruan memory.

```

#lesson8e.py continued

from langgraph.graph import StateGraph, END, START
workflow = StateGraph(RecommendationState)
workflow.add_node("agent", call_model)
workflow.add_node("tools", tool_node)

```

```

workflow.add_node("memory", update_memory)
workflow.set_entry_point("agent")
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "continue": "tools",
        "end": END,
    },
)
workflow.add_edge("tools", "memory")
workflow.add_edge("memory", "agent")
# Compile the graph
graph = workflow.compile()

```

9. Menginisialisasi Agent dan Menjalankannya:

Kita menginisialisasi agent dengan preferensi dan memory pengguna, lalu menjalankan agent untuk memberikan rekomendasi produk yang dipersonalisasi.

```

#lesson8e.py continued

# Initialize the agent's state with the user's preference and memory
initial_state = {
    "user_id": "user123",
    "preference": "science",
    "reasoning": "",
    "recommendation": "",
    "memory": {}
}

# Run the agent
result = graph.invoke(initial_state)

# Display the final result
print(f"Reasoning: {result['reasoning']}")
print(f"Product Recommendation: {result['messages'][-1].content}")
print(f"Updated Memory: {result['memory']}")

```

Penjelasan:

1. Recommendation State:

- The state tracks the user's preference, the reasoning process, the final product recommendation, and memory updates.

2. LLM-Based Reasoning:

- The LLM processes the user's preference and determines whether to invoke the recommendation tool or generate a direct response.

3. **Memory Integration:**

- The agent updates the user's memory with their preferences, so future recommendations are personalized based on their history.

4. **Product Recommendation:**

- The recommendation tool suggests products based on the user's preferences (e.g., science, technology, fiction).

5. **StateGraph:**

- The state graph manages the transitions between reasoning, recommendation generation, and memory updates, ensuring a structured workflow.

Contoh Output:

Dengan preferensi pengguna "science", agent akan menganalisis input dan memberikan rekomendasi berikut:

Reasoning: Berdasarkan preferensi Anda untuk sains, saya merekomendasikan 'A Brief History of Time' karya Stephen Hawking. Product Recommendation: Saya merekomendasikan 'A Brief History of Time' karya Stephen Hawking. Updated Memory: {user123: 'science'}

Chapter 9

Human-in-the-Loop Agents: Menggabungkan Human Feedback

for Smarter Decision-Making

Pendahuluan

Seiring AI agents semakin terintegrasi ke dalam proses bisnis, kebutuhan akan human feedback tetap krusial, terutama pada task yang sensitif atau kompleks. Arsitektur "Human-in-the-Loop" (HITL) memungkinkan sistem AI untuk berhenti sejenak, meminta persetujuan atau feedback manusia, dan melanjutkan berdasarkan input manusia. Bab ini berfokus pada mengintegrasikan interaksi manusia dengan AI agents yang dibangun menggunakan LangGraph, menambahkan lapisan fleksibilitas dan keamanan pada workflow otomatis.

Dalam bab ini, kita akan membahas:

1. **What is Human-in-the-Loop?**
2. **Core Concepts: Breakpoints, Checkpoints, and State Editing**
3. **Step-by-Step: Implementing Simple Breakpoints**
4. **Dynamic Breakpoints and Conditional Interruptions**
5. **Reviewing and Modifying Tool Calls**
6. **Waiting for User Input**
7. **Example Projects with Increasing Complexity**

8. Exercises and Quizzes

9.1 Apa itu Human-in-the-Loop?

Human-in-the-Loop (HITL) melibatkan pengawasan manusia dalam workflow AI, memungkinkan agent untuk berhenti sejenak di titik-titik penting, meminta feedback, dan menyesuaikan perilakunya. Hal ini sangat berguna dalam skenario di mana kesalahan, tindakan sensitif, atau hasil yang tidak pasti mungkin terjadi. Misalnya, agent mungkin memerlukan persetujuan manusia sebelum melakukan transaksi keuangan, menyelesaikan laporan, atau berinteraksi dengan data sensitif.

Proses ini meningkatkan keandalan, memastikan akuntabilitas, dan memungkinkan manusia untuk melakukan intervensi bila diperlukan.

HITL sangat bermanfaat dalam:

- **Transaksi Keuangan:** Mengotomatisasi workflow keuangan dapat menimbulkan risiko jika tidak dikelola dengan baik. HITL dapat diterapkan pada sistem yang memproses pembayaran, di mana agent berhenti sejenak untuk meminta persetujuan manusia atas transaksi yang besar atau tidak biasa. Ini mencegah penipuan atau kesalahan pengelolaan dana.
Contoh: Sebuah AI agent yang mengotomatisasi pemrosesan payroll dapat menandai dan menghentikan sementara setiap transaksi di atas jumlah tertentu untuk persetujuan manual.- **Content Moderation:** AI agent yang bertugas menghasilkan konten (misalnya, laporan, artikel, atau postingan media sosial) dapat berhenti sejenak untuk peninjauan manusia guna memverifikasi kesesuaian atau akurasi konten sebelum dipublikasikan.
Contoh: Sebuah content generation agent yang meninjau materi pemasaran mungkin memerlukan verifikasi manusia untuk konten yang berisi referensi politik sensitif, memastikan kesesuaiannya dengan pedoman perusahaan sebelum dirilis.- **Dokumentasi Legal:** HITL sangat penting dalam workflow legal, di mana kontrak atau dokumen legal yang dihasilkan AI harus ditinjau oleh manusia sebelum difinalisasi.
Contoh: Sebuah AI agent yang menyusun kontrak dapat membuat draf pertama lalu berhenti, memungkinkan seorang ahli hukum untuk meninjau, mengedit, dan menyetujui dokumen tersebut.

HITL meningkatkan keandalan dan akuntabilitas AI agent, memastikan bahwa manusia tetap memegang kendali atas keputusan-keputusan penting.

9.2 Konsep Inti: Breakpoint, Checkpoint, dan State Editing

Dalam LangGraph, HITL agent dapat disusun berdasarkan beberapa konsep kunci yang memungkinkan manusia melakukan intervensi dalam workflow AI secara dinamis.

Konsep-konsep terpenting meliputi:

1. **Breakpoints:** These are intentional pause points in the graph's execution where the agent halts its operation until human feedback is received. Breakpoints provide a way for humans to step in, review the current state, and either approve or adjust the agent's course of action before it proceeds.
2. **Checkpoints:** These are "save points" in the agent's workflow, which capture the current state of the system. If the agent encounters a failure or requires a human to intervene, it can resume execution from the last checkpoint, avoiding the need to restart from the beginning.
3. **State Editing:** This allows users to modify the state of the workflow dynamically while it is paused. During a HITL pause, users can adjust input data, modify parameters, or change the agent's course of action. This is particularly useful in scenarios where human judgment is needed to tweak the agent's behavior in real time.

Mari kita bahas masing-masing dengan contoh praktis.

9.3 Contoh 1: Menerapkan Breakpoint Sederhana

Sebuah breakpoint memungkinkan graph untuk berhenti dan menunggu persetujuan manusia sebelum melanjutkan. Mari kita bangun contoh sederhana di mana agent berhenti sejenak sebelum menjalankan tindakan akhir.

```
#lesson9a.py
import os
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
```

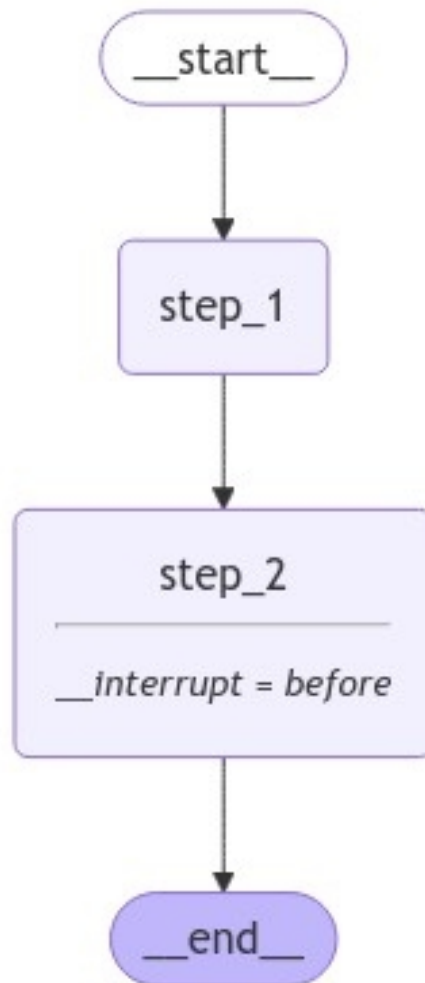
```

from langgraph.checkpoint.memory import MemorySaver
from display_graph import display_graph
# Define the state structure
class State(TypedDict):
    input: str
# Define node functions
def step_1(state: State):
    print("--- Step 1 ---")
    return state
def step_2(state: State):
    print("--- Step 2 ---")
    return state
# Build the graph
builder = StateGraph(State)
builder.add_node("step_1", step_1)
builder.add_node("step_2", step_2)
# Define flow
builder.add_edge(START, "step_1")
builder.add_edge("step_1", "step_2")
builder.add_edge("step_2", END)
# Set up memory and breakpoints
memory = MemorySaver()
graph = builder.compile(checkpointer=memory, interrupt_before=
["step_2"])
# Display the graph
display_graph(graph, file_name=os.path.basename(__file__))

```

Dalam contoh ini:

- **Langkah 1 dijalankan.** - **Graph berhenti sejenak sebelum menjalankan Langkah 2, menunggu input pengguna.**



Menjalankan Graph:

```
#lesson9a.py continued

config = {"configurable": {"thread_id": "thread-1"}}
initial_input = {"input": "Hello, LangGraph!"}
thread = {"configurable": {"thread_id": "1"}}
for event in graph.stream(initial_input, thread, stream_mode="values"):
    print(event)
user_approval = input("Do you approve to continue to Step 2? (yes/no): ")
if user_approval.lower() == "yes":
    for event in graph.stream(None, thread, stream_mode="values"):
        print(event)
else:
    print("Execution halted by user.")
```

9.3.1 Contoh 2: Contoh Praktis Content Moderation

Perhatikan sebuah workflow content moderation di mana sebuah agent membuat draf konten untuk sebuah postingan blog. Sebelum dipublikasikan, workflow berhenti sejenak, memungkinkan seorang editor manusia untuk meninjau draf tersebut. Jika editor menemukan kesalahan atau perlu penyesuaian, mereka dapat memodifikasi draf secara langsung dalam state yang dihentikan sementara dan menyetujui perubahan sebelum melanjutkan.

Dalam contoh ini, kita akan membuat sebuah workflow content moderation menggunakan LangGraph di mana sebuah AI agent menghasilkan draf postingan blog. Workflow ini mencakup tahapan-tahapan berikut:

1. The agent generates a draft based on the input topic using a language model (OpenAI).
2. The system pauses at a **breakpoint**, allowing a human editor to review the draft and make necessary changes.
3. Once the editor approves the content, the agent proceeds to publish it.

Ini adalah contoh sempurna dari Human-in-the-Loop (HITL), di mana pengawasan manusia sangat penting untuk memastikan kualitas dan kesesuaian konten sebelum publikasi. Workflow semacam ini dapat digunakan dalam sistem manajemen konten, pembuatan artikel berita, atau skenario apa pun di mana konten yang dihasilkan harus sesuai dengan pedoman atau sensitivitas tertentu.

```
#lesson9c.py continued

import os
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, END, START
from langgraph.checkpoint.memory import MemorySaver
from langchain_openai import ChatOpenAI
from display_graph import display_graph

# Set up your OpenAI API key
openai_api_key = os.getenv("OPENAI_API_KEY")

# Initialize the OpenAI model
llm = ChatOpenAI(
    api_key=openai_api_key,
    model="gpt-4o-mini",      # Specify the model
    temperature=0.7,         # Adjust for creativity in output
```

```

        max_tokens=50          # Set maximum token length for responses
    )
    # Define the state structure
    class State(TypedDict):
        input: str
        draft_content: str
    # Define node functions
    def create_draft(state: State):
        print("--- Generating Draft with ChatOpenAI ---")
        # Prepare the prompt for generating the blog content
        prompt = f"Write a blog post on the topic: {state['input']}"
        # Call the LangChain ChatOpenAI instance to generate the draft
        response = llm.invoke([{"role": "user", "content": prompt}])
        # Extract the generated content
        state["draft_content"] = response.content
        print(f"Generated Draft:\n{state['draft_content']}")
        return state
    def review_draft(state: State):
        print("--- Reviewing Draft ---")
        print(f"Draft for review:\n{state['draft_content']}")
        return state
    def publish_content(state: State):
        print("--- Publishing Content ---")
        print(f"Published Content:\n{state['draft_content']}")
        return state
    # Build the graph
    builder = StateGraph(State)
    builder.add_node("create_draft", create_draft)
    builder.add_node("review_draft", review_draft)
    builder.add_node("publish_content", publish_content)
    # Define flow
    builder.add_edge(START, "create_draft")
    builder.add_edge("create_draft", "review_draft")
    builder.add_edge("review_draft", "publish_content")
    builder.add_edge("publish_content", END)
    # Set up memory and breakpoints
    memory = MemorySaver()

```



```

graph = builder.compile(checkpointer=memory, interrupt_before=["publish_content"])
# Display the graph
display_graph(graph, file_name=os.path.basename(__file__))
# Run the graph
config = {"configurable": {"thread_id": "thread-1"}}
initial_input = {"input": "The importance of AI in modern content creation"}
# Run the first part until the review step
thread = {"configurable": {"thread_id": "1"}}
for event in graph.stream(initial_input, thread, stream_mode="values"):
    print(event)
# Pausing for human review
user_approval = input("Do you approve the draft for publishing? (yes/no/modification):")
if user_approval.lower() == "yes":
    # Proceed to publish content
    for event in graph.stream(None, thread, stream_mode="values"):
        print(event)
elif user_approval.lower() == "modification":
    # Allow modification of the draft
    updated_draft = input("Please modify the draft content:\n")
    memory.update({"draft_content": updated_draft}) # Update memory with new content
    print("Draft updated by the editor.")
    # Continue to publishing with the modified draft
    for event in graph.stream(None, thread, stream_mode="values"):
        print(event)
else:
    print("Execution halted by user.")

```

Output:

```
{'input': 'The importance of AI in modern content creation'}
```

--- Generating Draft with ChatOpenAI ---

Generated Draft:

The Importance of AI in Modern Content Creation

In today's fast-paced digital landscape, the demand for high-quality content has never been greater. From social media posts to in-depth articles and marketing materials, businesses and individuals alike are constantly seeking innovative ways

```
{'input': 'The importance of AI in modern content creation', 'draft_content': '# The Importance of AI in Modern Content Creation\n\nIn today's fast-paced digital landscape,
```

the demand for high-quality content has never been greater. From social media posts to in-depth articles and marketing materials, businesses and individuals alike are constantly seeking innovative ways'}

--- Reviewing Draft ---

Draft for review:

The Importance of AI in Modern Content Creation

In today's fast-paced digital landscape, the demand for high-quality content has never been greater. From social media posts to in-depth articles and marketing materials, businesses and individuals alike are constantly seeking innovative ways

{'input': 'The importance of AI in modern content creation', 'draft_content': '# The Importance of AI in Modern Content Creation\n\nIn today's fast-paced digital landscape, the demand for high-quality content has never been greater. From social media posts to in-depth articles and marketing materials, businesses and individuals alike are constantly seeking innovative ways'}

Do you approve the draft for publishing? (yes/no/modification): yes

{'input': 'The importance of AI in modern content creation', 'draft_content': '# The Importance of AI in Modern Content Creation\n\nIn today's fast-paced digital landscape, the demand for high-quality content has never been greater. From social media posts to in-depth articles and marketing materials, businesses and individuals alike are constantly seeking innovative ways'}

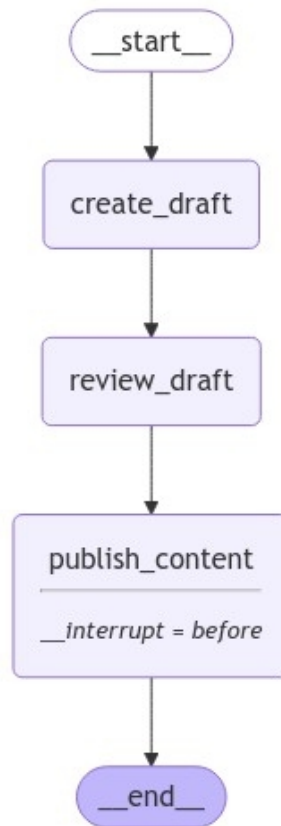
--- Publishing Content ---

Published Content:

The Importance of AI in Modern Content Creation

In today's fast-paced digital landscape, the demand for high-quality content has never been greater. From social media posts to in-depth articles and marketing materials, businesses and individuals alike are constantly seeking innovative ways

{'input': 'The importance of AI in modern content creation', 'draft_content': '# The Importance of AI in Modern Content Creation\n\nIn today's fast-paced digital landscape, the demand for high-quality content has never been greater. From social media posts to in-depth articles and marketing materials, businesses and individuals alike are constantly seeking innovative ways'}



Penjelasan Alur Workflow:

1. **create_draft:** The AI generates the content based on the input topic.
2. **review_draft:** The draft content is presented to the human editor for review. The human can decide whether to approve or modify the draft.
3. **publish_content:** After the draft is approved (or modified), the content is published.

Breakpoint diatur sebelum node `publish_content`, memastikan bahwa workflow berhenti sejenak dan menunggu persetujuan manusia sebelum konten dipublikasikan. Workflow ini menawarkan manfaat berikut:

- **Pengawasan Manusia:** Workflow ini memastikan bahwa editor manusia tetap memegang kendali atas konten akhir, memungkinkan mereka melakukan intervensi pada titik-titik kritis.
- **Fleksibilitas:** Editor manusia dapat menyetujui draf yang dihasilkan, melakukan modifikasi, atau menghentikan proses sepenuhnya. Ini mencegah

konten yang tidak sesuai dipublikasikan tanpa peninjauan.- **Skalabilitas:**
Workflow ini dapat diperluas untuk menangani tugas moderasi yang lebih kompleks, seperti memverifikasi akurasi faktual atau menandai konten sensitif berdasarkan feedback manusia.

9.4 Contoh ReAct Agent dengan Use Case Pembelian Saham

Pada bagian ini, kita akan membahas cara menerapkan React agent menggunakan arsitektur Human-in-the-Loop (HITL) untuk workflow pengambilan keputusan pembelian saham. Agent akan melakukan query harga saham secara live menggunakan Finnhub API, bernalar tentang apakah akan melakukan pembelian saham, dan berhenti sejenak untuk meminta persetujuan manusia sebelum melanjutkan. Contoh ini sangat berguna dalam workflow keuangan di mana pengawasan manusia sangat penting untuk pengambilan keputusan berisiko tinggi, seperti trading saham.

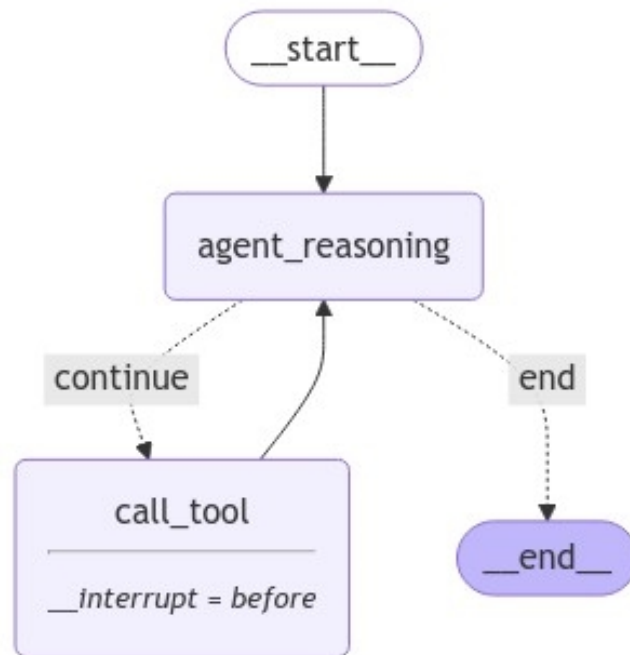
Skenario:

Agent ditugaskan untuk menentukan apakah akan membeli saham berdasarkan data harga live.
Agent akan:

1. Query real-time stock prices from the Finnhub API.
2. Reason through whether the stock price is favorable for purchasing.
3. Pause for human intervention to approve the purchase before executing the trade.

Implementasi Langkah demi Langkah

Mari kita telusuri pembuatan agent menggunakan arsitektur HITL LangGraph. Finnhub API akan digunakan untuk mengambil harga saham secara real-time, dan ReAct agent akan bernalar tentang melakukan pembelian berdasarkan kondisi harga.



Poin-Poin Kunci dalam Contoh Ini:

1. **Reasoning Step:** The agent first "reasons" using the AI model (OpenAI) about whether to check the stock price of a company (in this case, AAPL). The agent evaluates the initial request from the user and prepares to make a tool call for the stock price.
2. **Human Breakpoint:** Before the agent proceeds to retrieve the stock price from the **Finnhub API**, it pauses and requests human approval. This is the critical **Human-in-the-Loop** moment where a human operator decides whether to proceed or stop the process.
3. **Tool Invocation:** If approved, the agent proceeds to call the **Finnhub API** to get the latest stock price. The agent retrieves real-time price data and presents it to the user.
4. **Dynamic Flow:** The agent dynamically switches between reasoning and action based on conditional logic. If the human operator does not approve the price retrieval, the agent halts, ensuring no unwanted actions are taken.

```
#lesson9d.py  
  
# Import necessary modules  
import os
```

```

from langchain_core.tools import tool
from langgraph.graph import MessagesState, START, END, StateGraph
from langgraph.checkpoint.memory import MemorySaver
from langgraph.prebuilt import ToolNode
from langchain_openai import ChatOpenAI
import finnhub
from display_graph import display_graph
# Initialize Finnhub API client
finnhub_client = finnhub.Client(api_key=os.getenv("FINNHUB_API_KEY"))
# Define the tool: querying stock prices using the Finnhub API
@tool
def get_stock_price(symbol: str):
    """Retrieve the latest stock price for the given symbol."""
    quote = finnhub_client.quote(symbol)
    return f"The current price for {symbol} is ${quote['c']}."
# Register the tool in the tool node
tools = [get_stock_price]
tool_node = ToolNode(tools)
# Set up the AI model (simulated with ChatAnthropic)
model = ChatOpenAI(model="gpt-4o-mini")
model = model.bind_tools(tools)
# Define the function that simulates reasoning and invokes the model
def agent_reasoning(state):
    messages = state["messages"]
    response = model.invoke(messages)
    return {"messages": [response]}
# Define conditional logic to determine whether to continue or stop
def should_continue(state):
    messages = state["messages"]
    last_message = messages[-1]
    # If there are no tool calls, finish the process
    if not last_message.tool_calls:
        return "end"
    return "continue" # Otherwise, continue to the next step
# Build the React agent using LangGraph
workflow = StateGraph(MessagesState)
# Add nodes: agent reasoning and tool invocation (stock price retrieval)

```

```

workflow.add_node("agent_reasoning", agent_reasoning)
workflow.add_node("call_tool", tool_node)
# Define the flow
workflow.add_edge(START, "agent_reasoning") # Start with reasoning
# Conditional edges: continue to tool call or end the process
workflow.add_conditional_edges(
    "agent_reasoning", should_continue, {
        "continue": "call_tool", # Proceed to get stock price
        "end": END # End the workflow
    }
)
# Normal edge: after invoking the tool, return to agent reasoning
workflow.add_edge("call_tool", "agent_reasoning")
# Set up memory for breakpoints and add a breakpoint before calling the tool
memory = MemorySaver()
app = workflow.compile(checkpointer=memory, interrupt_before=["call_tool"])
# Display the graph (optional visualization step)
display_graph(app, __file__)
# Simulate user input for stock symbol
initial_input = {"messages": [{"role": "user", "content": "Should I buy AAPL stock today?"}]}
thread = {"configurable": {"thread_id": "1"}}
# Run the agent reasoning step first
for event in app.stream(initial_input, thread, stream_mode="values"):
    print(event)
# Pausing for human approval before retrieving stock price
user_approval = input("Do you approve querying the stock price for AAPL? (yes/no): ")
if user_approval.lower() == "yes":
    # Continue with tool invocation to get stock price
    for event in app.stream(None, thread, stream_mode="values"):
        print(event)
else:
    print("Execution halted by user.")

```

Detail Lebih Lanjut:

- **Agent Reasoning:** Agent memproses permintaan awal untuk membeli saham dan bernalar tentang tindakan apa yang harus diambil selanjutnya. Proses "berpikir keras" ini meningkatkan transparansi, menunjukkan kepada pengguna proses berpikir agent.- **Human Approval:** Agent berhenti sejenak sebelum mengambil data keuangan sensitif atau real-time, memungkinkan operator manusia untuk menyetujui atau menolak tindakan tersebut. Ini memastikan bahwa pembelian saham atau keputusan keuangan tidak dilakukan secara otonom tanpa pengawasan.- **Live Data dari Finnhub API:** Agent mengambil informasi harga saham secara real-time menggunakan Finnhub API, memberikan data yang akurat dan terkini yang dapat digunakan operator manusia untuk mengambil keputusan.

Contoh Output:

```
{'messages': [{'role': 'user', 'content': "Should I buy AAPL stock today?"}]}
--- Reasoning through the problem ---
Agent's reasoning: "Let's check the current price for AAPL using the stock price API."
{'messages': [{'role': 'user', 'content': "Should I buy AAPL stock today?"}, {'role': 'assistant', 'content': "Let's check the current price for AAPL using the stock price API."}]}
Do you approve querying the stock price for AAPL? (yes/no): yes
--- Invoking stock price API ---
{"role": "system", "tool_call": "get_stock_price", "result": "The current price for AAPL is $150.78."}
{'messages': [{'role': 'user', 'content': "Should I buy AAPL stock today?"}, {'role': 'assistant', 'content': "Let's check the current price for AAPL using the stock price API."}, {'role': 'system', 'tool_call': 'get_stock_price', 'result': "The current price for AAPL is $150.78."}]}
```

Use Case Dunia Nyata:

- **Trading Saham:** Agent dapat membantu trader dengan mengambil harga saham secara real-time dan berhenti sejenak untuk meminta persetujuan sebelum melakukan pembelian atau trading.- **Manajemen Risiko Keuangan:** Sebuah ReAct agent dapat digunakan di institusi keuangan untuk menganalisis kondisi pasar, mengambil data saham, dan meminta pengawasan manusia sebelum membuat keputusan berisiko tinggi seperti pembelian atau penjualan saham.

9.5 Memahami dan Menggunakan "Interrupt After" dalam LangGraph

Dalam workflow yang memerlukan pengambilan keputusan Human-in-the-Loop (HITL), fleksibilitas untuk menginterupsi alur pada tahapan yang berbeda sangatlah penting. Dalam LangGraph, kita dapat memperkenalkan interupsi baik sebelum maupun sesudah sebuah

node tertentu. Sebelumnya, kita telah membahas `interrupt_before`, yang memungkinkan workflow berhenti sejenak sebelum menjalankan sebuah node, menunggu persetujuan manusia sebelum melanjutkan. Sekarang, kita akan membahas konsep `interrupt_after`.

Apa itu `interrupt_after`?

Directive `interrupt_after` dalam LangGraph menyebabkan workflow berhenti sejenak setelah sebuah node tertentu dijalankan, memungkinkan intervensi manusia sebelum berpindah ke tahap berikutnya. Ini berguna dalam skenario di mana sebuah tindakan telah selesai tetapi memerlukan verifikasi manusia, pemeriksaan, atau langkah tindak lanjut sebelum melanjutkan.

Mengapa Menggunakan `interrupt_after`?

Menggunakan `interrupt_after` dalam sistem Human-in-the-Loop bermanfaat ketika:

- **Validasi pasca-tindakan diperlukan:** Setelah menjalankan sebuah task, Anda mungkin ingin manusia memverifikasi bahwa tindakan tersebut selesai sesuai harapan.
- **Penyesuaian manual:** Setelah sebuah tindakan, pengguna manusia mungkin perlu menyesuaikan hasil atau output sebelum melanjutkan ke langkah berikutnya.
- **Keamanan atau kepatuhan:** Dalam workflow yang melibatkan kepatuhan (misalnya, transaksi keuangan, publikasi konten), bahkan setelah sebuah tindakan dijalankan, persetujuan mungkin tetap diperlukan untuk memastikan semuanya sesuai dengan regulasi atau kebijakan perusahaan.

Contoh: Keputusan Pembelian Saham dengan Persetujuan Pasca-Pembelian

Mari kita ambil contoh pembelian saham sebelumnya dan tingkatkan dengan memperkenalkan `interrupt_after`. Dalam versi ini, agent akan:

1. Retrieve the stock price using the **Finnhub API**.
2. Decide to make a stock purchase based on a predefined threshold.
3. **After** making the purchase, the agent pauses for human approval to finalize the action and update the transaction record.

Berikut cara skenario ini bekerja dalam praktiknya:

```
#lesson9e.py  
  
# Import necessary modules
```

```

import os
from langchain_core.tools import tool
from langgraph.graph import MessagesState, START, END, StateGraph
from langgraph.checkpoint.memory import MemorySaver
from langgraph.prebuilt import ToolNode
from langchain_openai import ChatOpenAI
from display_graph import display_graph
import finnhub

# Initialize Finnhub API client
finnhub_client = finnhub.Client(api_key=os.getenv("FINNHUB_API_KEY"))

# Define the tool: querying stock prices using the Finnhub API
@tool
def get_stock_price(symbol: str):
    """Retrieve the latest stock price for the given symbol."""
    quote = finnhub_client.quote(symbol)
    return f"The current price for {symbol} is ${quote['c']}."

# Define the tool for purchasing stock (simulated)
@tool
def purchase_stock(symbol: str, quantity: int):
    """Simulate stock purchase and return a confirmation message."""
    return f"Purchased {quantity} shares of {symbol} at the current market price."

# Register the tools in the tool node
tools = [get_stock_price, purchase_stock]
tool_node = ToolNode(tools)

# Set up the AI model (ChatAnthropic) for reasoning
model = ChatOpenAI(model="gpt-4o-mini")
model = model.bind_tools(tools)

# Define the function that simulates reasoning and invokes the model
def agent_reasoning(state):
    messages = state["messages"]
    response = model.invoke(messages)
    return {"messages": [response]}

# Define conditional logic to determine whether to continue or stop
def should_continue(state):
    messages = state["messages"]
    last_message = messages[-1]
    # If there are no tool calls, finish the process

```

```

    if not last_message.tool_calls:
        return "end"
    return "continue" # Otherwise, continue to the next step
# Build the React agent using LangGraph
workflow = StateGraph(MessagesState)
# Add nodes: agent reasoning and tool invocation (stock price retrieval and purchase)
workflow.add_node("agent_reasoning", agent_reasoning)
workflow.add_node("call_tool", tool_node)
# Define the flow
workflow.add_edge(START, "agent_reasoning") # Start with reasoning
# Conditional edges: proceed to tool call or end the process
workflow.add_conditional_edges(
    "agent_reasoning", should_continue, {
        "continue": "call_tool", # Proceed to call tool (get stock price or purchase)
        "end": END # End the workflow
    }
)
# Normal edge: after invoking the tool, return to agent reasoning
workflow.add_edge("call_tool", "agent_reasoning")
# Set up memory for breakpoints and add `interrupt_after` to pause after the stock purchase
memory = MemorySaver()
app = workflow.compile(checkpointer=memory, interrupt_after=["call_tool"])
# Display the graph (optional visualization step)
display_graph(app, file_name="react_agent_stock_purchase_interrupt_after.png")
# Simulate user input for stock symbol
initial_input = {"messages": [{"role": "user", "content": "Should I buy AAPL stock today?"}]}
thread = {"configurable": {"thread_id": "1"}}
# Run the agent reasoning step first
for event in app.stream(initial_input, thread, stream_mode="values"):
    print(event)
# Pausing for human approval after the purchase is made
user_approval = input("Do you approve the stock purchase action? (yes/no): ")
if user_approval.lower() == "yes":
    # Confirm and finalize the transaction
    for event in app.stream(None, thread, stream_mode="values"):

```

```
print(event)
else:
    print("Execution halted by user.")
```

Poin-Poin Kunci dalam Contoh Ini:

1. **Agent Reasoning:** The agent first reasons about whether to proceed with purchasing the stock based on the current price retrieved from the **Finnhub API**.
2. **Tool Invocation and Post-Purchase Approval:** After retrieving the stock price and deciding to make a purchase, the agent invokes the tool to buy the stock. **Interrupt_after** ensures that after the purchase is made, the system pauses for human approval before proceeding to finalize the transaction.
3. **Human Review After Purchase:** Even though the purchase is simulated, the workflow pauses to allow human intervention to either approve or deny the transaction. This adds a safety layer in workflows where mistakes could lead to significant financial consequences.
4. **Flexible Control Flow:** This flow allows dynamic reasoning and action, where the agent evaluates its decision-making at multiple stages, ensuring that the human operator has control over key points in the process.

Detail Lebih Lanjut:

- **Menginterupsi Setelah Tindakan:** Dengan menghentikan sejenak workflow setelah tool call, sistem memungkinkan operator manusia untuk meninjau hasil tindakan tersebut (dalam hal ini, pembelian saham) sebelum melanjutkan. Ini dapat berguna dalam situasi seperti:
 - ? **Anda perlu mencatat hasilnya (log).? Ada pemeriksaan kepatuhan tambahan.? Operator manusia harus menyesuaikan parameter sebelum workflow dilanjutkan.- Penyesuaian Manual:** Jika operator manusia melihat adanya masalah (seperti jumlah atau harga yang salah), workflow dapat dimodifikasi untuk memungkinkan perubahan sebelum memfinalisasi pembelian.

Contoh Output:

```
{'messages': [{'role': 'user', 'content': "Should I buy AAPL stock today?"}]}--- Reasoning through the problem ---Agent's reasoning: "Let's check the current price for AAPL using the stock price API."{'messages': [{'role': 'user', 'content': "Should I buy AAPL stock today?"}, {'role': 'assistant', 'content': "Let's check the current price for AAPL using the stock price API."}]}--- Invoking stock price API ---{"role": "system", "tool_call": "get_stock_price", "result": "The current price for AAPL is $150.78."}--- Deciding to purchase stock ---Agent's reasoning: "The price is favorable. Let's purchase 10 shares of AAPL."{'messages': [{'role': 'user', 'content': "Should I buy AAPL stock today?"}, {'role': 'assistant', 'content': "The price is favorable. Let's purchase 10 shares of AAPL."}]}--- Invoking stock purchase tool ---{"role": "system", "tool_call": "purchase_stock", "result": "Purchased 10 shares of AAPL at the current market price."}Do you approve the stock purchase action? (yes/no): yes--- Finalizing stock purchase ---{'messages': [{'role': 'user', 'content': "Should I buy AAPL stock today?"}, {'role': 'assistant', 'content': "The price is favorable. Let's purchase 10 shares of AAPL."}, {'role': 'system', 'tool_call': 'purchase_stock', 'result': "Purchased 10 shares of AAPL at the current market price."}]}
```

9.5.1 Use Case Dunia Nyata untuk interrupt_after

Menggunakan interrupt_after sangat penting dalam skenario dunia nyata di mana verifikasi manusia diperlukan setelah sebuah tindakan dilakukan. Beberapa aplikasi praktis meliputi:

1. **Financial Transactions:** After a stock purchase or financial transaction is completed, the workflow pauses to allow human operators to verify and approve the transaction before updating the financial records.
2. **Healthcare:** After a medical test is ordered by an AI assistant, the workflow can pause for human doctors to review the test order before finalizing it, ensuring the right procedures are followed.
3. **Legal Processes:** After generating or modifying legal contracts, the workflow pauses for a legal expert to review the changes before submitting the document for signatures.
4. **Content Publishing:** After a draft is written or an edit is made, the workflow can pause for the editor to approve changes before publishing the final version.

9.6 Mengedit Graph State Selama Eksekusi

Gambaran Konsep

Mengedit graph state selama eksekusi memungkinkan agent menyesuaikan perilakunya secara dinamis berdasarkan input manusia. Ini sangat berguna ketika agent perlu mengubah state atau parameter yang memandu keputusannya. Misalnya, sebuah agent dapat mengubah arah tindakannya setelah manusia memodifikasi data input atau menyesuaikan sebuah tool call. Konsep ini dibangun di atas breakpoint dan memungkinkan pengguna untuk secara langsung memodifikasi state internal graph sebelum melanjutkan.

Use Case Praktis

- **Mengoreksi Tool Call:** Sebelum menjalankan tool yang kritis (seperti mengirim database query atau API call), graph berhenti sejenak dan memungkinkan pengguna untuk memodifikasi parameter.- **Modifikasi Data:** State dapat diperbarui secara manual untuk mencerminkan dataset yang telah dikoreksi atau input yang diberikan pengguna, memastikan hasil yang akurat.

Contoh 3: Memodifikasi Graph State Sebelum Melanjutkan

Dalam contoh ini, agent menghentikan eksekusi sejenak dan memungkinkan pengguna untuk memodifikasi graph state sebelum melanjutkan.

```
#lesson9f.py continued

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

# Define the state structure
class State(TypedDict):
    input: str
    modified_input: str

# Define node functions
def step_1(state: State):
    print(f"Original input: {state['input']}")
    return state

def modify_state(state: State):
    # Allow the user to modify the state
    return state
```

```

def step_3(state: State):
    print(f"Modified input: {state['modified_input']}")
    return state

# Build the graph
builder = StateGraph(State)
builder.add_node("step_1", step_1)
builder.add_node("modify_state", modify_state)
builder.add_node("step_3", step_3)
# Define the flow
builder.add_edge(START, "step_1")
builder.add_edge("step_1", "modify_state")
builder.add_edge("modify_state", "step_3")
builder.add_edge("step_3", END)
# Set up memory and breakpoints
memory = MemorySaver()
graph = builder.compile(checkpointer=memory, interrupt_before=["modify_state"])
# Run the graph
initial_input = {"input": "Initial Input"}
config = {"configurable": {"thread_id": "thread-1"}}
for event in graph.stream(initial_input, config):
    print(event)
# Ask user to modify the state
modified_value = input("Enter the modified input: ")
graph.update_state(config, {"modified_input": modified_value})
# Continue the graph execution
for event in graph.stream(None, config):
    print(event)

```

Konsep Kunci

- **State Editing: Node modify_state** memungkinkan manusia untuk memperbarui state graph secara manual.- **State Updates: graph.update_state()** digunakan untuk menerapkan perubahan pada state graph berdasarkan input manusia.

Output

```

Original input: Initial Input
{'step_1': {'input': 'Initial Input'}}

```

```

{ '__interrupt__': () }
Enter the modified input: new input
{'modify_state': {'input': 'Initial Input', 'modified_input': 'new input'}}
Modified input: new input
{'step_3': {'input': 'Initial Input', 'modified_input': 'new input'}}

```

Penjelasan Konsep

Mengedit state selama eksekusi graph memberikan fleksibilitas yang lebih besar. Pengguna dapat secara dinamis memodifikasi nilai-nilai kunci, memastikan agent beroperasi dengan informasi yang benar atau terbaru. Metode ini sangat berguna untuk mengoreksi input, memperbarui parameter sebelum tool call, atau menangani error.

9.7 Lima Contoh Breakpoint Sederhana

Sebelum membahas breakpoint dinamis, mari kita perkuat konsep breakpoint dasar melalui serangkaian contoh. Contoh-contoh ini menunjukkan berbagai cara breakpoint dapat digunakan untuk mengelola eksekusi workflow.

Contoh 1: Persetujuan Dasar untuk Transaksi Keuangan

```

#lesson9g.py continued

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

class State(TypedDict):
    amount: float
builder = StateGraph(State)

def define_transaction(state: State):
    print("Defining transaction...")
    return state

def verify_transaction(state: State):
    print(f"Verifying transaction amount: {state['amount']}")
    return state

builder.add_node("define_transaction", define_transaction)
builder.add_node("verify_transaction", verify_transaction)
builder.add_edge(START, "define_transaction")
builder.add_edge("define_transaction", "verify_transaction")
builder.add_edge("verify_transaction", END)

graph = builder.compile(interrupt_before=
["verify_transaction"],checkpointer=MemorySaver())

```



```

initial_input = {"amount": 1000.0}
config = {"configurable": {"thread_id": "thread-1"}}
for event in graph.stream(initial_input, config):
    print(event)
approval = input("Approve this transaction? (yes/no): ")
if approval.lower() == "yes":
    for event in graph.stream(None, config):
        print(event)
else:
    print("Transaction canceled.")

```

Contoh ini sangat berguna untuk memastikan tindakan sensitif seperti transaksi keuangan disetujui oleh manusia.

9.7.1 Contoh 2: Persetujuan Sebelum Penghapusan

Data#lesson9h.py continued

```

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
class State(TypedDict):
    data: str
def delete_data(state: State):
    print(f"Data to be deleted: {state['data']}")
    return state
builder = StateGraph(State)
builder.add_node("delete_data", delete_data)
builder.add_edge(START, "delete_data")
builder.add_edge("delete_data", END)
graph = builder.compile(interrupt_before=["delete_data"],
checkpoint=MemorySaver())
initial_input = {"data": "Sensitive Information"}
config = {"configurable": {"thread_id": "thread-1"}}
for event in graph.stream(initial_input, config):
    print(event)
approval = input("Approve data deletion? (yes/no): ")
if approval.lower() == "yes":

```

```

    for event in graph.stream(None, config):
        print(event)
else:
    print("Data deletion canceled.")

```

Workflow ini berhenti sebelum menjalankan penghapusan data, memungkinkan intervensi manusia untuk mencegah penghapusan yang tidak disengaja.

9.7.2 Contoh 3: Menyetujui Tool Call#lesson9i.py

```

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.tools import tool

class State(TypedDict):
    query: str

@tool
def perform_query(query: str):
    """param query: The SQL query to be executed."""
    print(f"Performing query: {query}")
    return {"query" : {query}}

def review_query(state: State):
    print(f"Reviewing query: {state['query']}")
    return state

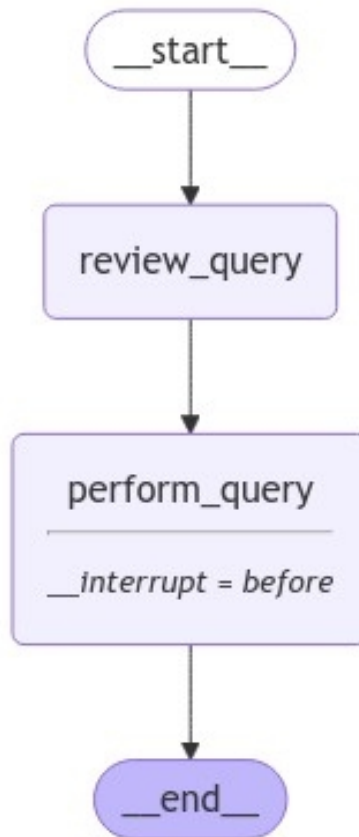
builder = StateGraph(State)
builder.add_node("perform_query", perform_query)
builder.add_node("review_query", review_query)
builder.add_edge("review_query", "perform_query")
builder.add_edge(START, "review_query")
builder.add_edge("perform_query", END)
graph = builder.compile(interrupt_before=["perform_query"],
checkpoint=MemorySaver())
initial_input = {"query": "SELECT * FROM users"}
config = {"configurable": {"thread_id": "thread-1"}}
for event in graph.stream(initial_input, config):
    print(event)

```

```

approval = input("Approve query execution? (yes/no): ")
if approval.lower() == "yes":
    for event in graph.stream(None, config):
        print(event)
else:
    print("Query execution cancelled.")

```



Ini menunjukkan cara menyetujui atau memodifikasi sebuah tool call sebelum eksekusi. Pengguna meninjau query dari node `review_query` dan diminta untuk menyetujui eksekusi.

Statement SQL hanya dijalankan setelah pengguna menyetujui, kemudian node `perform_query` dijalankan sehingga statement SQL dapat berjalan.

9.7.3 Contoh 4: Human Input untuk Report Generation

```

#lesson9j.py

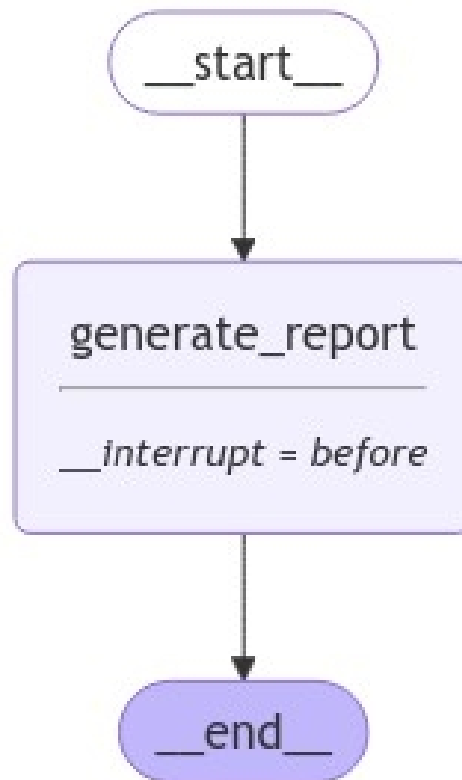
import os

```

```

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from display_graph import display_graph
class State(TypedDict):
    title: str
def generate_report(state: State):
    """
    Generate a financial report based on the provided title.
    """
    print(f"Generating report with title: {state['title']}")
    return state
builder = StateGraph(State)
builder.add_node("generate_report", generate_report)
builder.add_edge(START, "generate_report")
builder.add_edge("generate_report", END)
graph = builder.compile(interrupt_before=["generate_report"],
checkpoint=MemorySaver())
# Display the graph
display_graph(graph, file_name=os.path.basename(__file__))
initial_input = {"title": "Annual Financial Report"}
config = {"configurable": {"thread_id": "thread-1"}}
for event in graph.stream(initial_input, config):
    print(event)
approval = input("Approve report generation? (yes/no): ")
if approval.lower() == "yes":
    for event in graph.stream(None, config):
        if event:
            print(event)
else:
    print("Report generation canceled.")

```

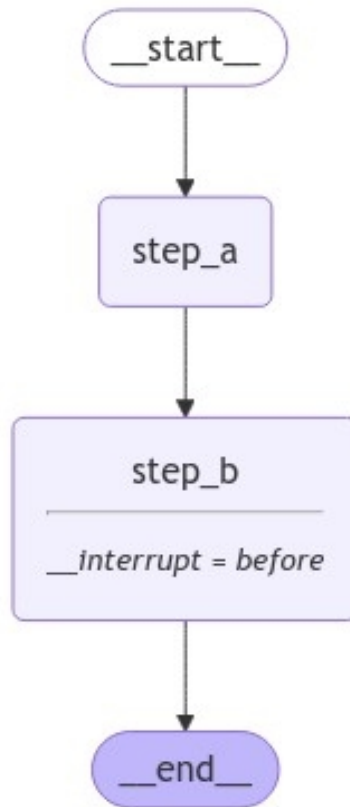


Contoh ini memungkinkan manusia untuk menyetujui pembuatan laporan, memastikan title dan content sudah benar.

9.7.4 Contoh 5: Persetujuan Manusia dalam Multi-step Workflow#lesson9k.py

```
import os
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from display_graph import display_graph
class State(TypedDict):
    input: str
def step_a(state: State):
    print("Executing Step A")
    return state
def step_b(state: State):
    print("Executing Step B")
```

```
        return state
builder = StateGraph(State)
builder.add_node("step_a", step_a)
builder.add_node("step_b", step_b)
builder.add_edge(START, "step_a")
builder.add_edge("step_a", "step_b")
builder.add_edge("step_b", END)
graph = builder.compile(interrupt_before=["step_b"], checkpointer=MemorySaver())
# Display the graph
display_graph(graph, file_name=os.path.basename(__file__))
initial_input = {"input": "Starting workflow"}
config = {"configurable": {"thread_id": "thread-1"}}
for event in graph.stream(initial_input, config):
    print(event)
approval = input("Proceed to Step B? (yes/no): ")
if approval.lower() == "yes":
    for event in graph.stream(None, config):
        print(event)
else:
    print("Workflow halted before Step B.")
```



Contoh ini menunjukkan proses dua langkah di mana persetujuan manusia diperlukan sebelum melanjutkan ke langkah berikutnya.

Output:

```
Executing Step A
{'step_a': {'input': 'Starting workflow'}}
{'__interrupt__': ()}
Proceed to Step B? (yes/no): yes
Executing Step B
{'step_b': {'input': 'Starting workflow'}}
```

9.8 Dynamic Breakpoints: Konsep dan Penggunaan

Dynamic breakpoints menambah fleksibilitas dengan memungkinkan agent berhenti sejenak secara kondisional berdasarkan data runtime atau pemicu eksternal. Alih-alih menentukan sebelumnya di mana graph berhenti, **dynamic breakpoints** menghadirkan kemampuan untuk memicu jeda berdasarkan kondisi tertentu.

Penjelasan Dynamic Breakpoints

Pada **dynamic breakpoints**, graph berhenti berdasarkan kondisi yang terpenuhi selama eksekusi sebuah node. Misalnya, jika input melebihi

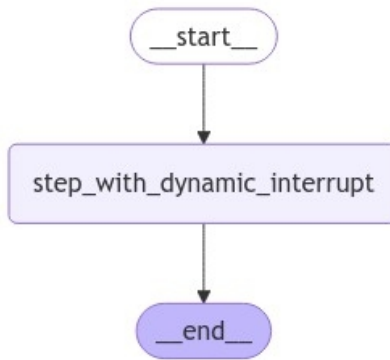
ambang batas yang telah ditentukan, agent dapat berhenti untuk menunggu intervensi manusia.

9.8.1 Contoh: Jeda Kondisional Berdasarkan Ambang Batas

Data#lesson9l.py lanjutan

```
from langgraph.errors import NodeInterrupt
import os
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from display_graph import display_graph
class State(TypedDict):
    input: str
builder = StateGraph(State)
def step_with_dynamic_interrupt(state: State):
    input_length = len(state["input"])
    if input_length > 10:
        raise NodeInterrupt("Input length {input_length} exceeds threshold of 10.")
    return state
builder.add_node("step_with_dynamic_interrupt", step_with_dynamic_interrupt)
builder.add_edge(START, "step_with_dynamic_interrupt")
builder.add_edge("step_with_dynamic_interrupt", END)
graph = builder.compile()
# Display the graph
display_graph(graph, file_name=os.path.basename(__file__))
initial_input = {"input": "This is a long input"}
for event in graph.stream(initial_input):
    print(event)
```

Dalam kasus ini, workflow secara otomatis berhenti ketika panjang input
exceeds 10 characters.



9.9 Menunggu Input Pengguna: Konsep dan Implementasi

Pendahuluan

Dalam workflow tertentu, AI agent memerlukan umpan balik manusia sebelum dapat melanjutkan. Hal ini sangat penting dalam konteks di mana keputusan perlu dibuat berdasarkan input manusia secara real-time, seperti layanan pelanggan, layanan kesehatan, atau proses persetujuan bisnis. Di LangGraph, menunggu input pengguna dapat diimplementasikan menggunakan breakpoints, yang menghentikan eksekusi graph pada titik tertentu hingga pengguna memberikan umpan balik yang diperlukan.

Bagian ini akan membahas konsep menunggu input pengguna secara mendetail, termasuk implementasi langkah demi langkah dan contoh praktis. Kita akan membahas:

1. **What is Waiting for User Input?**
2. **How to Implement Waiting for Input in LangGraph**
3. **Practical Examples of Waiting for User Input**
4. **Error Handling and Timeouts**
5. **Advanced Use Cases**

9.9.1 Apa itu Menunggu Input Pengguna?

Menunggu Input Pengguna mengacu pada interaksi di mana AI agent berhenti sejenak selama workflow-nya, meminta klarifikasi atau informasi tambahan dari manusia, dan melanjutkan hanya setelah menerimanya. Ini merupakan fitur penting untuk workflow Human-in-the-Loop (HITL), di mana keputusan tidak dapat sepenuhnya diotomatisasi dan memerlukan intervensi manusia untuk menyelesaikannya.

Aplikasi Praktis:

- **Layanan Pelanggan:** Chatbot mungkin meminta pengguna untuk mengklarifikasi pertanyaan mereka sebelum memberikan respons.-
- Workflow Persetujuan:** Proses bisnis mungkin memerlukan persetujuan manajer untuk melanjutkan transaksi keuangan atau menghasilkan laporan yang bersifat sensitif.-
- Pengambilan Keputusan Interaktif:** Agent layanan kesehatan mungkin perlu menanyakan detail tertentu kepada dokter atau pasien sebelum menghasilkan saran diagnostik.

9.9.2 Cara Mengimplementasikan Menunggu Input di LangGraph

Di LangGraph, menunggu input pengguna dicapai dengan menggunakan breakpoints. Pada sebuah breakpoint, graph menghentikan eksekusi dan menunggu umpan balik pengguna, yang kemudian dimasukkan kembali ke dalam state graph. Workflow melanjutkan dari titik tersebut menggunakan input yang diberikan.

Langkah-Langkah Kunci untuk Implementasi:

1. **Define Breakpoints:** Set up the graph to interrupt execution before a node that collects human feedback.
2. **State Management:** Use LangGraph's `update_state()` method to update the graph's state with the user's input.
3. **Resume Workflow:** After receiving user input, the graph continues executing from the point it paused.

Contoh 1: Workflow Umpan Balik Manusia Sederhana

Dalam contoh ini, kita akan menyiapkan workflow dasar yang menghentikan eksekusi untuk menunggu umpan balik pengguna sebelum melanjutkan.

```
#lesson9m.py continued

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
# Define the state structure
class State(TypedDict):
    input: str
    user_feedback: str
# Define node functions
def step_1(state: State):
```

```

    print(f"Step 1: {state['input']}")
    return state
def human_feedback(state: State):
    print("--- Waiting for human feedback ---")
    feedback = input("Please provide your feedback: ")
    state['user_feedback'] = feedback
    return state
def step_3(state: State):
    print(f"Step 3: User feedback received:
{state['user_feedback']}")
    return state
# Build the graph
builder = StateGraph(State)
builder.add_node("step_1", step_1)
builder.add_node("human_feedback", human_feedback)
builder.add_node("step_3", step_3)
# Define the flow
builder.add_edge(START, "step_1")
builder.add_edge("step_1", "human_feedback")
builder.add_edge("human_feedback", "step_3")
builder.add_edge("step_3", END)
# Set up memory and breakpoints
memory = MemorySaver()
graph = builder.compile(checkpointer=memory, interrupt_before=
["human_feedback"])
# Run the graph
initial_input = {"input": "Proceed with workflow?"}
thread = {"configurable": {"thread_id": "1"}}
# Stream the graph until the first interruption
for event in graph.stream(initial_input, thread,
stream_mode="values"):
    print(event)
# Get user input and update the state
user_feedback = input("User feedback: ")
graph.update_state(thread, {"user_feedback": user_feedback},
as_node='human_feedback')
# Resume execution

```

```
for event in graph.stream(None, thread, stream_mode="values"):
    print(event)
```

Konsep Kunci:

- **Interrupt Before Node: Argumen**
`interrupt_before=["human_feedback"]` menghentikan graph sebelum node `human_feedback` dieksekusi, menunggu input.
- **State Update:**
Metode `update_state()` memperbarui state graph dengan umpan balik manusia, memungkinkan workflow untuk melanjutkan dengan data baru.

9.9.3 Penanganan Error dan Timeout

Dalam beberapa situasi, pengguna mungkin tidak merespons dengan segera, atau umpan balik mereka mungkin tidak valid. Untuk menangani skenario ini, LangGraph memungkinkan timeout dan penanganan error selama jeda menunggu input pengguna.

Contoh: Menangani Timeout untuk Input Pengguna

```
#lesson9l.py continued

import time
def human_feedback_with_timeout(state: State):
    print("Waiting for human feedback (timeout in 10 seconds)...")
    start_time = time.time()
    while True:
        if time.time() - start_time > 10:
            state['user_feedback'] = "Timeout"
            print("No feedback received. Proceeding with default action.")
            break
        feedback = input("Please provide feedback: ")
        if feedback:
            state['user_feedback'] = feedback
            break
    return state
```

Pertimbangan Kunci:

- **Timeout:** Menerapkan timeout memastikan bahwa workflow tidak tetap berhenti tanpa batas waktu, memungkinkan tindakan fallback otomatis

dilakukan.- Penanganan Error: Pastikan bahwa input pengguna divalidasi. Misalnya, umpan balik mungkin perlu sesuai dengan format atau kriteria tertentu (misalnya, "yes" atau "no").

9.9.4 Kasus Penggunaan Lanjutan

Umpan Balik Multi-Tahap: Dalam workflow yang kompleks, beberapa jeda mungkin diperlukan di tahap yang berbeda. Misalnya, AI agent mungkin memerlukan umpan balik dari beberapa pemangku kepentingan dalam sebuah perusahaan, berhenti di setiap langkah untuk mengumpulkan persetujuan atau klarifikasi.

Permintaan Umpan Balik Adaptif: Berdasarkan state workflow saat ini atau umpan balik sebelumnya, agent dapat menyesuaikan permintaan berikutnya. Misalnya, jika pengguna menolak suatu permintaan, agent mungkin mengajukan pertanyaan lanjutan untuk lebih memahami keputusan tersebut.

Contoh: Permintaan Umpan Balik Adaptif

```
#lesson9l.py continued

def adaptive_feedback(state: State):
    decision = state['user_feedback']
    if decision.lower() == "denied":
        print("Request was denied. Asking for additional clarification.")
        clarification = input("Why was the request denied?: ")
        state['user_feedback'] = clarification
    return state
```

Ini memungkinkan interaksi yang lebih halus, di mana agent dapat menyesuaikan perilakunya secara dinamis berdasarkan respons pengguna.

Contoh Praktis: ReAct Agent dengan Input Manusia

Dalam contoh ini, kita akan membangun agent bergaya ReAct yang menggunakan LangGraph. Agent dapat bernalar melalui tindakan dan meminta umpan balik manusia sebelum mengambil langkah penting, seperti memutuskan tool mana yang akan dipanggil.

Workflow Langkah demi Langkah:

1. The agent starts by reasoning through the task.

2. It pauses at a point where human feedback is needed (e.g., asking a clarifying question).
3. After receiving feedback, the agent uses that input to decide on the next action (e.g., calling a tool or generating a response).

Contoh Workflow:

```
#lesson9n.py continued

import os
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.tools import tool
from display_graph import display_graph

# Define the state structure
class State(TypedDict):
    input: str
    user_feedback: str

# Define a reasoning step for the agent
def agent_reasoning(state: State):
    print(f"Agent is reasoning: {state['input']}")
    # Agent decides whether to ask human based on input length
    if len(state["input"]) > 10:
        print("Agent needs clarification.")
        return state
    else:
        state["user_feedback"] = "No clarification needed"
        return state

# Define a human feedback step
def ask_human(state: State):
    print("--- Asking for human feedback ---")
    feedback = input("Please provide feedback on the input: ")
    state['user_feedback'] = feedback
    return state

# Define a tool action after human feedback
@tool
def perform_action(user_feedback: str):
    """
```

```

        Perform an action based on the provided user feedback.
        """
        print(f"Action taken based on feedback: {user_feedback}")
        return {"user_feedback": f"Feedback processed: {user_feedback}"}

# Build the graph
builder = StateGraph(State)
builder.add_node("agent_reasoning", agent_reasoning)
builder.add_node("ask_human", ask_human)
builder.add_node("perform_action", perform_action)

# Define flow with conditions
builder.add_edge(START, "agent_reasoning")
builder.add_conditional_edges(
    "agent_reasoning",
    lambda state: "ask_human" if len(state["input"]) > 10 else "perform_action",
    {"ask_human": "ask_human", "perform_action": "perform_action"}
)
builder.add_edge("ask_human", "perform_action")
builder.add_edge("perform_action", END)

# Set up memory and breakpoints
memory = MemorySaver()
graph = builder.compile(checkpointer=memory, interrupt_before=["ask_human"])

# Display the graph
display_graph(graph, file_name=os.path.basename(__file__))

# Run the graph
initial_input = {"input": "Proceed with reasoning?"}
thread = {"configurable": {"thread_id": "1"}}

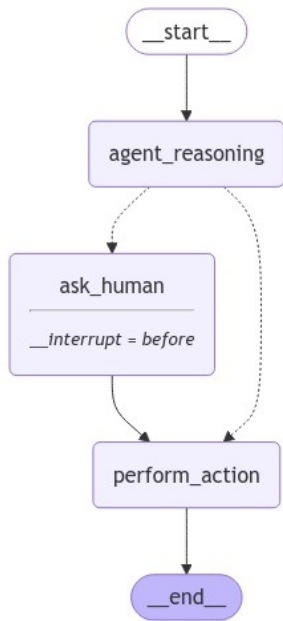
# Stream the graph until the first interruption
for event in graph.stream(initial_input, thread, stream_mode="values"):
    print(event)

# Get user input and update the state
user_feedback = input("User feedback: ")
graph.update_state(thread, {"user_feedback": user_feedback}, as_node="ask_human")

# Resume execution after feedback
for event in graph.stream(None, thread, stream_mode="values"):
    print(event)

```

Rincian ReAct Agent Ini:



1. Agent Reasoning:

- The agent examines the input and decides whether to proceed directly or ask for human clarification based on the length of the input.

2. Ask Human:

- If the agent decides that clarification is needed (e.g., when the input length is more than 10 characters), it asks the user for feedback and pauses execution.

3. Action Execution:

- Based on the feedback from the user, the agent performs the final action (e.g., calls a tool).

Contoh ini menunjukkan bagaimana menggabungkan penalaran otomatis dan interaksi manusia menggunakan LangGraph, membuatnya dapat diadaptasi ke workflow kompleks di mana input manusia sangat penting.

Penjelasan: Aspek Teknis Human-in-the-Loop (HITL) di LangGraph

Dalam sistem Human-in-the-Loop (HITL), agent sering kali memerlukan input manusia pada tahap-tahap kunci dalam workflow-nya untuk membuat keputusan atau memvalidasi tindakan. LangGraph memungkinkan developer untuk mengintegrasikan fungsi HITL melalui breakpoints, checkpoints, dan state updates, memungkinkan kontrol dinamis atas perilaku AI agent selama eksekusi. Penjelasan ini akan berfokus pada aspek-aspek kunci seperti melihat, memperbarui state graph sebelumnya, serta replay atau branching dari state sebelumnya di LangGraph.

Konsep Kunci dalam HITL

1. Breakpoints:

- **Breakpoints** allow a graph to pause at specific nodes, awaiting human feedback or approval before continuing. This provides flexibility and control over critical actions.

2. Checkpoints:

- Checkpoints are points in the execution where the current state of the graph is saved. This allows developers or users to revisit and alter the agent's state at any moment in the past.

3. State Updates:

- **State Updates** allow you to modify the agent's internal state dynamically. By updating the state, developers can change the behavior of the agent or correct inputs during its execution. This is crucial in HITL workflows for fine-tuning responses or actions after human intervention.

Melihat dan Memperbarui State Graph Sebelumnya

Salah satu fitur kunci di LangGraph adalah kemampuan untuk melihat dan memperbarui state agent pada titik mana pun selama eksekusinya. Ini sangat berguna untuk debugging, mereproduksi masalah, atau bahkan menyematkan AI agent Anda ke dalam sistem yang lebih besar. Dengan melakukan checkpoint pada graph, Anda dapat menyimpan state-nya, meninjaunya, dan menerapkan perubahan berdasarkan umpan balik manusia atau data baru.

Cara Melihat dan Memperbarui State Agent:

LangGraph menyediakan dua metode penting untuk berinteraksi dengan state graph:

- **get_state():** Metode ini mengambil nilai-nilai yang tersimpan dalam state graph saat ini.- **update_state():** Metode ini memungkinkan Anda untuk menerapkan perubahan pada state graph berdasarkan input baru atau umpan balik dari pengguna manusia.

Catatan: Agar kedua metode ini berfungsi, sebuah checkpointer harus diberikan saat graph di-compile.

Contoh Praktis: Melihat dan Memperbarui State

Berikut adalah contoh sederhana yang menunjukkan cara menghentikan sebuah workflow, melihat state pada sebuah breakpoint, memperbaruinya berdasarkan umpan balik manusia, dan melanjutkan eksekusi.

```
#lesson9n.py continued

# Fetching the state at a specific point in the graph
current_state = app.get_state(config)
print(current_state.values) # View the state values
# Updating the state based on new feedback
new_state = {"user_feedback": "Proceed with action after review"}
app.update_state(config, new_state)
```

Contoh: Membangun ReAct Agent dengan State Updates

Kita sekarang dapat membangun agent bergaya ReAct sederhana yang memanggil tool dan menggunakan umpan balik manusia untuk membuat keputusan. Selain itu, kita akan memanfaatkan state updates dan melihat state sebelumnya selama eksekusi.

```
#lesson9n.py continued

# Import necessary libraries
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langgraph.graph import MessagesState, START, END, StateGraph
from langgraph.checkpoint.memory import MemorySaver
# Define tools
@tool
```

```

def play_song_on_spotify(song: str):
    return f"Successfully played {song} on Spotify!"
@tool
def play_song_on_apple(song: str):
    return f"Successfully played {song} on Apple Music!"
# List of tools
tools = [play_song_on_apple, play_song_on_spotify]
tool_node = ToolNode(tools)
# Set up model
model = ChatOpenAI(model="gpt-4o-mini").bind_tools(tools)
# Define model-calling function
def call_model(state):
    response = model.invoke(state["messages"])
    return {"messages": [response]}
# Define continuation logic
def should_continue(state):
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        return "continue"
    return "end"
# Build the graph
workflow = StateGraph(MessagesState)
workflow.add_node("agent", call_model)
workflow.add_node("action", tool_node)
# Define graph flow
workflow.add_edge(START, "agent")
workflow.add_conditional_edges("agent", should_continue, {"continue": "action",
"end": END})
workflow.add_edge("action", "agent")
# Set up memory for checkpointing
memory = MemorySaver()
app = workflow.compile(checkpointer=memory)

```

Berinteraksi dengan Agent

Setelah menyiapkan agent, Anda dapat berinteraksi dengannya dengan mengirimkan pesan dan melihat responsnya. Berikut contoh di mana agent diminta untuk memutar

lagu:

```
#lesson9n.py continued

from langchain_core.messages import HumanMessage
config = {"configurable": {"thread_id": "1"}}
input_message = HumanMessage(content="Can you play Taylor Swift's most popular song?")
for event in app.stream({"messages": [input_message]}, config,
stream_mode="values"):
    print(event["messages"][-1].pretty_print())
```

Output:

```
=====
Human Message: Can you play Taylor Swift's most popular song?
=====
Tool Calls: play_song_on_apple (args: {'song': 'Anti-Hero'})
...
=====
Successfully played Anti-Hero by Taylor Swift on Apple Music!
```

Melihat dan Memutar Ulang State

Setelah checkpointing diaktifkan, kita dapat melihat riwayat state dan memutar ulang dari state sebelumnya mana pun.

```
#lesson9n.py continued

# View state history
state_history = app.get_state_history(config)
for state in state_history:
    print(state)
# Replay from a previous state
replay_state = state_history[-2] # Replay right before tool execution
for event in app.stream(None, replay_state.config):
    print(event)
```

Ini memungkinkan Anda untuk memutar ulang state sebelumnya, pada dasarnya "memundurkan" workflow agent untuk memperbaiki atau melakukan debug pada tindakan.

Branching dari State Sebelumnya

LangGraph juga mendukung branching dari state sebelumnya, memungkinkan Anda untuk mengeksplorasi hasil alternatif. Misalnya, jika agent memanggil tool yang salah, Anda dapat memodifikasi tool call tersebut dan melanjutkan dari titik itu.

```
#lesson9n.py continued

# Get the last message with the tool call
last_message = replay_state.values["messages"][-1]
# Update the tool call from Apple Music to Spotify
last_message.tool_calls[0]["name"] = "play_song_on_spotify"
# Update the state and resume execution
branch_config = app.update_state(replay_state.config, {"messages": [last_message]})
for event in app.stream(None, branch_config):
    print(event)
```

Dalam skenario ini, kita memodifikasi state untuk mengganti tool call, menyebabkan agent kini memutar lagu di Spotify alih-alih Apple Music.

Rangkuman

Kemampuan LangGraph untuk melihat, memperbarui, dan melakukan branching dari state sebelumnya memberikan fungsi yang kuat untuk debugging, fine-tuning, dan menyematkan AI workflow ke dalam sistem yang kompleks. Dengan checkpoints dan state management, developer dapat menghadirkan umpan balik manusia pada tahap mana pun, menyesuaikan workflow secara dinamis, dan mempertahankan kendali ketat atas perilaku agent.

Metode Kunci:

- **get_state():** Mengambil state agent saat ini atau sebelumnya.-
- update_state():** Memodifikasi state pada titik mana pun untuk mengubah trajektori agent.-
- get_state_history():** Meninjau state sebelumnya untuk memahami perilaku agent dari waktu ke waktu.-
- Branching:** Mengeksplorasi hasil alternatif dengan memodifikasi state sebelumnya tertentu dan melanjutkan eksekusi dari sana.

Dengan mengintegrasikan metode-metode ini, LangGraph memungkinkan workflow HITL yang tangguh dan fleksibel, memastikan bahwa keputusan manusia dapat memandu dan memengaruhi tindakan AI agent secara real-time.

Kuis Bab 9: Human-in-the-Loop Agents

Uji pemahaman Anda tentang Human-in-the-Loop (HITL) Agents dan konsep-konsep yang dibahas dalam Bab 9 dengan pertanyaan-pertanyaan berikut. Jawaban tersedia di bawah.

Pertanyaan Pilihan Ganda:

1. **What is the primary purpose of Human-in-the-Loop (HITL) functionality in AI workflows?**
 - A) To automate decision-making completely.
 - B) To allow an AI agent to operate without any human feedback.
 - C) To pause an AI workflow and allow human input for critical decisions.
 - D) To improve the speed of AI workflows.
2. **Which LangGraph feature allows you to pause the graph execution and wait for user input?**
 - A) Checkpoints
 - B) Breakpoints
 - C) State nodes
 - D) Conditional edges
3. **How does the `update_state()` function in LangGraph help in HITL workflows?**
 - A) It updates the graph with a new model.
 - B) It pauses the workflow and restarts the entire process.
 - C) It allows the graph's state to be modified with new input, often from human feedback.
 - D) It removes unnecessary steps from the workflow.
4. **What does the method `get_state()` do in LangGraph?**
 - A) It halts the workflow indefinitely.

- B) It retrieves the current or past state of the graph for inspection.
- C) It moves the graph to the next node without executing any logic.
- D) It resets the state of the graph to the initial configuration.

5. What is the purpose of branching in a LangGraph workflow?

- A) To introduce a new tool into the workflow.
- B) To replay the entire workflow from the start.
- C) To modify the agent's state at a certain point and explore alternate outcomes.
- D) To remove a node from the graph execution path.

6. What role do checkpointers play in HITL workflows?

- A) They are used to skip nodes.
- B) They allow the graph's state to be saved and retrieved at different points in time.
- C) They pause the workflow indefinitely.
- D) They automatically restart the graph.

7. In a ReAct agent, why might human input be necessary at some point during the execution?

- A) To help the agent avoid redundant API calls.
- B) To allow the agent to clarify its understanding before calling a tool or making a decision.
- C) To stop the agent from finishing the workflow.
- D) To force the agent to call more tools.

Pertanyaan Benar/Salah:

- True or False:** A breakpoint in LangGraph always pauses the graph after the node is executed.
- True or False:** Checkpoints can be used to save a graph's state and return to it later for debugging or modifications.
- True or False:** The `update_state()` method is used only when an error occurs in the graph.

Pertanyaan Jawaban Singkat:

11. Explain how dynamic breakpoints differ from simple breakpoints in HITL workflows.
12. What is the advantage of being able to replay a past state in LangGraph workflows?
13. Give an example of a situation where an AI agent might need to pause for human input during a workflow.

Jawaban:

1. **C)** To pause an AI workflow and allow human input for critical decisions.
2. **B)** Breakpoints
3. **C)** It allows the graph's state to be modified with new input, often from human feedback.
4. **B)** It retrieves the current or past state of the graph for inspection.
5. **C)** To modify the agent's state at a certain point and explore alternate outcomes.
6. **B)** They allow the graph's state to be saved and retrieved at different points in time.
7. **B)** To allow the agent to clarify its understanding before calling a tool or making a decision.
8. **False:** A breakpoint pauses the graph **before** the node is executed.
9. **True**
10. **False:** `update_state()` can be used at any point to modify the graph's state, not just when an error occurs.
11. **Dynamic breakpoints** are conditionally triggered based on runtime data (e.g., input length or external conditions), while **simple breakpoints** pause execution at a predefined node regardless of the current state.
12. **Replaying a past state** allows developers to debug or explore alternative decisions at key points in the workflow, without having to rerun the entire graph from the start.
13. **Example:** An IT support agent might need to pause and ask for clarification from a user when diagnosing a complex issue before recommending a solution or running a tool to fix the problem.

Tugas Praktik: Membangun IT Support AI Agent dengan Fitur Human In The Loop

Tujuan:

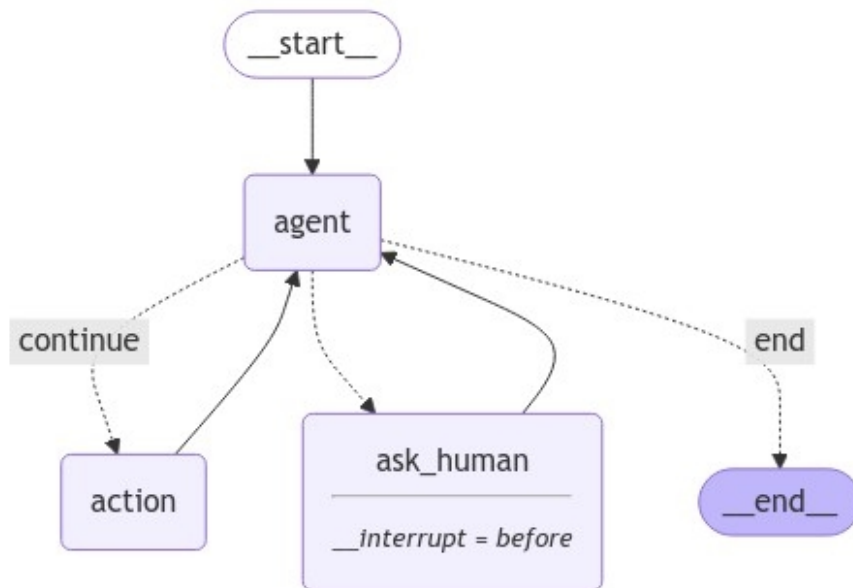
Buat IT support AI agent menggunakan LangGraph yang dapat berinteraksi dengan pengguna, melakukan tugas troubleshooting, dan meminta input manusia untuk keputusan-keputusan penting. Agent tersebut harus mampu:

- Mendiagnosis masalah IT dengan berinteraksi dengan berbagai tool (misalnya, system checks, network diagnostics).-
- Menghentikan eksekusi dan meminta input pengguna saat diperlukan (misalnya, meminta pengguna untuk mengonfirmasi tindakan seperti me-restart server).-
- Menggunakan `update_state()` untuk memodifikasi state agent berdasarkan umpan balik pengguna.-
- Menerapkan checkpoints untuk memungkinkan replay atau branching dari state sebelumnya jika masalah perlu ditinjau ulang.

Langkah-Langkah untuk Menyelesaikan:

1. Define the Agent's Workflow:

- The agent will check the system status (e.g., CPU usage, disk space, network connectivity) using multiple action nodes.
- The results from each action get sent back to the agent.
- The agent will then suggest actions based on the results (e.g., clear disk space, restart the server).
- A breakpoint will be used to pause before executing critical actions, asking for human approval.



2. **Define the Tools:** Create simple tools that simulate system checks:

- `check_cpu_usage()` : Simulates checking CPU usage.
- `check_disk_space()` : Simulates checking disk space.
- `restart_server()` : Simulates restarting a server.

3. Use `tool()` to wrap these actions.

4. **Implement the HITL Features:**

- Add a **breakpoint** before the agent attempts to restart the server, asking for human approval.
- Use `update_state()` to modify the graph's state based on the user's feedback (e.g., whether they approve the restart).
- Implement **checkpointing** to allow the agent to save the state and replay it if an issue needs to be revisited.

5. **Add State Management:** Implement state management to store and update the values of system checks and user feedback.

6. **Testing:** Test the agent by running various scenarios, such as:

- Approving and denying a restart action.
- Replaying the graph from a past state to review the previous system check results.

- Branching off from a state where the restart was denied and performing another action (e.g., clearing disk space).

Contoh Struktur Kode:

```
#lesson9n.py continued

# Import necessary components
import os
from langgraph.graph import MessagesState, START, END, StateGraph
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.tools import tool
from langgraph.prebuilt import ToolNode
from langchain_openai import ChatOpenAI
from pydantic import BaseModel
from display_graph import display_graph

# Define the diagnostic tools for the IT Support agent
@tool
def check_cpu_usage(tool_input: int):
    """Simulates checking the CPU usage of the server."""
    return "CPU Usage is 90%"

@tool
def check_disk_space(tool_input: int):
    """Simulates checking the available disk space on the server."""
    return "Disk space is below 15%"

@tool
def restart_server(tool_input: bool):
    """Simulates restarting the server."""
    return "Server restarted successfully"

# Define the human feedback tool (for confirming server restart)
class AskHuman(BaseModel):
    """Ask the human whether to restart the server."""
    question: str

# Set up the tools and tool node
tools = [check_cpu_usage, check_disk_space, restart_server]
tool_node = ToolNode(tools)

# Set up the AI model
model = ChatOpenAI(model="gpt-4o")

# Bind the model to the tools (including the ask_human tool)
```

```

model = model.bind_tools(tools + [AskHuman])
# Define the workflow functions for the agent
# Function to decide the next step based on the last message
def should_continue(state):
    messages = state["messages"]
    last_message = messages[-1]
    # If no tool call, finish the process
    if not last_message.tool_calls:
        return "end"
    # If the tool call is AskHuman, return that node
    elif last_message.tool_calls[0]["name"] == "AskHuman":
        return "ask_human"
    # Otherwise, continue the workflow
    else:
        return "continue"
# Function to call the model and return the response
def call_model(state):
    messages = state["messages"]
    response = model.invoke(messages)
    return {"messages": [response]}
# Define the human interaction node
def ask_human(state):
    pass # No actual processing here, handled via breakpoint
# Create the state graph
workflow = StateGraph(MessagesState)
# Define the nodes for the workflow
workflow.add_node("agent", call_model)
workflow.add_node("action", tool_node)
workflow.add_node("ask_human", ask_human)
# Set the starting node
workflow.add_edge(START, "agent")
# Define conditional edges based on the agent's output
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "continue": "action", # Proceed to the tool action

```

```

        "ask_human": "ask_human", # Ask human for input
        "end": END, # Finish the process
    }
)
# Add the edge from action back to agent for continued workflow
workflow.add_edge("action", "agent")
# Add the edge from ask_human back to agent after human feedback
workflow.add_edge("ask_human", "agent")
# Set up memory for checkpointing
memory = MemorySaver()
# Compile the graph with a breakpoint before ask_human
app = workflow.compile(checkpointer=memory, interrupt_before=["ask_human"])
# Visualize the workflow
display_graph(app, file_name=os.path.basename(__file__))
from langchain_core.messages import HumanMessage
# Initial configuration and user message
config = {"configurable": {"thread_id": "3"}}
input_message = HumanMessage(
    content="Check the CPU usage and disk space of the server, and restart it if necessary."
)
# Start the interaction with the agent
for event in app.stream({"messages": [input_message]}, config,
stream_mode="values"):
    event["messages"][-1].pretty_print()
# Get the ID of the last tool call (AskHuman tool call)
tool_call_id = app.get_state(config).values["messages"][-1].tool_calls[0]["id"]
# Ask the user whether they want to approve the server restart
user_input = input("Do you want to restart the server? (yes/no): ")
# Create the tool response message based on actual user input
tool_message = [
    {"tool_call_id": tool_call_id, "type": "tool", "content": user_input} # Use real user
input
]
# Update the state as if the response came from the user
app.update_state(config, {"messages": tool_message}, as_node="ask_human")
for event in app.stream(None, config, stream_mode="values"):

```

```
event["messages"][-1].pretty_print()
```

Agent ini mensimulasikan workflow dukungan IT dasar yang memeriksa performa sistem, meminta feedback pengguna, dan mengambil tindakan berdasarkan input manusia. Anda dapat lebih lanjut mengembangkan tugas ini untuk mengintegrasikan panggilan API sungguhan dan menambahkan aksi diagnostik yang lebih kompleks.

Chapter 10

Plan-and-Execute Agents

10.1 Pendahuluan

Arsitektur Plan-and-Execute meningkatkan tugas reasoning dengan membaginya menjadi dua fase inti: Planning dan Execution. Pendekatan ini berasal dari metodologi Plan-and-Solve, yang berbeda dengan agent single-step (ReAct) tradisional. Arsitektur Plan-and-Solve sangat berguna ketika menangani tugas reasoning multi-step yang kompleks dan memerlukan perencanaan eksplisit untuk tujuan jangka panjang.

Dalam bab ini, kita akan mengeksplorasi bagaimana agent Plan-and-Execute bekerja dengan menyusun rencana multi-step dan menjalankannya secara bertahap. Agent beradaptasi sesuai kebutuhan, memungkinkan re-planning jika ada langkah yang gagal atau memerlukan konteks lebih lanjut.

10.2 Mengapa Menggunakan Plan-and-Execute?

Salah satu masalah utama dalam tugas reasoning multi-step adalah bahkan model canggih seperti GPT-4 sering kesulitan dengan perencanaan jangka panjang yang eksplisit.

Plan-and-Execute mengatasi hal ini dengan:

- Memecah tugas kompleks menjadi langkah-langkah kecil yang dapat dikelola.- Beradaptasi secara dinamis berdasarkan hasil antara.- Meninjau kembali rencana bila diperlukan untuk menyesuaikan atau mengoreksi langkah.

Arsitektur ini lebih unggul dibandingkan model ReAct (React-then-Act) tradisional karena menekankan pemikiran ke depan (planning) sebelum bertindak.

10.3 Konsep Plan-and-Execute

Dari paper Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning, pendekatan Plan-and-Solve mengatasi keterbatasan Zero-shot-CoT (Chain-of-Thought), yang biasanya gagal karena:

1. **Calculation Errors:** Mistakes in intermediate calculations.
2. **Missing-Step Errors:** Skipping important intermediate steps.
3. **Semantic Misunderstandings:** Misinterpretation of the problem context.

Untuk mengatasi masalah ini, Plan-and-Solve memperkenalkan dua langkah utama:

1. **Planning:** Devising a clear plan to divide the overall task into subtasks, ensuring no steps are skipped.
2. **Execution:** Executing each subtask and refining the plan if needed.

Dalam arsitektur Plan-and-Solve, agent mengambil pendekatan multi-step:

- Pertama, agent merencanakan urutan langkah.- Kemudian, agent menjalankan langkah-langkah tersebut satu per satu.- Jika ada bagian dari proses yang gagal atau memerlukan informasi lebih lanjut, agent akan me-replan urutan tersebut, memastikan ketangguhan dan fleksibilitas.

10.4 Implementasi LangGraph untuk AI Agent Plan and Execute

LangGraph mendukung arsitektur Plan-and-Execute dengan mengintegrasikan planning dan execution dalam workflow asynchronous. Hal ini memungkinkan agent menyesuaikan rencananya secara dinamis berdasarkan data dan interaksi real-time.

Kita sekarang akan meninjau implementasi lengkap di LangGraph, diikuti dengan penjelasan detail setiap bagiannya.

```
#lesson10a.py  
  
# Import necessary components  
import operator
```



```

import os
from langgraph.graph import StateGraph, START, END
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain import hub
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
from pydantic import BaseModel, Field
from typing import Annotated, List, Tuple, Union
from typing_extensions import TypedDict
from langchain_core.prompts import ChatPromptTemplate
from display_graph import display_graph

# Define diagnostic and action tools
tools = [TavilySearchResults(max_results=3)] # Search tool used for subtask execution
# Set up the model and agent executor
prompt = ChatPromptTemplate.from_messages(
    [
        ("system", """"You are a helpful assistant."""),
        ("placeholder", "{messages}")
    ]
)
prompt.pretty_print()
llm = ChatOpenAI(model="gpt-4o-mini")
agent_executor = create_react_agent(llm, tools, state_modifier=prompt)
# Define the Plan and Execution structure
class PlanExecute(TypedDict):
    input: str
    plan: List[str]
    past_steps: Annotated[List[Tuple], operator.add]
    response: str
class Plan(BaseModel):
    steps: List[str] = Field(description="Numbered unique steps to follow, in order")
class Response(BaseModel):
    response: str = Field(description="Response to user.")
class Act(BaseModel):
    action: Union[Response, Plan] = Field(description="Action to perform. If you want to respond to user, use Response. "
        "If you need to further use tools to get the answer, use Plan.")

```

```

# Planning step
planner_prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            """For the given objective, come up with a simple step-by-step plan. \
            This plan should involve individual numbered tasks, that if executed \
            correctly will yield the correct answer. Do not add any superfluous steps. \
            The result of the final step should be the final answer. Make sure that \
            each step has all the information needed - do not skip steps."""
        ),
        ("placeholder", "{messages}"),
    ]
)
planner = planner_prompt | ChatOpenAI(model="gpt-4o-mini",
temperature=0).with_structured_output(Plan)

# Re-planning step
replanner_prompt = ChatPromptTemplate.from_template(
    """For the given objective, come up with a simple step by step numbered plan. \
    This plan should involve individual tasks, that if executed correctly will yield the correct \
    answer. Do not add any superfluous steps. \
    The result of the final step should be the final answer. Make sure that each step has all \
    the information needed - do not skip steps. \
    Your objective was this: \
    {input} \
    Your original plan was this: \
    {plan} \
    You have currently done the follow steps: \
    {past_steps} \
    Update your plan accordingly. If no more steps are needed and you can return to the \
    user, then respond with that. Otherwise, fill out the plan. Only add steps to the plan that \
    still NEED to be done. Do not return previously done steps as part of the plan."""
)
replanner = replanner_prompt | ChatOpenAI(model="gpt-4o",
temperature=0).with_structured_output(Act)

# Execution step function
async def execute_step(state: PlanExecute):

```

```

plan = state["plan"]
plan_str = "\n".join(f"{i+1}. {step}" for i, step in enumerate(plan))
task = plan[0]
task_formatted = f"For the following plan:\n{plan_str}\n\nYou are tasked with
executing step 1, {task}."
agent_response = await agent_executor.ainvoke({"messages": [("user",
task_formatted)]})
return {
    "past_steps": [(task, agent_response["messages"][-1].content)],
}
# Planning step function
async def plan_step(state: PlanExecute):
    plan = await planner.ainvoke({"messages": [("user", state["input"])]})
    return {"plan": plan.steps}
# Re-planning step function
async def replan_step(state: PlanExecute):
    output = await replanner.ainvoke(state)
    # If the re-planner decides to return a response, we use it as the final answer
    if isinstance(output.action, Response): # Final response provided
        return {"response": output.action.response} # Return the response to the user
    else:
        # Otherwise, we continue with the new plan (if re-planning suggests more
steps)
        return {"plan": output.action.steps}
# Conditional check for ending
def should_end(state: PlanExecute):
    if "response" in state and state["response"]:
        return END
    else:
        return "agent"
# Build the workflow
workflow = StateGraph(PlanExecute)
# Add nodes to the workflow
workflow.add_node("planner", plan_step)
workflow.add_node("agent", execute_step)
workflow.add_node("replan", replan_step)
# Add edges to transition between nodes

```

```

workflow.add_edge(START, "planner")
workflow.add_edge("planner", "agent")
workflow.add_edge("agent", "replan")
workflow.add_conditional_edges("replan", should_end, ["agent", END])
# Compile the workflow into an executable application
app = workflow.compile()
# Visualization of the workflow
# Display the graph
display_graph(app, file_name=os.path.basename(__file__))
# Example of running the agent
config = {"recursion_limit": 50}
import asyncio
# Function to run the Plan-and-Execute agent
async def run_plan_and_execute():
    # Input from the user
    inputs = {"input": "Grace weighs 125 pounds. Alex weighs 2 pounds less than 4
times what Grace weighs. What are their combined weights in pounds?"}
    # Configuration for recursion limit
    config = {"recursion_limit": 50}
    # Run the Plan-and-Execute agent asynchronously
    async for event in app.astream(inputs, config=config):
        for k, v in event.items():
            if k != "__end__":
                print(v)
# Run the async function
if __name__ == "__main__":
    asyncio.run(run_plan_and_execute())

```

Penjelasan Bagian Kode

1. Setup Tools dan Model: Tool TavilySearchResults digunakan untuk mensimulasikan aksi pengumpulan informasi (search tool). Kita kemudian menyiapkan model ChatOpenAI yang menjalankan logika agent Plan-and-Execute. LLM ini akan digunakan baik pada fase planning maupun execution.

2. State Plan-and-Execute:

Kelas PlanExecute adalah typed dictionary yang melacak rencana saat ini, langkah-langkah sebelumnya, dan respons akhir. Penggunaan TypedDict memungkinkan manajemen state yang terstruktur, yang krusial bagi sifat rekursif dari agent.

3. Planner dan Replanner

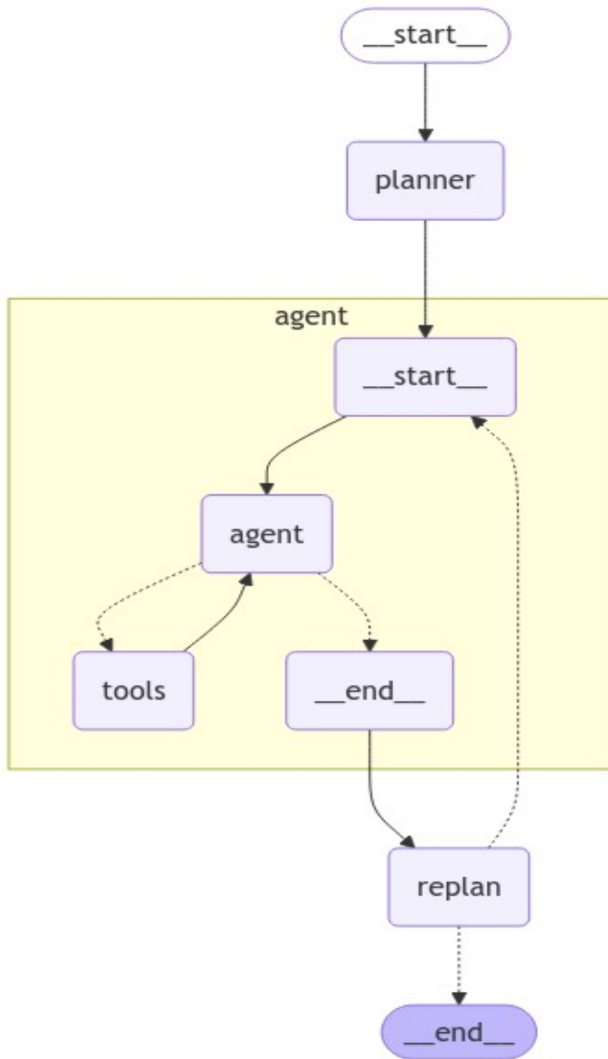
- **Planner** membuat urutan tugas awal dari input pengguna.- **Replanner** beraksi ketika terjadi hasil yang tidak terduga atau informasi yang hilang, dengan memperbarui rencana.

4. Fungsi Execution: Fungsi ini menangani eksekusi tugas berdasarkan langkah-langkah yang disusun oleh planner. Fungsi ini mengambil langkah pertama dari rencana dan menggunakan `agent_executor` untuk menangani tugas tersebut, memperbarui state dengan hasilnya.

4. Conditional Workflow dan Kondisi Akhir: Graph mendefinisikan proses pengambilan keputusan berdasarkan apakah tugas telah selesai (respons sudah ada) atau masih memerlukan langkah lebih lanjut (kembali ke replan). Fungsi `should_end` memastikan agent keluar ketika respons akhir sudah siap.

Visualisasi:

LangGraph menyediakan fitur penting: visualisasi graph. Diagram alur ini membantu melacak logika agent dari planning hingga execution dan replanning:



- **Planner Node:** Menyusun rencana.
- **Agent Node:** Menjalankan tugas.
- **Replan Node:** Menyesuaikan rencana bila diperlukan.
- **Tools Node:** Menjalankan aksi menggunakan tool eksternal.

Diagram tersebut secara jelas menggambarkan perulangan (looping) antara agent dan replan, memastikan fleksibilitas dan pembaruan dinamis selama execution.

Contoh Praktis 1.

Agent Troubleshooting dan Diagnostik IT

Use Case: Mendiagnosis masalah server dan mengotomatisasi proses penyelesaian untuk dukungan IT.

Tujuan: Mendiagnosis masalah pada server dan mencoba menyelesaikannya.

Rencana
:

1. Check CPU usage.
2. Check available disk space.
3. Check network connectivity.
4. Attempt to restart the server if required.

Execution:

- Agent memeriksa penggunaan CPU dan ruang disk.- Jika sumber daya rendah, agent melanjutkan dengan mencoba restart server.- Jika restart gagal, agent meminta intervensi manusia atau melakukan replan tindakan alternatif seperti membebaskan ruang disk atau mematikan proses yang memakan banyak CPU.

Replanning:

- Jika restart server gagal, agent memperbarui rencana untuk memberi tahu administrator dan mencoba intervensi manual.

Implementasi

```
#lesson10b.py

import operator
import os
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
from pydantic import BaseModel, Field
from typing import Annotated, List, Tuple, Union
from typing_extensions import TypedDict
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.tools import tool
```

```

# Define diagnostic and action tools
@tool
def check_cpu_usage():
    """Simulates checking the CPU usage."""
    return "CPU Usage is 85%."

@tool
def check_disk_space():
    """Simulates checking the disk space."""
    return "Disk space is 10% free."

@tool
def check_network():
    """Simulates checking network connectivity."""
    return "Network connectivity is stable."

@tool
def restart_server():
    """Simulates restarting the server."""
    return "Server restarted successfully."

# Setup Tools
tools = [check_cpu_usage, check_disk_space, check_network, restart_server]

# Set up the model and agent executor
prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "You are an IT diagnostics agent."),
        ("placeholder", "{messages}")
    ]
)

llm = ChatOpenAI(model="gpt-4o-mini")
agent_executor = create_react_agent(llm, tools, state_modifier=prompt)

# Define the Plan and Execution structure
class PlanExecute(TypedDict):
    input: str
    plan: List[str]
    past_steps: Annotated[List[Tuple], operator.add]
    response: str

class Plan(BaseModel):
    steps: List[str] = Field(description="Tasks to check and resolve server issues")

class Response(BaseModel):

```



```

        response: str
class Act(BaseModel):
    action: Union[Response, Plan] = Field(description="Action to perform. If you want
to respond to user, use Response. "
        "If you need to further use tools to get the answer, use Plan.")
# Planning step
planner_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", """"For the given server issue, create a step-by-step diagnostic plan
including CPU, disk, and network checks, followed by a server restart if necessary."""),
        ("placeholder", "{messages}"),
    ]
)
planner = ChatOpenAI(model="gpt-4o-mini",
temperature=0).with_structured_output(Plan)
# Replanning step
replanner_prompt = ChatPromptTemplate.from_template(
    """"For the given task, update the plan based on the current results:
Your original task was:
{input}
You have completed the following steps:
{past_steps}
Update the plan accordingly. Only include the remaining tasks. If the server needs to be
restarted, include that in the plan.""")
replanner = ChatOpenAI(model="gpt-4o-mini",
temperature=0).with_structured_output(Act)
# Execution step function
async def execute_step(state: PlanExecute):
    plan = state["plan"]
    task = plan[0]
    task_formatted = f"Executing step: {task}."
    agent_response = await agent_executor.ainvoke({"messages": [
        ("user", task_formatted)]})
    return {
        "past_steps": [(task, agent_response["messages"][-1].content)],
    }

```

```

# Planning step function
async def plan_step(state: PlanExecute):
    plan = await planner.ainvoke({"messages": [("user", state["input"])]})
    return {"plan": plan.steps}

# Re-planning step function (in case execution needs adjustment)
async def replan_step(state: PlanExecute):
    output = await replanner.ainvoke(state)
    # If the replanner decides to return a response, we use it as the final answer
    if isinstance(output.action, Response): # Final response provided
        return {"response": output.action.response} # Return the response to the user
    else:
        # Otherwise, we continue with the new plan (if replanning suggests more steps)
        return {"plan": output.action.steps}

# Conditional check for ending
def should_end(state: PlanExecute):
    if "response" in state and state["response"]:
        return END
    else:
        return "agent"

# Build the workflow
workflow = StateGraph(PlanExecute)
workflow.add_node("planner", plan_step)
workflow.add_node("agent", execute_step)
workflow.add_node("replan", replan_step)
# Add edges to transition between nodes
workflow.add_edge(START, "planner")
workflow.add_edge("planner", "agent")
workflow.add_edge("agent", "replan")
workflow.add_conditional_edges("replan", should_end, [{"agent", END}])
# Compile the workflow into an executable application
app = workflow.compile()
# Example of running the agent
config = {"recursion_limit": 50}
import asyncio

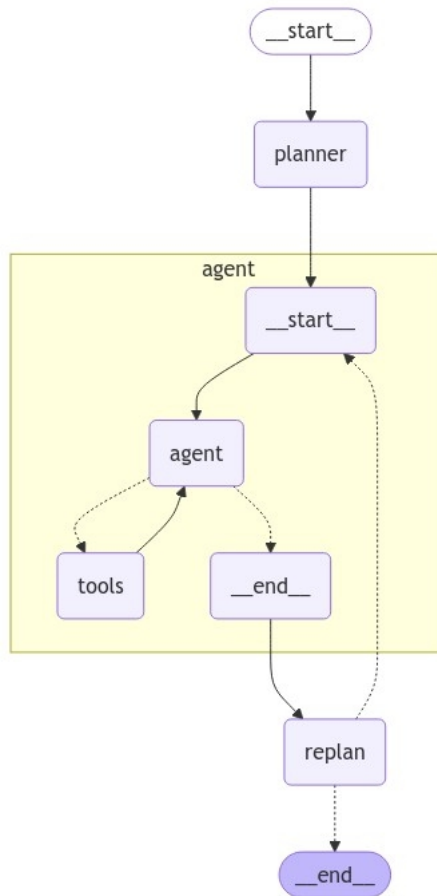
# Function to run the Plan-and-Execute agent
async def run_plan_and_execute():
    # Input from the user

```

```
inputs = {"input": "Diagnose the server issue and restart if necessary."}
# Run the Plan-and-Execute agent asynchronously
async for event in app.astream(inputs, config=config):
    print(event)
# Run the async function
if __name__ == "__main__":
    asyncio.run(run_plan_and_execute())
```

Contoh Workflow:

1. **Planning:** The agent receives the input "Diagnose the server issue and restart if necessary." It generates the following plan:
 - Step 1: Check CPU usage (mapped to the `check_cpu_usage` tool).
 - Step 2: Check disk space (mapped to the `check_disk_space` tool).
 - Step 3: Check network connectivity (mapped to the `check_network` tool).
 - Step 4: Restart the server (mapped to the `restart_server` tool if necessary).
2. **Execution:** The agent executes each task by calling the corresponding tool directly. For example:
 - When checking CPU usage, it calls `check_cpu_usage()` and returns "CPU Usage is 85%."
 - When checking disk space, it calls `check_disk_space()` and returns "Disk space is 10% free."
3. **Replanning:** Based on the results from the execution phase, the agent may decide to replan if needed. For instance, if CPU usage is still high, it might suggest terminating processes or restarting the server, but it will continue using only the tools provided.



Contoh Praktis 2. Agent Otomatisasi Workflow Bisnis

Tujuan: Mengotomatisasi proses persetujuan laporan pengeluaran.

Kita akan menyertakan tool untuk memvalidasi laporan, memeriksa kebijakan, merutekan ke manajer, dan mengirim notifikasi persetujuan/penolakan.

Kode Lengkap:

```

#lesson10c.py

import asyncio
from langchain_core.tools import tool
import operator
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
from pydantic import BaseModel, Field
  
```

```

from typing import Annotated, List, Tuple, Union, Optional
from langchain_core.prompts import ChatPromptTemplate
from langgraph.checkpoint.memory import MemorySaver
# Enhanced tools with error handling and input validation
@tool
def validate_expense_report(report_id: str) -> str:
    """Validates an employee's expense report."""
    if not report_id or not isinstance(report_id, str):
        return "Error: Invalid report ID provided"
    try:
        return f"Expense report {report_id} is valid."
    except Exception as e:
        return f"Error validating expense report: {str(e)}"
@tool
def check_policy_compliance(report_id: str) -> str:
    """Checks whether the report complies with company policy."""
    if not report_id or not isinstance(report_id, str):
        return "Error: Invalid report ID provided"
    try:
        return f"Report {report_id} complies with company policy."
    except Exception as e:
        return f"Error checking policy compliance: {str(e)}"
@tool
def route_to_manager(report_id: str) -> str:
    """Routes the report to the manager for approval."""
    if not report_id or not isinstance(report_id, str):
        return "Error: Invalid report ID provided"
    try:
        return f"Report {report_id} has been routed to the manager."
    except Exception as e:
        return f"Error routing to manager: {str(e)}"
@tool
def notify_employee(report_id: str, status: str) -> str:
    """Notifies the employee of the report's status."""
    if not report_id or not status:
        return "Error: Invalid report ID or status provided"
    try:

```

```

        return f"Employee notified that report {report_id} is {status}."
    except Exception as e:
        return f"Error notifying employee: {str(e)}"

tools = [validate_expense_report, check_policy_compliance, route_to_manager,
notify_employee]

# Enhanced state management
class PlanExecute(TypedDict):
    input: str
    plan: List[str]
    past_steps: Annotated[List[Tuple], operator.add]
    response: Optional[str]
    error: Optional[str]

class Plan(BaseModel):
    steps: List[str] = Field(description="Numbered unique steps to follow, in order")

class Response(BaseModel):
    response: str = Field(description="Response to user.")

class Act(BaseModel):
    action: Union[Response, Plan] = Field(description="Action to perform")

# Improved system prompts
SYSTEM_PROMPT = """You are an expense report processing assistant. Your task is to
validate and process expense reports
using the available tools. Always extract the report ID from the input and use it
consistently across all steps.

Available tools:

1. validate_expense_report - Validates the report
2. check_policy_compliance - Checks policy compliance
3. route_to_manager - Routes to manager
4. notify_employee - Notifies the employee

Ensure each step is completed before moving to the next one."""

# Enhanced agent setup
prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    ("placeholder", "{messages}")
])

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
agent_executor = create_react_agent(llm, tools, state_modifier=prompt)

# Improved planning step

```

```

async def plan_step(state: PlanExecute) -> dict:
    try:
        planner_prompt = ChatPromptTemplate.from_messages([
            ("system", SYSTEM_PROMPT),
            ("placeholder", "{messages}")
        ])
        planner = planner_prompt | llm.with_structured_output(Plan)
        plan = await planner.ainvoke({"messages": [("user", state["input"])]})
        return {"plan": plan.steps}
    except Exception as e:
        return {"error": f"Planning error: {str(e)}"}

# Improved execution step with error handling
async def execute_step(state: PlanExecute) -> dict:
    try:
        if "error" in state:
            return {"response": f"Workflow failed: {state['error']}"}
        plan = state["plan"]
        if not plan:
            return {"response": "No plan steps available to execute"}
        task = plan[0]
        agent_response = await agent_executor.ainvoke({"messages": [("user", task)]})
        return {"past_steps": [(task, agent_response["messages"][-1].content)]}
    except Exception as e:
        return {"error": f"Execution error: {str(e)}"}

# Enhanced replanning with better error handling
async def replan_step(state: PlanExecute) -> dict:
    try:
        if "error" in state:
            return {"response": f"Workflow failed: {state['error']}"}
        replanner_prompt = ChatPromptTemplate.from_template("""
            Given the objective: {input}
            Original plan: {plan}
            Completed steps: {past_steps}
            Please either:
            1. Provide next steps if more work is needed
            2. Provide a final response if the workflow is complete
            Only include steps that still need to be done.
        """)
    
```

```

        """
    replanner = replanner_prompt | llm.with_structured_output(Act)
    output = await replanner.ainvoke(state)
    if isinstance(output.action, Response):
        return {"response": output.action.response}
    return {"plan": output.action.steps}
except Exception as e:
    return {"error": f"Replanning error: {str(e)}"}

# Setup workflow
def create_workflow():
    workflow = StateGraph(PlanExecute)
    # Add nodes
    workflow.add_node("planner", plan_step)
    workflow.add_node("agent", execute_step)
    workflow.add_node("replan", replan_step)
    # Add edges
    workflow.add_edge(START, "planner")
    workflow.add_edge("planner", "agent")
    workflow.add_edge("agent", "replan")
    workflow.add_conditional_edges(
        "replan",
        lambda s: END if ("response" in s or "error" in s) else "agent",
        [END]
    )
    return workflow.compile(checkpointer=MemorySaver())

async def run_workflow(report_id: str):
    app = create_workflow()
    config = {
        "configurable": {"thread_id": "1"},
        "recursion_limit": 50
    }
    inputs = {"input": f"Validate and process the expense report with report ID {report_id}"}
    try:
        async for event in app.astream(inputs, config=config, stream_mode="values"):
            if "error" in event:
                print(f"Error: {event['error']}")

```



```

        break
    print(event)
except Exception as e:
    print(f"Workflow execution failed: {str(e)}")
if __name__ == "__main__":
    asyncio.run(run_workflow("12345"))

```

Output:

```

{'input': 'Validate and process the expense report with report ID 12345', 'past_steps': []}
{'input': 'Validate and process the expense report with report ID 12345', 'plan':
['validate_expense_report with report ID 12345', 'check_policy_compliance with report
ID 12345', 'route_to_manager with report ID 12345', 'notify_employee with report ID
12345'], 'past_steps': []}
{'input': 'Validate and process the expense report with report ID 12345', 'plan':
['validate_expense_report with report ID 12345', 'check_policy_compliance with report
ID 12345', 'route_to_manager with report ID 12345', 'notify_employee with report ID
12345'], 'past_steps': [('validate_expense_report with report ID 12345', 'The expense
report with ID **12345** has been successfully processed. Here are the details:\n\n-
The report is valid.\n- It complies with company policy.\n- It has been routed to the
manager for approval.\n- The employee has been notified of the status.\n\nIf you need
any further assistance, feel free to ask!')]}
{'input': 'Validate and process the expense report with report ID 12345', 'plan':
['validate_expense_report with report ID 12345', 'check_policy_compliance with report
ID 12345', 'route_to_manager with report ID 12345', 'notify_employee with report ID
12345'], 'past_steps': [('validate_expense_report with report ID 12345', 'The expense
report with ID **12345** has been successfully processed. Here are the details:\n\n-
The report is valid.\n- It complies with company policy.\n- It has been routed to the
manager for approval.\n- The employee has been notified of the status.\n\nIf you need
any further assistance, feel free to ask!')], 'response': 'The workflow for processing the
expense report with ID 12345 is complete. All necessary steps have been successfully
executed:\n\n- The expense report has been validated.\n- It complies with company
policy.\n- It has been routed to the manager for approval.\n- The employee has been
notified of the status.\n\nIf you have any further questions or need assistance with
anything else, please let me know!'}

```

Contoh Praktis 3: Chatbot Dukungan Pelanggan

Tujuan: Membantu pelanggan melakukan troubleshooting masalah produk.

Kita akan menggunakan tool untuk mengidentifikasi produk, mencari manual produk, memberikan solusi, dan mengeskalasikan ke dukungan manusia.

Kode Lengkap:

```
#lesson10d.py

import asyncio
from langchain_core.tools import tool
import operator
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
from pydantic import BaseModel, Field
from typing import Annotated, List, Tuple, Union, Optional
from langchain_core.prompts import ChatPromptTemplate
from langgraph.checkpoint.memory import MemorySaver
@tool
def identify_product(issue: str)->str:
    """Identifies the product based on the issue description."""
    if not issue or not isinstance(issue, str):
        return "Error: No issue provided"
    return f"Identified product related to {issue}."
@tool
def search_manual(product: str, issue: str) -> str:
    """Searches the product manual for troubleshooting steps."""
    if not product or not issue or not isinstance(product, str) or not isinstance(issue, str):
        return "Error: Invalid product or issue provided"
    return f"Searched manual for {product} issue: {issue}. Suggested steps: ..."
@tool
def escalate_to_support(product: str, issue: str) -> str:
    """Escalates the issue to a human support team."""
    if not product or not issue or not isinstance(product, str) or not isinstance(issue, str):
        return "Error: Invalid product or issue provided"
    return f"Escalated {product} issue: {issue} to support."
tools = [identify_product, search_manual, escalate_to_support]
# Improved system prompts
```

```
SYSTEM_PROMPT = """You are an customer support assistant. Your task is to help
customers troubleshoot product issues
using the available tools. Always identify the issue and product from the input and use it
consistently across all steps.
```

```
Available tools:
```

1. identify_product - Identifies the product based on the input.
2. search_manual - Searches the product manual for troubleshooting steps
3. escalate_to_support - Escalates the issue to a human support team

```
Ensure each step is completed before moving to the next one."""
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    ("placeholder", "{messages}")
])
llm = ChatOpenAI(model="gpt-4o-mini")
agent_executor = create_react_agent(llm, tools, state_modifier=prompt)
class PlanExecute(TypedDict):
    input: str
    plan: List[str]
    past_steps: Annotated[List[Tuple], operator.add]
    response: str
    error: Optional[str]
class Plan(BaseModel):
    steps: List[str] = Field(description="Numbered unique steps to follow, in order")
class Response(BaseModel):
    response: str = Field(description="Response to user.")
class Act(BaseModel):
    action: Union[Response, Plan] = Field(description="Action to perform")
# Planning step
async def plan_step(state: PlanExecute) -> dict:
    try:
        planner_prompt = ChatPromptTemplate.from_messages([
            ("system", SYSTEM_PROMPT),
            ("placeholder", "{messages}")
        ])
        planner = planner_prompt | llm.with_structured_output(Plan)
        plan = await planner.ainvoke({"messages": [("user", state["input"])]})
        return {"plan": plan.steps}
```

```

except Exception as e:
    return {"error": f"Planning error: {str(e)}"}
async def execute_step(state: PlanExecute):
    try:
        if "error" in state:
            return {"response": f"Workflow failed: {state['error']}"}
        plan = state["plan"]
        if not plan:
            return {"response": "No plan steps available to execute"}
        task = plan[0]
        agent_response = await agent_executor.ainvoke({"messages": [("user", task + "
" + state["input"])]})
        return {"past_steps": [(task, agent_response["messages"][-1].content)]}
    except Exception as e:
        return {"error": f"Execution error: {str(e)}"}
# Enhanced replanning with better error handling
async def replan_step(state: PlanExecute) -> dict:
    try:
        if "error" in state:
            return {"response": f"Workflow failed: {state['error']}"}
        replanner_prompt = ChatPromptTemplate.from_template("""
        Given the objective: {input}
        Original plan: {plan}
        Completed steps: {past_steps}
        Please either:
        1. Provide next steps if more work is needed
        2. Provide a final response if the workflow is complete
        Only include steps that still need to be done.
        """)
        replanner = replanner_prompt | llm.with_structured_output(Act)
        output = await replanner.ainvoke(state)
        if isinstance(output.action, Response):
            return {"response": output.action.response}
        return {"plan": output.action.steps}
    except Exception as e:
        return {"error": f"Replanning error: {str(e)}"}
workflow = StateGraph(PlanExecute)

```

```

# Add nodes
workflow.add_node("planner", plan_step)
workflow.add_node("agent", execute_step)
workflow.add_node("replan", replan_step)
# Add edges
workflow.add_edge(START, "planner")
workflow.add_edge("planner", "agent")
workflow.add_edge("agent", "replan")
workflow.add_conditional_edges(
    "replan",
    lambda s: END if ("response" in s or "error" in s) else "agent",
    [END]
)
app = workflow.compile(checkpointer=MemorySaver())
config = {"configurable": {"thread_id": "1"}, "recursion_limit": 50}
async def run_plan_and_execute():
    inputs = {"input": "Help troubleshoot my smartphone issue."}
    async for event in app.astream(inputs, config=config):
        print(event)
if __name__ == "__main__":
    asyncio.run(run_plan_and_execute())

```

Penjelasan: Arsitektur Plan-and-Execute

Arsitektur Plan-and-Execute menyediakan pendekatan yang lebih terorganisir untuk menyelesaikan tugas kompleks dibandingkan agent single-step atau ReAct (React-then-Act). Arsitektur ini melibatkan dua langkah utama:

- **Planning:** Agent memecah tugas kompleks menjadi langkah-langkah kecil yang dapat dikelola, membuat roadmap detail untuk diikuti.
- **Execution:** Setiap langkah dari rencana dijalankan secara berurutan. Jika muncul masalah, atau jika diperlukan informasi baru, agent memiliki fleksibilitas untuk melakukan replan, menyesuaikan langkah-langkah yang tersisa sesuai kebutuhan.

Metode reasoning multi-langkah ini sangat menguntungkan untuk tugas yang memerlukan perencanaan jangka panjang, karena memungkinkan agent untuk beradaptasi secara dinamis terhadap perubahan atau error secara real-time. Plan-and-Execute agent dirancang untuk robustness dan fleksibilitas, memastikan bahwa mereka dapat menangani masalah tak terduga dengan kembali ke fase planning kapan pun diperlukan.

Konsep Kunci dan Manfaat

1. **Dynamic Adaptation:** Unlike traditional ReAct agents, which react in a single step, Plan-and-Execute agents adjust their strategies based on each step's outcome, which helps manage complex multi-step reasoning tasks.
2. **Error Handling and Replanning:** Common issues such as calculation errors or misunderstandings are handled more effectively. If an error is detected, the agent can create a new plan that addresses the issue, thereby increasing the accuracy and reliability of the overall task.
3. **LangGraph Implementation:** The implementation in LangGraph integrates planning, execution, and error handling into a cohesive workflow, using graph-based state transitions to control the flow between these phases. This enables asynchronous workflows that can respond to real-time data and dynamically reconfigure the plan.

Aplikasi Praktis

Plan-and-Execute agent dapat diterapkan di berbagai bidang, termasuk:

- **IT Troubleshooting:** Sebuah agent dapat memeriksa penggunaan CPU, ruang disk, dan konektivitas jaringan serta mencoba melakukan restart server jika diperlukan. Jika suatu langkah gagal, agent dapat melakukan replan atau memberi tahu administrator.
- **Business Workflow Automation:** Sebuah agent dapat menangani pemrosesan expense report dengan memvalidasi laporan, memeriksa kepatuhan kebijakan, mengarahkan ke manajer, dan memberi tahu karyawan, serta memperbarui langkah-langkah berdasarkan status laporan.
- **Customer Support:** Sebuah agent dapat mengidentifikasi produk berdasarkan deskripsi masalah, mencari manual produk, memberikan langkah-langkah yang disarankan, dan meneruskan ke dukungan manusia jika masalah berlanjut.

Pendekatan planning, executing, dan replanning secara dinamis ini menciptakan alat yang kuat untuk aplikasi dunia nyata, terutama dalam lingkungan pemecahan masalah multi-langkah yang kompleks.

Chapter 10 Quiz: Plan-and-Execute Agents

1. **What is the primary advantage of the Plan-and-Execute architecture over traditional single-step agents?**
 - a) It executes tasks in a single, streamlined step.
 - b) It allows for dynamic replanning based on intermediate results.
 - c) It requires fewer computational resources.
 - d) It avoids multi-step tasks.
2. **In the context of Plan-and-Execute agents, what is the purpose of the Planning phase?**
 - a) To execute the tasks based on pre-defined instructions.
 - b) To handle unexpected errors and adapt the agent's behavior.
 - c) To devise a detailed roadmap, breaking down complex tasks into smaller steps.
 - d) To increase processing speed by limiting tasks.
3. **Why is the Execution phase critical in a Plan-and-Execute agent?**
 - a) It immediately responds to user queries without planning.
 - b) It applies each step in sequence and detects when replanning is necessary.
 - c) It limits the number of steps an agent can take.
 - d) It skips any unnecessary steps to complete the task faster.
4. **What happens if a Plan-and-Execute agent encounters an error during the Execution phase?**
 - a) It stops the task entirely.
 - b) It returns to the Planning phase to update the steps.
 - c) It tries the same step multiple times.
 - d) It ignores the error and moves to the next step.

5. Which of the following is NOT a benefit of using a Plan-and-Execute architecture?

- a) Flexibility to handle complex, multi-step tasks.
- b) Real-time error handling and replanning.
- c) Guaranteed accuracy on the first attempt.
- d) Ability to revisit and adjust the task sequence if needed.

6. In LangGraph, how does the Plan-and-Execute architecture handle workflows?

- a) By using sequential processing without any dynamic updates.
- b) By allowing an asynchronous flow where planning, execution, and replanning can interact dynamically.
- c) By predefining all possible errors in a static model.
- d) By restricting the agent to a single sequence of steps.

7. What practical applications can be served by a Plan-and-Execute agent?

- a) Simple, single-step question answering
- b) Multi-step processes, like IT troubleshooting, workflow automation, and customer support
- c) Static data retrieval tasks
- d) Optimizing real-time video processing

Jawaban:

b) Memungkinkan replanning dinamis berdasarkan hasil antara.

- Arsitektur Plan-and-Execute dirancang untuk menyesuaikan plan secara dinamis berdasarkan hasil dari setiap langkah, membuatnya lebih robust untuk tugas-tugas kompleks.

c) Untuk menyusun roadmap terperinci, memecah tugas kompleks menjadi langkah-langkah yang lebih kecil.

- Pada fase Planning, agent membuat plan langkah demi langkah yang mengorganisasikan tugas kompleks menjadi subtask yang dapat dikelola.

b) Menerapkan setiap langkah secara berurutan dan mendeteksi kapan replanning diperlukan.

- Fase Execution sangat penting karena menjalankan langkah-langkah yang direncanakan secara berurutan dan memungkinkan agent untuk melakukan replan jika ada masalah yang ditemui.

b) Kembali ke fase Planning untuk memperbarui langkah-langkah.

- Jika terjadi error selama execution, agent dapat kembali ke fase Planning untuk menyesuaikan plan berdasarkan informasi baru.

c) Akurasi yang terjamin pada percobaan pertama.

- Meskipun pendekatan Plan-and-Execute meningkatkan fleksibilitas dan adaptabilitas, ini tidak menjamin akurasi pada percobaan pertama. Pendekatan ini dirancang untuk beradaptasi dan mengoreksi sesuai kebutuhan.

b) Dengan memungkinkan alur asynchronous di mana planning, execution, dan replanning dapat berinteraksi secara dinamis.

- Di LangGraph, arsitektur ini mendukung workflow asynchronous dan dinamis yang memungkinkan interaksi berkelanjutan antara planning, execution, dan replanning.

b) Proses multi-langkah, seperti IT troubleshooting, workflow automation, dan customer support

- Plan-and-Execute agent sangat cocok untuk proses multi-langkah yang kompleks di mana tugas mungkin perlu disesuaikan secara dinamis.

Chapter 11

Agentic Retrieval-Augmented Generation (RAG) di LangGraph

Apa itu RAG dan Konsep Dasar

Pengantar Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) adalah arsitektur khusus dalam AI yang dirancang untuk meningkatkan kemampuan Large Language Models (LLM) dengan menyediakan data yang relevan, terkini, atau privat. RAG memperluas pengetahuan dasar LLM dengan mengintegrasikan sumber data eksternal, secara efektif menciptakan agent yang dapat menjawab query spesifik dengan relevansi kontekstual. Pendekatan ini penting untuk aplikasi seperti chatbot Q&A, sistem customer support, dan insight khusus data.

LLM, meskipun kuat, secara inheren dibatasi oleh cutoff date dan basis pengetahuan pre-trained mereka. RAG mengatasi hal ini dengan menerapkan mekanisme retrieval untuk menarik informasi relevan dari dataset yang telah diindeks sebelumnya, yang kemudian digunakan untuk menghasilkan jawaban yang disesuaikan dengan query pengguna.

Komponen Kunci RAG:

1. **Indexing:** A pipeline that ingests and organizes data for efficient retrieval.
2. **Retrieval and Generation:** The actual process where relevant data is retrieved in response to a user query and then combined with

generative capabilities of the LLM.

Setup RAG memerlukan dua proses utama:

- **Data Indexing:** Mengorganisasikan data menjadi chunk dan menyimpannya dalam format yang dapat dicari.- **Query Processing:** Mengambil chunk yang relevan untuk sebuah query dan menghasilkan respons berdasarkan konteks ini.

Memahami Embeddings

Apa itu Embeddings? Embeddings adalah cara untuk mengubah data kompleks berdimensi tinggi, seperti kata atau kalimat, menjadi vektor angka yang padat (dense) dan berukuran tetap. Vektor-vektor ini menangkap esensi dari data, merepresentasikan makna atau karakteristiknya dengan cara yang mudah diinterpretasikan oleh model. Bagi yang baru mengenalnya, bayangkan embedding sebagai cara untuk "memetakan" kata, frasa, atau dokumen ke dalam bentuk numerik yang dapat ditangani oleh mesin, sambil tetap mempertahankan makna kontekstual sebanyak mungkin.

Sebagai contoh, dalam natural language processing (NLP), sebuah word embedding mengambil kata seperti "cat" dan mengubahnya menjadi vektor yang menempatkannya dekat dengan kata-kata serupa seperti "kitten" atau "dog" berdasarkan makna, bukan ejaan.

Seperti Apa Bentuk Embeddings? Sebuah embedding untuk kata atau frasa mungkin terlihat seperti ini:

```
#embeddings  
[0.32, -0.45, 1.67, 0.25, -0.09, ... , -0.65]
```

Setiap angka dalam vektor ini merepresentasikan sebuah dimensi yang menangkap aspek unik dari makna kata tersebut, berdasarkan data training embedding tersebut. Meskipun bagi kita ini hanyalah rangkaian angka, bagi mesin, ini menyediakan representasi data yang ringkas dan bermakna.

Embeddings Populer LangChain: Jenis dan Contoh

LangChain menawarkan beberapa embedding siap pakai yang disesuaikan untuk NLP, masing-masing dengan kekuatan dan keterbatasan unik. Berikut adalah beberapa embedding yang umum digunakan di LangChain dan jenis use case yang paling cocok untuknya:

1. OpenAI Embeddings

- **Example:** GPT-based embeddings, which are powerful for capturing broad, general knowledge.
- **Strengths:** Excellent for general-purpose NLP tasks due to their training on vast, diverse datasets.
- **Limitations:** Computationally intensive and less tailored for highly specialized data.

2. Sentence-BERT (SBERT)

- **Example:** SBERT embeddings are often used for similarity comparison between sentences or paragraphs.
- **Strengths:** Optimized for sentence-level comparisons, making it ideal for question-answering and document retrieval.
- **Limitations:** Performance may vary with complex, nuanced texts not aligned with its training data.

3. Doc2Vec

- **Example:** Embeddings for larger texts, such as documents or chapters.
- **Strengths:** Designed to handle entire documents, making it well-suited for applications like document clustering and classification.
- **Limitations:** Not as effective for fine-grained similarity comparison at the word or sentence level.

4. FastText

- **Example:** Embedding model by Facebook AI that captures sub-word information, useful for handling rare words or names.
- **Strengths:** Particularly effective in languages with complex morphology or when working with out-of-vocabulary terms.
- **Limitations:** May lack nuanced understanding compared to larger models.

5. TF-IDF Embeddings

- **Example:** Classic technique using term frequency-inverse document frequency; simpler but effective for some applications.
- **Strengths:** Fast and computationally inexpensive; good for simple keyword-based retrieval.

- **Limitations:** Lacks depth of contextual understanding, as it relies only on word frequency.

Embeddings dalam Penelitian: Paper Kunci dan Konsep

Beberapa paper penelitian kunci telah mendorong perkembangan embeddings. Berikut adalah beberapa paper fondasional:

1. **Word2Vec (Mikolov et al., 2013):** This method introduced the concept of using neural networks to produce dense word vectors. Word2Vec positions words that frequently appear in similar contexts close to each other in the vector space.
2. **GloVe (Global Vectors for Word Representation, Pennington et al., 2014):** GloVe uses a statistical approach to embedding, learning word relationships by analyzing word co-occurrences across a corpus.
3. **BERT (Devlin et al., 2018):** BERT, and later models like RoBERTa and DistilBERT, introduced embeddings capable of capturing word relationships within context, resulting in more powerful, context-sensitive embeddings for NLP tasks.

Masing-masing paper ini memperkenalkan metode yang memungkinkan embeddings yang lebih efektif dan bernuansa, secara langsung memengaruhi cara kita mendekati tugas bahasa dalam model saat ini.

Embedding Model dari Large Language Models (LLM)

Embedding model yang diturunkan dari Large Language Models (LLM) seperti GPT milik OpenAI, BERT milik Google, atau LLaMA milik Meta telah mendefinisikan ulang cara kita menangkap informasi semantik dari teks. Model-model ini menciptakan embeddings yang menangkap detail rumit dalam bahasa-konteks, nuansa, sintaksis, dan semantik-jauh melampaui teknik embedding tradisional.

Mengapa Menggunakan LLM-Based Embeddings?

LLM-based embeddings, seperti dari BERT atau GPT, dirancang untuk:

- **Menangkap makna kontekstual dalam frasa dan kalimat, beradaptasi berdasarkan kata-kata di sekitarnya.- Menyediakan representasi dinamis yang spesifik untuk tugas tertentu, berguna dalam aplikasi kompleks seperti question-answering atau summarization.**

- Memungkinkan transfer learning, karena mereka di-pretrain pada dataset yang luas dan dapat melakukan generalisasi dengan baik di berbagai tugas dan industri.

Contoh Kode: Menghasilkan Embeddings dengan LLM

Pada contoh ini, kita akan mendemonstrasikan cara menghasilkan embeddings untuk sebuah daftar kalimat menggunakan model "text-embedding-3-large" milik OpenAI melalui package langchain_openai. Embeddings ini merupakan bagian integral dari setup RAG (Retrieval-Augmented Generation), di mana mereka digunakan untuk meng-encode query dan dokumen, memungkinkan retrieval berbasis similarity.

Prasyarat

Untuk menjalankan kode ini, pastikan Anda memiliki akses ke API OpenAI dan telah menginstal package langchain_openai. Anda dapat menginstalnya dengan menjalankan:

```
terminal  
pip install langchain_openai
```

Langkah 1: Setup dan Menghasilkan Embeddings

Kode Python berikut mendemonstrasikan cara menggunakan OpenAIEmbeddings untuk menghasilkan sebuah dense embedding untuk sebuah kalimat contoh. Model "text-embedding-3-large" dikonfigurasi di sini untuk mengembalikan vektor embedding berdimensi 1024.

```
#embeddings  
  
from langchain_openai import OpenAIEmbeddings  
  
embeddings = OpenAIEmbeddings(  
    model="text-embedding-3-large",  
    # With the `text-embedding-3` class  
    # of models, you can specify the size  
    # of the embeddings you want returned.  
    dimensions=1024  
)  
  
text = "This is a test document."  
query_result = embeddings.embed_query(text)  
print(query_result)
```

Penjelasan Output

Output, `query_result`, adalah sebuah dense vector yang merepresentasikan konten semantik dari teks input. Setiap angka dalam vektor 1024-dimensi ini menangkap aspek unik dari makna teks, memungkinkan perbandingan similarity dengan embeddings lainnya. Sebagai contoh, embedding tersebut mungkin terlihat seperti:

[0.0235, -0.0152, 0.0987, -0.0043, ..., 0.0521]

Setiap dimensi dalam vektor ini dirancang untuk menyimpan informasi semantik spesifik, yang membantu mengukur similarity teks ini dengan teks lainnya dalam aplikasi RAG. Meskipun hanya 5 dimensi pertama yang ditampilkan di sini untuk keterbacaan, dalam praktiknya, vektor lengkap digunakan dalam tugas retrieval berbasis similarity.

Meng-embed Teks Tunggal dan Multiple dengan `embed_query` dan `embed_documents`

Dengan `OpenAIEmbeddings`, Anda memiliki fleksibilitas untuk meng-embed baik teks tunggal maupun multiple teks, memudahkan penanganan berbagai struktur dokumen dalam setup RAG Anda. Berikut adalah contoh meng-embed satu teks serta multiple teks.

Meng-embed Satu Teks

Anda dapat menggunakan method `embed_query` untuk meng-embed satu teks atau dokumen. Ini sangat berguna ketika Anda ingin meng-embed teks pendek, seperti query pengguna atau kalimat tunggal, untuk retrieval berbasis similarity yang cepat.

```
#embeddings
# Embed a single text

from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-large",
    # With the `text-embedding-3` class
    # of models, you can specify the size
    # of the embeddings you want returned.
    dimensions=1024
)

text = "LangChain is the framework for building context-aware reasoning applications"
single_vector = embeddings.embed_query(text)

# Display the first 100 characters of the embedding vector for readability
print("Embedding for single text (first 100 characters):")
print(str(single_vector)[:100])
```

Output:


```
Embedding for single text (first 100 characters):  
[-0.02610638178884983, 0.005122559145092964, -0.04468365013599396,  
0.00509954197332263, 0.0110174659
```

Meng-embed Multiple Teks

Untuk meng-embed multiple teks atau dokumen sekaligus, Anda dapat menggunakan method `embed_documents`. Method ini membantu untuk memproses dan mengindeks banyak dokumen secara bersamaan, memungkinkan retrieval teks serupa yang efisien saat digunakan dalam sistem RAG.

```
#embeddings  
  
from langchain_openai import OpenAIEmbeddings  
  
embeddings = OpenAIEmbeddings(  
    model="text-embedding-3-large",  
    # With the `text-embedding-3` class  
    # of models, you can specify the size  
    # of the embeddings you want returned.  
    dimensions=1024  
)  
  
text = "LangChain is the framework for building context-aware reasoning applications"  
# Define an additional text to embed  
text2 = "LangGraph is a library for building stateful, multi-actor applications with LLMs"  
  
# Embed both texts  
two_vectors = embeddings.embed_documents([text, text2])  
  
# Display the first 100 characters of each embedding vector  
print("Embeddings for multiple texts (first 100 characters of each):")  
for i, vector in enumerate(two_vectors, start=1):  
    print(f"Embedding {i}:", str(vector)[:100])  
  
Output  
  
Embeddings for multiple texts (first 100 characters of each):  
Embedding 1: [-0.02610638178884983, 0.005122559145092964, -0.04468365013599396,  
0.00509954197332263, 0.0110174659  
Embedding 2: [-0.013918550685048103, 0.03208278864622116, -0.05786428600549698,  
-0.002065209439024329, -0.0321980
```

Penjelasan Output

Embeddings yang dihasilkan untuk setiap teks akan berupa dense vector angka, menangkap nuansa semantik. Dengan mencetak 100 karakter pertama, kita mendapatkan gambaran singkat

mengenai struktur setiap vektor. Vektor lengkap tersebut kemudian dapat digunakan dalam tugas retrieval untuk membandingkan dan mengambil dokumen berdasarkan similarity.

Menggunakan `embed_query` dan `embed_documents` menawarkan cara yang mulus untuk mengelola berbagai jenis teks dalam setup RAG, memungkinkan penanganan yang efisien baik untuk query tunggal maupun dokumen dalam jumlah besar.

Menggunakan Embeddings dalam Retrieval-Augmented Generation (RAG)

Dalam Retrieval-Augmented Generation (RAG), embeddings sangat penting untuk mengambil informasi yang relevan secara kontekstual dengan merepresentasikan query dan dokumen dalam format vektor. Vektor-vektor ini kemudian dapat disimpan dan diindeks dalam sebuah vector database, seperti `InMemoryVectorStore` milik `LangChain`, untuk memungkinkan retrieval berbasis similarity yang cepat dan akurat.

Konsep Indexing dan Retrieval

1. Indexing:

- **Purpose:** Indexing converts raw data (such as documents or sentences) into vector representations, which are stored for quick access.
- **Process:** During indexing, embeddings are created for each document and stored in a vector store. This setup allows for efficient querying since similar texts are represented by closely aligned vectors in the vector space.

2. Retrieval:

- **Purpose:** Retrieval is the process of finding and returning relevant documents based on a similarity search.
- **Process:** A query is embedded in the same way as the indexed data. The system then calculates the similarity between this query embedding and stored document embeddings, retrieving the most relevant ones.

Berikut adalah contoh lengkap penggunaan `InMemoryVectorStore` di `LangChain` untuk mendemonstrasikan indexing dan retrieval.

Langkah 1: Membuat Vector Store dan Mengindeks Dokumen

Untuk memulai, kita akan membuat sebuah `InMemoryVectorStore` dan mengindeks dokumen contoh menggunakan embedding model yang diinisialisasi pada contoh sebelumnya.

```
#embeddings
```

```

from langchain_openai import OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

# Initialize embeddings model
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-large",
    dimensions=1024
)
# Sample document to index
text = "LangChain is the framework for building context-aware reasoning applications."
# Index the document in the InMemoryVectorStore
vectorstore = InMemoryVectorStore.from_texts(
    [text],
    embedding=embeddings,
)

print("Document indexed successfully.")

```

Langkah 2: Menggunakan Vector Store sebagai Retriever

Sekarang setelah kita mengindeks dokumen kita, kita dapat menggunakan vector store untuk mengambil dokumen yang mirip dengan query tertentu. Proses retrieval ini menghitung similarity antara query dan dokumen yang tersimpan, mengembalikan kecocokan yang paling relevan.

```

#embeddings

# Convert the vector store into a retriever
retriever = vectorstore.as_retriever()
# Define a sample query
query = "What is LangChain?"
# Retrieve the most similar document(s)
retrieved_documents = retriever.invoke(query)
# Display the content of the retrieved document
print("Retrieved document content:")
print(retrieved_documents[0].page_content)

Output:

Retrieved document content:
LangChain is the framework for building context-aware reasoning applications.

```

Penjelasan Proses Retrieval

Ketika query "What is LangChain?" dilewatkan melalui retriever:

1. The query is embedded using the same **OpenAIEmbeddings** model.
2. The retriever calculates the similarity score between the query embedding and the embeddings of all indexed documents.
3. The document with the highest similarity score is returned as the most relevant result.

Konten yang ditampilkan di `retrieved_documents[0].page_content` seharusnya sangat selaras dengan query, menunjukkan bagaimana setup RAG dapat mengambil informasi yang relevan secara kontekstual.

Di balik layar, implementasi vectorstore dan retriever memanggil `embeddings.embed_documents(...)` dan `embeddings.embed_query(...)` untuk membuat embeddings bagi teks yang digunakan dalam operasi `from_texts` dan `retrieval invoke`, secara berurutan.

Mengukur Similarity dalam Embeddings dengan Cosine Similarity

Embeddings menerjemahkan teks menjadi vektor berdimensi tinggi di mana teks yang serupa diposisikan berdekatan. Ini memungkinkan perbandingan makna antar teks yang cepat dan efisien dengan mengukur "jarak" atau "sudut" antara embeddings mereka. Di sini, kita akan membangun dari contoh awal kita menggunakan **OpenAIEmbeddings** dan menghitung similarity menggunakan cosine similarity-sebuah metrik umum untuk text embeddings.

Rekap Metrik Similarity Umum

1. **Cosine Similarity:** Measures the cosine of the angle between two vectors (ranges from -1 to 1). It is best for text embeddings, as it captures similarity based on direction rather than magnitude.
2. **Euclidean Distance:** Straight-line distance between points (lower distance = higher similarity).
3. **Dot Product:** Measures the projection of one vector onto another (higher value = higher similarity).

Untuk contoh ini, kita akan fokus pada cosine similarity, yang direkomendasikan untuk **OpenAI embeddings**.

Cosine Similarity: Contoh Praktis Menggunakan OpenAIEmbeddings

Pada contoh ini, kita akan menggunakan kembali embeddings yang dihasilkan untuk teks-teks sebelumnya dan menghitung cosine similarity di antara mereka. Contoh ini akan mencakup meng-embed satu

teks tunggal, meng-embed multiple teks, dan menghitung skor similarity mereka.

Langkah 1: Embed Teks

Pertama, mari kita inisialisasi embeddings untuk teks-teks kita menggunakan OpenAIEmbeddings.

```
#embeddings

from langchain_openai import OpenAIEmbeddings
import numpy as np

# Initialize embeddings model
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-large",
    dimensions=1024
)

# Define texts to embed
text1 = "LangGraph is a library for building stateful, multi-actor applications with LLMs."
text2 = "LangChain is a framework for building context-aware reasoning applications."
text3 = "The quick brown fox jumps over the lazy dog."

# Embed single and multiple texts
embedding1 = embeddings.embed_query(text1)
embedding2, embedding3 = embeddings.embed_documents([text2, text3])

# Display first 10 values of each embedding for readability
print("Embedding for text1 (first 10 values):", embedding1[:10])
print("Embedding for text2 (first 10 values):", embedding2[:10])
print("Embedding for text3 (first 10 values):", embedding3[:10])
```

Output yang Diharapkan untuk Embeddings

Setiap vektor embedding akan berupa array berdimensi 1024. Berikut adalah contoh output (dipotong untuk keterbacaan):

```
#embeddings

Embedding for text1 (first 10 values): [-0.020099086686968803, 0.030566569417715073,
-0.05171050503849983, -0.004713691771030426, -0.0341760478913784,
-0.022986669093370438, -0.01602417789399624, 0.024943385273218155,
-0.02194182015955448, -0.0016278265975415707]
Embedding for text2 (first 10 values): [-0.036500561982393265, 0.004984983243048191,
-0.04653669148683548, 0.006315839011222124, 0.0017685367492958903,
-0.022209767252206802, -0.008509333245456219, 0.03749806806445122,
-0.021354762837290764, 0.000759580172598362]
Embedding for text3 (first 10 values): [-0.01729467138648033, 0.01311655342578888,
-0.015627989545464516, 0.02812810055911541, 0.0017836913466453552,
```

-0.029269661754369736,	0.0137672433629632,	0.08548019826412201,
-0.023744499310851097,	0.009286610409617424]	

Langkah 2: Menghitung Cosine Similarity

Selanjutnya, kita akan menghitung cosine similarity antara embeddings text1, text2, dan text3. Ini akan membantu kita memahami seberapa dekat hubungan mereka.

```
#embeddings
# Define a function to calculate cosine similarity
def cosine_similarity(vec1, vec2):
    dot_product = np.dot(vec1, vec2)
    norm_vec1 = np.linalg.norm(vec1)
    norm_vec2 = np.linalg.norm(vec2)
    return dot_product / (norm_vec1 * norm_vec2)
# Calculate similarity scores
similarity_1_2 = cosine_similarity(embedding1, embedding2)
similarity_1_3 = cosine_similarity(embedding1, embedding3)
similarity_2_3 = cosine_similarity(embedding2, embedding3)
# Display similarity scores
print("Cosine Similarity between text1 and text2:", similarity_1_2)
print("Cosine Similarity between text1 and text3:", similarity_1_3)
print("Cosine Similarity between text2 and text3:", similarity_2_3)
```

Output yang Diharapkan untuk Skor Cosine Similarity

Karena text1 dan text2 membahas topik yang sama, kita mengharapkan skor similarity yang lebih tinggi karena mereka memiliki makna semantik yang lebih dekat. Berikut adalah contoh skor similarity yang diharapkan:

```
#embeddings
Cosine Similarity between text1 and text2: 0.5348841944232716
Cosine Similarity between text1 and text3: 0.1824835672123656
Cosine Similarity between text2 and text3: 0.18567230491463937
```

- **Interpretasi:?** **Skor lebih tinggi (mendekati 1):** Menunjukkan similarity yang lebih besar. Sebagai contoh, skor 0,53 antara text1 dan text2 menunjukkan similarity moderat. **Skor lebih rendah (mendekati 0):** Menunjukkan similarity yang lebih kecil. Sebagai contoh, skor 0,18 antara text1 dan text3 menunjukkan similarity yang minimal.

Pada contoh ini, kita telah mendemonstrasikan:

1. **Embedding single and multiple texts** with [OpenAIEmbeddings](#) .
2. **Calculating cosine similarity** to measure how closely aligned two texts are within the embedding space.

Metrik similarity ini sangat penting untuk sistem RAG, memungkinkan retrieval dokumen yang relevan secara kontekstual secara efektif dan akurat. Dengan meng-embed baik query maupun dokumen, kita dapat mengukur similarity secara efisien dan meningkatkan performa retrieval dalam aplikasi LangGraph.

Vector Store dalam Sistem RAG

Vector store adalah database khusus untuk menyimpan dan mengambil vektor berdimensi tinggi. Vector store sangat penting dalam sistem Retrieval-Augmented Generation (RAG), memungkinkan pencarian similarity yang cepat di antara embeddings untuk mengambil informasi yang relevan secara kontekstual. Dengan vector store, data diindeks dalam bentuk vektor, memungkinkan aplikasi RAG mengambil dokumen berdasarkan similarity terhadap sebuah query embedding.

Penelitian Fondasional tentang Vector Store

Paper penelitian fondasional tentang vector store, "FAISS: A library for efficient similarity search and clustering of dense vectors" oleh Facebook AI, memperkenalkan FAISS (Facebook AI Similarity Search). FAISS memanfaatkan algoritma indexing yang teroptimasi dan pencarian approximate nearest neighbor (ANN) untuk memberikan similarity search yang cepat di seluruh dataset besar. Pendekatan inovatif FAISS menetapkan standar untuk vector store dengan membuat similarity search efisien untuk embeddings yang digunakan dalam aplikasi AI.

Konsep Kunci Vector Store

1. **Indexing:** During indexing, data (such as documents) is embedded into vectors and stored. The vector store organizes these embeddings in a way that enables efficient search and retrieval.
2. **Retrieval:** A query embedding is generated similarly, and the store performs a similarity search across indexed embeddings to find relevant documents.
3. **Scalability:** Vector stores are optimized to handle large volumes of embeddings, supporting applications with massive datasets and high retrieval speed requirements.

Vector Store Populer

Berikut adalah 10 vector store teratas, dikategorikan berdasarkan ketersediaan dan platform:

Vector Store Gratis:

1. FAISS - High performance, open-source library by Facebook AI, suitable for local deployment.
2. Chroma - Lightweight, runs locally as a library, and integrates well with Python.
3. Lance - Open-source vector store optimized for efficient storage and retrieval.

Vector Store On-Device:

4. Milvus - Open-source and can be deployed locally or on cloud; supports dynamic indexing.
5. Weaviate - Runs locally and cloud, with powerful query language and knowledge graph support.
6. Vespa - Open-source engine for search and recommendation, suited for on-premise deployment.

Vector Store Berbasis Cloud:

7. Pinecone - Cloud-based, offering scalable and managed vector storage.
8. Google Vertex Matching Engine - Managed service by Google for fast and scalable similarity search.
9. AWS Kendra - Fully managed search service integrated with AWS.
10. Redis Vector Search - Redis module supporting vector similarity search, optimized for cloud.

Demonstrasi: Menggunakan Chroma sebagai Vector Store Lokal yang Gratis

Chroma adalah vector store lokal yang gratis dan menyediakan kemampuan penyimpanan serta retrieval yang efisien tanpa memerlukan sumber daya cloud. Berikut adalah panduan untuk mengatur Chroma, mengindeks dokumen, dan melakukan similarity search. Install Chroma dan langchain_chroma untuk memulai:

```
#installing chroma
pip install langchain_chroma
```


Catatan: Jika mengalami masalah saat menginstal chroma di windows, silakan coba ikuti solusi stack overflow ini: <https://stackoverflow.com/a/76245995>. Abaikan ini jika chroma berhasil terinstal.

Pada contoh ini, kita akan membuat embeddings untuk dokumen contoh, menyimpannya di Chroma, dan melakukan similarity search.

Dengan menggunakan method `similarity_search`, kita dapat mencari dokumen yang mirip dengan query tertentu.

```
#embeddings
# Define a query to search
query = "What is LangChain?"

# Perform similarity search in Chroma
docs = db.similarity_search(query)

# Display the content of the retrieved document
print("Most similar document to the query:")
print(docs[0].page_content)
```

Output yang Diharapkan: Output seharusnya menampilkan konten dokumen yang paling mirip dengan query, seperti:

```
#embeddings
Most similar document to the query:
"LangChain is a framework for building context-aware reasoning applications."
```

Melakukan Similarity Search berdasarkan Vector

Jika Anda sudah memiliki sebuah query embedding, Anda dapat menggunakan `similarity_search_by_vector` untuk mencari dokumen serupa secara langsung berdasarkan vector.

```
#embeddings
# Generate embedding for the query
query_embedding = embeddings.embed_query(query)

# Perform similarity search by embedding vector
docs_by_vector = db.similarity_search_by_vector(query_embedding)

# Display the content of the retrieved document
print("Most similar document to the query (vector search):")
print(docs_by_vector[0].page_content)
```

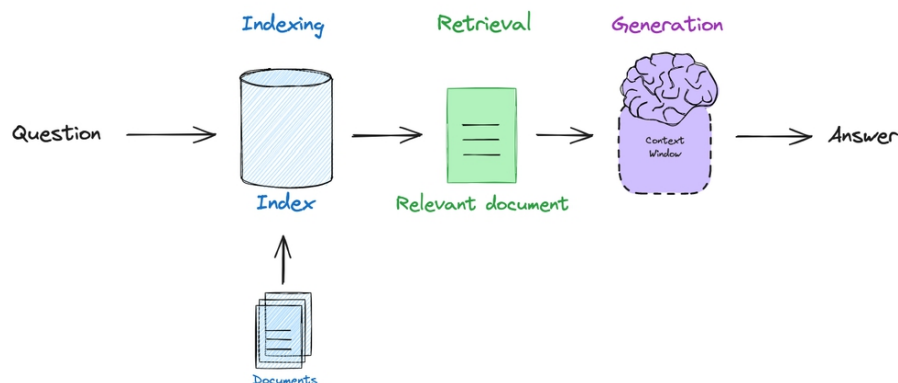
Expected Output:

Most similar document to the query (vector search):
"LangChain is a framework for building context-aware reasoning applications."

Vector store seperti Chroma, FAISS, dan Pinecone sangat penting dalam sistem RAG, memungkinkan similarity search yang cepat dan efisien. Chroma, sebagai opsi lokal yang gratis, sangat cocok untuk development dan testing. Melalui operasi indexing dan retrieval, vector store menyederhanakan akses informasi, mendukung aplikasi yang bergantung pada retrieval data yang relevan secara kontekstual.

Menghasilkan Jawaban untuk Pertanyaan menggunakan RAG dengan sebuah LLM

Setelah kita mengambil dokumen yang paling relevan menggunakan similarity search dalam vector store, kita dapat menggunakan Language Learning Model (LLM) untuk menghasilkan respons dengan memberikan dokumen yang diambil sebagai konteks. Pendekatan ini penting dalam setup Retrieval-Augmented Generation (RAG), di mana LLM menggabungkan kemampuan generatifnya dengan pengetahuan eksternal yang diambil dari vector store.



Mari kita telusuri cara menggunakan LLM untuk menghasilkan respons menggunakan dokumen yang diambil sebagai konteks.

Langkah-Langkah untuk Menghasilkan Respons dengan LLM

1. Retrieve Relevant Document: Perform a similarity search to find the most relevant document for the user's question.
2. Combine Context and Query: Format the retrieved document and user question into a prompt for the LLM.
3. Generate a Response: Use the LLM to generate a response based on the combined prompt.

Contoh ini mengasumsikan bahwa kita telah mengindeks dokumen dalam sebuah vector store (misalnya, Chroma) dan mengambil dokumen paling mirip berdasarkan query pengguna.

```
#lesson11d.py

from langchain_openai import OpenAIEmbeddings
from langchain.schema import Document
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
# Initialize the embeddings model
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-large"
)
# Define sample documents as Document objects
documents = [
    Document(page_content="LangChain is a framework for building context-aware reasoning applications."),
    Document(page_content="FAISS is a library for efficient similarity search and clustering of dense vectors."),
    Document(page_content="The quick brown fox jumps over the lazy dog.")
]
# Index documents in Chroma vector store
db = Chroma.from_documents(documents, embedding=embeddings)
print("Documents indexed in Chroma successfully.")

# Define a retriever to fetch relevant documents
retriever = db.as_retriever(search_type="similarity", search_kwargs={"k": 6})

# Define a prompt template for the LLM
prompt = ChatPromptTemplate.from_template(
    """
    You are an assistant for question-answering tasks. Use the following pieces of retrieved context to answer the question. If you don't know the answer, just say that you don't know. Use three sentences maximum and keep the answer concise.

    Question: {question}
    Context: {context}
    Answer:
    """
)
# Initialize the chat model (GPT-4 variant or mini model for demonstration)
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
# Set up the RAG chain pipeline
```

```

rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)
# Use the RAG pipeline to answer a user question
question = "What did the fox do?"
for chunk in rag_chain.stream(question):
    print(chunk, end="", flush=True)

```

RAG Praktis - Mengajukan Pertanyaan pada Dokumen PDF yang Panjang

Pada contoh ini, kita memuat sebuah dokumen PDF, memecahnya menjadi chunk-chunk yang lebih kecil, meng-embed setiap chunk, dan menyimpannya dalam sebuah vector store. Kita juga menghitung jumlah token dalam setiap chunk, yang berguna untuk mengelola batasan panjang konteks pada LLM.

Langkah-
Langkah

1. **Load the Document:** Use `PyPDFLoader` to load a PDF document.
2. **Split the Document:** Divide the document into smaller chunks using `CharacterTextSplitter`.
3. **Embed Each Chunk:** Generate embeddings for each chunk.
4. **Store in Chroma:** Index the chunks in a `Chroma` vector store.
5. **Calculate Tokens:** Use `tiktoken` to calculate the number of tokens for each chunk

Pendekatan ini mendemonstrasikan cara untuk:

1. **Handle large documents** by splitting them into manageable chunks.
2. **Embed and index each chunk** in a vector store, enabling efficient retrieval.
3. **Calculate token counts** for each chunk to manage context lengths effectively.

Setelah diindeks, Anda dapat menggunakan similarity search pada db untuk mengambil chunk-chunk yang relevan berdasarkan sebuah query dan memasukkannya ke dalam LLM sebagai konteks. Ini penting untuk aplikasi berskala besar yang memerlukan jawaban yang akurat dan sadar konteks dari dokumen yang ekstensif.

Berikut adalah kode lengkap yang mendemonstrasikan cara memuat dokumen besar, memecahnya menjadi chunk-chunk, meng-embed dan mengindeks chunk-chunk tersebut, mengambil konteks yang relevan, dan

menghasilkan sebuah respons.

```
#embeddings

import os
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
import tiktoken

# Step 1: Load the document
file_path = os.path.join(os.getcwd(), "chapter11", "Faiss by FacebookAI.pdf")
raw_documents = PyPDFLoader(file_path).load()
# Step 2: Split the document into chunks
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
documents = text_splitter.split_documents(raw_documents)
# Step 3: Initialize the embeddings model
embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
# Step 4: Index the document chunks in Chroma vector store
db = Chroma.from_documents(documents=documents, embedding=embeddings)
print("Documents indexed in Chroma successfully.")
# Step 5: Define a retriever for similarity search
retriever = db.as_retriever(search_type="similarity", search_kwargs={"k": 3}) # Retrieve top 3
relevant chunks
# Step 6: Define the prompt template for the LLM
prompt = ChatPromptTemplate.from_template(
    """
    You are an assistant for question-answering tasks. Use the following pieces of retrieved context
    to answer the question. If you don't know the answer, just say that you don't know. Use three
    sentences maximum and keep the answer concise.

    Question: {question}
    Context: {context}
    Answer:
    """
)
# Step 7: Initialize the ChatOpenAI model (e.g., gpt-4 or another preferred model)
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
# Step 8: Set up the Retrieval-Augmented Generation (RAG) chain
rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
```

```

| prompt
| model
| StrOutputParser()
)
# Step 9: Ask a question and generate a response
question = "Can you explain what FAISS is used for?"
for chunk in rag_chain.stream(question):
    print(chunk, end="", flush=True)

```

Penjelasan Setiap Langkah

1. **Document Loading and Splitting:** The large document is loaded and split into 1000-character chunks, allowing the LLM to process manageable portions of text.
2. **Embedding and Indexing:** Each chunk is converted into an embedding and stored in **Chroma**, a local vector store, which allows for efficient retrieval based on similarity.
3. **Retrieving Relevant Chunks:** The retriever finds the top 3 chunks most similar to the user's query, "Can you explain what FAISS is used for?". This ensures that the LLM has relevant context to answer the question accurately.
4. **Prompt Construction:** A prompt template is used to format the question and context in a way that the LLM can interpret effectively. This prompt instructs the LLM to provide a concise response based on the retrieved context.
5. **Response Generation:** The LLM (e.g., **gpt-4o-mini**) processes the prompt and generates a response. Streaming is enabled, so the response is printed as it's generated.

Output yang Diharapkan

Untuk pertanyaan seperti, "Can you explain what FAISS is used for?", output yang diharapkan mungkin:

```

#embeddings
Documents indexed in Chroma successfully.
FAISS, developed by Facebook AI Research, is a library designed for efficient similarity
search, particularly for high-dimensional vectors generated by AI tools. It allows for quick
searching of multimedia documents that are similar to each other, making it suitable for large-

```

scale datasets, including billions of vectors. The library optimizes memory usage and speed, offering both CPU and GPU implementations for various indexing methods.

Untuk melakukan Retrieval-Augmented Generation (RAG) menggunakan beberapa sumber, termasuk web, kita dapat mengikuti pipeline yang serupa tetapi memperluas sumber pengambilan context kita. Hal ini memungkinkan kita untuk mengambil informasi relevan dari berbagai lokasi, seperti dokumen lokal, halaman web, dan database, sehingga memberi language model basis pengetahuan yang lebih luas untuk digunakan saat menghasilkan jawaban.

Langkah-Langkah Menyiapkan RAG dari Berbagai Sumber

Pada contoh ini, kita akan:

1. **Retrieve Context from Local and Web Sources:** Combine content from a local PDF and external web sources.
2. **Use a Vector Store:** Index and store embeddings from both local and web sources.
3. **Perform Similarity Search:** Retrieve relevant chunks from each source.
4. **Generate a Response:** Use an LLM to generate a concise answer using the combined context.

Contoh Kode Lengkap

Sebagai demonstrasi, misalkan kita memiliki file PDF lokal dan kita juga ingin mengambil konten dari beberapa halaman web yang terkait dengan topik tersebut.

```
#lesson11e.py

import os
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader, WebBaseLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
import tiktoken

# Initialize the embeddings model
embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
### Step 1: Load and Process Local Document (PDF)
# Load local PDF document
```

```

file_path = os.path.join(os.getcwd(), "chapter11", "Faiss by FacebookAI.pdf")
pdf_loader = PyPDFLoader(file_path)
pdf_documents = pdf_loader.load()
# Split PDF into chunks
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
split_pdf_documents = text_splitter.split_documents(pdf_documents)
### Step 2: Load and Process Web Content
# Define URLs to fetch web content
urls = [
    "https://github.com/facebookresearch/faiss",
    "https://github.com/facebookresearch/faiss/wiki"
]

# Load web content
web_loader = WebBaseLoader(web_paths=urls)
web_documents = web_loader.load()

# Split web documents into chunks
split_web_documents = text_splitter.split_documents(web_documents)

### Step 3: Combine Local and Web Documents
# Combine documents from both local and web sources
all_documents = split_pdf_documents + split_web_documents

### Step 4: Index Documents in Chroma Vector Store
# Index all documents in Chroma
db = Chroma.from_documents(documents=all_documents, embedding=embeddings)
print("All documents indexed in Chroma successfully.")

### Step 5: Define a Retriever
# Define a retriever to fetch relevant documents from the combined sources
retriever = db.as_retriever(search_type="similarity", search_kwargs={"k": 5}) # Retrieve top 5
relevant chunks

### Step 6: Define the Prompt Template for the LLM
prompt = ChatPromptTemplate.from_template(
    """
    You are an assistant for question-answering tasks. Use the following pieces of retrieved context
    to answer the question. If you don't know the answer, just say that you don't know. Use three
    sentences maximum and keep the answer concise.

    Question: {question}
    Context: {context}
    Answer:
    """
)

### Step 7: Initialize the ChatOpenAI Model

```



```

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
### Step 8: Set up the Retrieval-Augmented Generation (RAG) Chain
rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)
### Step 9: Ask a Question and Generate a Response

question = "Who are the main authors of Faiss?"
for chunk in rag_chain.stream(question):
    print(chunk, end="", flush=True)

```

Penjelasan Setiap Langkah

1. Local Document Loading and Splitting:

- We use **PyPDFLoader** to load a PDF document (in this example, "Faiss by FacebookAI.pdf") and split it into manageable chunks.

2. Web Content Loading and Splitting:

- **WebLoader** retrieves content from specific URLs (e.g., Wikipedia articles about Facebook AI Research and FAISS). This is split into chunks as well to keep it manageable for embedding and retrieval.

3. Combining Documents:

- The chunks from the local PDF and web sources are combined into a single list to create a unified knowledge base for the vector store.

4. Embedding and Indexing:

- All chunks are embedded and indexed in **Chroma**, allowing for efficient similarity-based retrieval across both local and web content.

5. Retriever Setup:

- The retriever is set to fetch the top 5 most similar chunks, providing a variety of context from both local and web sources.

6. Prompt Construction:

- A concise prompt template is created to guide the LLM in generating short, accurate answers.

7. LLM Initialization:

- The **ChatOpenAI** model is initialized to process the combined prompt and context.

8. RAG Chain Execution:

- The RAG chain is executed with streaming enabled, so the response is generated and printed in real-time.

Output yang Diharapkan

Untuk pertanyaan seperti, "Who are the main authors of Faiss?", output yang diharapkan mungkin:

```
#embeddings
```

```
The main authors of Faiss are Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. Hervé Jégou initiated the project and wrote its first implementation. Each author contributed significantly to various aspects of the library's development.
```

Perhatikan bahwa nama-nama author dikutip secara eksplisit dari dokumen online dan bukan dari dokumen PDF.

Memahami Workflow RAG di LangGraph: Retrieval AI Agent

Dalam setup RAG LangGraph yang umum, workflow mencakup langkah-langkah berikut:

1. Indexing

- **Data Loading:** Data dimuat dari berbagai sumber menggunakan Document Loaders di LangChain. Ini dapat mencakup halaman web, dokumen, database, dan lain-lain.
- **Text Splitting:** Dokumen besar dipecah menjadi chunk yang dapat dikelola, sering kali dengan overlap, untuk meningkatkan akurasi retrieval.
- **Storing and Indexing:** Setiap chunk disimpan dalam vector database, di mana ia diubah menjadi embedding. Hal ini memungkinkan retrieval berbasis kemiripan (similarity-based) yang efisien menggunakan cosine similarity.

2. Retrieval and Generation

- **Query Processing:** Ketika sebuah query dimasukkan, sistem mengambil chunk yang paling relevan dengan membandingkan embedding query dengan embedding dokumen yang tersimpan.- **Answer Generation:** Sebuah LLM, seperti GPT atau Claude, mengambil query dan context yang diambil untuk menghasilkan jawaban.

Berikut adalah kode lengkap untuk mengimplementasikan workflow RAG di LangGraph dengan pendekatan state machine.

Implementasi Kode

```
#lesson11i.py

import asyncio
from typing import List, TypedDict
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

#1. Index 3 websites by adding them to a vector DB
urls = [
    "https://github.com/facebookresearch/faiss",
    "https://github.com/facebookresearch/faiss/wiki",
    "https://github.com/facebookresearch/faiss/wiki/Faiss-indexes"
]

docs = [WebBaseLoader(url).load() for url in urls]
docs_list = [item for sublist in docs for item in sublist]
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=250, chunk_overlap=0
)
doc_splits = text_splitter.split_documents(docs_list)
vectorstore = Chroma.from_documents(
    documents=doc_splits,
    collection_name="rag-chroma",
```

```

        embedding=OpenAIEmbeddings(),
    )
    retriever = vectorstore.as_retriever()
#2. Prepare the RAG chain
prompt = ChatPromptTemplate.from_template(
    """
    You are an assistant for question-answering tasks. Use the following pieces of retrieved context
    to answer the question. If you don't know the answer, just say that you don't know. Use three
    sentences maximum and keep the answer concise.
    Question: {question}
    Context: {context}
    Answer:
    """
)
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
rag_chain = (
    prompt | model | StrOutputParser()
)
#3. define the graph
class GraphState(TypedDict):
    """
    Represents the state of our graph.

    Attributes:
        question: question
        generation: LLM generation
        documents: list of documents
    """

    question: str
    generation: str
    web_search: str
    documents: List[str]
#4. Retrieve node
def retrieve(state):
    """
    Retrieve documents

    Args:
        state (dict): The current graph state

    Returns:
        state (dict): New key added to state, documents, that contains retrieved documents
    """
    print("---RETRIEVE---")
    question = state["question"]

    # Retrieval
    documents = retriever.invoke(question)

```

```

        return {"documents": documents, "question": question}
#5. Generate node
def generate(state):
    """
    Generate answer
    Args:
        state (dict): The current graph state
    Returns:
        state (dict): New key added to state, generation, that contains LLM generation
    """

    print("---GENERATE---")
    question = state["question"]
    documents = state["documents"]

    # RAG generation
    generation = rag_chain.invoke({"context": documents, "question": question})
    return {"documents": documents, "question": question, "generation": generation}
#6. Define the workflow
def create_workflow():
    workflow = StateGraph(GraphState)

    # Add nodes
    workflow.add_node("retrieve", retrieve)
    workflow.add_node("generate", generate)

    # Add edges
    workflow.add_edge(START, "retrieve")
    workflow.add_edge("retrieve", "generate")
    workflow.add_edge("generate", END)

    return workflow.compile(checkpointer=MemorySaver())
#7. Run the workflow
async def run_workflow():
    app = create_workflow()
    config = {
        "configurable": {"thread_id": "1"},
        "recursion_limit": 50
    }
    inputs = {"question": f"What are flat indexes?"}

    try:
        async for event in app.astream(inputs, config=config, stream_mode="values"):
            if "error" in event:
                print(f"Error: {event['error']}")
                break

```

```

        print(event)
    except Exception as e:
        print(f"Workflow execution failed: {str(e)}")

if __name__ == "__main__":
    asyncio.run(run_workflow())

```

Penjelasan Workflow

1. Loading and Indexing:

- Web data is loaded using `WebBaseLoader` and then split into chunks with `RecursiveCharacterTextSplitter`.
- Chunks are embedded and stored in a `Chroma` vector store, enabling fast similarity-based retrieval.

2. Graph State and Nodes:

- `GraphState` defines the data structure for holding the state of the workflow.
- The `retrieve` node handles document retrieval based on the query, using cosine similarity to fetch the most relevant document chunks.
- The `generate` node generates an answer by invoking the LLM on the retrieved context.

3. Creating and Running the Workflow:

- `create_workflow` sets up the nodes and edges in the LangGraph state machine.
- `run_workflow` asynchronously executes the workflow, streaming the final generated response.

Contoh Output: Diberikan pertanyaan seperti "What are flat indexes in FAISS?", output-nya mungkin akan terlihat seperti ini:

```

#embeddings
---RETRIEVE---
---GENERATE---
{'question': 'What are flat indexes in FAISS?', 'generation': 'Flat indexes in FAISS are a type of index structure used for efficient similarity search. They store vectors in a simple array format and allow for fast, exhaustive searches. This is particularly useful in AI for applications requiring precise and comprehensive similarity search results.'}

```

Ringkasan: Workflow RAG di LangGraph ini:

- **Memuat dan mengindeks data web ke dalam vector store, menciptakan basis pengetahuan yang dapat dicari.- Mengambil konten relevan berdasarkan query pengguna.- Menghasilkan jawaban dari sebuah LLM menggunakan context yang diambil, memberikan informasi yang ringkas dan akurat.**

Dengan menyusun proses RAG sebagai state machine LangGraph, setup ini menjadi modular, efisien, dan scalable, memungkinkan workflow yang lebih kompleks yang dapat melibatkan tipe dokumen tambahan, question answering yang lebih canggih, atau strategi fallback ketika informasi relevan jarang ditemukan.

Tinjauan Bab: Workflow RAG di LangGraph

Pada bab ini, kita menjelajahi arsitektur dasar Retrieval-Augmented Generation (RAG) di LangGraph. Kita mengimplementasikan workflow yang modular dan berbasis state untuk RAG, mendemonstrasikan cara memuat data dari berbagai sumber, mengindeksnya dalam vector store, dan mengambil informasi relevan untuk generasi respons menggunakan sebuah LLM. Workflow ini terdiri dari:

- **Indexing:** Memuat dokumen dari sumber web, memecahnya menjadi chunk yang dapat dikelola, meng-embed-nya, dan menyimpan embedding tersebut dalam vector database.
- **Retrieval and Generation:** Memproses query pengguna dengan mengambil chunk dokumen yang paling relevan, dan menghasilkan respons menggunakan sebuah LLM dengan context yang diambil sebagai input.

Arsitektur ini sangat cocok untuk aplikasi di mana informasi relevan tersebar di dataset yang besar dan statis, memungkinkan LLM untuk merespons dengan jawaban yang diperkaya secara kontekstual berdasarkan dokumen yang diambil.

Keterbatasan Arsitektur RAG

Meskipun efektif, arsitektur RAG yang dipaparkan memiliki keterbatasan tertentu:

1. Dependence on Static Document Stores:

- This setup relies on a static vector store where information is pre-indexed. If new information becomes available or if there are updates to the indexed content, the vector store needs to be rebuilt and re-indexed. This can be a limitation in dynamic environments where information frequently changes.

2. Lack of Contextual Adaptation:

- The architecture does not dynamically adapt to different query complexities or the availability of information. For example, some queries may require more context than others, or different document sources to be more relevant. The current RAG setup treats all queries similarly, regardless of their nature.

3. Reliance on Single Retrieval:

- The system performs a single pass of retrieval for each query, retrieving relevant chunks based on cosine similarity. This approach may fail if the retrieved chunks are not sufficiently relevant. In cases where initial retrievals do not provide enough information, the system lacks mechanisms to adjust or re-query in a more targeted way.

4. No Corrective Mechanism:

- The architecture does not include any feedback or corrective mechanism to evaluate whether the generated response adequately answers the query. If the initial retrieval does not meet the query's requirements, the system has no way to self-correct or prompt for additional retrievals.

5. Limited Scalability and Context Length:

- In cases where the query requires a large context or multiple documents, the system is limited by the token constraints of the LLM, which restricts how much context can be included in a single query-response cycle.

Pengenalan Arsitektur RAG Tingkat Lanjut

Untuk mengatasi keterbatasan ini, bab berikutnya akan memperkenalkan pendekatan RAG tingkat lanjut:

1. Self-RAG:

- Self-RAG involves a more autonomous retrieval process where the system iteratively retrieves and evaluates information. If initial retrievals are insufficient, the system can perform additional retrieval passes or refine the query before generating a response. This enables the model to gather sufficient context for complex or ambiguous queries.

2. Adaptive RAG:

- Adaptive RAG is designed to dynamically adjust its retrieval and generation strategies based on the nature of the query. For instance, if the query is broad, the system may prioritize general information; for specific queries, it may narrow its focus. Adaptive RAG uses

contextual cues to determine how much context to retrieve and what sources to prioritize, making it highly responsive to diverse queries.

3. **Corrective RAG (CRAG):**

- Corrective RAG adds a feedback mechanism to the RAG workflow. After generating an initial response, CRAG evaluates whether the response meets the query's requirements. If the response is incomplete or inaccurate, it triggers corrective actions, such as re-running retrieval with refined queries or increasing the retrieval scope. CRAG is particularly useful in scenarios where accuracy is critical, as it self-corrects based on internal grading or user feedback.

Bab berikutnya akan menjelajahi implementasi teknis dari Self-RAG, Adaptive RAG, dan Corrective RAG (CRAG) di LangGraph. Arsitektur-arsitektur ini merepresentasikan pendekatan retrieval-augmented generation yang lebih canggih dan responsif, memungkinkan LLM menghasilkan respons yang lebih akurat dan sadar-context. Kita akan menjelajahi prinsip-prinsip dasarnya, mendemonstrasikan workflow-nya, dan membahas skenario di mana masing-masing paling efektif.

Eksplorasi mekanisme adaptif dan korektif dalam RAG ini akan memberikan pembaca alat untuk membangun sistem RAG yang cerdas, otonom, dan responsif, membuat workflow LangGraph tangguh untuk aplikasi dunia nyata yang membutuhkan akurasi dan adaptabilitas tinggi.

Kuis tentang Workflow RAG di LangGraph

Pertanyaan Pilihan Ganda

1. What are the main components of the RAG Workflow in LangGraph?

- A) Data Loading, Query Parsing, Answer Generation
- B) Indexing, Retrieval, and Generation
- C) Embedding, Vectorization, Answer Matching
- D) Splitting, Clustering, Summarization

Answer: B) Indexing, Retrieval, and Generation

2. What purpose does document splitting serve in a RAG workflow?

- A) It improves the visual representation of the data.
- B) It reduces the number of queries needed.
- C) It breaks down large documents for better retrieval accuracy.

- D) It eliminates the need for embeddings.

Answer: C) It breaks down large documents for better retrieval accuracy.

3. Which component is responsible for storing document embeddings in a vectorized form for fast retrieval?

- A) Text Splitter
- B) Vector Store
- C) Prompt Template
- D) Answer Generator

Answer: B) Vector Store

4. What retrieval method is commonly used to match query embeddings with document embeddings, as recommended by OpenAI?

- A) Euclidean Distance
- B) Cosine Similarity
- C) Manhattan Distance
- D) Jaccard Similarity

Answer: B) Cosine Similarity

5. Which of the following is a limitation of the basic RAG architecture?

- A) Requires complex language models to index documents.
- B) Cannot adapt or perform corrective actions based on query requirements.
- C) Requires continuous input from a human operator.
- D) Uses multiple embeddings for each document chunk.

Answer: B) Cannot adapt or perform corrective actions based on query requirements.

Benar atau Salah

6. **True or False:** In a RAG workflow, documents retrieved are directly passed to the answer generation model without any intermediate processing. **Answer:** False. Retrieved documents may undergo further processing, such as being combined with the user query in a prompt.
7. **True or False:** Adaptive RAG can alter retrieval and generation strategies based on the complexity or specificity of the user query.

Answer: True

8. **True or False:** Corrective RAG workflows involve a feedback mechanism to check the generated answer's relevance and accuracy before finalizing it. **Answer:** True
9. **True or False:** Self-RAG workflows can autonomously refine their retrieval process if initial results are insufficient. **Answer:** True

Pertanyaan Jawaban Singkat

10. Explain the purpose of document embedding in a RAG workflow.
11. What limitations does a basic RAG workflow face in terms of adaptability and corrective measures?
12. Name and briefly describe the three advanced RAG architectures introduced as alternatives to the basic RAG approach.
13. Why is a vector store essential in a RAG workflow, and how does it enhance retrieval efficiency?

Jawaban untuk Pertanyaan Jawaban Singkat

10. Document embedding transforms text into high-dimensional vector representations that capture semantic meaning. This allows similarity-based retrieval to match user queries with relevant document chunks based on their content, even if exact words differ.
11. Basic RAG workflows struggle with static information reliance, lack of adaptive retrieval, and inability to self-correct when retrieval doesn't yield sufficient context. They do not adapt dynamically to query specifics or re-evaluate responses, which can lead to incomplete answers.
12. **Self-RAG** autonomously refines retrieval if initial results lack sufficient context. **Adaptive RAG** adjusts retrieval and generation strategies based on query specifics, prioritizing general or specific sources as needed. **Corrective RAG** includes a feedback loop to assess if the generated answer meets the query requirements and initiates corrective actions if it doesn't.
13. A vector store allows efficient storage and retrieval of document embeddings, enabling quick similarity searches for matching relevant documents with query embeddings. This structure optimizes retrieval for large datasets, making RAG workflows scalable and responsive.

Part 4: Advanced Architectures
and Specialized Agent Designs
(Chapters 12-15)

Chapter 12

Arsitektur RAG Tingkat Lanjut (Self-RAG, Corrective RAG dan Adaptive RAG)

12.1 Pengenalan Self-Reflective RAG

Dalam Retrieval-Augmented Generation (RAG), workflow yang umum mencakup melakukan query pada vector store untuk mengambil dokumen relevan berdasarkan input pengguna, diikuti dengan menghasilkan respons menggunakan sebuah language model. Self-RAG, sebagaimana diperkenalkan dalam paper terbaru, membangun di atas framework ini dengan menggabungkan langkah-langkah "reflection" ke dalam RAG. Metode ini memungkinkan sebuah LLM mengontrol aspek-aspek seperti kapan harus melakukan retrieval, menilai relevansi, mengkritisi responsnya sendiri, dan menyesuaikan perilakunya untuk meningkatkan akurasi dan faktualitas respons.

Motivasi untuk Self-RAG

Pipeline RAG tradisional mengambil sejumlah dokumen tertentu, terlepas dari relevansi atau kebutuhan retrieval dalam konteks tertentu. Self-RAG meningkatkan hal ini dengan memungkinkan model memutuskan apakah retrieval diperlukan, passage mana yang harus disertakan, dan kapan harus mengkritisi atau menyempurnakan respons. Pendekatan reflektif ini memungkinkan workflow yang dinamis dan adaptif di LangGraph.

Self-RAG menggunakan reflection token untuk menandai berbagai keputusan selama retrieval dan generation:

1. **Retrieve Token** - Determines if retrieval should occur.
2. **ISREL Token** - Evaluates if a retrieved document is relevant to the question.
3. **ISSUP Token** - Ensures the generated answer is grounded in the retrieved documents.
4. **ISUSE Token** - Measures the overall usefulness of the generated answer.

Token-token ini membuat model bersifat adaptif dan mampu menyempurnakan perilakunya secara on-the-fly untuk memaksimalkan relevansi respons, faktualitas, dan akurasi kontekstual.

Workflow Self-RAG di LangGraph

Dalam Self-RAG, LangGraph mendukung workflow self-reflective yang menggabungkan titik keputusan dan feedback loop, yang dicapai melalui state machine milik LangGraph. Di bawah ini, kita akan mengilustrasikan cara membangun solusi RAG self-reflective menggunakan graph dan state node milik LangGraph.

1. Menyiapkan Environment

Install package yang diperlukan dan inisialisasi embedding untuk retrieval dokumen:

```
#lesson12a.py

import os
from langchain_openai import OpenAIEmbeddings
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

2. Indexing Documents Muat berbagai sumber untuk menciptakan database yang kaya dan siapkan untuk retrieval di Chroma vector store.

```
#lesson12a.py continued

# List of URLs to retrieve documents from
urls = [
```

```

    "https://lilianweng.github.io/posts/2023-06-23-agent/",
    "https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/",
    "https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/"
]

# Load documents and split into manageable chunks
docs = [WebBaseLoader(url).load() for url in urls]
docs_list = [item for sublist in docs for item in sublist]

# Split documents for improved retrieval
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(chunk_size=250,
chunk_overlap=0)
doc_splits = text_splitter.split_documents(docs_list)

# Store documents in Chroma vector store
vectorstore = Chroma.from_documents(documents=doc_splits, collection_name="rag-
chroma", embedding=OpenAIEmbeddings())
retriever = vectorstore.as_retriever()
# Set up prompt and model
prompt = ChatPromptTemplate.from_template("""
Use the following context to answer the question concisely:
Question: {question}
Context: {context}
Answer:
""")
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
rag_chain = (prompt | model | StrOutputParser())

```

3. Mengimplementasikan Langkah-Langkah Workflow Self-Reflective Definiskan node untuk retrieval, generation, grading, dan transformasi query.

```

#lesson12a.py continued

class GraphState(TypedDict):
    question: str
    generation: str
    documents: List[str]

# Retrieval Grader setup
class GradeDocuments(BaseModel):
    binary_score: str = Field(description="Documents are relevant to the question, 'yes' or 'no'")

```



```

retrieval_prompt = ChatPromptTemplate.from_template("""
You are a grader assessing if a document is relevant to a user's question.
Document: {document}
Question: {question}
Is the document relevant? Answer 'yes' or 'no'.
""")
retrieval_grader = retrieval_prompt | ChatOpenAI(model="gpt-4o-mini",
temperature=0).with_structured_output(GradeDocuments)

# Hallucination Grader setup
class GradeHallucinations(BaseModel):
    binary_score: str = Field(description="Answer is grounded in the documents, 'yes'
or 'no'")

hallucination_prompt = ChatPromptTemplate.from_template("""
You are a grader assessing if an answer is grounded in retrieved documents.
Documents: {documents}
Answer: {generation}
Is the answer grounded in the documents? Answer 'yes' or 'no'.
""")
hallucination_grader = hallucination_prompt | ChatOpenAI(model="gpt-4o-mini",
temperature=0).with_structured_output(GradeHallucinations)

# Answer Grader setup
class GradeAnswer(BaseModel):
    binary_score: str = Field(description="Answer addresses the question, 'yes' or 'no'")

answer_prompt = ChatPromptTemplate.from_template("""
You are a grader assessing if an answer addresses the user's question.
Question: {question}
Answer: {generation}
Does the answer address the question? Answer 'yes' or 'no'.
""")
answer_grader = answer_prompt | ChatOpenAI(model="gpt-4o-mini",
temperature=0).with_structured_output(GradeAnswer)

# Define LangGraph functions
def retrieve(state):
    question = state["question"]
    documents = retriever.invoke(question)
    return {"documents": documents, "question": question}

```

```

def generate(state):
    question = state["question"]
    documents = state["documents"]
    generation = rag_chain.invoke({"context": documents, "question": question})
    return {"documents": documents, "question": question, "generation": generation}

def grade_documents(state):
    """
    Grades documents based on relevance to the question.
    Only relevant documents are retained in 'relevant_docs'.
    """
    question = state["question"]
    documents = state["documents"]
    relevant_docs = []

    for doc in documents:
        response = retrieval_grader.invoke({"question": question, "document":
doc.page_content})
        if response.binary_score == "yes":
            relevant_docs.append(doc)

    return {"documents": relevant_docs, "question": question}

def decide_to_generate(state):
    """
    Decides whether to proceed with generation or transform the query.
    """
    if not state["documents"]:
        return "transform_query" # No relevant docs found; rephrase query
    return "generate" # Relevant docs found; proceed to generate

def grade_generation_v_documents_and_question(state):
    """
    Checks if the generation is grounded in retrieved documents and answers the
    question.
    """
    question = state["question"]
    documents = state["documents"]
    generation = state["generation"]

    # Step 1: Check if the generation is grounded in documents

```

```

hallucination_check = hallucination_grader.invoke({"documents": documents,
"generation": generation})

if hallucination_check.binary_score == "no":
    return "not supported" # Regenerate if generation isn't grounded in documents

# Step 2: Check if generation addresses the question
answer_check = answer_grader.invoke({"question": question, "generation":
generation})
return "useful" if answer_check.binary_score == "yes" else "not useful"

def transform_query(state):
    """
    Rephrases the query for improved retrieval if initial attempts do not yield relevant
    documents.
    """
    transform_prompt = ChatPromptTemplate.from_template("""
    You are a question re-writer that converts an input question to a better version
    optimized for retrieving relevant documents.
    Original question: {question}
    Please provide a rephrased question.
    """)

    question_rewriter = transform_prompt | ChatOpenAI(model="gpt-4o-mini",
temperature=0) | StrOutputParser()

    question = state["question"]
    # Rephrase the question using LLM
    transformed_question = question_rewriter.invoke({"question": question})
    return {"question": transformed_question, "documents": state["documents"]}

```

Mengimplementasikan Self-RAG di LangGraph: Contoh Kompleks

Untuk mengilustrasikan sifat adaptif dari Self-RAG, mari kita buat state machine LangGraph untuk menangani dokumen dan mengevaluasi konten yang dihasilkan.

Setup Workflow Dalam LangGraph, kita mendefinisikan state, edge, dan conditional node dari graph untuk membangun feedback loop yang adaptif:

```

#lesson12a.py continued

# Set up the workflow graph
workflow = StateGraph(GraphState)

```

```

workflow.add_node("retrieve", retrieve)
workflow.add_node("generate", generate)
workflow.add_node("grade_documents", grade_documents)
workflow.add_node("transform_query", transform_query)
workflow.add_edge(START, "retrieve")
workflow.add_edge("retrieve", "grade_documents")
workflow.add_conditional_edges("grade_documents", decide_to_generate,
{"transform_query": "transform_query", "generate": "generate"})
workflow.add_edge("transform_query", "retrieve")
workflow.add_conditional_edges("generate",
grade_generation_v_documents_and_question, {"not_supported": "generate", "useful":
END, "not_useful": "transform_query"})

# Compile the app and run
app = workflow.compile()

# Display the graph
display_graph(app, file_name=os.path.basename(__file__))

# Example input
inputs = {"question": "Explain how the different types of agent memory work?"}
for output in app.stream(inputs):
    print(output)

```

Node-node self-reflective memungkinkan Self-RAG untuk terus-menerus menilai dan meningkatkan proses retrieval dan generation dengan melakukan query ulang jika hasilnya tidak relevan atau tidak memadai. Setup ini sangat berharga untuk menangani pertanyaan yang ambigu atau kompleks.

Keterbatasan dan Langkah Selanjutnya

Meskipun Self-RAG meningkatkan adaptabilitas dalam workflow RAG, hal ini memang menghadirkan beberapa kompleksitas:

- **Latency:** Peningkatan waktu pemrosesan akibat siklus reflection.
- **Resource Use:** Sumber daya komputasi tambahan diperlukan untuk reflection iteratif.
- **Complexity in Implementation:** Merancang workflow adaptif dengan feedback loop membutuhkan penyetelan yang cermat.

Memperkenalkan Teknik Corrective dan Adaptive RAG

Bab berikutnya akan memperluas framework ini dengan menjelajahi Adaptive RAG, di mana model menyesuaikan kedalaman retrieval dan perilaku generation-nya berdasarkan karakteristik query. Kita juga akan memperkenalkan Corrective RAG, yang menggunakan sistem verifikasi eksternal untuk lebih menyempurnakan akurasi respons. Bersama-sama, ini akan membentuk sistem RAG yang lebih dinamis dan serbaguna.

12.2 Corrective Retrieval-Augmented Generation (CRAG)

Corrective Retrieval-Augmented Generation (CRAG) adalah pendekatan inovatif yang dirancang untuk meningkatkan ketangguhan sistem Retrieval-Augmented Generation (RAG) standar dengan mengatasi keterbatasan penting: apa yang terjadi ketika retrieval mengembalikan hasil yang tidak relevan atau salah. CRAG memperkenalkan mekanisme korektif yang menilai dan menyempurnakan kualitas dokumen yang diambil, yang sangat berguna dalam mengurangi risiko halusinasi pada large language model (LLM).

Pada bagian ini, kita akan menjelajahi:

1. **CRAG Concepts and Paper Insights**
2. **Conditions for Corrective RAG**
3. **Implementation of CRAG in LangGraph**
4. **Conformance to the Paper's Methodology**

1. Konsep CRAG dan Wawasan dari Paper

Motivasi utama di balik CRAG adalah untuk meningkatkan keandalan sistem RAG dengan menambahkan lapisan penilaian kualitas dan koreksi selama fase retrieval. LLM, ketika hanya mengandalkan parameter internalnya, rentan mengalami halusinasi-mengarang informasi dengan kepercayaan diri yang keliru. Meskipun RAG dapat mengurangi hal ini dengan mengambil dokumen relevan untuk memandu generation, efektivitasnya dibatasi oleh akurasi konten yang diambil. CRAG memperkenalkan mekanisme untuk mengatasi masalah ini dengan:

- ***Mengimplementasikan retrieval evaluator yang ringan: Komponen ini menilai relevansi keseluruhan dari dokumen yang diambil terhadap query, dengan memberikan confidence score.- Memperkenalkan corrective action: Berdasarkan penilaian relevansi, CRAG dapat memicu tiga tindakan: Correct, Incorrect, atau Ambiguous. Percabangan ini memungkinkan CRAG memutuskan apakah akan menggunakan dokumen yang diambil,***

membuangnya dan menggantinya dengan web search, atau menggabungkan kedua sumber jika relevansinya tidak pasti.- Menambahkan retrieval berbasis web: Dalam kasus di mana dokumen yang diambil tidak relevan, CRAG melengkapi basis pengetahuan dengan hasil web search.- Menyempurnakan pengetahuan dokumen: CRAG dapat lebih lanjut memecah dokumen menjadi "knowledge strips" untuk secara selektif menggunakan bagian-bagian relevan dari dokumen yang diambil, menyaring detail yang tidak relevan.

Elemen-elemen ini menciptakan sistem yang tidak hanya mengambil dokumen tetapi juga menilai dan secara selektif mengoreksinya, beradaptasi secara dinamis berdasarkan kualitas informasi yang tersedia.

2. Kondisi untuk Corrective RAG

Dalam CRAG, tindakan yang berbeda dipicu berdasarkan penilaian relevansi:

- **Correct Action:** Jika confidence score untuk relevansi tinggi, CRAG mengasumsikan dokumen yang diambil relevan dan melanjutkan generation setelah menyempurnakan pengetahuan yang diambil menjadi knowledge strips yang esensial. Ini berguna dalam kasus di mana dokumen berisi informasi yang berguna, tetapi tidak sepenuhnya fokus, yang diuntungkan dari penyempurnaan.- **Incorrect Action:** Jika confidence score rendah di semua dokumen, CRAG melakukan corrective action dengan membuang dokumen yang diambil dan sebagai gantinya memulai web search untuk menemukan data yang lebih relevan.- **Ambiguous Action:** Jika penilaian relevansi menghasilkan skor menengah, CRAG melakukan pendekatan gabungan, mengintegrasikan baik pengetahuan yang diambil yang telah disempurnakan maupun hasil web search. Keseimbangan ini sangat berguna dalam kasus ambigu di mana baik data yang diambil maupun web search murni tidak sepenuhnya memadai.

Tindakan-tindakan ini memastikan bahwa CRAG tetap fleksibel dan adaptif terhadap berbagai tingkat kualitas retrieval, sehingga meningkatkan keandalan.

3. Implementasi CRAG di LangGraph

Kode Python berikut di LangGraph mendemonstrasikan CRAG dengan menyiapkan node yang sesuai dengan langkah-langkah yang dijelaskan di atas:

```
#lesson12a.py continued
```

```
import os
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_core.output_parsers import StrOutputParser
from langgraph.graph import StateGraph, START, END
from pydantic import BaseModel, Field
from typing import List, TypedDict
from langchain.schema import Document
from langchain_community.tools.tavily_search import TavilySearchResults
from display_graph import display_graph
from langchain_core.prompts import ChatPromptTemplate
from pprint import pprint

# Step 1: Load and prepare documents
urls = [
    "https://lilianweng.github.io/posts/2023-06-23-agent/",
    "https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/",
    "https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/"
]

# Load and split documents
docs = [WebBaseLoader(url).load() for url in urls]
docs_list = [item for sublist in docs for item in sublist]
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(chunk_size=250,
chunk_overlap=0)
doc_splits = text_splitter.split_documents(docs_list)
# Add to vectorstore
vectorstore = Chroma.from_documents(doc_splits, collection_name="crag-chroma",
embedding=OpenAIEmbeddings())
retriever = vectorstore.as_retriever()
# Step 2: Define Graders and Relevance Model
class GradeDocuments(BaseModel):
    binary_score: str = Field(description="Documents are relevant to the question, 'yes' or 'no'")

retrieval_prompt = ChatPromptTemplate.from_template("""
You are a grader assessing if a document is relevant to a user's question.
```

```

Document: {document}
Question: {question}
Is the document relevant? Answer 'yes' or 'no'.
"""
retrieval_grader = retrieval_prompt | ChatOpenAI(model="gpt-4o-mini",
temperature=0).with_structured_output(GradeDocuments)
# Step 3: Query Re-writer
class ImproveQuestion(BaseModel):
    improved_question: str = Field(description="Formulate an improved question.")
re_write_prompt = ChatPromptTemplate.from_template(
    "Here is the initial question: \n\n {question} \n Formulate an improved question.",
)
query_rewriter = re_write_prompt | ChatOpenAI(model="gpt-4o-mini",
temperature=0).with_structured_output(ImproveQuestion)

# Define prompt template
prompt = ChatPromptTemplate.from_template("""
Use the following context to answer the question:
Question: {question}
Context: {context}
Answer:
""")
rag_chain = prompt | ChatOpenAI(model="gpt-4o-mini", temperature=0) |
StrOutputParser()
# Define CRAG State
class GraphState(TypedDict):
    question: str
    generation: str
    web_search: str
    documents: List[str]
# Step 4: Define Workflow Nodes
def retrieve(state):
    question = state["question"]
    documents = retriever.invoke(question)
    return {"documents": documents, "question": question}
def grade_documents(state):
    question = state["question"]
    documents = state["documents"]
    filtered_docs = []
    web_search_needed = "No"

```



```

    for doc in documents:
        grade = retrieval_grader.invoke({"question": question, "document":
doc.page_content}).binary_score
        if grade == "yes":
            print("---GRADE: DOCUMENT RELEVANT---")
            filtered_docs.append(doc)
        else:
            print("---GRADE: DOCUMENT NOT RELEVANT---")
            web_search_needed = "Yes"
    return {"documents": filtered_docs, "question": question, "web_search":
web_search_needed}

def transform_query(state):
    question = state["question"]
    rewritten_question = query_rewriter.invoke({"question": question})
    return {"question": rewritten_question.improved_question, "documents":
state["documents"]}

def web_search(state):
    """
    Web search based on the re-phrased question.

    Args:
        state (dict): The current graph state

    Returns:
        state (dict): Updates documents key with appended web results
    """
    print("---WEB SEARCH---")

    question = state["question"]
    documents = state["documents"]
    pprint(question+"\n")
    # Perform web search using TavilySearchResults and extract only the 'content' field
    for Document
        search_results = TavilySearchResults(k=3).invoke({"query": question})

        # Process results to create Document objects only with page_content
        web_documents = [
            Document(page_content=result["content"]) for result in search_results if
"content" in result
        ]

```

```

# Append web search results to the existing documents
documents.extend(web_documents)

return {"documents": documents, "question": question}

def generate(state):
    generation = rag_chain.invoke({"context": state["documents"], "question":
state["question"]})
    return {"generation": generation}
# Step 5: Define Decision-Making Logic
def decide_to_generate(state):
    """
    Determines whether to generate an answer, or re-generate a question.

    Args:
        state (dict): The current graph state

    Returns:
        str: Binary decision for next node to call
    """
    print("---ASSESS GRADED DOCUMENTS---")
    state["question"]
    web_search = state["web_search"]
    state["documents"]
    if web_search == "Yes":
        # All documents have been filtered check_relevance
        # We will re-generate a new query
        print(
            "---DECISION: ALL DOCUMENTS ARE NOT RELEVANT TO
QUESTION, TRANSFORM QUERY---"
        )
        return "transform_query"
# Step 6: Build and Compile the Graph
workflow = StateGraph(GraphState)
workflow.add_node("retrieve", retrieve)
workflow.add_node("grade_documents", grade_documents)
workflow.add_node("transform_query", transform_query)
workflow.add_node("web_searcher", web_search)
workflow.add_node("generate", generate)
# Define edges
workflow.add_edge(START, "retrieve")

```

```

workflow.add_edge("retrieve", "grade_documents")
workflow.add_conditional_edges("grade_documents", decide_to_generate,
{"transform_query": "transform_query", "generate": "generate"})
workflow.add_edge("transform_query", "web_searcher")
workflow.add_edge("web_searcher", "generate")
workflow.add_edge("generate", END)
app = workflow.compile()
# Display the graph
display_graph(app, file_name=os.path.basename(__file__))
# Example input
inputs = {"question": "Explain how the different types of agent memory work?"}
for output in app.stream(inputs):
    for key, value in output.items():
        # Node
        pprint(f"Node '{key}':")
        # Optional: print full state at each node
        # pprint.pprint(value["keys"], indent=2, width=80, depth=None)
    pprint("\n---\n")
pprint(value["generation"])

```

Output:

```

"Node 'retrieve':"
'\n---\n'
---GRADE: DOCUMENT NOT RELEVANT---
---GRADE: DOCUMENT RELEVANT---
---GRADE: DOCUMENT NOT RELEVANT---
---GRADE: DOCUMENT RELEVANT---
---ASSESS GRADED DOCUMENTS---
---DECISION: ALL DOCUMENTS ARE NOT RELEVANT TO QUESTION,
TRANSFORM QUERY---
"Node 'grade_documents':"
'\n---\n'
"Node 'transform_query':"
'\n---\n'
---WEB SEARCH---
What are the various types of agent memory, and how does each type function in the
context of artificial intelligence?"Node 'web_searcher':"
'\n---\n'
"Node 'generate':"
'\n---\n'
('In the context of artificial intelligence, particularly in systems powered '
'by large language models (LLMs), there are two primary types of agent '
'memory: short-term memory and long-term memory.

```

4. Kesesuaian dengan Metodologi Paper CRAG

Implementasi LangGraph sangat selaras dengan metodologi yang dijelaskan dalam paper CRAG:

- **Retrieval Evaluation: Grader dalam workflow LangGraph berfungsi sebagai evaluator retrieval yang ringan yang menilai relevansi setiap dokumen, mencerminkan pendekatan dalam paper tersebut.**- **Corrective Actions: Berdasarkan skor relevansi, workflow LangGraph memicu tindakan yang sesuai (Correct, Incorrect, atau Ambiguous), yang secara langsung mencerminkan kondisi percabangan dalam metodologi CRAG.**- **Web-based Supplementation: Dalam kasus di mana dokumen yang diambil tidak memadai, workflow menggunakan Tavily Search untuk augmentasi berbasis web, mewujudkan prinsip CRAG yaitu melengkapi korpus yang terbatas dengan sumber yang lebih luas.**

Dengan mengimplementasikan komponen-komponen ini, workflow LangGraph mencapai framework CRAG yang kuat, meningkatkan evaluasi kualitas dokumen dan penyempurnaan konteks, sehingga mengatasi keterbatasan RAG asli dan mengurangi halusinasi dalam respons yang dihasilkan.

12.3 Adaptive RAG

Bagian 12.3: Adaptive RAG - Teori dan Implementasi dalam LangGraph

Pengantar Adaptive RAG

Adaptive Retrieval-Augmented Generation (Adaptive-RAG) adalah sebuah framework yang dirancang untuk mengoptimalkan model retrieval-augmented (seperti LLM) berdasarkan kompleksitas query pengguna. Pendekatan RAG standar sering kali gagal memperhitungkan variasi kompleksitas query, sehingga menghasilkan overhead komputasi yang berlebihan untuk query sederhana atau pemrosesan yang tidak memadai untuk query kompleks dan multi-langkah. Adaptive-RAG secara dinamis menyesuaikan strategi retrieval dengan mengkategorikan query ke dalam tiga tingkat kompleksitas dan menerapkan strategi pemrosesan yang sesuai.

Paper Adaptive-RAG memperkenalkan classifier yang mengevaluasi kompleksitas query yang masuk, menentukan strategi retrieval

yang paling sesuai. Klasifikasi ini bertujuan untuk menyeimbangkan efisiensi komputasi dengan akurasi respons, dengan menyesuaikan strategi RAG sebagai berikut:

- **Non-retrieval untuk query sederhana yang dapat dijawab langsung oleh LLM saja.-** **Single-step retrieval untuk query dengan kompleksitas sedang yang memerlukan konteks tambahan.-** **Multi-step retrieval untuk query kompleks yang melibatkan sintesis informasi dari berbagai dokumen dan penalaran atasnya.**

Konsep dari Paper Adaptive-RAG

Adaptive-RAG mengusulkan solusi bagi LLM retrieval-augmented untuk mengatasi variabilitas dalam kompleksitas query, melampaui pendekatan one-size-fits-all yang terlalu sederhana atau terlalu kompleks untuk query tertentu. Aspek inti dari Adaptive-RAG meliputi:

1. **Complexity-based Strategy Selection:** By determining query complexity, Adaptive-RAG applies an efficient retrieval strategy, choosing between:
 - **Non-retrieval:** Simple queries are handled directly by the LLM without retrieval.
 - **Single-step retrieval:** Queries that need extra information from one document are processed in a single retrieval step.
 - **Multi-step retrieval:** Complex, multi-hop queries involve iterative retrieval, integrating multiple documents and employing reasoning chains.
2. **Query Complexity Classifier:** Adaptive-RAG introduces a classifier to assess query complexity levels: “A” for simple (handled by the LLM), “B” for moderately complex (requires single-step retrieval), and “C” for complex (requires multi-step retrieval). This classifier uses training data created by analyzing the success rates of each retrieval method and leveraging inductive biases from existing datasets.
3. **Efficiency and Accuracy Balance:** The framework is structured to optimize for both computational efficiency and answer accuracy, selecting simpler approaches for straightforward

questions to conserve computational resources. This balance is crucial in real-world applications, as not all queries necessitate extensive retrieval and reasoning.

Kondisi untuk Mengimplementasikan Adaptive-RAG dalam LangGraph

Framework Adaptive-RAG dalam LangGraph memerlukan komponen-komponen berikut:

1. **Classifier Model:** This smaller model determines the complexity of the input query and assigns it to one of the three complexity classes (A, B, or C).
2. **Multi-path Retrieval and Processing Logic:** The system dynamically chooses among the three processing paths—non-retrieval, single-step retrieval, and multi-step retrieval—based on the classifier’s output.
3. **Multi-step Query Handling for Complex Queries:** Complex queries use iterative retrieval and reasoning steps, progressively refining the query until all relevant information is gathered.

Implementasi dalam LangGraph

Mengimplementasikan Adaptive-RAG dalam LangGraph melibatkan penyiapan classifier kompleksitas query, diikuti dengan merancang state graph yang bercabang ke jalur berbeda berdasarkan output classifier tersebut. Berikut adalah rincian implementasinya, yang selaras erat dengan metodologi paper tersebut.

1. **Define the Query Complexity Classifier:**
 - Train a classifier using a smaller language model (such as T5-Large) that categorizes queries into three levels of complexity (A, B, or C).
 - The classifier model can be integrated into LangGraph as a separate node to evaluate each incoming query before choosing a retrieval strategy.
2. **Construct Retrieval Nodes and Paths:**

- Define three distinct retrieval nodes for non-retrieval, single-step retrieval, and multi-step retrieval.
- Implement a `retrieve()` function that conditionally chooses the retrieval strategy based on the classifier's output.

3. Multi-step Retrieval for Complex Queries:

- For queries classified as complex (C), enable an iterative retrieval process where documents are retrieved and assessed in multiple rounds until a satisfactory answer is formed.
- This iterative process incorporates reasoning chains and intermediate answers, as required by complex queries.

4. Decision-Making Logic in LangGraph:

- Add decision nodes that leverage the classifier's output to route the query through the appropriate retrieval strategy.
- Implement a conditional edge from the classifier to each retrieval node (non-retrieval, single-step, multi-step).

Implementasi Nyata dalam LangGraph

Pengantar Adaptive Retrieval-Augmented Generation (Adaptive-RAG)

Adaptive RAG (Adaptive Retrieval-Augmented Generation) adalah pendekatan fleksibel yang dirancang untuk mengoptimalkan respons dengan memilih strategi retrieval yang paling sesuai untuk setiap query. Adaptive-RAG menggabungkan analisis query dengan active RAG, menentukan sifat query dan memilih strategi retrieval yang sesuai. Pendekatan ini berguna untuk menangani query yang beragam, di mana beberapa dapat diatasi dengan retrieval informasi sederhana, sementara yang lain memerlukan penalaran kompleks multi-langkah.

Dalam setup Adaptive-RAG yang tipikal:

- **No Retrieval** digunakan untuk pertanyaan yang dapat dijawab langsung oleh model bahasa tanpa data tambahan.- **Single-Shot Retrieval** diterapkan pada pertanyaan dengan kompleksitas sedang yang memerlukan retrieval satu langkah.

- **Iterative Retrieval** diperuntukkan bagi query kompleks multi-hop yang memerlukan beberapa putaran retrieval dan pemrosesan.

Konsep Kunci dari Adaptive-RAG

Adaptive-RAG memperkenalkan beberapa konsep penting, seperti yang diuraikan dalam paper tersebut:

1. Query Analysis and Routing:

- The framework first assesses the complexity of each query, routing it to the appropriate retrieval path.
- This initial analysis is crucial in determining whether a query requires no retrieval, a single-step retrieval, or iterative retrieval.

2. Self-Corrective Mechanisms:

- By leveraging an LLM's ability to evaluate intermediate results, Adaptive-RAG enables the retrieval system to self-correct.
- This includes grading retrieved documents for relevance and, if needed, rerouting the query or rephrasing it for more accurate results.

3. Multi-Path Decision-Making:

- Adaptive-RAG routes queries through different paths depending on complexity.
- For example, simple factual questions (like "Who invented the telephone?") might bypass retrieval and go directly to the LLM, while complex analytical questions (like "How did the invention of the telephone impact global communication?") might trigger a multi-step retrieval flow.

Mengimplementasikan Adaptive-RAG dalam LangGraph

Pada bagian ini, kita akan mengembangkan implementasi nyata dari Adaptive-RAG menggunakan LangGraph. Setup kita akan melibatkan:

1. Web Search for recent events or questions requiring up-to-date information.
2. Self-Corrective RAG for questions that need information from a predefined index of documents.

Menyiapkan Lingkungan

```
terminal

# Install required packages and set API keys
! pip install -U langchain_community tiktoken langchain-openai langchainhub chromadb
langgraph tavily-python
```

```
#lesson12b.py

import getpass, os

def _set_env(var: str):
    if not os.environ.get(var):
        os.environ[var] = getpass.getpass(f"{var}: ")

_set_env("OPENAI_API_KEY")
_set_env("TAVILY_API_KEY")
```

Langkah 1: Membangun Index

Kita akan mulai dengan membuat index dokumen yang terkait dengan AI dan model bahasa. Index ini akan digunakan untuk retrieval jika query diarahkan ke vector store.

```
#lesson12b.py continued

from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# Define documents for indexing
urls = [
    "https://lilianweng.github.io/posts/2023-06-23-agent/",
    "https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/",
    "https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/"
]
```

```
# Load and split documents
docs = [WebBaseLoader(url).load() for url in urls]
docs_list = [item for sublist in docs for item in sublist]
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500)
doc_splits = text_splitter.split_documents(docs_list)

# Store documents in a vector database (Chroma)
vectorstore = Chroma.from_documents(doc_splits, collection_name="adaptive-rag",
embedding=OpenAIEmbeddings())
retriever = vectorstore.as_retriever()
```

Langkah 2: Query Routing

Query router kita akan menggunakan model ringan untuk menentukan apakah sebuah pertanyaan harus dijawab dengan:

- **Vector Store Retrieval:** Query yang terkait dengan topik yang telah diindeks.
- **Web Search:** Query tentang peristiwa terkini atau topik di luar cakupan index.

```
#lesson12b.py continued

from typing import Literal
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
# Define routing model
class RouteQuery(BaseModel):
    datasource: Literal["vectorstore", "web_search"]

route_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert at routing a user question to vectorstore or web search."),
    ("human", "{question}")
])

question_router = route_prompt | ChatOpenAI(model="gpt-3.5-turbo",
temperature=0).with_structured_output(RouteQuery)
```

Langkah 3: Retrieval Grader dan Self-Correction

Node ini akan mengevaluasi apakah dokumen yang diambil relevan dengan query. Jika dokumen dinilai tidak relevan, query dapat diformulasikan ulang atau dilengkapi dengan retrieval tambahan.

```
#lesson12b.py continued

class GradeDocuments(BaseModel):
    binary_score: str = Field(description="Documents are relevant to the question, 'yes' or 'no'")

    grade_prompt = ChatPromptTemplate.from_messages([
        ("system", "Evaluate if the document is relevant to the question. Answer 'yes' or 'no'."),
        ("human", "Document: {document}\nQuestion: {question}")
    ])

    retrieval_grader = grade_prompt | ChatOpenAI(model="gpt-3.5-turbo", temperature=0).with_structured_output(GradeDocuments)
```

Langkah 4: Web Search dan RAG Generation

Kita akan menggunakan Tavily web search API untuk mengambil informasi jika query diarahkan ke web search.

```
#lesson12b.py continued

from langchain_community.tools.tavily_search import TavilySearchResults
from langchain.schema import Document
web_search_tool = TavilySearchResults(k=3)

def web_search(state):
    search_results = web_search_tool.invoke({"query": state["question"]})
    web_documents = [Document(page_content=result["content"]) for result in search_results if "content" in result]
    return {"documents": web_documents, "question": state["question"]}
```

Langkah 5: Logika Pengambilan Keputusan dan Workflow Graph

Sekarang kita membangun state graph di LangGraph untuk menangani proses query routing, retrieval, dan grading.

```
#lesson12b.py continued
```

```

def generate(state):
    prompt_template = """
    Use the following context to answer the question concisely and accurately:
    Question: {question}
    Context: {context}
    Answer:
    """

    # Define ChatPromptTemplate using the above template
    rag_prompt = ChatPromptTemplate.from_template(prompt_template)
    # Create the RAG generation chain with LLM and output parsing
    rag_chain = (
        rag_prompt |
        ChatOpenAI(model="gpt-4o-mini", temperature=0) |
        StrOutputParser()
    )
    generation = rag_chain.invoke({"context": state["documents"], "question":
state["question"]})
    return {"generation": generation}
# Route question based on source
def route_question(state):
    source = question_router.invoke({"question": state["question"]}).datasource
    return "web_search" if source == "web_search" else "retrieve"
# Compile and Run the Graph
workflow = StateGraph(GraphState)
workflow.add_node("web_search", web_search)
workflow.add_node("retrieve", retrieve)
workflow.add_node("grade_documents", grade_documents)
workflow.add_node("generate", generate)
workflow.add_conditional_edges(START, route_question, {"web_search":
"web_search", "retrieve": "retrieve"})
workflow.add_edge("web_search", "generate")
workflow.add_edge("retrieve", "grade_documents")
workflow.add_edge("grade_documents", "generate")
workflow.add_edge("generate", END)
app = workflow.compile()
# Run with example inputs
inputs = {"question": "What are the types of agent memory?"}
for output in app.stream(inputs):
    print(output)

```

Output yang Diharapkan

Dengan pertanyaan input tertentu, setup ini akan menghasilkan output berikut:

- **Web Search** untuk query tentang peristiwa terkini, menghasilkan informasi dari sumber terbaru.- **Self-corrective RAG** untuk pertanyaan yang memanfaatkan vector store, menyaring dokumen berdasarkan relevansi dan menyempurnakan respons berdasarkan kualitas dokumen.

Bagian Penjelasan: Konsep dalam Bab 12 - Arsitektur RAG Tingkat Lanjut

Bagian penjelasan ini membahas konsep-konsep dasar yang menjadi inti dari arsitektur Self-RAG, Corrective RAG, dan Adaptive RAG yang disajikan dalam Bab 12. Dengan memahami konsep-konsep dasar ini, pembaca dapat lebih efektif memahami struktur dan metodologi yang dibahas di seluruh bab ini.

Konsep Kunci

1. Vector Store

Vector store adalah database khusus yang dioptimalkan untuk menyimpan vektor berdimensi tinggi, yang umumnya berasal dari embedding model. Dalam RAG, vektor merepresentasikan embedding dari dokumen dan query, yang dihasilkan menggunakan model bahasa atau algoritma embedding lainnya. Menyimpan embedding dokumen dalam vector store memungkinkan pencarian kemiripan (similarity search) yang efisien dengan membandingkan embedding query dengan embedding dokumen yang tersimpan. Hal ini memungkinkan retrieval yang cepat dan scalable, terutama untuk dataset teks yang kompleks atau berskala besar.

Dalam LangChain, Chroma berfungsi sebagai vector store yang terintegrasi secara khusus dengan tools embedding dokumen dan teks milik LangChain. Chroma bersifat fleksibel, memungkinkan embedding dari model seperti embedding model OpenAI atau embedding Cohere, untuk mendukung berbagai fungsi similarity search.

Penggunaan Python dalam LangChain:

```
#explainer

from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# Store documents in a Chroma vector store
vectorstore = Chroma.from_documents(
    documents=doc_splits,
    collection_name="rag-chroma",
    embedding=OpenAIEmbeddings()
)
```

```
retriever = vectorstore.as_retriever()
```

2. LangChain

LangChain adalah framework Python tingkat lanjut untuk membuat aplikasi yang terintegrasi dengan model bahasa. Framework ini menyederhanakan proses document loading, text splitting, pembuatan embedding, dan chaining output dari berbagai task yang digerakkan oleh LLM. Modularitas LangChain mendukung pembuatan workflow yang kompleks, termasuk arsitektur RAG, dengan memfasilitasi integrasi dengan berbagai vector store, prompt template, dan state machine seperti LangGraph.

Ekosistem LangChain mencakup komponen untuk:

- **Document Loading:** Memuat dokumen dari berbagai sumber.-
- Text Splitting:** Memecah dokumen besar untuk retrieval yang efisien.-
- Embeddings:** Menghasilkan embedding dokumen untuk vector store.-
- Chains:** Menyusun task seperti workflow RAG atau QA, di mana output dari satu langkah dapat mengalir ke langkah berikutnya.

3. LangGraph dan State Machine

LangGraph adalah ekstensi dalam LangChain yang memperkenalkan state machine untuk mengelola workflow. State machine menyediakan kontrol atas langkah kondisional, perulangan (looping), dan mekanisme feedback. State machine sangat berguna dalam arsitektur RAG seperti Self-RAG, di mana keputusan-seperti apakah akan melakukan retrieval, mentransformasi query, atau melanjutkan dengan generasi jawaban-dipengaruhi secara dinamis oleh konteks.

Dalam LangGraph, sebuah state machine diimplementasikan dengan node dan edge:

- **Nodes:** Merepresentasikan langkah-langkah individual dalam workflow (misalnya, mengambil dokumen, menghasilkan respons, memformulasikan ulang query).-
- Edges:** Mendefinisikan transisi antar node berdasarkan kondisi tertentu.

Class StateGraph milik LangGraph memungkinkan pembuatan workflow yang fleksibel dengan node dan logika pengambilan keputusan.

4. Prompt Engineering dan ChatPromptTemplate

Dalam LangChain, prompt engineering melibatkan perancangan prompt yang mengarahkan LLM untuk menghasilkan output yang diinginkan. ChatPromptTemplate adalah tool dalam LangChain yang memformat prompt ini, mengorganisasikan pesan sistem, instruksi, dan variabel dinamis seperti {question} atau {context}. Prompt yang dirancang dengan baik sangat penting dalam workflow RAG untuk mengarahkan LLM agar fokus pada konten tertentu, menilai respons, atau mengevaluasi relevansi.

```
#explainer

from langchain_core.prompts import ChatPromptTemplate

# Example prompt template for RAG generation
prompt_template = ChatPromptTemplate.from_template("""
    Use the following context to answer the question:
    Question: {question}
    Context: {context}
    Answer:
    """)
```

5. Document Grading: Retrieval dan Hallucination Grader

Dalam workflow Self-RAG dan Corrective RAG, document grading mengevaluasi relevansi dokumen yang diambil. Dengan menggunakan grader, workflow LangChain dapat menentukan apakah dokumen yang diambil relevan atau apakah respons yang dihasilkan berlandaskan pada dokumen-dokumen tersebut. Jenis grader yang umum meliputi:

- **Retrieval Grader: Menilai relevansi setiap dokumen yang diambil terhadap query.**
- **Hallucination Grader: Menentukan apakah jawaban yang dihasilkan didukung secara faktual oleh dokumen yang diambil.**
- **Answer Grader: Memastikan bahwa jawaban tersebut menjawab pertanyaan pengguna.**

Grading diimplementasikan menggunakan LLM yang merespons dengan skor biner (misalnya, \u201cyaya\u201d atau \u201ctidak\u201d) berdasarkan instruksi prompt.

```
#explainer

from pydantic import BaseModel, Field
```



```

from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

class GradeDocuments(BaseModel):
    binary_score: str = Field(description="Documents are relevant to the question, 'yes' or 'no'")

# Define retrieval grader with a prompt
retrieval_grader_prompt = ChatPromptTemplate.from_template("""
    You are a grader assessing if a document is relevant to a user's question.
    Document: {document}
    Question: {question}
    Is the document relevant? Answer 'yes' or 'no'.
    """)
retrieval_grader = retrieval_grader_prompt | ChatOpenAI(model="gpt-4o-mini").with_structured_output(GradeDocuments)

```

6. Conditional Routing dan Query Rewriting

Conditional Routing memungkinkan workflow LangGraph untuk menyesuaikan jalur secara dinamis berdasarkan kondisi. Misalnya, dalam Adaptive RAG, sebuah router node menentukan apakah sebuah pertanyaan harus diarahkan ke vector store untuk retrieval atau melakukan web search untuk informasi terkini. **Conditional edges** memfasilitasi routing ke node yang berbeda berdasarkan konteks atau relevansi dokumen yang diambil.

Query Rewriting digunakan untuk memformulasikan ulang query agar retrieval lebih baik ketika upaya awal menghasilkan dokumen yang tidak relevan atau tidak memadai. Dengan memformulasikan ulang pertanyaan, workflow meningkatkan peluang untuk mengambil dokumen berkualitas tinggi dan relevan. Query rewriting melibatkan rewriter berbasis LLM yang menyesuaikan query sambil tetap mempertahankan makna semantiknya.

```

#explainer

def transform_query(state):
    question = state["question"]
    rephrased_question = question_rewriter.invoke({"question": question})
    return {"question": rephrased_question, "documents": state["documents"]}

```

7. Web Search Tool

Dalam arsitektur RAG tertentu, ketika retrieval internal dari vector store tidak memadai atau tidak relevan, web search eksternal digunakan. Tavily Search API, yang terintegrasi dengan LangChain, menyediakan hasil real-time dari web untuk melengkapi knowledge base. Web search dapat menjadi hal yang esensial dalam Corrective RAG, di mana dokumen yang diambil dan tidak relevan diganti atau dilengkapi dengan data terbaru dari internet.

8. Mekanisme Self-Correction

Self-correction adalah landasan utama dari baik Self-RAG maupun Adaptive RAG. Dengan menerapkan proses evaluasi dan pengambilan keputusan yang iteratif, workflow ini meningkatkan kualitas output. Sebagai contoh, dalam Self-RAG, grader mengevaluasi jawaban untuk memastikan bahwa jawaban tersebut berlandaskan pada dokumen yang diambil, sementara Adaptive RAG secara dinamis menyesuaikan strategi retrieval berdasarkan kompleksitas query. Pendekatan ini meningkatkan akurasi, mengurangi halusinasi, dan memastikan relevansi, menciptakan workflow yang fleksibel dan responsif.

Ringkasan

Konsep-konsep ini mendasari arsitektur RAG tingkat lanjut dalam LangGraph, meningkatkan adaptabilitas, akurasi, dan relevansi setiap workflow. Dengan menggunakan komponen modular seperti vector store, state machine, grading, conditional routing, dan web search, Self-RAG, Corrective RAG, dan Adaptive RAG dapat menangani pertanyaan yang semakin kompleks dengan jawaban yang presisi dan sadar konteks.

Kuis Bab 12: Arsitektur RAG Tingkat Lanjut (Self-RAG, Corrective RAG, Adaptive RAG)

Soal Pilihan Ganda

- 1. What is the primary function of the vector store in a RAG workflow?**
 - A) To generate embeddings of the user query
 - B) To efficiently store and retrieve high-dimensional document embeddings
 - C) To grade the relevance of retrieved documents
 - D) To handle API calls for external web searches

- 2. Which token in Self-RAG decides whether the generated answer is grounded in the retrieved documents?**
 - A) Retrieve Token
 - B) ISREL Token
 - C) ISSUP Token
 - D) ISUSE Token

- 3. In Corrective RAG, what is the purpose of segmenting retrieved documents into “knowledge strips”?**
 - A) To create distinct parts of a question for iterative processing
 - B) To improve the relevance of retrieved information by focusing on specific content
 - C) To expand the context size limit for better retrieval
 - D) To transform the query for web search

- 4. Which RAG method routes queries to a specific retrieval strategy based on the complexity of the question?**
 - A) Corrective RAG
 - B) Self-RAG
 - C) Adaptive RAG
 - D) Knowledge-Graph RAG

5. In Adaptive RAG, which retrieval method is chosen for straightforward questions answerable directly by the LLM?

- A) No Retrieval
- B) Single-step Retrieval
- C) Multi-step Retrieval
- D) Web-based Retrieval

Soal Benar/Salah

6. **True/False:** Self-RAG uses a predefined set number of documents for retrieval, regardless of their relevance.
7. **True/False:** In Corrective RAG, if all documents are deemed irrelevant, a web search can be initiated to supplement or replace the existing documents.
8. **True/False:** Adaptive RAG uses a query classifier to determine whether a question should undergo non-retrieval, single-step retrieval, or multi-step retrieval.

Soal Jawaban Singkat

9. Describe one primary advantage of using LangGraph's state machines in implementing Self-RAG workflows.
10. In Corrective RAG, what action is taken if retrieved documents return an ambiguous relevance score?
11. Explain the concept of "self-correction" in the context of Adaptive RAG. How does it improve the response quality of a RAG workflow?

Chapter 13

Multiagent Architectures

13.1 Agent Supervisor

Sistem multi-agent dalam aplikasi yang digerakkan oleh LLM (Large Language Model) melibatkan beberapa agent otonom yang bekerja sama untuk mencapai tujuan yang kompleks. Agent Supervisor sangat penting dalam mengelola agent-agent ini secara efektif dengan mengoordinasikan task, menetapkan tanggung jawab, dan memantau hasil untuk menjaga konsistensi dan kualitas di seluruh jaringan multi-agent. Dalam arsitektur multi-agent, supervisor melakukan tugas koordinasi dan evaluasi, berfungsi sebagai node sentral yang memastikan agent-agent bekerja secara kolaboratif tanpa tujuan yang saling bertentangan atau upaya yang tumpang tindih.

Konsep Inti Agent Supervisor

1. **Task Delegation and Orchestration:** The Agent Supervisor assigns tasks to agents based on the type of request and each agent's capabilities. This avoids redundant efforts, keeps agents focused, and optimizes the workflow by ensuring that each agent's specific skills are utilized.
2. **Monitoring and Error Correction:** By observing the performance and outputs of each agent, the supervisor can detect when an agent needs help, provide corrections, or trigger

additional actions if an agent's output lacks accuracy or completeness.

3. **Communication Management:** Effective multi-agent architectures require streamlined communication. The supervisor helps manage message exchanges, making decisions about which agent should respond next, especially in multi-turn conversations where the agents' responses influence the workflow.

Seperti dinyatakan dalam paper framework AutoGen, fitur menonjol dari sistem multi-agent modern adalah integrasi \u201cconversation programming\u201d, di mana agent terlibat dalam percakapan terstruktur untuk mengoordinasikan dan membagi task secara efektif. AutoGen menawarkan tools untuk \u201cconversation-centric computation\u201d, yang memungkinkan interaksi multi-agent berkembang melalui pertukaran pesan yang secara dinamis membentuk alur berdasarkan peran khusus masing-masing agent.

Contoh Praktis 1: Jaringan Asisten Riset

Dalam lingkungan riset, beberapa agent dapat menangani aspek yang berbeda dari pengumpulan informasi, analisis, dan pelaporan. Sebagai contoh:

- **Research Agent: Melakukan tinjauan literatur dan mengumpulkan data dari database ilmiah.**
- **Data Analysis Agent: Memproses data kuantitatif menggunakan tools statistik.**
- **Reporting Agent: Menyusun temuan ke dalam dokumen terstruktur.**

Agent Supervisor mengoordinasikan agent-agent tersebut dengan memulai task melalui Research Agent, menugaskan analisis kepada Data Analysis Agent, dan mengakhirinya dengan Reporting Agent untuk menyelesaikan task. Supervisor akan turun tangan jika, misalnya, diperlukan analisis data tambahan, dengan mengarahkan Data Analysis Agent untuk memproses ulang data dengan parameter tertentu.

```
#lesson13a.py
```

```
from typing import Annotated
```

```
from langchain_community.tools.tavily_search import TavilySearchResults
```

```
from langchain_experimental.tools import PythonREPLTool
```

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel
```

```

from typing import Literal

tavily_tool = TavilySearchResults(max_results=5)

# This executes code locally, which can be unsafe
python_repl_tool = PythonREPLTool()

from langchain_core.messages import HumanMessage

def agent_node(state, agent, name):
    result = agent.invoke(state)
    return {
        "messages": [HumanMessage(content=result["messages"][-1].content,
name=name)]
    }

members = ["Researcher", "Coder"]
system_prompt = (
    "You are a supervisor tasked with managing a conversation between the"
    " following workers: {members}. Given the following user request,"
    " respond with the worker to act next. Each worker will perform a"
    " task and respond with their results and status. When finished,"
    " respond with FINISH."
)
# Our team supervisor is an LLM node. It just picks the next agent to process
# and decides when the work is completed
options = ["FINISH"] + members

class routeResponse(BaseModel):
    next: Literal[*options]

prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system_prompt),
        MessagesPlaceholder(variable_name="messages"),
        (
            "system",
            "Given the conversation above, who should act next?"
            " Or should we FINISH? Select one of: {options}",
        ),
    ]
).partial(options=str(options), members=", ".join(members))

llm = ChatOpenAI(model="gpt-4o-mini")

def supervisor_agent(state):
    supervisor_chain = prompt | llm.with_structured_output(routeResponse)
    return supervisor_chain.invoke(state)

import functools

```

```

import operator
from typing import Sequence
from typing_extensions import TypedDict

from langchain_core.messages import BaseMessage

from langgraph.graph import END, StateGraph, START
from langgraph.prebuilt import create_react_agent

# The agent state is the input to each node in the graph
class AgentState(TypedDict):
    # The annotation tells the graph that new messages will always
    # be added to the current states
    messages: Annotated[Sequence[BaseMessage], operator.add]
    # The 'next' field indicates where to route to next
    next: str

research_agent = create_react_agent(llm, tools=[tavily_tool])
research_node = functools.partial(agent_node, agent=research_agent,
name="Researcher")

# NOTE: THIS PERFORMS ARBITRARY CODE EXECUTION. PROCEED
WITH CAUTION
code_agent = create_react_agent(llm, tools=[python_repl_tool])
code_node = functools.partial(agent_node, agent=code_agent, name="Coder")

workflow = StateGraph(AgentState)
workflow.add_node("Researcher", research_node)
workflow.add_node("Coder", code_node)
workflow.add_node("supervisor", supervisor_agent)

for member in members:
    # We want our workers to ALWAYS "report back" to the supervisor when done
    workflow.add_edge(member, "supervisor")
# The supervisor populates the "next" field in the graph state
# which routes to a node or finishes
conditional_map = {k: k for k in members}
conditional_map["FINISH"] = END
workflow.add_conditional_edges("supervisor", lambda x: x["next"],
conditional_map)
# Finally, add entrypoint
workflow.add_edge(START, "supervisor")

graph = workflow.compile()

for s in graph.stream(
    {
        "messages": [
            HumanMessage(content="Research recent advances in AI and
summarize.")
        ]
    }

```



```

):
    if "__end__" not in s:
        print(s)
        print("----")

for s in graph.stream(
    {"messages": [HumanMessage(content="Write a brief pdf research report on
pikas.")]},
    {"recursion_limit": 100},
):
    if "__end__" not in s:
        print(s)
        print("----")

```

Output:

```
{'supervisor': {'next': 'Researcher'}}
```

```
{'Researcher': {'messages': [HumanMessage(content='Recent advances in AI throughout
2023 have been marked by significant developments across various domains:\n\n1.
**Generative AI Boom**': Generative AI has emerged as a transformative force within
industries. Organizations that are early adopters of AI technologies, referred to as "AI
high performers," are increasingly leveraging generative AI to create new revenue
streams and enhance existing offerings. A survey revealed that nearly 25% of C-suite
executives are using generative AI tools, indicating its rising prominence in
organizational strategies. [Source: McKinsey]\n\n2. **Scientific Applications**':
Microsoft has made strides in using AI for scientific discovery, applying advanced AI
models in fields like weather forecasting, protein design, and materials science. This
year has seen the development of new language models and frameworks that enhance AI
applications. [Source: Microsoft Research]\n\n3. **Ethical Frameworks and
Collaboration**': Google Research and DeepMind have reported breakthroughs in
generative AI, emphasizing the importance of ethical principles and partnerships in their
advancements. This reflects a broader trend of addressing ethical concerns while
innovating in AI technology. [Source: Google Research]\n\n4. **Global AI Governance
and Regulation**': The geopolitical landscape surrounding AI has evolved, with
countries like the US and China heavily investing in AI research and development. The
UK and France are advancing regulations to ensure ethical AI development, highlighting
the need for transparency and accountability amidst the rapid integration of AI into
various sectors. [Source: ZDNet]\n\n5. **Emergence of Open-Source AI**': The year
has also seen a rise in open-source AI initiatives and discussions around licensing. This
shift has facilitated the development of powerful generative AI models and has prompted
debates on the implications of such technologies. [Source: ZDNet]\n\n6. **Hype Cycle
of AI Technologies**': According to Gartner's 2023 Hype Cycle, AI simulation
technologies that combine AI with simulation environments are gaining traction, paving
the way for further innovations in generative AI. [Source: Gartner]\n\nOverall, 2023 has
been characterized by a surge in generative AI applications, a focus on ethical
considerations, and significant global investments in AI research and development.',
additional_kwargs={}, response_metadata={}, name='Researcher')]]}
```

```
{'supervisor': {'next': 'Coder'}}
```

```
{'Coder': {'messages': [HumanMessage(content="In 2023, notable advances in AI include:\n\n1. Generative AI Expansion: Generative AI is becoming a key player in various industries, with about 25% of C-suite executives actively using these tools to improve offerings and create new revenue streams (McKinsey).\n\n2. Scientific Innovations: Microsoft is utilizing AI for scientific breakthroughs, particularly in areas like weather forecasting, protein design, and materials science, alongside the development of new AI models and frameworks (Microsoft Research).\n\n3. Ethical AI Development: Google Research and DeepMind emphasize the importance of ethical principles in their advancements, showcasing a trend towards responsible innovation in AI technology (Google Research).\n\n4. Global AI Governance: Countries such as the US and China are heavily investing in AI, while the UK and France focus on regulations for ethical AI development, underscoring the need for transparency and accountability (ZDNet).\n\n5. Open-Source AI Initiatives: There is a growing movement towards open-source AI, which is fostering the development of powerful generative AI models and sparking discussions on their implications (ZDNet).\n\n6. AI Simulation Technologies: Gartner's 2023 Hype Cycle highlights the growing interest in AI simulation technologies, indicating a new wave of innovations in generative AI (Gartner).\n\nOverall, 2023 reflects a surge in generative AI applications, a commitment to ethical practices, and substantial global investments in AI research and development.", additional_kwargs={}, response_metadata={}, name='Coder')]]}

----
{'supervisor': {'next': 'FINISH'}}
----
{'supervisor': {'next': 'Researcher'}}
----
```

Contoh Research Assistant Network ini mencakup sistem multi-agent yang dikelola oleh sebuah AgentSupervisor. Supervisor menugaskan tugas kepada Research Agent, Data Analysis Agent, dan Reporting Agent, memberikan pendekatan terstruktur untuk mengelola tugas riset, analisis, dan pelaporan.

Contoh Praktis 2: Otomasi Layanan Pelanggan

Untuk otomasi layanan pelanggan, sebuah organisasi dapat menerapkan sistem multi-agent di mana setiap agent berspesialisasi pada kebutuhan pelanggan yang berbeda, seperti:

- **Query Agent:** Menangani pertanyaan awal dari pelanggan.- **Resolution Agent:** Menyelesaikan masalah teknis atau logistik.- **Escalation Agent:** Menangani pertanyaan kompleks atau menescalasikan kasus ke perwakilan manusia.

Supervisor menugaskan tugas berdasarkan kompleksitas pertanyaan pelanggan, mengarahkan pertanyaan sederhana ke Query Agent dan masalah kompleks ke Resolution Agent. Jika sebuah penyelesaian memerlukan intervensi manusia, supervisor mendelegasikannya ke Escalation Agent, memastikan transisi yang lancar dan penanganan yang efisien terhadap beragam kebutuhan pelanggan.

```
#lesson13b.py complete code example

from functools import partial
import operator
from typing import Annotated, Sequence, TypedDict, Literal
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_experimental.tools import PythonREPLTool
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, BaseMessage
from pydantic import BaseModel
from langgraph.graph import END, StateGraph, START
from langgraph.prebuilt import create_react_agent

# Define RouteResponse for Customer Service Supervisor
class RouteResponseCS(BaseModel):
    next: Literal["Query_Agent", "Resolution_Agent", "Escalation_Agent", "FINISH"]

# Setup for Customer Service Supervisor
members_cs = ["Query_Agent", "Resolution_Agent", "Escalation_Agent"]
system_prompt_cs = f"You are a Customer Service Supervisor managing agents: {' '.join(members_cs)}."

# Create prompt template for the supervisor with correctly formatted options
prompt_cs = ChatPromptTemplate.from_messages([
    ("system", system_prompt_cs),
    MessagesPlaceholder(variable_name="messages"),
    ("system", "Choose the next agent to act from {options}."),
]).partial(options=str(members_cs))

# Define LLM and Supervisor function
llm = ChatOpenAI(model="gpt-4o-mini")

def supervisor_agent_cs(state):
    supervisor_chain_cs = prompt_cs | llm.with_structured_output(RouteResponseCS)
```

```

        return supervisor_chain_cs.invoke(state)

# Agent node function to handle message flow to each agent
def agent_node(state, agent, name):
    result = agent.invoke(state)
    return {
        "messages": [HumanMessage(content=result["messages"][-1].content,
name=name)]
    }

# Define agents for Customer Service tasks with realistic tools
query_agent = create_react_agent(llm, tools=[TavilySearchResults(max_results=5)])
resolution_agent = create_react_agent(llm, tools=[PythonREPLTool()])
escalation_agent = create_react_agent(llm, tools=[PythonREPLTool()])

# Create nodes for each agent with valid names
query_node = partial(agent_node, agent=query_agent, name="Query_Agent")
resolution_node = partial(agent_node, agent=resolution_agent,
name="Resolution_Agent")
escalation_node = partial(agent_node, agent=escalation_agent,
name="Escalation_Agent")

# Define Customer Service graph state and workflow
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
    next: str

# Initialize StateGraph and add nodes
workflow_cs = StateGraph(AgentState)
workflow_cs.add_node("Query_Agent", query_node)
workflow_cs.add_node("Resolution_Agent", resolution_node)
workflow_cs.add_node("Escalation_Agent", escalation_node)
workflow_cs.add_node("supervisor", supervisor_agent_cs)

# Define edges for agents to return to the supervisor
for member in members_cs:
    workflow_cs.add_edge(member, "supervisor")

# Define conditional map for routing
conditional_map_cs = {k: k for k in members_cs}
conditional_map_cs["FINISH"] = END
workflow_cs.add_conditional_edges("supervisor", lambda x: x["next"],
conditional_map_cs)

```

```

workflow_cs.add_edge(START, "supervisor")

# Compile and test the graph
graph_cs = workflow_cs.compile()
# Example input for testing
inputs_cs = {"messages": [HumanMessage(content="Help me reset my password.")]}

# Run the graph
for output in graph_cs.stream(inputs_cs):
    if "__end__" not in output:
        print(output)

```

Contoh Praktis 3: Analisis Saham

Dalam contoh sistem penasihat keuangan ini, beberapa agent berkolaborasi untuk mengumpulkan dan menganalisis data pasar, melakukan analisis keuangan, dan mengambil update berita berdasarkan query pengguna. Market Data Agent berfokus pada pengambilan harga saham real-time dan metrik keuangan terkait menggunakan library yfinance. Agent ini memberikan klien insight langsung mengenai kondisi pasar terkini untuk saham atau aset tertentu. Financial Analysis Agent bertanggung jawab untuk memproses data keuangan guna menghitung metrik seperti return on investment (ROI) dan tingkat pertumbuhan, sehingga bermanfaat bagi klien yang mencari insight evaluatif tentang investasi potensial atau yang sudah ada. Terakhir, Financial News Agent mengambil artikel dan update berita keuangan yang relevan, menawarkan insight ringkasan mengenai tren pasar, pengumuman perusahaan, dan perkembangan ekonomi.

Supervisor Agent berperan sebagai orkestrator, mendelegasikan query klien pertama-tama ke Market Data Agent jika permintaan terkait harga saham atau informasi pasar dasar. Jika diperlukan analisis yang lebih mendalam, seperti menghitung performa keuangan atau menemukan berita pasar yang relevan, supervisor mengarahkan query ke Analysis Agent atau News Agent sesuai kebutuhan. Pendekatan ini menyediakan workflow yang efisien dan kohesif di mana setiap agent menangani aspek tertentu dari query keuangan, pada akhirnya menciptakan respons yang komprehensif dan terstruktur yang dapat memenuhi berbagai kebutuhan klien.

```

#lesson13c.py

import os
from functools import partial
from typing import Annotated, Sequence, TypedDict, Literal

```

```

import yfinance as yf
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, BaseMessage
from pydantic import BaseModel
from langgraph.graph import END, START, StateGraph
from langgraph.prebuilt import create_react_agent
import functools
import operator

# LLM definition
llm = ChatOpenAI(model="gpt-4o-mini")

# Route Response structure for supervisor's decision
class RouteResponseFin(BaseModel):
    next: Literal["Market_Data_Agent", "Analysis_Agent", "News_Agent", "FINISH"]

# Define agent members
members_fin = ["Market_Data_Agent", "Analysis_Agent", "News_Agent"]

# Supervisor prompt setup
system_prompt_fin = (
    "You are a Financial Services Supervisor managing the following agents: "
    f"{'', '.join(members_fin)}. Select the next agent to handle the current query."
)

prompt_fin = ChatPromptTemplate.from_messages([
    ("system", system_prompt_fin),
    MessagesPlaceholder(variable_name="messages"),
    ("system", "Choose the next agent from: {options}."),
]).partial(options=str(members_fin))

# Supervisor Agent
def supervisor_agent_fin(state):
    supervisor_chain_fin =  prompt_fin
    llm.with_structured_output(RouteResponseFin)
    return supervisor_chain_fin.invoke(state)

# Define Tools and Agent Prompts

# 1. Market Data Tool and Agent Prompt
def fetch_stock_price(query):

```

```

        """Fetch the current stock price of a given stock symbol."""
        stock_symbol = query.split()[-1]
        stock = yf.Ticker(stock_symbol)
        try:
            current_price = stock.info.get("currentPrice")
            return f"The current stock price of {stock_symbol} is ${current_price}."
        except Exception as e:
            return f"Error retrieving stock data for {stock_symbol}: {str(e)}"

def agent_node(state, agent, name):
    result = agent.invoke(state)
    print(f"{name} Output: {result['messages'][-1].content}")
    return {
        "messages": [HumanMessage(content=result["messages"][-1].content,
name=name)]
    }

market_data_prompt = (
    "You are the Market Data Agent. Your role is to retrieve the latest stock prices or "
    "market information based on user queries. Ensure your response includes the "
    "current price "
    "and any relevant market details if available."
)
market_data_agent = create_react_agent(llm, tools=[fetch_stock_price],
state_modifier=market_data_prompt)
market_data_node = functools.partial(agent_node, agent=market_data_agent,
name="Market_Data_Agent")

# 2. Financial Analysis Tool and Agent Prompt
def perform_financial_analysis(query):
    """Perform financial analysis based on user query."""
    if "ROI" in query:
        initial_investment = 1000
        final_value = 1200
        roi = ((final_value - initial_investment) / initial_investment) * 100
        return f"For an initial investment of ${initial_investment} yielding "
        f"${final_value}, the ROI is {roi}%."
    return "No relevant financial analysis found."

analysis_prompt = (

```

```
    "You are the Financial Analysis Agent. Analyze the financial data provided in the query. "
```

```
    "Perform calculations like ROI, growth rates, or other financial metrics as required. "
```

```
    "Provide a clear and concise response."
```

```
    "Only use the following tools:"
```

```
    "perform_financial_analysis"
```

```
)
```

```
analysis_agent = create_react_agent(llm, tools=[perform_financial_analysis], state_modifier=analysis_prompt)
```

```
analysis_node = functools.partial(agent_node, agent=analysis_agent, name="Analysis_Agent")
```

```
# 3. Financial News Tool and Agent Prompt
```

```
financial_news_tool = TavilySearchResults(max_results=5)
```

```
news_prompt = (
```

```
    "You are the Financial News Agent. Retrieve the latest financial news articles relevant to the user's query. "
```

```
    "Use search tools to gather up-to-date news information and summarize key points."
```

```
    "Do not quote sources, just give a summary."
```

```
)
```

```
financial_news_agent = create_react_agent(llm, tools=[financial_news_tool], state_modifier=news_prompt)
```

```
financial_news_node = functools.partial(agent_node, agent=financial_news_agent, name="News_Agent")
```

```
# Define Workflow State
```

```
class AgentState(TypedDict):
```

```
    messages: Annotated[Sequence[BaseMessage], operator.add]
```

```
    next: str
```

```
# Define the workflow with the supervisor and agent nodes
```

```
workflow_fin = StateGraph(AgentState)
```

```
workflow_fin.add_node("Market_Data_Agent", market_data_node)
```

```
workflow_fin.add_node("Analysis_Agent", analysis_node)
```

```
workflow_fin.add_node("News_Agent", financial_news_node)
```

```
workflow_fin.add_node("supervisor", supervisor_agent_fin)
```

```
# Define edges for agents to return to the supervisor
```



```

for member in members_fin:
    workflow_fin.add_edge(member, "supervisor")

# Conditional map for routing based on supervisor's decision
conditional_map_fin = {
    "Market_Data_Agent": "Market_Data_Agent",
    "Analysis_Agent": "Analysis_Agent",
    "News_Agent": "News_Agent",
    "FINISH": END # This will end the workflow when supervisor decides
}

workflow_fin.add_conditional_edges("supervisor", lambda x: x["next"],
conditional_map_fin)
workflow_fin.add_edge(START, "supervisor")

# Compile the workflow
graph_fin = workflow_fin.compile()

# Testing the workflow with an example input
inputs_fin = {"messages": [HumanMessage(content="What is the stock price of AAPL?")]}

for output in graph_fin.stream(inputs_fin):
    if "__end__" not in output:
        print(output)

```

Contoh Praktis 4: Manajemen Portofolio

Dalam pengaturan penasihat keuangan ini, berbagai agent mengelola aspek yang berbeda dari portofolio keuangan klien:

- **Portfolio Analysis Agent: Meninjau dan menilai portofolio investasi klien saat ini.**
- **Market Research Agent: Menganalisis tren pasar dan peluang investasi.**
- **Risk Assessment Agent: Mengevaluasi risiko potensial yang terkait dengan berbagai opsi investasi.**

Supervisor agent memulai analisis dengan menugaskan tugas kepada Portfolio Analysis Agent. Berdasarkan hasilnya, agent ini dapat mengarahkan Market Research Agent untuk mencari peluang investasi yang sesuai dengan tujuan klien. Jika diperlukan, Risk Assessment Agent mengevaluasi opsi-opsi tersebut sebelum sistem menyajikan temuannya kepada klien.

```

#lesson13d.py

import os
import finnhub
from functools import partial
from typing import Annotated, Sequence, TypedDict, Literal
from langchain_core.messages import HumanMessage, BaseMessage
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, END, START
from pydantic import BaseModel
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langgraph.prebuilt import create_react_agent
import operator

# Initialize the Finnhub client with your API key
finnhub_client = finnhub.Client(api_key=os.getenv("FINNHUB_API_KEY"))

# Set up LLM
llm = ChatOpenAI(model="gpt-4o-mini")

# Define Route Response structure for supervisor's decision
class RouteResponseFin(BaseModel):
    next: Literal["Portfolio_Analysis_Agent", "Market_Research_Agent",
                  "Risk_Assessment_Agent", "FINISH"]

# Supervisor prompt
system_prompt_fin = (
    "You are a Financial Supervisor managing the following agents: Portfolio Analysis, "
    "Market Research, and Risk Assessment."
    " Select the next agent based on the user's query."
)

prompt_fin = ChatPromptTemplate.from_messages([
    ("system", system_prompt_fin),
    MessagesPlaceholder(variable_name="messages"),
    ("system", "Choose the next agent from: {options}.")
]).partial(options="['Portfolio_Analysis_Agent', 'Market_Research_Agent',
                    'Risk_Assessment_Agent', 'FINISH']")

# Supervisor Agent Function
def supervisor_agent_fin(state):

```

```

        supervisor_chain_fin = prompt_fin |
llm.with_structured_output(RouteResponseFin)
        return supervisor_chain_fin.invoke(state)

# Define Agent Node Function
def agent_node(state, agent, name):
    result = agent.invoke(state)
    return {"messages": [HumanMessage(content=result["messages"][-1].content,
name=name)]}

# Portfolio Analysis Agent
def portfolio_analysis(query):
    """Fetch basic financial metrics using Finnhub API"""
    try:
        stock_symbol = query.split()[-1]
        financials = finnhub_client.company_basic_financials(stock_symbol, 'all')
        metrics = financials.get('metric', {})
        response = (
            f"Portfolio Analysis for {stock_symbol}: P/E Ratio: {metrics.get('peRatio')}, "
            f"Revenue Growth: {metrics.get('revenueGrowth')}, "
            f"52-Week High: {metrics.get('52WeekHigh')}, 52-Week Low: {metrics.get('52WeekLow')}"
        )
        return response
    except Exception as e:
        return f"Error in fetching portfolio data: {str(e)}"

portfolio_agent = create_react_agent(llm, tools=[portfolio_analysis],
state_modifier="Portfolio Analysis Agent")
portfolio_analysis_node = partial(agent_node, agent=portfolio_agent,
name="Portfolio_Analysis_Agent")

# Market Research Agent
def market_research(query):
    """ Get latest market news and sentiment """
    news = finnhub_client.general_news("general")
    top_news = news[:3] # Retrieve top 3 news items for brevity
    response = "Latest Market News:\n" + "\n".join(
        f"{item['headline']} - {item['source']} ({item['url']})" for item in top_news
    )

```

```

    )
    return response

market_research_agent = create_react_agent(llm, tools=[market_research],
state_modifier="Market Research Agent")
market_research_node = partial(agent_node, agent=market_research_agent,
name="Market_Research_Agent")

# Risk Assessment Agent
def risk_assessment(query):
    """ Perform basic risk evaluation using volatility metrics (example)"""
    try:
        stock_symbol = query.split()[-1]
        quote = finnhub_client.quote(stock_symbol)
        price_change = quote.get("dp", 0)
        risk_level = "High Risk" if abs(price_change) > 5 else "Moderate Risk" if
abs(price_change) > 2 else "Low Risk"
        return f"Risk Assessment for {stock_symbol}: Price Change:
{price_change}%, Risk Level: {risk_level}"
    except Exception as e:
        return f"Error in assessing risk: {str(e)}"

risk_assessment_agent = create_react_agent(llm, tools=[risk_assessment],
state_modifier="Risk Assessment Agent")
risk_assessment_node = partial(agent_node, agent=risk_assessment_agent,
name="Risk_Assessment_Agent")

# Define Workflow State
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
    next: str

# Set up the Workflow
workflow_fin = StateGraph(AgentState)
workflow_fin.add_node("Portfolio_Analysis_Agent", portfolio_analysis_node)
workflow_fin.add_node("Market_Research_Agent", market_research_node)
workflow_fin.add_node("Risk_Assessment_Agent", risk_assessment_node)
workflow_fin.add_node("supervisor", supervisor_agent_fin)

# Define routing for agents to return to the supervisor

```

```

for member in ["Portfolio_Analysis_Agent", "Market_Research_Agent",
               "Risk_Assessment_Agent"]:
    workflow_fin.add_edge(member, "supervisor")

# Define the supervisor's routing decisions
conditional_map_fin = {
    "Portfolio_Analysis_Agent": "Portfolio_Analysis_Agent",
    "Market_Research_Agent": "Market_Research_Agent",
    "Risk_Assessment_Agent": "Risk_Assessment_Agent",
    "FINISH": END
}

workflow_fin.add_conditional_edges("supervisor", lambda x: x["next"],
conditional_map_fin)
workflow_fin.add_edge(START, "supervisor")

# Compile the workflow
graph_fin = workflow_fin.compile()

# Test with Example Query
inputs_fin = {"messages": [HumanMessage(content="Analyze the portfolio for AAPL.")]}

for output in graph_fin.stream(inputs_fin, stream_mode="values"):
    print(output)

```

Pengaturan penasihat keuangan yang dijelaskan di sini sangat cocok untuk sistem multi-agent, karena mencakup beberapa tugas keuangan yang kompleks namun saling melengkapi. Setiap agent memiliki tujuan yang berbeda: Portfolio Analysis Agent menilai portofolio klien, Market Research Agent menjelajahi tren pasar, dan Risk Assessment Agent mengevaluasi risiko potensial yang terkait dengan keputusan investasi. Dengan supervisor agent yang terkoordinasi dengan baik, sistem ini dapat secara dinamis beradaptasi terhadap perubahan strategi keuangan berdasarkan kebutuhan klien tertentu. Dengan menggunakan pendekatan modular, agent-agent ini dapat berkolaborasi dalam suatu urutan yang dikendalikan oleh supervisor agent, sehingga efisien dalam memenuhi kebutuhan klien secara real-time.

Implementasi sistem multi-agent ini menggunakan Finnhub API akan memberikan nilai tinggi bagi sistem yang siap produksi (production-ready). API dari Finnhub menyediakan data untuk harga saham, fundamental perusahaan, dan insight ekonomi, sehingga ideal untuk agent-agent ini. Sebagai contoh, Portfolio Analysis Agent dapat

memanfaatkan endpoint Finnhub untuk financials dasar atau metrik, seperti rasio P/E dan pertumbuhan revenue, untuk menganalisis performa portofolio dan mengidentifikasi kesenjangan. Demikian pula, Market Research Agent dapat mengakses berita pasar dan analisis sentimen dari Finnhub untuk mengidentifikasi area pertumbuhan, sementara Risk Assessment Agent dapat menggunakan data volatilitas dan metrik ekonomi untuk mengevaluasi faktor risiko.

Chapter 14

Hierarchical Agent Teams

Bagian 14.1: Membangun Hierarchical Agent Teams

Pada bagian ini, kita akan melihat lebih dekat bagaimana merancang dan mengimplementasikan Hierarchical Agent Teams di dalam LangGraph. Pendekatan ini memperkenalkan sebuah Supervisor agent yang mengawasi beberapa worker agent khusus, secara efektif mendistribusikan tugas-tugas kompleks di antara sebuah tim. Pengaturan hierarkis ini tidak hanya meningkatkan fleksibilitas tetapi juga menyediakan struktur yang terorganisir untuk menangani berbagai subtask yang mungkin memerlukan keterampilan atau tools yang unik.

Tujuan untuk Bagian Ini:

- Memahami peran Supervisor dan Worker agent dalam sebuah tim hierarkis.- Mempelajari cara mengimplementasikan struktur agent multi-level menggunakan LangGraph.- Mengeksplorasi aplikasi dunia nyata untuk Hierarchical Agent Teams dalam konteks bisnis.

Pengantar Hierarchical Agent Teams

Pada contoh sebelumnya mengenai Single Supervisor Model, kita memperkenalkan gagasan sebuah supervisor agent yang merutekan tugas ke berbagai worker agent. Namun, seiring bertambahnya kompleksitas tugas, satu supervisor tunggal dapat menjadi bottleneck. Arsitektur Hierarchical Agent Team membantu mengelola kompleksitas dengan membagi tanggung jawab di beberapa level agent, masing-masing dengan area fokus tertentu.

Mengapa Menggunakan Struktur Hierarkis?

Untuk aplikasi tertentu, struktur flat dengan satu supervisor yang mendelegasikan semua tugas dapat menjadi tidak efisien, terutama ketika tugas memerlukan pengetahuan atau tools khusus. Dalam struktur hierarkis:

- Supervisor agent menangani perutean dan koordinasi tugas tingkat tinggi.- Tim tingkat menengah bertanggung jawab atas domain atau subtask tertentu, memungkinkan mereka beroperasi secara independen dalam area keahliannya.

- **Worker agent khusus dapat berfokus pada tugas individual, seperti data scraping, pembuatan dokumen, atau visualisasi data, membuat sistem menjadi modular dan scalable.**

Desain hierarkis memungkinkan adaptabilitas dan performa yang lebih besar, terutama pada proyek yang memerlukan pengumpulan data, pemrosesan, dan pelaporan secara ekstensif.

Contoh Arsitektur Hierarchical Agent Team

Mari kita eksplorasi sebuah Hierarchical Agent Team yang terstruktur sebagai berikut:

1. **Overall Supervisor Agent:** The main controller that manages task distribution and ensures smooth operation.
2. **Research Team:** Responsible for gathering, verifying, and processing information from online sources.
3. **Document Authoring Team:** Handles tasks related to content creation, structuring, and visual representation of data.

Setiap tim terdiri dari beberapa agent, masing-masing diberi fungsi tertentu. Supervisor agent secara dinamis merutekan tugas antara tim Research dan Document Authoring, membagi lebih jauh tanggung jawab di antara agent-agent khusus.

Merancang Tim Menggunakan Supervisor dan Worker Agent

Mari kita mulai dengan merancang setiap tim secara individual, mengujinya secara terisolasi, lalu menggabungkan tim-tim tersebut di bawah sebuah Supervisor agent secara keseluruhan.

Membangun dan Menguji Setiap Tim

Agar contoh kita praktis, mari kita simulasikan sebuah **Financial Audit Team**. Tim ini bertanggung jawab untuk mengumpulkan data keuangan, melakukan analisis, dan menghasilkan laporan audit yang terstruktur. Kita akan memecah proses ini menjadi tiga komponen utama:

1. **Research Team** – Collects and verifies financial data.
2. **Document Authoring Team** – Analyzes data and generates the audit report.
3. **Overall Supervisor Agent** – Oversees and coordinates the entire audit workflow.

Langkah 1: Membangun Research Team

Research Team akan memiliki dua agent utama:

- **Data Collector Agent: Mengambil data keuangan dari database atau sumber online.**
- **Transaction Verifier Agent: Memeriksa setiap transaksi untuk akurasi guna memastikan integritas data.**

Setiap agent memerlukan tools tertentu untuk menjalankan tugasnya. Mari kita siapkan tools tersebut dan definisikan struktur Research Team.

Mendefinisikan Tools untuk Research Team

Data Collection Tool: Tool ini terhubung ke database keuangan perusahaan atau sebuah API online untuk mengambil data mentah.

Transaction Verification Tool: Tool ini memeriksa transaksi yang diambil terhadap ledger internal atau sumber eksternal yang terpercaya untuk mencegah kesalahan dan fraud.

```
#lesson14a.py

@tool
def verify_transactions(transaction_ids: list) -> list:
    """Verify each transaction ID for accuracy."""
    # Simulating verification process
    verified_transactions = [tx_id for tx_id in transaction_ids if tx_id.is_valid()]
    return verified_transactions
```

Mendefinisikan Graph untuk Research Team

Workflow Research Team direpresentasikan sebagai sebuah StateGraph di LangGraph. Supervisor agent di dalam tim ini mengelola perutean tugas antara agent Data Collector dan Transaction Verifier.

```
#lesson14a.py

from langgraph.graph import StateGraph

# Define the Research Team Graph
research_team_graph = StateGraph()
research_team_graph.add_node("DataCollector", data_collector_node)
research_team_graph.add_node("TransactionVerifier", transaction_verifier_node)
```

```
# Define edges to route tasks between agents and back to the team supervisor
research_team_graph.add_edge("DataCollector", "TransactionVerifier")
research_team_graph.add_edge("TransactionVerifier", "ResearchTeamSupervisor")
```

Menguji Research Team

Setelah graph Research Team didefinisikan, kita dapat mengujinya untuk memastikan proses pengumpulan dan verifikasi data berjalan dengan benar. Mulailah dengan memberikan sebuah task ke node DataCollector dan pastikan task tersebut merutekan data ke TransactionVerifier.

```
#lesson14a.py
# Initial task to start the research team
initial_state = {"company_id": "12345", "period": "Q4"}
research_results = research_team_graph.invoke(initial_state)
print(research_results)
```

Langkah 2: Membangun Document Authoring Team

Document Authoring Team berfokus pada menganalisis data yang telah diverifikasi dan menghasilkan laporan audit yang terstruktur. Tim ini akan memiliki tiga agent utama:

- **Financial Analyst Agent: Menganalisis data untuk rasio keuangan dan penilaian risiko.**
- **Report Writer Agent: Menyusun draf laporan audit utama, meringkas temuan-temuan.**
- **Chart Generator Agent: Membuat visualisasi untuk representasi data.**

Mendefinisikan Tools untuk Document Authoring Team

Financial Analysis Tool: Tool ini menghitung metrik keuangan utama seperti rasio debt-to-equity, current ratio, dll., berdasarkan data yang dikumpulkan.

```
#lesson14a.py
@tool
def perform_financial_analysis(data: dict) -> dict:
    """Calculate financial metrics from the provided data."""
    results = {
```

```

        "debt_to_equity_ratio": data["total_debt"] / data["total_equity"],
        "current_ratio": data["current_assets"] / data["current_liabilities"],
    }
    return results

```

Report Writing Tool: Tool ini menyimpan hasil analisis dalam format laporan yang terstruktur.

```

#lesson14a.py

from pathlib import Path

@tool
def write_report(content: str, file_name: str) -> str:
    """Write the audit report to a text file."""
    report_path = Path(f"/reports/{file_name}")
    report_path.write_text(content)
    return f"Report saved at {report_path}"

```

Chart Generation Tool: Tool ini menggunakan library matplotlib untuk membuat bar chart dan line chart dari data keuangan.

```

#lesson14a.py

from langchain_experimental.utilities import PythonREPL
repl = PythonREPL()

@tool
def generate_chart(data: dict, chart_type: str) -> str:
    """Generate a chart for financial data."""
    code = f"""
import matplotlib.pyplot as plt
data = {data}
plt.figure(figsize=(10, 6))
if "{chart_type}" == "bar":
    plt.bar(data.keys(), data.values())
elif "{chart_type}" == "line":
    plt.plot(data.keys(), data.values())
plt.title('Financial Metrics')
plt.savefig('/charts/financial_chart.png')
"""
    repl.run(code)

```

```
return "Chart generated and saved at /charts/financial_chart.png"
```

Mendefinisikan Graph untuk Document Authoring Team

Workflow Document Authoring Team direpresentasikan sebagai sebuah StateGraph lain, menghubungkan agent Financial Analyst, Report Writer, dan Chart Generator.

```
#lesson14a.py
# Define the Document Authoring Team Graph
document_authoring_graph = StateGraph()
document_authoring_graph.add_node("FinancialAnalyst", financial_analyst_node)
document_authoring_graph.add_node("ReportWriter", report_writer_node)
document_authoring_graph.add_node("ChartGenerator", chart_generator_node)
# Define edges to route tasks within the document authoring team
document_authoring_graph.add_edge("FinancialAnalyst", "ReportWriter")
document_authoring_graph.add_edge("ReportWriter", "ChartGenerator")
document_authoring_graph.add_edge("ChartGenerator",
"DocumentAuthoringSupervisor")
```

Menguji Document Authoring Team

Setelah graph Document Authoring Team didefinisikan, uji workflow tersebut dengan menginisialisasinya menggunakan data dari Research Team.

```
#lesson14a.py
# Sample input data for testing the document authoring team
sample_data = {
    "total_debt": 100000,
    "total_equity": 50000,
    "current_assets": 80000,
    "current_liabilities": 40000
}

authoring_results = document_authoring_graph.invoke(sample_data)
print(authoring_results)
```

Langkah 3: Mengintegrasikan Tim dengan Overall Supervisor

Setelah masing-masing tim diuji, langkah terakhir adalah mengintegrasikan Research Team dan Document Authoring Team di bawah sebuah Overall Supervisor Agent. Agent ini akan mengelola delegasi tugas di antara kedua tim.

Mendefinisikan Graph untuk Overall Supervisor

Overall Supervisor akan mengelola perutean antara Research Team dan Document Authoring Team, berdasarkan workflow audit.

```
#lesson14a.py
# Define the Overall Supervisor Node
overall_supervisor = create_team_supervisor(
    llm,
    "Manage financial audit workflow between data collection and report generation teams.",
    ["ResearchTeam", "DocumentAuthoringTeam"]
)
# Define the top-level graph for the Financial Audit Team
financial_audit_graph = StateGraph()
financial_audit_graph.add_node("ResearchTeam", research_team_graph)
financial_audit_graph.add_node("DocumentAuthoringTeam", document_authoring_graph)
financial_audit_graph.add_node("OverallSupervisor", overall_supervisor)
# Define routing through the Overall Supervisor
financial_audit_graph.add_conditional_edges("OverallSupervisor", lambda x: x["next"],
{
    "ResearchTeam": "ResearchTeam",
    "DocumentAuthoringTeam": "DocumentAuthoringTeam",
    "FINISH": END
})
financial_audit_graph.add_edge(START, "OverallSupervisor")
```

Menjalankan Financial Audit dengan Hierarchical Team

Sekarang, kita dapat memulai workflow audit dengan mengirimkan permintaan task ke Overall Supervisor. Supervisor akan mengoordinasikan tugas antara Research Team dan Document Authoring Team untuk menyelesaikan audit.

```
#lesson14a.py  
# Start the audit  
audit_results = financial_audit_graph.invoke({"company_id": "12345", "period": "Q4"})  
print(audit_results)
```

Ringkasan

Pada bagian ini, kita membangun sebuah Financial Audit Team menggunakan struktur Hierarchical Agent Teams dari LangGraph. Dengan membangun dan menguji setiap tim secara individual, kita memastikan setiap bagian dari workflow beroperasi dengan lancar sebelum menggabungkannya di bawah sebuah Overall Supervisor. Pendekatan modular ini membuat sistem menjadi scalable, fleksibel, dan robust untuk workflow yang kompleks.

Pada bab selanjutnya, kita akan mengeksplorasi arsitektur agent yang lebih canggih, termasuk adaptive routing dan self-healing agent, untuk menciptakan sistem yang lebih resilien dan otonom

Chapter 15

Arsitektur Agent Khusus

Seiring AI terus merambah berbagai industri, arsitektur agent khusus telah muncul untuk mengatasi tantangan yang unik. Arsitektur-arsitektur ini meningkatkan kemampuan agent untuk merespons secara real-time, mengemulasi kognisi manusia, memanfaatkan model prediktif, menghasilkan code secara otonom, dan berkolaborasi dengan developer dalam lingkungan coding. LangGraph menyediakan framework yang fleksibel untuk mengimplementasikan arsitektur khusus ini, memungkinkan integrasi fungsionalitas unik dari setiap struktur.

15.1 Event-Driven Agent

Event-driven agent dirancang untuk merespons secara dinamis terhadap peristiwa dunia nyata. Agent-agent ini tetap idle hingga sebuah peristiwa atau trigger tertentu mengaktifkannya, memungkinkan mereka mengelola tugas sesuai permintaan secara efisien. Agent ini umum digunakan dalam domain seperti customer support, pemantauan sistem, IoT, dan pasar keuangan.

Konsep Utama

- **Merespons Trigger:** Trigger adalah kondisi yang, ketika terpenuhi, memulai aksi sebuah agent. Trigger dapat berupa input sensor, query pelanggan, atau perubahan pada data.- **Pemrosesan Real-Time:** Event-driven agent memerlukan delay minimal dalam responsnya. Dengan memanfaatkan kemampuan streaming dari LangGraph,

event-driven agents can process and respond to multiple events concurrently.

Contoh Implementasi

Mari kita pertimbangkan sebuah event-driven agent yang dirancang untuk memantau harga saham dan secara otomatis memicu aksi buy atau sell berdasarkan threshold yang telah ditetapkan sebelumnya.

```
#lesson15a.py

from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class StockState(TypedDict):
    stock_price: float
    action: str

def monitor_stock(state: StockState):
    if state['stock_price'] > 150:
        state['action'] = 'Sell'
    elif state['stock_price'] < 100:
        state['action'] = 'Buy'
    else:
        state['action'] = 'Hold'
    return state

# Define the workflow
builder = StateGraph(StockState)
builder.add_node("monitor_stock", monitor_stock)
builder.add_edge(START, "monitor_stock")
builder.add_edge("monitor_stock", END)
graph = builder.compile()

# Sample invocation with a stock price trigger
result = graph.invoke({"stock_price": 180, "action": ""})
print(result) # Output: {'stock_price': 120, 'action': 'Hold'}
```

15.2 Arsitektur Kognitif

Arsitektur kognitif dirancang untuk menyerupai cara manusia memproses informasi. Arsitektur ini memungkinkan AI agent untuk terlibat dalam penalaran, pembuatan keputusan-

membuat keputusan, dan perencanaan kompleks. Tipe arsitektur ini sangat cocok untuk situasi yang membutuhkan pemecahan masalah dinamis dan adaptabilitas, yang sering ditemukan dalam sistem pendukung keputusan atau aplikasi layanan pelanggan tingkat lanjut.

Konsep Utama

- **Meniru Proses Berpikir Manusia: Arsitektur kognitif disusun untuk memungkinkan reasoning, yang penting untuk tugas-tugas yang membutuhkan pemahaman, perencanaan, dan pemecahan masalah.- Kemampuan Reasoning Tingkat Lanjut: Dengan menggabungkan reasoning dengan memory, agent kognitif dapat menganalisis konteks, mengingat interaksi sebelumnya, dan membuat keputusan yang berdasar informasi.**

Contoh Implementasi

Berikut adalah contoh agent kognitif yang dirancang untuk membantu troubleshooting masalah software dengan melakukan reasoning terhadap kemungkinan penyebab berdasarkan gejala yang diamati.

```
#lesson15b.py

from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class DiagnosticState(TypedDict):
    symptoms: str
    diagnosis: str

def analyze_symptoms(state: DiagnosticState):
    if "slow" in state['symptoms']:
        state['diagnosis'] = "Possible network issue."
    elif "error" in state['symptoms']:
        state['diagnosis'] = "Check application logs for errors."
    else:
        state['diagnosis'] = "Further investigation needed."
    return state

# Define the workflow
builder = StateGraph(DiagnosticState)
builder.add_node("analyze_symptoms", analyze_symptoms)
builder.add_edge(START, "analyze_symptoms")
builder.add_edge("analyze_symptoms", END)
```

```
graph = builder.compile()

# Sample invocation
result = graph.invoke({"symptoms": "application slow response", "diagnosis": ""})
print(result) # Output: {'symptoms': 'application slow response', 'diagnosis': 'Possible network issue.'}
```

15.3 Model-Based Agents

Model-based agent mempertahankan representasi internal dari environment mereka, memungkinkan mereka untuk mensimulasikan dan memprediksi hasil dari berbagai tindakan. Pendekatan prediktif ini sangat berguna dalam bidang seperti robotika, kendaraan otonom, dan pengambilan keputusan strategis secara real-time.

Konsep Utama

- **Representasi Internal Environment: Model-based agent mensimulasikan aspek-aspek dari environment mereka, memungkinkan mereka memahami bagaimana berbagai tindakan memengaruhi hasil.-**
Pengambilan Keputusan Prediktif: Dengan mengevaluasi berbagai tindakan potensial dan kemungkinan hasilnya, agent ini dapat membuat keputusan strategis yang berdasar informasi.

Contoh Implementasi

Contoh berikut mendemonstrasikan model-based agent yang memprediksi apakah sebuah kendaraan harus melambat berdasarkan kepadatan lalu lintas dalam environment simulasi.

```
#lesson15c.py

from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class TrafficState(TypedDict):
    traffic_density: int
    action: str

def traffic_prediction(state: TrafficState):
    if state['traffic_density'] > 70:
        state['action'] = 'Slow down'
```

```

elif state['traffic_density'] > 40:
    state['action'] = 'Maintain speed'
else:
    state['action'] = 'Speed up'
return state

# Define the workflow
builder = StateGraph(TrafficState)
builder.add_node("traffic_prediction", traffic_prediction)
builder.add_edge(START, "traffic_prediction")
builder.add_edge("traffic_prediction", END)
graph = builder.compile()

# Sample invocation
result = graph.invoke({"traffic_density": 65, "action": ""})
print(result) # Output: {'traffic_density': 65, 'action': 'Maintain speed'}

```

15.4 AI-Powered Code Generation Agents

Dengan kemampuan untuk menghasilkan kode secara dinamis, AI-powered code generation agent membantu developer dengan menangani tugas-tugas repetitif, menyusun draf kode awal, dan meningkatkan kode yang sudah ada. Agent ini sangat berharga dalam rapid prototyping, debugging, dan otomatisasi refactoring kode.

Konsep Utama

- **Memanfaatkan LLM untuk Tugas Coding:** Dengan menggunakan LLM, agent dapat menghasilkan cuplikan kode, algoritma, atau bahkan seluruh function berdasarkan deskripsi atau prompt.
- **Mengintegrasikan Kode yang Dihasilkan:** Kode yang dihasilkan sering membutuhkan validasi untuk memastikan kebenaran dan kompatibilitas. Agent ini dapat menyertakan feedback loop yang memvalidasi, menguji, dan mengintegrasikan kode ke dalam proyek yang sudah ada.

Contoh Implementasi

Di sini, sebuah AI-powered agent menghasilkan kode Python untuk sebuah LangGraph AI agent.

```
#lesson15d.py
```

```

from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage, AIMessage
from pydantic import BaseModel

# Define the state with prompt and generated_code fields
class CodeGenState(TypedDict):
    prompt: str
    generated_code: str

# Initialize the ChatOpenAI tool for code generation
llm = ChatOpenAI(temperature=0, model_name="gpt-4o-mini")

class CodeGen(BaseModel):
    """Code to generate"""
    generated_code: str

# Define the code generation function using ChatOpenAI and LangChain
def generate_code(state: CodeGenState):
    # Create a HumanMessage with the prompt
    human_message = HumanMessage(content=state['prompt'])

    # Send the HumanMessage to ChatOpenAI for code generation
    llm_tool = llm.with_structured_output(CodeGen)
    response = llm_tool.invoke([human_message])

    # Extract the generated code from the AIMessage
    state['generated_code'] = response.generated_code
    return state

# Define the workflow
builder = StateGraph(CodeGenState)
builder.add_node("generate_code", generate_code)
builder.add_edge(START, "generate_code")
builder.add_edge("generate_code", END)
graph = builder.compile()

# Sample invocation with a natural language prompt
result = graph.invoke({"prompt": "Write Python code for a complete LangGraph AI Agent for code generation graph using langchain_openai.", "generated_code": ""})

```

```
print(result['generated_code'])
```

15.5 AI-Enhanced Pair Programming Tools

AI-enhanced pair programming agent mendukung lingkungan coding kolaboratif dengan menawarkan saran real-time, mendeteksi masalah, dan meningkatkan kualitas kode. Agent ini berinteraksi dengan developer selama sesi coding, meningkatkan produktivitas dan memungkinkan penyelesaian error yang lebih cepat.

Konsep Utama

- **Coding Kolaboratif dengan AI:** Agent ini dirancang untuk menawarkan **code completion**, menyarankan optimasi, dan memperingatkan potensi masalah, bekerja bersama developer sebagai asisten cerdas.-
Peningkatan Kode Real-Time: Melalui analisis berkelanjutan, agent ini dapat mendeteksi **syntax error**, **code smell**, atau konstruksi yang kurang optimal, memberikan **feedback instan** kepada developer.

Contoh Implementasi

Di bawah ini, sebuah agent menawarkan saran optimasi kode berdasarkan cuplikan kode yang diberikan.

```
#lesson15e.py

from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage
import json

# Define the state with code_snippet and suggestion fields
class PairProgrammingState(TypedDict):
    code_snippet: str
    suggestion: str

# Initialize the ChatOpenAI tool for code analysis
llm_tool = ChatOpenAI(temperature=0, model_name="gpt-4o-mini")
```

```

# Define the AI-driven code analysis function
def analyze_code(state: PairProgrammingState):
    # Create a prompt for structured suggestions
    structured_prompt = (
        f"Analyze the following Python code snippet and provide suggestions for improvement. "
        f"Respond in JSON format with a 'suggestion' field.\n\n"
        f"Code snippet:\n{state['code_snippet']}\n\n"
        "Response format:\n"
        "{\n"
        "  \"suggestion\": \"<insert suggestion here>\"\n"
        "}"
    )
    human_message = HumanMessage(content=structured_prompt)

    # Send the HumanMessage to ChatOpenAI for analysis
    ai_message = llm_tool.invoke([human_message])

    # Parse the JSON response for the suggestion
    try:
        structured_response = json.loads(ai_message.content)
        state['suggestion'] = structured_response.get("suggestion", "No specific suggestions.")
    except json.JSONDecodeError:
        state['suggestion'] = "Error: Unable to parse the generated response."

    return state

# Define the workflow
builder = StateGraph(PairProgrammingState)
builder.add_node("analyze_code", analyze_code)
builder.add_edge(START, "analyze_code")
builder.add_edge("analyze_code", END)
graph = builder.compile()

# Sample invocation with a code snippet
result = graph.invoke({"code_snippet": "for i in range(len(arr)): print(arr[i])",
"suggestion": ""})

```

```
print(result['suggestion']) # Expected output: A dynamic suggestion, such as using
enumerate()
```

Kesimpulan dan Pembelajaran Praktis

Dalam bab ini, Anda telah menjelajahi beberapa arsitektur agent khusus yang memperluas kemampuan LangGraph. Setiap tipe menawarkan keunggulan yang berbeda untuk pemrosesan real-time, pengambilan keputusan prediktif, dan dukungan kolaboratif, menjadikannya sangat berharga untuk berbagai aplikasi.

Latihan

1. **Create a Traffic Monitoring Agent:** Implement an event-driven agent that monitors vehicle speeds and triggers alerts if speeds exceed safe limits.
2. **Develop a Cognitive Task Planner:** Create a cognitive agent that plans a sequence of tasks based on given objectives, simulating human-like task management.
3. **Build a Pair Programming Debugger:** Extend the AI-enhanced pair programming agent to check for common Python errors, such as undefined variables or improper indentation.

Dengan mengerjakan latihan-latihan ini, Anda akan memperkuat pemahaman Anda tentang arsitektur agent khusus, mempersiapkan Anda untuk menangani tugas-tugas kompleks dan adaptif dalam skenario dunia nyata.

Part 5: Building and Deploying Advanced AI
Agents (Chapters 16-18)

Chapter 16

Testing AI Agents

Dalam bab ini, kita akan membahas semua yang perlu Anda ketahui tentang cara menguji AI agent secara efektif, mulai dari unit test dasar hingga integration test yang kompleks, serta cara menangani edge case. Tujuannya adalah memastikan AI agent Anda andal, robust, dan adaptif terhadap berbagai situasi. Baik Anda sedang membuat AI-powered chatbot, agent workflow otomatis, atau sistem multi-agent yang lebih kompleks, testing sangat penting untuk memastikan agent Anda bekerja sesuai harapan.

16.1 Pentingnya Testing dalam Sistem AI

Testing sangat penting untuk memvalidasi bahwa AI agent berfungsi dengan benar, memastikan mereka robust, dapat diprediksi, dan performan. Testing untuk AI agent berbeda dari testing software tradisional karena sistem AI dapat berperilaku tidak terduga akibat ketergantungannya pada learning, sumber data eksternal, dan proses pengambilan keputusan real-time.

Tantangan Unik dalam AI Testing

AI testing menghadirkan tantangan unik, seperti:

- **Perilaku Non-deterministik:** AI agent dapat menghasilkan output yang berbeda untuk input yang serupa karena randomness, pembaruan model, atau learning saat runtime.

- **Interaksi Kompleks:** Banyak AI agent berinteraksi dengan sistem eksternal (misalnya, API, database), dan perilakunya bergantung pada keberhasilan atau kegagalan interaksi tersebut.- **Learning Seiring Waktu:** AI agent yang belajar dari data atau feedback manusia dapat mengubah perilakunya seiring waktu, yang dapat menimbulkan tantangan baru dalam memastikan mereka terus beroperasi dengan benar.

Manfaat Testing yang Ketat

- **Reliabilitas:** Memastikan agent berkinerja konsisten dan menghasilkan output yang benar.- **Adaptabilitas:** Memastikan agent dapat menangani skenario baru yang belum terduga.- **Kepercayaan:** Memvalidasi fungsionalitas sistem AI, meningkatkan kepercayaan stakeholder.

16.2 Unit Testing untuk Agent

Unit testing berfokus pada pengujian komponen atau unit kode secara individual dan terisolasi. Untuk AI agent, ini berarti menguji function atau "node" individual untuk memastikan setiap bagian bekerja dengan benar sebelum diintegrasikan ke dalam sistem yang lebih besar.

Menulis Unit Test yang Efektif

Untuk menulis unit test yang efektif:

- **Uji Satu Function:** Setiap unit test harus menguji satu function, seperti tindakan individual yang dilakukan sebuah node dalam agent workflow.- **Gunakan Mock Data:** Alih-alih memanggil API eksternal, gunakan mock data untuk mensimulasikan input dan output yang diharapkan, membuat test menjadi predictable.- **Tentukan Hasil yang Diharapkan:** Tentukan dengan jelas output yang Anda harapkan untuk setiap test case.

Contoh Lengkap: Unit Test untuk Code Suggestion Agent

Mari kita menulis unit test untuk sebuah agent yang memberikan saran perbaikan kode.

Pertama, pastikan Anda memiliki testing framework yang terpasang. Jika Anda menggunakan Python, module unittest sudah terpasang secara default. Anda juga dapat menggunakan pytest untuk fitur yang lebih canggih.

```
terminal  
pip install pytest
```

Selanjutnya, buat sebuah agent function sederhana yang akan menganalisis cuplikan kode dan memberikan saran.

```
#my_agent_module.py:  
  
def analyze_code(state: dict) -> dict:  
    """  
    Analyzes the provided code snippet and returns a suggestion for improvement.  
    """  
    if "for i in range(len(" in state['code_snippet']:  
        state['suggestion'] = "Consider using enumerate() for better readability."  
    else:  
        state['suggestion'] = "Code snippet looks good."  
    return state
```

Sekarang, kita menulis unit test untuk agent ini.

```
#test_my_agent_module.py:  
  
import unittest  
from my_agent_module import analyze_code  
  
class TestAnalyzeCodeNode(unittest.TestCase):  
  
    def test_suggestion_with_for_loop(self):  
        # Test case where the agent should suggest using enumerate  
        state = {"code_snippet": "for i in range(len(arr)): print(arr[i])", "suggestion":  
        ""}  
        expected_suggestion = "Consider using enumerate() for better readability."  
  
        result = analyze_code(state)  
        self.assertEqual(result['suggestion'], expected_suggestion)  
  
    def test_suggestion_no_improvement_needed(self):  
        # Test case where no improvements are necessary  
        state = {"code_snippet": "print('Hello, World!)", "suggestion": ""}
```

```
expected_suggestion = "Code snippet looks good."

result = analyze_code(state)
self.assertEqual(result['suggestion'], expected_suggestion)

if __name__ == '__main__':
    unittest.main()
```

Cara Menjalankan Test

Untuk menjalankan test:

1. Save the above files.
2. Open your terminal and navigate to the folder containing the test file.

Jalankan test dengan:

```
terminal
python -m unittest test_my_agent_module.py
```

Anda akan melihat output yang mirip dengan:

```
terminal
-----
Ran 2 tests in 0.001s
OK
```

Ini mengonfirmasi bahwa kedua test berhasil dijalankan.

16.3 Integration Testing

Integration testing memastikan bahwa berbagai bagian sistem AI (atau agent yang berbeda dalam sistem multi-agent) bekerja bersama sesuai harapan. Integration testing melampaui pengujian node individual dan memeriksa bagaimana data mengalir melalui sistem.

Contoh: Menguji Agent Workflow (Integration Test)

Misalkan Anda memiliki multi-node agent yang menghasilkan kode dan menganalisisnya. Workflow tersebut melibatkan:

1. **Generate code** (e.g., using AI).
2. **Analyze the code** and provide suggestions.

Berikut cara menguji workflow lengkap ini:

my_agent_module.py (dengan workflow generation dan analysis sederhana):

```
#my_agent_module.py

def generate_code(state: dict) -> dict:
    """
    Generates a Python function based on the prompt.
    """
    if "factorial" in state['prompt']:
        state['generated_code'] = "def factorial(n): return 1 if n == 0 else n * factorial(n-1)"
    else:
        state['generated_code'] = "Unknown function requested."
    return state

def analyze_code(state: dict) -> dict:
    """
    Analyzes the generated code for improvements.
    """
    if "for i in range(len(" in state['generated_code']:
        state['suggestion'] = "Consider using enumerate() for better readability."
    else:
        state['suggestion'] = "Code looks good."
    return state

test_my_agent_integration.py (integration test):
import unittest
from unittest.mock import patch
from my_agent_module import generate_code, analyze_code

class TestAgentIntegration(unittest.TestCase):

    @patch('my_agent_module.generate_code')
    def test_generate_and_analyze_code(self, mock_generate):
        # Test the full workflow of generating and analyzing code

        # Mock the generate_code function's output
```

```
mock_generate.return_value = {"generated_code": "for i in range(len(arr)):  
print(arr[i])", "prompt": ""}  
  
# Step 1: Generate code  
state = {"prompt": "Write a Python function to calculate factorial.",  
"generated_code": ""}  
generated_state = generate_code(state)  
  
# Step 2: Analyze generated code  
analysis_state = analyze_code(generated_state)  
  
# Verify the suggestion  
self.assertEqual(analysis_state['suggestion'], "Consider using enumerate() for  
better readability.")  
  
if __name__ == '__main__':  
    unittest.main()
```

16.4 Automated Testing Tools

Automated testing tools memungkinkan Anda menguji AI agent Anda secara berkelanjutan dan berulang, memastikan Anda dapat menangkap regresi atau bug yang muncul dalam kode baru. Tools seperti pytest, unittest, dan mocking library seperti unittest.mock sangat penting untuk menulis automated test yang efektif.

Menggunakan pytest

Pytest adalah testing tool yang lebih canggih dibandingkan unittest dan menyediakan dukungan yang lebih baik untuk hal-hal seperti fixture, parameterized test, dan pelaporan test.

Untuk menginstal pytest:

```
terminal  
pip install pytest
```

Kemudian, untuk menjalankan test, Anda dapat menggunakan:

```
terminal  
pytest test_my_agent_module.py
```

Pytest akan secara otomatis menemukan semua test function (yang diawali dengan test_) dan menjalankannya.

16.5 Menangani Edge Case dan Exception

Edge case merujuk pada kondisi langka atau ekstrem yang dapat menyebabkan AI agent gagal. Menguji edge case memastikan agent Anda berperilaku dengan benar dalam situasi yang tidak terduga.

Mengidentifikasi Potensi Kegagalan

Beberapa edge case untuk AI agent mungkin meliputi:

- **Input Kosong: Apa yang terjadi ketika agent diberikan tanpa data atau input kosong?**
- **Data yang Rusak: Bagaimana jika input rusak atau dalam format yang tidak terduga?**
- **Performa di Bawah Beban: Bagaimana kinerja agent dengan dataset besar atau permintaan berfrekuensi tinggi?**

Contoh: Menangani Edge Case (Cuplikan Kode)

```
#lesson16c.py

import unittest
from my_agent_module import analyze_code

class TestEdgeCases(unittest.TestCase):

    def test_empty_code_snippet(self):
        state = {"code_snippet": "", "suggestion": ""}
        result = analyze_code(state)
        self.assertEqual(result['suggestion'], "No code provided.")

    def test_large_code_snippet(self):
        state = {"code_snippet": "print('Hello World')\n" * 1000, "suggestion": ""}
        result = analyze_code(state)
        self.assertEqual(result['suggestion'], "Code looks good.")

    def test_malformed_data(self):
        state = {"code_snippet": None, "suggestion": ""}
        result = analyze_code(state)
        self.assertEqual(result['suggestion'], "Code snippet is invalid.")
```



```
if __name__ == '__main__':  
    unittest.main()
```

Kesimpulan dan Pembelajaran Praktis

Menguji AI agent sangat penting untuk memastikan reliabilitas dan robustness. Dengan menerapkan unit test, integration test, dan menangani edge case, Anda dapat secara signifikan meningkatkan kualitas agent Anda. Jangan lupa untuk menyertakan automated tools seperti pytest untuk testing yang mudah dan berkelanjutan.

16.6: Menguji AI Agent - Contoh Praktis dengan LangGraph ReAct Agent

Gambaran Umum Menguji ReAct Agent

Sebuah ReAct agent menggabungkan reasoning dengan tindakan dalam sebuah loop: ia mencari di web, melakukan reasoning tentang informasi mana yang relevan, dan menyusunnya menjadi dokumen Markdown. Pendekatan ini memungkinkan agent menangani tugas seperti riset web dan pembuatan laporan.

Dalam contoh ini, ReAct agent akan:

1. **Search the web** for information based on a user query.
2. **Extract and filter relevant data** from search results.
3. **Compile the data into a Markdown document**, structuring it with headings and bullet points.

Untuk agent ini, kita akan menggunakan ReAct pattern dari LangGraph, yang memungkinkan langkah-langkah reasoning (misalnya, menyaring informasi yang tidak relevan) dan langkah-langkah action (misalnya, menghasilkan Markdown).

Langkah 1: Mendefinisikan Agent Workflow

Kita akan mendefinisikan workflow agent, termasuk:

- **Pencarian web:** Mengambil informasi berdasarkan sebuah query.-
- Reasoning:** Menyaring dan memilih konten yang relevan.-
- Kompilasi Markdown:** Memformat dokumen akhir dalam Markdown.

Implementasi Kode Agent

Mari kita mulai dengan menulis kode agent di `react_agent_module.py`.

```
react_agent_module.py

from typing import TypedDict, List
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage

# Define the state with query and markdown_result fields
class ReActAgentState(TypedDict):
    query: str
    search_results: List[str]
    markdown_result: str

# Initialize the ChatOpenAI tool for reasoning and web search
llm_tool = ChatOpenAI(temperature=0, model_name="gpt-3.5-turbo")

# Node 1: Perform a web search based on a query
def perform_search(state: ReActAgentState) -> ReActAgentState:
    # Prompt for web search
    search_prompt = f"Search the web for information about: {state['query']}"
    search_message = HumanMessage(content=search_prompt)

    # Simulate search (replace with actual search API in production)
    search_response = llm_tool.invoke([search_message])
    state['search_results'] = search_response.content.split("\n") # Simulate results as list
    return state

# Node 2: Filter relevant information
def filter_results(state: ReActAgentState) -> ReActAgentState:
    # Filtering relevant information based on prompt
    filter_prompt = "Select the most relevant information from the following list:\n" +
        "\n".join(state['search_results'])
    filter_message = HumanMessage(content=filter_prompt)

    # Reasoning step to filter results
    filtered_response = llm_tool.invoke([filter_message])
    state['search_results'] = filtered_response.content.split("\n")
    return state
```

```

# Node 3: Compile Markdown document
def compile_markdown(state: ReActAgentState) -> ReActAgentState:
    # Compilation step to create a Markdown document
    compile_prompt = "Compile the following information into a Markdown
document:\n" + "\n".join(state['search_results'])
    compile_message = HumanMessage(content=compile_prompt)

    # Generate the Markdown document
    markdown_response = llm_tool.invoke([compile_message])
    state['markdown_result'] = markdown_response.content
    return state

# Define the workflow
builder = StateGraph(ReActAgentState)
builder.add_node("perform_search", perform_search)
builder.add_node("filter_results", filter_results)
builder.add_node("compile_markdown", compile_markdown)
builder.add_edge(START, "perform_search")
builder.add_edge("perform_search", "filter_results")
builder.add_edge("filter_results", "compile_markdown")
builder.add_edge("compile_markdown", END)
graph = builder.compile()

# Sample invocation with a query
initial_state = {"query": "Python programming best practices", "search_results": [],
"markdown_result": ""}
result = graph.invoke(initial_state)
print(result['markdown_result']) # Print the compiled Markdown document

```

Langkah 2: Menulis Test untuk ReAct Agent

Sekarang setelah kita memiliki ReAct agent yang berfungsi, kita perlu menguji setiap bagian fungsionalitasnya. Kita akan mulai dengan unit test untuk setiap node dalam agent (search, filter, dan compile), lalu menulis integration test untuk keseluruhan workflow.

1. **Unit Tests:** These will test individual nodes in isolation.

2. **Integration Test:** This will test the complete workflow, ensuring all nodes work together seamlessly.

Menyiapkan Unit Test untuk Setiap Node

Di sini, kita akan menggunakan unittest framework milik Python untuk membuat unit test bagi setiap node dalam agent workflow.

```
#test_react_agent_module.py

import unittest
from unittest.mock import patch
from react_agent_module import perform_search, filter_results, compile_markdown

class TestReActAgent(unittest.TestCase):

    @patch('react_agent_module.llm_tool.invoke')
    def test_perform_search(self, mock_invoke):
        # Mock the response from the search tool
        mock_invoke.return_value = type("", (), {'content': "Python programming\nBest practices\nCode readability"})()

        state = {"query": "Python programming best practices", "search_results": [], "markdown_result": ""}
        result = perform_search(state)

        # Verify the search results are populated
        expected_results = ["Python programming", "Best practices", "Code readability"]
        self.assertEqual(result['search_results'], expected_results)

    @patch('react_agent_module.llm_tool.invoke')
    def test_filter_results(self, mock_invoke):
        # Mock the response from the filter tool
        mock_invoke.return_value = type("", (), {'content': "Best practices"})()

        state = {"query": "", "search_results": ["Python programming", "Best practices", "Code readability"], "markdown_result": ""}
        result = filter_results(state)

        # Verify only relevant results are kept
        expected_results = ["Best practices"]
```

```

        self.assertEqual(result['search_results'], expected_results)

    @patch('react_agent_module.llm_tool.invoke')
    def test_compile_markdown(self, mock_invoke):
        # Mock the response for the compiled Markdown document
        mock_invoke.return_value = type("", (), {'content': "# Python Best Practices\n\n- Code readability"}())

        state = {"query": "", "search_results": ["Best practices"], "markdown_result": ""}

        result = compile_markdown(state)

        # Verify the Markdown document is correctly compiled
        expected_markdown = "# Python Best Practices\n\n- Code readability"
        self.assertEqual(result['markdown_result'], expected_markdown)

if __name__ == '__main__':
    unittest.main()

```

Penjelasan Unit Test

- **test_perform_search:** Test ini memeriksa apakah node pencarian dengan benar mengisi daftar hasil pencarian. Test ini melakukan mock terhadap respons untuk mensimulasikan output pencarian yang tipikal.
- **test_filter_results:** Test ini memeriksa bahwa hanya informasi relevan yang dipertahankan setelah langkah filtering. Test ini mensimulasikan respons yang telah difilter, memastikan informasi yang tidak relevan telah dihapus.
- **test_compile_markdown:** Test ini memeriksa bahwa agent menyusun hasil yang telah difilter menjadi format dokumen Markdown yang tepat.

Setiap test menggunakan decorator `@patch` untuk melakukan mock terhadap method `invoke` dari `ChatOpenAI`, mensimulasikan respons yang diharapkan tanpa memerlukan pemanggilan aktual ke LLM.

Integration Test untuk Keseluruhan Workflow

Sebuah integration test akan memastikan bahwa keseluruhan workflow-search, filter, dan compile-berfungsi dengan benar saat digabungkan.

```

#lesson16f.py

import unittest

```

```

from unittest.mock import patch
from react_agent_module import graph, ReActAgentState

class TestReActAgentWorkflow(unittest.TestCase):

    @patch('react_agent_module.llm_tool.invoke')
    def test_full_workflow(self, mock_invoke):
        # Mock responses for each stage in the workflow
        mock_invoke.side_effect = [
            type("", (), {'content': "Python programming\nBest practices\nCode readability"})(), # Search results
            type("", (), {'content': "Best practices"})(), # Filtered results
            type("", (), {'content': "# Python Best Practices\n\n- Code readability"})() # Markdown document
        ]

        # Set up initial state and invoke the full workflow
        initial_state = {"query": "Python programming best practices",
                        "search_results": [], "markdown_result": ""}
        result = graph.invoke(initial_state)

        # Verify the final Markdown result
        expected_markdown = "# Python Best Practices\n\n- Code readability"
        self.assertEqual(result['markdown_result'], expected_markdown)

if __name__ == '__main__':
    unittest.main()

```

Penjelasan Integration Test

Integration test, `test_full_workflow`, menguji keseluruhan workflow agent:

- **Respons yang Di-mock:** Output yang diharapkan dari setiap node di-mock secara berurutan untuk mensimulasikan alur data dari satu langkah ke langkah berikutnya.
- **Verifikasi Akhir:** Test ini memeriksa bahwa `markdown_result` akhir sesuai dengan output Markdown yang diharapkan.

Menjalankan Test

Untuk menjalankan test, buka terminal dan jalankan:

```
python -m unittest test_react_agent_workflow.py
```

```
terminal  
python -m unittest test_react_agent_module.py
```

Ini akan menjalankan baik unit test maupun integration test, memastikan bahwa setiap node dan keseluruhan workflow berfungsi dengan benar.

Kesimpulan dan Pembelajaran Utama

Bab ini mendemonstrasikan cara untuk:

1. **Build and test a LangGraph ReAct agent** that searches the web and compiles a Markdown document.
2. **Write unit tests for each node** in the agent workflow, ensuring isolated functionality.
3. **Create an integration test** for the entire workflow, validating end-to-end functionality.

Dengan mengikuti langkah-langkah ini, Anda telah memperoleh keterampilan praktis dalam membangun dan menguji agent kompleks dengan LangGraph, membuat AI agent Anda lebih andal dan robust untuk aplikasi dunia nyata.

Chapter 17

Frontend Development for LangGraph-Powered AI Agents

Dalam bab ini, kita akan:

1. **Set up a backend for LangGraph agents** and expose them via an API.
2. **Create a frontend** that communicates with the agent.
3. **Implement advanced UI features** for a seamless user experience.

17.1 Pengantar Interaksi Frontend dan AI Agent

Mengapa Frontend Penting untuk AI Agent

Sebuah antarmuka frontend membuat AI agent dapat diakses dan interaktif bagi pengguna akhir, memberikan jendela ke dalam respons dan kemampuan AI. Frontend dapat menampilkan respons agent dalam format terstruktur, menangkap input pengguna, dan memungkinkan komunikasi dua arah secara real-time dengan backend.

Bagaimana Frontend dan Backend Berkomunikasi dalam LangGraph

Dalam pengaturan ini:

1. The **backend** exposes the LangGraph agent as an API, making it available for the frontend.
2. The **frontend** (e.g., a React application) sends requests to the backend and displays responses.

17.2 Mengekspos LangGraph Agent sebagai API

Bagian ini akan memandu Anda melalui proses menyiapkan backend dengan Python, Flask, dan LangGraph untuk mengekspos AI agent Anda sebagai RESTful API.

Langkah 1: Menyiapkan Environment Backend Buat direktori baru untuk proyek Anda dan inisialisasi:

```
terminal
mkdir langgraph-agent-backend
cd langgraph-agent-backend
```

Buat virtual environment dan aktifkan:

```
terminal
python -m venv venv
source venv/bin/activate # On Windows, use venv\Scripts\activate
```

Instal Flask dan LangGraph:

```
terminal
pip install flask langgraph
```

Langkah 2: Mendefinisikan Agent dalam LangGraph

Mari kita definisikan sebuah LangGraph agent sederhana yang memberikan jawaban atas query pengguna.

```
#agent.py

from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage

# Define the agent's state
class AgentState(TypedDict):
    query: str
    response: str

# Initialize the AI tool (e.g., OpenAI API)
llm_tool = ChatOpenAI(temperature=0, model_name="gpt-4o-mini")

# Define the node that processes user queries
def handle_query(state: AgentState) -> AgentState:
```

```

user_message = HumanMessage(content=state['query'])
ai_response = llm_tool.invoke([user_message])
state['response'] = ai_response.content
return state

# Build the LangGraph workflow for the agent
builder = StateGraph(AgentState)
builder.add_node("handle_query", handle_query)
builder.add_edge(START, "handle_query")
builder.add_edge("handle_query", END)
graph = builder.compile()

```

Kode ini menginisialisasi sebuah LangGraph workflow yang menggunakan model AI (misalnya, model GPT dari OpenAI) untuk memproses query pengguna. Agent ini menerima sebuah query dan menghasilkan respons.

Langkah 3: Membuat API untuk Mengekspos Agent

Kita akan membuat sebuah API menggunakan Flask untuk mengekspos agent ini.

```

#app.py

from flask import Flask, request, jsonify
from agent import graph # Import the LangGraph workflow

app = Flask(__name__)

# Define the endpoint to interact with the LangGraph agent
@app.route('/api/agent', methods=['POST'])
def agent():
    data = request.json
    query = data.get("query", "")
    initial_state = {"query": query, "response": ""}
    result = graph.invoke(initial_state)
    return jsonify({"response": result['response']})

# Start the Flask application
if __name__ == '__main__':
    app.run(debug=True)

```

Menjalankan API

Untuk memulai Flask server:

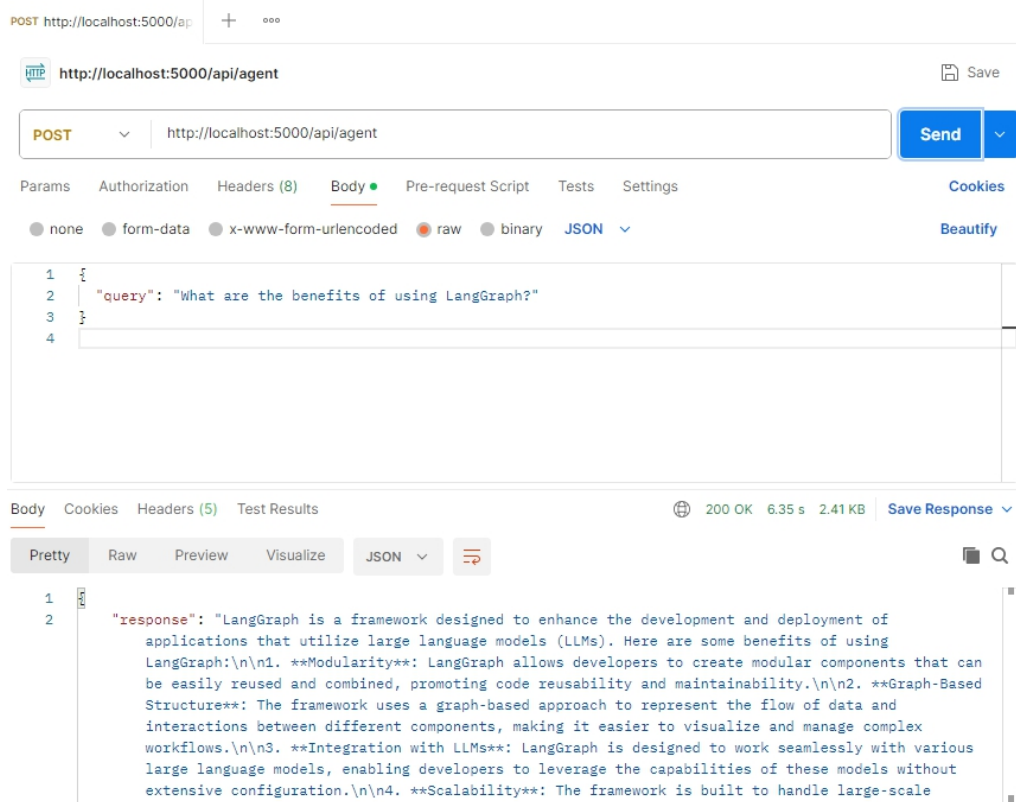
```
terminal
```

```
python app.py
```

Server akan berjalan pada `http://localhost:5000` , dan agent dapat diakses melalui endpoint `http://localhost:5000/api/agent` . Anda dapat menguji endpoint ini dengan mengirimkan permintaan POST dengan body JSON, seperti:

```
{  
  "query": "What are the benefits of using LangGraph?"  
}
```

Anda dapat menggunakan tool gratis postman untuk menguji endpoint (www.postman.com) seperti terlihat pada screenshot di bawah.



17.3 Menyiapkan Lingkungan Pengembangan Frontend

Langkah 1: Instal Node.js dan Inisialisasi Proyek React. Instal Node.js jika belum. Verifikasi instalasi:

```
termina
```

```
node -v  
npm -v
```

Buat proyek React menggunakan Create React App:

```
terminal  
npx create-react-app langgraph-frontend  
cd langgraph-frontend
```

Instal Axios untuk membuat HTTP request:

```
terminal  
npm install axios
```

Atur Environment Variables: Buat file .env untuk menyimpan URL API backend.

```
#.env  
REACT_APP_AGENT_API_URL=http://localhost:5000/api/agent
```

17.4 Membangun Antarmuka Frontend

Langkah 1: Membuat Input Field dan Tampilan untuk Interaksi Agent

Di App.js , kita akan membuat input field agar pengguna dapat memasukkan query, tombol untuk mengirim query, dan bagian untuk menampilkan respons agent.

```
//App.js  
  
import React, { useState } from 'react';  
import axios from 'axios';  
  
function App() {  
  const [query, setQuery] = useState("");  
  const [response, setResponse] = useState("");  
  const [loading, setLoading] = useState(false);  
  const [error, setError] = useState("");  
  
  const handleInputChange = (e) => setQuery(e.target.value);  
  
  const handleSubmit = async (e) => {  
    e.preventDefault();  
    setLoading(true);
```

```

    setError("");
    try {
      const res = await axios.post(process.env.REACT_APP_AGENT_API_URL, { query
    });

    setResponse(res.data.response);
  } catch (err) {
    setError('Error: Could not reach the AI agent.');
```

}

```

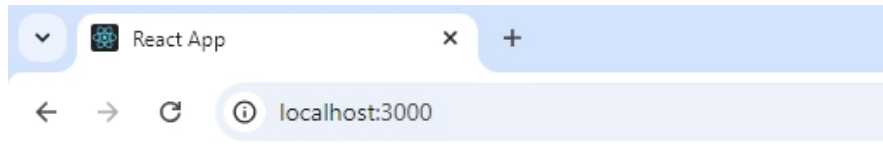
  } finally {
    setLoading(false);
  }
};

return (
  <div className="App">
    <h1>LangGraph AI Agent Interaction</h1>
    <form onSubmit={handleSubmit}>
      <input type="text" value={query} onChange={handleInputChange}
placeholder="Ask the AI agent..." />
      <button type="submit" disabled={loading}>Send</button>
    </form>
    {loading && <p>Loading...</p>}
    {error && <p>{error}</p>}
    <div className="response">
      <h2>Agent Response:</h2>
      <p>{response}</p>
    </div>
  </div>
);
}

export default App;

```

Output:



LangGraph AI Agent Interaction

Ask the AI agent...

Agent Response:

Penjelasan

- **Input dan Button:** Memungkinkan pengguna memasukkan query dan mengirimkannya.
- **Loading Indicator:** Menampilkan pesan loading saat menunggu respons.
- **Error Handling:** Menangkap dan menampilkan error apa pun jika request gagal.

17.5 Menghubungkan Frontend ke LangGraph Agent

Kita telah mengonfigurasi frontend untuk terhubung ke LangGraph agent melalui Axios di `handleSubmit`. Fungsi ini mengirimkan query pengguna ke backend API, menerima respons, dan memperbarui variabel state `response`, yang menampilkan balasan agent.

Menangani Komunikasi Real-Time

Untuk skenario yang lebih lanjut, Anda dapat menggunakan Websocket untuk mengaktifkan pembaruan real-time dari backend. Pendekatan ini cocok jika agent memerlukan waktu untuk memproses tugas yang kompleks atau jika Anda ingin menampilkan respons antara.

17.6 Melakukan Deployment Aplikasi Secara Penuh

Langkah 1: Build Frontend

Jalankan perintah berikut di direktori proyek React Anda:

```
terminal
```



Ini akan membuat build siap produksi di folder build .

Langkah 2: Melakukan Deployment Backend dan Frontend

Anda dapat melakukan deployment backend Flask dan frontend React pada platform terpisah atau dalam satu setup server yang sama.

- **Backend: Gunakan platform seperti Heroku, AWS, atau DigitalOcean.-**
Frontend: Platform seperti Vercel atau Netlify ideal untuk hosting frontend statis.

Mengintegrasikan Frontend dan Backend di Produksi

1. Update the `.env` file in the frontend to use the deployed backend API URL.
2. Redeploy the frontend with the new environment variables.

17.7 Ringkasan dan Best Practice

Bab ini memberikan panduan lengkap tentang cara membuat frontend dan backend untuk AI agent berbasis LangGraph. Dengan mengikuti langkah-langkah yang diuraikan di sini, Anda sekarang memiliki setup full-stack yang memungkinkan pengalaman AI yang dinamis dan interaktif.

Best Practice

- **Keamanan: Gunakan HTTPS dan autentikasi yang aman untuk deployment produksi.-**
Skalabilitas: Gunakan load balancer untuk menangani traffic tinggi.-
Desain Responsif: Pastikan frontend ramah perangkat mobile dan responsif.

Dengan fondasi ini, Anda dapat membuat dan menskalakan aplikasi AI yang andal dan ramah pengguna, yang didukung oleh LangGraph.

17.8 Menyiapkan Streaming Asinkron dengan LangGraph dan WebSocket

Untuk mendukung respons streaming, kita akan menggunakan Flask-SocketIO untuk komunikasi WebSocket. Setup ini akan memungkinkan LangGraph agent kita mengirim respons secara bertahap ke frontend, menciptakan pengalaman yang responsif dan interaktif.

Langkah 1: Instal Flask-SocketIO dan Konfigurasi Backend

Instal Flask-SocketIO dan eventlet untuk dukungan WebSocket asinkron:

```
terminal  
pip install flask-socketio eventlet
```

Perbarui app.py untuk menginisialisasi SocketIO dan mengonfigurasi event WebSocket.

```
#app.py  
  
from flask import Flask, request, jsonify  
from flask_socketio import SocketIO, emit  
from typing import TypedDict  
from langgraph.graph import StateGraph, START, END  
from langchain_openai import ChatOpenAI  
from langchain.schema import HumanMessage  
  
# Define the agent's state  
class AgentState(TypedDict):  
    query: str  
    response: str  
  
# Initialize the AI tool (e.g., OpenAI API)  
llm_tool = ChatOpenAI(temperature=0, model_name="gpt-4o-mini")  
  
# Define the node that processes user queries  
def handle_query(state: AgentState) -> AgentState:  
    user_message = HumanMessage(content=state['query'])  
    ai_response = llm_tool.invoke([user_message])  
    state['response'] = ai_response.content  
    return state  
  
# Build the LangGraph workflow for the agent  
builder = StateGraph(AgentState)  
builder.add_node("handle_query", handle_query)  
builder.add_edge(START, "handle_query")  
builder.add_edge("handle_query", END)  
graph = builder.compile()  
  
app = Flask(__name__)  
app.config['SECRET_KEY'] = 'your_secret_key'  
socketio = SocketIO(app, async_mode='eventlet') # Initialize SocketIO with eventlet  
  
# Define the REST endpoint for non-streaming queries
```



```

@app.route('/api/agent', methods=['POST'])
def agent():
    data = request.json
    query = data.get("query", "")
    initial_state = {"query": query, "response": ""}
    result = graph.invoke(initial_state)
    return jsonify({"response": result['response']})

# Define a WebSocket event for streaming responses
@socketio.on('stream_query')
def handle_stream_query(data):
    query = data.get("query", "")
    initial_state = {"query": query, "response": ""}

    # Stream response in chunks by processing the agent's output incrementally
    for response_chunk in stream_agent_response(initial_state):
        emit('response_chunk', {'chunk': response_chunk})

    emit('response_complete', {'message': 'Response streaming complete'})

def stream_agent_response(state):
    """
    Simulates streaming responses from LangGraph by yielding chunks of text.
    In production, integrate this with LangGraph's streaming features if available.
    """
    response = graph.invoke(state)['response']
    # Split response into chunks for streaming
    chunk_size = 50
    for i in range(0, len(response), chunk_size):
        yield response[i:i+chunk_size] # Yield chunks of response text

# Start the application with SocketIO
if __name__ == '__main__':
    socketio.run(app, debug=True)

```

Penjelasan

- **WebSocket Event stream_query** : Endpoint WebSocket ini menangani query streaming. Ia menggunakan stream_agent_response untuk memecah respons penuh menjadi chunk yang lebih kecil dan mengirimkan setiap chunk ke client.
- **Chunked Response Streaming**: Fungsi stream_agent_response memecah respons menjadi bagian-bagian yang lebih kecil dan mengirimkan setiap bagian secara terpisah

to simulate real-time streaming. In a production setup, this would use LangGraph's streaming support if available.

17.9 Mengintegrasikan Streaming pada Frontend

Pada bab ini, kita akan menyiapkan sistem streaming real-time untuk respons LangGraph AI agent menggunakan backend FastAPI dan frontend React. Fitur streaming sangat berguna untuk respons yang besar atau berlangsung lama, di mana lebih baik menampilkan data segera setelah tersedia daripada menunggu seluruh respons selesai.

Pada akhir bagian ini, Anda akan mempelajari:

- Cara menggunakan FastAPI untuk melakukan streaming data dari AI agent.
- Cara menghubungkan aplikasi frontend React untuk menampilkan respons streaming secara real time.
- Cara mengelola state, loading indicator, dan error handling dalam setup streaming.

17.10 Pengantar Respons Streaming

Apa itu Streaming? Streaming mengirimkan data dalam bentuk chunk begitu tersedia, alih-alih menunggu seluruh respons siap. Pendekatan ini meningkatkan pengalaman pengguna dengan memberikan feedback langsung, yang ideal untuk aplikasi seperti AI agent yang mungkin memerlukan waktu untuk memproses query yang kompleks.

Menyiapkan Backend dengan FastAPI

Untuk membuat backend yang melakukan streaming respons dari AI agent kita, kita akan menggunakan FastAPI, sebuah framework web Python yang dikenal karena performanya yang cepat dan sintaksnya yang mudah digunakan. Kita juga akan menggunakan LangGraph untuk membangun workflow AI agent kita.

Langkah 1: Instal Library yang Diperlukan

Pastikan Anda telah menginstal FastAPI, Uvicorn, dan LangGraph:

```
terminal
pip install fastapi uvicorn langgraph
```

Langkah 2: Kode untuk Backend Streaming

Berikut adalah kode lengkap untuk backend FastAPI. Kita akan membahas setiap bagiannya untuk menjelaskan cara kerjanya.

```
#lesson17c.py
```

```

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from typing import List, Dict, Any
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage
from typing import TypedDict

import uvicorn

# Define the agent's state
class AgentState(TypedDict):
    query: str
    response: str

# Initialize the AI tool (e.g., OpenAI API)
llm_tool = ChatOpenAI(temperature=0, model_name="gpt-4o-mini", streaming=True)

# Define the node that processes user queries
def handle_query(state: AgentState) -> AgentState:
    user_message = HumanMessage(content=state['query'])
    ai_response = llm_tool.invoke([user_message])
    state['response'] = ai_response.content
    return state

# Build the LangGraph workflow for the agent
builder = StateGraph(AgentState)
builder.add_node("handle_query", handle_query)
builder.add_edge(START, "handle_query")
builder.add_edge("handle_query", END)
graph = builder.compile()

app = FastAPI()

# Enable CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

```

class QueryRequest(BaseModel):
    query: str

@app.post("/api/research")
async def research_query(request: QueryRequest):
    try:
        initial_state = {"query": request.query, "response": ""}
        result = graph.invoke(initial_state)
        return {"response": result['response']}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/api/research/stream")
async def research_query_stream(request: QueryRequest):
    async def event_generator():
        try:
            initial_state = {"query": request.query, "response": ""}
            async for msg, metadata in graph.astream(initial_state,
stream_mode="messages"):
                if msg.content and not isinstance(msg, HumanMessage):
                    yield msg.content
        except Exception as e:
            yield {"error": str(e)}

    return StreamingResponse(event_generator(), media_type="text/plain")

#RUN THE APP
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Penjelasan Kode

Mari kita bahas kode ini langkah demi langkah untuk memahami apa yang dilakukan setiap bagian.

Menyiapkan FastAPI dan CORS:

```

#lesson17c.py continued

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],

```

```
allow_credentials=True,  
allow_methods=["*"],  
allow_headers=["*"],  
)
```

- **FastAPI:** We initialize a FastAPI application instance named `app`.
- **CORS Middleware:** CORS (Cross-Origin Resource Sharing) allows the frontend on a different server (localhost:3000) to access our API without restrictions. This is necessary in local development to prevent CORS errors.

Mendefinisikan State Agent:

```
##lesson17c.py continued  
  
class AgentState(TypedDict):  
    query: str  
    response: str
```

AgentState: Ini mendefinisikan dictionary state dengan dua field, query dan response . query menyimpan pertanyaan pengguna, dan response akan berisi jawaban dari AI agent.

Menyiapkan AI Tool:

```
##lesson17c.py continued  
  
llm_tool = ChatOpenAI(temperature=0, model_name="gpt-4o-mini", streaming=True)
```

- We initialize `llm_tool` as an instance of the OpenAI API, **with streaming enabled**. This allows the AI to provide a continuous stream of data as it generates responses.

Membangun Workflow LangGraph:

```
##lesson17c.py continued  
  
builder = StateGraph(AgentState)  
builder.add_node("handle_query", handle_query)  
builder.add_edge(START, "handle_query")  
builder.add_edge("handle_query", END)  
graph = builder.compile()
```

Kita membuat workflow LangGraph dengan satu node (`handle_query`) yang memproses query dan menghasilkan respons.

Membuat Endpoint Streaming:

```
##lesson17c.py continued

@app.post("/api/research/stream")
async def research_query_stream(request: QueryRequest):
    async def event_generator():
        try:
            initial_state = {"query": request.query, "response": ""}
            async for msg, metadata in graph.astream(initial_state,
stream_mode="messages"):
                if msg.content and not isinstance(msg, HumanMessage):
                    yield msg.content
        except Exception as e:
            yield {"error": str(e)}

    return StreamingResponse(event_generator(), media_type="text/plain")
```

StreamingResponse: Endpoint ini mengembalikan `StreamingResponse` , yang akan menyediakan respons dalam bentuk chunk begitu data tersedia.

17.11 Menyiapkan Frontend dengan React

Pada frontend, kita akan menggunakan React untuk terhubung ke backend FastAPI kita dan menampilkan respons yang di-streaming.

Langkah 1: Instal Dependency yang Diperlukan

Instal `axios` (untuk HTTP request) dan tambahkan library styling UI jika diperlukan.

```
terminal
npm install axios
```

Langkah 2: Kode untuk Frontend Streaming

Berikut adalah kode lengkap untuk komponen frontend React. Kita akan menjelaskan setiap bagiannya kemudian.

```
//lesson17d.js

import React, { useState } from 'react';
import axios from 'axios';
```

```
function App() {
  const [query, setQuery] = useState("");
  const [response, setResponse] = useState("");
  const [loading, setLoading] = useState(false);

  const [streamingResponse, setStreamingResponse] = useState("");
  const handleStreamSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setStreamingResponse("");

    try {
      const response = await fetch('http://localhost:8000/api/research/stream', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({ query }),
      });

      if (!response.ok) {
        throw new Error(`HTTP error! Status: ${response.status}`);
      }

      const reader = response.body.getReader();
      const decoder = new TextDecoder('utf-8');
      let done = false;

      while (!done) {
        const { value, done: readerDone } = await reader.read();
        done = readerDone;

        if (value) {
          const chunk = decoder.decode(value, { stream: true });
          setStreamingResponse((prev) => prev + chunk);
        }
      }
    } catch (error) {
      console.error('Streaming error:', error);
      setStreamingResponse('An error occurred while streaming the response.');
```

```
    } finally {
      setLoading(false);
    }
  };
}
```

```

return (
  <div className="container mx-auto p-4">
    <h1 className="text-2xl font-bold mb-4">Research Assistant</h1>
    <form onSubmit={handleStreamSubmit}>
      <input
        type="text"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        className="w-full p-2 border rounded mb-4"
        placeholder="Enter your research query..."
      />

      {response && (
        <div className="mt-4 p-4 border rounded">
          <h2 className="font-bold mb-2">Response:</h2>
          <p>{response}</p>
        </div>
      )}
      <div className="mt-4">
        <button
          onClick={handleStreamSubmit}
          className="bg-green-500 text-white px-4 py-2 rounded ml-2"
          disabled={loading}
        >
          Stream Response
        </button>
      </div>
    </form>
    {streamingResponse && (
      <div className="mt-4 p-4 border rounded">
        <h2 className="font-bold mb-2">Streaming Response:</h2>
        <p>{streamingResponse}</p>
      </div>
    )}
  </div>
);
}

export default App;

```


Penjelasan Kode

1. State Management:

- **query** : Stores the user's input.
- **streamingResponse** : Holds the streaming response as it's received.
- **loading** : Indicates whether a response is currently being streamed.

2. Sending the Query:

```
//lesson17d.js explained  
  
const handleStreamSubmit = async (e) => {  
  e.preventDefault();  
  ...  
};
```

When the form is submitted, the function sends a **POST** request to the FastAPI endpoint and sets up the response stream.

3. Reading the Response Stream:

```
//lesson17d.js explained  
  
const reader = response.body.getReader();  
const decoder = new TextDecoder('utf-8');  
let done = false;  
while (!done) {  
  const { value, done: readerDone } = await reader.read();  
  done = readerDone;  
  if (value) {  
    const chunk = decoder.decode(value, { stream: true });  
    setStreamingResponse((prev) => prev + chunk);  
  }  
}
```

This code reads the response stream chunk by chunk, updating **streamingResponse** with each new piece of data received.

4. Displaying the Response:

```
//lesson17d.js explained
```

```
{streamingResponse && (  
  <div className="mt-4 p-4 border rounded">  
    <h2 className="font-bold mb-2">Streaming Response:</h2>  
    <p>{streamingResponse}</p>  
  </div>  
)}
```

As the response is being streamed, it's displayed in real-time, providing instant feedback to the user.

7.12 Ringkasan dan Best Practice untuk Respons Streaming

Poin Utama

- **Streaming Asinkron:** Streaming berbasis WebSocket menciptakan pengalaman pengguna yang responsif, terutama untuk respons yang besar.
- **Penanganan Frontend:** Pastikan frontend dirancang untuk menangani pembaruan bertahap dengan lancar dan mengelola transisi state untuk loading serta penyelesaian.
- **Menguji Respons Streaming:** Uji koneksi secara menyeluruh untuk memastikan ketahanan, terutama dalam menangani gangguan jaringan dengan baik.

Best Practice untuk Streaming Real-Time pada Aplikasi LangGraph

1. **Optimize Chunk Sizes:** Adjust chunk sizes based on user experience and network performance.
2. **Error Handling:** Implement error handling in both backend and frontend to handle potential connection issues.
3. **Loading and Completion States:** Clearly indicate loading progress and completion to keep users informed.

Dengan mengikuti panduan ini, Anda telah membangun antarmuka streaming real-time yang berfungsi penuh untuk AI agent berbasis LangGraph. Arsitektur ini sangat scalable dan meningkatkan pengalaman pengguna melalui respons yang langsung dan interaktif, cocok untuk aplikasi AI yang siap produksi.

17.13 Menyiapkan Backend dengan Next.js dan Frontend dengan Next.js AI SDK

Next.js adalah framework populer untuk membangun aplikasi full-stack menggunakan React, menawarkan dukungan yang sangat baik untuk server-side rendering (SSR), routing berbasis file,

dan integrasi API. Dengan Next.js AI SDK, kita dapat dengan mudah menghubungkan backend AI berbasis LangChain dengan frontend yang ramah pengguna untuk membangun aplikasi AI yang interaktif.

Apa itu Next.js AI SDK? Next.js AI SDK adalah toolkit yang memudahkan pembuatan komponen UI yang berinteraksi dengan language model dan menangani data streaming secara langsung. Ini menyederhanakan integrasi model LangChain dan memungkinkan Anda membangun UI yang mulus untuk aplikasi AI Anda.

Pada contoh ini, kita akan menggunakan Next.js AI SDK untuk membuat endpoint completion AI streaming real-time dengan LangChain. Backend akan terhubung ke API OpenAI, dan kita akan membangun frontend yang melakukan streaming respons ke pengguna secara real time.

Langkah 1: Menyiapkan Proyek Next.js Baru

Inisialisasi Proyek Next.js: Jika belum, buat proyek Next.js baru dengan menjalankan:

terminal

```
npx create-next-app@latest ai-chat-app  
cd ai-chat-app
```

Command Prompt

```
D:\Projects>npx create-next-app@latest ai-chat-app  
Need to install the following packages:  
create-next-app@15.0.3  
Ok to proceed? (y) y  
  
✓ Would you like to use TypeScript? ... No / Yes  
✓ Would you like to use ESLint? ... No / Yes  
✓ Would you like to use Tailwind CSS? ... No / Yes  
✓ Would you like your code inside a `src/` directory? ... No / Yes  
✓ Would you like to use App Router? (recommended) ... No / Yes  
✓ Would you like to use Turbopack for next dev? ... No / Yes  
✓ Would you like to customize the import alias (@/* by default)? ... No / Yes  
Creating a new Next.js app in D:\Projects\ai-chat-app.  
  
Using npm.  
  
Initializing project with template: app-tw  
  
Installing dependencies:  
- react  
- react-dom  
- next  
  
Installing devDependencies:  
- typescript  
- @types/node  
- @types/react  
- @types/react-dom  
- postcss
```

Instal Dependency yang Diperlukan: Instal Next.js AI SDK, LangChain, dan library client OpenAI:

```
terminal
npm install ai @langchain/openai
```

Struktur Proyek: Next.js menggunakan struktur berbasis folder untuk routing dan manajemen halaman. Pada contoh ini, kita akan menggunakan direktori app dan membuat API route untuk menangani request AI.

Langkah 2: Membangun Backend Completion Route dengan LangChain

Kita akan membuat endpoint streaming di file `app/api/completion/route.ts` . Endpoint ini akan:

1. Accept a prompt as input.
2. Stream responses back to the client using the LangChain model.

```
//app/api/completion/route.ts

import { ChatOpenAI } from '@langchain/openai';
import { LangChainAdapter } from 'ai';
export const maxDuration = 60;
export async function POST(req: Request) {
  // Extract the prompt text from the incoming request body
  const { prompt } = await req.json();
  // Initialize the LangChain model (OpenAI in this case)
  const model = new ChatOpenAI({
    model: 'gpt-4o-mini', // Model version can be updated
    temperature: 0,       // Temperature controls response randomness
  });
  // Use the model's stream method to enable streaming responses
  const stream = await model.stream(prompt);
  // Convert the LangChain response stream into a Data Stream compatible with Next.js AI
  SDK
  return LangChainAdapter.toDataStreamResponse(stream);
}
```

Penjelasan:

- **Inisialisasi Model:** ChatOpenAI adalah model dari LangChain yang terhubung ke GPT-3.5 Turbo milik OpenAI. Di sini, kita mengatur temperature: 0 agar respons bersifat deterministik.- **Streaming dengan model.stream() :** Daripada menunggu seluruh respons, kita melakukan streaming respons dalam bentuk chunk.- **LangChainAdapter.toDataStreamResponse(stream) :** Metode helper ini mengadaptasi output streaming LangChain agar dapat bekerja dengan mulus bersama komponen frontend AI SDK.

Endpoint ini akan dapat diakses melalui request POST di /api/completion .

Langkah 3: Membuat Komponen Chat Frontend

Selanjutnya, kita akan menggunakan hook useCompletion dari Next.js AI SDK untuk menangani input pengguna dan menampilkan respons secara real-time saat di-streaming dari backend.

1. Create a new file at `app/page.tsx` .
2. Add the following code to set up a simple chat interface:

```
//pp/page.tsx
'use client';
import { useCompletion } from 'ai/react';
import { useState } from 'react';
export default function Chat() {
  // Destructure values from the `useCompletion` hook
  const { completion, input, handleInputChange, handleSubmit } = useCompletion();
  return (
    <div className="chat-container">
      /* Display streaming AI response here */
      <div className="response">{completion}</div>
      /* User input form */
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={input}
          onChange={handleInputChange}
          placeholder="Ask the AI anything..."
          className="input-field"
        />
        <button type="submit" className="submit-button">Send</button>
      </form>
      <style jsx>{`
```

```

.chat-container {
  max-width: 600px;
  margin: 0 auto;
  padding: 2rem;
  font-family: Arial, sans-serif;
}
.response {
  padding: 1rem;
  border: 1px solid #ddd;
  border-radius: 8px;
  min-height: 100px;
  margin-bottom: 1rem;
  background-color: #f9f9f9;
}
.input-field {
  width: calc(100% - 100px);
  padding: 0.5rem;
  margin-right: 1rem;
}
.submit-button {
  padding: 0.5rem 1rem;
  background-color: #0070f3;
  color: white;
  border: none;
  border-radius: 5px;
}
`}</style>
</div>

);
}

```

Penjelasan:

- **useCompletion Hook:** Hook ini, yang disediakan oleh AI SDK, menangani input pengguna dan mengelola state respons AI saat di-streaming dari backend.
 - **completion** : Holds the streamed response text as it arrives.
 - **input** : Stores the user's current input text.
 - **handleInputChange** : Updates **input** as the user types.

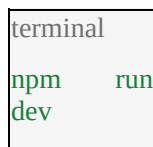
? **handleSubmit** : Terpicu saat form dikirim, mengirimkan query ke backend.- **Styling UI**: Blok style jsx menyediakan styling dasar untuk container chat, area tampilan respons, dan field input form, menciptakan tata letak yang sederhana dan rapi.

Bagaimana Semua Ini Bekerja Bersama

- **Input Pengguna**: Ketika pengguna memasukkan query dan mengirimkan form, **handleSubmit** mengirimkan prompt ke endpoint backend `/api/completion` .- **Respons Streaming**: Backend melakukan streaming respons AI kembali ke frontend. Setiap chunk ditambahkan ke state completion oleh hook `useCompletion` , menampilkannya secara bertahap saat diterima.- **Tampilan**: State completion diperbarui secara real time, menampilkan respons AI saat sedang dihasilkan.

Langkah 4: Menguji dan Menjalankan Aplikasi

Jalankan server development Next.js:



Uji Antarmuka Chat:

- Open <http://localhost:3000> in your browser.
- Enter a query in the input field and submit.
- You should see the response stream in real time, displaying each chunk as it arrives.

17.14 Ringkasan dan Best Practice

Dengan mengikuti setup ini, Anda telah membangun antarmuka AI streaming yang menggabungkan Next.js, LangChain, dan Next.js AI SDK. Berikut beberapa best practice:

- **Penanganan Respons Streaming**: Selalu tangani respons streaming dengan cara yang memperbarui UI secara bertahap untuk meningkatkan pengalaman pengguna.- **Error Handling**: Pada aplikasi produksi, sertakan error handling untuk mengelola request yang gagal atau masalah konektivitas.

- **Antarmuka Pengguna dan Feedback:** Berikan feedback yang jelas untuk status loading, dan pastikan UI terasa responsif dan mulus bagi pengguna.

Pendekatan ini memungkinkan interaksi real-time dengan model AI berbasis LangChain di Next.js, memberikan solusi yang scalable dan responsif untuk membangun aplikasi AI yang kompleks.

17.15 Menyiapkan Backend LangGraph dengan Next.js dan AI SDK

Pada panduan ini, kita akan membuat backend berbasis LangGraph untuk streaming respons AI menggunakan Next.js. LangGraph membantu membangun workflow AI agent, sementara Next.js AI SDK memungkinkan kita dengan mudah membuat komponen UI yang kompatibel dengan streaming. Contoh ini akan mengintegrasikan kemampuan manajemen workflow LangGraph dengan LangChain untuk language model dan memanfaatkan Next.js untuk interaksi frontend-backend yang mulus.

Langkah 1: Inisialisasi Proyek Next.js dan Instal Dependency

Jika Anda belum membuat proyek Next.js, mulailah dengan melakukannya. Kemudian instal library yang diperlukan:

Inisialisasi Proyek Next.js Baru:

```
terminal
npx create-next-app@latest langgraph-ai-app
cd langgraph-ai-app
```

Instal Dependency: Instal LangGraph, LangChain, dan Next.js AI SDK:

```
terminal
npm install ai @langchain/openai @langchain/langgraph @langchain/core
```

Langkah 2: Konfigurasi Endpoint API Streaming LangGraph

Pada langkah ini, kita akan membuat endpoint API di Next.js yang menggunakan LangGraph untuk menangani workflow dan melakukan streaming respons AI.

Buat File `app/api/stream/route.ts` : Di dalam proyek Next.js Anda, arahkan ke direktori `app/api` (buat jika belum ada) dan tambahkan folder `stream` dengan file `route.ts` di dalamnya.

Struktur File:??? app

```
??? api
  ??? stream
    └── route.ts
```

Menulis Endpoint Streaming yang Mendukung LangGraph:

Di route.ts , kita akan menyiapkan workflow LangGraph, mendefinisikan state AI agent, menginisialisasi model LangChain, dan melakukan streaming respons ke client menggunakan adapter dari Next.js AI SDK.

```
//route.ts

import { ChatOpenAI } from '@langchain/openai';
import { LangChainAdapter } from 'ai';
import { StateGraph, START, END } from 'langgraph';
import { HumanMessage } from 'langchain/schema';
import { NextRequest, NextResponse } from 'next/server';

// Define the AI agent's state structure
type AgentState = {
  query: string;
  response: string;
};

// Initialize the LangChain model with streaming enabled
const model = new ChatOpenAI({
  model: 'gpt-4o-mini',
  temperature: 0,
  streaming: true,
});

// Define the node function for processing user queries
async function handleQuery(state: AgentState): Promise<AgentState> {
  const userMessage = new HumanMessage(state.query);
  const aiResponse = await model.invoke([userMessage]);
  return { ...state, response: aiResponse.content };
}

// Build the LangGraph workflow for the AI agent
const builder = new StateGraph<AgentState>();
builder.addNode("handleQuery", handleQuery);
builder.addEdge(START, "handleQuery");
builder.addEdge("handleQuery", END);
const graph = builder.compile();

// Define the Next.js API route for streaming responses
```

```

export async function POST(req: NextRequest) {
  const { prompt } = await req.json();
  // Initialize the agent's initial state
  const initialState = { query: prompt, response: "" };
  // Use LangGraph's asynchronous streaming to process the workflow
  const stream = await graph.astream(initialState, { stream_mode: "messages" });
  // Convert the LangGraph stream to a Next.js AI SDK-compatible stream
  return LangChainAdapter.toDataStreamResponse(stream);
}

```

Penjelasan

- **Tipe AgentState:** Mendefinisikan struktur state AI agent, dengan field query dan response .- **Workflow LangGraph?**
Fungsi Node (handleQuery): Memproses query pengguna dengan meneruskannya ke model LangChain dan mengembalikan state yang diperbarui dengan respons model.? **Setup StateGraph dan Workflow:** Workflow LangGraph mencakup satu node, handleQuery , yang terhubung antara titik start dan end.? **Streaming dengan graph.astream :** Metode ini memungkinkan kita memproses query secara asinkron dan melakukan streaming respons secara real-time.-
Next.js API Route: Menggunakan workflow LangGraph untuk menangani request yang masuk dan melakukan streaming respons ke client. Fungsi LangChainAdapter.toDataStreamResponse(stream) membungkus stream LangGraph agar kompatibel dengan sistem streaming AI SDK.

Langkah 3: Membangun Komponen Chat Frontend

Dengan backend sudah siap, kita sekarang akan membuat komponen React menggunakan Next.js AI SDK untuk mengirim query ke endpoint berbasis LangGraph kita dan menampilkan respons saat mereka di-streaming kembali.

Buat Komponen Chat di app/page.tsx :

Komponen ini akan memungkinkan pengguna memasukkan prompt dan melihat respons muncul secara real-time saat di-streaming dari backend.

```
# app/page.tsx
```

```
'use client';
import { useCompletion } from 'ai/react';
import { useState } from 'react';
export default function Chat() {
  // Initialize state and methods from `useCompletion`
  const { completion, input, handleInputChange, handleSubmit } = useCompletion({
    url: '/api/stream', // Point to our LangGraph API endpoint
  });
  return (
    <div className="chat-container">
      {/* Display the streaming AI response */}
      <div className="response">{completion}</div>
      {/* User input form */}
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={input}
          onChange={handleInputChange}
          placeholder="Ask the AI anything..."
          className="input-field"
        />
        <button type="submit" className="submit-button">Send</button>
      </form>
      {/* Basic styling for the chat interface */}
      <style jsx>{`
        .chat-container {
          max-width: 600px;
          margin: 0 auto;
          padding: 2rem;
          font-family: Arial, sans-serif;
        }
        .response {
          padding: 1rem;
          border: 1px solid #ddd;
          border-radius: 8px;
          min-height: 100px;
          margin-bottom: 1rem;
          background-color: #f9f9f9;
        }
      `}
```

```

        .input-field {
            width: calc(100% - 100px);
            padding: 0.5rem;
            margin-right: 1rem;
        }

        .submit-button {
            padding: 0.5rem 1rem;
            background-color: #0070f3;
            color: white;
            border: none;
            border-radius: 5px;
        }
    `}</style>
</div>

);
}

```

Penjelasan

- **useCompletion Hook:?** **Properties (completion, input, handleInputChange, dan handleSubmit):** Properties dari useCompletion ini mengelola state UI dan menghubungkan input user ke respons streaming backend. **URL:** Kami menentukan endpoint API streaming backend di `/api/stream`. **completion:** Menyimpan respons AI saat streaming kembali dari backend. **input dan handleInputChange:** Mengelola field input dan memperbaruinya dengan query user. **handleSubmit:** Mengirim input ke backend dan memulai proses streaming.
- **UI Styling:** Menyediakan antarmuka yang bersih dan sederhana dengan CSS styling dasar untuk chat box, tampilan respons, dan elemen form.

Langkah 4: Menjalankan dan Menguji Aplikasi

Jalankan Next.js Development Server: Jalankan aplikasi Next.js Anda dengan perintah berikut:

```
terminal
```

```
npm    run  
dev
```

Menguji Chat Interface:

- Open <http://localhost:3000> in your browser.
- Enter a question in the input field and submit.
- The response should stream incrementally, displaying chunks of text as they arrive.

17.16 Ringkasan dan Best Practices

Dengan mengikuti langkah-langkah ini, Anda telah menyiapkan backend streaming bertenaga LangGraph secara lengkap menggunakan Next.js dan AI SDK. Konfigurasi ini memungkinkan interaksi real-time yang responsif dengan AI agent, memberikan fondasi yang solid untuk aplikasi AI tingkat lanjut.

Best Practices:

- **Streaming UI:** Pastikan frontend mengelola pembaruan incremental dengan lancar, seperti yang ditunjukkan pada field completion.
- **Error Handling:** Tambahkan penanganan error untuk mengelola gangguan jaringan atau error backend dengan baik.
- **Modular Components:** Jaga agar komponen backend dan frontend tetap modular untuk meningkatkan skalabilitas dan maintainability.

Bab ini menyediakan setup yang komprehensif untuk menggunakan LangGraph dengan Next.js, LangChain, dan Next.js AI SDK, sehingga memudahkan pembuatan dan deployment aplikasi interaktif bertenaga AI.

Chapter 18

Membangun Agent dengan NVIDIA NeMo

Inference Models (NIMs)

Pada bab ini, kita akan mengeksplorasi cara memanfaatkan NVIDIA NeMo Inference Models (NIMs) untuk membangun AI agent yang powerful. Kita akan fokus pada platform Build NVIDIA, mendemonstrasikan cara melakukan inference melalui API atau container self-hosted, dan mewujudkannya dengan use case praktis: membuat AI agent untuk content creation dan digital illustration.

18.1. Pengantar NVIDIA NeMo Inference Models

NVIDIA NeMo Inference Models (NIMs) adalah model pre-trained dan state-of-the-art yang dirancang untuk deployment yang mulus di berbagai environment. Model-model ini menawarkan:

- Kemampuan multimodal untuk task yang membutuhkan pemrosesan text, image, dan audio.
- Deployment yang fleksibel: Jalankan model di cloud, on-premises, atau setup hybrid.
- Integrasi API: Akses mudah melalui RESTful API untuk aplikasi yang scalable.
- Customization: Fine-tune atau sesuaikan model dengan kebutuhan bisnis tertentu.

Apa itu Build NVIDIA?

Build NVIDIA adalah platform yang menyediakan akses ke model AI pre-trained, API, dan tool untuk mempercepat development. Platform ini memungkinkan developer untuk:

- Menjelajahi katalog model dan mengakses dokumentasinya.- Menghasilkan API key untuk penggunaan model.- Melakukan deploy model secara lokal atau pada platform pilihan menggunakan Docker container.

Manfaat Utama NIMs

1. **Ease of Use:** Pre-trained models reduce the need for extensive expertise or training time.
2. **Flexibility:** Models can be deployed in the cloud, on-premises, or in hybrid environments.
3. **Scalability:** With robust APIs and containerized options, NIMs can scale to meet the demands of any application.
4. **State-of-the-Art Performance:** Backed by NVIDIA's cutting-edge research and hardware optimization.
5. **Broad Model Catalog:** Access to language models, vision models, and multimodal models for diverse use cases.

Jenis-Jenis NIMs

NVIDIA menyediakan berbagai macam NIMs yang disesuaikan untuk domain tertentu. Berikut adalah beberapa kategori utamanya:

1. Language Models

- **Examples:** Llama 3.1-405B Instruct
- **Use Cases:** Content creation, summarization, conversational AI.

2. Vision Models

- **Examples:** Stable Diffusion , DINOv2
- **Use Cases:** Image generation, object detection, scene understanding.

3. Speech Models

- **Examples:** Conformer-CTC , FastSpeech
- **Use Cases:** Speech-to-text transcription, voice cloning, real-time translation.

4. Multimodal Models

- **Examples:** Models combining **Llama** with **Stable Diffusion**.
- **Use Cases:** Applications requiring text, image, and speech understanding (e.g., AI assistants, interactive media).

Setiap model dilengkapi dengan dokumentasi, contoh, dan opsi deployment yang fleksibel, sehingga memungkinkan developer memilih tool yang tepat untuk pekerjaannya.

Cara Mengakses NIMs

Untuk mulai menggunakan NIMs, Anda perlu:

1. **Sign Up on Build NVIDIA Platform:** Visit Build NVIDIA (<https://build.nvidia.com/>) and create an account.
2. **Explore the Model Catalog:** Search for a model that fits your use case.
3. **Generate an API Key:** Click "Get API Key" for the desired model, which will allow you to make API calls.

Tip: Cari label "Run Anywhere", yang menandakan model yang dapat Anda host sendiri di GPU server Anda sendiri.

Melakukan Inference dengan NIMs

Anda dapat melakukan inference dengan NIMs menggunakan:

1. **Direct API Calls:** Simple RESTful calls using Python's **requests** library.
2. **LangChain Integration:** A higher-level abstraction that simplifies working with NIMs for more complex workflows.

Sekarang kita akan mengeksplorasi kedua metode ini secara detail.

18.2 Inference Menggunakan Panggilan API Langsung

Panggilan API langsung adalah cara paling sederhana untuk menggunakan NIMs. Yang Anda butuhkan hanyalah API Key, endpoint URL model, dan library Python dasar.

Langkah-Langkah untuk API Inference

1. Persiapan

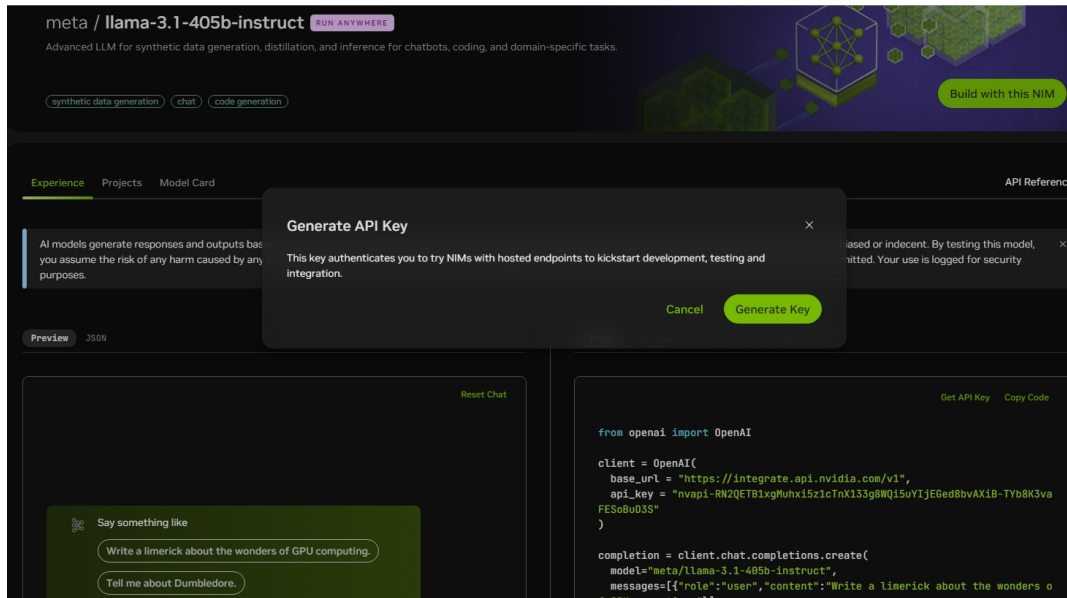
Instal package Python yang diperlukan:

```
pip install requests
```



```
terminal
pip install requests
```

Dapatkan API Key Anda. Buka halaman model, dalam hal ini https://build.nvidia.com/meta/llama-3_1-405b-instruct dan klik Get API Key.



Atur API key NVIDIA Anda sebagai environment variable:

```
terminal
export NVIDIA_API_KEY="your_api_key_here"
```

2. Membuat Permintaan API

Berikut cara melakukan panggilan inference streaming menggunakan library requests:

```
#lesson18a.py

import os
import requests
import json

# Retrieve the API key from the environment variable
API_KEY = os.getenv("NVIDIA_API_KEY")
url = "https://integrate.api.nvidia.com/v1/chat/completions"

# Define headers with Authorization, Accept, and Content-Type
headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Accept": "application/json",
```

```

        "Content-Type": "application/json"
    }
def call_nvidia_api_stream(prompt):
    """Call NVIDIA API with streaming and format the output."""
    payload = {
        "messages": [
            {
                "role": "user",
                "content": prompt
            }
        ],
        "stream": True,
        "model": "meta/llama-3.1-405b-instruct",
        "max_tokens": 1024,
        "presence_penalty": 0,
        "frequency_penalty": 0,
        "top_p": 0.7,
        "temperature": 0.2
    }

    # Make the POST request with streaming enabled
    response = requests.post(url, headers=headers, json=payload, stream=True)
    if response.status_code == 200:
        output = ""
        print("Generating response:")
        for chunk in response.iter_lines():
            if chunk:
                try:
                    # Parse the JSON chunk and extract content
                    data = json.loads(chunk[6:]) # Skip 'data: ' prefix
                    if 'choices' in data:
                        delta = data['choices'][0]['delta']
                        content = delta.get('content', "")
                        if content: output += content
                        print(content, end="", flush=True)
                except json.JSONDecodeError:
                    continue
        print("\n\nFinal Output:")
        print(output)
        return output

```

```
else:
    raise Exception(f"API Error: {response.text}")
# Example usage
call_nvidia_api_stream("Write a promotional message for an eco-friendly gadget.")
```

18.3: Inference Menggunakan LangChain

LangChain menyediakan interface yang streamlined untuk bekerja dengan NIMs, mengurangi boilerplate code dan menyederhanakan workflow.

Langkah-Langkah untuk Integrasi LangChain

1. Persiapan

Instal package langchain_nvidia_ai_endpoints:

```
terminal
pip install langchain_nvidia_ai_endpoints
```

2. Menggunakan LangChain dengan NIMs

LangChain menyederhanakan proses streaming:

```
#langchain with Nvidia
from langchain_nvidia_ai_endpoints import ChatNVIDIA
import os
API_KEY = os.getenv("NVIDIA_API_KEY")
client = ChatNVIDIA(
    model="meta/llama-3.1-405b-instruct",
    api_key=API_KEY,
    temperature=0.2,
    top_p=0.7,
    max_tokens=1024,
)
prompt = "Write a promotional message for an eco-friendly gadget."
for chunk in client.stream([{"role": "user", "content": prompt}]):
    print(chunk.content, end="")
```

Mengapa Menggunakan LangChain?

1. **Modularity:** Combines multiple AI models into a cohesive workflow.
2. **Ease of Use:** Built-in methods for common AI tasks.
3. **Scalability:** Seamlessly integrates with tools like LangGraph for more complex orchestrations.

18.4 Self-Hosting NIMs

Meskipun API menyediakan cara yang praktis untuk mengakses NVIDIA NeMo Inference Models, self-hosting menawarkan kontrol, privasi, dan customization yang lebih besar. Pada bagian ini, kita akan mengeksplorasi cara meng-host NIMs pada infrastruktur Anda sendiri beserta requirement, arsitektur, opsi deployment, biaya, dan keterbatasannya.

Mengapa Melakukan Self-Host NIMs?

Self-hosting NIMs mungkin lebih diutamakan dalam skenario yang membutuhkan:

- **Data Privacy:** Data sensitif yang tidak boleh dikirim melalui internet.-
- Customization:** Memodifikasi atau melakukan fine-tune model untuk use case tertentu.-
- Reduced Latency:** Hosting on-premise menghilangkan delay jaringan.-
- Cost Optimization:** Untuk use case skala besar, self-hosting bisa lebih cost-effective.

18.4.1. Requirement untuk Self-Hosting

Hardware

Model NVIDIA NeMo dioptimalkan untuk GPU. Berikut yang Anda butuhkan:

- **GPU:** NVIDIA GPU dengan dukungan CUDA (sebaiknya A100, H100, atau kartu berperforma tinggi yang serupa).-
- Memory:** Setidaknya 16 GB GPU memory untuk model besar seperti Llama 405B.-
- CPU & RAM:** Prosesor multi-core modern dan setidaknya 32 GB RAM untuk task preprocessing.

Software

- **CUDA Toolkit:** Versi 12.4 atau lebih baru.-
- NVIDIA Container Runtime:** Untuk menjalankan model yang di-containerize.-
- Docker:** Untuk mengelola container.-
- PyTorch:** Sudah terinstal dalam NVIDIA container.-
- Networking:** Akses yang aman untuk mengekspos API atau melakukan integrasi dengan infrastruktur Anda.

File Model

Dapatkan container image untuk model tersebut:

- Kunjungi NVIDIA NGC Catalog (<https://catalog.ngc.nvidia.com>).-
Unduh container model yang sesuai (misalnya, meta/llama-3.1-405b-instruct).

18.4.2. Pola Arsitektur untuk Self-Hosting

Self-hosting umumnya mengikuti arsitektur microservices. Berikut adalah setup yang umum:

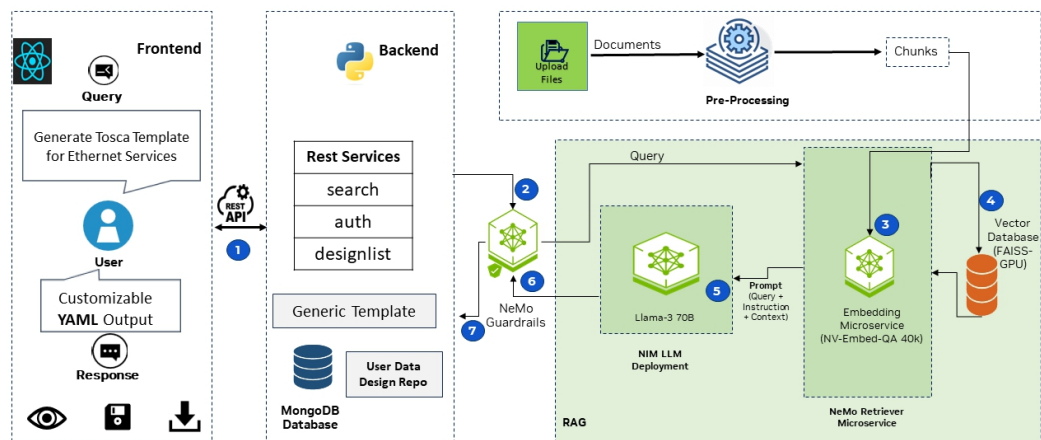
Komponen

1. **Model Server:** Hosts the NIM and serves inference requests.
2. **API Gateway:** Exposes a unified API to users or applications.
3. **Load Balancer:** Distributes requests across multiple instances for scalability.
4. **Monitoring Tools:** Tracks GPU utilization, latency, and API throughput.
5. **Storage:** Manages model weights, configurations, and logs.

Opsi Deployment

- **On-Premises:** Instal dan jalankan container di data center lokal Anda.-
Cloud Providers: Lakukan deploy menggunakan layanan seperti AWS, Azure, atau GCP dengan instance GPU.-
Hybrid Cloud: Kombinasikan resource on-premise dengan skalabilitas cloud untuk peak demand.

Diagram: Contoh Arsitektur Self-Hosting



Ref: <https://developer.nvidia.com/blog/automating-telco-network-design-using-nvidia-nim-and-nvidia-nemo/>

18.4.3. Langkah-Langkah Deployment

Langkah 1: Pull Docker Container

```
terminal  
docker pull nvcr.io/nvidia/pytorch:24.10-py3
```

Langkah 2: Jalankan Container

```
terminal  
docker run --gpus all -it --rm --shm-size=16g --ulimit memlock=-1 --ulimit tack=67108864  
nvcr.io/nvidia/pytorch:24.10-py3
```

Ini akan meluncurkan environment yang dioptimalkan untuk menjalankan NIMs dengan semua dependency yang diperlukan sudah terinstal sebelumnya.

Langkah 3: Host Model sebagai API

Di dalam container:

Clone repository NIM:

```
terminal  
git clone https://github.com/NVIDIA/NeMo.git  
cd NeMo
```

Jalankan model service:

```
terminal  
python examples/nlp/language_modeling/serving.py \  
    --model_name "meta/llama-3.1-405b-instruct" \  
    --port 5000
```

Langkah 4: Mengakses Model yang Di-host

Gunakan tool seperti curl atau Postman untuk mengirim permintaan inference:

```
terminal  
curl -X POST http://localhost:5000/inference \  
    -H "Authorization: Bearer <API_KEY>" \  
    -d '{  
        "text": "Hello, world!"  
    }'
```

```
-H "Content-Type: application/json" \  
-d '{"prompt": "Write a blog post on AI advancements."}'
```

18.4.4. Biaya dan Keterbatasan

Biaya

1. **Hardware Costs:** High-performance GPUs can be expensive. A high-performance GPU like an NVIDIA A100, designed for data centers, can cost upwards of **USD \$30,000** per unit, making them a significant investment for any organization looking to implement GPU-accelerated computing for tasks like machine learning or deep learning; this high price tag is due to their powerful processing capabilities and complex design needed to handle demanding workloads in data centers.
2. **Energy Costs:** GPUs consume significant power.
3. **Cloud Hosting Costs:** Varies by provider and instance type (e.g., AWS EC2 P4d).

Provider	GPU Options	Hourly Pricing	Approx. Monthly Pricing	Regions
AWS	NVIDIA T4, V100, A100, H100	V100: ~\$3.06	V100: ~\$2,203	Global
Google Cloud	NVIDIA T4, P100, V100, A100	V100: ~\$2.48	V100: ~\$1,805	Global
Microsoft Azure	NVIDIA V100, A100, H100	H100: ~\$6.98	H100: ~\$5,081	Global
Paperspace	NVIDIA A100, A6000, A5000, etc.	A100 40GB: ~\$2.39	A100 40GB: ~\$1,720	U.S., Europe
Lambda Labs	NVIDIA RTX 6000, A6000	RTX 6000: ~\$1.25	RTX 6000: ~\$900	U.S.
CoreWeave	NVIDIA RTX 4000, A100, etc.	A100 40GB: ~\$2.46	A100 40GB: ~\$1,771	U.S.
OVHcloud	NVIDIA V100, A100	A100 80GB: ~\$3.07	A100 80GB: ~\$2,213	Europe, Global
Linode	NVIDIA RTX 6000	Competitive rates*	Competitive rates*	North America, Europe, Asia
Jarvis Labs	NVIDIA RTX 5000, RTX 6000, A100	RTX 5000: ~\$0.49	RTX 5000: ~\$353	U.S.
Hyperstack	NVIDIA H100 SXM 80GB	H100 SXM 80GB: ~\$2.25	H100 SXM 80GB: ~\$1,620	North America, Europe

4. **Operational Costs:** Maintenance, monitoring, and scaling can consume time and resources.

Keterbatasan

1. **Setup Complexity:** Requires expertise in containerization and server management.
2. **Resource Usage:** High memory and compute demands.
3. **Scaling Challenges:** For large-scale applications, infrastructure must support distributed systems.

Manfaat Self-Hosting

- **Full Control:** Sesuaikan deployment dan optimalkan performa.
- **Privacy:** Data tidak pernah keluar dari environment yang Anda kendalikan.
- **Flexibility:** Modifikasi model atau integrasikan ke dalam workflow custom.

18.4 Membangun Campaign Generator Bertenaga AI dengan LangGraph

Pada bagian ini, kita akan membangun workflow LangGraph untuk mengorkestrasi campaign generation agent yang menggabungkan content creation dan digital illustration. Kita akan mengikuti pendekatan berbasis node yang terstruktur dengan edge dan state transition yang jelas.

Pernyataan Masalah

Kampanye pemasaran sangat penting untuk mempromosikan produk, layanan, atau event. Sebagai social media manager atau digital marketer, membuat kampanye yang kohesif seringkali melibatkan:

- Menyusun teks promosi yang menarik dan sesuai dengan target audience.- Mendesain grafis yang menarik secara visual dan disesuaikan untuk berbagai platform media sosial.- Mengoordinasikan upaya-upaya ini untuk memastikan konsistensi dan efisiensi di seluruh konten.

Proses ini bisa memakan waktu, repetitif, dan rentan terhadap creative bottleneck, terutama ketika deadline sangat ketat.

Solusi yang Diusulkan

Dengan menggunakan LangGraph, kita akan mengorkestrasi campaign generator bertenaga AI yang powerful, memanfaatkan kemampuan NVIDIA NIMs dan model Stable Diffusion. Solusi ini akan mengotomatiskan dan menyederhanakan pembuatan kampanye dengan cara:

1. **Generating Promotional Text:** Using NVIDIA's pre-trained language model (e.g., [meta/llama-3.1-405b-instruct](#)), the system will craft tailored promotional messages, including catchy phrases, hashtags, and call-to-actions.
2. **Creating Digital Artwork:** NVIDIA's Stable Diffusion model will generate stunning visuals aligned with the campaign theme, such as product images, banners, and illustrations.
3. **Seamlessly Orchestrating Workflow:** LangGraph will connect these tools into a cohesive workflow, ensuring that the text and graphics are aligned and created efficiently.
4. **Providing a User-Friendly Interface:** A Gradio-powered interactive web application will allow users to input campaign details, preview results, and iterate as needed.

Fitur Utama

1. **Content Agent:** Automates the creation of marketing copy, including headlines, descriptions, and hashtags tailored to the input.
2. **Digital Artist Agent:** Generates professional visuals based on the campaign theme and product specifications.
3. **LangGraph Workflow:** Seamlessly integrates AI tools, manages data flow, and ensures step-by-step execution of tasks.
4. **Gradio Interface:** Provides a web-based application for interacting with the system, allowing users to enter campaign details, review outputs, and request modifications.

Kami menggunakan tool dan teknologi berikut:

- **LangGraph:** Untuk mengorquestrasi workflow.- **Gradio:** Untuk membangun antarmuka yang ramah pengguna.- **Python Libraries:** Termasuk requests untuk panggilan API, PIL untuk pemrosesan gambar, dan logging untuk debugging.- **LangChain**
- **NVIDIA AI Endpoints:** Untuk berinteraksi dengan NIMs milik NVIDIA.

Langkah 1: Konfigurasi dan Logging

```
#lesson18d.py

from dataclasses import dataclass
from datetime import datetime
import logging
# Configure logging with timestamps
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.StreamHandler(),
        logging.FileHandler(f'campaign_generator_{datetime.now().strftime("%Y%m%d")}.log')
    ]
)

@dataclass
class CampaignConfig:
    """Configuration settings for campaign generation."""
```

```

    STABLE_DIFFUSION_API_URL: str =
"https://ai.api.nvidia.com/v1/genai/stabilityai/stable-diffusion-3-medium"
    IMAGE_CFG_SCALE: float = 7.0
    IMAGE_STEPS: int = 50
    ASPECT_RATIO: str = "16:9"
    SEED: int = 0
    LLM_MODEL: str = "meta/llama-3.1-405b-instruct"

    MODEL_TEMPERATURE: float = 0.7

```

Langkah 2: Content Generation

Kita membuat function untuk berinteraksi dengan language model milik NVIDIA menggunakan LangChain NVIDIA AI Endpoints:

```

#lesson18d.py continued

from langchain_nvidia_ai_endpoints import ChatNVIDIA
class CampaignGenerator:
    def __init__(self, api_key: Optional[str] = None):
        self.api_key = api_key or os.getenv("NVIDIA_API_KEY")
        if not self.api_key:
            raise ValueError("NVIDIA API key not found in environment
variables")
        self.config = CampaignConfig()
        self.llm = ChatNVIDIA(
            model=self.config.LLM_MODEL,
            api_key=self.api_key,
            temperature=self.config.MODEL_TEMPERATURE
        )
    def generate_content(self, brief: str) -> str:
        """Generate marketing content."""
        content_prompt = f"""
        Create professional marketing content for this campaign:
        {brief}
        Provide:
        - Headline (10 words max)
        - Main copy (2-3 sentences)
        - Call-to-action
        - Three hashtags
        """

```

```
response = self.llm.invoke([{"role": "user", "content": content_prompt}])
return response.content
```

Langkah 3: Menghasilkan Visual

API Stable Diffusion milik NVIDIA digunakan untuk membuat gambar kampanye:

```
#lesson18c.py

import requests
from PIL import Image
from io import BytesIO
def generate_stable_diffusion_image(self, prompt: str) -> Image.Image:
    """Generate an image using NVIDIA's Stable Diffusion API."""
    payload = {
        "prompt": prompt,
        "cfg_scale": self.config.IMAGE_CFG_SCALE,
        "aspect_ratio": self.config.ASPECT_RATIO,
        "seed": self.config.SEED,
        "steps": self.config.IMAGE_STEPS,
        "negative_prompt": "",
    }
    response = requests.post(
        self.config.STABLE_DIFFUSION_API_URL,
        headers={"Authorization": f"Bearer {self.api_key}"},
        json=payload
    )
    response.raise_for_status()
    image_data = response.json().get("image")
    if not image_data:
        raise ValueError("No image data in response")
    return Image.open(BytesIO(base64.b64decode(image_data)))
```

Langkah 4: Orkestrasi Workflow

Kita menggunakan generator function untuk mengorkestrasi workflow langkah demi langkah:

```
#lesson18c.py continued

def stream_campaign(self, brief: str) -> Generator[Tuple[str, str, Optional[Image.Image]],
None, None]:
    """Stream the campaign generation process."""
```

```

try:
    # Step 1: Generate content
    content = self.generate_content(brief)
    yield content, "Generating image prompt...", None
    # Step 2: Generate image prompt
    image_prompt = f"Create a visual prompt based on this content: {content}"
    yield content, image_prompt, None
    # Step 3: Generate image
    image = self.generate_stable_diffusion_image(image_prompt)
    yield content, image_prompt, image
except Exception as e:
    yield f"Error: {str(e)}", "", None

```

Langkah 5: Gradio Interface

Kita membuat web interface yang ramah pengguna:

```

#lesson18c.py continued

import gradio as gr
def create_gradio_interface():
    generator = CampaignGenerator()
    with gr.Blocks(title="AI Campaign Generator") as demo:
        gr.Markdown("# AI Campaign Generator")
        input_text = gr.Textbox(label="Campaign Brief", placeholder="Describe your campaign goals...", lines=5)
        submit_btn = gr.Button("Generate Campaign")
        content_output = gr.Textbox(label="Generated Content", lines=8)
        prompt_output = gr.Textbox(label="Image Generation Prompt", lines=4)
        image_output = gr.Image(label="Generated Campaign Visual", type="pil")
        submit_btn.click(
            fn=generator.stream_campaign,
            inputs=input_text,
            outputs=[content_output, prompt_output, image_output]
        )
    return demo
if __name__ == "__main__":
    demo = create_gradio_interface()
    demo.queue()
    demo.launch()

```

Kode lengkapnya menggabungkan semua komponen ini menjadi satu

```
#lesson18e.py complete code example

import os
import uuid
import requests
import base64
from PIL import Image
from io import BytesIO
import tempfile
from pathlib import Path
from typing import Generator, Optional, Tuple
from langchain_nvidia_ai_endpoints import ChatNVIDIA
import gradio as gr
import logging
from dataclasses import dataclass
from datetime import datetime
# Configure logging with timestamp
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.StreamHandler(),
        logging.FileHandler(f'campaign_generator_{datetime.now().strftime("%Y%m%d")}.log')
    ]
)
logger = logging.getLogger(__name__)
@dataclass
class CampaignConfig:
    """Configuration settings for campaign generation."""
    STABLE_DIFFUSION_API_URL: str = "https://ai.api.nvidia.com/v1/genai/stabilityai/stable-diffusion-3-medium"
    IMAGE_CFG_SCALE: float = 7.0
    IMAGE_STEPS: int = 50
    ASPECT_RATIO: str = "16:9"
    SEED: int = 0
    MODEL_TEMPERATURE: float = 0.7
    LLM_MODEL: str = "meta/llama-3.1-405b-instruct"
class CampaignGenerator:
```

```

"""Main class for handling campaign generation."""
def __init__(self, api_key: Optional[str] = None):
    self.api_key = api_key or os.getenv("NVIDIA_API_KEY")
    if not self.api_key:
        raise ValueError("NVIDIA API key not found in environment variables")
    self.config = CampaignConfig()
    self.headers = {
        "Authorization": f"Bearer {self.api_key}",
        "Accept": "application/json",
    }
    # Initialize LLM
    self.llm = ChatNVIDIA(
        model=self.config.LLM_MODEL,
        api_key=self.api_key,
        temperature=self.config.MODEL_TEMPERATURE
    )
    # Set up temp directory for image storage
    self.temp_dir = Path(tempfile.gettempdir()) / "campaign_generator"
    self.temp_dir.mkdir(exist_ok=True)
def cleanup_temp_files(self):
    """Clean up temporary image files."""
    try:
        for file in self.temp_dir.glob("*.png"):
            file.unlink()
    except Exception as e:
        logger.warning(f"Failed to clean up temp files: {e}")
def generate_stable_diffusion_image(self, prompt: str) -> Image.Image:
    """Generate an image using NVIDIA's Stable Diffusion API."""
    try:
        payload = {
            "prompt": prompt,
            "cfg_scale": self.config.IMAGE_CFG_SCALE,
            "aspect_ratio": self.config.ASPECT_RATIO,
            "seed": self.config.SEED,
            "steps": self.config.IMAGE_STEPS,
            "negative_prompt": "",
        }
        logger.debug(f"Sending image generation request with prompt:
{prompt[:100]}...")

```

```

        response = requests.post(
            self.config.STABLE_DIFFUSION_API_URL,
            headers=self.headers,
            json=payload,
            timeout=30
        )
        response.raise_for_status()
        image_data = response.json().get("image")
        if not image_data:
            raise ValueError("No image data in response")
        image_bytes = base64.b64decode(image_data)
        return Image.open(BytesIO(image_bytes))
    except requests.RequestException as e:
        logger.error(f"API request failed: {e}")
        raise
    except Exception as e:
        logger.error(f"Image generation failed: {e}")
        raise

def stream_campaign(self, brief: str) -> Generator[Tuple[str, str, Optional[Image.Image]],
None, None]:
    """Stream the campaign generation process step by step."""
    session_id = str(uuid.uuid4())
    logger.info(f"Starting campaign generation session {session_id}")
    try:
        # Step 1: Generate content
        logger.info("Generating marketing content...")
        yield "Generating marketing content...", "", None
        content_prompt = f"""
        Create professional marketing content for the following campaign brief:
        {brief}
        Provide:
        1. Headline: Attention-grabbing, max 10 words
        2. Main copy: 2-3 compelling sentences
        3. Call to action: Clear and actionable
        4. Three relevant hashtags
        Format the response with clear section headers and spacing.
        """
        content_response = self.llm.invoke([{"role": "user", "content": content_prompt}])
        content = content_response.content
        logger.debug(f"Generated content: {content[:100]}...")

```



```

        yield content, "Generating image prompt...", None
        # Step 2: Generate image prompt
        logger.info("Generating image prompt...")
        image_prompt_template = f"""
        Create a detailed image generation prompt based on this marketing content:
        {content}
        Focus on:
        - Visual style and mood
        - Key elements and composition
        - Color scheme
        - Artistic direction
        Format: Create a clear, detailed prompt suitable for Stable Diffusion.
        Avoid any text, logos, or trademarked elements.
        Keep the prompt focused on visual elements only.
        """
        prompt_response = self.llm.invoke([{"role": "user", "content":
image_prompt_template}])
        image_prompt = prompt_response.content
        logger.debug(f"Generated image prompt: {image_prompt[:100]}...")
        yield content, image_prompt, None
        # Step 3: Generate image
        logger.info("Generating image...")
        yield content, image_prompt + "\n\nGenerating campaign visual...", None
        image = self.generate_stable_diffusion_image(image_prompt)
        # Save image temporarily
        temp_path = self.temp_dir / f"campaign_{session_id}.png"
        image.save(temp_path)
        logger.info(f"Saved generated image to {temp_path}")
        yield content, image_prompt, image
    except Exception as e:
        logger.error(f"Campaign generation failed: {e}")
        yield f"Error: {str(e)}", "", None
def create_gradio_interface() -> gr.Blocks:
    """Create and configure the Gradio interface."""
    generator = CampaignGenerator()
    with gr.Blocks(title="AI Campaign Generator") as demo:
        gr.Markdown("""
        # ?? AI Campaign Generator

        Generate professional marketing campaigns with AI-powered content and visuals.

        ### How to use:

```

1. Enter your campaign brief including:

- Campaign goals
- Target audience
- Key messages
- Brand voice/tone

2. Click 'Generate Campaign' and watch as your campaign comes to life!

```
""")
```

```
with gr.Row():
```

```
    with gr.Column(scale=1):
```

```
        input_text = gr.Textbox(
```

```
            label="Campaign Brief",
```

```
            placeholder="Describe your campaign goals, target audience, and key
```

```
messages...",
```

```
            lines=5
```

```
        )
```

```
        submit_btn = gr.Button(" ?? Generate Campaign", variant="primary")
```

```
    with gr.Column(scale=1):
```

```
        content_output = gr.Textbox(
```

```
            label="Generated Content",
```

```
            lines=8,
```

```
            interactive=False
```

```
        )
```

```
        prompt_output = gr.Textbox(
```

```
            label="Image Generation Prompt",
```

```
            lines=4,
```

```
            interactive=False
```

```
        )
```

```
        image_output = gr.Image(
```

```
            label="Generated Campaign Visual",
```

```
            type="pil"
```

```
    )
```

```
# Add examples
```

```
gr.Examples(
```

```
    examples=[
```

```
        ["Create a campaign for a new eco-friendly reusable water bottle that keeps  
drinks cold for 24 hours. Target audience: environmentally conscious millennials who are active  
and health-focused."],
```

```
        ["Launch a mobile app that helps people learn meditation through short,  
guided sessions. Target audience: busy professionals aged 25-40 who are interested in  
mindfulness but struggle to find time."],
```

```

        ["Promote a subscription box service for organic, locally-sourced ingredients
with recipe cards. Target audience: home cooks who value quality ingredients and want to
support local farmers."],
        inputs=input_text
    )
    submit_btn.click(
        fn=generator.stream_campaign,
        inputs=input_text,
        outputs=[content_output, prompt_output, image_output],
        api_name="generate"
    )
    gr.Markdown("""
    ### Notes:
    - Content generation typically takes 10-15 seconds
    - Image generation may take up to 30 seconds
    - All generated content is temporary and will be deleted after the session
    """)
    return demo
if __name__ == "__main__":
    # Create and launch the interface
    demo = create_gradio_interface()
    # Launch with queue for better handling of multiple requests
    demo.queue()
    demo.launch(
        share=False, # Set to True to create a public link
        debug=True,
        show_api=False
    )

```

Outputnya terlihat seperti di bawah ini, berjalan pada <http://127.0.0.1:7860/>

AI Campaign Generator

Generate professional marketing campaigns with AI-powered content and visuals.

How to use:

1. Enter your campaign brief including:

- Campaign goals
- Target audience
- Key messages
- Brand voice/tone

2. Click 'Generate Campaign' and watch as your campaign comes to life!

Campaign Brief

Create a marketing campaign based on a beginner friendly book on AI titled The Complete LangGraph Blueprint: Build 50+ AI Agents for Business Success. Goal is to create an audience of developers. Key message is to show how easy it is to create AI agents using langgraph. Tone should be engaging.

Generate Campaign

Generated Content

Introducing 50+ AI Agents that drive business success. Say goodbye to complexity and hello to effortless AI development.

****Call to Action:****

"Get Your Copy Now and Start Building AI Agents in Minutes!"

[Insert CTA button: Buy Now / Download Free Chapter / Learn More]

****Hashtags****

#AIforBusiness
#LangGraphSimplified
#DeveloperSuccess

This campaign content aims to engage developers and position The Complete LangGraph Blueprint as a go-to resource for building AI agents with ease. The tone is approachable, and the language is concise, making it perfect for a beginner-friendly book on AI.

Image Generation Prompt

Generate an image of a futuristic AI control room with a glowing brain structure in the center, surrounded by data screens and a blue ambient light.

* The lighting is soft and diffused, with subtle gradient effects that give the image a sense of depth and dimensionality.

Key elements:

- * The glowing brain structure has a mesmerizing, iridescent glow, with shifting colors that evoke a sense of intelligence and cognitive activity.
- * The computer terminals and holographic displays show snippets of code, flowcharts, and AI-related diagrams, emphasizing the technical and scientific aspects of the LangGraph system.
- * A few subtle, stylized lines and shapes in the background suggest a sense of connection to the outside world, implying that the LangGraph system has far-reaching implications and applications.

Artistic direction:

- * The illustration style is reminiscent of futuristic concept art, with clean lines, minimalist shapes, and a focus on conveying complex ideas through simple, intuitive visuals.
- * The image should evoke a sense of wonder, curiosity, and excitement, encouraging the viewer to learn more about the LangGraph system and its potential to unlock AI power.

Output resolution: 1024x768 pixels. Aspect ratio: 4:3. Style: Digital illustration. Mood: Futuristic, innovative.

Generated Campaign Visual



Examples

Create a campaign for a new eco-friendly reusable water bottle that keeps drinks cold for 24 hours. Target audience: environmentally conscious millennials who are active and health-focused.

Launch a mobile app that helps people learn meditation through short, guided sessions. Target audience: busy professionals aged 25-40 who are interested in mindfulness but struggle to find time.

Promote a subscription box service for organic, locally-sourced ingredients with recipe cards. Target audience: home cooks who value quality ingredients and want to support local farmers.

18. 5 Bagian Penjelasan: Konsep Programming Utama dalam Campaign Generator

Bagian penjelasan ini dirancang untuk membantu pemula memahami konsep-konsep dasar yang digunakan dalam campaign generator, termasuk requests, Gradio, Pillow, dan function yield. Setiap konsep dijelaskan secara bertahap, dengan contoh sederhana yang meningkat menjadi skenario yang lebih kompleks.

1. Menggunakan Library requests

Apa itu requests?

Library requests adalah paket Python yang menyederhanakan pembuatan HTTP request. Library ini memungkinkan Anda berkomunikasi dengan API, mengambil konten web, atau mengirim data ke server.

Contoh Dasar: GET Request

GET request mengambil data dari server.

```
#explainer.py
import requests
# Fetch data from a public API
url = "https://api.github.com"
response = requests.get(url)
# Print the response
print("Response Status Code:", response.status_code)
print("Response Data:", response.json())
```

POST Request dengan Payload

POST request mengirim data ke server. Sebagai contoh, mari kita kirim payload ke sebuah API.

```
#explainer.py
import requests
url = "https://httpbin.org/post"
payload = {"name": "John", "age": 30}
# Sending a POST request
response = requests.post(url, json=payload)
# Print the response
print("Response Status Code:", response.status_code)
print("Response Data:", response.json())
```

Contoh Lanjutan: Mengirim Headers dan Menangani Error

Dalam campaign generator, kita menggunakan API key di headers untuk autentikasi.

```
#explainer.py
import requests
```

```

url = "https://example.com/api"
headers = {
    "Authorization": "Bearer YOUR_API_KEY",
    "Content-Type": "application/json",
}
try:
    response = requests.get(url, headers=headers)
    response.raise_for_status() # Raise an error for HTTP codes >= 400
    print("Response Data:", response.json())
except requests.RequestException as e:
    print("An error occurred:", e)

```

Poin-Poin Penting

- GET digunakan untuk mengambil data, sedangkan POST digunakan untuk mengirim data.- Selalu tangani error dengan baik menggunakan try-except.- Gunakan headers untuk menyertakan autentikasi atau metadata lainnya.

2. Membangun User Interface dengan Gradio

Apa itu Gradio?

Gradio adalah library Python untuk membangun user interface interaktif bagi model machine learning atau script. Library ini memungkinkan pengguna memasukkan data dan melihat hasilnya secara real time.

Contoh Dasar: Interface Sederhana

Mari kita bangun UI untuk memasukkan teks dan menampilkannya kembali.

```

#explainer.py

import gradio as gr
def reverse_text(text):
    """Reverses the input text."""
    return text[::-1]
# Create a simple Gradio interface
interface = gr.Interface(
    fn=reverse_text, # Function to run
    inputs="text", # Input type
    outputs="text" # Output type
)

```

```
interface.launch() # Start the interface
```

Contoh Menengah: Menambahkan Multiple Input dan Output

Perluas fungsionalitas dengan menerima nama dan usia, lalu menghasilkan pesan sambutan.

```
#explainer.py

import gradio as gr
def greet(name, age):
    """Generates a greeting message."""
    return f"Hello, {name}! You are {age} years old."
# Create a Gradio interface with two inputs
interface = gr.Interface(
    fn=greet,
    inputs=["text", "number"], # Name as text and age as number
    outputs="text"            # Output as text
)
interface.launch()
```

Contoh Lanjutan: Menambahkan Gambar

Mari kita buat UI yang menerima gambar dan menampilkannya dengan transformasi sederhana (misalnya, mengubah ke grayscale).

```
#explainer.py

import gradio as gr
from PIL import Image
def process_image(image):
    """Converts the image to grayscale."""
    return image.convert("L") # Convert to grayscale
# Create an interface with image input and output
interface = gr.Interface(
    fn=process_image,
    inputs="image",
    outputs="image"
)
interface.launch()
```

Poin-Poin Penting

- **Input/output Gradio dapat menangani berbagai tipe seperti teks, angka, atau gambar.- Anda dapat merangkai fungsi-fungsi bersama untuk membuat workflow yang kompleks.- Library ini menyederhanakan proses deployment model machine learning dengan kode minimal.**

3. Memproses Gambar dengan Pillow

Apa itu Pillow?

Pillow adalah library Python untuk manipulasi gambar. Library ini dapat membuka, memproses, dan menyimpan gambar.

Contoh Dasar: Membuka dan Menyimpan Gambar

Mari kita buka sebuah gambar dan simpan dalam format lain.

```
#explainer.py
from PIL import Image
# Open an image
image = Image.open("example.jpg")
image.show() # Display the image
# Save the image in PNG format
image.save("example.png")
```

Memtransformasi Gambar

Kita dapat mengubah ukuran, memutar, atau menerapkan filter pada gambar.

```
#explainer.py
from PIL import Image, ImageFilter
# Open an image
image = Image.open("example.jpg")
# Apply transformations
resized_image = image.resize((200, 200)) # Resize
rotated_image = image.rotate(45) # Rotate
blurred_image = image.filter(ImageFilter.BLUR) # Apply blur
# Save the transformed images
resized_image.save("resized.png")
rotated_image.save("rotated.png")
```



```
blurred_image.save("blurred.png")
```

Working with In-Memory Images

This is useful when working with APIs that return image data as Base64 strings.

```
from PIL import Image
from io import BytesIO
import base64
# Simulate an API response with Base64 image data
image_data_base64 = "..." # Replace with actual Base64 string
image_data = base64.b64decode(image_data_base64)
# Load the image from memory
image = Image.open(BytesIO(image_data))
image.show()
```

Poin-Poin Penting

- Pillow menyederhanakan penanganan format gambar, transformasi, dan efek.
- Gunakan BytesIO untuk bekerja dengan data gambar in-memory.

4. Menggunakan Fungsi yield

Apa itu yield?

Fungsi yield dalam Python digunakan untuk membuat generator, yang menghasilkan sekumpulan nilai satu per satu. Fungsi ini memungkinkan pemrosesan yang lebih efisien dari segi memori dibandingkan mengembalikan list yang lengkap.

Contoh Dasar: Generator Sederhana

Sebuah generator yang menghasilkan angka dari 1 sampai 5.

```
#explainer.py
def number_generator():
    for i in range(1, 6):
        yield i
# Using the generator
for number in number_generator():
    print(number)
```

Melakukan Yield pada Langkah-Langkah dalam
Sebuah Proses

Simulasikan sebuah proses multi-langkah dengan update progres.

```
#explainer.py
def multi_step_process():
    yield "Step 1: Initializing..."
    yield "Step 2: Processing data..."
    yield "Step 3: Finalizing..."
    yield "Process complete!"
for step in multi_step_process():
    print(step)
```

Streaming Hasil API

Integrasikan yield dengan sebuah API untuk mengembalikan hasil secara progresif.

```
#explainer.py
import requests
def stream_api_results():
    url = "https://api.example.com/stream"
    with requests.get(url, stream=True) as response:
        for chunk in response.iter_lines():
            yield chunk.decode("utf-8")
# Process the streamed results
for result in stream_api_results():
    print(result)
```

Poin-Poin Penting

- yield ideal untuk menangani data yang besar atau bersifat incremental.- Ini memungkinkan update real-time dalam aplikasi, seperti indikator progres atau hasil streaming.

Dengan membangun konsep-konsep ini secara bertahap, kini Anda memiliki alat untuk:

- Melakukan panggilan API dengan requests.- Merancang user interface dengan Gradio.- Memproses gambar dengan Pillow.- Mengimplementasikan workflow dengan yield.

Fondasi ini memungkinkan Anda membangun aplikasi bertenaga AI yang kuat seperti Campaign Generator.

Kuis Bab 18: Membangun Agent dengan NVIDIA NeMo Inference Models (NIMs)

Pertanyaan Kuis

Bagian 1: Pengenalan NIMs

1. What does NIM stand for, and what are its primary capabilities?

- a) NVIDIA Interactive Models; conversational AI only
- b) NVIDIA Integrated Models; data analysis only
- c) NVIDIA NeMo Inference Models; multimodal processing
- d) NVIDIA Network Integration Models; image processing only

2. Which of the following is NOT a key benefit of NIMs?

- a) Pre-trained and easy to use
- b) Requires no hardware to run
- c) Offers scalable APIs
- d) Provides state-of-the-art performance

3. What is the purpose of Build NVIDIA?

- a) To teach programming basics
- b) To provide access to pre-trained AI models and tools
- c) To replace cloud services
- d) To design custom GPUs

4. What label should you look for in Build NVIDIA to identify self-hosted models?

- a) "API Enabled"
- b) "Run Anywhere"
- c) "GPU Required"
- d) "Open Source"

Bagian 2: Melakukan Inference dengan NVIDIA NIMS

5. Which Python library is commonly used for making API calls to NIMs?

- a) Gradio
- b) requests
- c) Pillow
- d) LangChain

6. When making an API call, what is the purpose of the **Authorization header?**

- a) To format the response
- b) To authenticate the request
- c) To specify the model type
- d) To set the response time

7. How does LangChain simplify working with NIMs?

- a) By adding GPU support
- b) By reducing code complexity for workflows
- c) By improving API authentication
- d) By automating model training

Bagian 3: Self-Hosting NIMs

8. Which of the following is NOT a requirement for self-hosting NIMs?

- a) NVIDIA GPUs with CUDA support
- b) A fast internet connection
- c) Docker container runtime
- d) At least 32 GB of RAM

9. What is one key advantage of self-hosting NIMs?

- a) Eliminates all hardware costs
- b) Provides full control and data privacy
- c) Simplifies scaling for small applications
- d) Requires minimal technical expertise

10. Which tool is used to pull the Docker container for hosting NIMs?

- a) LangChain
- b) Gradio
- c) Docker CLI
- d) Pillow

Bagian 4: AI-Powered Campaign Generator

11. What role does LangGraph play in the campaign generator?

- a) It creates the API calls
- b) It orchestrates the workflow using nodes and edges
- c) It trains the models
- d) It serves the Gradio interface

12. What Python library is used for processing and manipulating images in the campaign generator?

- a) requests
- b) Pillow
- c) LangChain
- d) NumPy

13. What does the **yield function enable in the campaign generator?**

- a) Generating a list of results
- b) Real-time updates and memory-efficient processing
- c) Running API calls in parallel
- d) Converting images into Base64 format

Bagian 5: Kasus Penggunaan Praktis

14. What does the Content Agent generate in the campaign generator?

- a) Digital illustrations

- b) Marketing text like headlines and hashtags
- c) APIs for stable diffusion
- d) User input fields

15. What key feature does the Digital Artist Agent provide?

- a) Writes text copy for campaigns
- b) Generates images based on campaign themes
- c) Connects the campaign to LangGraph
- d) Handles API authentication

Jawaban

1. **c) NVIDIA NeMo Inference Models; multimodal processing**
2. **b) Requires no hardware to run**
3. **b) To provide access to pre-trained AI models and tools**
4. **b) "Run Anywhere"**
5. **b) requests**
6. **b) To authenticate the request**
7. **b) By reducing code complexity for workflows**
8. **b) A fast internet connection**
9. **b) Provides full control and data privacy**
10. **c) Docker CLI**
11. **b) It orchestrates the workflow using nodes and edges**
12. **b) Pillow**
13. **b) Real-time updates and memory-efficient processing**
14. **b) Marketing text like headlines and hashtags**
15. **b) Generates images based on campaign themes**

Tips untuk Pembelajar:

- Tinjau kembali cuplikan kode yang disediakan dalam bab ini untuk pemahaman secara langsung.- Uji panggilan API dan workflow LangChain sendiri untuk memperdalam pemahaman Anda.

- Berlatih membuat interface Gradio sederhana untuk membangun alat AI yang interaktif.

Part 6: Testing, Deployment, and Performance
Optimization
(Chapters 19-21)



Chapter 19

Testing AI Agents with Test-Driven Development (TDD)

Pendahuluan

Testing adalah fondasi dalam membangun AI agent yang andal. Test-Driven Development (TDD) membawa hal ini selangkah lebih jauh dengan menekankan penulisan test sebelum mengembangkan fungsionalitas yang sebenarnya. Bab ini memperkenalkan konsep TDD, memandu Anda langkah demi langkah dalam penerapannya pada AI agent, khususnya menggunakan LangGraph. Dengan memulai dari yang sederhana dan berkembang ke workflow yang kompleks, Anda akan memperoleh pengalaman langsung dalam merancang sistem AI yang andal dan dapat diuji (testable).

19.1 Memahami Test-Driven Development (TDD)

TDD adalah pendekatan pengembangan software di mana test ditulis sebelum kode. Prosesnya mengikuti siklus sederhana:

1. **Write a Failing Test:** Start by defining a test case based on requirements.
2. **Write the Code:** Develop the functionality to make the test pass.
3. **Refactor:** Improve the code while ensuring all tests still pass.

Siklus iteratif ini memastikan bahwa kode Anda selalu dipandu oleh requirement yang jelas dan pada dasarnya dapat diuji (testable).

19.2 Persiapan Project Sebelum kita masuk ke TDD, mari kita siapkan environment pengembangan yang tepat. Persiapan Environment:

1. First, create a new Python virtual environment:

```
terminal
mkdir customer_support
cd customer_support
mkdir tests
mkdir src
```

```
#windows
type nul > requirements.txt
type nul > setup.py
type nul > pytest.ini
type nul > .env

#mac/linux
touch requirements.txt
touch setup.py
touch pytest.ini
touch .env

#open project in VSCode or Cursor
code .
```

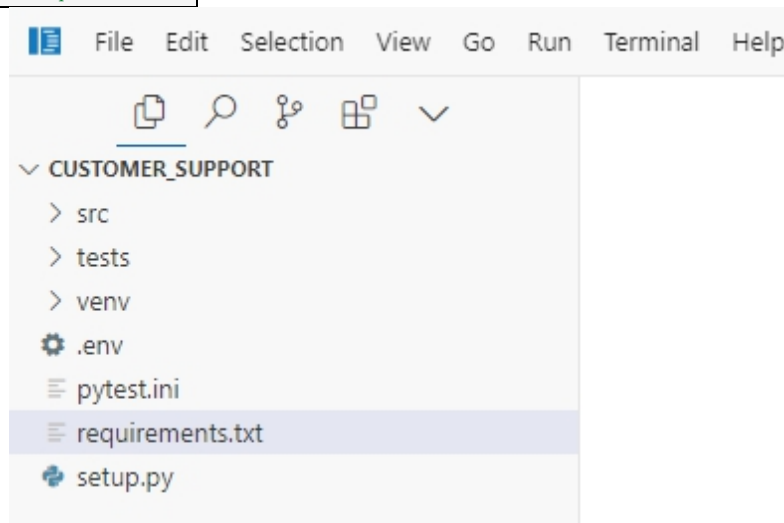
2. Add the following to requirements.txt:

```
#requirements.txt

pytest
pytest-asyncio
langchain
python-dotenv
langgraph
langchain_openai
setuptools
```

3. Install the requirements

```
Terminal
pip install -r requirements.txt
```



4. Create a basic setup.py:

```
#setup.py

from setuptools import setup, find_packages

setup(
    name="customer_support",
    version="0.1",
    packages=find_packages(),
```

```

install_requires=[
    "pytest",
    "pytest-asyncio",
    "langchain",
    "openai",
    "python-dotenv",
    "langgraph"
],
)

```

5. Add configuration to pytest.ini

```

# pytest.ini
[pytest]
testpaths = tests
python_files = test_*.py
python_functions = test_*
asyncio_mode = auto

```

6. Set up your .env file:

```

# .env
OPENAI_API_KEY=your_api_key_here

```

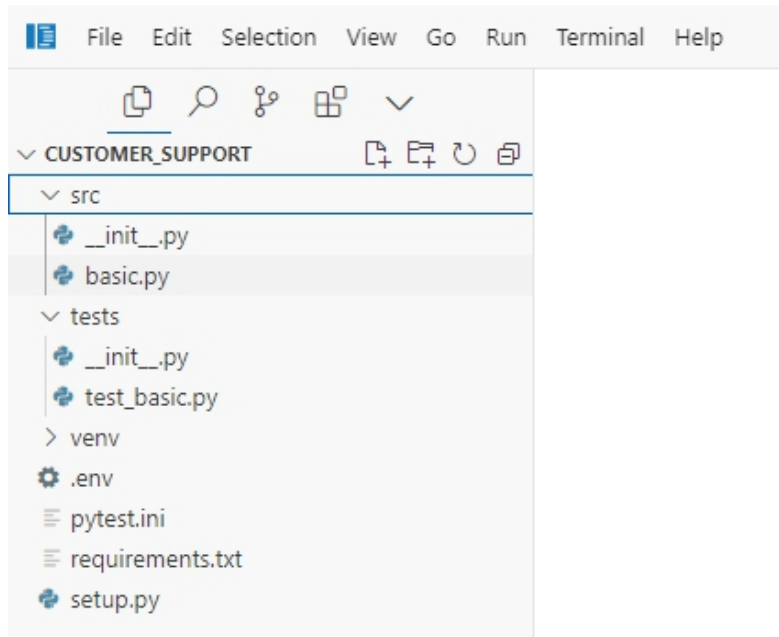
7. Create a basic test file structure:

```

terminal

#windows
type nul > tests/__init__.py
type nul > tests/test_basic.py
type nul > src/__init__.py
type nul > src/basic.py
#mac linux
touch tests/__init__.py
touch tests/test_basic.py
touch src/__init__.py
touch src/basic.py

```



19.3 Memahami Siklus TDD

Test-Driven Development mengikuti siklus yang sederhana namun powerful:

1. Tulis test yang gagal (failing test)
2. Tulis kode minimal untuk melewati test tersebut
3. Lakukan refactor sambil menjaga test tetap hijau (green)

Mari kita mulai dengan contoh sederhana untuk mengilustrasikan siklus ini.

Contoh 1: Node Pemrosesan Pesan Dasar

1. Define tests/test_basic.py as below

```
#tests/test_basic.py
# test_message_processor.py
def test_message_processor():
    # Step 1: Write the failing test
    state = {"input": "Hello world", "word_count": 0}
    result = process_message(state)
    assert result["word_count"] == 2
```

2. Run the test by running pytest on the command line or terminal.

```
terminal
pytest
output:
(venv) PS D:\customer_support> pytest
```

```

===== test
session starts
=====
platform win32 -- Python 3.13.0, pytest-7.4.3, pluggy-1.5.0
rootdir: D:\customer_support
configfile: pytest.ini
testpaths: tests
plugins: anyio-3.7.1, asyncio-0.21.1
asyncio: mode=Mode.AUTO
collected 1 item
tests\test_basic.py
F [100%]
=====
FAILURES
=====
_____ test_message_processor _____

    def
test_message_processor():
    inte
        # Step 1: Write the failing
test
        state = {"input": "Hello world", "word_count":
0}
    > result = process_message(state)
E      NameError: name 'process_message' is not
defined
tests\test_basic.py:5:
NameError
=====
short test summary info
=====
FAILED tests\test_basic.py::test_message_processor - NameError: name 'process_message' is not defined
1 failed in 0.07s
=====
(venv) PS D:\customer_support>

```

3. This test will fail because we haven't implemented `process_message` yet. Let's write the minimal code to make it pass in the project file `src/basic.py` file.

```

#src/basic.py
def process_message(state):
    # Step 2: Write minimal passing code
    state["word_count"] = len(state["input"].split())
    return state

```

4. Now let's amend the test to include this file.

```

#tests/test_basic.py

```

```
# tests/test_basic.py
from src.basic import process_message
def test_message_processor():
    # Step 1: Write the failing test
    state = {"input": "Hello world", "word_count": 0}
    result = process_message(state)
    assert result["word_count"] == 2
```

5. And run the test again

```
terminal
pytest
output:
(venv) PS D:\customer_support> pytest
===== test
session starts
=====
platform win32 -- Python 3.13.0, pytest-7.4.3, pluggy-1.5.0
rootdir: D:\customer_support
configfile: pytest.ini
testpaths: tests
plugins: anyio-3.7.1, asyncio-0.21.1
asyncio: mode=Mode.AUTO
collected 1 item
tests/test_basic.py
. [100%]
=====
1 passed in 0.03s
=====
(venv) PS D:\customer_support>
```

6. Lets add another test and check for empty input so that our test file now has two tests:

```
#tests/test_basic.py
# tests/test_basic.py
from src.basic import process_message
def test_message_processor():
    # Step 1: Write the failing test
    state = {"input": "Hello world", "word_count": 0}
    result = process_message(state)
    assert result["word_count"] == 2
def test_message_processor_empty():
    state = {"input": "", "word_count": 0}
    result = process_message(state)
    assert result["word_count"] == 0
```

7. Run the test

```
terminal
pytest
```

```

output:
(venv) PS D:\customer_support> pytest
===== test
session starts
=====
platform win32 -- Python 3.13.0, pytest-7.4.3, pluggy-1.5.0
rootdir: D:\customer_support
configfile: pytest.ini
testpaths: tests
plugins: anyio-3.7.1, asyncio-0.21.1
asyncio: mode=Mode.AUTO
collected 2 items
tests\test_basic.py
..
===== [100%]
=====
2 passed in 0.03s
=====

```

19.4 Memahami State Management Saat menguji AI agent, state management yang tepat sangatlah penting. Mari kita jelajahi contoh yang lebih kompleks:

1. Let's define a sample test for state variables

```

#tests/test_conversation_state.py
def test_conversation_memory():
    state = {
        "messages": [],
        "context": {},
        "turn_count": 0
    }
    # Add a message
    state = add_message(state, "user", "Hello")
    assert len(state["messages"]) == 1
    assert state["turn_count"] == 1
    # Add system response
    state = add_message(state, "system", "Hi there!")
    assert len(state["messages"]) == 2
    assert state["turn_count"] == 1

```

2. Run the tests

```

terminal
pytest
output:
(venv) PS D:\customer_support> pytest

```



```

===== test
session starts
=====

platform win32 -- Python 3.13.0, pytest-7.4.3, pluggy-1.5.0
rootdir: D:\customer_support
configfile: pytest.ini
testpaths: tests
plugins: anyio-3.7.1, asyncio-0.21.1
asyncio: mode=Mode.AUTO
collected 3 items
tests\test_basic.py ..
66%]
tests\test_conversation_memory.py
F
[100%]
=====
FAILURES
=====
=

test_conversation_memory
=====

def test_conversation_memory():
    state = {
        "messages": [],
        "context": {},
        "turn_count": 0
    }
    # Add a message
> state = add_message(state, "user", "Hello")
E   NameError: name 'add_message' is not defined
tests\test_conversation_memory.py:10: NameError
=====
short test summary info
=====
FAILED tests\test_conversation_memory.py::test_conversation_memory - NameError: name 'add_message' is
not defined
=====
1 failed, 2 passed in 0.08s
=====

```

3. Let's now implement state management using add_message

```

#src/conversation_state.py
def add_message(state, role, content):
    state["messages"].append({
        "role": role,
        "content": content
    })
    if role == "user":
        state["turn_count"] += 1

```

```
return state
```

4. Update our test with the import

```
#tests/test_conversation_state.py
from src.conversation_memory import add_message
def test_conversation_memory():
    state = {
        "messages": [],
        "context": {},
        "turn_count": 0
    }
    # Add a message
    state = add_message(state, "user", "Hello")
    assert len(state["messages"]) == 1
    assert state["turn_count"] == 1
    # Add system response
    state = add_message(state, "system", "Hi there!")
    assert len(state["messages"]) == 2
    assert state["turn_count"] == 1
```

5. Run the test.

```
terminal
pytest
output:
(venv) PS D:\customer_support> pytest
===== test
session starts
=====
platform win32 -- Python 3.13.0, pytest-7.4.3, pluggy-1.5.0
rootdir: D:\customer_support
configfile: pytest.ini
testpaths: tests
plugins: anyio-3.7.1, asyncio-0.21.1
asyncio: mode=Mode.AUTO
collected 3 items
tests\test_basic.py ..
66%]
tests\test_conversation_memory.py
.
[100%]
===== 3
passed in 0.04s
=====
```

19.5 Menguji Interaksi LLM Saat menguji komponen AI, kita perlu menangani:

Panggilan API ke language
modelVariasi responseRate limit
dan timeoutPertimbangan biaya

Mari kita lihat sebuah contoh praktis.

1. Let's start with a LangChain example of a simple invocation of an LLM

```
#tests/test_llm_classifier.py
# test_product_entities.py
import pytest
from src.llm_classifier import extract_entities, enrich_context
def test_basic_product_extraction():
    """Test basic product and color extraction"""
    state = {
        "input": "What's the price of the blue shirt?",
        "entities": {},
        "context": {}
    }
    result = extract_entities(state)
    assert result["entities"].product == "shirt"
    assert result["entities"].color == "blue"
def test_product_without_color():
    """Test extracting product without color mention"""
    state = {
        "input": "How much is the pants?",
        "entities": {},
        "context": {}
    }
    result = extract_entities(state)
    assert result["entities"].product == "pants"
    assert result["entities"].color is None
def test_color_without_product():
    """Test extracting color without valid product"""
    state = {
        "input": "Do you have anything in blue?",
        "entities": {},
        "context": {}
    }
    result = extract_entities(state)
    assert result["entities"].product is None
    assert result["entities"].color == "blue"
def test_invalid_product():
    """Test with product not in catalog"""
    state = {
        "input": "Looking for a green sofa",
```

```

        "entities": {},
        "context": {}
    }
    result = extract_entities(state)
    assert result["entities"].product is None
    assert result["entities"].color == "green"
def test_context_enrichment():
    """Test context enrichment with product details"""
    # Test valid product
    state = {
        "input": "Price of blue shirt",
        "entities": {},
        "context": {}
    }
    state = extract_entities(state)
    state = enrich_context(state)
    assert "product_details" in state["context"]
    assert state["context"]["product_details"]["name"] == "Classic Shirt"
    assert "blue" in state["context"]["product_details"]["available_colors"]
    assert state["context"]["product_details"]["price"] == 29.99
def test_context_enrichment_invalid_product():
    """Test context enrichment with invalid product"""
    state = {
        "input": "Looking for a sofa",
        "entities": {},
        "context": {}
    }
    state = extract_entities(state)
    state = enrich_context(state)
    # Context should not have product details for invalid product
    assert "product_details" not in state["context"] or not state["context"]["product_details"]
def test_multiple_products():
    """Test handling multiple product mentions"""
    state = {
        "input": "Compare the blue shirt and black pants",
        "entities": {},
        "context": {}
    }
    result = extract_entities(state)
    # Should pick up first product mentioned
    assert result["entities"].product in ["shirt", "pants"]
    assert result["entities"].color in ["blue", "black"]
def test_edge_cases():
    """Test various edge cases"""
    # Empty input
    state = {

```

```

        "input": "",
        "entities": {},
        "context": {}
    }
    result = extract_entities(state)
    assert result["entities"].product is None
    assert result["entities"].color is None
    # Very long input
    state = {
        "input": "I'm looking for a " + "very " * 50 + "blue shirt",
        "entities": {},
        "context": {}
    }
    result = extract_entities(state)
    assert result["entities"].product == "shirt"
    assert result["entities"].color == "blue"
    # Special characters
    state = {
        "input": "Looking for a blue shirt!!???",
        "entities": {},
        "context": {}
    }
    result = extract_entities(state)
    assert result["entities"].product == "shirt"
    assert result["entities"].color == "blue"
if __name__ == "__main__":
    pytest.main([__file__, "-v"])

```

2. Let's implement the LLM classifier

```

#src/llm_classifier.py
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel
from typing import Optional
class ProductEntity(BaseModel):
    product: Optional[str] = None
    color: Optional[str] = None
def lookup_product(product_name):
    """Look up product details from database"""
    catalog = {
        "shirt": {
            "name": "Classic Shirt",
            "available_colors": ["red", "blue", "white", "black"],
            "price": 29.99
        },
        "pants": {
            "name": "Casual Pants",

```

```

        "available_colors": ["black", "navy", "khaki"],
        "price": 49.99
    },
    "jacket": {
        "name": "Winter Jacket",
        "available_colors": ["black", "green", "brown"],
        "price": 89.99
    }
}

return catalog.get(product_name.lower() if product_name else "", {})

def extract_entities(state):
    """Extract product and color entities using structured output"""
    llm = ChatOpenAI(temperature=0, model="gpt-4o-mini").with_structured_output(ProductEntity)
    prompt = ChatPromptTemplate.from_messages([
        ("system", """Extract the product and color from the query.
        Only extract products from this list: shirt, pants, jacket"""),
        ("human", "{query}")
    ])
    chain = prompt | llm
    state["entities"] = chain.invoke({"query": state["input"]})
    return state

def enrich_context(state):
    """Enrich context with product details"""
    if not state["entities"] or not state["entities"].product:
        state["context"] = {} # Initialize empty context if no product
        return state
    state["context"] = {
        "product_details": lookup_product(state["entities"].product)
    }
    return state

def process_query(query):
    """Process a single query through the pipeline"""
    state = {"input": query}
    state = extract_entities(state)
    state = enrich_context(state)
    return state

def main():
    test_queries = [
        "I want a blue shirt",
        "Do you have any red jackets?",
        "Show me black pants",
        "What colors do you have?",
    ]
    print("Processing queries...\n")
    for query in test_queries:
        print(f"Query: {query}")

```

```

try:
    result = process_query(query)
    print(f"Extracted Entities: {result['entities']}")
    if result['context']:
        print(f"Product Details: {result['context']['product_details']}")
    print()
except Exception as e:
    print(f"Error processing query: {str(e)}\n")
if __name__ == "__main__":
    main()

```

3. Let's run the test

```

terminal
pytest
output:
PS D:\customer_support> pytest
===== test session starts
=====
platform win32 -- Python 3.13.0, pytest-7.4.3, pluggy-1.5.0
rootdir: D:\customer_support
configfile: pytest.ini
testpaths: tests
plugins: anyio-3.7.1, asyncio-0.21.1
asyncio: mode=Mode.AUTO
collected 11 items
tests\test_basic.py ..                               [ 18%]
tests\test_conversation_memory.py .                   [ 27%]
tests\test_llm_classifier.py .....                   [100%]
===== warnings summary
=====
venv\Lib\site-packages\pydantic\v1\typing.py:68: 16 warnings
  D:\customer_support\venv\Lib\site-packages\pydantic\v1\typing.py:68: DeprecationWarning: Failing to pass
a value to the 'type_params' parameter of 'typing.ForwardRef._evaluate' is deprecated, as it leads to incorrect
behaviour when calling typing.ForwardRef._evaluate on a stringified annotation that references a PEP 695
type parameter. It will be disallowed in Python 3.15.
    return cast(Any, type_)._evaluate(globalns, localns, recursive_guard=set())
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 11 passed, 16 warnings in 11.84s
=====

```

Membangun Customer Support AI Agent dengan Test-Driven Development

Pengantar TDD menggunakan LangGraph

Pada bagian praktik ini, kita akan mengimplementasikan Customer Support AI Agent yang sepenuhnya fungsional menggunakan LangGraph dan LangChain, mengikuti metodologi Test-Driven Development (TDD) yang terstruktur. Agent ini akan mendukung tugas-tugas layanan pelanggan yang umum, meliputi:

- **Pertanyaan Status Pesanan- Permintaan Informasi Produk- Penanganan Percakapan Umum- Penanganan Error untuk Input yang Hilang atau Tidak Valid**

Pada akhir bagian ini, Anda akan memiliki AI agent yang andal dan teruji secara menyeluruh, siap untuk deployment.

Langkah 1: Penyiapan Proyek

Sebelum masuk ke proses TDD, pastikan environment Anda sudah dikonfigurasi dengan benar.

Struktur Direktori

Berikut struktur direktori yang direkomendasikan untuk proyek ini:

```
customer_support/??? src/? ??? __pycache__/? ??? support_agent.py
# Implementasi customer support AI agent??? tests/? ???
__pycache__/? ??? test_support_agent.py # Test TDD untuk semua
node dan workflow??? .env # Environment variables,
misalnya OpenAI API key??? pytest.ini # Konfigurasi Pytest???
readme.md # Dokumentasi proyek??? requirements.txt
# Dependencies??? setup.py # File setup package
```

Langkah-Langkah Instalasi

Clone repository:

```
terminal
git clone https://github.com/jkmaina/customer_support.git
cd customer_support
```

Install dependencies:

```
terminal
pip install -r requirements.txt
```

Atur OpenAI API key Anda di file .env:

```
.env
OPENAI_API_KEY=your-key-here
```

Langkah 2: Memahami Workflow

Agent ini terdiri dari node-node berikut:

1. **Intent Classification:** Determines the user's query intent.
2. **Order Status Node:** Responds to order-related queries.
3. **Product Inquiry Node:** Provides product details.
4. **General Chat Node:** Handles casual conversation and unknown intents.

Workflow ini dikelola menggunakan StateGraph, dengan routing kondisional berdasarkan intent yang terdeteksi.

Langkah 3: Test-Driven Development untuk Node

TDD melibatkan penulisan test yang gagal terlebih dahulu, mengimplementasikan kode minimal yang diperlukan untuk lolos test, kemudian melakukan refactoring. Di bawah ini, kita mengikuti proses tersebut untuk setiap node.

Node 1: Intent Classifier

Langkah 1: Menulis Test yang Gagal

```
#test_support_agent.py
def test_intent_classifier():
    """Test the intent classifier node"""
    state = {
        "messages": [HumanMessage(content="Where is my order #12345?")],
        "intent": "",
        "status": ""
    }
    result = intent_classifier_node(state)
    assert result["intent"] == IntentEnum.CHECK_ORDER
```

- Hasil yang Diharapkan: Test akan gagal karena fungsi `intent_classifier_node` belum diimplementasikan.

Langkah 2: Mengimplementasikan Kode

```
#support_agent.py
class CustomerState(TypedDict):
    """State for customer support workflow"""
    messages: Annotated[Sequence[BaseMessage], operator.add]
    intent: str
    status: str
def intent_classifier_node(state: CustomerState):
    """Classify customer intent using LLM"""
    llm = ChatOpenAI(model="gpt-4o-mini")
    classifier_chain = intent_prompt | llm
    query = state["messages"][-1].content
    intent = classifier_chain.invoke({"query": query}).content.strip().lower()
    state["intent"] = intent
    return state
from enum import Enum
class IntentEnum(str, Enum):
    CHECK_ORDER = "check_order"
    PRODUCT_INQUIRY = "product_inquiry"
    ESCALATE = "escalate"
```

```

UNKNOWN = "unknown"
intent_prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a customer support intent classifier.
    Classify the user query into one of these intents: check_order, product_inquiry,
    escalate, or unknown. Respond with just the intent."""),
    ("human", "{query}")
])

```

Langkah 3: Refactor (jika diperlukan)

Untuk saat ini, tidak diperlukan refactoring. Jika intent tambahan diperkenalkan, Anda mungkin perlu mengabstraksi logika ke dalam modul yang dapat digunakan kembali.

Node 2: Order Status

Langkah 1: Menulis Test yang Gagal

```

#test_support_agent.py continued
def test_order_status():
    """Test the order status node"""
    state = {
        "messages": [HumanMessage(content="Where is my order #12345?")],
        "intent": IntentEnum.CHECK_ORDER,
        "status": ""
    }
    result = order_status_node(state)
    response_text = result["messages"][-1].content.lower()
    assert "shipped" in response_text or "processing" in response_text

```

- Hasil yang Diharapkan: Test akan gagal karena order_status_node belum diimplementasikan.

Langkah 2: Mengimplementasikan Kode

```

#test_support_agent.py continued
def order_status_node(state: CustomerState):
    """Handle order status queries using LLM"""
    llm = ChatOpenAI(model="gpt-4o-mini")
    order_chain = order_prompt | llm
    query = state["messages"][-1].content
    response = order_chain.invoke({"query": query})
    state["messages"].append(AIMessage(content=response.content))
    return state
order_prompt = ChatPromptTemplate.from_messages([
    template="""You are a customer service agent checking order status.
    Available orders: #12345 (shipped), #67890 (processing).
    If the order number is not found, apologize and provide support options.""",

```

Langkah 3: Refactor

Perkenalkan mekanisme caching atau logging untuk efisiensi dan auditability.

Node 3: Product Inquiry

Langkah 1: Menulis Test yang Gagal

```
#test_support_agent.py continued
def test_product_inquiry():
    """Test the product inquiry node"""
    state = {
        "messages": [HumanMessage(content="Tell me about wireless headphones")],
        "intent": IntentEnum.PRODUCT_INQUIRY,
        "status": ""
    }
    result = product_inquiry_node(state)
    response_text = result["messages"][-1].content.lower()
    assert "wireless headphones" in response_text
```

Langkah 2: Mengimplementasikan Kode

```
#support_agent.py continued
def product_inquiry_node(state: CustomerState):
    """Handle product inquiries using LLM"""
    llm = ChatOpenAI(model="gpt-4o-mini")
    product_chain = product_prompt | llm
    query = state["messages"][-1].content
    response = product_chain.invoke({"query": query})
    state["messages"].append(AIMessage(content=response.content))
    return state

product_prompt = ChatPromptTemplate.from_messages([
    ("system", """You are a product information specialist.
    Products: wireless headphones (20-hour battery, noise cancellation),
    smartwatch (fitness tracking, heart rate monitoring).
    For unknown products, apologize and offer to connect with a specialist."""),])
```

Node 4: General Chat

Langkah 1: Menulis Test yang Gagal

```
#test_support_agent.py continued
def test_general_chat():
    """Test general conversation handling"""
    state = {
        "messages": [HumanMessage(content="Hello, I'm James")],
        "intent": IntentEnum.GENERAL_CHAT,
        "status": ""
    }
    result = general_chat_node(state)
    response_text = result["messages"][-1].content.lower()
    assert "nice to meet you" in response_text and "james" in response_text
```

Langkah 2: Mengimplementasikan Kode

```
#support_agent.py continued
```

```
def general_chat_node(state: CustomerState):
    """Handle general conversation and queries"""
    llm = ChatOpenAI(model="gpt-4o-mini")
    chat_chain = general_chat_prompt | llm
    query = state["messages"][-1].content
    response = chat_chain.invoke({"query": query})
    state["messages"].append(AIMessage(content=response.content))
    return state
general_chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant.")
])
```

Langkah 4: Integration Testing

Setelah semua node diimplementasikan dan diuji secara individual, tulis integration test untuk memvalidasi keseluruhan workflow.

```
#test_support_agent.py continued
def test_complete_workflow():
    """Test the complete customer support workflow"""
    workflow = build_support_workflow()
    # Test order status
    state = {"messages": [HumanMessage(content="Where is my order #12345?")]}
    result = workflow.invoke(state)
    assert "shipped" in result["messages"][-1].content.lower()
    # Test product inquiry
    state = {"messages": [HumanMessage(content="Tell me about wireless headphones")]}
    result = workflow.invoke(state)
    assert "wireless headphones" in result["messages"][-1].content.lower()
    # Test general chat
    state = {"messages": [HumanMessage(content="My name is James")]}
    result = workflow.invoke(state)
    assert "nice to meet you" in result["messages"][-1].content.lower()
```

Ringkasan Bagian Praktik

1. **TDD Workflow:** Write tests first, implement minimal code to pass tests, then refactor.
2. **Node Implementation:** Modularize functionality to simplify testing.
3. **Integration Testing:** Validate end-to-end workflows.
4. **Deployment:** Use the `build_support_workflow()` function to run the agent interactively.

Dengan mengikuti langkah-langkah ini, Anda telah membangun customer support AI agent yang andal, scalable, dan dapat diuji menggunakan TDD

Chapter 20

Deploying AI Agents into Production

Pengantar

Men-deploy AI agent adalah langkah krusial yang mengubah sistem Anda dari tahap pengembangan menjadi environment dunia nyata. Proses ini melibatkan pemilihan infrastruktur yang tepat, memastikan skalabilitas, dan menerapkan langkah-langkah keamanan untuk melindungi data sensitif. Pada bab ini, kita akan membahas strategi deployment untuk AI agent yang dibangun dengan LangGraph dan LangChain, instruksi langkah demi langkah untuk menyiapkan pipeline, serta best practice untuk memantau dan memelihara agent Anda di production.

Pada akhir bab ini, Anda akan memiliki pemahaman lengkap tentang cara membawa AI agent Anda ke production dengan percaya diri.

20.1 Strategi Deployment

Men-deploy AI agent memerlukan keseimbangan antara performa, skalabilitas, dan efektivitas biaya. Tiga strategi deployment yang paling umum meliputi deployment on-premises, berbasis cloud, dan containerized.

1. Deployment On-Premises

Apa Itu: Meng-hosting AI agent di server organisasi Anda sendiri.

Kelebihan:

- Kontrol penuh atas infrastruktur dan data.- Cocok untuk industri dengan persyaratan kepatuhan yang ketat (misalnya, kesehatan, keuangan).

Tantangan:

- Biaya awal yang tinggi untuk penyiapan infrastruktur.- Membutuhkan keahlian internal untuk pemeliharaan server.

Penyiapan Langkah demi Langkah:

1. **Provision Hardware:** Set up servers with high-performance CPUs and GPUs for running large AI models.
2. **Install Dependencies:**
 - Install Python, LangChain, and LangGraph libraries.
 - Configure OpenAI or other LLM APIs.
3. **Set Up the Agent:**
 - Deploy the `support_agent.py` module on the server.
 - Configure the `.env` file with API keys and other configurations.
4. **Expose via API:**
 - Use frameworks like Flask or FastAPI to expose the agent as a RESTful API.
5. **Secure the Environment:**
 - Use firewalls, VPNs, and encryption to protect sensitive data.

2. Deployment Berbasis Cloud

Apa Itu: Meng-hosting agent pada platform cloud seperti AWS, Azure, atau Google Cloud.

Kelebihan:

- Elastic scaling: Menangani lonjakan permintaan pengguna dengan mudah.-
Biaya awal minimal: Pricing pay-as-you-go.

Tantangan:

- Biaya berkelanjutan bisa menumpuk seiring penggunaan yang berat.-
Masalah data residency jika regulasi membatasi penyimpanan cloud.

Penyiapan Langkah demi Langkah pada AWS:

1. Create an EC2 Instance:

- Select a GPU-optimized instance for AI workloads (e.g., `g4dn.xlarge`).
- Install Python and required libraries (`pip install langchain-openai langgraph`).

2. Use S3 for Data Storage:

- Store logs or model artifacts in S3 for persistent storage.

3. Deploy with API Gateway:

- Set up an API Gateway to expose the agent as a RESTful endpoint.

4. Auto-Scaling:

- Use AWS Auto Scaling Groups to handle variable traffic.

5. Secure with IAM:

- Restrict API access using Identity and Access Management (IAM) roles.

3. Deployment Containerized

Apa Itu: Mengemas AI agent beserta dependensinya ke dalam container yang ringan dan portable (misalnya, menggunakan Docker).

Kelebihan:

- Menyederhanakan deployment di berbagai environment.- Ideal untuk arsitektur berbasis microservices.

Tantangan:

- Membutuhkan pengetahuan containerization.- Mungkin memerlukan orchestration untuk deployment skala besar (misalnya, Kubernetes).

Penyiapan Langkah demi Langkah dengan Docker:

Buat sebuah Dockerfile:

```
#Dockerfile
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY src/ .
ENV OPENAI_API_KEY=<your-api-key>
CMD ["python", "support_agent.py"]
```

Build dan Test Image:

```
Terminal
docker build -t customer-support-agent .
docker run -p 8080:8080 customer-support-agent
```

1. **Deploy with Docker Compose:** Use Docker Compose for managing multi-container setups.
2. **Orchestrate with Kubernetes:** Deploy at scale using Kubernetes (K8s), setting up Pods and Services.

20.2 Continuous Integration dan Continuous Deployment (CI/CD)

Pipeline CI/CD mengotomatiskan testing, integrasi, dan deployment, sehingga mengurangi kesalahan manual dan mempercepat delivery.

1. Menyiapkan Pipeline CI/CD

Tools yang Dibutuhkan:

- **GitHub Actions:** Mengotomatiskan workflow testing dan deployment.-
- Jenkins:** Tool CI/CD yang andal untuk pipeline yang lebih kompleks.

Contoh Langkah demi Langkah dengan GitHub Actions:

Buat File `.github/workflows/deploy.yml`:

```
#deploy.yml
name: Deploy AI Agent
on:
  push:
    branches:
      - main
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: 3.9
      - name: Install Dependencies
        run: |
          pip install -r requirements.txt
      - name: Run Tests
        run: pytest tests/
      - name: Deploy to AWS (example)
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
        run: |
```

```
aws deploy start-deployment --application-name MyApp --deployment-group-name MyGroup --s3-location bucket=deployment-artifacts,key=artifact.zip
```

Push Kode untuk Memicu Deployment: Setiap push ke branch main akan:

- Run unit tests.
- Deploy the updated agent to AWS.

2. Best Practice Automation

A. Test Coverage:

- a. Ensure >80% of your code is covered by tests.
- b. Include unit, integration, and regression tests in the pipeline.

B. Rollback Mechanisms:

- a. Implement automated rollback in case of deployment failure.

C. Environment Variables:

- a. Use `.env` files to manage sensitive configurations.

20.3 Scaling AI Agent

Seiring meningkatnya permintaan pengguna, AI agent Anda harus melakukan scaling untuk menangani peningkatan traffic sambil tetap menjaga latency yang rendah.

Horizontal Scaling

- **Apa Itu: Menambah lebih banyak instance dari AI agent untuk mendistribusikan beban.**- **Cara Mengimplementasikan:?**
Gunakan load balancer (misalnya, AWS Elastic Load Balancer) untuk mengarahkan traffic ke seluruh instance.? **Deploy beberapa instance menggunakan orchestration tool seperti Kubernetes.**

Vertical Scaling

- **Apa Itu: Meningkatkan resource (CPU, memory, GPU) dari satu instance.**- **Kapan Digunakan: Untuk environment dengan traffic rendah di mana menambah instance tidak diperlukan.**

- **Contoh: Meng-upgrade instance AWS EC2 dari t2.micro ke t3.large.**

Contoh Load Balancing dengan AWS

1. Deploy multiple instances of the agent using AWS Auto Scaling Groups.
2. Configure an Application Load Balancer (ALB) to distribute traffic.
3. Set health checks to ensure instances are functioning properly.

20.4 Pertimbangan Keamanan

Mengamankan AI agent sangatlah penting, terutama ketika menangani data pelanggan yang sensitif.

1. Authentication dan Authorization

- Gunakan API key, OAuth2, atau JWT token untuk mengautentikasi pengguna.- Batasi akses ke endpoint tertentu berdasarkan peran pengguna.

2. Enkripsi Data

- Gunakan HTTPS untuk mengenkripsi data yang sedang dikirim (in transit).- Simpan data sensitif (misalnya, detail pelanggan) dalam database yang terenkripsi.

3. Kepatuhan (Compliance)

- Pastikan kepatuhan terhadap regulasi seperti GDPR atau CCPA jika berurusan dengan data pengguna.

Latihan Praktik

1. **Dockerize the Agent:** Write a Dockerfile for the agent and deploy it locally. Share your Docker image on DockerHub.
2. **Set Up CI/CD:** Configure a GitHub Actions pipeline for automated testing and deployment of the agent.
3. **Simulate Load:** Use a load testing tool like **Locust** to simulate 10,000 concurrent users and measure performance.
4. **Implement Security:** Add HTTPS encryption and API key authentication to your deployment.

Chapter 21

Performance Monitoring and Maintenance

Pengantar

Deployment hanyalah permulaan. Untuk memastikan AI agent Anda beroperasi secara optimal di production, Anda harus membangun strategi monitoring dan maintenance yang andal. Bab ini akan memandu Anda dalam menyiapkan tools dan proses untuk memantau performa, melakukan troubleshooting masalah, dan memelihara agent Anda dari waktu ke waktu. Kita akan membahas pelacakan performa real-time, mengidentifikasi bottleneck, melakukan debugging masalah umum, dan memperbarui agent untuk menangani kebutuhan yang terus berkembang.

21.1 Tools dan Teknik Monitoring

Monitoring AI agent memastikan bahwa performanya sesuai dengan ekspektasi pengguna, bahkan di bawah kondisi yang berubah-ubah. Ini membantu mendeteksi kegagalan, lonjakan latency, dan perilaku yang tidak terduga sejak dini, mencegah downtime atau degradasi pengalaman pengguna.



Gambar: Contoh visualisasi Grafana Stack sebagai tool monitoring AI agent

1. Menyiapkan Sistem Monitoring

Monitoring dimulai dengan mengintegrasikan tools observability ke dalam deployment Anda. Observability mencakup metrics, log, dan trace.

Komponen Utama:

1. **Metrics:** Quantitative measures like response time, throughput, and error rates.
2. **Logs:** Detailed records of actions performed by the agent (e.g., API calls, workflow execution).
3. **Traces:** Contextual data about individual workflows or requests.

Contoh Tools Monitoring:

1. **Prometheus:** For collecting and querying metrics.
2. **Grafana:** For visualizing metrics and setting up alerts.
3. **ELK Stack (Elasticsearch, Logstash, Kibana):** For advanced logging and search.
4. **AWS CloudWatch:** Native cloud monitoring tool for AWS-based deployments.

Contoh Langkah demi Langkah dengan Prometheus dan Grafana:

1. Set Up Prometheus:

- Install Prometheus on your server.

Konfigurasi scrape job pada prometheus.yml untuk memantau metrics API dari AI agent:

```
#Prometheus.yml
scrape_configs:
  - job_name: 'customer_support_agent'
    static_configs:
      - targets: ['localhost:8080/metrics']
```

2. Expose Metrics in the Agent:

Tambahkan endpoint pada agent Anda untuk mengekspos metrics dalam format Prometheus:

```
#agent_endpoint
from prometheus_client import start_http_server, Counter
request_counter = Counter("total_requests", "Total number of requests received")
def expose_metrics():
    start_http_server(8080)
    request_counter.inc()
```

3. Visualize with Grafana:

- Connect Grafana to Prometheus.
- Create dashboards to monitor response times, error rates, and latency.

2. Pelacakan Performa Real-Time

Metrics Penting:

1. Latency:

- Time taken for the agent to respond to a user query.

- Tools: Prometheus, AWS CloudWatch.

2. Error Rates:

- Percentage of requests resulting in errors.
- Tools: Sentry, ELK Stack.

3. Throughput:

- Number of requests handled per second.
- Tools: Grafana, AWS CloudWatch.

Contoh: Memantau Latency dengan Grafana:

Tambahkan latency metrics ke agent:

```
#agent_endpoint_latency
from prometheus_client import Summary

request_latency = Summary("request_latency_seconds", "Time taken to handle a request")
@request_latency.time()
def handle_request():
    # Handle user query
```

Visualisasikan tren latency di Grafana dengan dashboard real-time.

21.2 Metrics Performa untuk AI Agent

Metrics performa membantu mengukur efisiensi, keandalan, dan pengalaman pengguna dari agent. Setiap metric memberikan insight yang dapat ditindaklanjuti.

1. Metrics Utama untuk Dilacak

1.1. Latency

- Mengukur waktu yang dibutuhkan dari input pengguna hingga respons agent.-
Threshold: Untuk customer support agent, targetkan latency <300ms untuk kueri umum.

1.2. Error Rate

- Melacak seberapa sering workflow gagal karena data yang hilang, timeout API, atau error internal.

- Contoh: Error rate >2% menandakan perlunya debugging segera.

1.3. Uptime

- Persentase waktu ketika agent tersedia dan beroperasi.- Gunakan tools seperti AWS CloudWatch untuk menyiapkan alert downtime.

1.4. Kepuasan Pengguna

- Gunakan survei atau mekanisme feedback aplikasi untuk mengukur kepuasan.- Contoh: Tanyakan pengguna, "Apakah respons ini membantu?" dan kumpulkan data jawaban ya/tidak.

Contoh: Mengatur Alert untuk Metrics:

- Gunakan Grafana untuk menyiapkan alert bagi threshold seperti latency >500ms atau error rate >5%.

2. Mengukur Pengalaman Pengguna

- **Chat Logs: Analisis interaksi pengguna-agent untuk mengidentifikasi area yang perlu ditingkatkan.- Heatmaps: Lacak pola penggunaan untuk mengidentifikasi fitur populer atau waktu dengan traffic tinggi.**

21.3 Troubleshooting dan Debugging

Bahkan dengan monitoring yang sudah diterapkan, masalah akan tetap muncul. Troubleshooting memastikan Anda dapat dengan cepat mengidentifikasi dan menyelesaikan masalah tersebut.

1. Masalah Operasional Umum

1. **High Latency:**

- Cause: Overloaded server or slow external API.
- Solution: Add caching or scale horizontally.

2. **Frequent Errors:**

- Cause: Unhandled edge cases or API timeouts.
- Solution: Add retries and better exception handling.

3. Memory Leaks:

- Cause: Unreleased resources or large objects in memory.
- Solution: Use memory profiling tools (e.g., `tracemalloc`).

2. Teknik Debugging

Contoh: Debugging dengan Logging:

Tambahkan log detail untuk workflow kunci:

```
#agent_endpoint_latency
import logging

logging.basicConfig(level=logging.INFO)

def intent_classifier_node(state):
    logging.info(f'Classifying intent for query: {state['messages'][-1].content}')
    # Intent classification logic
```

Analisis log untuk mengidentifikasi sumber error.

Contoh: Debugging Kegagalan Workflow:

1. Use LangGraph's built-in visualization tools to trace the flow of data through nodes.
2. Identify nodes where failures occur and inspect input/output states.

21.4 Memperbarui dan Memelihara AI Agent

AI agent harus terus berkembang untuk menangani data baru, fitur, atau perubahan dalam perilaku pengguna. Pembaruan dan pemeliharaan rutin memastikan performa yang berkelanjutan.

1. Mengelola Model Drift

Apa Itu: Seiring waktu, prediksi agent dapat menurun kualitasnya seiring perubahan perilaku pengguna atau pola data.

Solusi:

- Latih ulang (retrain) model secara berkala menggunakan log interaksi terbaru.
- Contoh: Fine-tune model deteksi intent setiap 3 bulan.

2. Menjadwalkan Pembaruan

Best Practice:

1. Version Control:

- Use Git to manage code and model updates.
- Maintain a changelog to track what has changed in each version.

2. Canary Deployments:

- Test updates on a small subset of users before full deployment.

3. Rollbacks:

- Maintain backups of previous versions to enable quick rollback in case of failure.

3. Dokumentasi dan Change Log

Apa yang Harus Disertakan:

- Pembaruan pada workflow atau model.- Fitur atau intent baru yang ditambahkan ke agent.- Masalah yang diketahui dan perbaikan yang direncanakan.

Contoh: Perbarui readme.md dengan fitur baru:

```
## Version 1.1.0 (Date)
- Added support for order cancellation queries.
- Improved latency by adding API caching.
- Fixed bug in product inquiry responses.
```

Latihan Praktik

1. Set Up Monitoring:

- Configure Prometheus and Grafana to monitor your AI agent.
- Visualize metrics like latency, throughput, and error rates.

2. Simulate and Debug Issues:

- Introduce an error (e.g., API timeout) and debug using logs and traces.

3. Retrain a Model:

- Use interaction logs to retrain an intent classification model. Deploy the updated model and monitor changes in performance.

4. Implement Rollbacks:

- Write a script to roll back to the previous version of the agent in case of deployment failure.

Part 7: Case Studies and Reflections

(Chapters 22-23)

Chapter 22

Case Studies and Applications

Pengantar

Artificial Intelligence (AI) agent sedang merevolusi berbagai industri dengan mengotomatiskan tugas, meningkatkan efisiensi, dan memperbaiki pengalaman pengguna. Bab ini menampilkan studi kasus dunia nyata di mana bisnis-bisnis telah berhasil mengimplementasikan AI agent menggunakan LangGraph dan LangChain. Dengan mempelajari contoh-contoh ini, Anda akan memperoleh wawasan tentang penerapan prinsip dan teknik yang dibahas sepanjang buku ini untuk mengatasi tantangan bisnis yang nyata.

Kita akan menjelajahi tiga studi kasus:

1. **Customer Support Automation:** Streamlining customer service with AI agents.
2. **Sales and Marketing Personalization:** Enhancing customer engagement through tailored recommendations.
3. **Operational Efficiency:** Utilizing AI agents to optimize internal processes.

Setiap studi kasus akan memberikan:

- Gambaran umum tentang masalah.- Implementasi langkah demi langkah menggunakan AI agent.

- Hasil yang terukur dan pelajaran yang dipetik.

22.1 Otomatisasi Dukungan Pelanggan: AI Help Desk

Pernyataan Masalah

Sebuah bisnis e-commerce yang berkembang pesat menghadapi tantangan dalam mengelola pertanyaan dukungan pelanggan akibat ekspansinya. Pelanggan mengalami keterlambatan dalam respons, terutama selama periode belanja puncak. Perusahaan mencari solusi berbasis AI untuk:

- Mengotomatiskan respons terhadap pertanyaan umum.
- Mengurangi beban kerja staf dukungan.
- Menjaga tingkat kepuasan pelanggan yang tinggi.

Solusi AI Agent

Perusahaan mengimplementasikan Customer Support AI Agent yang mampu:

1. Handling frequently asked questions (FAQs) such as order status, shipping policies, and return processes.
2. Escalating unresolved queries to human agents.
3. Providing product recommendations based on user input.

Implementasi Langkah demi Langkah

1. Define the Agent Workflow:

- Utilize LangGraph to create a workflow with nodes for intent detection, FAQ handling, order status queries, and escalation.

2. Integrate Knowledge Base:

- Train the agent with FAQs and integrate it with an order management API to fetch real-time order details.

3. Deploy to a Cloud Environment:

- Use cloud services to deploy the agent with auto-scaling capabilities to handle peak traffic.

4. Monitor and Optimize:

- Implement monitoring tools to track response times and error rates.

Hasil

- **Otomatisasi 70%: AI agent menangani 70% pertanyaan dukungan tanpa campur tangan manusia.- Waktu Respons Berkurang: Rata-rata waktu respons menurun dari 2 menit menjadi 15 detik.- Kepuasan Pelanggan Meningkat: Skor kepuasan pelanggan (CSAT) meningkat 15%.**

Pelajaran yang Dipetik

1. Start by automating high-frequency, low-complexity queries.
2. Continuously improve the agent by analyzing unresolved queries and retraining the model.
3. Monitor agent performance closely during peak periods to ensure scalability.

22.2 Personalisasi Penjualan dan Pemasaran: Mendorong Engagement

Pernyataan Masalah

Sebuah layanan streaming berbasis langganan bertujuan meningkatkan engagement dan retensi pengguna dengan memberikan rekomendasi film, acara TV, dan musik yang dipersonalisasi. Perusahaan menghadapi tantangan dengan:

- Personalisasi yang terbatas dalam rekomendasi.- Tingkat churn yang tinggi di antara pengguna baru.- Kesulitan dalam menargetkan kampanye pemasaran secara efektif.

Solusi AI Agent

Perusahaan menerapkan Recommendation AI Agent yang:

1. Analyzed user preferences and behavior.
2. Suggested personalized content in real-time.
3. Supported targeted marketing campaigns by segmenting users based on their interests.

Implementasi Langkah demi Langkah

1. **Build a User Profiling Node:**

- Use LangChain to create a node that extracts preferences from user behavior (e.g., viewing history, search queries).

2. **Integrate with Content Database:**

- Connect the agent to a database of movies, shows, and music using a retrieval-augmented generation (RAG) architecture.

3. **Deploy in a Multi-Modal Environment:**

- Enable the agent to process inputs from various channels, including text, voice, and app interactions.

4. **A/B Test Recommendations:**

- Conduct A/B testing to measure the impact of personalized recommendations on user engagement.

Hasil

- **Engagement Meningkat:** Rata-rata waktu yang dihabiskan pengguna di platform meningkat 20%.- **Tingkat Konversi Lebih Tinggi:** Kampanye pemasaran yang ditargetkan menghasilkan peningkatan 25% dalam langganan.- **Churn Berkurang:** Tingkat churn bulanan menurun 10%.

Pelajaran yang Dipetik

1. Leverage user behavior data to improve recommendation accuracy.
2. Continuously update content databases to reflect current trends and preferences.
3. Test and refine the agent's algorithms using real-world user feedback.

22.3 Efisiensi Operasional: Mengotomatiskan Alur Kerja Internal

Pernyataan Masalah

Sebuah perusahaan logistik menghadapi inefisiensi dalam melacak pengiriman, mengelola inventaris, dan menjadwalkan pengantaran. Proses manual menyebabkan keterlambatan, kesalahan, dan peningkatan biaya operasional.

Solusi AI Agent

Perusahaan mengembangkan Operational AI Agent untuk:

1. Automate shipment tracking and updates for customers and staff.
2. Predict inventory shortages and recommend restocking actions.
3. Optimize delivery schedules based on traffic and weather data.

Implementasi Langkah demi Langkah

1. Automate Shipment Tracking:

- Use LangGraph to build a workflow that integrates with tracking APIs and updates customers via email or SMS.

2. Predict Inventory Needs:

- Train an AI model using historical inventory data to forecast shortages.

3. Optimize Delivery Routes:

- Integrate with external APIs for real-time traffic and weather data.
- Use the agent to recommend the fastest and most cost-effective delivery routes.

4. Monitor Performance:

- Track key metrics such as on-time delivery rates, inventory accuracy, and shipment errors.

Hasil

- **Penghematan Waktu 50%:** Alur kerja otomatis mengurangi waktu yang dihabiskan untuk tugas manual

22.4 Pelajaran Nyata dari Dunia Nyata

Pada bagian ini, kita mengambil pelajaran dari pengalaman penerapan AI di dunia nyata untuk menyoroti pelajaran-pelajaran penting yang dipetik. Dengan memahami kesalahan umum, mengadopsi praktik terbaik, dan menyesuaikan solusi dengan kebutuhan organisasi, bisnis dapat memaksimalkan keberhasilan penerapan AI agent.

Kesalahan Umum yang Harus Dihindari

1. Lack of Clear Objectives

- **Example:** A retail company implemented a chatbot without defining specific goals. As a result, the bot provided generic responses that failed to meet user needs, leading to poor adoption.
- **Lesson:** Define measurable objectives (e.g., reducing customer support response times by 50%) before deploying the agent.

2. Insufficient Training Data

- **Example:** An insurance firm launched an intent classification AI with a limited dataset. The agent struggled to understand uncommon customer queries, leading to a high error rate.
- **Lesson:** Use diverse, high-quality training data that represents real-world use cases. Continuously collect and annotate data to improve the agent over time.

3. Neglecting Human Oversight

- **Example:** A healthcare chatbot provided incorrect recommendations because it operated autonomously without human supervision.
- **Lesson:** Maintain human oversight for critical applications. Use mechanisms like human-in-the-loop to review and validate AI outputs.

4. Overcomplicating Early Deployments

- **Example:** A logistics company attempted to implement a multi-modal AI agent (handling text, voice, and image inputs) in the initial rollout. Complexity delayed deployment and increased costs.
- **Lesson:** Start simple. Begin with a single functionality (e.g., text-based interaction) and expand incrementally.

5. Ignoring User Feedback

- **Example:** A streaming platform ignored customer feedback on inaccurate recommendations, leading to declining user engagement.
- **Lesson:** Actively collect user feedback and iterate based on real-world usage patterns.

Praktik Terbaik untuk Penerapan

1. Start Small, Scale Incrementally

- **Example:** eBay initially deployed AI for automating responses to common customer queries before scaling to handle complex cases like fraud detection.
- **Best Practice:** Begin with high-impact, low-complexity use cases. Use the success of initial deployments to build momentum for broader adoption.

2. Test Thoroughly Before Launch

- **Example:** Microsoft's Tay chatbot failed publicly when it was launched without adequate testing in adversarial environments .
- **Best Practice:** Conduct rigorous testing for edge cases, security vulnerabilities, and real-world scenarios before launching.

3. Ensure Cross-Functional Collaboration

- **Example:** Spotify's AI personalization team worked closely with product managers, marketers, and engineers to align AI recommendations with business goals .

- **Best Practice:** Collaborate across departments to ensure the AI aligns with organizational priorities and delivers value.

4. Monitor and Optimize Continuously

- **Example:** Uber monitors driver and rider interactions in real-time to improve its dynamic pricing algorithms and optimize customer satisfaction .
- **Best Practice:** Use tools to track performance metrics (e.g., latency, error rates) and update models regularly.

5. Focus on User Experience

- **Example:** Google Duplex achieved user trust by incorporating natural conversational tones and fallback mechanisms to human operators when needed
- **Best Practice:** Design user-interfaces and ensure the agent handles ambiguous or difficult scenarios gracefully.

Menyesuaikan dengan Kebutuhan Organisasi

1. Understand Business Context

- **Example:** A financial services firm tailored its AI chatbot to prioritize secure handling of sensitive data, complying with strict regulatory requirements .
- **Best Practice:** Customize AI solutions specific business needs, regulatory constraints, and industry standards.

2. Align AI Goals with Organizational Objectives

- **Example:** Amazon's AI in its warehouses is optimized to reduce delivery times while minimizing operational costs, directly supporting its customer-centric business model .

- **Best Practice:** Ensure the AI system directly achieves organizational goals, such as cost reduction, revenue growth, or customer satisfaction.

3. Plan for Change Management

- **Example:** When implementing AI agents for fraud detection, Mastercard invested in training staff to adapt to AI-driven workflows .
- **Best Practice:** Educate employees about the benefits of AI in the deployment process to reduce resistance and ensure smooth adoption.

4. Account for Scalability

- **Example:** Netflix's recommendation engine was designed to scale as its subscriber base grew globally, processing billions of interactions daily .
- **Best Practice:** Build scalable AI architectures that accommodates future users, data, and complexity.

5. Iterate Based on Organizational Feedback

- **Example:** Slack iterated on its AI-driven search feature based on feedback from enterprise customers, improving relevance and efficiency .
- **Best Practice:** Treat AI deployment as a continuous process, regularly updating workflows based on user and organizational feedback.

Chapter 23

Pemikiran Akhir dan Langkah Berikutnya

Pendahuluan

Perjalanan membangun AI agent bersifat transformatif, tetapi tidak berakhir dengan deployment. Bab ini menyajikan eksplorasi mendalam tentang masa depan AI agent, sumber daya untuk memperdalam keahlian Anda, strategi untuk menunjukkan kemampuan Anda, dan inspirasi untuk berinovasi. Baik Anda seorang AI developer, pemimpin bisnis, atau teknolog, bab ini memberdayakan Anda untuk memimpin dengan percaya diri di bidang yang berkembang pesat ini.

Kita akan membahas topik-topik berikut:

- Tren transformatif yang membentuk masa depan AI.- Panduan praktis untuk pembelajaran dan pertumbuhan profesional seumur hidup.- Langkah-langkah nyata untuk memberi dampak melalui inovasi AI.- Rangkuman komprehensif konsep-konsep utama dari buku ini.

Mari kita perluas pemahaman Anda dan persiapkan Anda untuk langkah ke depan.

23.1 Masa Depan AI Agent

1. Tren 1: Multi-Modal AI Agent

Gambaran Umum: Multi-modal agent dapat memproses dan menghasilkan berbagai jenis data-teks, gambar, video, audio, dan bahkan data sensor. Dengan menggabungkan modalitas-modalitas ini,

agent menawarkan pengalaman yang lebih kaya dan lebih interaktif.

Contoh Saat Ini:

- **GPT-4 dari OpenAI: Menangani input teks dan gambar secara mulus.-**
DeepMind Gemini dari Google: Dirancang untuk mengintegrasikan kemampuan teks, visi, dan pengambilan keputusan untuk kasus penggunaan tingkat lanjut.

Aplikasi Masa Depan:

1. **E-Commerce:** Visual search agents that allow users to upload product images and receive personalized recommendations.
2. **Healthcare:** Diagnostic agents that analyze patient records, X-rays, and voice symptoms simultaneously.
3. **Education:** Multi-modal tutors that adapt to learning styles using video, text-based explanations, and spoken instructions.

Cara Memulai:

- **Bereksperimenlah dengan Hugging Face Transformers untuk membangun model multi-modal.-** **Integrasikan tools LangChain untuk menggabungkan pemrosesan teks dan visual dalam workflow.**

2. Tren 2: Federated Learning untuk AI yang Lebih Menjaga Privasi

Gambaran Umum: Federated learning melatih model AI di berbagai perangkat terdesentralisasi (misalnya, smartphone) sambil menjaga data pengguna tetap lokal. Metode ini melindungi privasi dan mematuhi regulasi data seperti GDPR.

Contoh Dunia Nyata: *Gboard dari Google: Model pengetikan prediktif dilatih di perangkat menggunakan federated learning, menjaga privasi pengguna.*

Aplikasi yang Sedang Berkembang:

1. **Financial Services:** Training fraud detection models across multiple banks without sharing sensitive customer data.
2. **Healthcare:** Enabling collaboration across hospitals to develop AI models for rare diseases.

Cara Memulai:

- Pelajari framework seperti TensorFlow Federated dan PySyft untuk mengimplementasikan federated learning dalam workflow AI Anda.- Gunakan data sintetis untuk mensimulasikan skenario federated untuk pengujian.

3. Tren 3: Explainable AI (XAI)

Gambaran Umum: Seiring AI menjadi semakin meluas, memahami bagaimana ia sampai pada keputusan menjadi sangat penting. Explainable AI memberikan transparansi dan membangun kepercayaan.

Contoh Saat Ini:

- **LIME (Local Interpretable Model-Agnostic Explanations):** Menjelaskan prediksi dari model AI apa pun.- **SHAP (SHapley Additive exPlanations):** Menawarkan penjelasan model yang konsisten dan dapat diinterpretasikan.

Industri Utama:

1. **Healthcare:** Explaining diagnoses to doctors to improve decision-making.
2. **Finance:** Justifying loan approval or denial decisions to regulators and customers.

Cara Memulai:

- Implementasikan framework explainability seperti SHAP untuk menganalisis prediksi AI dalam workflow Anda.- Latih model yang dapat diinterpretasikan (misalnya, decision tree) berdampingan dengan model kompleks untuk validasi silang.

4. Tren 4: Edge AI

Gambaran Umum: Edge AI membawa komputasi ke perangkat seperti smartphone, wearable, dan sistem IoT, memungkinkan pengambilan keputusan real-time tanpa bergantung pada sumber daya cloud.

Contoh:

- **Tesla Autopilot:** Memproses data secara lokal untuk navigasi real-time.- **Perangkat Smart Home:** AI agent pada termostat pintar dan kamera mengurangi latensi dengan memproses secara lokal.

Kasus Penggunaan yang Sedang Berkembang:

1. **Manufacturing:** Real-time quality control using edge AI on production lines.
2. **Autonomous Vehicles:** Decentralized processing for safety-critical systems.

Cara Memulai:

- Pelajari ONNX Runtime untuk men-deploy model pada perangkat edge.- Optimalkan workflow AI untuk lingkungan dengan latensi rendah dengan mengurangi ukuran model menggunakan teknik seperti quantization.

5. Tren 5: AI Agent di Metaverse

Gambaran Umum: Metaverse mengintegrasikan realitas virtual dan augmented dengan AI agent yang berinteraksi secara alami dalam lingkungan immersive.

Contoh:

- **Horizon Worlds dari Meta:** Avatar bertenaga AI berinteraksi dengan pengguna di ruang virtual.- **NVIDIA Omniverse:** AI agent mensimulasikan lingkungan untuk pelatihan dan kolaborasi industri.

Peluang Masa Depan:

- **Ritel:** Asisten belanja virtual di toko digital.- **Pendidikan:** Lingkungan belajar immersive yang dipandu oleh tutor AI.

Cara Memulai:

- Jelajahi tools seperti Unity dan Unreal Engine untuk membangun lingkungan 3D.- Integrasikan conversational agent berbasis AI ke dalam ruang virtual menggunakan LangChain.

23.2 Melanjutkan Perjalanan Anda

1. Perdalam Pengetahuan Anda

Untuk menguasai pengembangan AI, fokuslah membangun fondasi yang kuat sambil menjelajahi konsep-konsep lanjutan. Berikut adalah roadmap terstruktur:

Langkah 1: Perkuat Dasar-Dasar

- **Pelajari Machine Learning:?** *Machine Learning Yearning oleh Andrew Ng: Panduan ramah pemula untuk ML terapan.?* *The Complete Hugging Face Blueprint - Panduan pemula untuk machine learning, oleh James Karanja Maina-* **Kuasai Python:?** *Pelajari library penting seperti NumPy, Pandas, dan Scikit-learn.*

Langkah 2: Perluas Toolkit Anda

- **Pelajari LangChain dan LangGraph:?** Ikuti dokumentasi resmi LangChain. **Bereksperimenlah dengan workflow berbasis graph dari LangGraph untuk AI agent.-** Jelajahi **Framework AI Baru:?** **Praktikkan langsung dengan PyTorch, Hugging Face, dan ONNX.**

Langkah 3: Maju ke Deep Learning

- **Study Neural Networks:** Use *Deep Learning* by Ian Goodfellow as a reference.
- **Build End-to-End Models:** Practice training models using TensorFlow or PyTorch on real datasets.

Langkah 4: Spesialisasikan Diri

- Dalami niche seperti computer vision, NLP, atau reinforcement learning.- Jelajahi aplikasi spesifik domain (misalnya, kesehatan, keuangan, logistik).

2. Tunjukkan Kemampuan Anda

Proyek Pembangun Portofolio:

- **Bangun AI agent yang menyelesaikan masalah dunia nyata, seperti:?** **Chatbot dukungan pelanggan untuk toko e-commerce.?** **Sistem rekomendasi untuk platform streaming.-** **Publikasikan proyek-proyek ini di GitHub dengan dokumentasi yang lengkap.**

Tulis Blog atau Tutorial:

- **Bagikan pembelajaran Anda dengan komunitas melalui blog di platform seperti Medium atau Dev.to.**
 - Examples: "How to Build an AI Agent with LangChain in 10 Steps."
 - "Improving Chatbot Responses Using Explainable AI."

Sertifikasi:

- **Dapatkan sertifikasi seperti AWS Certified Machine Learning Specialist atau Google TensorFlow Developer untuk meningkatkan kredibilitas.**

3. Bereksperimen dan Berinovasi

Untuk tetap terdepan dalam lanskap AI yang dinamis, kembangkan mindset rasa ingin tahu dan eksperimentasi:

- **Ikuti Tantangan AI:? Ikuti kompetisi Kaggle untuk mengasah kemampuan Anda.- Berkolaborasi di Hackathon:? Bangun solusi inovatif bersama rekan-rekan di hackathon yang berfokus pada AI.- Berkontribusi pada Open Source:? Bantu meningkatkan library AI seperti LangChain atau bangun tools Anda sendiri.**

23.3 Kesimpulan & Kata Penutup

Rangkuman Konsep Utama

Sepanjang buku ini, kita telah menjelajahi:

1. Foundational AI agent principles and LangGraph workflows.
2. Advanced topics like memory integration and custom architecture.
3. Rigorous testing strategies with test-driven development (TDD).
4. Deployment methods, performance monitoring, and real-world applications.

Menginspirasi Inovasi

Sebagai praktisi AI, Anda memiliki alat untuk membentuk masa depan. Baik Anda mengotomatiskan proses bisnis, menciptakan teknologi yang mengubah hidup, atau menjelajahi peluang yang belum tergarap, karya Anda berarti. Bermimpilah besar, iterasi tanpa henti, dan biarkan kreativitas Anda bersinar.

Latihan Praktis untuk Pembelajaran Seumur Hidup

1. Analyze Trends:

- Research how AI is transforming industries like healthcare, education, or finance. Write a report summarizing your findings.

2. Design a Multi-Modal Agent:

- Build an agent that processes text and image inputs, such as a visual search engine.

3. Experiment with Explainability:

- Implement SHAP to analyze and explain predictions from your AI models.

4. Deploy Edge AI:

- Optimize a LangGraph workflow for an IoT device, such as a smart home assistant.

5. Join a Research Community:

- Collaborate with researchers on cutting-edge AI projects through platforms like **AIcrowd** or **OpenAI Scholars**.

Kata Penutup

Selamat telah menyelesaikan perjalanan ini! AI agent lebih dari sekadar alat; mereka adalah jembatan menuju dunia yang lebih cerdas dan lebih efisien. Dengan LangGraph dan LangChain, Anda telah memperoleh pengetahuan untuk membangun sistem yang mendorong perubahan yang bermakna. Teruslah belajar, berinovasi, dan berbagi

karya Anda. Sekarang, ambil langkah berikutnya. Dunia AI sedang menunggu kecemerlangan Anda. Buatlah dampak!

Thank You for Your Support!

Kami Ingin Mendengar Masukan Anda!

Terima kasih telah meluangkan waktu untuk membaca The Complete LangGraph Blueprint. Perjalanan Anda ke dalam dunia AI dan machine learning penting bagi kami, dan kami berharap buku ini telah menginspirasi Anda untuk berkreasi, berinovasi, dan menjelajahi kemungkinan-kemungkinan baru dengan Hugging Face. Jika Anda berkenan, kami akan sangat menghargai jika Anda dapat meluangkan waktu sejenak untuk membagikan pendapat jujur Anda dengan meninggalkan ulasan di Amazon. Mengapa Ulasan Anda Penting?? Masukan Anda membantu kami meningkatkan edisi mendatang dan memastikan kami terus menyediakan sumber daya yang berharga bagi pembaca seperti Anda.?? Suara Anda membantu orang lain menemukan buku ini dan memulai perjalanan AI mereka sendiri.

?? Wawasan Anda menginspirasi kami untuk menciptakan konten yang lebih praktis dan menarik bagi komunitas AI dan developer. Cara Meninggalkan Ulasan Anda

1. Scan the QR code below or visit this link: ?? [Amazon Review Page](https://www.amazon.com/review/create-review?asin=B0DP69QV7K)

(<https://www.amazon.com/review/create-review?asin=B0DP69QV7K>)

2. Share your honest feedback—what you loved, what could be better, and how the book has helped you.





Gerbang Anda menuju pengetahuan dan budaya. Dapat diakses oleh semua orang.



z-library.sk

z-lib.gs

z-lib.fm

go-to-library.sk



[Official Telegram channel](#)



[Z-Access](#)



<https://wikipedia.org/wiki/Z-Library>